BİLGİSAYAR MÜHENDİSLİĞİ · 1. SINIF

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba, C!\n");
    return 0;
}
```

C ile Programlamaya Giriş

Konu anlatımı, sınav tuzakları ve çözümlü soru bankası

14	178	1	2
bölüm	soru	mini proje	deneme sınavı

HAZIRLAYAN

Ahmet Şerif Kart · BARÜ – Bartın Üniversitesi

C17 · VS Code · gcc · 2026

İçindekiler

Bu Kitap Nasıl Kullanılır? . . . 7

1 Programlamaya ve C'ye Giriş . . . 10

1.1 Program ve programlama dili . . . 10
1.2 C'nin kısa tarihi . . . 11
1.3 Derleme süreci . . . 12
1.4 Kurulum: gcc ve VS Code . . . 12
1.5 İlk program: Merhaba Dünya . . . 14
1.6 Terminalden derleme ve çalıştırma . . . 15
1.7 printf ve kaçış dizileri . . . 17
1.8 Yorum satırları . . . 19
1.9 Hata türleri . . . 20
1.10 main fonksiyonu ve return 0 . . . 24
1.11 Büyük/küçük harf duyarlılığı . . . 25

2 Değişkenler, Veri Tipleri ve Operatörler . . . 32

2.1 Değişken nedir? . . . 32
2.2 Tanımlama, ilk değer verme ve atama . . . 32
2.3 Değişken adları ve anahtar sözcükler . . . 34
2.4 Temel veri tipleri . . . 35
2.5 İlk değer verilmemiş değişken . . . 36
2.6 Sabitler . . . 37
2.7 Aritmetik operatörler . . . 39
2.8 Tip dönüşümü . . . 41
2.9 char: küçük bir tamsayı . . . 44
2.10 Bileşik atama, ++ ve -- . . . 45
2.11 Operatör önceliği . . . 46
2.12 Taşma ve ondalıklı sayıların hassasiyeti . . . 48
2.13 Bit düzeyinde operatörler . . . 50
2.14 math.h kütüphanesi . . . 51

3 Klavyeden Girdi ve Biçimli Çıktı . . . 59

3.1 printf biçim belirteçleri . . . 59
3.2 Alan genişliği, hizalama ve hassasiyet . . . 61
3.3 Yanlış biçim belirteci . . . 64
3.4 scanf ile klavyeden okuma . . . 65
3.5 scanf'te en sık iki hata . . . 67
3.6 scanf'in dönüş değeri . . . 68
3.7 Girdi tamponu ve %c tuzağı . . . 69
3.8 Tamponu temizlemek: fflush(stdin) yanlış . . . 71
3.9 Örnek uygulamalar . . . 73
3.10 Türkçe karakterler ve Windows konsolu . . . 75

4 Koşullu İfadeler . . . 83

4.1 Karar vermek ve ilişkisel operatörler . . . 83
4.2 C'de doğru ve yanlış . . . 84
4.3 if, if-else ve else if . . . 85
4.4 Süslü parantez tuzakları . . . 87
4.5 İç içe if ve sarkan else . . . 88
4.6 Mantıksal operatörler . . . 89
4.7 = ile == karışıklığı . . . 91
4.8 0 < x < 10 tuzağı . . . 92
4.9 Operatör önceliği . . . 92
4.10 Üçlü operatör ?: . . . 94
4.11 switch deyimi . . . 95
4.12 double değerlerini karşılaştırmak . . . 98
4.13 Örnek uygulamalar . . . 99

5 Döngüler . . . 108

5.1 Neden döngü? while . . . 108
5.2 do-while . . . 109
5.3 for döngüsü . . . 109
5.4 Sayaç, toplam ve çarpım . . . 110
5.5 break ve continue . . . 111
5.6 Noktalı virgül tuzağı . . . 113
5.7 İç içe döngüler ve desenler . . . 114
5.8 İzleme tablosu ve off-by-one . . . 115
5.9 Klasik algoritmalar . . . 116
5.10 Kullanıcı girdisiyle döngüler . . . 120
5.11 double sayaçla döngü tuzağı . . . 122

6 Fonksiyonlar . . . 130

6.1 Neden fonksiyon? . . . 130
6.2 Fonksiyonun anatomisi . . . 130
6.3 void fonksiyonlar, dönüş değeri ve erken çıkış . . . 131
6.4 Prototip (bildirim) . . . 132
6.5 Parametre ve argüman . . . 134
6.6 Değer ile geçirme . . . 136
6.7 Çağrı yığını . . . 136
6.8 Kapsam . . . 137
6.9 static yerel değişken . . . 139
6.10 Fonksiyonu return olmadan bitirmek . . . 140
6.11 Hazır fonksiyonlar . . . 141
6.12 Algoritmaları fonksiyona dönüştürme . . . 142
6.13 Menü programı . . . 144
6.14 main de bir fonksiyondur . . . 146
6.15 Önişlemci: makro ve koşullu derleme . . . 146
6.16 Programı dosyalara bölmek . . . 150

7 Diziler . . . 161

7.1 Neden dizi? Tanımlama ve bellek . . . 161
7.2 İlk değer verme . . . 163
7.3 İndeks ve sınır taşması . . . 164
7.4 Eleman sayısı: sizeof . . . 164
7.5 Dolaşma, doldurma ve yazdırma . . . 165
7.6 Temel algoritmalar . . . 166
7.7 Diziler ve fonksiyonlar . . . 167
7.8 Diziyi kopyalama ve karşılaştırma . . . 168
7.9 Ters çevirme ve kaydırma . . . 170
7.10 Arama: doğrusal ve ikili . . . 171
7.11 Sıralama: kabarcık ve seçmeli . . . 173
7.12 İki boyutlu diziler . . . 175

8 Karakter Dizileri (String) . . . 184

8.1 String nedir? . . . 184
8.2 Tanımlama ve ilk değer . . . 185
8.3 Yazdırma ve karakter karakter dolaşma . . . 186
8.4 Klavyeden string okumak . . . 186
8.5 string.h fonksiyonları . . . 188
8.6 == ile karşılaştırma ve = ile atama tuzağı . . . 192
8.7 ctype.h: karakter denetimi ve dönüşümü . . . 193
8.8 String ile sayı dönüşümleri . . . 194
8.9 Türkçe karakterler ve UTF-8 . . . 196
8.10 String dizisi . . . 197
8.11 Klasik algoritmalar . . . 198

P1 Öğrenci Not Sistemi . . . 208

S Deneme Vize Sınavı . . . 221

9 İşaretçiler (Pointers) . . . 237

9.1 Bellek ve adres . . . 237
9.2 İşaretçi tanımlama ve dolaylama . . . 238
9.3 İşaretçinin tipi . . . 240
9.4 NULL ve başlatılmamış işaretçi . . . 240
9.5 Adres ile geçirme . . . 241
9.6 İşaretçi aritmetiği . . . 243
9.7 Diziler ve işaretçiler . . . 245
9.8 Stringler ve işaretçiler . . . 247
9.9 const ve işaretçiler . . . 248
9.10 Yerel değişkenin adresini döndürmek . . . 250
9.11 İşaretçinin işaretçisi . . . 250
9.12 İşaretçi dizisi ve diziye işaretçi . . . 251
9.13 Komut satırı argümanları . . . 253
9.14 void işaretçi ve fonksiyon işaretçisi . . . 254

10 Dinamik Bellek Yönetimi . . . 264
10.1 Neden dinamik bellek? . . . 264
10.2 Bellek bölgeleri . . . 264
10.3 malloc ve free . . . 265
10.4 calloc . . . 266
10.5 Boyutu kullanıcıdan alıp dinamik dizi . . . 267
10.6 realloc ile büyüyen dizi . . . 268
10.7 Fonksiyondan dinamik dizi döndürmek . . . 270
10.8 Dinamik string . . . 271
10.9 2 boyutlu dinamik dizi . . . 272
10.10 Bellek hataları . . . 273
10.11 Bellek hatalarını bulan araçlar . . . 275
10.12 sizeof ve ayrılan boyut . . . 276

11 Yapılar: struct, union ve enum . . . 286
11.1 Neden struct? . . . 286
11.2 struct tanımı ve üye erişimi . . . 286
11.3 typedef ile kısa ad . . . 287
11.4 Atama ve karşılaştırma . . . 288
11.5 Struct dizisi . . . 290
11.6 Fonksiyona struct: değer ve işaretçi . . . 292
11.7 qsort ile yapı dizisi sıralamak . . . 294
11.8 İç içe struct . . . 294
11.9 Struct ve dinamik bellek . . . 295
11.10 sizeof ve hizalama . . . 296
11.11 union . . . 298
11.12 enum . . . 299
11.13 Uygulama: sınıf kayıt sistemi . . . 300

12 Dosya İşlemleri . . . 310
12.1 Dosya neden gerekir? . . . 310
12.2 FILE, fopen ve fclose . . . 310
12.3 Dosyaya yazma: fprintf, fputs, fputc . . . 312
12.4 Dosyadan okuma: fscanf, fgets, fgetc . . . 313
12.5 feof tuzağı . . . 315
12.6 Uygulamalar . . . 316
12.7 CSV dosyaları . . . 318
12.8 stdin, stdout, stderr . . . 320
12.9 İkili dosyalar: fwrite ve fread . . . 321
12.10 Rastgele erişim: fseek, ftell, rewind . . . 322
12.11 Metin ve ikili mod, dosya yolu . . . 324
12.12 Türkçe karakterli dosyalar . . . 325

13 Özyineleme (Recursion) . . . 333
13.1 Özyineleme nedir . . . 333
13.2 Temel durum ve özyinelemeli durum . . . 334
13.3 Çağrı yığını ve faktoriyel . . . 334
13.4 Sayılarla özyineleme . . . 336
13.5 Fibonacci . . . 338
13.6 Dizi ve string ile özyineleme . . . 340
13.7 Hanoi kuleleri . . . 342
13.8 Özyinelemeli ikili arama . . . 344
13.9 Yazdırmayı çağrıdan önce mi sonra mı yapmalı . . . 345
13.10 Temel durumu unutmak: yığın taşması . . . 346
13.11 Özyineleme mi, döngü mü? . . . 348
14 Veri Yapılarına Giriş: Bağlı Liste, Yığın ve Kuyruk . . . 356
14.1 Neden veri yapısı? . . . 356
14.2 Öz-referanslı yapı ve düğüm . . . 356
14.3 Düğüm oluşturma, bağlama ve dolaşma . . . 357
14.4 Tüm listeyi serbest bırakma . . . 359
14.5 Ekleme: başa, sona, sıralı . . . 360
14.6 Silme . . . 363
14.7 Yığın (stack) . . . 365
14.8 Kuyruk (queue) . . . 368
14.9 Dizi mi, bağlı liste mi? . . . 371
S Deneme Final Sınavı . . . 380
A C Cheat Sheet . . . 404
B Terimler Sözlüğü . . . 409
C Sık Karşılaşılan Hata Mesajları . . . 417
D ASCII Tablosu . . . 433

Bu Kitap Nasıl Kullanılır?

Bu kitap, bilgisayar mühendisliğinin ilk yılında C dersini alan ve daha önce hiç kod yazmamış öğrenci için yazıldı. Bilgisayarın ne olduğunu, derleyicinin ne işe yaradığını kabaca biliyorsun; ama değişken, döngü, işaretçi gibi kavramlarla henüz tanışmadın. Hedef iki tane: **C'yi gerçekten öğrenmen** ve **vize/finalde soru kaçırmaman**.

Anlatım bilerek sade tutuldu: kısa cümleler, bol örnek, "sınavda ne sorulur?" uyarıları. Kitabı bir roman gibi okumana gerek yok; bilgisayarın başında, kodu yazarak ilerle.

Kitabın yapısı

Kitap, tipik bir "Programlamaya Giriş" dersinin C ile işlenen sırasını takip eder:

Kısım	Bölümler	Ne öğreneceksin
Temeller	1–8	Derleme süreci, değişken ve operatörler, `printf` / `scanf`, koşul, döngü, fonksiyon, dizi, string
Proje: Öğrenci Not Sistemi + Vize denemesi	—	İlk sekiz bölümü birleştiren uygulama ve vize sınavı provası
Bellek ve işaretçiler	9–10	İşaretçi (pointer), dinamik bellek (`malloc`, `free`)
Veriyi düzenlemek	11–12	`struct`, `union`, `enum`; dosya okuma ve yazma
Özyineleme ve veri yapıları	13–14	Özyineleme (recursion), bağlı liste, yığın, kuyruk
Final denemesi	—	Tüm dönemi kapsayan sınav provası
Ekler	A–D	A: Cheat sheet, B: Terimler sözlüğü, C: Hata mesajları, D: ASCII tablosu

Her bölüm aynı düzende ilerler: önce **"Bu bölümde öğreneceklerin"** listesi, sonra konu anlatımı ve örnekler, ardından **Bölüm Özeti**, en sonda **Sorular** ve yeni bir sayfada **Çözümler**. Derleyici ve editör kurulumu Bölüm 1'de adım adım anlatılıyor; önce onu yap, sonra örnekleri yazarak ilerle.

Kutular

Metnin içinde dört tür renkli kutu göreceksin. Hepsinin bir görevi var:

> **İpucu**
>
> İşini kolaylaştıran kısa yollar, VS Code ve terminal numaraları, akılda tutma yöntemleri.

> **Sık Yapılan Hata**
>
> Öğrencilerin en sık düştüğü tuzaklar. Sınavda puan kaybettiren hataların çoğu bu kutulardan çıkar.

> **Sınavda Çıkar**
>
> Hocaların sormayı çok sevdiği konular. Vize/final öncesi sadece bu kutuları okuyarak bile hızlı tekrar yapabilirsin.

> **C23 Notu**
>
> C'nin en yeni standardı olan C23 ile gelen yenilikler ve kolaylaştırmalar. Bilmek iyidir, ama sınavda klasik C17 yazımını kullan.

Kod ve çıktılar

Kitaptaki **her program gcc ile derlendi ve çalıştırıldı** (gcc 16, `-std=c17 -Wall -Wextra` ayarlarıyla, uyarısız). Çıktı kutularındaki yazılar elle yazılmadı; programın gerçek çıktısı olarak kitaba eklendi. Yani bir örneği kendi bilgisayarında çalıştırdığında aynı sonucu görmelisin. Kod kutusunun üstündeki etiket (`ornek.c`) dosyanın adıdır.

ornek.c

```c
#include <stdio.h>

int main(void) {
    printf("Kod kutusu böyle görünür.\n");
    return 0;
}
```

Çıktı

```
Kod kutusu böyle görünür.
```

Programın klavyeden bir şey okuduğu örneklerde, senin yazdığın kısım çıktı kutusunda **mavi** renkte gösterilir; tıpkı terminalde gördüğün gibi. Aşağıdaki `scanf` satırını Bölüm 3'te öğreneceksin; şimdilik sadece çıktının nasıl göründüğüne bak:

selam.c

```c
#include <stdio.h>

int main(void) {
    int yas;
    printf("Yaşın: ");
    scanf("%d", &yas);
    printf("Gelecek yıl %d yaşında olacaksın.\n", yas + 1);
    return 0;
}
```

Çıktı

```
Yaşın: 19
Gelecek yıl 20 yaşında olacaksın.
```

Çıktı kutularının renkleri de bir şey söyler. Koyu zeminli normal kutu programın çıktısıdır. **Sarı** kutu, derleyicinin verdiği bir **uyarıdır** (warning): program derlenir ama gcc "buraya dikkat et" der. **Kırmızımsı** kutu bir **derleme hatasıdır** (error): program derlenmez ve kutuda gcc'nin gerçek hata mesajı görünür. Program çalışırken çökerse çıktı kutusunun etiketi bunu söyler. Bu mesajları okumayı Bölüm 1'de öğreneceksin.

> **Not: Çıktılar her bilgisayarda aynı mı?**
>
> Çıktılar ondalık **nokta** ile verilmiştir (`3.75`). Windows konsolunda Türkçe karakterler (`ş`, `ğ`, `ü`) bozuk görünebilir; nedenini ve çözümünü Bölüm 3'te göreceksin. Rastgele sayılar ve bellek adresleri gibi bilgisayardan bilgisayara değişen çıktılarda bunu metinde ayrıca belirtiyoruz.

Sorular nasıl çözülmeli?

Her bölümün sonunda üç tür soru var:

- **Çoktan seçmeli (5 soru):** Kavramları ölçer. Önce cevabı kendin bul, sonra şıklara bak.
- **Çıktı tahmini (3 soru):** "Bu kod ne yazdırır?" Sınavların vazgeçilmezi. Kodu **kâğıt üzerinde, satır satır** izle; değişkenlerin değerini bir tabloya yaz. Bilgisayarda çalıştırmadan önce cevabını mutlaka kâğıda yaz.
- **Kod yazma (2 soru):** Küçük programlar. Önce kendin yaz, terminalde `gcc` ile derleyip çalıştır, sonra çözümle karşılaştır. Çözümdeki kod tek doğru cevap değildir; senin kodun da doğru çıktı veriyorsa doğrudur.

Cevap anahtarı, çözümlerin başında tek bir tabloda toplanır. Sınavda bir sorunun cevabını bulmakla yetinme; **neden öbür şıkların yanlış olduğunu** da çözümden oku.

> **İpucu: Çalışma önerisi**
>
> Bölümü oku → örnekleri **elle yazarak** (kopyala-yapıştır yok!) derleyip çalıştır → soruları çöz → yanlışlarını çözümlerden kontrol et → bir hafta sonra sadece kutuları ve özeti tekrar oku. Programlama izleyerek değil, yazarak öğrenilir. Derleyici hata verdiğinde mesajı atlama, oku: hata mesajını okumak da öğrenilecek bir beceridir.

BÖLÜM 1

Programlamaya ve C'ye Giriş

Bu Bölümde Öğreneceklerin

- Programın, makine dilinin ve derleyici ile yorumlayıcı arasındaki farkın ne olduğunu
- C'nin kısa tarihini, neden hâlâ öğrenildiğini ve standartlarını (C89'dan C23'e)
- Bir C programının çalıştırılabilir dosyaya dönüşme aşamalarını (önişlemci → derleyici → çevirici → bağlayıcı)
- gcc ve VS Code kurulumunu
- İlk programını yazıp terminalden derleyip çalıştırmayı
- `printf` ile metin yazdırmayı ve kaçış dizilerini (`\n`, `\t`, `\"`, `\\`, `%%`)
- Yorum satırlarını ve iç içe `/* */` tuzağını
- Derleme, bağlama, çalışma zamanı ve mantık hatası arasındaki farkı; gcc'nin hata mesajını okumayı
- `int main(void)` ve `return 0` ifadelerinin anlamını

1.1 Program ve programlama dili

Program, bilgisayara ne yapacağını adım adım anlatan komutlar dizisidir. İşlemci (CPU) yalnızca kendi **makine dilini** (0 ve 1'lerden oluşan komutlar) anlar. Makine diliyle program yazmak çok zor ve hataya açık olduğu için **yüksek seviyeli programlama dilleri** (high-level language) geliştirilmiştir: C, Java, Python bunlardan birkaçıdır.

Yüksek seviyeli dilde yazdığımız metne **kaynak kod** (source code) denir. İşlemci kaynak kodu doğrudan anlamaz; önce makine koduna çevrilmesi gerekir. Bunun iki klasik yolu vardır:

	Derleyici (compiler)	Yorumlayıcı (interpreter)
Nasıl çalışır?	Bütün programı **önceden** makine koduna çevirir, sonra çıkan dosya çalıştırılır	Kodu satır satır okur ve **o anda** çalıştırır
Sonuç	Ayrı bir çalıştırılabilir dosya (*executable*, `.exe` gibi)	Ayrı bir dosya üretmez
Hatalar	Sözdizimi hatalarını program çalışmadan önce yakalar	Hataya o satıra gelindiğinde rastlanır
Hız	Genellikle çok hızlıdır	Genellikle daha yavaştır
Örnek diller	C, C++	Python (en basit anlatımla)

C bir **derlenen dildir** (compiled language). Derleme bir kez yapılır; çıkan dosya, derleyicisi olmayan bir bilgisayarda bile çalışır. Ama bu dosya, derlendiği işlemci ve işletim sistemine özgüdür: Windows'ta derlediğin `.exe` dosyası Linux'ta çalışmaz. Kaynak kodu ise başka bir platformda yeniden derleyerek çalıştırabilirsin.

Sınavda Çıkar

"Derleyici ile yorumlayıcı arasındaki fark nedir?" klasik bir sorudur. Anahtar cümle: Derleyici programın **tamamını çalıştırmadan önce** çevirir; yorumlayıcı **satır satır, çalışırken** çevirir. "C hangisini kullanır?" sorusunun cevabı: derleyici.

1.2 C'nin kısa tarihi

C dilini **Dennis Ritchie**, 1970'lerin başında (1972 civarında) ABD'deki **Bell Laboratuvarları**'nda (Bell Labs) geliştirdi. Amaç, aynı yerde yazılan **UNIX** işletim sistemini assembly yerine daha okunaklı bir dille yazabilmekti; UNIX'in çekirdeği 1973'te C ile yeniden yazıldı. 1978'de Brian Kernighan ve Ritchie'nin birlikte yazdığı "The C Programming Language" kitabı dili dünyaya yaydı.

Neden hâlâ C öğreniyoruz?

Elli yılı aşkın geçmişine rağmen C hâlâ bilgisayar mühendisliğinin ilk derslerinde okutulur, çünkü:

- **İşletim sistemleri:** Linux çekirdeği ve Windows ile macOS'un önemli bölümleri C (ve C++) ile yazılmıştır.
- **Gömülü sistemler:** Arabalardaki, ev aletlerindeki, Arduino gibi kartlardaki mikrodenetleyicilerin yazılımı çoğunlukla C'dir.
- **Diğer dillerin altındaki dünya:** Python'un standart yorumlayıcısı C ile yazılmıştır. Java, C++, C# ve JavaScript'in `if`, `for`, `{ }` gibi söz dizimi C'den miras kalmıştır.
- **Bilgisayarın gerçekte nasıl çalıştığını gösterir:** C bellek, adres ve işaretçi gibi kavramları senden saklamaz. Bunları bir kez öğrenince diğer dillerde arka planda ne olduğunu da anlarsın.

C standartları

C zaman içinde **standartlaştırılmıştır**: Dilin kurallarını bir komite belirler, derleyiciler bu kurallara uyar. Standartlar çıktıkları yıla göre adlandırılır:

Standart	Yıl	Öne çıkanlar
C89 / C90	1989 / 1990	İlk resmî standart (ANSI C). `//` yorumu yoktur
C99	1999	`//` yorumu, `long long`, `stdbool.h` ile `bool`, değişkeni blok ortasında tanımlama
C11	2011	Çoklu iş parçacığı desteği; tehlikeli `gets` fonksiyonu dilden çıkarıldı
C17	2018	C11'in hata düzeltmesi, yeni özellik getirmez (C18 de denir). **Bu kitabın standardı**
C23	2024	`bool`, `true`, `false` anahtar sözcük oldu; `nullptr` ve `constexpr` geldi

Standartlar büyük ölçüde geriye dönük uyumludur: C99 ya da C17'ye göre yazılmış klasik kod yeni derleyicilerde de çalışır. Sınavlarda da çoğunlukla bu klasik tarzla karşılaşırsın.

Sık Yapılan Hata: C ile C++ aynı şey değildir

C++, C'nin üzerine kurulmuş **ayrı bir dildir**. `cout`, `class`, `std::string` gibi C++ özellikleri C'de yoktur; internette bulduğun C++ kodunu bir `.c` dosyasına yapıştırırsan derlenmez. Bu kitap yalnızca C'yi anlatır: dosya uzantısı `.c`, derleyici komutu `gcc`'dir (C++ için `g++`).

1.3 Derleme süreci

Elinde yalnızca bir metin dosyası (`merhaba.c`) varken çalışan bir programa nasıl ulaşırsın? Bu iş tek adımda olmaz; dört aşamalı bir hattan geçer. `gcc` komutu bu aşamaları sırayla kendisi çalıştırır:

Aşama	Ne yapar?	Girdi → Çıktı
1. **Önişlemci** (preprocessor)	`#` ile başlayan satırları işler: `#include <stdio.h>` satırını `stdio.h` dosyasının içeriğiyle değiştirir, yorumları siler	`merhaba.c` → genişletilmiş kaynak kod
2. **Derleyici** (compiler)	Yazım kurallarını denetler ve kodu işlemcinin komutlarını metin olarak yazan **assembly** diline çevirir	genişletilmiş kaynak → `merhaba.s` (assembly kodu)
3. **Çevirici** (assembler)	Assembly kodunu makine koduna çevirir	`merhaba.s` → `merhaba.o` (nesne dosyası, *object file*)
4. **Bağlayıcı** (linker)	Nesne dosyasını, kullandığın hazır fonksiyonların kodunu taşıyan kütüphanelerle (library) birleştirir	`merhaba.o` + kütüphane → `merhaba.exe` (Linux ve macOS'ta `merhaba`)

Şu noktalara dikkat:

- `stdio.h` bir **başlık dosyasıdır** (header file). İçinde `printf` gibi fonksiyonların "tanıtımı" bulunur; derleyici bir fonksiyonu görünce nasıl çağrılacağını bilmek zorundadır. `.h` uzantısı *header* sözcüğünden gelir.
- `printf`'in **kendi kodu** ise `stdio.h`'de değil, C'nin **standart kütüphanesinde** (standard library) bulunur. Onu bağlayıcı programına ekler. Yani sen yazmadığın bir fonksiyonu kullanabilirsin, ama bağlayıcı o fonksiyonu **bulabilmelidir**.
- Günlük dilde "derlemek" çoğu zaman dört aşamayı birden anlatır. Dar anlamda "derleyici" ise yalnızca ikinci aşamadır. Bazı kaynaklar derleyici ile çeviriciyi tek aşama sayıp "üç aşama" der; sınavda hocanın derste anlattığı modeli kullan.

Aşamaları istersen tek tek de çalıştırabilirsin. Aşağıdaki komutları kurulumdan sonra deneyebilirsin:

```
gcc -E merhaba.c          ← yalnız önişlemci: genişletilmiş kodu ekrana yazar
gcc -S merhaba.c          ← önişlemci + derleyici: merhaba.s (assembly) oluşur
gcc -c merhaba.c          ← önişlemci + derleyici + çevirici: merhaba.o oluşur
gcc merhaba.o -o merhaba  ← bağlayıcı: çalıştırılabilir dosya oluşur
```

`gcc -E` çıktısına bak: Tek satırlık `#include <stdio.h>`, yüzlerce satıra dönüşür. Önişlemcinin yaptığı iş budur.

> **Sınavda Çıkar**
>
> "C programının derleme aşamalarını sırayla yaz", "`#include` hangi aşamada işlenir?" ve "Bağlayıcı ne işe yarar?" sık sorulur. Sıra: **önişlemci → derleyici → çevirici (assembler) → bağlayıcı**. `#include` önişlemcinin, `printf` gibi hazır fonksiyonların kodunu programa eklemek bağlayıcının işidir.

1.4 Kurulum: gcc ve VS Code

Programı yazmak için bir **editör**, derlemek için bir **derleyici** gerekir. Kitap boyunca derleyici olarak **gcc**'yi (GNU Compiler Collection), editör olarak **VS Code**'u kullanıyoruz. Önce gcc'yi kur: işletim sistemine uygun olanı yap (1 ya da 2), sonra 3'e geç. Adımlar zamanla küçük değişiklikler gösterebilir; takılırsan kurulum sayfalarındaki güncel yönergelere bak.

1. Windows: MSYS2 ile gcc

Windows'ta gcc'nin **MinGW-w64** adlı sürümü kullanılır. Kurmanın en kolay yolu **MSYS2**'dir:

1. msys2.org adresinden kurulum dosyasını (`msys2-x86_64-....exe`) indir ve çalıştır. Her şeyi varsayılan bırakıp kur; klasör `C:\msys64` olur.
2. Kurulum bitince Başlat menüsünden **MSYS2 UCRT64** terminalini aç ve aşağıdaki komutu yaz. Sorular çıkarsa Enter ile varsayılanı kabul et, onay istenince `Y` yaz.

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

1. gcc'yi her terminalden çağırabilmek için klasörünü **PATH**'e ekle. Başlat menüsünde "ortam değişkenleri" (*environment variables*) ara ve açılan pencerede **Ortam Değişkenleri / Environment Variables** düğmesine bas. Kullanıcı değişkenlerinden `Path`'i seç, **Düzenle / Edit** → **Yeni / New** ile şu satırı ekle ve pencereleri **Tamam / OK** ile kapat:

```
C:\msys64\ucrt64\bin
```

1. Açık bütün terminalleri kapat; **yeni** bir PowerShell ya da Komut İstemi aç ve şunu yaz:

```
gcc --version
```

İlk satırda `gcc` sözcüğünü ve bir sürüm numarasını görmelisin (sürüm numaran farklı olabilir). Görüyorsan kurulum tamamdır.

> **Sık Yapılan Hata: "gcc tanınmıyor" hatası**
>
> `gcc --version` yazınca "'gcc' is not recognized..." (Türkçe Windows'ta "...tanınmıyor") benzeri bir mesaj alıyorsan iki ihtimal var: PATH'e eklediğin klasör yanlıştır (`C:\msys64\ucrt64\bin` klasörünün içinde `gcc.exe` olduğunu Dosya Gezgini'nde kontrol et) ya da terminali PATH'i ekledikten **sonra** yeniden açmadın. VS Code açıksa onu da kapatıp yeniden aç.

2. macOS ve Linux

macOS: Terminal uygulamasını aç ve şunu yaz; açılan pencerede **Install**'a bas. Bu komut Apple'ın komut satırı araçlarını (derleyici dahil) kurar.

```
xcode-select --install
```

Linux (Debian/Ubuntu): Terminalde şunu yaz. Fedora'da `sudo dnf install gcc` kullanılır.

```
sudo apt install gcc
```

Kurulumdan sonra her iki sistemde de `gcc --version` ile kontrol et.

> **Not: macOS'ta "gcc" aslında clang'dir**
>
> macOS'ta `gcc` yazınca çalışan program GNU gcc değil, Apple'ın **clang** derleyicisidir; `gcc --version` çıktısında "Apple clang" yazar. Kitaptaki programlar ikisinde de çalışır, ama hata ve uyarı **mesajlarının sözcükleri** biraz farklıdır; mesajın düzeni (dosya:satır:sütun) aynıdır. Gerçek gcc istersen Homebrew ile `brew install gcc` kurabilirsin; komutun adı sürüm numaralı olur (ör. `gcc-16`).

3. VS Code ve C/C++ eklentisi

1. code.visualstudio.com adresinden VS Code'u kur.
2. Sol menüden **Extensions**'a gir (Ctrl+Shift+X; macOS'ta Cmd+Shift+X), **"C/C++"** eklentisini (yayıncı: Microsoft) bul ve **Install**'a bas. Eklenti kod renklendirme, otomatik tamamlama ve hata altı çizme getirir. Derleyici **değildir**; derleyiciyi 1 ya da 2'de kurdun.
3. Bilgisayarında bir klasör oluştur (ör. `CDersleri`) ve VS Code'da **File → Open Folder** ile aç.
4. Klasörde yeni bir dosya oluştur ve adını `merhaba.c` koy. Uzantının `.c` olması şarttır.
5. **View → Terminal** ile (Ctrl+`) alttaki terminal panelini aç. Terminal, açık klasörde başlar.

> **İpucu: Derlemeyi terminalden yap**
>
> VS Code'un sağ üstündeki ▷ (Run C/C++ File) düğmesi de çalışır, ama arka planda `-Wall` olmadan derler. Biz derlemeyi terminalde elle yapacağız: komutu kendin yazarsan laboratuvarda ya da sınavda hangi bilgisayar olursa olsun aynı komutu bilirsin. ▷ düğmesini kullanacaksan, açılan `.vscode/tasks.json` dosyasındaki `"args"` listesine `"-std=c17"`, `"-Wall"` ve `"-Wextra"` ekle.

4. Alternatif: Dev-C++

Okulundaki laboratuvarda **Dev-C++** (Embarcadero sürümü) kullanılıyor olabilir. Derleyicisiyle birlikte gelen basit bir ortamdır; yalnız gcc'si kitabınkinden eskidir, bazı mesajlar farklı çıkabilir. Kitaptaki gibi davranması için **Tools → Compiler Options → General** sekmesinde "Add the following commands when calling the compiler" kutusunu işaretle ve şunu yaz:

```
-std=c17 -Wall -Wextra
```

Dev-C++'ta F11 derler ve çalıştırır. (Menü adları sürüme göre biraz değişebilir.)

5. Alternatif: online derleyici

Hiçbir şey kurmadan hızlı bir deneme için onlinegdb.com gibi bir online derleyiciye gir, dil olarak **C** seç, kodu yaz ve **Run**'a bas. Küçük denemeler için iyidir; ama derleyici ayarları sitenin elindedir. Asıl çalışmanı kendi bilgisayarındaki gcc ile yap.

1.5 İlk program: Merhaba Dünya

Programlama geleneğidir: Her yeni dilde ilk program ekrana bir selam yazar. `merhaba.c` dosyasına şunu yaz:

merhaba.c

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba Dünya!\n");
    return 0;
}
```

Çıktı

```
Merhaba Dünya!
```

Beş satırlık bu program C'nin neredeyse bütün temel kurallarını içinde barındırır. Satır satır inceleyelim:

Kod	Anlamı
`#include <stdio.h>`	**Önişlemci komutu.** `stdio.h` (*standard input/output*) başlık dosyasını programa ekler. `printf` bu dosyada tanıtılmıştır. `#` ile başlar, `;` ile **bitmez**.
`int main(void)`	**main fonksiyonu**: Programın başladığı yer. Program çalışınca işletim sistemi ilk olarak burayı çağırır. `int` ve `(void)` sözcüklerinin anlamını bu bölümün sonunda göreceksin.
`{ ... }`	Süslü parantezler bir **blok** oluşturur: burada `main`'in gövdesi. Her açılan `{` mutlaka kapanmalıdır.
`printf("Merhaba Dünya!\n");`	Parantez içindeki metni ekrana yazar. Çift tırnak içindeki metne **string sabiti** (string literal) denir. Sondaki `\n` alt satıra geçirir.
`;`	Her **deyim** (statement) noktalı virgülle biter.
`return 0;`	`main` fonksiyonunu bitirir ve işletim sistemine "program başarıyla tamamlandı" der.

Boş satırlar ve girintiler derleyiciyi ilgilendirmez, insanlar için vardır: Bloğun içindeki satırları **4 boşluk** içeri yaz, kodun okunması kolaylaşır. `main` satırındaki kelimelerin ayrıntısını Bölüm 6'da fonksiyonlarla birlikte göreceksin; şimdilik bu kalıbı her programda **aynen** yaz.

> **Not: Windows'ta "Dünya" bozuk çıkarsa**
>
> Windows konsolu Türkçe karakterleri (`ü`, `ş`, `ğ`) varsayılan olarak yanlış gösterebilir; program doğru çalışıyordur, sorun konsolun karakter kodlamasındadır. Hızlı çözüm olarak konsolda `chcp 65001` yazıp programı yeniden çalıştırabilirsin. Nedenini Bölüm 3'te anlatacağız.

> **Sık Yapılan Hata: Dosya uzantısı**
>
> Windows varsayılan olarak dosya uzantılarını gizler. Dosyayı Not Defteri ile kaydedersen adı `merhaba.c.txt` olabilir; gcc böyle bir dosyayı C kodu olarak tanımaz. Dosyayı VS Code'da `merhaba.c` adıyla oluştur. Uzantıları görmek için Dosya Gezgini'nin **Görünüm** menüsünden (Windows 11'de **Görüntüle → Göster**) "Dosya adı uzantıları" kutusunu işaretle.

> **Sık Yapılan Hata: `#include` satırının sonuna `;` konmaz**
>
> `#include`, `#define` gibi `#` ile başlayan satırlar C deyimi değil, **önişlemci komutlarıdır**; noktalı virgülle bitmezler. `#include <stdio.h>;` yazarsan gcc `extra tokens at end of '#include' directive` uyarısı verir. Uyarı programı durdurmaz ama hatalı alışkanlıktır.

1.6 Terminalden derleme ve çalıştırma

Terminali `merhaba.c` dosyasının bulunduğu klasörde aç (VS Code terminali zaten açık klasörde başlar; başka bir terminalde `cd klasorAdi` ile git, `dir` ya da `ls` ile dosyanı gör). Derleme komutu:

```
gcc merhaba.c -o merhaba
```

`gcc` derleyiciyi çağırır, `merhaba.c` kaynak dosyadır, `-o merhaba` (*output*) çıktı dosyasının adını belirler. Windows'ta dosya `merhaba.exe` olarak oluşur. `-o` yazmazsan Linux ve macOS'ta `a.out`, Windows'ta `a.exe` adı kullanılır.

Çalıştırma komutu işletim sistemine göre değişir:

Terminal	Komut
Linux, macOS	`./merhaba`
Windows PowerShell	`.\merhaba`
Windows Komut İstemi (cmd)	`merhaba`

`./` neden gerekli? Linux, macOS ve PowerShell güvenlik gereği içinde bulunduğun klasörü otomatik aramaz; "şu klasördeki `merhaba`" demek için `./` yazarsın. Bütün oturum şöyle görünür:

```
gcc merhaba.c -o merhaba     ← derle: çalıştırılabilir dosya oluşur
./merhaba                    ← çalıştır (PowerShell'de .\merhaba)
Merhaba Dünya!
```

Uyarıları açma alışkanlığı

gcc'yi yalnızca `gcc merhaba.c -o merhaba` ile çağırırsan çok az uyarı verir. Kitap boyunca derlemeyi şu komutla yapacağız:

```
gcc -std=c17 -Wall -Wextra merhaba.c -o merhaba
```

Seçenek	Anlamı
`-std=c17`	C17 kurallarına göre derle. gcc 15 ile birlikte varsayılan standart C23 oldu; kitapla aynı davranışı görmek için C17'yi açıkça istiyoruz
`-Wall`	*Warnings: all.* Şüpheli görünen yaygın durumlar için uyarıları aç
`-Wextra`	`-Wall`'ın kaçırdığı ek uyarıları da aç
`-o ad`	Çıktı dosyasının adı

Uyarı (warning) bir hata değildir; program yine de derlenir. Ama uyarılar çoğu zaman gerçek bir hatanın erken işaretidir. Aşağıdaki program geçerlidir, fakat içinde hiç kullanılmayan bir değişken vardır (`int kullanilmayan = 5;` satırını Bölüm 2'de öğreneceksin):

kullanilmayan.c

```c
#include <stdio.h>

int main(void) {
    int kullanilmayan = 5;
    printf("Merhaba\n");
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
kullanilmayan.c: In function 'main':
kullanilmayan.c:4:9: warning: unused variable 'kullanilmayan' [-Wunused-variable]
    4 |     int kullanilmayan = 5;
      |         ^~~~~~~~~~~~
```

Program yine de derlendi ve çalıştı:

Çıktı

```
Merhaba
```

`-Wall` olmadan derleseydin gcc bu programı sessizce derlerdi ve hatayı fark etmezdin. Kural basit: **Uyarısız derle.** Ödevde ve sınavda uyarı veren kod, çoğu zaman puan kaybettirir.

1.7 printf ve kaçış dizileri

`printf` (*print formatted*) ekrana yazı yazar. Çift tırnak içindeki metni olduğu gibi basar. Bir ayrıntıya dikkat: `printf` kendiliğinden **alt satıra geçmez**.

satirSonuYok.c

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba");
    printf("Dünya");
    printf("!");
    return 0;
}
```

Çıktı

```
MerhabaDünya!
```

Üç `printf` çağrısı arka arkaya, boşluksuz yazdı. Bazı terminallerde komut satırı (prompt) da çıktının hemen yanına yapışır. Alt satıra geçmek için metne **yeni satır karakteri** `\n` eklenir:

satirSonuVar.c

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba\n");
    printf("Dünya\n");
    printf("\n");
    printf("Bu satırdan önce boş satır var.\n");
    return 0;
}
```

Çıktı

```
Merhaba
Dünya

Bu satırdan önce boş satır var.
```

`\n` iki karakter gibi görünse de C için **tek bir karakterdir**. Ters bölü ile başlayan bu özel yazımlara **kaçış dizisi** (escape sequence) denir; yazılamayan ya da özel anlamı olan karakterleri metne sokmak için kullanılır:

Yazım	Anlamı
`\n`	Yeni satır (alt satıra geç)
`\t`	Sekme (tab), genelde birkaç boşluk kadar ileri atlar
`\"`	Çift tırnak karakteri
`\'`	Tek tırnak karakteri
`\\`	Ters bölü karakterinin kendisi
`%%`	Yüzde işaretinin kendisi (bu bir kaçış dizisi değil, `printf`'e özgü bir kuraldır)
`\0`	Boş karakter (null character); Bölüm 8'de string'leri anlatırken göreceksin

kacis.c

```c
#include <stdio.h>

int main(void) {
    printf("Ad\tSoyad\tNot\n");
    printf("Ali\tYılmaz\t85\n");
    printf("Hoca \"çok kolay\" dedi.\n");
    printf("Dosya: C:\\c\\merhaba.c\n");
    printf("Başarı oranı: %%95\n");
    printf("Satır 1\nSatır 2\n");
    return 0;
}
```

Çıktı

```
Ad      Soyad   Not
Ali     Yılmaz  85
Hoca "çok kolay" dedi.
Dosya: C:\c\merhaba.c
Başarı oranı: %95
Satır 1
Satır 2
```

`printf` içinde `%` işareti özeldir: `%d` gibi **biçim belirteçleri** (format specifier), değişkenlerin değerini metnin içine yerleştirmek için kullanılır. `%d` yer tutucusunu Bölüm 3'te göreceksin. Şimdilik tek bilmen gereken şu: Ekrana gerçekten `%` yazdırmak istiyorsan `%%` yazarsın.

Sık Yapılan Hata: Ters bölü, eğik çizgi ve yüzde işareti

Kaçış dizileri **ters bölü** `\` ile yazılır. `printf("Merhaba/n");` yazarsan ekrana olduğu gibi `Merhaba/n` basılır ve alt satıra geçilmez. Yüzde için de `\%` diye bir kaçış dizisi yoktur: `%%` yaz. `printf("Başarı: %100\n");` gibi yalnız bir `%` bırakırsan `printf` onu biçim belirtecinin başlangıcı sanır. Geçersiz bir biçim belirteci **tanımsız davranıştır** (undefined behavior): sonuç derleyiciye ve sisteme göre değişir, güvenilecek bir çıktı yoktur.

Sınavda Çıkar

Çıktı sorularında `\n`, `\t` ve `\\` çok sık kullanılır; en sık tuzaklardan biri de `printf("Ali\\n")` ile `printf("Ali\n")` farkıdır. Kural: Ekrandaki "imleç" yalnızca **gerçek** `\n` ile alt satıra iner. Soruyu çözerken imlecin hangi satırda kaldığını adım adım takip et. Son `printf`'te `\n` yoksa imleç o satırda kalır.

1.8 Yorum satırları

Yorumlar (comments), derleyicinin tamamen görmezden geldiği, kodu okuyan insana yönelik notlardır:

yorumlar.c

```c
/* Bu program yorum türlerini gösterir.
   Çok satırlı yorum birden fazla satıra yayılabilir. */
#include <stdio.h>

int main(void) {
    // Tek satırlık yorum: satırın sonuna kadar geçerlidir
    printf("Merhaba\n"); // kodun yanına da yazılabilir

    /* Bu iki satır arasındaki her şey yok sayılır.
    printf("Bu satır çalışmaz\n"); */
    printf("Bitti\n");
    return 0;
}
```

Çıktı

```
Merhaba
Bitti
```

Yorum türü	Sözdizimi	Kullanım
Tek satır	`// ...`	Kısa açıklamalar. C99'dan beri vardır
Çok satır	`/* ... */`	Uzun açıklamalar, bir kod parçasını geçici olarak devre dışı bırakma

Tersi de geçerlidir: **string'in içindeki** `//` ya da `/*` yorum başlatmaz. `printf("a // b\n");` ekrana `a // b` yazar; yorum işareti ancak tırnak dışında yorum işaretidir.

İç içe / / tuzağı

C'de `/* */` yorumları **iç içe yazılamaz**. Yorum, ilk gördüğü `*/` işaretinde kapanır:

icIceYorum.c

```c
#include <stdio.h>

int main(void) {
    /* Başlangıç yorumu
       printf("Bir\n"); /* iç yorum */
       printf("İki\n");
    */
    printf("Üç\n");
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
icIceYorum.c: In function 'main':
icIceYorum.c:5:25: warning: '/*' within comment [-Wcomment]
    5 |        printf("Bir\n"); /* iç yorum */
icIceYorum.c:7:6: error: expected expression before '/' token
    7 |     */
      |      ^
```

Ne oldu? Dıştaki yorum, **içteki** `*/` işaretinde kapandı. Bundan sonra `printf("İki\n");` normal kod gibi okundu, ardından gelen yalnız `*/` ise derleyiciye anlamsız geldi: `expected expression before '/' token`. gcc önce bir de uyarı verdi: `'/*' within comment`, yani "yorumun içinde yorum başlatmaya mı çalıştın?". Bir kod parçasını geçici olarak yorum yapacaksan ve içinde `/* */` varsa, satırları tek tek `//` ile yorumla.

> **İpucu: Yorum yapma kısayolu**
>
> VS Code'da bir veya birkaç satırı seçip **Ctrl+/** (macOS'ta **Cmd+/**) tuşlarına basarsan satırların başına `//` eklenir; tekrar basarsan kaldırılır. İç içe yorum sorunu yaşamamanın en kolay yolu da budur.

1.9 Hata türleri

Program yazarken dört farklı türde hata ile karşılaşırsın. Hangisi olduğunu anlamak, hatayı çözmenin yarısıdır. Hataları ortaya çıktıkları sırayla ele alıyoruz: derleme, bağlama, çalışma zamanı, mantık.

1. Derleme hatası (compile-time error)

Kod C'nin **yazım kurallarına** (sözdizimi, *syntax*) uymuyorsa gcc derlemez ve çalıştırılabilir dosya oluşmaz. En klasik örnek unutulan noktalı virgüldür:

noktaliVirgul.c

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba\n")
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
noktaliVirgul.c: In function 'main':
noktaliVirgul.c:4:24: error: expected ';' before 'return'
    4 |     printf("Merhaba\n")
      |                        ^
      |                        ;
    5 |     return 0;
      |     ~~~~~~
```

gcc mesajı **dosya:satır:sütun: tür: açıklama** düzeninde yazar. İlk mesaj satırını parçalara ayıralım:

Parça	Örnekteki değeri	Anlamı
Dosya	`noktaliVirgul.c`	Hata hangi dosyada
Satır	`4`	Hata kaçıncı satırda
Sütun	`24`	O satırın kaçıncı karakterinde
Tür	`error`	`error` hata demektir ve derleme durur; `warning` uyarıdır ve derleme sürer
Açıklama	`expected ';' before 'return'`	`return`'den önce `;` bekleniyordu

Altındaki satırlarda gcc sorunlu satırı gösterir, `^` işaretiyle noktayı işaretler ve bazen çözümü önerir (burada `;` eklemeyi). `In function 'main':` satırı da hatanın hangi fonksiyonun içinde olduğunu söyler. Bir ayrıntı: Eksik `;` gibi hatalarda derleyici şikâyeti, asıl hatalı satırdan **sonraki** satırda bildirebilir (sürüme göre değişir). Mesajdaki satırda bir şey göremezsen bir üst satıra da bak.

Kitaptaki mesajlar gcc 16'dan alınmıştır. Senin gcc sürümün ya da macOS'taki clang sözcükleri biraz farklı yazabilir, ama dosya:satır:sütun düzeni hepsinde aynıdır.

İkinci klasik hata, fonksiyon adının yanlış yazılmasıdır:

printfYazim.c

```c
#include <stdio.h>

int main(void) {
    prinft("Merhaba\n");
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
printfYazim.c: In function 'main':
printfYazim.c:4:5: error: implicit declaration of function 'prinft'; did you mean 'printf'? [-
Wimplicit-function-declaration]
    4 |     prinft("Merhaba\n");
      |     ^~~~~~
      |     printf
```

`implicit declaration of function 'prinft'`, "tanıtılmamış bir fonksiyon çağırdın" demektir. gcc `did you mean 'printf'?` diye kendisi de tahmin yürütür. Dikkat: gcc 14'ten önceki sürümlerde bu durum yalnızca bir **uyarıydı**; asıl hata bağlama aşamasında `undefined reference to 'prinft'` olarak çıkardı. Bu yüzden eski bir derleyicide aynı hatanın başka bir mesajla geldiğini görebilirsin.

Üçüncü klasik hata, `#include` satırını unutmaktır:

includeYok.c

```c
int main(void) {
    printf("Merhaba\n");
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
includeYok.c: In function 'main':
includeYok.c:2:5: error: implicit declaration of function 'printf' [-Wimplicit-function-declaration]
    2 |     printf("Merhaba\n");
      |     ^~~~~~
includeYok.c:1:1: note: include '<stdio.h>' or provide a declaration of 'printf'
  +++ |+#include <stdio.h>
    1 | int main(void) {
includeYok.c:2:5: warning: incompatible implicit declaration of built-in function 'printf' [-
Wbuiltin-declaration-mismatch]
    2 |     printf("Merhaba\n");
      |     ^~~~~~
includeYok.c:2:5: note: include '<stdio.h>' or provide a declaration of 'printf'
```

Mesajı baştan sona oku: gcc `printf`'i tanımadığını söylüyor, ardından da çözümü veriyor: `include '<stdio.h>' or provide a declaration of 'printf'`. Hatta eklenecek satırı `+++ |+#include <stdio.h>` ile gösteriyor. Hata mesajları çoğu zaman çözümün kendisini taşır; okumak zorundasın.

> **İpucu: İlk hatadan başla**
>
> Bir hata çoğu zaman arkasından başka hata mesajlarını da getirir (örneğin kapanmayan bir `{` yüzünden dosyanın sonuna kadar şikâyet edilir). Mesaj listesi uzun olsa bile **en üstteki ilk hatayı** düzelt ve yeniden derle. Çoğu zaman diğerleri kendiliğinden kaybolur.

2. Bağlama hatası (linker error)

Kodun yazımı doğrudur ve derleyici aşamasını geçer; ama **bağlayıcı** programın ihtiyaç duyduğu bir parçayı bulamaz. En tipik örnek, `main` yerine `Main` yazmaktır:

```c
#include <stdio.h>

int Main(void) {          // main değil, Main!
    printf("Merhaba\n");
    return 0;
}
```

C için `Main` sıradan bir fonksiyon adıdır, bu yüzden dosya sorunsuz **derlenir**. Ama bağlayıcı, programın başlayacağı `main` fonksiyonunu bulamaz. Windows'ta (MinGW gcc) şuna benzer bir mesaj çıkar (uzun yol bilgileri `...` ile kısaltıldı):

```
.../libmingw32.a(lib64_libmingw32_a-crt0_c.o): in function `main':
.../crt0_c.c:18:(.text.startup+0x2e): undefined reference to `WinMain'
collect2.exe: error: ld returned 1 exit status
```

Linux'ta gcc şunu yazar:

```
/usr/bin/ld: .../Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
```

Windows'taki `WinMain`, Windows'un grafik programlar için kullandığı başlangıç fonksiyonudur; anlam yine aynıdır: "programın başlangıç noktası bulunamadı". macOS'ta ise `Undefined symbols for architecture arm64: "_main"` ile başlayan bir mesaj görürsün. Hepsinin ortak noktası **`undefined reference`** (macOS'ta *undefined symbols*) ifadesidir; Windows ve Linux mesajları `ld returned 1 exit status` satırıyla biter (`ld`, bağlayıcının adıdır). Ayrıca senin kaynak dosyandaki bir satırı gösteren `dosya.c:satır:sütun:` konumu yoktur, çünkü bağlayıcı kaynak satırlarına değil nesne dosyalarına bakar.

Bağlama hatasının başka bir sebebi, çağırdığın bir fonksiyonun kodunun bulunamamasıdır. Örneğin Linux'ta `math.h`'deki `sqrt` gibi fonksiyonları kullanıp derleme komutuna `-lm` eklemezsen bu hatayı alırsın. Bunu Bölüm 2'de göreceksin.

3. Çalışma zamanı hatası (runtime error)

Kod kurallara uygundur, derlenir ve bağlanır; ama **çalışırken** yapılamayacak bir işlemle karşılaşılır. Tipik örnekler: bir tamsayıyı sıfıra bölmek, olmayan bir dosyayı açıp açılıp açılmadığını kontrol etmeden okumaya çalışmak (açma işlemi kendisi çökmez, Bölüm 12'de göreceğin gibi `NULL` döndürür), geçersiz bir bellek adresine erişmek.

```c
int bolen = 0;
int sonuc = 10 / bolen;   // derlenir, ama çalışırken sorun çıkar
```

Bu kod derlenir. Çalıştırdığında ise program çoğu bilgisayarda çöker; ama davranış işlemciye göre değişebilir, çünkü C bu durumu **tanımsız davranış** (undefined behavior) sayar: C standardı sonucu vaat etmez. Program çökebilir, yanlış değer üretebilir ya da sanki çalışıyormuş gibi görünebilir. Geçersiz bellek adresine erişimde Linux ve macOS terminali `Segmentation fault` gibi bir mesaj yazar; Windows'ta program çoğu zaman sessizce kapanır. Bu yüzden C'de uyarıları ciddiye almak çok önemlidir. Çalışma zamanı hatalarının ayrıntılarını konularıyla birlikte ileriki bölümlerde göreceksin.

4. Mantık hatası (logic error)

En sinsi hata budur: Program derlenir, çalışır, çökmez; ama **yanlış sonuç** üretir. Bilgisayar sana bir şey söylemez, hatayı sen bulmak zorundasın.

geriSayim.c

```c
#include <stdio.h>

int main(void) {
    // Amaç: 3, 2, 1, Başla! şeklinde geri sayım yapmak
    printf("1\n");
    printf("2\n");
    printf("3\n");
    printf("Başla!\n");
    return 0;
}
```

Çıktı

```
1
2
3
Başla!
```

Hiçbir hata mesajı yok, ama geri sayım ters çıktı. Gerçek programlarda mantık hataları çoğunlukla hesaplarda ortaya çıkar; örneğin parantezi unutup işlemleri yanlış sırayla yapmak. Bunları Bölüm 2'de operatör öncelikleriyle göreceksin.

Hata türü	Ne zaman ortaya çıkar?	Program çalışır mı?	Örnek
Derleme hatası	gcc'nin derleme aşamasında	Hayır, çalıştırılabilir dosya oluşmaz	Eksik `;`, yanlış yazılmış `printf`, unutulan `#include`
Bağlama hatası	Bağlayıcı aşamasında	Hayır, çalıştırılabilir dosya oluşmaz	`main` yerine `Main` yazmak
Çalışma zamanı hatası	Program çalışırken	Başlar; çökebilir ya da yanlış davranabilir	Tamsayıyı sıfıra bölmek
Mantık hatası	Sonuçlara bakınca	Evet, ama yanlış sonuç verir	Yanlış sırayla yazdırmak, unutulan parantez

> **Sınavda Çıkar**
>
> "Aşağıdaki hata hangi türdendir?" sorusu çok sık çıkar. İpucu: Eksik `;`, kapanmayan `}`, yanlış yazılmış `printf` → **derleme**. `undefined reference` → **bağlama**. Sıfıra bölme, geçersiz bellek erişimi → **çalışma zamanı**. Yanlış formül, yanlış sıra → **mantık**. Derleme ve bağlama hatalarında program hiç çalışmaz; ikisini ayırt etmenin yolu mesajdır: Mesajda `undefined reference` geçiyorsa bağlama hatasıdır, `dosya.c:satır:sütun: error` biçimindeki hatalar derleme hatasıdır.

1.10 main fonksiyonu ve return 0

Her C programı `main` fonksiyonundan başlar. Birden fazla `.c` dosyası olsa bile programda tam bir tane `main` bulunur. İlk satırı parçalayalım:

Parça	Anlamı
`int`	`main` işini bitirince işletim sistemine bir **tamsayı** (integer) döndürür
`main`	Fonksiyonun adı. Bu ad sabittir; bağlayıcı tam olarak `main`'i arar
`(void)`	`main` hiçbir **parametre** (dışarıdan değer) almıyor demektir

`return 0;` bu tamsayıyı döndürür. Buna **çıkış kodu** (exit status) denir. Gelenek şudur: `0` "her şey yolunda, program başarıyla bitti", sıfırdan farklı bir sayı "bir sorun oldu" demektir. Aşağıdaki program bilerek `3` döndürüyor:

donus3.c

```c
#include <stdio.h>

int main(void) {
    printf("Bir sorun oluştu!\n");
    return 3;
}
```

Çıktı – program 3 çıkış koduyla biter

```
Bir sorun oluştu!
```

Ekranda bir fark görmezsin, program normal bitmiş gibidir; ama çıkış kodu `3`'tür. Terminalde son programın çıkış kodunu şu komutlarla görebilirsin:

Terminal	Komut
Linux, macOS (bash, zsh)	`echo $?`
Windows PowerShell	`$LASTEXITCODE`
Windows Komut İstemi (cmd)	`echo %ERRORLEVEL%`

`./donus3` çalıştırıp hemen ardından `echo $?` yazarsan ekrana `3` gelir. Bu bilgi, bir programı başka bir programın ya da betiğin çalıştırdığı gerçek hayatta çok işe yarar (derleme sistemleri çıkış koduna bakarak devam eder ya da durur). Derste ise kuralı bil: Normal biten programlar `return 0;` ile biter.

C99'dan beri `main`'in sonuna `return 0;` yazmazsan derleyici sonuna kendisi `return 0;` ekler. Yani yazmasan da program çalışır. Ama sınavda ve bu kitapta **yaz**: Okuyan kişi, programın nasıl bittiğini açıkça görsün.

`int main(void)` yerine `int main()` yazıldığını da görebilirsin. C17'de ikisi de derlenir. `int main() { ... }` gibi gövdesiyle yazılan bir tanımda boş `()` da "parametre yok" demektir. Fark, Bölüm 6'da göreceğin **bildirimlerde** (prototip, gövdesiz satır) ortaya çıkar: orada `(void)` "hiç parametre yok" der, boş `()` ise "parametreler belirtilmedi" anlamına gelir. Alışkanlık olsun diye sınavda ve kitapta `int main(void)` kullanılır.

> **C23 Notu: main() ve main(void)**
>
> C23'te boş parantez `()` artık "hiç parametre yok" anlamına gelir; yani `int main()` ile `int main(void)` **eşdeğer** olur. Üstelik gcc 15'ten beri varsayılan standart C23'tür. Yine de bilmek iyidir, sınavda klasik C17 yaz: `int main(void)`. Bu yüzden derleme komutuna `-std=c17` ekliyoruz.

● Sık Yapılan Hata: void main yazma

İnternetteki eski örneklerde şu satırı görürsün:

```
void main() {          // YANLIŞ: standart dışı
```

`main` **her zaman** `int` döndürmelidir. `void main` standart değildir; gcc bunu `-Wall` ile `return type of 'main' is not 'int'` uyarısıyla belirtir. Bazı derleyiciler kabul eder, ama işletim sistemine belirsiz bir çıkış kodu gidebilir ve hocalar bunu yanlış sayar. Aynı eski kaynaklardan gelen şunları da kullanma: `conio.h` ve `getch()` yalnızca eski Windows derleyicilerinde vardır; `system("pause")` işletim sistemine bağımlıdır ve gereksiz yere başka bir program çalıştırır; `gets` güvenlik açığı yüzünden C11'de dilden çıkarılmıştır.

1.11 Büyük/küçük harf duyarlılığı

C, **büyük/küçük harf duyarlı** (case-sensitive) bir dildir: `printf` ile `Printf`, `main` ile `Main` bambaşka şeylerdir. Dilin sözcükleri (`int`, `return`, `void`) ve standart fonksiyonlar (`printf`, `main`) küçük harfle yazılır.

buyukPrintf.c

```c
#include <stdio.h>

int main(void) {
    Printf("Merhaba\n");
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
buyukPrintf.c: In function 'main':
buyukPrintf.c:4:5: error: implicit declaration of function 'Printf'; did you mean 'printf'? [-
Wimplicit-function-declaration]
    4 |     Printf("Merhaba\n");
      |     ^~~~~~
      |     printf
```

gcc `Printf` adında bir fonksiyon bilmiyor; yine de `did you mean 'printf'?` diyerek doğrusunu önerir. Aynı hatayı `Main` yazarsan bağlama aşamasında alırsın (az önce gördün).

`int`, `return`, `void` gibi anahtar sözcükler de tamamen küçük harftir: `Int` ya da `RETURN` yazarsan derleme hatası alırsın.

Windows dosya adlarında büyük/küçük harfi umursamaz; `Merhaba.c` ile `merhaba.c` aynı dosya sayılır. Linux'ta ise farklı dosyalardır. Dosya adlarını küçük harf ve ASCII karakterlerle yazmayı alışkanlık haline getir.

Bölüm Özeti – Bilmen Gerekenler

- Program, bilgisayara ne yapacağını söyleyen komutlar dizisidir. **Derleyici** kodu önceden bütün olarak makine koduna çevirir; **yorumlayıcı** satır satır çalıştırır. **C derlenen bir dildir.**
- C, 1972 civarında Dennis Ritchie tarafından Bell Labs'ta geliştirildi; UNIX'i yazmak için kullanıldı. Standartlar: C89, C99, C11, **C17 (kitabın standardı)**, C23.
- Derleme hattı: **önişlemci** (`#include`) → **derleyici** (→ `.s` assembly) → **çevirici** (`.s` → `.o`) → **bağlayıcı** (`.o` + kütüphane → çalıştırılabilir dosya).
- Derleme komutu: `gcc -std=c17 -Wall -Wextra merhaba.c -o merhaba`. Çalıştırma: Linux ve macOS'ta `./merhaba`, PowerShell'de `.\merhaba`, cmd'de `merhaba`. Uyarısız derle.
- Program `int main(void)` fonksiyonundan başlar; sonunda `return 0;` programın başarıyla bittiğini söyler. `#include` satırı `;` ile **bitmez**; her deyim `;` ile biter, her `{` bir `}` ile kapanır.
- `printf` alt satıra kendiliğinden geçmez. `\n` yeni satır, `\t` sekme, `\"` tırnak, `\\` ters bölü, `%%` yüzde işaretidir.
- Yorumlar: `//` tek satır, `/* */` çok satır. `/* */` yorumları **iç içe girmez**; string içindeki `//` yorum değildir.
- Hata türleri: **derleme** (sözdizimi, `file.c:satır:sütun: error`), **bağlama** (`undefined reference`), **çalışma zamanı** (çökme ya da tanımsız davranış), **mantık** (yanlış sonuç).
- C **büyük/küçük harf duyarlıdır**: `Printf` ve `Main` hatadır. `void main` standart dışıdır.

Sorular

A) Çoktan Seçmeli

1 Bir C programı çalıştırılabilir dosyaya dönüşürken geçtiği aşamalar hangi sıradadır?

(A) Bağlayıcı, çevirici, derleyici, önişlemci
(B) Derleyici, önişlemci, çevirici, bağlayıcı
(C) Önişlemci, derleyici, çevirici, bağlayıcı
(D) Önişlemci, çevirici, derleyici, bağlayıcı
(E) Yorumlayıcı, derleyici, çevirici, bağlayıcı

2 Aşağıdakilerden hangisi derleyici (compiler) ile yorumlayıcı (interpreter) arasındaki farkı doğru anlatır?

(A) Derleyici programın tamamını çalıştırmadan önce makine koduna çevirir; yorumlayıcı kodu satır satır okuyup o anda çalıştırır.
(B) Derleyici kodu satır satır çalıştırır; yorumlayıcı programın tamamını önceden çevirir.
(C) İkisi de aynı işi yapar, yalnızca adları farklıdır.
(D) Derleyici yalnızca hata mesajı verir, programı hiçbir şeye çevirmez.
(E) Yorumlayıcı yalnızca C programlarında kullanılır.

3 Aşağıdaki durumlardan hangisi bir **mantık hatasıdır**?

(A) `printf("Merhaba\n")` satırının sonundaki `;` unutulmuştur.

(B) `printf` yerine `Printf` yazılmıştır.

(C) `main` yerine `Main` yazılmıştır.

(D) Program çalışırken bir tamsayı sıfıra bölünmektedir.

(E) Program hatasız derlenir ve çalışır, ama 3, 2, 1 diye sayması gereken satırlar 1, 2, 3 diye yazdırılır.

4 `main` fonksiyonunun sonundaki `return 0;` satırı ne anlama gelir?

(A) Ekrana 0 sayısını yazdırır.

(B) Programın başarıyla bittiğini işletim sistemine bildirir.

(C) Programı 0 kez çalıştırır.

(D) Programın bir hata ile bittiğini bildirir.

(E) Bütün değişkenlerin değerini sıfırlar.

5 Aşağıdaki komutlardan hangisi ekrana tam olarak `Yol: C:\ders` yazdırır? (Satır sonu önemsizdir.)

(A) `printf("Yol: C:\ders");`

(B) `printf("Yol: C:/ders");`

(C) `printf("Yol: C:\\\\ders");`

(D) `printf("Yol: C:\\ders");`

(E) `printf('Yol: C:\ders');`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    printf("Not: %%85\n");
    printf("C:\\ders\\c\n");
    printf("\"Merhaba\"\n");
    printf("A\tB\tC\n");
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    printf("1\n");
    // printf("2\n");
    printf("3\n"); // printf("4\n");
    /* printf("5\n");
       printf("6\n"); */
    printf("7 // 8\n");
    printf("9 /* 10 */\n");
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır? Boş satır olup olmadığına dikkat et.

```c
#include <stdio.h>

int main(void) {
    printf("Ali");
    printf(" Veli\n");
    printf("Ayse\\n");
    printf("Fatma\n\n");
    printf("Son\n");
    return 0;
}
```

C) Kod Yazma

9 Adını ve okulunu aşağıdaki gibi bir **kutu içinde** yazdıran `kutu` programını yaz. Her satır için ayrı bir `printf` kullan. Kenarların hizalı olması için boşlukları dikkatle say.

```
+----------------------+
| Ad    : Elif Demir   |
| Okul  : Ankara Univ. |
+----------------------+
```

10 Aşağıdaki **desenli isim kartını** yazdıran `isimKarti` programını yaz. Desende ters bölü, çift tırnak ve yüzde işareti geçtiğine dikkat et.

```
/\/\/\/\/\/\/\/\/\
|                  |
|    Elif Demir    |
|  "Kod Yazarim"   |
|   Hedef: %100    |
|                  |
\/\/\/\/\/\/\/\/\/
```

Çözümler

Cevap anahtarı

1 C	2 A	3 E	4 B	5 D

1 Cevap: C

Sıra **önişlemci → derleyici → çevirici → bağlayıcı**'dır. Önişlemci `#include` gibi `#` satırlarını işler, derleyici kodu assembly diline (`.s`), çevirici de assembly'yi makine koduna (`.o`) çevirir, bağlayıcı da nesne dosyasını kütüphanelerle birleştirip çalıştırılabilir dosyayı üretir. A, B ve D sıralaması yanlıştır; E'de yorumlayıcı yoktur, C derlenen bir dildir.

2 Cevap: A

Derleyici bütün programı **çalışmadan önce** çevirir, yorumlayıcı **çalışırken satır satır** çevirir. B bu tanımı tersine çevirmiştir. C yanlış, ikisi farklı çalışır. D yanlış, derleyicinin asıl işi çevirmektir. E yanlış, yorumlayıcı C'ye özgü değildir; Python gibi dillerde kullanılır.

3 Cevap: E

Mantık hatasında program derlenir ve çalışır, ama yanlış sonuç verir; E tam olarak bunu anlatıyor. A ve B **derleme hatasıdır** (eksik `;`, tanınmayan `Printf`), C **bağlama hatasıdır** (`main` bulunamaz), D **çalışma zamanı hatasıdır**.

4 Cevap: B

`main`'in döndürdüğü `0`, **çıkış kodu**dur ve işletim sistemine "program başarıyla tamamlandı" der. A yanlış, ekrana yazdırma `printf`'in işidir. C ve E anlamsızdır. D tersini söylüyor: Hata bildirmek için sıfırdan farklı bir değer döndürülür (ör. `return 3;`).

5 Cevap: D

`\\` ters bölü karakterinin kendisini yazdırır; `Yol: C:\ders` için `C:\\ders` gerekir. A'da `\d` C standardında tanımlı bir kaçış dizisi değildir; gcc `unknown escape sequence` uyarısı verip `d` yazdırır (`Yol: C:ders`), ama bu gcc'nin kendi tercihidir, başka bir derleyici farklı davranabilir. B eğik çizgiyi `/` yazdırır. C iki ters bölü yazdırır (`C:\\ders`). E'de çift tırnak yerine tek tırnak kullanılmıştır; tek tırnak tek bir karakter içindir, metin için değil.

6

Satır satır izle:

1. `%%` tek bir `%` yazdırır: `Not: %85`.
2. `\\` tek ters bölü yazdırır, son `\n` alt satıra geçirir: `C:\ders\c`.
3. `\"` çift tırnak karakteridir: `"Merhaba"`.
4. `\t` sekme boşluğudur: `A`, `B`, `C` arasında birer tab vardır.

Çıktı

```
Not: %85
C:\ders\c
"Merhaba"
A	B	C
```

7

Yorumların içindekiler çalışmaz, string'in içindeki işaretler yorum değildir.

1. `printf("1\n");` normal çalışır: `1`.
2. `// printf("2\n");` tamamen yorumdur: **yok**.
3. `printf("3\n");` çalışır: `3`. Satır sonundaki `// printf("4\n");` yoktur.
4. `/* ... */` iki satırı birden yok sayar: 5 ve 6 **yok**.
5. `"7 // 8\n"`: `//` tırnağın içinde olduğu için yorum değildir, aynen yazılır.
6. `"9 /* 10 */\n"`: aynı şekilde aynen yazılır.

Çıktı

```
1
3
7 // 8
9 /* 10 */
```

8

İmlecin yerini izleyelim:

1. `printf("Ali");` → `Ali`, imleç aynı satırda.
2. `printf(" Veli\n");` → `Veli` aynı satıra eklenir, `\n` alt satıra geçirir: `Ali Veli`.
3. `printf("Ayse\\n");` → `\\` tek ters bölüdür, ardından gelen `n` sıradan bir harftir. Ekrana `Ayse\n` yazılır ve **alt satıra geçilmez**.
4. `printf("Fatma\n\n");` → `Fatma` aynı satırda devam eder, ilk `\n` satırı bitirir, ikinci `\n` boş bir satır bırakır.
5. `printf("Son\n");` → `Son`.

Çıktı

```
Ali Veli
Ayse\nFatma

Son
```

9

Her satırı ayrı bir `printf` ile yazdırmak yeterli. Kutunun sağ kenarı hizalı olsun diye "Elif Demir" satırının sonunda üç boşluk bırakıldığına dikkat et.

kutu.c

```c
#include <stdio.h>

int main(void) {
    printf("+----------------------+\n");
    printf("| Ad    : Elif Demir   |\n");
    printf("| Okul  : Ankara Univ. |\n");
    printf("+----------------------+\n");
    return 0;
}
```

Çıktı

```
+----------------------+
| Ad    : Elif Demir   |
| Okul  : Ankara Univ. |
+----------------------+
```

10

Ters bölü için `\\`, çift tırnak için `\"`, yüzde işareti için `%%` kullanılır. Üst ve alt kenardaki her `\` karakteri çift yazılır; satır sonu için ayrıca `\n` eklenir. Üst kenarın sonundaki `\\\n`, "bir ters bölü, ardından yeni satır" demektir.

isimKarti.c

```c
#include <stdio.h>

int main(void) {
    printf("/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\n");
    printf("|                                 |\n");
    printf("|    Elif Demir    |\n");
    printf("|  \"Kod Yazarim\"   |\n");
    printf("|   Hedef: %%100    |\n");
    printf("|                                 |\n");
    printf("\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\\/\n");
    return 0;
}
```

Çıktı

```
/\/\/\
```

BÖLÜM 2

Değişkenler, Veri Tipleri ve Operatörler

- **Bu Bölümde Öğreneceklerin**
  - Değişkenin ne olduğunu; tanımlamayı, ilk değer vermeyi ve atamayı
  - Geçerli değişken adı kurallarını, anahtar sözcükleri ve `camelCase` alışkanlığını
  - `char`, `short`, `int`, `long`, `long long`, `float`, `double` ve `unsigned` tiplerini; `sizeof` ve `limits.h` ile boyut ve sınır bulmayı
  - İlk değer verilmemiş değişkenin neden tehlikeli olduğunu (çöp değer)
  - `const`, `#define` ve sabit (literal) türlerini
  - Aritmetik operatörleri; tamsayı bölmesini ve `%` operatörünü (negatif sayılarla birlikte)
  - Örtük ve açık tip dönüşümünü (cast); `char`'ın aslında küçük bir tamsayı olduğunu
  - Bileşik atamayı (`+=`, `%=` ...) ve `++`/`--` operatörlerinin ön ek / son ek farkını
  - Operatör önceliğini, tamsayı taşmasını ve ondalıklı sayıların hassasiyetini
  - Bit düzeyinde operatörleri (`&`, `|`, `^`, `~`, `<<`, `>>`) ve `&` ile `&&` farkını
  - `math.h` fonksiyonlarını ve Linux'ta `-lm` gerektiğini

2.1 Değişken nedir?

Bir program veriyle çalışır: öğrencinin notu, ürünün fiyatı, kullanıcının yaşı. Bu verileri program çalıştığı sürece bir yerde tutman gerekir. Bu yer bilgisayarın **belleğidir** (RAM). **Değişken** (variable), bellekte **adı olan** ve içinde bir değer tutan kutudur. Her değişkenin üç özelliği vardır: **ad** (kutunun etiketi), **tip** (kutunun büyüklüğü ve içine ne konabileceği) ve **değer** (kutunun o anki içeriği).

```
   yas          boy            harf
+--------+   +----------+   +-------+
|   19   |   |   1.75   |   |   A   |
+--------+   +----------+   +-------+
  int          double         char
 4 bayt        8 bayt         1 bayt
```

Değişkene "değişken" dedirten şey, değerinin program boyunca **değişebilmesidir**: kutudaki sayıyı çıkarıp yenisini koyabilirsin. Tip ise değişmez; `int` açtığın kutu sonuna kadar `int` kalır. Bellekteki her kutunun bir de **adresi** vardır; adresleri Bölüm 9'da (işaretçiler) göreceksin.

2.2 Tanımlama, ilk değer verme ve atama

Bir değişkeni kullanmadan önce **tanımlaman** (declaration) gerekir: derleyiciye "şu tipte, şu adda bir kutu aç" dersin. Üç kalıbı ayır:

```
tip ad;                 // tanımlama
tip ad = ilkDeger;      // tanımlama + ilk değer verme (initialization)
ad = yeniDeger;         // atama (assignment): içeriği sonradan değiştirme
```

İlk değer verme, kutuyu açarken içine ilk değeri koymaktır; atama ise kutu zaten varken içeriği değiştirmektir. Sınavda "ikisi arasındaki fark nedir?" diye sorulabilir.

ilkDegisken.c

```c
#include <stdio.h>

int main(void) {
    int yas = 19;
    double boy = 1.75;
    char harf = 'A';

    printf("Yaş: %d\n", yas);
    printf("Boy: %f\n", boy);
    printf("Boy: %.2f\n", boy);
    printf("Harf: %c\n", harf);
    return 0;
}
```

Çıktı

```
Yaş: 19
Boy: 1.750000
Boy: 1.75
Harf: A
```

`printf` içindeki `%d`, `%f`, `%c` birer **yer tutucudur** (Bölüm 1'deki biçim belirteçleri): Virgülden sonra yazdığın değişkenin değeri o yere yerleşir. Şimdilik dört kalıp yeter, ayrıntısı Bölüm 3'te: `%d` tamsayı (`int`), `%f` ondalıklı sayı (`double`, virgülden sonra 6 basamak), `%.2f` virgülden sonra 2 basamaklı ondalıklı sayı, `%c` tek karakter. Ondalık ayırıcı virgül değil **noktadır** (`1.75`). Karakter **tek tırnakla** (`'A'`), metin çift tırnakla yazılır.

`=` işareti matematikteki "eşittir" değil, **atama operatörüdür**: Sağ taraf hesaplanır, sonuç soldaki kutuya konur. Bu yüzden `sayi = sayi + 1;` C'de gayet geçerlidir: "kutudaki değeri al, 1 ekle, aynı kutuya geri koy."

atamaOrnek.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 10;
    sayi = 25;                  // eski değer silinir
    sayi = sayi + 1;            // önce sağ taraf hesaplanır: 25 + 1
    printf("sayi = %d\n", sayi);

    int a = 3, b = 8;           // birden çok tanım tek satırda
    int gecici = a;             // takas: a'nın değerini sakla
    a = b;
    b = gecici;
    printf("a = %d, b = %d\n", a, b);

    int x, y, z;
    x = y = z = 0;              // sağdan sola: önce z, sonra y, sonra x
    printf("%d %d %d\n", x, y, z);
    return 0;
}
```

Çıktı
```
sayi = 26
a = 8, b = 3
0 0 0
```

Üç nokta: (1) Aynı tipte birden çok değişkeni virgülle **tek satırda** tanımlayabilirsin; her birinin kendi ilk değeri olur. (2) İki değişkenin değerini değiştirmek (**takas**, swap) için üçüncü bir kutu gerekir; `a = b; b = a;` yazarsan iki kutuda da `8` kalır. (3) C99'dan beri değişkeni bloğun ortasında tanımlayabilirsin; tek koşul onu **kullanmadan önce** tanımlamaktır.

2.3 Değişken adları ve anahtar sözcükler

Değişkene istediğin adı veremezsin. Kurallar:

1. Ad **harf, rakam ve alt çizgiden** (`_`) oluşur ve **rakamla başlamaz**.
2. Boşluk, `-`, `+`, `.`, `$` gibi özel karakterler kullanılamaz. (gcc `$` işaretini eklenti olarak kabul eder, ama standart C'de geçerli değildir; kullanma.)
3. **Anahtar sözcük** (keyword) olamaz (`int`, `for`, `return` ...).
4. Büyük/küçük harf farklıdır (Bölüm 1): `sayi`, `Sayi` ve `SAYI` üç ayrı değişkendir.
5. Türkçe karakterleri (`ş`, `ğ`, `ı`, `ö`, `ü`, `ç`) kullanma. Yeni gcc sürümleri kabul etse de eski derleyiciler ve farklı dosya kodlamaları sorun çıkarır.

Ad	Geçerli mi?	Neden
`ogrenciSayisi`, `sayi2`, `_toplam`	Evet	Rakam sonda olabilir, `_` ile başlanabilir (önerilmez)
`2sayi`	**Hayır**	Rakamla başlıyor
`ogrenci sayisi`	**Hayır**	Boşluk var
`ortalama-not`	**Hayır**	`-` çıkarma işaretidir: `ortalama - not` diye okunur
`int`	**Hayır**	Anahtar sözcük

Hatalı ad gcc'de şöyle görünür:

adRakam.c
```c
int main(void) {
    int 2sayi = 5;
    return 0;
}
```

Derleyici çıktısı – program derlenmez
```
adRakam.c: In function 'main':
adRakam.c:2:9: error: invalid suffix 'sayi' on integer constant
    2 |     int 2sayi = 5;
      |         ^~~~
adRakam.c:2:9: error: expected identifier or '(' before numeric constant
```

gcc `2sayi`'yi "2 sayısına yapışmış `sayi` eki" gibi okudu, bu yüzden ilk mesaj tuhaf. Asıl ipucu ikinci satırda: Ad bekleniyordu, sayı geldi. `int for = 5;` gibi anahtar sözcük kullanımı da `expected identifier or '(' before 'for'` hatası verir.

C17'de **44 anahtar sözcük** vardır; hepsini ezberleme, editör onları farklı renkte gösterir. Tip adları (`char`, `int`, `double`, `unsigned` ...), karar ve döngü sözcükleri (`if`, `else`, `switch`, `for`, `while`, `do`, `break`, `continue`, `goto`), `return`, `sizeof`, `const`, `static`, `struct`, `enum`, `typedef`, `void` bunlardandır; kitapta zamanla hepsini tanıyacaksın. `main` ve `printf` anahtar sözcük değildir, ama onların adını da değişkene verme.

Adlandırma alışkanlığı. Kitapta **camelCase** kullanıyoruz: İlk sözcük küçük, sonrakilerin baş harfi büyük (`ogrenciSayisi`, `ortalamaNot`). Ad, değişkenin ne tuttuğunu söylemeli: `x` yerine `yas`. Sabitlerin adı BÜYÜK_HARFLE yazılır (`PI`, `MAX_NOT`). Koddaki adları Türkçe ASCII yaz; Türkçe karakter yalnız metinlerde (string) kullanılır. `i`, `j` gibi tek harfli adlar yalnızca döngü sayaçlarında (Bölüm 5) kabul görür.

2.4 Temel veri tipleri

Bellek **bayt** (byte) denen küçük parçalara bölünmüştür; **1 bayt = 8 bit** ve n bitle 2^n farklı değer saklanabilir. Tip, değişkene **kaç bayt ayrılacağını** ve baytların nasıl yorumlanacağını belirler:

Tip	Ne tutar?	Boyut	Aralık (işaretli)	`printf`
`char`	Karakter ya da küçük tamsayı	1 bayt	-128 ... 127	`%c` ya da `%d`
`short`	Küçük tamsayı	2 bayt	-32768 ... 32767	`%hd` ya da `%d`
`int`	Tamsayı	4 bayt	yaklaşık ±2,1 milyar	`%d`
`long`	Büyük tamsayı	4 ya da 8 bayt	platforma göre	`%ld`
`long long`	Çok büyük tamsayı	8 bayt	yaklaşık $\pm 9{,}2 \times 10^{18}$	`%lld`
`float`	Ondalıklı sayı, tek duyarlık	4 bayt	yaklaşık 7 anlamlı basamak	`%f`
`double`	Ondalıklı sayı, çift duyarlık	8 bayt	yaklaşık 15 anlamlı basamak	`%f`

`%hd`, `%ld` ve `%lld` yeni belirteçler; şimdilik kalıp olarak al, Bölüm 3'te ayrıntısı var. Birkaç kural:

- Tamsayı tipleri varsayılan olarak **işaretlidir** (signed): negatif de olabilir. Başına `unsigned` (işaretsiz) yazarsan negatif değer tutmaz, ama pozitif tarafı iki katına çıkar: `unsigned char` 0 ... 255, `unsigned int` 0 ... 4294967295 (`printf`'te `%u`).
- `short int` yerine `short`, `long int` yerine `long`, `unsigned int` yerine `unsigned` yazabilirsin; anlamları aynıdır.
- `char`'ın işaretli mi işaretsiz mi olduğunu standart derleyiciye bırakır; PC ve Mac'teki gcc'de işaretlidir.
- Ondalıklı sayı için kitapta **`double`** kullanacağız. `float`'ın sabitleri `3.14f` diye yazılır. C'nin ilk sürümlerinde `bool` tipi yoktu; C99'dan beri `stdbool.h` ile gelir. Doğru/yanlış kavramını Bölüm 4'te göreceksin.

Standart boyutları kesin vermez, yalnızca alt sınır koyar ve `char` ≤ `short` ≤ `int` ≤ `long` ≤ `long long` sıralamasını garanti eder. Tek kesin değer `sizeof(char) == 1`'dir. Boyutları `sizeof` operatörüyle öğrenirsin:

boyutlar.c

```c
#include <stdio.h>
#include <limits.h>

int main(void) {
    printf("char      : %zu\n", sizeof(char));
    printf("short     : %zu\n", sizeof(short));
    printf("int       : %zu\n", sizeof(int));
    printf("long      : %zu\n", sizeof(long));
    printf("long long : %zu\n", sizeof(long long));
    printf("float     : %zu\n", sizeof(float));
    printf("double    : %zu\n", sizeof(double));
    printf("INT_MIN   : %d\n", INT_MIN);
    printf("INT_MAX   : %d\n", INT_MAX);
    printf("UINT_MAX  : %u\n", UINT_MAX);
    printf("LLONG_MAX : %lld\n", LLONG_MAX);
    return 0;
}
```

Çıktı

```
char      : 1
short     : 2
int       : 4
long      : 8
long long : 8
float     : 4
double    : 8
INT_MIN   : -2147483648
INT_MAX   : 2147483647
UINT_MAX  : 4294967295
LLONG_MAX : 9223372036854775807
```

`sizeof` bir fonksiyon değil **operatördür**. Sonucu bayt sayısıdır; tipi `size_t` (işaretsiz tamsayı) olduğu için `%zu` ile yazdırılır. Değişkene de uygulanır: `sizeof(yas)`, `int` bir değişken için `4` verir. Alt satırlar `limits.h` başlık dosyasının sunduğu hazır sınır sabitleridir (`SHRT_MAX`, `LONG_MAX` gibi başkaları da vardır); sınırları ezberlemene gerek yok. Sınavda en çok `INT_MAX = 2147483647` ve `INT_MIN = -2147483648` sorulur. İşaretli *n* bitlik tamsayının aralığı `-2^(n-1)` … `2^(n-1) - 1`'dir; negatif taraf bir fazladır, çünkü `0` pozitif tarafta sayılır.

> **İpucu: Ekranda `zu` mu görüyorsun?**
>
> Eski Dev-C++ sürümlerindeki derleyici Windows'un eski C kütüphanesini kullanır ve `%zu`'yu tanımayabilir; o zaman ekrana sayı yerine `zu` yazılır. Çözüm: `printf("%u\n", (unsigned)sizeof(int));` biçimini kullan ya da Bölüm 1'deki MSYS2 UCRT64 derleyicisine geç. Aynı sorun `%lld` için de görülebilir.

> **Not: long kaç bayt?**
>
> Bu çıktı macOS'ta (Apple M4) alındı ve `long` satırı **8** yazıyor. Windows'ta `long` **4** bayttır; Linux ve macOS'ta 8. Yani aynı program senin bilgisayarında `long` satırında farklı bir sayı yazabilir; diğer satırlar bugünkü PC ve Mac'lerde aynıdır. Standart bunu da garanti etmez: Örneğin Arduino gibi küçük mikrodenetleyicilerde `int` 2 bayttır. Taşınabilir kod için 4 bayttan büyük sayı gerekiyorsa `long long` kullan: o her yerde 8 bayttır.

2.5 İlk değer verilmemiş değişken

Değişkeni tanımlayıp içine hiçbir şey koymazsan kutuda **o bellek bölgesinde daha önce kalmış rastgele bir değer** bulunur. Buna **çöp değer** (garbage value) denir:

```c
int sayi;                   // ilk değer verilmedi
printf("%d\n", sayi);       // çöp değer yazar
```

Bu değeri okumak **tanımsız davranıştır** (Bölüm 1). Ekrana `0` da yazabilir, `21845` de, her çalıştırmada farklı bir sayı da. Kötüsü, tesadüfen `0` yazıp programın "çalışıyormuş" gibi görünmesidir; hata başka bilgisayarda ya da sınav günü ortaya çıkar. Bu yüzden böyle kodun çıktısını göstermiyoruz. `-Wall` bu hatayı çoğu zaman yakalar:

copDeger.c

```c
#include <stdio.h>

int main(void) {
    int sayi;
    printf("%d\n", sayi);
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
copDeger.c: In function 'main':
copDeger.c:5:5: warning: 'sayi' is used uninitialized [-Wuninitialized]
    5 |     printf("%d\n", sayi);
      |     ^~~~~~~~~~~~~~~~~~~~
copDeger.c:4:9: note: 'sayi' was declared here
    4 |     int sayi;
      |         ^~~~
```

Kural: Değişkene ilk değerini tanımlarken ver. Değeri sonradan klavyeden okuyacak olsan bile `int sayi = 0;` yaz.

> **Sık Yapılan Hata: int a, b, c = 0; kalıbı**
>
> `int a, b, c = 0;` satırı **üç değişkeni de** sıfırlamaz; yalnız `c` sıfır olur. `a` ile `b` çöp değerle kalır. Her değişkenin ilk değeri ayrı yazılır: `int a = 0, b = 0, c = 0;`.

2.6 Sabitler

Bazı değerler program boyunca değişmemelidir: `π`, haftadaki gün sayısı, KDV oranı. Bunları koda sayı olarak gömmek (**sihirli sayı**, magic number) kötü alışkanlıktır: Kodda `0.20` yazan yüz yerin hangisi KDV'dir? Sabit tanımlamanın iki yolu vardır.

const ile sabit

Tanımlamanın başına `const` yazarsan değişken **salt okunur** olur: Değeri ilk değer verirken koyarsın, sonra değiştiremezsin.

constAlan.c

```c
#include <stdio.h>

int main(void) {
    const double PI = 3.14159;
    const int HAFTA_GUN = 7;
    double yaricap = 2.0;

    printf("Çevre: %.2f\n", 2 * PI * yaricap);
    printf("Haftada %d gün var.\n", HAFTA_GUN);
    return 0;
}
```

```
Çıktı
Çevre: 12.57
Haftada 7 gün var.
```

Sabite sonradan değer atamaya çalışırsan (`HAFTA_GUN = 8;`) gcc `assignment of read-only variable 'HAFTA_GUN'` hatası verir ve program derlenmez.

#define ile sabit

`#define` bir **önişlemci komutudur** (Bölüm 1): Önişlemci, derleme başlamadan önce metindeki adı karşılığıyla **bul-değiştir** gibi değiştirir. `#define` değişken oluşturmaz, bellekte yer kaplamaz.

defineSabit.c

```c
#include <stdio.h>
#define PI 3.14159
#define MAX_NOT 100

int main(void) {
    printf("Alan: %.2f\n", PI * 2 * 2);
    printf("En yüksek not: %d\n", MAX_NOT);
    return 0;
}
```

```
Çıktı
Alan: 12.57
En yüksek not: 100
```

Önişlemci `PI * 2 * 2` ifadesini `3.14159 * 2 * 2` yaptı. Sözdizimi `#define AD DEGER`'dir: sonunda `;` yok, `=` yok, tip yok.

> **Sık Yapılan Hata: `#define` satırına `;` koymak**
>
> `#define` düz **metin değiştirmedir**; `;` da değiştirilen metne dahildir. Aşağıdaki program masum görünür, ama `KDV_ORANI` yerine `0.20;` yazılır:
>
> defineNokta.c
>
> ```c
> #define KDV_ORANI 0.20;
>
> int main(void) {
>     double fiyat = 100 * (1 + KDV_ORANI);
>     return 0;
> }
> ```
>
> Satır `100 * (1 + 0.20;);` olur ve gcc `expected ')' before ';' token` hatası verir. Hata `#define` satırını gösterir ve `in expansion of macro 'KDV_ORANI'` notu düşer. Aynı mantıkla `#define PI = 3.14` de yanlıştır: `=` işareti de metne karışır.

`const` ile `#define` arasındaki farklar: `const`'ı **derleyici** işler, tipi vardır (`const int`), bir değişkendir ve normal tanımlama gibi `;` ile biter. `#define`'ı **önişlemci** işler, tipi yoktur (yalnız metin), bellekte yer kaplamaz ve `;` ile **bitmez**. İkisi de kabul edilir; tip denetimi sağladığı için modern kodda `const` tercih edilir, ders kitaplarında ve sınavlarda `#define` daha sık görünür (özellikle dizi boyutlarında, Bölüm 7).

Sabit değer türleri (literal)

Koda doğrudan yazdığın değerlere **sabit değer** (literal) denir. Yazılışı tipini belirler:

Yazılış	Tip	Not
`5`	`int`	
`5L` / `5LL` / `5U`	`long` / `long long` / `unsigned int`	Sonek harfi tipi seçer
`5.0`	`double`	Noktalı sayı varsayılan olarak `double`'dır
`5.0f`	`float`	`f` soneki
`'A'`	`int` (C'de!)	Tek tırnak, tek karakter; değeri karakter kodudur (65)
`"A"`	metin (string)	Çift tırnak; `'A'` ve `'\0'` olmak üzere iki karakterlik dizi, Bölüm 8
`0x1F`, `017`	`int`	Onaltılık (hexadecimal) 31; **sekizlik** (octal, başında `0`) 15

sabitTipler.c

```c
#include <stdio.h>

int main(void) {
    printf("%zu %zu %zu\n", sizeof(5), sizeof(5.0), sizeof(5.0f));
    printf("%zu\n", sizeof('A'));
    printf("%d %d %d\n", 0x1F, 017, 010);
    return 0;
}
```

Çıktı

```
4 8 4
4
31 15 8
```

İki tuzak: C'de `'A'` bir `char` değil **`int`** tipindedir, bu yüzden `sizeof('A')` 4'tür (C++'ta 1). Ayrıca başına `0` yazılan sayı **sekizliktir**: `010` sekiz eder, on değil. Sayıların başına anlamsızca sıfır koyma.

2.7 Aritmetik operatörler

Operatör (operator) bir işlem yapan simgedir; işlemin uygulandığı değerlere **işlenen** (operand) denir. `a + b` ifadesinde `+` operatör, `a` ve `b` işlenendir. Dört işlemi bildiğin gibi kullanırsın (`+ - * /`); beşincisi `%`, bölümden **kalanı** verir. Önce hepsini görelim, sonra bölmeye ve `%`'ye yakından bakalım.

aritmetik.c

```c
#include <stdio.h>

int main(void) {
    int a = 17, b = 5;
    printf("%d + %d = %d\n", a, b, a + b);
    printf("%d - %d = %d\n", a, b, a - b);
    printf("%d * %d = %d\n", a, b, a * b);
    printf("%d / %d = %d\n", a, b, a / b);
    printf("%d %% %d = %d\n", a, b, a % b);
    printf("1 / 2    = %d\n", 1 / 2);
    printf("-7 / 2   = %d\n", -7 / 2);
    printf("7 / 2.0  = %.1f\n", 7 / 2.0);
    return 0;
}
```

Çıktı

```
17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3
17 % 5 = 2
1 / 2    = 0
-7 / 2   = -3
7 / 2.0  = 3.5
```

`17 / 5` neden `3.4` değil de `3`? Çünkü **tamsayı bölmesi** vardır: İki işlenen de tamsayıysa bölmenin sonucu da tamsayıdır; kesirli kısım **atılır** (yuvarlanmaz, kesilir). `1 / 2` bu yüzden `0`'dır. İşlenenlerden en az biri ondalıklıysa (`7 / 2.0`) sonuç ondalıklı olur. `-7 / 2` sonucu `-4` değil `-3`'tür: C99'dan beri bölme **sıfıra doğru** keser (`-3.5` → `-3`). Tamsayıyı sıfıra bölmek ise çalışma zamanı hatasıdır (Bölüm 1); `x / 0` ve `x % 0` yazma.

Kalan operatörü: %

`a % b`, `a`'yı `b`'ye böldüğünde artan kalandır. Bölme sıfıra doğru kesildiği için `a == (a / b) * b + a % b` eşitliği hep sağlanır ve **kalanın işareti bölünenin işaretiyle aynıdır**:

modNegatif.c

```c
#include <stdio.h>

int main(void) {
    printf("7 %% 2    = %d\n", 7 % 2);
    printf("-7 %% 2   = %d\n", -7 % 2);
    printf("7 %% -2   = %d\n", 7 % -2);
    printf("-7 %% -2  = %d\n", -7 % -2);
    printf("2 %% 5    = %d\n", 2 % 5);
    return 0;
}
```

Çıktı

```
7 % 2    = 1
-7 % 2   = -1
7 % -2   = 1
-7 % -2  = -1
2 % 5    = 2
```

Python biliyorsan dikkat: Python'da `-7 % 2` sonucu `1`'dir, C'de `-1`. Küçük sayıyı büyüğe bölünce kalan, sayının kendisidir: `2 % 5 == 2`.

`%` sınavın en sevilen operatörüdür ve hep aynı işlere yarar. `1453` sayısı için: `1453 % 10` son basamağı (`3`), `1453 / 10` son basamağı atılmış sayıyı (`145`) verir; `sayi % 2` çift/tek ayrımı yapar (çiftse `0`); `(22 + 5) % 24` ise saat gibi başa dönen sayaçlar içindir (`3`).

`%` yalnızca **tamsayılarla** çalışır; işlenenlerden biri `double` ise program derlenmez (ondalıklı kalan için `math.h`'deki `fmod`, bölümün sonunda):

modDouble.c

```c
int main(void) {
    double a = 7.5;
    return a % 2;
}
```

Derleyici çıktısı – program derlenmez

```
modDouble.c: In function 'main':
modDouble.c:3:14: error: invalid operands to binary % (have 'double' and 'int')
    3 |     return a % 2;
      |              ^
```

> **Sınavda Çıkar**
>
> "`-17 % 5` kaçtır?" ve "`7 / 2 * 2.0` kaçtır?" tipi sorular her yıl çıkar. Üç kural: (1) İki tamsayının bölmesi tamsayıdır. (2) `%`'de kalan bölünenin işaretini taşır: `-17 % 5 = -2`, `17 % -5 = 2`. (3) Aynı öncelikteki işlemler soldan sağa yapılır: `7 / 2 * 2.0` önce `7 / 2 = 3`, sonra `3 * 2.0 = 6.0`.

2.8 Tip dönüşümü

Bir ifadede farklı tipler karışırsa ne olur? İki durum vardır: derleyici kendiliğinden dönüştürür (örtük) ya da sen zorlarsın (açık).

Örtük dönüşüm

Bir işlemin iki işleneni farklı tipteyse C **küçük tipi büyük tipe yükseltir**. Sıra kabaca: `char`, `short` → `int` → `long` → `long long` → `float` → `double`. `int + double` işleminin sonucu `double` olur. Atamada ise sağdaki değer, soldaki değişkenin tipine çevrilir: `double`'dan `int`'e atarken ondalık kısım **atılır** (yuvarlama yapılmaz, sıfıra doğru kesilir); `int`'ten `double`'a atama sorunsuzdur.

ortukDonusum.c

```c
#include <stdio.h>

int main(void) {
    int tamsayi = 5;
    double toplam = tamsayi + 2.5;   // int, double'a yükselir
    int a = 3.99;                    // ondalık kısım atılır
    int b = -3.99;                   // sıfıra doğru: -3
    double c = 7 / 2;                // önce tamsayı bölmesi (3), sonra 3.0
    double d = 7 / 2.0;

    printf("toplam = %.1f\n", toplam);
    printf("a = %d, b = %d\n", a, b);
    printf("c = %.1f, d = %.1f\n", c, d);
    printf("5 / 2 + 0.5 = %.1f\n", 5 / 2 + 0.5);
    return 0;
}
```

Çıktı

```
toplam = 7.5
a = 3, b = -3
c = 3.0, d = 3.5
5 / 2 + 0.5 = 2.5
```

Dönüşüm **işlem işlem** yapılır, ifadenin tamamına bakılmaz. `double c = 7 / 2;` klasik tuzaktır: Değişkenin `double` olması bölmenin tamsayı yapılmasını engellemez, çünkü bölme **atamadan önce** yapılır. Aynı şekilde `5 / 2 + 0.5` ifadesinde önce `5 / 2` iki tamsayı olduğu için `2` çıkar; `double` yalnız sonraki toplamada devreye girer ve sonuç `3.0` değil `2.5` olur. gcc `-Wall -Wextra` ile bu satırlar için uyarı vermez; farkı sen görmek zorundasın.

> **Sık Yapılan Hata: Ortalamayı tamsayı bölmesiyle hesaplamak**
>
> `toplam` ve `adet` `int` iken `double ortalama = toplam / adet;` yazarsan `ortalama` değişkeni `double` olsa bile sonuç **kesilmiş** çıkar (toplam `145`, adet `2` ise `72.0`; doğrusu `72.5`). Çözüm işlenenlerden birini `double` yapmaktır; yolu aşağıda.

Açık dönüşüm (cast)

Dönüşümü kendin yapmak için ifadenin önüne parantez içinde tip yazarsın: `(tip) ifade`. Buna **tip dönüştürme** (type casting) ya da kısaca **cast** denir.

castFark.c

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = 2;
    printf("a / b           : %d\n", a / b);
    printf("(double)a / b   : %.2f\n", (double)a / b);
    printf("(double)(a / b) : %.2f\n", (double)(a / b));
    printf("(int)(-3.99)    : %d\n", (int)(-3.99));
    return 0;
}
```

Çıktı

```
a / b           : 3
(double)a / b   : 3.50
(double)(a / b) : 3.00
(int)(-3.99)    : -3
```

İlk iki ifade arasındaki fark sınavın klasik sorusudur. `(double)a / b` ifadesinde cast, `/` operatöründen **önce** uygulanır: `a` önce `7.0` olur, sonra `7.0 / 2` ondalıklı bölme olarak `3.5` verir. `(double)(a / b)` ifadesinde ise parantez içi önce hesaplanır: `7 / 2 = 3` olur ve bu `3` sonradan `3.0` yapılır; kesilen `.5` geri gelmez.

Cast değişkenin **kendisini değiştirmez**; yalnız o ifadede kullanılan değerin başka tipte bir kopyasını üretir. `a` hâlâ `int`'tir.

> **Sınavda Çıkar**
>
> Cast'in **yeri** puanı belirler: `(double)toplam / adet` doğru ortalamadır, `(double)(toplam / adet)` yanlış. Cast bölmeden önce, bölünene (ya da böleni) uygulanmalıdır. Pozitif sayıyı yuvarlamak için `(int)(x + 0.5)` kalıbını da bil; negatiflerde çalışmaz, orada `math.h`'deki `round` kullanılır.

Tamsayı yükseltmesi

`char` ve `short` gibi `int`'ten küçük tipler aritmetikte kendi boyutlarıyla işlem görmez: işlemden önce otomatik olarak `int`'e yükseltilir (**tamsayı yükseltmesi**, integer promotion). İki `char`'ın toplamı `char` değil `int`'tir; bu yüzden `char`'a sığmayan sonuç bile doğru hesaplanır:

yukseltme.c

```c
#include <stdio.h>

int main(void) {
    char c = 100;
    short s = 30000;

    printf("c + c = %d\n", c + c);
    printf("s * 2 = %d\n", s * 2);
    printf("sizeof(c) = %zu, sizeof(c + c) = %zu\n", sizeof(c), sizeof(c + c));
    return 0;
}
```

Çıktı

```
c + c = 200
s * 2 = 60000
sizeof(c) = 1, sizeof(c + c) = 4
```

`c` en çok 127 tutabilen bir `char`'dır, ama `c + c` `int` olarak hesaplandığı için `200` çıktı; `short` için de aynıdır. `sizeof` farkı açıkça gösterir: `c` 1 bayt, `c + c` 4 bayttır. Sonucu tekrar `char` değişkene atarsan sığmayan değer bozulur, bu yüzden ara hesapları `int` değişkende tut. `printf`'e verilen `char` da `int`'e yükseltilir; `%d` ile `%c`'nin ikisinin de çalışmasının sebebi budur.

İşaretli ile işaretsiz karşılaştırma tuzağı

`int` ile `unsigned int` aynı işlemde karşılaşırsa C **işaretliyi işaretsize çevirir**, tersini değil. `-1` işaretsize çevrilince en büyük değere, `4294967295`'e sarar (taşma bölümündeki mantık). Karşılaştırma bu yüzden şaşırtıcı sonuç verir:

isaretKarsilastirma.c

```c
#include <stdio.h>

int main(void) {
    int a = -1;
    unsigned int b = 1;

    printf("a'nın işaretsiz hali: %u\n", (unsigned int)a);
    if (a < b) {
        printf("-1, 1'den küçük\n");
    } else {
        printf("-1, 1'den küçük DEĞİL (!)\n");
    }
    if (a < (int)b) {
        printf("cast ile doğru: -1 < 1\n");
    }
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
isaretKarsilastirma.c: In function 'main':
isaretKarsilastirma.c:8:11: warning: comparison of integer expressions of different signedness:
'int' and 'unsigned int' [-Wsign-compare]
    8 |     if (a < b) {
      |           ^
```

Çıktı

```
a'nın işaretsiz hali: 4294967295
-1, 1'den küçük DEĞİL (!)
cast ile doğru: -1 < 1
```

`a < b` ifadesinde önce `a` işaretsize çevrildi: `4294967295 < 1` yanlıştır. gcc bu tuzağı yalnızca `-Wextra` ile (`-Wsign-compare`) uyarır; `-Wall` tek başına susar, bu yüzden ikisini birden kullan. Çözüm iki tarafı aynı işarette tutmaktır: değişkenleri baştan aynı tipte tanımla ya da, değerin `int`'e sığdığından eminsen, `(int)b` gibi cast kullan. `<`, `>`, `<=`, `>=` ve `==` aynı kuralı izler. Bölüm 8'de `strlen`'in işaretsiz `size_t` döndürmesi yüzünden aynı tuzakla `strlen(s) - 1` ve `i < strlen(s)` biçiminde karşılaşacaksın; derleyici mesajı Ek C'de.

> **Sınavda Çıkar**
>
> `int i = -1; unsigned int u = 1;` iken `i < u` **yanlış** (0) çıkar: işaretli olan işaretsize çevrilir. Aynı sebeple `unsigned int i` ile yazılan `for (i = 5; i >= 0; i--)` döngüsü hiç bitmez; `0 - 1` en büyük değere sarar ve `i >= 0` işaretsiz bir sayı için her zaman doğrudur.

2.9 char: küçük bir tamsayı

`char` tek karakter tutar, ama bellekte karakteri değil onun **kodunu** saklar: `'A'` için `65`, `'a'` için `97`, `'0'` için `48`. Bu eşleştirme **ASCII tablosudur** (tamamı Ek D'de). Yani `char` aslında 1 baytlık küçük bir tamsayıdır; aynı değeri `%c` ile karakter, `%d` ile sayı olarak yazdırabilirsin:

charTamsayi.c

```c
#include <stdio.h>

int main(void) {
    char harf = 'A';
    printf("%c %d\n", harf, harf);
    printf("%c %d\n", harf + 1, harf + 1);
    printf("%d\n", 'a' - 'A');

    char rakam = '7';
    printf("%d\n", rakam - '0');

    char kucuk = 'g';
    char buyuk = kucuk - 'a' + 'A';
    printf("%c -> %c\n", kucuk, buyuk);
    return 0;
}
```

Çıktı

```
A 65
B 66
32
7
g -> G
```

Buradan çıkan sonuçlar:

- Büyük ve küçük harf arasında hep `32` fark vardır; küçük harfi büyüğe çevirmek `kucuk - 'a' + 'A'` ile yapılır.
- `'7'` karakteriyle `7` sayısı **aynı şey değildir**: `'7'` ASCII'de `55`'tir. Rakam karakterinin sayı değerini bulmak için `'0'` çıkarılır: `'7' - '0' == 7`.
- Aritmetikte `char` önce `int`'e yükseltilir. `harf + 1` bir `int`'tir; `%c` ile yazdırırsan karaktere (`B`), `%d` ile yazdırırsan koda (`66`) çevrilir.

2.10 Bileşik atama, ++ ve --

Bileşik atama

`x = x + 5` gibi "değişkeni hesapla ve aynı değişkene geri koy" kalıbı o kadar sık geçer ki C'de kısaltması vardır: **bileşik atama** (compound assignment). `x op= y`, `x = x op y` demektir: `+=`, `-=`, `*=`, `/=`, `%=`.

bilesik.c

```c
#include <stdio.h>

int main(void) {
    int x = 10;
    x += 5;                 // 15
    x -= 3;                 // 12
    x *= 2;                 // 24
    x /= 5;                 // 24 / 5: tamsayı bölmesi, 4
    x %= 3;                 // 4 % 3 = 1
    printf("x = %d\n", x);

    int y = 4;
    y *= 2 + 3;             // y = y * (2 + 3)
    printf("y = %d\n", y);
    return 0;
}
```

Çıktı

```
x = 1
y = 20
```

Dikkat: `y *= 2 + 3`, `y = y * 2 + 3` **değildir**. Sağ taraf bir bütündür; `y = y * (2 + 3)` olur ve sonuç `11` değil `20`'dir.

Artırma ve azaltma: ++ ve --

Bir değişkeni 1 artırmak o kadar yaygındır ki ayrı operatörü vardır: `x++` ve `x--`, `x = x + 1` ve `x = x - 1` demektir. Operatör değişkenin **önüne** (ön ek, prefix) ya da **arkasına** (son ek, postfix) yazılabilir; bir ifadenin içinde kullanılınca fark yaratır. `x++` (son ek) ifade olarak `x`'in **eski** değerini verir, sonra `x` 1 artar. `++x` (ön ek) önce `x`'i 1 artırır, ifade olarak **yeni** değeri verir. `--` için mantık aynıdır.

onSonEk.c

```c
#include <stdio.h>

int main(void) {
    int n = 5;
    printf("%d\n", n++);
    printf("%d\n", n);
    printf("%d\n", ++n);
    printf("%d\n", n--);
    printf("%d\n", --n);
    return 0;
}
```

Çıktı

```
5
6
7
7
5
```

İzle: `n++` eski değer `5`'i verir, sonra `n = 6` olur; `n` yazdırılınca `6`. `++n` önce `7` yapar, `7` yazdırır. `n--` eski değer `7`'yi verir, sonra `n = 6`; `--n` önce `5` yapar ve `5` yazdırır. `x++;` satırı **tek başına** yazılırsa ön ek ile son ek arasında fark yoktur; fark yalnız ifadenin değeri kullanılırken çıkar. `++` ve `--` yalnız değişkenlere uygulanır: `5++` yazılamaz.

Bir değişkeni **tek bir ifadede iki kez değiştirmek** (ya da `++`/`--` ile değiştirip aynı ifadede bir de okumak) tanımsız davranıştır. Derleyici hangi `i++`'in önce yapılacağını garanti etmez:

```c
int i = 5;
int x = i++ + i++;          // tanımsız
i = i++;                    // tanımsız
printf("%d %d", i, i++);    // tanımsız
```

gcc bunları `-Wall` ile yakalar: `operation on 'i' may be undefined [-Wsequence-point]`. Sonuç gcc sürümüne, hatta optimizasyon ayarına göre değişebilir; o yüzden çıktıyı göstermiyoruz. **Kural:** Bir değişkeni bir ifadede en fazla bir kez değiştir; `++`/`--` kullanan satırı sade tut.

> **Sınavda Çıkar**
>
> `int a = 5; int b = a++ + ++a;` gibi sorular "çıktısı nedir?" diye gelebilir. Gerçek C'de cevap **tanımsızdır**. Soruyu çözmek zorundaysan hocanın kuralını (genellikle soldan sağa) uygula, ama bunun C standardının garantisi olmadığını bil. Tek başına `x++;` kullanan sorular güvenlidir.

2.11 Operatör önceliği

Bir ifadede birden çok operatör varsa hangisinin önce yapılacağını **öncelik** (precedence) belirler: çarpma, bölme ve kalan, toplama ve çıkarmadan önce yapılır. Aynı seviyedeki operatörler çoğunlukla **soldan sağa** işlenir. En yüksekten düşüğe:

Seviye	Operatörler	Yön
1	`( )` parantez, son ek `++` `--`	soldan sağa
2	Tekli: `+` `-` (işaret), ön ek `++` `--`, `(tip)` cast, `sizeof`	**sağdan sola**
3	`*` `/` `%`	soldan sağa
4	`+` `-`	soldan sağa
5	`=` `+=` `-=` `*=` `/=` `%=`	**sağdan sola**

Karşılaştırma ve mantık operatörlerini (`<`, `==`, `&&` ...) Bölüm 4'te göreceksin; bunlar 4. ile 5. seviye arasına girecek.

oncelik.c

```c
#include <stdio.h>

int main(void) {
    printf("2 + 3 * 4     = %d\n", 2 + 3 * 4);
    printf("(2 + 3) * 4   = %d\n", (2 + 3) * 4);
    printf("20 - 6 - 4    = %d\n", 20 - 6 - 4);
    printf("100 / 10 * 5  = %d\n", 100 / 10 * 5);
    printf("7 + 8 %% 3 * 2 = %d\n", 7 + 8 % 3 * 2);
    return 0;
}
```

Çıktı

```
2 + 3 * 4     = 14
(2 + 3) * 4   = 20
20 - 6 - 4    = 10
100 / 10 * 5  = 50
7 + 8 % 3 * 2 = 11
```

`100 / 10 * 5` ifadesi `100 / 50 = 2` **değil**: `/` ile `*` aynı seviyedir, soldan sağa gidilir, `(100 / 10) * 5 = 50`. `7 + 8 % 3 * 2` ifadesinde önce `%` (`2`), sonra `*` (`4`), en son `+` (`11`) yapılır. Atama sağdan sola işlediği için `x = y = z = 0` zincirinde önce `z` sıfırlanır.

Emin değilsen **parantez kullan**; kodu okuyana niyetini de gösterir. Parantezi unutmak, Bölüm 1'de söz verdiğimiz sinsi mantık hatasının ta kendisidir:

ortalama.c

```c
#include <stdio.h>

int main(void) {
    int vize = 65;
    int finalNotu = 80;

    double hatali = vize + finalNotu / 2;
    double yarim = (vize + finalNotu) / 2;
    double dogru = (vize + finalNotu) / 2.0;

    printf("Hatalı : %.2f\n", hatali);
    printf("Yarım  : %.2f\n", yarim);
    printf("Doğru  : %.2f\n", dogru);
    return 0;
}
```

```
Çıktı
Hatalı : 105.00
Yarım  : 72.00
Doğru  : 72.50
```

`vize + finalNotu / 2` işleminde `/` önce yapıldığı için yalnız `finalNotu` ikiye bölünür (`65 + 40 = 105`). Parantez eklenince toplam doğru bölünür, ama `145 / 2` iki tamsayı olduğu için `72` çıkar. Bölen `2.0` olunca bölme ondalıklı yapılır: `72.50`. Program hiçbir hata mesajı vermedi, ekranda hep makul bir sayı vardı. Mantık hatası böyle gizlenir.

> **İpucu: Karmaşık ifadeyi adım adım aç**
>
> Sınavda uzun bir ifade görürsen kâğıtta parantezlerle açarak çöz. `2 + 3 * 4 % 5` için önce `3 * 4 → 12`, sonra `12 % 5 → 2`, en son `2 + 2 → 4`. Her adımı bir alt satıra yaz; bir adımı zihinden atlamaya çalışırsan hata yaparsın.

2.12 Taşma ve ondalıklı sayıların hassasiyeti

Tamsayı taşması

Her tamsayı tipinin bir aralığı vardır. Hesabın sonucu bu aralığın dışına çıkarsa **taşma** (overflow) olur. C bu durumda iki türlü davranır.

İşaretli taşma tanımsız davranıştır. En büyük `int`'e 1 eklemek bunun örneğidir:

```c
int enBuyuk = INT_MAX;       // 2147483647
enBuyuk = enBuyuk + 1;       // tanımsız davranış
```

Birçok bilgisayarda sonuç `-2147483648` çıkar ("başa sarma"), ama derleyici bunu garanti etmez; optimizasyonla başka bir şey de yapabilir. Çıktısını göstermiyoruz. Büyük sayıların çarpımında da aynısı olur ve çoğu kişi bunu fark etmez:

```c
int a = 100000;
int b = a * a;               // 10 milyar, int'e sığmaz: tanımsız
long long c = a * a;         // bu da hatalı: çarpma yine int ile yapılır
long long d = (long long)a * a;   // doğru: çarpmadan ÖNCE büyük tipe çevir
```

İşaretsiz taşma tanımlıdır ve kuralı bellidir: Sonucun, tipin değer sayısına göre **modu** alınır; sayaç başa **sarar**.

unsignedSarma.c

```c
#include <stdio.h>
#include <limits.h>

int main(void) {
    unsigned int u = UINT_MAX;
    u = u + 1;
    printf("%u\n", u);
    u = u - 1;
    printf("%u\n", u);

    unsigned char kucuk = 250;
    kucuk = kucuk + 10;
    printf("%d\n", kucuk);

    unsigned int negatif = -1;
    printf("%u\n", negatif);
    return 0;
}
```

Çıktı

```
0
4294967295
4
4294967295
```

`UINT_MAX + 1` sıfıra sarar; sıfırdan 1 çıkarınca en büyük değere dönülür. `unsigned char` 0 … 255 tutar; `250 + 10 = 260` olunca `260 % 256 = 4` kalır. `unsigned` değişkene negatif değer atamak da aynı mantıkla büyük pozitif sayı üretir (`-1` → `4294967295`). Yani `unsigned` değişken sıfırın altına inemez: `0 - 1`, `-1` değil `4294967295` olur.

Ondalıklı sayıların hassasiyeti

Bilgisayar sayıları **ikilik tabanda** saklar. `0.5` ya da `0.25` ikilikte sonlu yazılır, ama `0.1` yazılamaz (tıpkı `1/3`'ün onluk tabanda `0.333...` olması gibi). Sayı, yakınındaki temsil edilebilir bir değere **yuvarlanır**. Hatayı `%.20f` ile (virgülden sonra 20 basamak) görebiliriz. Aynı programda `float` ile `double`'ın kaç basamağı doğru tuttuğuna da bakalım:

hassasiyet.c

```c
#include <stdio.h>

int main(void) {
    double toplam = 0.1 + 0.2;
    float kucuk = 0.1f;
    float f = 123456789.0f;
    double d = 123456789.0;

    printf("%.1f\n", toplam);
    printf("%.20f\n", toplam);
    printf("%.20f\n", kucuk);
    printf("%d\n", toplam == 0.3);
    printf("float : %.1f\n", f);
    printf("double: %.1f\n", d);
    return 0;
}
```

```
Çıktı
0.3
0.30000000000000004441
0.10000000149011611938
0
float : 123456792.0
double: 123456789.0
```

`%.1f` ile yuvarlanınca `0.3` görünür, ama gerçek değer `0.30000000000000004...`'tür. `float` olan `0.1`'in hatası daha da büyüktür. Dördüncü satırdaki `==` operatörünü Bölüm 4'te göreceksin; şimdilik "eşitse `1`, değilse `0`" bil: `0.1 + 0.2` ile `0.3` **eşit çıkmadı**. Bu yüzden ondalıklı sayılar `==` ile karşılaştırılmaz. Son iki satır hassasiyetin sınırını gösterir: `float` yaklaşık **7**, `double` yaklaşık **15-16** anlamlı basamak tutar. Dokuz basamaklı sayıyı `float` yaklaşık saklayıp `123456792` yazdı, `double` tam tuttu. Para ve not hesaplarında `double` kullanmanın nedeni budur.

2.13 Bit düzeyinde operatörler

Bilgisayar her sayıyı **bitlerle** (0 ve 1) saklar. Onluk bir sayıyı ikilik tabana çevirmek için onu 8, 4, 2, 1 gibi ikinin kuvvetlerinin toplamı olarak yaz: `12 = 8 + 4` olduğu için ikilikte `1100`, `10 = 8 + 2` olduğu için `1010` olur. Bitler **sağdan sola** 0, 1, 2, ... diye numaralanır; en sağdaki bit 0. bittir. `unsigned int` 32 bitliktir.

C'de sayıların bitlerine tek tek dokunan altı operatör vardır. Bunlar sayıyı bir bütün olarak değil, **aynı sıradaki bit çiftlerini ayrı ayrı** işler:

Operatör	Adı	Ne yapar?	`a = 1100`, `b = 1010` için
`a & b`	VE (AND)	İkisinde de 1 olan bit 1 olur	`1000` (8)
`a | b`	VEYA (OR)	En az birinde 1 olan bit 1 olur	`1110` (14)
`a ^ b`	ÖZEL VEYA (XOR)	Yalnız birinde 1 olan bit 1 olur	`0110` (6)
`~a`	DEĞİL (NOT)	Bütün bitleri ters çevirir (tekli)	32 bitte `4294967283`
`a << n`	Sola kaydırma	Bitleri `n` basamak sola iter, sağdan `0` gelir	`a << 2` → `110000` (48)
`a >> n`	Sağa kaydırma	Bitleri `n` basamak sağa iter, sağdaki `n` bit düşer	`a >> 2` → `11` (3)

İşaretsiz sayılarda `a << n`, `a`'yı 2^n ile çarpmak; `a >> n`, 2^n'e bölüp kesiri atmak demektir. Hepsini bir arada görelim:

bitOperatorleri.c

```c
#include <stdio.h>

int main(void) {
    unsigned int a = 12;                // 1100
    unsigned int b = 10;                // 1010

    printf("a & b  = %u\n", a & b);     // 1000
    printf("a | b  = %u\n", a | b);     // 1110
    printf("a ^ b  = %u\n", a ^ b);     // 0110
    printf("~a     = %u\n", ~a);        // 32 bitin hepsi ters çevrilir
    printf("a << 2 = %u\n", a << 2);    // 110000
    printf("a >> 2 = %u\n", a >> 2);    // 11
    printf("a && b = %d\n", a && b);    // mantıksal VE: yalnız 1 ya da 0

    unsigned int x = 13;                // 1101
    printf("x & 1  = %u\n", x & 1);     // son bit: 1 ise sayı tek
    printf("1u << 3 = %u\n", 1u << 3);  // yalnız 3. biti 1 olan sayı
    x = x | (1u << 1);                  // 1. biti aç:    1111
    printf("açıldı  = %u\n", x);
```

```
    x = x & ~(1u << 3);                  // 3. biti kapat: 0111
    printf("kapandı = %u\n", x);
    return 0;
}
```

Çıktı

```
a & b  = 8
a | b  = 14
a ^ b  = 6
~a     = 4294967283
a << 2 = 48
a >> 2 = 3
a && b = 1
x & 1  = 1
1u << 3 = 8
açıldı  = 15
kapandı = 7
```

İlk altı satır tablodaki sonuçlardır. `~a`, 32 bitin hepsini çevirdiği için büyük bir sayı verir (`UINT_MAX - 12`). Yedinci satır `&&` içindir (aşağıda). Son dört satır bit operatörlerinin tipik işleridir:

- **Tek/çift:** `x & 1` sayının son bitidir; `1` ise sayı tektir, `0` ise çifttir (`x % 2` ile aynı sonuç).
- **`1u << n`:** yalnız `n`. biti 1 olan sayıdır (`1u << 3` = 8). Buna **bit maskesi** denir ve bir sayının tek bir bitini değiştirmek ya da okumak için kullanılır: `x | (1u << n)` biti açar, `x & ~(1u << n)` kapatır, `x ^ (1u << n)` tersler, `x & (1u << n)` sıfırdan farklıysa bit açıktır.

Bit operatörlerini hep `unsigned` sayılarla kullan. İşaretli `int` ile iki risk vardır: `1 << 31` gibi sonucu `int`'e sığmayan kaydırmalar ve negatif sayıyı sola kaydırmak **tanımsız davranıştır**; negatif sayıyı sağa kaydırmanın sonucu ise derleyiciye bağlıdır. Kaydırma miktarı negatifse ya da tipin bit sayısına (`int` için 32) eşit veya büyükse de tanımsızdır. `unsigned char` ve `short` değerlerde `~` ve kaydırma önce `int`'e yükseltme yaptığı için sonuç şaşırtabilir; bu yüzden örneklerde `unsigned int` kullandık.

`&` ile `&&` farkı. `&` bitlere tek tek bakıp bir **sayı** üretir. `&&` (mantıksal VE, Bölüm 4) ise iki ifadenin de sıfırdan farklı olup olmadığına bakar ve yalnız `1` ya da `0` verir: `12 & 10` sonucu `8`, `12 && 10` sonucu `1`'dir. `|` ile `||` arasındaki fark da aynıdır.

Öncelik. `~` tekli operatörler seviyesindedir. `<<` ve `>>` toplamanın hemen altına, `&`, `^`, `|` ise karşılaştırma operatörlerinin (`==`, Bölüm 4) bile altına düşer. Bu yüzden `x & 1 == 0` beklediğin gibi çalışmaz; `x & (1 == 0)` diye okunur. Doğrusu `(x & 1) == 0`'dır. Şüphede parantez kullan.

> **Sınavda Çıkar**
>
> C'de **üs alma operatörü yoktur**. `2 ^ 3` sonucu `8` değil `1`'dir, çünkü `^` XOR'dur (`010` ile `011` yalnız son bitte farklıdır). Üs için `pow`, ikinin kuvveti için `1 << n` kullanılır. Diğer klasikler: `x & 1` tek/çift sorar, `12 & 10` ile `12 && 10` farklı sonuç verir (`8` ve `1`), `5 << 1` sonucu `10`, `20 >> 2` sonucu `5`'tir.

2.14 math.h kütüphanesi

Kök alma, üs alma gibi işlemler operatör değil **fonksiyondur**; `math.h` başlık dosyasında tanıtılmıştır ve hepsi `double` alıp `double` döndürür: `sqrt` (karekök), `pow(x, y)` (x üssü y), `fabs` (mutlak değer), `ceil` (yukarı yuvarla), `floor` (aşağı yuvarla), `round` (en yakına yuvarla; tam yarımda sıfırdan uzağa), `fmod` (ondalıklı sayılarda kalan). Tamsayının mutlak değeri için `stdlib.h`'deki `abs` kullanılır; ona `double` verirsen sonuç kesilir ve gcc `-Wabsolute-value` uyarısı verir.

matematik.c

```c
#include <stdio.h>
#include <math.h>
#include <stdlib.h>

int main(void) {
    printf("sqrt(144)    = %.1f\n", sqrt(144.0));
    printf("pow(2, 10)   = %.0f\n", pow(2, 10));
    printf("fabs(-3.7)   = %.1f\n", fabs(-3.7));
    printf("ceil(3.2)    = %.1f\n", ceil(3.2));
    printf("floor(-3.2)  = %.1f\n", floor(-3.2));
    printf("round(3.5)   = %.1f\n", round(3.5));
    printf("round(-3.5)  = %.1f\n", round(-3.5));
    printf("fmod(7.5, 2) = %.1f\n", fmod(7.5, 2.0));
    printf("abs(-9)      = %d\n", abs(-9));
    return 0;
}
```

Çıktı

```
sqrt(144)    = 12.0
pow(2, 10)   = 1024
fabs(-3.7)   = 3.7
ceil(3.2)    = 4.0
floor(-3.2)  = -4.0
round(3.5)   = 4.0
round(-3.5)  = -4.0
fmod(7.5, 2) = 1.5
abs(-9)      = 9
```

`floor(-3.2)` sonucu `-3.0` değil `-4.0`'tır: "aşağı" her zaman daha küçük sayı demektir; kesmeyle (`(int)-3.2 == -3`) karıştırma. `pow` bir `double` döndürür; `int`'e atarken kesilir ve bazı eski derleyicilerde `pow(10, 2)` ifadesi `99.999...` çıkıp `99`'a düşebilir. Tamsayı istiyorsan `(int)round(pow(10, 2))` yaz.

Sık Yapılan Hata: -lm: Linux'ta math.h bağlama hatası

`math.h` fonksiyonlarının kodu standart kütüphanede değil, ayrı bir **matematik kütüphanesinde** (`libm`) bulunur. Linux'ta bunu bağlayıcıya söylemezsen Bölüm 1'deki gibi bir bağlama hatası alırsın. Örneğin klavyeden okunan bir sayının karekökünü alan `karekok.c` programını `-lm` olmadan derlersen Linux gcc'si şunu yazar (yol bilgileri kısaltıldı):

```
/usr/bin/ld: ...: in function `main':
karekok.c:(.text+0x2d): undefined reference to `sqrt'
collect2: error: ld returned 1 exit status
```

Dikkat: Yukarıdaki `matematik.c` bu hatayı vermezdi, çünkü `sqrt(144.0)` gibi sabit sayılı çağrıları gcc derleme sırasında kendisi hesaplar. Değeri çalışırken belli olan çağrılar (`sqrt(sayi)`) kütüphane ister.

Çözüm, derleme komutunun **sonuna** (kaynak dosyadan *sonra*) `-lm` (library m) eklemektir: `gcc -std=c17 -Wall -Wextra matematik.c -o matematik -lm`. macOS'ta ve Windows'ta (MinGW) matematik fonksiyonları zaten standart kütüphanededir, `-lm` yazmasan da çalışır; ama her yerde çalışsın diye yazmak zarar vermez. Kitaptaki tüm programlar `-lm` ile derlenmiştir.

Bölüm Özeti – Bilmen Gerekenler

- **Değişken**, bellekte adı olan kutudur: ad, tip ve değeri vardır. `tip ad = deger;` ilk değer verir, `ad = deger;` sonradan atar. Tanımlarken ilk değer ver; ilk değersiz değişkeni okumak çöp değer verir (tanımsız davranış).
- **Ad kuralları:** harf, rakam, `_`; rakamla başlamaz; boşluk/özel karakter yok; anahtar sözcük olamaz; büyük/küçük harf farklıdır. Gelenek `camelCase`, sabitler `BUYUK_HARF`.
- **Tipler:** `char` 1, `short` 2, `int` 4, `long` 4 (Windows) ya da 8, `long long` 8, `float` 4, `double` 8 bayt. `sizeof` bayt sayısını verir (`%zu`). `INT_MAX = 2147483647`, `INT_MIN = -2147483648`. Ondalıklı sayı için `double`.
- **Sabit:** `const double PI = 3.14;` (tipli, `;` var) ve `#define PI 3.14` (metin değiştirme, `;` **yok**). `'A'` `int`'tir, `5.0` `double`'dır, `010` sekizliktir.
- **Tamsayı bölmesi:** iki tamsayının bölmesi tamsayıdır, kesir atılır, sıfıra doğru kesilir: `7 / 2 == 3`, `-7 / 2 == -3`. `%`: kalanın işareti bölünenin işaretidir: `-7 % 2 == -1`. `%` double ile kullanılamaz.
- **Dönüşüm:** karışık ifadede küçük tip büyüğe yükselir. `double`'dan `int`'e atama kesir kısmını atar. Cast: `(double)a / b` (doğru) ile `(double)(a / b)` (bölme önce, kesilmiş) farklıdır.
- `char` küçük bir tamsayıdır: `'A' + 1` bir `int`'tir (`66`), `%c` ile `B` yazılır; `'7' - '0' == 7`.
- **Tamsayı yükseltmesi:** `char` ve `short` aritmetikte `int`'e yükselir (`char c = 100; c + c` 200 verir, `sizeof(c + c)` 4'tür). `int` ile `unsigned` karşılaşırsa işaretli olan işaretsize çevrilir: `-1 < 1u` yanlıştır (`-Wsign-compare`, `-Wextra` ile uyarır).
- **Bileşik atama:** `x += 5` ≡ `x = x + 5`; `y *= 2 + 3` ≡ `y = y * (2 + 3)`. `x++` ifade olarak eski değeri, `++x` yeni değeri verir. Bir değişkeni bir ifadede iki kez değiştirmek tanımsızdır.
- **Öncelik:** `( )` > tekli (`++`, `-`, cast) > `* / %` > `+ -` > atama. Aynı seviyede soldan sağa (atama hariç). Şüphede parantez kullan.
- **Taşma:** işaretsiz taşma tanımlıdır (mod alır, sarar); işaretli taşma tanımsızdır. `0.1 + 0.2` tam `0.3` değildir; `float` ~7, `double` ~15 basamak tutar.
- **Bit operatörleri:** `&` VE, `|` VEYA, `^` XOR, `~` DEĞİL, `<<` / `>>` kaydırma; `unsigned` sayılarla kullan. `x & 1` tek/çift sorar, `1u << n` bit maskesidir. `&` sayı üretir, `&&` yalnız `1`/`0`. `^` üs alma **değildir**.
- `math.h`: `sqrt`, `pow`, `fabs`, `ceil`, `floor`, `round`; tamsayı için `abs` (`stdlib.h`). Linux'ta `-lm`.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki kod parçasından sonra ekrana ne yazılır?

```c
int a = 17, b = 5;
double c = a / b;
printf("%.1f", c);
```

(A) 3.4
(B) 3.0
(C) 3.5
(D) 4.0
(E) Program derlenmez

2 `printf("%d", -17 % 5);` satırı ekrana ne yazdırır?

(A) 3 (B) 2 (C) -3 (D) 0 (E) -2

3 Aşağıdakilerden hangisi geçerli bir C değişken adıdır?

(A) `_toplam2` (B) `2toplam` (C) `toplam-not` (D) `double` (E) `ogrenci sayisi`

4 Bir programın başında `#define SINIR 50;` satırı vardır ve `main` içinde şu satır yazılmıştır. Program derlenmeye çalışılırsa ne olur?

```
int x = SINIR * 2;
```

(A) Ekrana `100` yazılır.
(B) Ekrana `50` yazılır.
(C) Ekrana `52` yazılır.
(D) Program derlenmez, çünkü önişlemci `int x = 50; * 2;;` metnini üretir.
(E) Program derlenir, ama `x` çöp değer alır.

5 Aşağıdaki kod parçasından sonra ekrana ne yazılır?

```
unsigned char c = 250;
c = c + 10;
printf("%d", c);
```

(A) 260
(B) 255
(C) 4
(D) 10
(E) Davranış tanımsızdır, çıktı tahmin edilemez.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
#include <stdio.h>

int main(void) {
    int a = 4, b = 3;
    int c = a++ + b;
    int d = ++a * 2;
    b += a--;
    printf("%d %d %d %d\n", a, b, c, d);
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int x = 17, y = 5;
    double d1 = x / y;
    double d2 = (double)x / y;
    double d3 = (double)(x / y);
    int m = -x % y;
    printf("%.1f %.1f %.1f\n", d1, d2, d3);
    printf("%d %d\n", m, x / -y);
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    char c = 'a';
    int fark = 'd' - c;
    c = c + 2;
    printf("%c %d\n", c, fark);
    printf("%c %d\n", 'A' + 1, 'A' + 1);
    printf("%c\n", c - 'a' + 'A');
    return 0;
}
```

C) Kod Yazma

9 Bir dersin **vize** notu `70`, **final** notu `85` olsun (ikisi de `int`). Ders ortalaması, vizenin `%40`'ı ile finalin `%60`'ı toplanarak hesaplanır. Ağırlıkları `const double` olarak tanımla, ortalamayı `double` bir değişkende tut ve virgülden sonra iki basamakla yazdır. `agirlikliOrtalama` programını yaz.

```
Vize: 70
Final: 85
Ortalama: 79.00
```

10 Verilen toplam saniyeyi **saat, dakika, saniye** olarak gösteren `sureCevir` programını yaz. Toplam saniye `10000` olsun; yalnız `/` ve `%` operatörlerini kullan (1 saat = 3600 saniye, 1 dakika = 60 saniye).

```
10000 saniye = 2 saat 46 dakika 40 saniye
```

Çözümler

Cevap anahtarı

1 B	2 E	3 A	4 D	5 C

1 Cevap: B

`a` ve `b` iki tamsayıdır, bu yüzden `17 / 5` **tamsayı bölmesidir** ve sonuç `3` olur. Bu `3`, atamada `double`'a çevrilip `3.0` olur. `double c` olması bölmenin ondalıklı yapılmasını sağlamaz. A ve C (`3.4`, `3.5`) ondalıklı bölmeyi varsayar, D yuvarlama yapar, E yanlış: kod sorunsuz derlenir.

2 Cevap: E

Bölme sıfıra doğru kesilir: `-17 / 5 = -3`. Kalan `a - (a / b) * b` ile bulunur: `-17 - (-3 * 5) = -2`. Kalanın işareti bölünenin (`-17`) işaretidir. A (`3`) matematikteki mod değeridir (Python'un verdiği), B (`2`) işareti yok sayar, C (`-3`) bölümün kendisidir, D yanlış çünkü `17`, `5`'in katı değil.

3 Cevap: A

Ad harf, rakam ve alt çizgiden oluşur, rakamla başlamaz; `_toplam2` buna uyar. B rakamla başlıyor. C'deki `-` çıkarma operatörüdür (`toplam - not` diye okunur). D `double`, bir anahtar sözcük. E'de boşluk var.

4 Cevap: D

`#define` düz metin değiştirmedir; `;` da değiştirilen metne dahildir. `SINIR * 2`, `50; * 2` olur ve satır `int x = 50; * 2;;` hâline gelir. `* 2;` ifadesi `2` sayısına işaretçi operatörünü uygulamaya çalışan anlamsız bir ifadedir; gcc `invalid type argument of unary '*'` hatası verir. A, B ve C çalışan bir program varsayar; E'de program hiç derlenmediği için `x` diye bir şey de oluşmaz.

5 Cevap: C

`c + 10` ifadesi `int` olarak hesaplanır (`260`), ama sonuç `unsigned char`'a atanır. İşaretsiz tipte taşma **tanımlıdır** ve mod alınır: `260 % 256 = 4`. A (`260`) tipin aralığını yok sayar, B (`255`) sonucun "tavana yapışacağını" varsayar (öyle olmaz, sarar), D (`10`) eklenen sayıdır, E yanlış: tanımsız olan **işaretli** taşmadır.

6

Değişkenleri adım adım izle:

Satır	İşlem	a	b	c	d
başta		4	3		
`c = a++ + b`	`4 + 3 = 7` atanır, sonra `a` artar	5	3	7	
`d = ++a * 2`	önce `a = 6`, sonra `6 * 2`	6	3	7	12
`b += a--`	`a--` eski değer `6`'yı verir: `b = 3 + 6`; sonra `a` azalır	5	9	7	12

Son durumda `a = 5`, `b = 9`, `c = 7`, `d = 12`; `printf` bu sırayla yazar:

```
Çıktı
5 9 7 12
```

7

`x = 17`, `y = 5`.

1. `d1 = x / y`: iki `int` bölünür (`3`), `3.0`'a çevrilir: `3.0`.
2. `d2 = (double)x / y`: cast önce `x`'e uygulanır, bölme ondalıklı yapılır: `3.4`.
3. `d3 = (double)(x / y)`: bölme önce yapılır (`3`), sonra `3.0`: `3.0`.
4. `m = -x % y`: tekli eksi `%`'den önce uygulanır, `(-17) % 5 = -2`.
5. `x / -y = 17 / -5`: sıfıra doğru kesilir: `-3`.

```
Çıktı
3.0 3.4 3.0
-2 -3
```

8

`'a'` kodu `97`, `'d'` kodu `100`.

1. `fark = 'd' - c = 100 - 97 = 3`. `c = c + 2` → `99`, yani `'c'`. İlk `printf`: `c 3`.
2. `'A' + 1` bir `int`'tir (`66`). `%c` onu `B` karakteri olarak, `%d` sayı olarak yazar: `B 66`.
3. `c - 'a' + 'A' = 99 - 97 + 65 = 67` → `%c` ile `C`.

```
Çıktı
c 3
B 66
C
```

9

Ağırlıklar `const double`, notlar `int`. `0.40 * vize` işleminde `int`, `double`'a yükseltilir. Ağırlığı `40 / 100` diye yazsaydın tamsayı bölmesi `0` verirdi.

agirlikliOrtalama.c

```c
#include <stdio.h>

int main(void) {
    const double VIZE_AGIRLIK = 0.40;
    const double FINAL_AGIRLIK = 0.60;
    int vize = 70;
    int finalNotu = 85;

    double ortalama = VIZE_AGIRLIK * vize + FINAL_AGIRLIK * finalNotu;

    printf("Vize: %d\n", vize);
    printf("Final: %d\n", finalNotu);
    printf("Ortalama: %.2f\n", ortalama);
    return 0;
}
```

Çıktı
```
Vize: 70
Final: 85
Ortalama: 79.00
```

10

Saat için `10000 / 3600 = 2` (tamsayı bölmesi tam saati verir). Geriye `10000 % 3600 = 2800` saniye kalır. Bu kalandan dakika `2800 / 60 = 46`, saniye `2800 % 60 = 40` bulunur.

sureCevir.c
```c
#include <stdio.h>

int main(void) {
    int toplamSaniye = 10000;

    int saat = toplamSaniye / 3600;
    int kalan = toplamSaniye % 3600;
    int dakika = kalan / 60;
    int saniye = kalan % 60;

    printf("%d saniye = %d saat %d dakika %d saniye\n",
           toplamSaniye, saat, dakika, saniye);
    return 0;
}
```

Çıktı
```
10000 saniye = 2 saat 46 dakika 40 saniye
```

BÖLÜM 3

Klavyeden Girdi ve Biçimli Çıktı

- **Bu Bölümde Öğreneceklerin**
  - `printf` biçim belirteçlerini (`%d %i %u %ld %lld %f %e %g %c %s %x %X %o %zu %%`) ve alan genişliği, hizalama, hassasiyet ayarını (`%5d`, `%-5d`, `%05d`, `%.2f`, `%8.3f`, `%-10s`)
  - Yanlış biçim belirtecinin neden hata olduğunu ve gcc'nin bunu nasıl uyardığını
  - `scanf` ile klavyeden `int`, `float`, `double` ve `char` okumayı; `&` işaretinin neden gerektiğini
  - Birden çok değeri ve `15/03/2005` gibi kalıplı girdiyi okumayı; `scanf`'in dönüş değerini ve sayı yerine harf girilince ne olduğunu
  - Girdi tamponunu (buffer), `%c` tuzağını ve `" %c"` çözümünü; `getchar` ile `putchar`'ı; `fflush(stdin)`'in neden yanlış olduğunu ve tamponu temizlemenin doğru kalıbını
  - Vücut kitle indeksi, saniye çevirme ve fiş yazdırma uygulamalarını
  - Türkçe karakterlerin Windows konsolunda neden bozulduğunu ve çözümünü (`chcp 65001`, UTF-8)

Şimdiye kadar yazdığın programlar tek yönlüydü: program ekrana bir şey yazdı, sen izledin. Bu bölümde program **sana soru soracak ve cevabını bekleyecek**. Bunun için `scanf` ile klavyeden okumayı öğreneceksin. Bölüm 2'de `printf`'i çoğunlukla `%d`, `%f`, `%.2f` ve `%c` ile kullanmıştın; şimdi bütün biçim belirteçlerini ve çıktıyı sütunlara hizalamayı göreceksin. Bölüm 1'de sözünü verdiğimiz gibi, Windows'ta Türkçe karakterlerin neden bozulduğunu da bu bölümün sonunda anlatacağız.

3.1 printf biçim belirteçleri

`printf`'in ilk argümanı **biçim metnidir** (format string). Bu metindeki `%` ile başlayan her parça bir **biçim belirteci**dir (format specifier): `printf` onu, sonraki argümanlardan sırayla birinin değeriyle değiştirir. Belirtecin son harfi, değerin **hangi türde yorumlanacağını** söyler:

```
printf("%d + %d = %d\n", a, b, a + b);
```

Burada birinci `%d` yerine `a`, ikinci `%d` yerine `b`, üçüncü `%d` yerine `a + b` yazılır. Belirteç sayısı ile argüman sayısı ve **türleri** uyuşmak zorundadır. Kitabın geri kalanında göreceğin belirteçler:

Belirteç	Argüman türü	Ne yazdırır?
`%d`, `%i`	`int`	İşaretli onluk tamsayı. `printf`'te ikisi aynıdır
`%u`	`unsigned int`	İşaretsiz onluk tamsayı
`%ld`	`long`	`long` tamsayı
`%lld`	`long long`	`long long` tamsayı
`%f`	`float`, `double`	Onluk kesirli sayı, noktadan sonra 6 hane
`%e`	`float`, `double`	Bilimsel gösterim: `3.141590e+00` (3.14159 çarpı 10 üzeri 0)
`%g`	`float`, `double`	`%f` ya da `%e`'den kısa olanı seçer, gereksiz sıfırları atar
`%c`	`char`	Tek karakter
`%s`	string	Metin (ayrıntısı Bölüm 8'de)
`%x`, `%X`	`unsigned int`	Onaltılık (hexadecimal) gösterim; `%x` küçük, `%X` büyük harfle
`%o`	`unsigned int`	Sekizlik (octal) gösterim
`%zu`	`size_t`	`sizeof` sonucu (Bölüm 2)
`%%`	(argüman yok)	`%` işaretinin kendisi

Hepsini tek programda görelim. Köşeli parantezler yalnız okumayı kolaylaştırmak için konuldu:

belirtecler.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 42;
    unsigned int buyuk = 3000000000u;
    long uzun = 123456789L;
    long long cokBuyuk = 9000000000LL;
    double pi = 3.14159;
    char harf = 'A';

    printf("[%d] [%i]\n", sayi, sayi);
    printf("[%u] [%ld] [%lld]\n", buyuk, uzun, cokBuyuk);
    printf("[%f] [%e] [%g]\n", pi, pi, pi);
    printf("[%g] [%g]\n", 1234567.0, 0.00001);
    printf("[%c] [%d]\n", harf, harf);
    printf("[%s]\n", "metin");
    printf("[%x] [%X] [%o]\n", 255, 255, 8);
    printf("[%zu]\n", sizeof(int));
    printf("Başarı: %d%%\n", 85);
    return 0;
}
```

Çıktı

```
[42] [42]
[3000000000] [123456789] [9000000000]
[3.141590] [3.141590e+00] [3.14159]
[1.23457e+06] [1e-05]
[A] [65]
[metin]
[ff] [FF] [10]
[4]
Başarı: 85%
```

Çıktıdan çıkarılacak dersler:

- `%f` her zaman noktadan sonra **6 hane** yazar (`3.141590`). Hane sayısını hassasiyetle değiştireceksin (sonraki başlık).
- `%g` gereksiz sıfırları atar (`3.14159`), çok büyük ya da küçük sayıda bilimsel gösterime geçer (`1.23457e+06`, `1e-05`).
- `harf` değişkeni `%c` ile `A` karakteri, `%d` ile `65` sayısı olarak yazıldı: `char` küçük bir tamsayıdır (ASCII tablosu Ek D'de).
- `255` onaltılık tabanda `ff`'dir ($15 \times 16 + 15$), `8` sekizlik tabanda `10`'dur. `%#x` kullanırsan önüne `0x` eklenir.
- `printf("%d%%")`: önce sayı, sonra `%%` ile `%` işareti yazıldı.

`%i`, `printf`'te `%d` ile aynıdır; `scanf`'te ise farklıdır (`0x` gibi ön ekleri yorumlar). Sınavda ve bu kitapta `%d` kullanılır.

`printf`'e bir `float` verdiğinde C onu sessizce `double`'a çevirir; bu yüzden `float` ve `double` için **aynı** belirteç, `%f`, kullanılır (`%lf` de C99'dan beri zararsızdır). `scanf`'te böyle bir çeviri olmaz, orada ikisi farklıdır.

> **Sınavda Çıkar**
>
> "`double` bir değişkeni yazdırmak ve okumak için hangi belirteçler kullanılır?" sorusu kesin çıkar. Cevap: `printf`'te `%f`, `scanf`'te `%lf`. `float` için ise ikisinde de `%f`. Bu tabloyu ezberle: `int` → `%d`, `float` → `%f`, `double` → `%f` (yazdırma) / `%lf` (okuma), `char` → `%c`, `long long` → `%lld`.

3.2 Alan genişliği, hizalama ve hassasiyet

Belirtecin `%` ile harfi arasına bazı sayılar ve işaretler koyarak çıktının görünümünü ayarlayabilirsin. Genel biçim şöyledir:

```
%[bayrak][genişlik][.hassasiyet]harf
```

Parça	Anlamı	Örnek
genişlik	Çıktı **en az** bu kadar karakter yer kaplar. Kısaysa boşlukla doldurulur ve varsayılan olarak **sağa yaslanır**	`%5d`
`-` bayrağı	**Sola yasla**, boşlukları sağa koy	`%-5d`
`0` bayrağı	Boşluk yerine **sıfırla** doldur (sayılarda)	`%05d`
`+` bayrağı	Pozitif sayıya da `+` işareti koy	`%+d`
`.hassasiyet`	`%f` ile noktadan sonra kaç hane yazılacağını belirler (`%8.3f`'de genişlik **noktayı da sayar**); `%s` ile en çok kaç karakter yazılacağını	`%.2f`, `%.3s`

Her birini deneyelim:

genislik.c

```c
#include <stdio.h>

int main(void) {
    printf("[%5d]\n", 42);
    printf("[%-5d]\n", 42);
    printf("[%05d]\n", 42);
    printf("[%+d] [%+d]\n", 42, -42);
    printf("[%2d]\n", 123456);
    printf("[%.2f]\n", 3.14159);
    printf("[%8.3f]\n", 3.14159);
    printf("[%-8.1f]\n", 3.14159);
    printf("[%08.2f]\n", 3.14159);
    printf("[%10s]\n", "Ali");
    printf("[%-10s]\n", "Ali");
    printf("[%.3s]\n", "Elmalar");
    return 0;
}
```

Çıktı

```
[   42]
[42   ]
[00042]
[+42] [-42]
[123456]
[3.14]
[   3.142]
[3.1     ]
[00003.14]
[       Ali]
[Ali       ]
[Elm]
```

Köşeli parantezler boşlukları görmeni sağlar; hizalamayı denerken sen de değeri `[%5d]` gibi parantez içine al. Satır satır oku:

1. `%5d`: `42` iki karakter, genişlik 5, önüne **üç boşluk** konur. `%-5d` sayıyı sola yaslar, **üç boşluk sağda** kalır. `%05d` boşluk yerine sıfır koyar. `%+d` artı işaretini de yazar.
2. `%2d` ile `123456`: Genişlik **en az** demektir; sayı sığmıyorsa **kesilmez**.
3. `%8.3f` değerinde `3.142` beş karakterdir (nokta dahil), genişlik 8 olduğu için önüne üç boşluk gelir. `%-8.1f` sola yaslar (`3.1` ve beş boşluk), `%08.2f` sıfırla doldurur (`00003.14`).
4. `%10s` ve `%-10s` string'i sağa ya da sola yaslar, `%.3s` yalnız ilk üç karakteri yazar.

> **Sınavda Çıkar**
>
> Genişlik sorularında üç kural: (1) Genişlik **en az** demektir, sayı sığmazsa kesilmez. (2) `%8.3f` gibi belirteçlerde nokta da genişliğe dahildir. (3) Bayrak yoksa sağa yaslanır, `-` varsa sola yaslanır ve boşluk sağda kalır. `%5d|` ile `%-5d|` çıktılarını yan yana yazarak farkı bir kez görürsen unutmazsın.

Yuvarlama

`%.2f` sayıyı iki haneye **yuvarlar**, kesmez. Tamsayıya dönüştürmek (`(int)` ile) ise ondalık kısmı **keser** (Bölüm 2). İkisinin farkına bak:

yuvarlama.c

```c
#include <stdio.h>

int main(void) {
    printf("%.2f\n", 3.14159);
    printf("%.2f\n", 3.999);
    printf("%.0f\n", 2.7);
    printf("%d\n", (int)2.7);
    printf("%.2f\n", 2.675);
    printf("%.2f\n", 1.005);
    return 0;
}
```

Çıktı

```
3.14
4.00
3
2
2.67
1.00
```

İlk dört satır beklediğin gibi: `3.999` yuvarlanınca `4.00`, `2.7` yuvarlanınca `3`, kesilince `2` olur. Son iki satır şaşırtıcı: Elle yuvarlasan `2.675` için `2.68`, `1.005` için `1.01` bulurdun, ama ekranda `2.67` ve `1.00` yazıyor. Sebep, ondalık sayıların ikilik tabanda saklanmasıdır: `2.675` tam saklanamaz, bellekteki değer `2.67499999...` biçimindedir ve `printf` bu gerçek değeri yuvarlar. Bu bir hata değil, ondalık kesirlerin doğal sonucudur.

Tam ortada kalan ve ikilik tabanda **tam** saklanabilen sayılarda (ör. `%.1f` ile `2.25`) sonuç kütüphaneye göre değişebilir: çoğu sistem `2.2`, bazı kütüphaneler `2.3` yazar. Sınavda böyle sınır durumu beklemezsin; kural "`%.nf` yuvarlar, kesmez"dir.

Hizalı tablo

Genişlik ve hassasiyeti birleştirince düzgün sütunlu tablolar yazdırabilirsin. Bir not tablosu yapalım: İsimler sola, sayılar sağa yaslı, ortalama iki ondalıklı. Başlıktaki `%5s` gibi belirteçler de başlıkları sayıların üstüne hizalar:

notTablosu.c

```c
#include <stdio.h>

int main(void) {
    printf("%-10s %5s %5s %7s\n", "Ad", "Vize", "Final", "Ort.");
    printf("---------- ----- ----- -------\n");
    printf("%-10s %5d %5d %7.2f\n", "Ali", 70, 85, 0.4 * 70 + 0.6 * 85);
    printf("%-10s %5d %5d %7.2f\n", "Zeynep", 95, 90, 0.4 * 95 + 0.6 * 90);
    printf("%-10s %5d %5d %7.2f\n", "Mehmet", 55, 62, 0.4 * 55 + 0.6 * 62);
    return 0;
}
```

Çıktı

```
Ad          Vize Final    Ort.
---------- ----- ----- -------
Ali           70    85   79.00
Zeynep        95    90   92.00
Mehmet        55    62   59.20
```

Sütun genişliklerini (10, 5, 5, 7) başlık satırı, ayırıcı çizgi ve veri satırlarında **aynı** tuttuğun sürece tablo hizalı çıkar. `0.4 * 70 + 0.6 * 85` gibi bir ifadeyi doğrudan argüman olarak verebilirsin; `printf` önce ifadeyi hesaplar, sonucu `%7.2f` ile yazar.

3.3 Yanlış biçim belirteci

`printf` argümanlarının türünü **bilmez**. Biçim metnindeki harfe güvenir: `%d` görürse "bu bir `int`" der, `%f` görürse "bu bir `double`" der. Sen farklı türde bir değer verirsen `printf` yine de belleği o türmüş gibi okur ve **anlamsız bir sayı** yazar. Bu bir **tanımsız davranıştır** (undefined behavior): Çıktı derleyiciye, bilgisayara ve hatta çalıştırmaya göre değişebilir. Bu yüzden bu tür hataların çıktısını göstermiyoruz.

Neyse ki gcc bu hataları derleme sırasında yakalar. `-Wall` açıkken devreye giren `-Wformat` uyarısı, biçim metnini argümanlarla karşılaştırır:

yanlisBelirtec.c

```c
#include <stdio.h>

int main(void) {
    int adet = 5;
    double fiyat = 2.5;
    printf("%f\n", adet);
    printf("%d\n", fiyat);
    printf("%d %d\n", adet);
    printf("%d\n", adet, fiyat);
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
yanlisBelirtec.c: In function 'main':
yanlisBelirtec.c:6:14: warning: format '%f' expects argument of type 'double', but argument 2 has
type 'int' [-Wformat=]
    6 |     printf("%f\n", adet);
      |             ~^     ~~~~
      |              |     |
      |              |     int
      |              double
      |             %d
yanlisBelirtec.c:7:14: warning: format '%d' expects argument of type 'int', but argument 2 has type
'double' [-Wformat=]
    7 |     printf("%d\n", fiyat);
      |             ~^     ~~~~~
      |              |     |
      |              int   double
      |             %f
yanlisBelirtec.c:8:17: warning: format '%d' expects a matching 'int' argument [-Wformat=]
    8 |     printf("%d %d\n", adet);
      |                ~^
      |                 |
      |                 int
yanlisBelirtec.c:9:12: warning: too many arguments for format [-Wformat-extra-args]
    9 |     printf("%d\n", adet, fiyat);
      |            ^~~~~
```

Program yine derlenir, çünkü bunlar hata değil **uyarıdır**; ama dört satırın dördü de yanlıştır. Uyarıları birer birer oku:

- `format '%f' expects argument of type 'double', but argument 2 has type 'int'`: `int` değeri `%f` ile yazdırmaya çalıştın. gcc doğru belirteci de önerir.

- `format '%d' expects argument of type 'int', but argument 2 has type 'double'`: Tersi; `double` değeri `%d` ile yazdırmaya çalıştın.
- `format '%d' expects a matching 'int' argument`: İki belirteç var ama tek değer verdin. İkinci `%d` bellekte rastgele bir değer okur.
- `too many arguments for format`: Belirteçten fazla değer verdin. Fazla argüman yoksayılır; gcc uyarmasa fark etmezdin.

Buradan çıkan alışkanlık: **Uyarı varsa kodu çalıştırma, önce uyarıyı düzelt.** `scanf` için de aynı kontrol çalışır; birazdan göreceksin.

> **Sık Yapılan Hata: Tamsayı bölmesini `%f` ile yazdırmak**
>
> Bölüm 2'deki tuzağı hatırla: `5 / 2` iki `int` olduğu için sonuç `2`'dir ve türü **`int`**'tir. Bu yüzden şu satır yanlıştır:
>
> ```
> printf("%.2f\n", 5 / 2);      // YANLIŞ: argüman int, belirteç double bekliyor
> printf("%d\n", 5 / 2.0);      // YANLIŞ: argüman double, belirteç int bekliyor
> ```
>
> İkisi de tanımsız davranıştır (gcc uyarı verir). Doğrusu: `printf("%d\n", 5 / 2);` ya da `printf("%.2f\n", 5 / 2.0);`. Belirteci seçmeden önce ifadenin **türünü** bul: iki tamsayıdan çıkan sonuç tamsayıdır, aralarında bir kesirli sayı varsa sonuç kesirlidir.

3.4 scanf ile klavyeden okuma

`scanf` (*scan formatted*), `printf`'in tersini yapar: Klavyeden yazılan metni okur ve biçim belirtecine göre sayıya ya da karaktere çevirip değişkene koyar. `scanf` de `printf` gibi `stdio.h` içinde tanıtılmıştır. Genel kalıp:

```
scanf("biçim metni", &değişken1, &değişken2, ...);
```

İlk örnek, kullanıcıdan yaşını alıp bir yıl sonraki yaşını hesaplıyor. Okunan değer (**19**) çıktı kutusunda mavi görünür; bunu kullanıcı yazmıştır:

yasOku.c

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    printf("Yaşını gir: ");
    scanf("%d", &yas);
    printf("Gelecek yıl %d yaşında olacaksın.\n", yas + 1);
    return 0;
}
```

Çıktı

```
Yaşını gir: 19
Gelecek yıl 20 yaşında olacaksın.
```

`scanf` çağrılınca program **durur** ve sen Enter'a basana kadar bekler. Bu yüzden `scanf`'ten önce mutlaka `printf` ile ne istediğini yazarsın (kullanıcıya soru, ya da *prompt*). Prompt'un sonunda `\n` yoktur ki cevap aynı satırda yazılsın.

Hangi değişken, hangi belirteç?

`scanf` belirteçleri `printf`'inkilere benzer, ama **bir fark çok önemlidir**: `double` için `scanf`'te `%lf` yazılır.

Değişken türü	`scanf` belirteci	`printf` belirteci
`short`	**`%hd`**	`%hd` ya da `%d`
`int`	`%d`	`%d`
`long`	`%ld`	`%ld`
`long long`	`%lld`	`%lld`
`float`	`%f`	`%f`
`double`	**`%lf`**	`%f`
`char`	`%c`	`%c`

`lf` içindeki `l`, *long float* yani `double` demektir. `scanf` okuduğu değeri değişkenin belleğine yazar; kaç bayt yazacağını belirteçten anlar. `%f` görürse 4 baytlık `float` yazar, `%lf` görürse 8 baytlık `double`. Yanlış seçersen bellek bozulur. Aynı tehlike `short` için de vardır: `short s; scanf("%d", &s);` yazarsan `scanf` 2 baytlık `s`'ye 4 bayt yazar (tanımsız davranış); doğrusu `%hd`'dir. `printf`'te ise `short` zaten `int`'e yükseltilerek gönderildiği için `%d` de çalışır.

& neden gerekli? `scanf`'e değişkeninin **değerini** değil, **bellekteki yerini** (adresini) vermen gerekir; çünkü okuduğu sayıyı oraya yazacaktır. `&yas` ifadesi "yas değişkeninin adresi" demektir. Adres ve işaretçi kavramını Bölüm 9'da ayrıntılı göreceksin. Şimdilik kural şu: **`scanf`'te sayı ve karakter değişkenlerinin önüne `&` koyarsın, `printf`'te koymazsın.** (String okumayı Bölüm 8'de göreceksin.)

Birden çok değer okumak

Tek `scanf` ile birden çok değer okuyabilirsin; her belirteç için bir `&değişken` verirsin. Girdiler birbirinden **boşlukla** ya da **Enter ile** ayrılır: `scanf("%d %d", &a, &b)` için `12 30` yazmakla `12`, Enter, `30` yazmak aynıdır. Bu, `scanf`'in boşluk kurallarından gelir:

Biçimde	Girdide ne olur?
`%d`, `%f`, `%lf`	Sayıdan önceki boşluk, sekme ve Enter'ları **kendisi atlar**
`%c`	**Hiçbir şey atlamaz.** Sıradaki karakteri, boşluk ya da Enter bile olsa alır
Biçimdeki boşluk ( )	Girdideki sıfır ya da daha fazla boşluk, sekme ve Enter'ı atlar
Başka karakter (`/`, `:`, `,`, `-`)	Girdide **aynen** bulunmalıdır; bulunmazsa okuma orada durur

Üç farklı türü, üç ayrı satırdan okuyan bir örnek:

uclu.c

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    double boy = 0.0;
    char grup = ' ';
    printf("Yaş, boy (m) ve kan grubu harfi: ");
    scanf("%d %lf %c", &yas, &boy, &grup);
    printf("Yaş: %d, boy: %.2f m, grup: %c\n", yas, boy, grup);
    return 0;
}
```

```
Çıktı
Yaş, boy (m) ve kan grubu harfi: 19
1.78
A
Yaş: 19, boy: 1.78 m, grup: A
```

Biçimdeki `%c`'nin **önündeki boşluk** önemlidir: `1.78`'den sonra gelen Enter karakterini atlar, böylece `%c` `A`'yı okur. Bunun neden gerekli olduğunu birazdan "tampon" başlığında göreceksin.

Kalıplı girdi. Biçim metnine koyduğun sıradan karakterler girdide **aynen** beklenir; böylece `15/03/2005` ya da `09:30` gibi girdileri tek hamlede ayrıştırırsın:

tarih.c

```c
#include <stdio.h>

int main(void) {
    int gun = 0, ay = 0, yil = 0;
    printf("Doğum tarihin (gg/aa/yyyy): ");
    scanf("%d/%d/%d", &gun, &ay, &yil);
    printf("Tarih: %02d.%02d.%d\n", gun, ay, yil);
    return 0;
}
```

```
Çıktı
Doğum tarihin (gg/aa/yyyy): 5/3/2005
Tarih: 05.03.2005
```

`scanf` üç sayıyı `/` karakterlerini atlayarak okudu. Yazdırırken `%02d` gün ve ayı iki haneli, başına sıfır koyarak yazdı (`05`, `03`). Saat için de `scanf("%d:%d", &saat, &dakika)` yazılır. Kullanıcı kalıba uymazsa (`5 3 2005` yazarsa) `scanf` ilk `/` beklediği yerde durur ve kalan değişkenler değişmez. Bu yüzden Bölüm 2'deki kurala uyup değişkenlere ilk değer (`= 0`) verdik: vermeseydik `ay` ve `yil` için çöp değer yazdırılırdı (tanımsız davranış).

3.5 scanf'te en sık iki hata

Biçim metninin argümanlarla uyuşmasını `printf`'te olduğu gibi gcc `scanf` için de denetler. Başlangıçta en çok yapılan iki hata: `&` unutmak ve `double`'ı `%f` ile okumak.

unutulanAmpersand.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 0;
    double boy = 0.0;
    scanf("%d", sayi);
    scanf("%f", &boy);
    printf("%d %f\n", sayi, boy);
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
unutulanAmpersand.c: In function 'main':
unutulanAmpersand.c:6:13: warning: format '%d' expects argument of type 'int *', but argument 2 has
type 'int' [-Wformat=]
    6 |     scanf("%d", sayi);
      |            ~^   ~~~~
      |             |   |
      |             |   int
      |             int *
unutulanAmpersand.c:7:13: warning: format '%f' expects argument of type 'float *', but argument 2
has type 'double *' [-Wformat=]
    7 |     scanf("%f", &boy);
      |            ~^   ~~~~
      |             |   |
      |             |   double *
      |             float *
      |            %lf
```

İlk uyarı `&`'yi unuttuğunu söylüyor: `scanf` `int *` (int'in adresi) bekliyor, sen `int` verdin. Bu durumda `scanf`, `sayi` değişkeninin **değerini** (burada `0`) bir adres sanır ve okuduğu sayıyı oraya yazmaya çalışır. Sonuç tanımsız davranıştır; program çoğunlukla çöker. İkinci uyarı `double` için `%f` yazdığını söylüyor: `scanf` `float *` bekliyor, sen `double *` verdin. gcc çözümü de gösteriyor: `%lf`. Bu hatada program çökmeyebilir; `scanf` 8 baytlık `boy` değişkeninin yalnızca 4 baytını yazar ve `boy` **bozuk bir sayı** olur. Hata sessiz olduğu için daha da tehlikelidir. İkisinde de çalıştırmadan önce uyarıyı düzeltmelisin. Tersi de hatalıdır: `printf("%d", &sayi)` değişkenin değerini değil adresini yazar. Kural: `scanf`'te `&` **var**, `printf`'te **yok**.

> **Sık Yapılan Hata: `scanf`'in biçim metnine soru yazmak**
>
> `scanf("Yaşını gir: %d", &yas);` yazma. `scanf`'in biçim metni ekrana bir şey yazdırmaz; girdide aranacak şablondur. Yukarıdaki satırda kullanıcı `Yaşını gir: 19` yazmazsa okuma başarısız olur. Soruyu `printf` ile sor, `scanf`'e yalnız belirteç ver. Bir de `scanf("%d\n", &x);` yazma: Biçimin sonundaki `\n` bir boşluk gibi çalışır ve sonraki **boşluk olmayan** karakter gelene kadar okur; program sen bir şey daha yazana kadar takılmış gibi görünür.

> **Sık Yapılan Hata: Ondalık sayıda virgül**
>
> Türkçe bölge ayarlarında sayı tuş takımındaki ondalık tuşu virgül (`,`) yazar, ama C ondalık ayırıcı olarak **noktayı** bekler. `scanf("%lf", &boy)` ile `1,78` yazarsan `boy` değişkenine yalnız `1` okunur, `,78` tamponda kalır. Programlarına ondalık sayıyı `1.78` biçiminde gir. Çıktıda da C her zaman nokta yazar (`%.2f` ile `1.78`).

3.6 scanf'in dönüş değeri

`scanf` bir de değer **döndürür**: başarıyla okuyup değişkene yerleştirdiği öğe sayısını. Bu değer, kullanıcının doğru girdi yazıp yazmadığını anlamanın tek yoludur.

Dönüş değeri	Anlamı
`1`, `2`, `3`, …	O kadar öğe başarıyla okundu
`0`	Hiçbir öğe okunamadı (girdi beklenen biçime uymuyor)
`EOF` (genellikle `-1`)	Girdi bitti (Linux ve macOS'ta Ctrl+D, Windows'ta Ctrl+Z ve Enter)

Sayı beklenirken harf girilirse ne olduğunu görelim. Okunamayan değişkene **önceden ilk değer** verdik; vermeseydik içinde çöp kalırdı ve yazdırmak tanımsız davranış olurdu:

harfGirdim.c

```c
#include <stdio.h>

int main(void) {
    int sayi = -1;
    printf("Bir sayı gir: ");
    int okunan = scanf("%d", &sayi);
    printf("scanf %d öğe okudu, sayi = %d\n", okunan, sayi);
    return 0;
}
```

Çıktı

```
Bir sayı gir: abc
scanf 0 öğe okudu, sayi = -1
```

Kullanıcı `abc` yazdı; `%d` bir rakam bekliyordu, `a` rakam değil. `scanf` hiçbir şey okuyamadı ve `0` döndürdü. `sayi` olduğu gibi (`-1`) kaldı. Program çökmedi, uyarı vermedi; `scanf` sessizce vazgeçti. Dönüş değerine bakmazsan hatayı fark edemezsin.

Birden çok değer okurken dönüş değeri, **kaçına kadar** başarıldığını söyler. Örneğin `scanf("%d %d", &a, &b)` çağrısına `7 abc` yazılırsa `7` okunur, `abc` yüzünden ikinci okuma başarısız olur: dönüş değeri `1`, `a` değişir, `b` değişmez. Başarısız olunca `scanf` geçersiz karakterlerin (`abc`) üstünden **atlamaz**; onlar tamponda bekler ve sonraki bir `scanf("%d")` aynı yerde yine takılır. Bunu sonraki iki başlıkta ele alacağız.

Dönüş değerini kullanmanın standart kalıbı, `if` ile `!= 1` kontrolüdür (`if`'i Bölüm 4'te öğreneceksin, şimdilik kalıbı olduğu gibi oku):

```c
if (scanf("%d", &yas) != 1) {
    printf("Geçersiz giriş!\n");
    return 1;
}
```

"`scanf` bir öğe okuyamadıysa hata mesajı ver ve programı bitir" demektir. Böylece çöp değerle devam etmezsin.

> **Sınavda Çıkar**
>
> "`scanf` ne döndürür?" sorusunun cevabı "başarıyla okunan öğe sayısı"dır; `0` döndürmesi hata değil, "okunamadı" demektir. Çıktı sorusu şöyle gelir: `scanf("%d-%d", &a, &b)` ile `5 3` girilirse dönüş değeri **1**'dir (`5` okunur, `-` beklenirken boşluk görülür), `b` değişmez. "Ne kadar okundu?" sorusunda ilk uyuşmazlığın olduğu yerde sayımı durdur.

3.7 Girdi tamponu ve %c tuzağı

Klavyede yazdığın karakterler programa **tek tek** gitmez. İşletim sistemi yazdıklarını bir **tampon**da (buffer) biriktirir ve sen **Enter**'a basınca satırı programa verir. Enter'a bastığında da tampona `'\n'` (yeni satır) karakteri eklenir. `scanf` bu tampondan **ihtiyacı kadarını** alır; geri kalan tamponda bekler.

```
Klavyede yazdığın:      2  0  [Enter]
Tamponun içi:          '2' '0' '\n'

scanf("%d", &yas) okur: '2' '0'  → yas = 20
Tamponda kalan:                  '\n'
```

`%d` sayıyı okuyunca durur; Enter'ın bıraktığı `'\n'` tamponda **kalır**. Bir sonraki `scanf("%d")` için sorun değildir, çünkü `%d` baştaki boşluk ve Enter'ları kendisi atlar. Ama `%c` **hiçbir şey atlamaz**: Tamponda ne varsa alır. İşte klasik tuzak:

tuzak.c

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    char harf = ' ';
    printf("Yaşın: ");
    scanf("%d", &yas);
    printf("Baş harfin: ");
    scanf("%c", &harf);
    printf("yas = %d, harf kodu = %d\n", yas, harf);
    return 0;
}
```

Çıktı

```
Yaşın: 20
Baş harfin: yas = 20, harf kodu = 10
```

Kullanıcı `Baş harfin:` sorusuna cevap **yazamadı**: `scanf("%c")` tamponda hazır bekleyen `'\n'` karakterini okudu ve program devam etti. Harfin kodunu `%d` ile yazdırdık: `10`, yani yeni satır karakterinin ASCII kodu (Ek D). Programı kendi bilgisayarında çalıştırırsan ikinci soruda durmadığını gözünle görürsün.

Çözüm: `%c`'nin önüne boşluk. `%c` öncesinde biçime bir **boşluk** koyarsan `scanf` tamponda kalan boşluk, sekme ve Enter karakterlerini atlar, sonra gerçek karakteri okur:

cozumBosluk.c

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    char harf = ' ';
    printf("Yaşın: ");
    scanf("%d", &yas);
    printf("Baş harfin: ");
    scanf(" %c", &harf);
    printf("yas = %d, harf = %c\n", yas, harf);
    return 0;
}
```

Çıktı

```
Yaşın: 20
Baş harfin: E
yas = 20, harf = E
```

Bu sefer `scanf(" %c")` önce `'\n'` karakterini atladı, tamponun boşaldığını görünce yeni bir satır girilmesini bekledi ve `E`'yi okudu. **Kural: `%c`'yi her zaman `" %c"` yaz.** (Bir önceki `scanf`'in ne okuduğundan bağımsız, hep işe yarar.)

> **Sınavda Çıkar**
>
> "Aşağıdaki programda ikinci `scanf` neden beklemeden geçer?" sorusu klasiktir. Cevap: İlk `scanf("%d")` Enter'ın bıraktığı `'\n'` karakterini tamponda bıraktı; `scanf("%c")` onu okudu. Çözüm: `scanf(" %c", ...)` (boşluklu) ya da tamponu temizlemek. Cevabını şöyle yaz: "`%c` boşluk atlamaz, tamponda kalan `\n`'i alır."

getchar ve putchar

Tek bir karakter okumak ya da yazmak için `scanf` ve `printf` yerine iki küçük fonksiyon vardır: `getchar()` tampondan **bir karakter** alır, `putchar(c)` **bir karakter** yazar. `getchar` boşluk atlamaz; `%c` gibi, tamponda ne varsa onu verir. Sonucunu `char` değil `int` değişkene koyman gerekir: Girdi bittiğinde `getchar` `EOF` döndürür ve bu değer `char`'a sığmayabilir.

tekKarakter.c

```c
#include <stdio.h>

int main(void) {
    printf("Bir karakter yaz: ");
    int c = getchar();
    printf("Yazdığın karakter: ");
    putchar(c);
    putchar('\n');
    printf("ASCII kodu: %d\n", c);
    int sonraki = getchar();
    printf("Sıradaki karakterin kodu: %d\n", sonraki);
    return 0;
}
```

Çıktı

```
Bir karakter yaz: K
Yazdığın karakter: K
ASCII kodu: 75
Sıradaki karakterin kodu: 10
```

Kullanıcı yalnız `K` yazıp Enter'a bastı, ama tamponda iki karakter vardı: `K` ve `'\n'`. İkinci `getchar()` Enter'ı aldı: kodu `10`. Enter da tamponda sıradan bir karakterdir; `scanf` ile aynı mantık.

> **İpucu: Çıktı tamponu: yazı geç görünürse**
>
> `printf` ile ekrana giden metin de bir tamponda biriktirilir ve çoğunlukla satır sonunda (`\n`) boşaltılır. Sonunda `\n` olmayan bir prompt (`printf("Yaşın: ");`) bazı ortamlarda `scanf` beklerken ekranda hiç görünmeyebilir. Böyle olursa `printf("Yaşın: "); fflush(stdout);` yaz. `fflush(stdout)` çıktı tamponunu zorla boşaltır ve C standardına uygundur. Ama `stdin` için aynısını **yapma**; bir sonraki başlığın konusu bu.

3.8 Tamponu temizlemek: fflush(stdin) yanlış

Bazen tamponda artık işine yaramayan karakterler kalır: `scanf("%d")` sonrası `'\n'`, ya da kullanıcının sayı yerine yazdığı `abc`. İnternetteki eski örneklerde ve bazı ders notlarında bunu `fflush(stdin)` ile temizleme önerisi görürsün. **Yapma:**

● Sık Yapılan Hata: `fflush(stdin)` kullanma

```
scanf("%d", &yas);
fflush(stdin);          // YANLIŞ: C standardına göre tanımsız davranış
scanf("%c", &harf);
```

C standardı `fflush`'ı yalnızca **çıktı** akımları için tanımlar. `stdin` bir girdi akımıdır; `fflush(stdin)` standartta **tanımsız davranıştır**. Eski Microsoft kütüphanesi bunu tamponu temizleyen bir eklenti olarak sunduğu için Windows'ta çalışıyormuş gibi görünür; Linux ve macOS'ta ise genellikle tamponu temizlemez ve hata kalır. Aynı kod bilgisayardan bilgisayara farklı davrandığı için sınavda da yanlış sayılır. `fflush(stdout)` doğrudur, `fflush(stdin)` yanlıştır.

Doğru temizleme kalıbı, tampondaki karakterleri `'\n'` görene kadar **tek tek okuyup atmaktır**:

```
while (getchar() != '\n');
```

Bu satır, "`getchar()` ile bir karakter oku, `'\n'` değilse tekrar oku" demektir. Döngüyü (`while`) Bölüm 5'te öğreneceksin; şimdilik tek satırlık bir kalıp olarak al: satırın **kalanını**, Enter dahil, tampondan atar. Bu kısa hâlin bir eksiği var: Girdi tamamen biterse (dosya sonu, `EOF`) `'\n'` hiç gelmez ve döngü sonsuza kadar döner. Bölüm 5'te `EOF`'u da kontrol eden güvenli hâlini göreceksin; döngüleri öğrendikten sonra hep onu kullan. İşte `%c` tuzağının ikinci çözümü:

tamponTemizle.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 0;
    char harf = ' ';
    printf("Sayı: ");
    scanf("%d", &sayi);
    while (getchar() != '\n');
    printf("Harf: ");
    scanf("%c", &harf);
    printf("sayi = %d, harf = %c\n", sayi, harf);
    return 0;
}
```

Çıktı

```
Sayı: 20
Harf: E
sayi = 20, harf = E
```

Bu kez `scanf("%c")` kendiliğinden doğru çalıştı: Temizleme kalıbı `'\n'`'yi attığı için tampon boştu, `scanf` yeni satırı bekledi. İki çözümün farkı:

Çözüm	Ne yapar?	Ne zaman?
`scanf(" %c", &harf);`	Okumadan önce boşluk ve Enter'ları atlar	Karakter okumadan hemen önce, en kolayı
`while (getchar() != '\n');`	Satırın kalanını tamamen atar	Satırda fazla bir şey (`abc`, `20 abc`) kalmış olabilirse

İkincisini Bölüm 8'de `fgets` ile satır okurken de kullanacaksın: `scanf`'ten sonra `fgets` çağırırsan aynı `'\n'` tuzağına düşersin.

> **Sınavda Çıkar**
>
> "Tamponu temizlemek için `fflush(stdin)` kullanılır" ifadesi **yanlıştır**; doğru cevap `while (getchar() != '\n');`. Sınavda "bu kod her derleyicide aynı çalışır mı?" diye sorulursa, `fflush(stdin)` içeren kod için cevap "hayır, tanımsız davranış"tır.

3.9 Örnek uygulamalar

Öğrendiklerini birleştiren üç kısa program. Her birinde önce ne yapacağımızı, sonra kodu, sonra çıktıyı görüyoruz.

Vücut kitle indeksi

Vücut kitle indeksi (VKİ), kiloyu boyun karesine bölerek bulunur: `kilo / (boy * boy)`; kilo kilogram, boy metre cinsindendir. Kilo ve boy kesirli olabileceği için `double` ve `%lf` kullanıyoruz; sonucu iki ondalıkla yazdırıyoruz:

vki.c

```c
#include <stdio.h>

int main(void) {
    double kilo = 0.0, boy = 0.0;
    printf("Kilon (kg) ve boyun (m): ");
    scanf("%lf %lf", &kilo, &boy);
    double vki = kilo / (boy * boy);
    printf("Vücut kitle indeksi: %.2f\n", vki);
    return 0;
}
```

Çıktı

```
Kilon (kg) ve boyun (m): 72.5 1.78
Vücut kitle indeksi: 22.88
```

`72.5 / (1.78 × 1.78)` = `72.5 / 3.1684` ≈ `22.88`. Sonucu "zayıf, normal, fazla kilolu" gibi sınıflandırmak için `if` gerekir; onu Bölüm 4'te bu programa ekleyeceksin.

Saniyeyi sa:dk:sn biçimine çevirme

Toplam saniyeden saat, dakika ve saniyeyi çıkarmak için Bölüm 2'deki tamsayı bölmesini (`/`) ve kalan operatörünü (`%`) kullanırız. 1 saat 3600 saniyedir; bölümden saat çıkar, kalan saniyeyi 60'a bölünce dakika ve artan saniye çıkar. Çıktıyı saat `01:23:45` biçiminde yazmak için `%02d` kullanıyoruz:

saniyeCevir.c

```c
#include <stdio.h>

int main(void) {
    int toplam = 0;
    printf("Saniye: ");
    scanf("%d", &toplam);
    int saat = toplam / 3600;
    int kalan = toplam % 3600;
    int dakika = kalan / 60;
    int saniye = kalan % 60;
    printf("%d saniye = %02d:%02d:%02d\n", toplam, saat, dakika, saniye);
    return 0;
}
```

```
Çıktı
Saniye: 5025
5025 saniye = 01:23:45
```

Adım adım: `5025 / 3600 = 1` saat, `5025 % 3600 = 1425` saniye kalır. `1425 / 60 = 23` dakika, `1425 % 60 = 45` saniye. `%02d` olmasaydı `1:23:45` yazılırdı; dakika ya da saniye tek haneli olduğunda (`1:5:9`) saat gibi görünmezdi.

Fiş yazdırma

Son program, genişlik, hizalama ve hassasiyeti bir arada kullanan küçük bir market fişi. Kullanıcıdan üç ürünün adetlerini alıyor. Sütun başlıklarında bilerek Türkçe karakter kullanmadık; nedenini bu bölümün son başlığında göreceksin.

fis.c

```c
#include <stdio.h>

int main(void) {
    const double ekmekFiyat = 12.50;
    const double peynirFiyat = 185.90;
    const double zeytinFiyat = 94.75;
    const double kdvOrani = 0.10;
    int ekmekAdet = 0, peynirAdet = 0, zeytinAdet = 0;

    printf("Ekmek, peynir, zeytin adetleri: ");
    scanf("%d %d %d", &ekmekAdet, &peynirAdet, &zeytinAdet);

    double ekmekTutar = ekmekAdet * ekmekFiyat;
    double peynirTutar = peynirAdet * peynirFiyat;
    double zeytinTutar = zeytinAdet * zeytinFiyat;
    double araToplam = ekmekTutar + peynirTutar + zeytinTutar;
    double kdv = araToplam * kdvOrani;

    printf("\n           MARKET FİŞİ\n");
    printf("-----------------------------------\n");
    printf("%-10s %4s %9s %9s\n", "Urun", "Adet", "Fiyat", "Tutar");
    printf("-----------------------------------\n");
    printf("%-10s %4d %9.2f %9.2f\n", "Ekmek", ekmekAdet,
           ekmekFiyat, ekmekTutar);
    printf("%-10s %4d %9.2f %9.2f\n", "Peynir", peynirAdet,
           peynirFiyat, peynirTutar);
    printf("%-10s %4d %9.2f %9.2f\n", "Zeytin", zeytinAdet,
           zeytinFiyat, zeytinTutar);
    printf("-----------------------------------\n");
    printf("%-25s %9.2f\n", "Ara toplam:", araToplam);
    printf("%-25s %9.2f\n", "KDV (%10):", kdv);
    printf("%-25s %9.2f\n", "GENEL TOPLAM:", araToplam + kdv);
    return 0;
}
```

```
Çıktı
Ekmek, peynir, zeytin adetleri: 3 1 2

           MARKET FİŞİ
-----------------------------------
Urun       Adet     Fiyat     Tutar
-----------------------------------
Ekmek         3     12.50     37.50
Peynir        1    185.90    185.90
Zeytin        2     94.75    189.50
-----------------------------------
Ara toplam:                 412.90
KDV (%10):                   41.29
GENEL TOPLAM:               454.19
```

Birkaç ayrıntı: Başlık satırı ile ürün satırlarının genişlikleri aynıdır (`10`, `4`, `9`, `9`); bu yüzden sütunlar altlı üstlü oturur. Toplam satırlarında `%-25s` etiketi sola yaslar, `%9.2f` sayıyı sağdaki sütunun altına koyar. `"KDV (%10):"` satırında `%` işaretini tek yazdık, çünkü o yazı **biçim metninde değil**, bir string argümanın içinde; `printf` argümanların içindeki `%` karakterlerini yorumlamaz. Biçim metni olsaydı `%%` yazmak zorundaydın.

3.10 Türkçe karakterler ve Windows konsolu

Bölüm 1'de bırakılan soruyu cevaplayalım: `printf("Merhaba Dünya!\n");` neden Windows'ta `Merhaba D╝nya!` gibi çıkabilir? Program doğru çalışıyor; sorun **karakter kodlamasında** (character encoding). Üç ayrı parça vardır ve üçü aynı kodlamada buluşmalı:

1. **Kaynak dosya.** Editörün `.c` dosyasını diske hangi kodlamayla yazdığı. Biz **UTF-8** kullanıyoruz. UTF-8'de `a`, `b`, `1` gibi karakterler 1 bayt, `ç`, `ğ`, `ı`, `ö`, `ş`, `ü` gibi Türkçe harflerin her biri **2 bayttır**.
2. **gcc.** String'in içindeki baytları olduğu gibi çalıştırılabilir dosyaya koyar. Dosya UTF-8 ise programın ürettiği çıktı da UTF-8 baytlarıdır.
3. **Konsol.** Gelen baytları ekranda hangi harfe çevireceğini kendi **kod sayfası**na (code page) göre seçer. Türkçe Windows'ta varsayılan kod sayfası UTF-8 değildir. Türkçe bir harfin iki UTF-8 baytı konsolda iki ayrı garip karakter olarak görünür; `ü` yerine `Ã¼` ya da `╝` gibi şeyler görürsün.

Yani bozulma Türkçe karakterleri yazarken değil, **gösterirken** olur; program bellekte doğru baytları üretiyordur. Linux ve macOS terminalleri varsayılan olarak UTF-8 kullandığı için orada sorun çıkmaz.

Çözüm 1: Konsolu UTF-8 yap. Programı çalıştırmadan önce aynı terminalde şunu yaz:

```
chcp 65001
```

`chcp` (*change code page*), konsolun kod sayfasını değiştirir; `65001` UTF-8'dir. Komut yalnızca **o terminal penceresi** için geçerlidir, terminali kapatıp açınca eski haline döner; her yeni terminalde yeniden yazman gerekir. Ekranda yazılar hâlâ bozuksa terminalin yazı tipini (font) değiştirmeyi dene; bazı eski yazı tipleri Türkçe harfleri çizemez.

Çözüm 2: Kaynak dosya UTF-8 olsun. VS Code dosyaları varsayılan olarak UTF-8 kaydeder ve pencerenin **sağ alt köşesindeki alt bilgi çubuğunda** kodlamayı gösterir (`UTF-8`). Orada `Windows 1254` gibi başka bir şey yazıyorsa dosya başka kodlamayla kaydedilmiştir: O yazıya tıkla, **Save with Encoding** seç, **UTF-8**'i seç. Dev-C++ gibi bazı eski editörler dosyayı Windows-1254 ile kaydedebilir; o durumda `chcp 65001` yetmez, çünkü baytlar UTF-8 değildir.

Her platformda geçerli bir ayrıntı daha var: Türkçe karakterli string'leri **hizalarken** dikkat etmelisin. `%-8s` genişliği **bayt** sayar, harf saymaz:

turkceGenislik.c

```c
#include <stdio.h>

int main(void) {
    printf("[%-8s]\n", "Ali");
    printf("[%-8s]\n", "Ayşe");
    return 0;
}
```

Çıktı

```
[Ali     ]
[Ayşe    ]
```

`Ali` 3 bayt olduğu için 5 boşlukla tamamlandı ve parantez 10. sütunda kapandı. `Ayşe` ise `ş` yüzünden 5 bayttır (4 harf), 3 boşluk eklendi; ekranda bir karakter **eksik** görünüyor. Sebep `printf` değil, C'nin string'i bayt dizisi olarak görmesidir (Bölüm 8). Bu yüzden hizalı tablolarda Türkçe karakter kullanmamayı, fiş örneğindeki gibi `Urun` yazmayı tercih ediyoruz.

Tuzak: setlocale çözümü. İnternette "Türkçe karakter sorununu `setlocale(LC_ALL, "Turkish")` çözer" diye bir tavsiye görürsün (`locale.h`):

Sık Yapılan Hata: `setlocale` neden iyi çözüm değil

```c
#include <locale.h>
setlocale(LC_ALL, "Turkish");   // Bu bir "çözüm" gibi anlatılır
```

Bu çağrı konsol kod sayfasını **değiştirmez**; asıl sorunu çözmez. Yan etkisi ise gerçektir: Türkçe yerel ayarda ondalık ayırıcı virgüldür. Ondan sonra `printf("%.2f", 3.14159)` ekrana `3,14` yazar ve `scanf("%lf")` `3.14` girişini `3` olarak okuyup `.14`'ü tamponda bırakır; bütün ondalık hesapların bozulur. Üstelik `"Turkish"` adı yalnız Windows'ta geçerlidir; Linux'ta `tr_TR.UTF-8` gerekir. Bu kitapta ve sınavda `setlocale` kullanılmaz; çözüm `chcp 65001` ve UTF-8 dosyadır.

Bir de tek tırnak tuzağı: `'ş'` gibi tek bir `char` sabiti yazamazsın, çünkü `ş` 2 bayttır ve `char` 1 baytlıktır; gcc `multi-character character constant` uyarısı verir. Türkçe karakter yalnız **string** içinde (çift tırnakta) kullanılır. Ayrıntısını Bölüm 8'de göreceksin.

Bölüm Özeti – Bilmen Gerekenler

- `printf` belirteçleri: `%d %i` (int), `%u` (unsigned), `%ld` (long), `%lld` (long long), `%f` (float ve double), `%e` (bilimsel), `%g` (kısa olanı), `%c`, `%s`, `%x %X` (hex), `%o` (sekizlik), `%zu` (`sizeof`), `%%` (yüzde işareti).
- Biçim: `%[bayrak][genişlik][.hassasiyet]harf`. Genişlik **en az** demektir, sığmayan sayı kesilmez. `-` sola yaslar, `0` sıfırla doldurur, `%8.3f` genişliğinde nokta da sayılır.
- `%.nf` yuvarlar, `(int)` keser. Ondalık kesirler ikilik tabanda tam saklanamadığı için `2.675` `%.2f` ile `2.67` çıkar.
- Belirteç türle uyuşmak zorundadır. `int` değeri `%f` ile, `double` değeri `%d` ile yazdırmak **tanımsız davranıştır**; gcc `-Wformat` uyarısı verir. İki tamsayının bölümü `int`'tir.
- `scanf("%d", &x)`: `&` adres demektir ve `scanf`'te gerekir, `printf`'te gerekmez. Okuma belirteçleri: `int` → `%d`, `float` → `%f`, **`double`** → **`%lf`**, `char` → `%c`.
- `%d`, `%f`, `%lf` baştaki boşluk ve Enter'ları atlar; `%c` atlamaz. Biçimdeki boşluk boşluk/Enter atlar. `%d/%d/%d` ya da `%d:%d` gibi kalıplarda ara karakterler girdide aynen bulunmalıdır.

- `scanf` **başarıyla okunan öğe sayısını** döndürür (`0`: hiç okunamadı, `EOF`: girdi bitti). Sayı yerine harf girilirse değişken değişmez, harfler tamponda kalır.
- Tampon: Girdi Enter'a basınca programa gelir ve `'\n'` tamponda kalabilir. `scanf("%c")` onu okur (**`%c` tuzağı**); çözüm `scanf(" %c", &c)`. `getchar()` bir karakter okur (`int` değişkene koy), `putchar(c)` bir karakter yazar.
- `fflush(stdin)` **tanımsız davranıştır**; doğru temizleme `while (getchar() != '\n');` (`EOF`'u da kontrol eden güvenli hâli Bölüm 5'te). `fflush(stdout)` doğrudur.
- Türkçe karakterler UTF-8'de 2 bayttır. Windows konsolunda bozuk görünürse `chcp 65001`; VS Code alt bilgisinde kodlamanın UTF-8 olduğunu kontrol et. `setlocale(LC_ALL, "Turkish")` ondalık ayırıcıyı virgül yapar, kullanma.

Sorular

A) Çoktan Seçmeli

1 `double` türündeki bir değişkeni klavyeden okumak ve ekrana yazdırmak için sırasıyla hangi `scanf` ve `printf` belirteçleri kullanılır?

(A) `scanf` için `%f`, `printf` için `%lf`
(B) `scanf` için `%d`, `printf` için `%d`
(C) `scanf` için `%lf`, `printf` için `%f`
(D) `scanf` için `%f`, `printf` için `%f`
(E) `scanf` için `%lf`, `printf` için `%d`

2 `int a = 0, b = 0;` tanımlıyken `scanf("%d-%d", &a, &b);` çağrılıyor. Aşağıdaki girdilerden hangisinde `a` değişkenine `10`, `b` değişkenine `20` atanır?

(A) `10 20` (B) `10-20` (C) `10,20` (D) `10 - 20` (E) `10/20`

3 Aşağıdaki programı çalıştıran kullanıcı ilk soruya `20` yazıp Enter'a basıyor.

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    char harf = ' ';
    printf("Yaş: ");
    scanf("%d", &yas);
    printf("Harf: ");
    scanf("%c", &harf);
    printf("harf kodu = %d\n", harf);
    return 0;
}
```

İkinci `scanf` çağrısı sırasında ne olur?

(A) Program, kullanıcı bir harf yazıp Enter'a basana kadar bekler.

(B) Derleme hatası oluşur.

(C) `harf` değişkenine `0` atanır.

(D) `scanf` hata verir ve program çöker.

(E) Tamponda kalan Enter (`'\n'`) karakteri `harf`'e okunur ve program beklemeden devam eder.

4 Aşağıdaki biçim belirteçlerinden hangisi `7` sayısını ekrana `007` olarak yazdırır?

(A) `%03d` (B) `%3d` (C) `%-3d` (D) `%d03` (E) `%+3d`

5 `int x;` tanımlıyken `scanf("%d", x);` yazılırsa (yani `&` unutulursa) aşağıdakilerden hangisi doğrudur?

(A) Derleyici hata verir, çalıştırılabilir dosya oluşmaz.

(B) Program normal çalışır ve `x`'e girilen sayı atanır.

(C) Program yalnızca bir uyarı verir ve her zaman sorunsuz çalışır.

(D) gcc `-Wall` ile uyarı verir; program çalışınca tanımsız davranış oluşur ve çoğunlukla çöker.

(E) `x` değişkenine otomatik olarak `0` atanır.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır? Köşeli parantezlerin içindeki boşlukları dikkatle say.

```c
#include <stdio.h>

int main(void) {
    printf("[%5d]\n", 42);
    printf("[%-5d]\n", 42);
    printf("[%05d]\n", 42);
    printf("[%8.3f]\n", 3.14159);
    printf("[%.0f]\n", 2.7);
    printf("[%6s]\n", "abc");
    printf("[%2d]\n", 12345);
    return 0;
}
```

7 Aşağıdaki program çalışırken klavyeden `4,7 9` yazılıp Enter'a basılıyor. Program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int a = 0, b = 0, c = 0;
    int n = scanf("%d,%d,%d", &a, &b, &c);
    printf("%d %d %d %d\n", n, a, b, c);
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    printf("%d%%\n", 50);
    printf("%x %X %o\n", 255, 255, 8);
    printf("%c %d\n", 'A' + 2, 'A' + 2);
    printf("%6.1f|%-4d|%03d\n", 9.96, 7, 5);
    return 0;
}
```

C) Kod Yazma

9 Kullanıcıdan vize, ödev ve final notunu (üç tamsayı, aralarında boşlukla) okuyan ve **notKarti** programını yaz. Ortalama `vize * 0.3 + ödev * 0.2 + final * 0.5` formülüyle bulunur. Çıktı iki satırlık bir tablo olmalı: ilk üç sütun 6, son sütun 8 karakter genişliğinde ve **sağa yaslı**; ortalama iki ondalıklı. Örnek girdi `60 80 70` için, girdi isteyen yazıdan sonra şu tablo çıkmalı:

```
  Vize  Odev Final    Ort.
    60    80    70   69.00
```

10 Önce bir tarihi `gg/aa/yyyy` biçiminde, sonra bir saati `ss:dd` biçiminde okuyan ve **tarihSaat** programını yaz. Tarih ve saati `yyyy-aa-gg ss:dd` biçiminde, gün, ay, saat ve dakika **iki haneli** (gerekirse başına sıfır konarak), yıl dört haneli yazdır. Örnek: İlk satıra `7/3/2026`, ikinci satıra `9:5` yazılırsa son satır `2026-03-07 09:05` olmalı.

Çözümler

Cevap anahtarı

1 C	2 B	3 E	4 A	5 D

1 Cevap: C

`scanf` ile `double` okurken `%lf` gerekir, `printf` ile yazdırırken `%f` kullanılır. A tersini söylüyor: `scanf("%f")` `float` içindir, `double` değişkende belleği bozar. B ve E'deki `%d` tamsayı belirtecidir. D'de `scanf` için `%f` yanlış. (A'daki `printf("%lf")` C99'dan beri zararsızdır, ama `scanf` kısmı yüzünden yine yanlıştır.)

2 Cevap: B

Biçimdeki `-` karakteri girdide **aynen** bulunmalıdır. `10-20` girdisinde `%d` `10`'u okur, `-` eşleşir, ikinci `%d` `20`'yi okur. A'da `-` yok; `%d` `10`'u okuduktan sonra `-` yerine boşluk görür ve okuma durur. C'de `,` var ama `-` beklenir. D'de `10` ile `-` arasında boşluk var; biçimde boşluk olmadığı için `-` yerine boşluk görülür ve okuma durur. E'de `/` var, `-` değil.

3 Cevap: E

`scanf("%d")` `20`'yi okuyup Enter'ın bıraktığı `'\n'` karakterini tamponda bırakır. `%c` boşluk ve Enter atlamadığı için bu `'\n'`'yi okur: `harf` değişkenine `10` (yeni satır) atanır ve program beklemeden devam eder. A yanlış, tampon boş değil. B yanlış, kod derlenir. C yanlış, `harf` `0` değil `10` olur. D yanlış, `scanf` başarıyla bir karakter okur, hata yoktur. Çözüm: `scanf(" %c", &harf);`.

Çıktı

```
Yaş: 20
Harf: harf kodu = 10
```

4 Cevap: A

`0` bayrağı boşluk yerine sıfır koyar: `%03d` ile `7` `007` olur. B `7` (iki boşluk ve 7) yazar. C `7` (7 ve iki boşluk) yazar. D `%d` ile `7`, ardından sıradan `03` metni gelir: `703`. E artı işaretiyle birlikte genişlik 3: `+7`.

Çıktı

```
[007] [  7] [7  ] [703] [ +7]
```

5 Cevap: D

`&` unutulunca `scanf` `x`'in **değerini** adres sanır. gcc `-Wformat` ile `format '%d' expects argument of type 'int *'` uyarısı verir, ama bu bir hata değil uyarıdır (A yanlış). Program çalışınca `scanf` okuduğu sayıyı geçersiz bir adrese yazmaya çalışır; bu tanımsız davranıştır, çoğunlukla program çöker. B, C ve E yanlış: Hiçbirinde "sorunsuz çalışır" ya da "otomatik 0" garantisi yoktur.

6

Her satırda köşeli parantezlerin içindeki karakterleri say:

1. `%5d` ile `42`: genişlik 5, sağa yaslı, üç boşluk + `42`.
2. `%-5d`: sola yaslı, `42` + üç boşluk.

3. `%05d`: boşluk yerine sıfır, `00042`.
4. `%8.3f` ile `3.14159`: `3.142` (nokta dahil beş karakter) + önüne üç boşluk.
5. `%.0f` ile `2.7`: yuvarlanır, `3`.
6. `%6s` ile `"abc"`: üç karakterlik metnin önüne üç boşluk.
7. `%2d` ile `12345`: genişlik sığmıyor, sayı kesilmez: `12345`.

Çıktı
```
[   42]
[42   ]
[00042]
[   3.142]
[3]
[   abc]
[12345]
```

7

`scanf("%d,%d,%d")` biçimdeki `,` karakterlerini girdide arar. Girdi `4,7 9`:

1. İlk `%d` `4`'ü okur → `a = 4`.
2. Biçimdeki `,` girdideki `,` ile eşleşir.
3. İkinci `%d` `7`'yi okur → `b = 7`.
4. Biçimdeki ikinci `,` aranır, ama girdide `7`'den sonra **boşluk** vardır; eşleşme başarısız olur ve okuma durur. `9` hiç okunmaz.
5. `scanf` **2** öğe okuduğu için `2` döndürür; `c` ilk değeri olan `0`'da kalır.

Yazdırılan değerler sırayla `n`, `a`, `b`, `c`:

Çıktı
```
4,7 9
2 4 7 0
```

8

1. `%d%%`: `50` yazılır, ardından `%` işareti: `50%`.
2. `255` onaltılık tabanda `ff` (küçük harf) ve `FF` (büyük harf), `8` sekizlik tabanda `10`.
3. `'A' + 2` işleminin sonucu `int` olarak `67`'dir. `%c` bu değeri karakter olarak (`C`), `%d` sayı olarak (`67`) yazar.
4. `%6.1f` ile `9.96`: tek ondalığa yuvarlanınca `10.0` olur (dört karakter), genişlik 6 olduğu için önüne iki boşluk gelir. `%-4d` ile `7`: `7` ve üç boşluk. `%03d` ile `5`: `005`. Aralardaki `|` sıradan metindir.

Çıktı
```
50%
ff FF 10
C 67
  10.0|7   |005
```

9

Girdileri `%d %d %d` ile okuruz. Ortalamada `0.3`, `0.2`, `0.5` kesirli sabitler olduğu için sonuç `double` olur ve `%8.2f` ile yazdırılır. Başlıkları da aynı genişliklerle `%6s` ve `%8s` ile yazarak sütunların hizalı çıkmasını sağlarız.

notKarti.c

```c
#include <stdio.h>

int main(void) {
    int vizeNot = 0, odevNot = 0, finalNot = 0;
    printf("Vize, ödev ve final notları: ");
    scanf("%d %d %d", &vizeNot, &odevNot, &finalNot);
    double ort = vizeNot * 0.3 + odevNot * 0.2 + finalNot * 0.5;
    printf("%6s%6s%6s%8s\n", "Vize", "Odev", "Final", "Ort.");
    printf("%6d%6d%6d%8.2f\n", vizeNot, odevNot, finalNot, ort);
    return 0;
}
```

Çıktı

```
Vize, ödev ve final notları: 60 80 70
  Vize  Odev Final    Ort.
    60    80    70   69.00
```

10

`gg/aa/yyyy` için biçimde `/`, `ss:dd` için `:` kullanılır. İkinci `scanf`'in başında Enter karakteri tamponda bekler, ama `%d` baştaki Enter'ı kendisi atladığı için sorun çıkmaz. Yazdırırken `%02d` (iki hane, sıfır dolgulu) ve yıl için `%04d` kullanılır.

tarihSaat.c

```c
#include <stdio.h>

int main(void) {
    int gun = 0, ay = 0, yil = 0, saat = 0, dakika = 0;
    printf("Tarih (gg/aa/yyyy): ");
    scanf("%d/%d/%d", &gun, &ay, &yil);
    printf("Saat (ss:dd): ");
    scanf("%d:%d", &saat, &dakika);
    printf("%04d-%02d-%02d %02d:%02d\n", yil, ay, gun, saat, dakika);
    return 0;
}
```

Çıktı

```
Tarih (gg/aa/yyyy): 7/3/2026
Saat (ss:dd): 9:5
2026-03-07 09:05
```

BÖLÜM 4

Koşullu İfadeler

- **Bu Bölümde Öğreneceklerin**
  - İlişkisel operatörleri (`<`, `>`, `<=`, `>=`, `==`, `!=`) ve sonuçlarının neden `1` ya da `0` olduğunu
  - C'de "doğru" ve "yanlış" kavramını: sıfır yanlıştır, sıfır dışı her değer doğrudur; `stdbool.h` ile `bool`
  - `if`, `if-else` ve `else if` merdivenini; süslü parantezsiz `if` ve `if (x > 5);` tuzaklarını
  - İç içe `if`'leri ve sarkan `else` (dangling else) sorununu
  - Mantıksal operatörleri (`&&`, `||`, `!`), doğruluk tablosunu ve kısa devre (short-circuit) değerlendirmeyi
  - `=` ile `==` karışıklığını ve `0 < x < 10` tuzağını
  - Operatör önceliğini: aritmetik → ilişkisel → eşitlik → `&&` → `||` → `?:` → atama
  - Üçlü operatör `?:` ile `switch` deyimini (`break`, `default`, fall-through)
  - `double` değerleri `==` ile karşılaştırmanın neden yanlış olduğunu ve `fabs` ile doğru yolu

Şimdiye kadarki programlar yukarıdan aşağıya, her satırı sırayla çalıştırıyordu. Gerçek programlar ise **karar verir**: "Yaş 18 ya da daha büyükse şunu yap, değilse bunu yap." Bu bölümde akışı koşullara göre dallandırmayı öğreneceksin. Sınavlarda en çok tuzak sorusu çıkan bölümlerden biridir; çünkü küçük bir `=` ya da `;` programın anlamını tamamen değiştirir.

4.1 Karar vermek ve ilişkisel operatörler

Karar vermek için önce iki değeri karşılaştırmak gerekir. Bunu **ilişkisel operatörler** (relational operators) yapar:

Operatör	Anlamı	Örnek	Sonuç
`<`	küçüktür	`3 < 5`	doğru
`>`	büyüktür	`3 > 5`	yanlış
`<=`	küçük eşittir	`5 <= 5`	doğru
`>=`	büyük eşittir	`4 >= 5`	yanlış
`==`	eşittir	`5 == 5`	doğru
`!=`	eşit değildir	`5 != 5`	yanlış

Eşitlik denetimi **iki** eşittir işaretiyle yapılır; tek `=` atama operatörüdür. İki karakterli operatörlerin sırası sabittir: `=<` ya da `=>` diye bir operatör yoktur, `<>` de "eşit değil" anlamına gelmez.

C'de bu operatörlerin sonucu özel bir "doğru/yanlış" türü değil, sıradan bir `int`'tir: karşılaştırma doğruysa `1`, yanlışsa `0`. `printf` ile doğrudan görebilirsin:

iliskisel.c

```c
#include <stdio.h>

int main(void) {
    int a = 7;
    int b = 4;

    printf("a > b       : %d\n", a > b);
    printf("a >= 7      : %d\n", a >= 7);
    printf("a == b      : %d\n", a == b);
    printf("a != b      : %d\n", a != b);
    printf("'a' < 'b'   : %d\n", 'a' < 'b');
    printf("a + b == 11 : %d\n", a + b == 11);
    return 0;
}
```

Çıktı

```
a > b       : 1
a >= 7      : 1
a == b      : 0
a != b      : 1
'a' < 'b'   : 1
a + b == 11 : 1
```

Karakterler de sayı olduğu için karşılaştırılabilir: `'a' < 'b'` doğrudur çünkü ASCII kodları 97 ve 98'dir (Ek D). Son satırda önce toplama, sonra karşılaştırma yapıldı; aritmetik operatörler karşılaştırmalardan önce işlenir. Bunu "Operatör önceliği" başlığında tabloyla toplayacağız.

4.2 C'de doğru ve yanlış

Karşılaştırma `1` ya da `0` üretiyorsa, karar verirken bir sayı nasıl yorumlanır? C'nin kuralı çok basittir:

- `0` **yanlıştır** (false).
- **Sıfırdan farklı her değer doğrudur** (true): `1`, `5`, `-1`, hatta `0.5` bile.

Bunu görmek için `if` deyimini kullanıyoruz: `if (koşul) { ... }`, parantez içi doğruysa süslü parantezin içini çalıştırır. Ayrıntısı bir sonraki başlıkta; şimdilik yalnız kural önemli:

dogruYanlis.c

```c
#include <stdio.h>

int main(void) {
    int sifir = 0;
    int bes = 5;
    int eksiBir = -1;

    if (bes) {
        printf("5 doğru sayılır\n");
    }
    if (eksiBir) {
        printf("-1 doğru sayılır\n");
    }
    if (sifir) {
        printf("0 doğru sayılır\n");
    }
    printf("Bitti\n");
    return 0;
}
```

```
Çıktı
5 doğru sayılır
-1 doğru sayılır
Bitti
```

`5` ve `-1` doğru sayıldı; yalnız `0` için blok çalışmadı. Özellikle **`-1` doğrudur**: "negatif sayı yanlıştır" diye düşünmek klasik bir hatadır. Aynı mantıkla `if (5)` her zaman, `if (0)` hiçbir zaman çalışır. `if (sayi != 0)` yerine kısaca `if (sayi)` yazıldığını da göreceksin; ikisi aynı anlama gelir.

> **Sınavda Çıkar**
>
> "`if (-1)` doğru mu yanlış mı?" ve "`printf("%d", 3 > 5);` ne yazdırır?" tipi sorular çok çıkar. Cevaplar: `-1` **doğrudur** (sıfır dışı); `3 > 5` yanlıştır, yani `0` yazdırılır. Sıfır dışı = doğru, sıfır = yanlış. Başka kural yok.

bool türü: stdbool.h

C99'dan beri `stdbool.h` ile okunaklı bir doğru/yanlış türü kullanabilirsin: `bool`, `true` (1) ve `false` (0). Başlık dosyasını kendin eklemelisin:

boolKullanimi.c

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    bool yetiskin = 20 >= 18;
    bool ogrenci = false;
    bool cokluDeger = 42;

    printf("%d %d %d\n", yetiskin, ogrenci, cokluDeger);
    return 0;
}
```

```
Çıktı
1 0 1
```

`printf`'te `bool` için `%d` kullanılır. `bool` değişkenine `42` gibi sıfır dışı bir değer atarsan değişken `1` olur: `bool` yalnızca iki değer taşır. `bool` ad olarak da niyeti gösterir (`yetiskin`, `ogrenciMi`, `bulundu`): `if (yetiskin)` satırı cümle gibi okunur.

> **C23 Notu: bool, true, false**
>
> C23'te `bool`, `true` ve `false` artık **anahtar sözcüktür**; `#include <stdbool.h>` yazmadan kullanılır (gcc 14 ve sonrasında `-std=c23` ile; gcc 15'ten itibaren C23 zaten varsayılandır). Biz C17 yazıyoruz, bu yüzden `stdbool.h`'i eklemeye devam et.

4.3 if, if-else ve else if

`if` koşul doğruysa bir bloğu çalıştırır; `else` koşul yanlışken çalışacak yolu verir. Koşul yanlışsa ve `else` yoksa blok atlanır, program bir sonraki satırdan devam eder:

```c
if (koşul) {
    // koşul doğruysa çalışır
} else {
    // koşul yanlışsa çalışır (else kısmı isteğe bağlıdır)
}
```

Sayının tek mi çift mi olduğunu `if (sayi % 2 == 0) { ... } else { ... }` ile bulursun: iki bloktan **tam biri** çalışır. Negatif sayılarda dikkat: `%` sonucu bölünenin işaretini taşır, yani `-7 % 2` sonucu `-1`'dir. Bu yüzden tek sayıyı `sayi % 2 == 1` ile aramak negatiflerde yanılır; `sayi % 2 != 0` güvenlidir.

else if merdiveni

İkiden fazla yol olduğunda `else if` ile **merdiven** kurarsın. Koşullar **yukarıdan aşağıya** denenir; ilk doğru çıkan bloğun içi çalışır, merdivenin geri kalanı atlanır. Hiçbiri doğru değilse en sondaki `else` çalışır:

harfNotu.c

```c
#include <stdio.h>

int main(void) {
    int puan = 0;

    printf("Puanın (0-100): ");
    scanf("%d", &puan);

    if (puan >= 90) {
        printf("Harf notu: A\n");
    } else if (puan >= 80) {
        printf("Harf notu: B\n");
    } else if (puan >= 70) {
        printf("Harf notu: C\n");
    } else if (puan >= 60) {
        printf("Harf notu: D\n");
    } else {
        printf("Harf notu: F\n");
    }
    return 0;
}
```

Çıktı

```
Puanın (0-100): 74
Harf notu: C
```

`74` için sırayla bakıldı: `>= 90` yanlış, `>= 80` yanlış, `>= 70` doğru. `C` yazıldı ve `>= 60` koşuluna **hiç bakılmadı**. Bu yüzden `puan >= 70 && puan < 80` yazmana gerek yok: `puan >= 80` zaten yukarıda elenmiştir. Merdivenin **sırası** önemlidir; `puan >= 60` koşulunu en üste koysaydın 95 alan öğrenci de `D` alırdı.

> **Sık Yapılan Hata: Ayrı `if`'ler ile `else if` aynı şey değildir**
>
> Her koşulu ayrı bir `if` ile yazarsan her biri bağımsız denenir ve birden fazlası çalışabilir. Yukarıdaki merdiveni `else` olmadan yazıp `puan = 95` verirsen hem `A` hem `B` hem `C` hem `D` yazılır; çünkü `95` her koşulu sağlar. Tek sonuç istiyorsan `else if` kullan.

4.4 Süslü parantez tuzakları

Gövdesinde tek deyim olan `if`, süslü parantezsiz de yazılabilir. Ama bu durumda koşula **yalnızca ilk deyim** bağlıdır. Girinti derleyiciyi ilgilendirmez; yalnız insanı yanıltır:

suslusuz.c

```c
#include <stdio.h>

int main(void) {
    int puan = 80;

    if (puan < 50)
        printf("Kaldın.\n");
        printf("Tekrar dene.\n");
    return 0;
}
```

gcc girintinin gerçek akışla uyuşmadığını fark edip uyarır:

Derleyici uyarısı – gcc -Wall -Wextra

```
suslusuz.c: In function 'main':
suslusuz.c:6:5: warning: this 'if' clause does not guard... [-Wmisleading-indentation]
    6 |     if (puan < 50)
      |     ^~
suslusuz.c:8:9: note: ...this statement, but the latter is misleadingly indented as if it were
guarded by the 'if'
    8 |         printf("Tekrar dene.\n");
      |         ^~~~~~
```

Puan `80` olduğu için koşul yanlış ve `Kaldın.` yazılmadı; ama ikinci `printf` `if`'e bağlı **değildi**, yine de çalıştı:

Çıktı

```
Tekrar dene.
```

> **İpucu: Süslü parantezleri her zaman yaz**
>
> Gövde tek satır bile olsa `{ }` kullan; sonradan araya satır eklediğinde bu hataya düşmezsin. Sınavda süslü parantezsiz kodla karşılaşırsan girintiye değil, koşula bağlı **tek deyime** bak.

if (koşul); tuzağı

İkinci tuzak, koşulun hemen arkasına yanlışlıkla `;` koymaktır. C'de tek başına `;` bir **boş deyimdir** (empty statement): hiçbir şey yapmaz. Yani `if (x > 5);` "koşul doğruysa hiçbir şey yap" demektir; alttaki blok ise sıradan bir blok olarak **her zaman** çalışır:

bosGovde.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 3;

    if (sayi > 5);
    {
        printf("Sayı 5'ten büyük\n");
    }
    return 0;
}
```

gcc bu alışkanlığı `-Wextra` içindeki `-Wempty-body` ile (ve yanıltıcı girinti uyarısıyla) yakalar:

Derleyici uyarısı – gcc -Wall -Wextra

```
bosGovde.c: In function 'main':
bosGovde.c:6:18: warning: suggest braces around empty body in an 'if' statement [-Wempty-body]
    6 |     if (sayi > 5);
      |                  ^
bosGovde.c:6:5: warning: this 'if' clause does not guard... [-Wmisleading-indentation]
    6 |     if (sayi > 5);
      |     ^~
bosGovde.c:7:5: note: ...this statement, but the latter is misleadingly indented as if it were
guarded by the 'if'
    7 |     {
      |     ^
```

Çıktı

```
Sayı 5'ten büyük
```

`sayi` 3 olduğu hâlde “5’ten büyük” yazıldı. Sınavda `)` sonrasındaki `;` işaretini ara.

4.5 İç içe if ve sarkan else

Bir `if`’in gövdesine başka bir `if` yazabilirsin (iç içe if, nested if). **Sarkan else** (dangling else) sorunu süslü parantez yazılmadığında çıkar. Kural: **`else`, kendisinden önceki, henüz `else` almamış en yakın `if`’e bağlanır.** Girinti buna bakmaz:

sarkanElse.c

```c
#include <stdio.h>

int main(void) {
    int a = 0;
    int b = 0;

    if (a == 1)
        if (b == 1)
            printf("A\n");
    else
        printf("B\n");
    printf("Bitti\n");
    return 0;
}
```

`else`’i dış `if`’in hizasına yazdık; “a 1 değilse B yaz” demek istediğimizi sanırsın. gcc şüphelenir (`-Wdangling-else`, `-Wall` içindedir):

Derleyici uyarısı – gcc -Wall -Wextra

```
sarkanElse.c: In function 'main':
sarkanElse.c:7:8: warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
    7 |     if (a == 1)
      |        ^
```

Gerçekte `else` iç `if`’e (`b == 1`) bağlanır. `a` sıfır olduğu için iç `if`’e hiç girilmez ve `B` de yazılmaz:

Çıktı

```
Bitti
```

Niyet `else`'in dış `if`'e ait olması idiyse iç `if`'i süslü parantezle kapatırsın: `if (a == 1) { if (b == 1) { ... } } else { ... }`. Bu kez `else` dış `if`'e bağlanır ve aynı `a` ile `b` değerleri için program `B` yazar.

> **Sınavda Çıkar**
>
> Sarkan else sorularında kod bilerek yanıltıcı girintilenir. Önce girintiyi **yok say**, `else`'i en yakın `if`'e bağla, sonra çalıştır. Süslü parantez varsa bağlama süslü parantezlere göredir. "Girinti öyle gösteriyor" cevabı sınavda yanlıştır.

4.6 Mantıksal operatörler

Birden fazla koşulu birleştirmek için üç **mantıksal operatör** (logical operators) vardır: `&&` (VE), `||` (VEYA), `!` (DEĞİL). Doğruluk tablosu (`0` yanlış, `1` doğru):

a	b	`a && b`	`a || b`	`!a`
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

`&&` yalnız **ikisi de** doğruysa, `||` **en az biri** doğruysa doğrudur; `!` doğruyu yanlış, yanlışı doğru yapar. Sonuç yine `1` ya da `0` olan bir `int`'tir. İşlenen olarak sıfır dışı her değer doğru sayılır; yani `5 && 3` sonucu `3` değil `1`'dir. En çok kullanıldığı yer **aralık denetimidir**: Puanın geçerli olması için `puan >= 0 && puan <= 100` ("içinde mi?"), geçersiz olması için `puan < 0 || puan > 100` ("dışında mı?") yazarsın.

Bu iki koşul birbirinin tersidir; bu eşdeğerliğe **De Morgan kuralları** denir: `!(a && b)` ile `!a || !b` aynıdır, `!(a || b)` ile `!a && !b` aynıdır. Parantezin önündeki `!`'yi içeri dağıtırken `&&` ile `||` yer değiştirir, her karşılaştırma da tersine döner (`>=` yerine `<`, `<=` yerine `>`).

> **Sık Yapılan Hata: x == 1 || 2 yazmak**
>
> "x, 1 ya da 2'ye eşit mi?" sorusunu `if (x == 1 || 2)` diye yazmak çok yaygın bir hatadır. İfade `(x == 1) || (2)` diye okunur ve `2` sıfır dışı olduğu için **her zaman doğrudur**: `x` kaç olursa olsun blok çalışır. Doğrusu her koşulu ayrı yazmaktır: `x == 1 || x == 2`. (Tek karakterli `&` ve `|` ise bit düzeyinde işlem yapan başka operatörlerdir; mantıksal VE/VEYA için her zaman çift karakter yaz.)

Kısa devre değerlendirme

`&&` ve `||` işlenenlerini **soldan sağa** değerlendirir ve sonuç belli olur olmaz durur. Buna **kısa devre** (short-circuit) denir:

İfade	Sağ taraf ne zaman hiç hesaplanmaz?
`sol && sag`	`sol` yanlışsa (sonuç zaten yanlış)
`sol || sag`	`sol` doğruysa (sonuç zaten doğru)

Bu yalnız hız için değil, **güvenlik** için de işe yarar. Sıfıra bölmeyi engellemek için önce `x != 0` denetimini yaz:

guvenliBolme.c

```c
#include <stdio.h>

int main(void) {
    int x = 0;

    printf("x: ");
    scanf("%d", &x);

    if (x != 0 && 10 / x > 1) {
        printf("10 / x, 1'den büyük.\n");
    } else {
        printf("Ya x sıfır ya da 10 / x, 1'den büyük değil.\n");
    }
    return 0;
}
```

Çıktı

```
x: 0
Ya x sıfır ya da 10 / x, 1'den büyük değil.
```

`x` sıfırken `x != 0` yanlış olduğundan `10 / x` **hiç hesaplanmadı**. Sırayı ters yazarsan (`10 / x > 1 && x != 0`) önce bölme yapılır. Tamsayıyı sıfıra bölmek tanımsız davranıştır: program çökebilir de çökmeyebilir de. Yani koruyucu koşul **solda** durur.

Kısa devrenin ikinci sonucu, sağ taraftaki **yan etkilerin** (side effect) de atlanmasıdır. `++b` sağ taraftaysa her zaman çalışacağını sanma:

yanEtki.c

```c
#include <stdio.h>

int main(void) {
    int a = 0;
    int b = 0;

    int s1 = a > 0 && ++b > 0;
    printf("s1 = %d, b = %d\n", s1, b);
    int s2 = a == 0 || ++b > 0;
    printf("s2 = %d, b = %d\n", s2, b);
    int s3 = a == 0 && ++b > 0;
    printf("s3 = %d, b = %d\n", s3, b);
    return 0;
}
```

Çıktı

```
s1 = 0, b = 0
s2 = 1, b = 0
s3 = 1, b = 1
```

1. `a > 0` yanlış → `&&`'in sağı atlanır, `++b` çalışmaz. `b` hâlâ `0`.
2. `a == 0` doğru → `||`'nin sağı atlanır, `++b` yine çalışmaz. `b` hâlâ `0`.
3. `a == 0` doğru → `&&`'nin sonucu henüz belli değil, sağ taraf hesaplanır. `++b` çalışır, `b` olur `1`.

> **Sınavda Çıkar**
>
> "`&&` ya da `||` içindeki `++` çalıştı mı?" sorusunun kuralı hep aynıdır: `&&`'de sol yanlışsa, `||`'de sol doğruysa sağ taraf **atlanır**. Çıktı sorularında değişkenleri adım adım yaz ve sağ tarafın çalışıp çalışmadığını işaretle. En sık hata, `++b`'nin "her halükarda" çalıştığını varsaymaktır.

4.7 = ile == karışıklığı

`=` **atama**, `==` **karşılaştırma** operatörüdür. Koşula yanlışlıkla `=` yazarsan derleyici hata vermez; çünkü atama da bir ifadedir ve **değeri atanan değerdir**: `x = 5` ifadesinin değeri `5`, yani doğrudur:

atamaTuzagi.c

```c
#include <stdio.h>

int main(void) {
    int x = 3;

    if (x = 5) {
        printf("x 5'e eşit\n");
    }
    printf("x = %d\n", x);
    return 0;
}
```

gcc koşulun etrafına parantez önerir:

Derleyici uyarısı – gcc -Wall -Wextra

```
atamaTuzagi.c: In function 'main':
atamaTuzagi.c:6:9: warning: suggest parentheses around assignment used as truth value [-
Wparentheses]
    6 |     if (x = 5) {
      |         ^
```

Program yine de derlenir ve çalışır:

Çıktı

```
x 5'e eşit
x = 5
```

Kötü haber iki tane: `x` 3 iken "5'e eşit" yazıldı **ve** `x` kalıcı olarak 5 oldu. `if (x = 0)` yazsaydın değer `0` (yanlış) olacağından blok hiç çalışmaz, `x` de sıfırlanırdı. Bilerek atama yapıp sonucuna bakacaksan uyarıyı çift parantezle susturursun: `if ((x = 5))`.

> **İpucu: Yoda yazımı: sabiti sola yaz**
>
> Karşılaştırmada sabiti sola yazarsan (`if (5 == x)`) bu hatadan korunursun: Yanlışlıkla `5 = x` yazarsan 5'e değer atanamayacağı için gcc `lvalue required as left operand of assignment` hatası verir ve hemen fark edersin (*lvalue*, atamanın sol tarafında durabilen şeydir, yani değişkendir). Adı "Yoda koşulu"dur (Yoda gibi ters konuşur). Bu kitapta okunaklı olduğu için `x == 5` yazıyoruz, ama ikisini de tanımalısın.

4.8 0 < x < 10 tuzağı

Matematikte “x, 0 ile 10 arasındadır” diye `0 < x < 10` yazılır. C’de bu derlenir ama **matematikteki anlamı yoktur**. İlişkisel operatörler soldan sağa işlenir: önce `0 < x`, sonra bu sonucun (`0` ya da `1`) `10`’dan küçük olup olmadığına bakılır. `0` ya da `1` her zaman 10’dan küçüktür, yani ifade **her zaman doğrudur**:

zincir.c

```c
#include <stdio.h>

int main(void) {
    int x = 15;

    if (0 < x < 10) {
        printf("x 0 ile 10 arasında\n");
    } else {
        printf("x aralıkta değil\n");
    }
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
zincir.c: In function 'main':
zincir.c:6:15: warning: comparison of constant '10' with boolean expression is always true [-Wbool-compare]
    6 |     if (0 < x < 10) {
      |               ^
zincir.c:6:11: warning: comparisons like 'X<=Y<=Z' do not have their mathematical meaning [-Wparentheses]
    6 |     if (0 < x < 10) {
      |         ~~^~~
```

Çıktı

```
x 0 ile 10 arasında
```

gcc iki uyarı verdi: Karşılaştırmanın sonucu hep doğru (`-Wbool-compare`) ve bu yazımın matematikteki anlamı yok (`-Wparentheses`). `x` 15 olduğu hâlde “arasında” yazıldı: `0 < 15` doğru → `1`; `1 < 10` doğru. Doğrusu iki koşulu `&&` ile bağlamaktır: `x > 0 && x < 10`. Aynı tuzak eşitlikte de var: `a == b == c`, “üçü de eşit” değil `(a == b) == c` demektir; doğrusu `a == b && b == c`.

4.9 Operatör önceliği

Bir ifadede birden fazla operatör varsa hangisinin önce işleneceğini **öncelik** (precedence) belirler. Bölüm 2’de aritmetik operatörlerin sırasını gördün; şimdi hepsini tek tabloda topluyoruz (yüksekten düşüğe):

Sıra	Operatörler	Birleşme
1	`( )` gruplama	içten dışa
2	son ek `x++`, `x--`; dizi indeksi `a[i]` ve fonksiyon çağrısı `f(x)` (Bölüm 6–7)	soldan sağa
3	`!`, `~`, ön ek `++x`, `--x`, tekli `-` ve `+`, `(tip)` dönüşümü, `sizeof` (Bölüm 9'daki `*p` ve `&x` de burada)	sağdan sola
4	`*`, `/`, `%`	soldan sağa
5	`+`, `-`	soldan sağa
6	`<<`, `>>`	soldan sağa
7	`<`, `<=`, `>`, `>=`	soldan sağa
8	`==`, `!=`	soldan sağa
9	`&` (bit VE)	soldan sağa
10	`^` (bit XOR)	soldan sağa
11	`|` (bit VEYA)	soldan sağa
12	`&&`	soldan sağa
13	`||`	soldan sağa
14	`?:` (üçlü operatör)	sağdan sola
15	`=`, `+=`, `-=`, `*=`, `/=`, `%=` ve bit atamaları (`&=`, `|=`, `^=`, `<<=`, `>>=`)	sağdan sola

Son ek `x++` ile ön ek `++x` **farklı satırlardadır**: son ek daha güçlüdür ve soldan sağa birleşir. Bu yüzden `!x++` ifadesi `!(x++)` diye okunur.

Hatırlama yolu: **önce aritmetik, sonra karşılaştırma, sonra mantık, en son atama.** Bit operatörleri araya girer: kaydırma (`<<`, `>>`) aritmetikten hemen sonra, `&`, `^`, `|` ise karşılaştırmadan sonra gelir. Bu yüzden `x & 1 == 0` ifadesi `x & (1 == 0)` diye işlenir; bit işleminde parantez kullan. `&&` her zaman `||`'den güçlüdür (matematikte çarpmanın toplamadan önce gelmesi gibi).

oncelik.c

```c
#include <stdio.h>

int main(void) {
    int a = 5;
    int b = 3;
    int c = 2;

    printf("%d\n", a + b > 7 && c * 2 == 4);
    printf("%d\n", a > 3 || b > 5 && c > 5);
    printf("%d\n", (a > 3 || b > 5) && c > 5);
    return 0;
}
```

Çıktı

```
1
1
0
```

1. `a + b > 7 && c * 2 == 4`: önce `a + b` (8) ve `c * 2` (4); sonra `8 > 7` doğru, `4 == 4` doğru; sonuç `1`.
2. `a > 3 || b > 5 && c > 5`: önce `&&`: `b > 5` yanlış olduğundan `b > 5 && c > 5` yanlış. `a > 3` doğru olduğu için `||` sonucu `1`.
3. Parantez önceliği değiştirir: `(a > 3 || b > 5)` doğru, ama `c > 5` yanlış; sonuç `0`.

İkinci satırda `&&` ile `||` parantezsiz karıştığı için `-Wall` uyarır:

Derleyici uyarısı – gcc -Wall -Wextra

```
oncelik.c: In function 'main':
oncelik.c:9:35: warning: suggest parentheses around '&&' within '||' [-Wparentheses]
    9 |     printf("%d\n", a > 3 || b > 5 && c > 5);
      |                                   ~~~~~~^~~~~~~
```

İpucu: Karışık &&/|| için parantez koy

`&&`'nin `||`'den önce işlendiğini bilsen bile parantez koy: `a > 3 || (b > 5 && c > 5)`. Hem gcc susar hem de kodu okuyan (hoca dahil) hesap yapmak zorunda kalmaz. Sınavda ise parantezsiz yazılmış ifadeyi **sen** tabloyla çözersin.

! operatörünün önceliği

`!` tekli (unary) bir operatördür ve tabloda karşılaştırmalardan **önce** gelir. Bu yüzden "x, 5 değil mi?" diye `if (!x == 5)` yazmak klasik bir hatadır: ifade `(!x) == 5` diye okunur. `!x` yalnızca `0` ya da `1` verebildiği için bu koşul **hiçbir `x` için doğru olamaz**; gcc de `-Wlogical-not-parentheses` ile uyarır. Doğru yazım `x != 5` ya da `!(x == 5)`'tir. Aynı sebeple `!x + 1` ifadesi `!(x + 1)` değil `(!x) + 1` demektir.

4.10 Üçlü operatör ?:

C'de üç işlenenli tek operatör `?:`'dir. Bir koşula göre iki değerden birini **seçer**:

```
koşul ? değerDoğruysa : değerYanlışsa
```

`if-else` bir **deyimdir** (statement) ve değer üretmez; `?:` ise bir **ifadedir** (expression) ve değer üretir. Bu yüzden sonucu doğrudan bir değişkene atayabilir ya da `printf`'in içine koyabilirsin:

ucluOperator.c

```c
#include <stdio.h>

int main(void) {
    int a = 12;
    int b = 20;
    int sayi = 7;
    int puan = 85;

    int enBuyuk = a > b ? a : b;
    printf("En büyük: %d\n", enBuyuk);
    printf("%d sayısı %s\n", sayi, sayi % 2 == 0 ? "çift" : "tek");
    printf("Not: %s\n", puan >= 90 ? "A" : puan >= 80 ? "B" : "C");
    return 0;
}
```

Çıktı

```
En büyük: 20
7 sayısı tek
Not: B
```

`%s`, ekrana bir metin (string) yazdıran yer tutucudur; string'leri Bölüm 8'de ayrıntılı göreceksin. Son satırda `?:` **iç içe** yazıldı. `?:` sağdan sola birleştiği için `a ? b : c ? d : e`, `a ? b : (c ? d : e)` demektir. Çalışır, ama okunması hızla zorlaşır. Kural: `?:` iki değerden birini seçmek için güzeldir (`enBuyuk`, tek/çift); iç içe geçtiğinde ya da seçeneklerin içinde birden fazla işlem gerektiğinde `else if` merdiveni kullan. Okunabilirlik sınavda ve ödevde puanın bir parçasıdır.

4.11 switch deyimi

Bir değişkeni **belirli sabit değerlerle** karşılaştırıyorsan uzun bir `else if` merdiveni yerine `switch` kullanabilirsin:

```c
switch (ifade) {
    case sabit1:
        // ifade == sabit1 ise buradan başlar
        break;
    case sabit2:
        break;
    default:
        // hiçbir case uymazsa (isteğe bağlı)
}
```

`ifade` hesaplanır, değeri hangi `case` etiketine eşitse program **o etiketin altından** çalışmaya başlar. `break` görülünce `switch`'ten çıkılır; hiçbir etiket uymazsa `default`'tan devam edilir:

sesliHarf.c

```c
#include <stdio.h>

int main(void) {
    char harf = ' ';

    printf("Bir harf yaz (a–z): ");
    scanf(" %c", &harf);

    switch (harf) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
            printf("'%c' bir sesli harftir.\n", harf);
            break;
        default:
            printf("'%c' sesli harf değil.\n", harf);
    }
    return 0;
}
```

Çıktı

```
Bir harf yaz (a–z): e
'e' bir sesli harftir.
```

İlk satırdaki beş etiket alt alta (ya da yan yana) yazılmış ve aralarında deyim yok: Hangisi eşleşirse aynı `printf`'e düşülür. Bu, aşağıda göreceğin kazara düşmeden farklı olarak **bilerek** yapılır.

`switch`'in kuralları:

- `switch (ifade)` içindeki ifade **tamsayı türünde** olmalıdır: `int`, `char`, `short`, `long` ve ileride göreceğin `enum`. `double` ya da `float` olmaz.
- `case` etiketleri derleme zamanında bilinen **sabit** olmalıdır: `3`, `'a'`, `2 + 3`, `#define` ile tanımlı bir ad. Değişken olmaz.
- `default` isteğe bağlıdır ve herhangi bir yerde olabilir; sona koymak iyi alışkanlıktır. Aynı değer iki `case`'de yazılamaz.
- `switch`, yalnız **eşitlik** denetler. "Puan 70'ten büyükse" gibi aralıklar için `else if` merdiveni kullan.

break unutulursa: fall-through

`break` yazmazsan program etiketi geçince durmaz; **bir alttaki case'in kodunu da çalıştırmaya devam eder**. Buna düşme (fall-through) denir ve `switch`'in en bilinen tuzağıdır:

breakYok.c

```c
#include <stdio.h>

int main(void) {
    int secim = 1;

    switch (secim) {
        case 1: printf("Bir\n");
        case 2: printf("İki\n");
        default: printf("Başka\n");
    }
    return 0;
}
```

gcc, `-Wextra` içindeki `-Wimplicit-fallthrough` ile yakalar:

Derleyici uyarısı — gcc -Wall -Wextra

```
breakYok.c: In function 'main':
breakYok.c:7:17: warning: this statement may fall through [-Wimplicit-fallthrough=]
    7 |         case 1: printf("Bir\n");
      |                 ^~~~~~~~~~~~~~~
breakYok.c:8:9: note: here
    8 |         case 2: printf("İki\n");
      |         ^~~~
breakYok.c:8:17: warning: this statement may fall through [-Wimplicit-fallthrough=]
    8 |         case 2: printf("İki\n");
      |                 ^~~~~~~~~~~~~~~
breakYok.c:9:9: note: here
    9 |         default: printf("Başka\n");
      |         ^~~~~~~
```

Çıktı

```
Bir
İki
Başka
```

`secim` 1 olduğu için `case 1`'in altından başlandı ve `break` olmadığı için `case 2` ile `default`'taki satırlar da çalıştı. Etiket yalnızca **nereden başlanacağını** söyler; nerede duracağını `break` söyler. Son bloğa `break` gerekmez. Sınavda yöntem şudur: eşleşen etiketi bul, oradan aşağı doğru ilk `break`'e kadar (ya da `switch`'in sonuna kadar) her satırı çalıştır; aradaki etiketler yalnız giriş noktasıdır, kendileri bir şey yapmaz.

Bilerek gruplanan case'ler

Birden fazla değer için **aynı işi** yapmak istediğinde, `sesliHarf`'teki gibi etiketleri alt alta dizersin. Araya hiç deyim girmeyen boş etiketler gcc'yi uyarmaz; `-Wimplicit-fallthrough` yalnız içinde deyim olan bir case'in aşağı düşmesine uyarır. (C23'te bilerek düşmeyi `[[fallthrough]];` ile işaretleyebilirsin; gcc'ye özgü eski yolu `__attribute__((fallthrough));` biçimindedir.) Örnek İngilizce alfabedeki harfler içindir: `ü`, `ö` gibi Türkçe karakterler UTF-8'de iki bayt tuttuğu için tek bir `char`'a sığmaz, onları Bölüm 8'de string'lerle ele alacağız.

case etiketinde neler yazılamaz

`case` etiketi derleme zamanında bilinen bir **tamsayı sabit** olmalıdır. Üç hatayı birlikte görelim:

caseHatalari.c

```c
#include <stdio.h>

int main(void) {
    int sinir = 5;
    int n = 3;

    switch (n) {
        case sinir: printf("a\n"); break;
        case 2.5: printf("b\n"); break;
        case "evet": printf("c\n"); break;
    }
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
caseHatalari.c: In function 'main':
caseHatalari.c:8:9: error: case label does not reduce to an integer constant
    8 |         case sinir: printf("a\n"); break;
      |         ^~~~
caseHatalari.c:9:9: error: case label does not reduce to an integer constant
    9 |         case 2.5: printf("b\n"); break;
      |         ^~~~
caseHatalari.c:10:9: error: pointers are not permitted as case values
   10 |         case "evet": printf("c\n"); break;
      |         ^~~~
```

- `case sinir:` değişken olduğu için yazılamaz; `#define` ile tanımlı bir ad ya da `2 + 3` gibi sabit ifade yazabilirsin. C'de `const int sinir = 5;` bile `case` için sabit sayılmaz.
- `case 2.5:` ondalıklı sayı tamsayı sabit değildir. (`switch (d)` ile `double` bir ifadeyi seçmeye kalkarsan gcc `switch quantity not an integer` der.)
- `case "evet":` string yazılamaz; metinleri `==` ile de karşılaştıramazsın. Doğru yol Bölüm 8'deki `strcmp` fonksiyonudur.

gcc'nin `case 1 ... 5:` biçiminde aralık yazan bir eklentisi vardır ama standart C değildir; sınavda kullanma. `case 1, 2:` gibi virgüllü yazım da yoktur; bunun yerine `case 1: case 2:` yaz.

4.12 double değerlerini karşılaştırmak

`double` değerler bellekte ikilik tabanda tutulur ve `0.1` gibi sayılar tam yazılamaz (Bölüm 2'deki hassasiyet konusu). Bu yüzden iki hesabın sonucu "matematikte eşit" olsa da bitleri aynı olmayabilir. `==` ise **tam** eşitlik arar:

ondalik.c

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double a = 0.1 + 0.2;
    double b = 0.3;
    const double tolerans = 1e-9;

    printf("a = %.17f\n", a);
    printf("b = %.17f\n", b);
    printf("a == b : %d\n", a == b);
    printf("fark   : %.20f\n", fabs(a - b));
    printf("fabs(a - b) < tolerans : %d\n", fabs(a - b) < tolerans);
    return 0;
}
```

Çıktı

```
a = 0.30000000000000004
b = 0.29999999999999999
a == b : 0
fark   : 0.00000000000000005551
fabs(a - b) < tolerans : 1
```

`0.1 + 0.2`, `0.3`'e çok yakın ama tam eşit değil; `==` yanlış çıktı. Doğru yöntem, farkın mutlak değerinin çok küçük bir **tolerans**tan (burada `1e-9`: "1 çarpı 10 üzeri eksi 9", yani `0.000000001`) küçük olup olmadığına bakmaktır. `fabs` (mutlak değer) `math.h`'dedir; Linux'ta derlerken `-lm` eklemeyi unutma. Sınavda tolerans olarak genellikle `1e-9` ya da `0.000001` kullanılır.

Sorun asıl `==` ve `!=` ile olur; `<` ve `>` yalnızca iki değerin matematikte tam eşit olması gereken sınır durumunda aynı tuzağa düşürür (`0.1 + 0.2 > 0.3` doğru çıkar). `0.25 + 0.25 == 0.5` gibi ikinin kuvvetlerinden oluşan sayılar tam gösterilebildiği için doğru çıkabilir, bu seni yanıltmasın.

4.13 Örnek uygulamalar

Artık yıl

Bir yıl **4'e bölünüyorsa artık yıldır; ama 100'e de bölünüyorsa artık değildir; 400'e de bölünüyorsa yine artıktır.** Örneğin 2024 artıktır; 1900 artık değildir (100'e bölünür, 400'e bölünmez); 2000 artıktır (400'e bölünür). Kural tek bir koşula çevrilir:

artikYil.c

```c
#include <stdio.h>

int main(void) {
    int yil = 0;

    printf("Yıl: ");
    scanf("%d", &yil);

    if ((yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0) {
        printf("%d artık yıldır.\n", yil);
    } else {
        printf("%d artık yıl değildir.\n", yil);
    }
    return 0;
}
```

Çıktı

```
Yıl: 1900
1900 artık yıl değildir.
```

Üçgen: geçerlilik ve tür

Üç kenar uzunluğu (pozitif tamsayılar) bir üçgen oluşturur **ancak ve ancak** her kenar diğer ikisinin toplamından küçükse (üçgen eşitsizliği). Geçerliyse üç kenar eşitse eşkenar, ikisi eşitse ikizkenar, hiçbiri eşit değilse çeşitkenar olur:

ucgen.c

```c
#include <stdio.h>

int main(void) {
    int a = 0, b = 0, c = 0;

    printf("Üç kenar uzunluğu: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a + b <= c || a + c <= b || b + c <= a) {
        printf("Bu kenarlarla üçgen olmaz.\n");
    } else if (a == b && b == c) {
        printf("Eşkenar üçgen\n");
    } else if (a == b || a == c || b == c) {
        printf("İkizkenar üçgen\n");
    } else {
        printf("Çeşitkenar üçgen\n");
    }
    return 0;
}
```

Çıktı

```
Üç kenar uzunluğu: 3 4 5
Çeşitkenar üçgen
```

Sıra yine önemli: Eşkenar denetimi ikizkenardan **önce** gelir; çünkü eşkenar üçgenin de iki kenarı eşittir ve sıra ters olsaydı hep “ikizkenar” yazılırdı.

Basit hesap makinesi

İki sayıyı ve aralarındaki işlemi okuyan program, işlemi `switch` ile seçer. `scanf`’teki `%c` öncesindeki boşluk, sayı ile işlem arasındaki boşlukları atlar (Bölüm 3):

hesapMakinesi.c

```c
#include <stdio.h>

int main(void) {
    double sayi1 = 0.0, sayi2 = 0.0;
    double sonuc = 0.0;
    char islem = ' ';

    printf("İşlem (ör. 12.5 * 4): ");
    scanf("%lf %c %lf", &sayi1, &islem, &sayi2);

    switch (islem) {
        case '+': sonuc = sayi1 + sayi2; break;
        case '-': sonuc = sayi1 - sayi2; break;
        case '*': sonuc = sayi1 * sayi2; break;
        case '/':
            if (sayi2 == 0) {
                printf("Sıfıra bölünemez!\n");
                return 0;
            }
            sonuc = sayi1 / sayi2;
            break;
        default:
            printf("Bilinmeyen işlem: %c\n", islem);
            return 0;
```

```
    }
    printf("%.2f %c %.2f = %.2f\n", sayi1, islem, sayi2, sonuc);
    return 0;
}
```

Çıktı

```
İşlem (ör. 12.5 * 4): 12.5 * 4
12.50 * 4.00 = 50.00
```

Bölmede önce `sayi2 == 0` denetlenir (kullanıcının yazdığı `0` tam sıfırdır, bu karşılaştırma güvenlidir); `default` kolu bilinmeyen işleme karşı korur. `char` ile `switch` kullanırken etiketlerin tek tırnaklı yazıldığına dikkat et: `'+'`, `"+"` değil.

Bölüm Özeti – Bilmen Gerekenler

- İlişkisel operatörler (`<`, `>`, `<=`, `>=`, `==`, `!=`) sonuç olarak `int` üretir: doğruysa `1`, yanlışsa `0`.
- **Sıfır yanlıştır, sıfır dışı her değer doğrudur** (`-1` ve `0.5` dahil). `stdbool.h` ile `bool`, `true`, `false` kullanılır; C23'te bunlar anahtar sözcüktür.
- `if`, `if-else`, `else if`: koşullar yukarıdan aşağıya denenir, ilk doğru olan çalışır. Gövde tek satır bile olsa `{ }` yaz. `if (x > 5);` boş gövdedir; `else` en yakın `if`'e bağlanır (sarkan else).
- `&&` (VE), `||` (VEYA), `!` (DEĞİL). **Kısa devre:** `&&`'de sol yanlışsa, `||`'de sol doğruysa sağ taraf hiç hesaplanmaz (`++` dahil).
- `=` atama, `==` karşılaştırmadır; `if (x = 5)` hata vermez ama her zaman doğrudur. `0 < x < 10` her zaman doğrudur; doğrusu `x > 0 && x < 10`. `x == 1 || 2` de her zaman doğrudur.
- Öncelik: `( )` → `!` → `* / %` → `+ -` → `< <= > >=` → `== !=` → `&&` → `||` → `?:` → `=`.
- `koşul ? a : b` bir değer üretir; iç içe yazma, `else if` kullan.
- `switch` tamsayı/`char` ifadesini sabit `case`'lerle karşılaştırır. `break` yoksa sonraki case'ler de çalışır (fall-through); `case`'de değişken, aralık, `double` ve string olmaz.
- `double` değerleri `==` ile değil `fabs(a - b) < tolerans` ile karşılaştır.

Sorular

A) Çoktan Seçmeli

1 `x` değişkeninin değerinin 0 ile 100 arasında (uçlar dahil) olduğunu doğru biçimde denetleyen ifade hangisidir?

(A) `0 <= x <= 100`

(B) `x >= 0 || x <= 100`

(C) `x > 0 && x < 100`

(D) `x >= 0 && x <= 100`

(E) `x >= 0 and x <= 100`

2 Aşağıdaki ifadelerden hangisinde `x` değeri `0` iken `10 / x` işlemi **hiçbir zaman çalıştırılmaz**?

(A) `10 / x > 1 && x != 0`
(B) `x != 0 && 10 / x > 1`
(C) `x != 0 || 10 / x > 1`
(D) `x == 0 && 10 / x > 1`
(E) `10 / x > 1 || x != 0`

3 Aşağıdakilerden hangisi `switch` içinde `case` etiketi olarak **kullanılamaz**? (Programda `#define MAKS 10` ve `int sinir = 10;` tanımlıdır.)

(A) `case 3:` (B) `case 'a':` (C) `case 2 + 3:` (D) `case MAKS:` (E) `case sinir:`

4 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int n = 15;

    printf("%d\n", n > 10 ? n > 20 ? 3 : 2 : 1);
    return 0;
}
```

(A) `2`
(B) `3`
(C) `1`
(D) `15`
(E) Derleme hatası verir

5 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int x = 4;

    if (x = 0) { printf("A\n"); } else { printf("B\n"); }
    printf("%d\n", x);
    return 0;
}
```

(A) `A` ve alt satırda `4`
(B) `B` ve alt satırda `4`
(C) `B` ve alt satırda `0`
(D) `A` ve alt satırda `0`
(E) Derleme hatası verir

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int x = 8;
    int y = 1;

    if (x > 3)
        if (y > 5)
            printf("A\n");
    else
        printf("B\n");
    printf("C\n");
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int a = 3;
    int b = 0;
    int c = 0;

    if (a > 5 && ++b > 0) { c = 10; }
    if (a < 5 || ++b > 0) { c += 1; }
    if (!b) { c += 2; }
    if (a) { c += 4; }
    printf("%d %d %d\n", a, b, c);
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int kod = 2;
    int toplam = 0;

    switch (kod) {
        case 1: toplam += 1;
        case 2: toplam += 10;
        case 3: toplam += 100; break;
        case 4: toplam += 1000;
        default: toplam += 5;
    }
    printf("%d\n", toplam);
    return 0;
}
```

C) Kod Yazma

9 Vize ve final notunu (ikisi de 0–100 arası `int`) okuyan `gecmeDurumu` programını yaz. Notlardan biri 0–100 dışındaysa `Geçersiz not!` yazıp bitsin. Aksi halde ortalamayı `vize * 0.4 + final * 0.6` formülüyle hesapla ve iki ondalıkla yazdır. Ortalama en az 50 **ve** final notu en az 45 ise `Geçtin.`, değilse `Kaldın.` yazdır.

```
Vize ve final notu: 90 40
Ortalama: 60.00
Kaldın.
```

10 Ay numarasını (1–12) ve yılı okuyan `gunSayisi` programını yaz. `switch` kullanarak o ayın kaç gün çektiğini `ay/yıl: N gün` biçiminde yazdır. Şubat için bu bölümdeki artık yıl kuralını kullan. Ay geçersizse `Geçersiz ay!` yazdır. Aynı sonucu veren ayları `break` yazmadan gruplayarak kod tekrarını azalt.

```
Ay ve yıl: 2 2024
2/2024: 29 gün
```

Çözümler

Cevap anahtarı

1	2	3	4	5
D	B	E	A	C

1 Cevap: D

Aralığın içinde olmak, iki koşulun **birlikte** sağlanmasıdır: `x >= 0 && x <= 100`. A'da `0 <= x` sonucu `0` ya da `1` olur ve bu sayı `100`'den hep küçüktür; ifade her zaman doğrudur. B her `x` için doğrudur: Hiçbir sayı hem 0'dan küçük hem 100'den büyük olamaz. C uçları (0 ve 100) dışarıda bırakır. E'deki `and` Python'a aittir, C'de operatör değildir.

2 Cevap: B

Kısa devre: `x != 0 && ...` ifadesinde `x` sıfırsa sol taraf yanlıştır ve sağ taraf **hiç hesaplanmaz**. A ve E'de bölme en başta, solda yapılır. C'de sol taraf yanlış olunca `||` sağ tarafa geçer; D'de `x == 0` doğru olunca `&&` sağ tarafı hesaplar. Yani A, C, D ve E'de sıfıra bölme gerçekleşir.

3 Cevap: E

`case` etiketi derleme zamanında bilinen bir sabit olmalıdır. `3` ve `'a'` sabittir (`'a'` 97 değerinde bir tamsayıdır), `2 + 3` sabit bir ifadedir, `MAKS` ise önişlemci tarafından derlemeden önce `10` ile değiştirilir. `sinir` ise bir **değişkendir**; gcc `case label does not reduce to an integer constant` hatası verir.

4 Cevap: A

İç `?:` ortadaki işlenendir: `n > 10 ? (n > 20 ? 3 : 2) : 1`. `15 > 10` doğru → iç ifade: `15 > 20` yanlış → `2`.

Çıktı
```
2
```

B, `n > 20`'nin doğru olduğunu varsayar; C dış koşulun yanlış çıktığını varsayar; D `n`'in kendisidir; ifade geçerli olduğundan E de yanlıştır.

5 Cevap: C

`if (x = 0)` karşılaştırma değil **atamadır**: `x`'e `0` atanır ve ifadenin değeri `0`, yani yanlış olur. `else` çalışır (`B`) ve `x` artık `0`'dır. gcc uyarır ama program derlenir (E yanlış). A ve D'de `if` bloğu çalışmaz; B'de `x`'in değişmediği sanılmıştır.

Çıktı
```
B
0
```

6

Sarkan else: `else`, girintisine rağmen **en yakın `if`'e**, yani `y > 5`'e bağlanır.

1. `x > 3` → `8 > 3` doğru, iç `if`'e girilir.
2. `y > 5` → `1 > 5` yanlış, `A` yazılmaz; iç `if`'in `else`'i çalışır: `B`.

3. `printf("C\n");` `if`'lerin dışında olduğu için her zaman çalışır.

```
Çıktı
B
C
```

7

Kısa devre ve değişken izleme (`a` hiç değişmez):

Adım	Açıklama	b	c
Başlangıç		0	0
1. `if`	`a > 5` yanlış → `++b` **atlanır**, blok çalışmaz	0	0
2. `if`	`a < 5` doğru → `||` kısa devre, `++b` **atlanır**, `c += 1`	0	1
3. `if`	`!b` = `!0` = `1` doğru, `c += 2`	0	3
4. `if`	`a` = 3 sıfır dışı, doğru, `c += 4`	0	7

```
Çıktı
3 0 7
```

8

`kod` 2 olduğundan program `case 2` etiketinin altından başlar:

1. `case 2:` `toplam` 0 → 10. `break` yok, aşağı düşer.
2. `case 3:` `toplam` 10 → 110. `break` var, `switch`'ten çıkılır.
3. `case 4` ve `default` hiç çalışmaz.

```
Çıktı
110
```

9

Önce geçerlilik denetlenir; `||` ile "dışında mı?" sorusu sorulur. Geçerliyse ortalama hesaplanır ve iki koşul `&&` ile bağlanır. Girdi `90 40`: ortalama 60.00, ama final 45'in altında olduğu için `Kaldın.`

gecmeDurumu.c

```c
#include <stdio.h>

int main(void) {
    int vizeNotu = 0, finalNotu = 0;
    double ortalama = 0.0;

    printf("Vize ve final notu: ");
    scanf("%d %d", &vizeNotu, &finalNotu);

    if (vizeNotu < 0 || vizeNotu > 100 || finalNotu < 0 || finalNotu > 100) {
        printf("Geçersiz not!\n");
        return 0;
    }
    ortalama = vizeNotu * 0.4 + finalNotu * 0.6;
    printf("Ortalama: %.2f\n", ortalama);
```

```
    if (ortalama >= 50 && finalNotu >= 45) {
        printf("Geçtin.\n");
    } else {
        printf("Kaldın.\n");
    }
    return 0;
}
```

Çıktı

```
Vize ve final notu: 90 40
Ortalama: 60.00
Kaldın.
```

10

31 ve 30 günlük aylar kendi içinde gruplanır; araya deyim girmeyen etiketler aynı kola düşer. Şubat kolunda artık yıl koşulu `?:` ile seçim yapar, geçersiz ayda `default` mesaj yazıp çıkar.

gunSayisi.c

```c
#include <stdio.h>

int main(void) {
    int ay = 0, yil = 0;
    int gun = 0;

    printf("Ay ve yıl: ");
    scanf("%d %d", &ay, &yil);

    switch (ay) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            gun = 31;
            break;
        case 4: case 6: case 9: case 11:
            gun = 30;
            break;
        case 2:
            gun = ((yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0) ? 29 : 28;
            break;
        default:
            printf("Geçersiz ay!\n");
            return 0;
    }
    printf("%d/%d: %d gün\n", ay, yil, gun);
    return 0;
}
```

Çıktı

```
Ay ve yıl: 2 2024
2/2024: 29 gün
```

BÖLÜM 5

Döngüler

Bu Bölümde Öğreneceklerin

- `while`, `do-while` ve `for` döngülerini ve hangisini ne zaman seçeceğini
- `for`'un üç kısmını, `for` ile `while` dönüşümünü ve döngü değişkeninin kapsamını
- Sayaç, toplam ve çarpım değişkenlerini; ilk değer vermenin neden şart olduğunu
- `break`, `continue`, sonsuz döngüleri ve `for (...);` noktalı virgül tuzağını
- İç içe döngülerle yıldız ve sayı desenlerini
- İzleme tablosunu (trace table), off-by-one hatasını ve "kaç kez döner" sorusunu
- Klasik algoritmaları: faktöriyel, basamaklar, asal sayı, EBOB-EKOK, Fibonacci, mükemmel sayı
- Sentinel, girdi doğrulama, tampon temizleme, girdi bitene kadar okuma ve `double` sayaç tuzağını

5.1 Neden döngü? while

1'den 1000'e kadar sayıları yazdırmak için bin `printf` yazmazsın. **Döngü** (loop), bir kod bloğunu koşul doğru olduğu sürece tekrar tekrar çalıştırır. En yalın döngü `while`'dır:

```
while (koşul) {
    // koşul doğru olduğu sürece tekrarlanan gövde
}
```

Sıra şöyledir: koşul denetlenir; doğruysa gövde bir kez çalışır; başa dönülüp koşul yeniden denetlenir; yanlış olunca döngü biter. Bölüm 4'te gördüğün gibi sıfırdan farklı her değer doğrudur.

whileTemel.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 1;
    while (sayi <= 5) {
        printf("%d ", sayi);
        sayi++;
    }
    printf("\nDöngü bitti, sayi = %d\n", sayi);
    return 0;
}
```

Çıktı

```
1 2 3 4 5
Döngü bitti, sayi = 6
```

Her `while`'da üç parça aranır: **ilk değer** (`sayi = 1`), **koşul** (`sayi <= 5`) ve koşulu sonunda bozacak bir **değişiklik** (`sayi++`). Döngü bitince `sayi` 5 değil 6'dır: döngü, koşulu bozan ilk değerde durur.

`sayi++;` satırını unutursan koşul hep doğru kalır ve program sonsuza kadar `1 1 1 ...` yazar (terminalde **Ctrl+C** ile durdurulur). Her döngüde kendine sor: "Bu döngü hangi satırda biter?"

5.2 do-while

`while`'da koşul gövdeden **önce** denetlenir; baştan yanlışsa gövde hiç çalışmaz. `do-while`'da koşul gövdeden **sonra** denetlenir, yani gövde **en az bir kez** çalışır:

```
do {
    // en az bir kez çalışan gövde
} while (koşul);       // ← bu noktalı virgül ZORUNLU
```

Koşul baştan yanlışsa `while` gövdeyi 0 kez, `do-while` 1 kez çalıştırır. Klasik kullanım yeri **menü**dür: menü en az bir kez gösterilmeli, seçime göre tekrarlanmalıdır.

menu.c

```c
#include <stdio.h>

int main(void) {
    int secim = 0;
    do {
        printf("1) Merhaba  0) Çıkış: ");
        scanf("%d", &secim);
        if (secim == 1) {
            printf("Merhaba!\n");
        }
    } while (secim != 0);
    printf("Program bitti.\n");
    return 0;
}
```

Çıktı

```
1) Merhaba  0) Çıkış: 1
Merhaba!
1) Merhaba  0) Çıkış: 1
Merhaba!
1) Merhaba  0) Çıkış: 0
Program bitti.
```

Mavi satırlar senin klavyeden yazdıklarındır. Seçim gövdenin içinde okunduğu için `do-while` burada doğal seçimdir. Seçim kuralı: kaç kez döneceği belliyse `for`; koşula bağlıysa ve hiç dönmeyebilirse `while`; en az bir kez çalışmalıysa (menü, girdi doğrulama) `do-while`.

> **Sınavda Çıkar**
>
> "Bu kod kaç kez çalışır?" sorularında önce döngü türüne bak. Koşul baştan yanlışsa `while` ve `for` **0 kez**, `do-while` **1 kez** çalışır. `do-while`'ın sonundaki `;` unutulursa derleme hatası olur; "kodda hata var mı?" sorusunun cevabı çoğu kez budur.

5.3 for döngüsü

Sayaçlı işlerde ilk değer, koşul ve değişikliği tek satıra toplayan `for` kullanılır:

```
for (başlangıç; koşul; adım) {
    // gövde
}
```

Sıra: (1) **başlangıç** yalnız bir kez, döngüye girerken çalışır; (2) **koşul** denetlenir, yanlışsa döngü biter; (3) **gövde** çalışır; (4) **adım** çalışır ve 2'ye dönülür.

forTemel.c

```c
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
    printf("\n");
    for (int i = 10; i >= 0; i -= 5) {
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
1 2 3 4 5
10 5 0
```

Adım `i++` ile sınırlı değildir: `i--` geriye sayar, `i += 5` beşer atlar, `i *= 2` ikiye katlar (1, 2, 4, 8, ...). Virgülle birden fazla değişken de yönetilebilir: `for (int a = 0, b = 10; a < b; a++, b--)`.

for ile while dönüşümü

Her `for` bir `while`'a çevrilebilir: başlangıç döngünün **önüne**, adım gövdenin **sonuna** gider. `for (int i = 1; i <= 3; i++) { gövde }` ile `int i = 1; while (i <= 3) { gövde i++; }` aynı işi yapar.

Döngü değişkeninin kapsamı

C99'dan beri başlangıçta değişken tanımlayabilirsin: `for (int i = 0; ...)`. Bu `i` yalnızca bu `for`'un içinde yaşar (kapsamı, *scope*, döngüyle sınırlıdır). Örneğin döngüden sonra `printf("%d", i);` yazarsan gcc `'i' undeclared (first use in this function)` hatası verir. Değer döngüden sonra lazımsa değişkeni **dışarıda** tanımla: `int i; for (i = 0; ...)`. Ardışık iki `for` aynı `i` adını kullanabilir; her biri kendi `i`'sini yaratır (`forTemel`'de gördün).

5.4 Sayaç, toplam ve çarpım

Döngülerde üç tür değişken sürekli karşına çıkar:

Rol	Görevi	İlk değer	Döngüde
Sayaç (counter)	Bir şeyin kaç kez olduğunu sayar	`0`	`sayac++;`
Toplam (sum)	Değerleri biriktirir	`0`	`toplam += x;`
Çarpım (product)	Değerleri çarpar	`1`	`carpim *= x;`

İlk değerler rastgele değildir: `0` toplamanın, `1` çarpmanın etkisiz elemanıdır. Çarpımı `0` ile başlatırsan sonuç hep `0` çıkar. Program, `n` için 1'den `n`'e **toplamı**, `n!` çarpımını ve çift sayıların sayısını bulur:

roller.c

```c
#include <stdio.h>

int main(void) {
    int n = 0;
    printf("n = ");
    scanf("%d", &n);

    int toplam = 0;
    int carpim = 1;
    int sayac = 0;
    for (int i = 1; i <= n; i++) {
        toplam += i;
        carpim *= i;
        if (i % 2 == 0) {
            sayac++;
        }
    }
    printf("1'den %d'e toplam: %d\n", n, toplam);
    printf("Çarpım (%d!): %d\n", n, carpim);
    printf("Çift sayı adedi: %d\n", sayac);
    return 0;
}
```

Çıktı

```
n = 5
1'den 5'e toplam: 15
Çarpım (5!): 120
Çift sayı adedi: 2
```

Toplam için kapalı formül de vardır: `n * (n + 1) / 2` (n = 5 için 15). Çarpım `n` büyüyünce `int`'e sığmaz; bunu faktöriyelde göreceksin.

> **Sık Yapılan Hata: Toplam değişkenine ilk değer vermemek**
>
> `int toplam;` yazıp ilk değer vermeden döngüde `toplam += i;` dersen toplam çöp değere eklenir. İlk değer verilmemiş yerel değişkenin içinde bellekten kalma rastgele bir sayı (**çöp değer**) vardır; sonuç `15 + çöp` olur. Daha kötüsü, çöp bazen tesadüfen `0` çıkar: kod senin bilgisayarında doğru çalışır, sınavda yanlış sonuç verir. C bunu tanımsız davranış saydığı için çıktısını göstermiyoruz. Kural: sayaç, toplam ve çarpım **tanımlandıkları satırda** ilk değerlerini alır.

5.5 break ve continue

`break`, içinde bulunduğu döngüyü **tamamen** bitirir. `continue` gövdenin **kalanını atlar** ve bir sonraki tura geçer.

breakContinue.c

```c
#include <stdio.h>

int main(void) {
    for (int i = 50; i <= 100; i++) {
        if (i % 7 == 0) {
            printf("İlk 7'ye bölünen: %d\n", i);
            break;
        }
    }
    for (int i = 1; i <= 10; i++) {
        if (i % 3 == 0) {
            continue;
        }
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
İlk 7'ye bölünen: 56
1 2 4 5 7 8 10
```

İlk döngü 56'da bitti (57..100 hiç denenmedi); ikinci döngü 3, 6 ve 9'da `printf`'i atladı. İki kural: (1) `break` yalnızca **en içteki** döngüyü bitirir; (2) `for`'da `continue` yine de **adım kısmını** çalıştırır, `while`'da ise adım gövdede olduğu için atlanabilir:

```c
int i = 1;
while (i <= 5) {
    if (i == 3) {
        continue;       // i++ atlanır, i hep 3 kalır: SONSUZ DÖNGÜ
    }
    i++;
}
```

Sonsuz döngü ve break ile çıkış

Koşulu hep doğru olan döngüye **sonsuz döngü** denir: `while (1)` ya da boş koşullu `for (;;)`. Hata değildir; çıkışı `break` ile yapılır ve kaç tur döneceği önceden bilinmeyen işlerde kullanılır:

sonsuzDongu.c

```c
#include <stdio.h>

int main(void) {
    int kuvvet = 1;
    while (1) {
        kuvvet *= 2;
        if (kuvvet > 1000) {
            break;
        }
    }
    printf("1000'i ilk geçen 2 kuvveti: %d\n", kuvvet);
    return 0;
}
```

Çıktı
```
1000'i ilk geçen 2 kuvveti: 1024
```

`break` olmasaydı bu döngü sonsuza kadar dönerdi. Sonsuz döngü yazdığında `break` yolunun gerçekten çalışacağından emin ol.

5.6 Noktalı virgül tuzağı

`for (...)` ya da `while (...)` satırının sonuna yanlışlıkla `;` koyarsan C bunu **boş bir gövde** sayar. Altındaki satır döngüden bağımsız, sıradan bir satır olur:

noktaliVirgul.c
```c
#include <stdio.h>

int main(void) {
    int i = 0;
    for (i = 1; i <= 3; i++);
        printf("Merhaba i=%d\n", i);
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra
```
noktaliVirgul.c: In function 'main':
noktaliVirgul.c:5:5: warning: this 'for' clause does not guard... [-Wmisleading-indentation]
    5 |     for (i = 1; i <= 3; i++);
      |     ^~~
noktaliVirgul.c:6:9: note: ...this statement, but the latter is misleadingly indented as if it were
guarded by the 'for'
    6 |         printf("Merhaba i=%d\n", i);
      |         ^~~~~~
```

Girinti insanı kandırır, derleyiciyi kandırmaz. Döngü boş gövdeyle 3 tur döndü (`i` 4 oldu), `printf` ise **bir kez** çalıştı:

Çıktı
```
Merhaba i=4
```

`while`'daki karşılığı daha tehlikelidir, çünkü sonsuz döngüye yol açar (çalıştırmıyoruz):

```c
int i = 1;
while (i <= 3);          // boş gövde: i hiç değişmez → sonsuz döngü
    i++;                 // buraya hiç gelinmez
```

> **Sık Yapılan Hata: Noktalı virgül nerede olur, nerede olmaz?**
>
> `for (...)` ve `while (...)` başlığından sonra `;` **konmaz**; `do-while`'ın sonunda ise `} while (...);` diye **konur**. Karıştırırsan bir döngü "bir kez çalışıyor" ya da "hiç ilerlemiyor" diye şaşırtır. (Bir istisna: aşağıda **bilerek** boş gövdeli bir `while` göreceksin.)

5.7 İç içe döngüler ve desenler

Bir döngünün gövdesine başka bir döngü yazabilirsin. Kural: dış döngünün **her turunda** iç döngü baştan sona çalışır. İç döngü bir **satırı**, dış döngü satırların **sayısını** üretir; satır sonundaki `printf("\n")` iç döngünün dışında, dış döngünün içindedir. `i` satır numarasıysa dik üçgende `i` yıldız vardır.

Ters üçgende yalnız dış döngü geriye sayar. Sayı üçgeninde `*` yerine iç sayaç `j` yazdırılır. **Piramit**te satırda boşluk da vardır; desen sorularında önce her satır için "kaç boşluk, kaç yıldız" tablosunu kâğıda çiz. `n = 4` için: satır 1 → 3 boşluk, 1 yıldız; satır 2 → 2 boşluk, 3 yıldız; satır 3 → 1 boşluk, 5 yıldız; satır 4 → 0 boşluk, 7 yıldız. Yani satır `i`'de `n - i` boşluk ve `2i - 1` yıldız vardır. Dört deseni tek programda görelim:

desenler.c

```c
#include <stdio.h>

int main(void) {
    int n = 4;
    printf("Dik üçgen:\n");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    printf("Ters üçgen:\n");
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf("*");
        }
        printf("\n");
    }
    printf("Sayı üçgeni:\n");
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d", j);
        }
        printf("\n");
    }
    printf("Piramit:\n");
    for (int i = 1; i <= n; i++) {
        for (int bosluk = 1; bosluk <= n - i; bosluk++) {
            printf(" ");
        }
        for (int yildiz = 1; yildiz <= 2 * i - 1; yildiz++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

Çıktı

```
Dik üçgen:
*
**
***
****
Ters üçgen:
****
***
**
*
Sayı üçgeni:
1
12
123
1234
Piramit:
   *
  ***
 *****
*******
```

Piramitte her satırda **iki ayrı iç döngü** art arda çalışır: önce boşluklar, sonra yıldızlar. Sınırları kâğıtta çıkardıysan döngüler kendiliğinden yazılır.

> **Sınavda Çıkar**
>
> "Bu kod kaç `*` yazdırır?" sorusunda iç gövdenin kaç kez çalıştığını say. İç sınır dış sayaca bağlıysa (`j <= i`) çarpma, tur başına değişen sayıları **topla**: 1 + 2 + 3 + 4 = 10. İç sınır sabitse (`j <= 4`) çarp: 3 dış tur × 4 = 12. İç döngüdeki `break` yalnızca iç döngüyü bitirir, dış döngü sürer.

5.8 İzleme tablosu ve off-by-one

Çıktı sorusunun en güvenilir çözümü **izleme tablosu**dur (trace table): her değişken için bir sütun açılır, her turdan sonra yeni değerler yazılır.

izleme.c

```c
#include <stdio.h>

int main(void) {
    int a = 1;
    int b = 10;
    int tur = 0;
    while (a < b) {
        a += 2;
        b -= 1;
        tur++;
    }
    printf("a=%d b=%d tur=%d\n", a, b, tur);
    return 0;
}
```

Tur	Koşul `a < b`	`a`	`b`	`tur`
başlangıç		1	10	0
1	1 < 10 doğru	3	9	1
2	3 < 9 doğru	5	8	2
3	5 < 8 doğru	7	7	3
bitiş	7 < 7 yanlış			

Çıktı
```
a=7 b=7 tur=3
```

> **İpucu: İzleme tablosunun kuralları**
>
> (1) İlk değerleri yaz. (2) Her tur başında **koşulu** yaz, doğru mu yanlış mı. (3) Gövdeyi sırayla uygulayıp değişkenleri güncelle. (4) Son satır koşulun yanlış çıktığı satırdır; döngü sonrası `printf`'i o satırdaki değerlerle çöz.

Off-by-one ("bir eksik/fazla") hatası, döngünün bir tur eksik ya da fazla dönmesidir; `<` ile `<=` karışıklığından doğar. Tur sayısı:

Döngü başlığı	Tur sayısı
`for (i = a; i < b; i++)`	`b - a` (örnek: `i = 0; i < 5` → 5 tur)
`for (i = a; i <= b; i++)`	`b - a + 1` (örnek: `i = 0; i <= 5` → 6 tur)
`for (i = 0; i < 10; i += 3)`	`i` = 0, 3, 6, 9 → 4 tur (adım k ise say)

1'den `n`'e toplarken `i <= n` yerine `i < n` yazarsan `n` toplama girmez: `n = 5` için 15 yerine 10 bulursun. Döngü sonrası değer de buradan çıkar: `for (i = 0; i < 5; i++)` biterken `i` **5**'tir; `break` ile bitmiş döngüde `i`, `break`'in çalıştığı değerde kalır.

5.9 Klasik algoritmalar

Bunların hepsi sınavda kod yazma sorusu olarak çıkar. Bölüm 6'da fonksiyonlara dönüştüreceksin; şimdilik hepsi `main` içindedir. (1'den `n`'e toplam için `roller` programına bak.)

Faktöriyel ve int taşması

`n! = 1 · 2 · ... · n` ve `0! = 1` olduğundan çarpım `1` ile başlar. Faktöriyel hızla büyür: `12! = 479001600` bir `int`'e (en fazla 2147483647) sığar, `13!` sığmaz. İşaretli tamsayı taşması tanımsız davranıştır; bu yüzden `long long` kullanılır ve `%lld` ile yazdırılır. `long long`'a `20!` sığar, `21!` sığmaz.

faktoriyel.c

```c
#include <stdio.h>

int main(void) {
    long long f = 1;
    for (int n = 1; n <= 20; n++) {
        f *= n;
        if (n % 5 == 0 || n == 12 || n == 13) {
            printf("%2d! = %lld\n", n, f);
        }
    }
    return 0;
}
```

Çıktı

```
 5! = 120
10! = 3628800
12! = 479001600
13! = 6227020800
15! = 1307674368000
20! = 2432902008176640000
```

`13!`'in `long long`'da doğru göründüğüne dikkat et; `int` ile yapsaydın sonuç yanlış (ve tanımsız) olurdu.

Basamak sayısı, toplamı ve ters çevirme

Anahtar fikir: `n % 10` **son basamağı** verir, `n / 10` son basamağı **atar**. Sayı 0 olana kadar bu iki işlem tekrarlanır:

basamaklar.c

```c
#include <stdio.h>

int main(void) {
    int n = 0;
    printf("Pozitif bir sayı: ");
    scanf("%d", &n);

    int kalan = n;
    int basamakSayisi = 0;
    int basamakToplami = 0;
    int ters = 0;
    while (kalan > 0) {
        int son = kalan % 10;
        basamakSayisi++;
        basamakToplami += son;
        ters = ters * 10 + son;
        kalan /= 10;
    }
    printf("Basamak sayısı: %d\n", basamakSayisi);
    printf("Basamak toplamı: %d\n", basamakToplami);
    printf("Ters: %d\n", ters);
    return 0;
}
```

Çıktı

```
Pozitif bir sayı: 4712
Basamak sayısı: 4
Basamak toplamı: 14
Ters: 2174
```

`ters` değişkeni tur tur şöyle büyür: 2, 21, 217, 2174. Üç nokta: (1) `n` bozulmasın diye kopyası `kalan` üzerinde çalışıldı. (2) `n` sıfırsa `while` hiç dönmez ve basamak sayısı `0` çıkar, oysa `0` tek basamaklıdır; bunu `do-while` çözer. (3) Sondaki sıfırlar tersi kısaltır (`1200` → `21`); on basamaklı sayıların tersi `int`'e sığmayabilir.

Asal sayı testi

Asal sayı, 1 ve kendisinden başka böleni olmayan 1'den büyük sayıdır. Bölen aramak için `n`'e kadar gitmek gerekmez: `n = a · b` ise küçük çarpan `√n`'i geçemez, yani `i * i <= n` yeter. Bir **bayrak** (flag) değişkeni "bölen bulundu mu?" bilgisini taşır. Program 2'den `n`'e kadar asalları listeler:

asalListe.c

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int n = 0;
    printf("Üst sınır: ");
    scanf("%d", &n);

    for (int sayi = 2; sayi <= n; sayi++) {
        bool asal = true;
        for (int i = 2; i * i <= sayi; i++) {
            if (sayi % i == 0) {
                asal = false;
                break;
            }
        }
        if (asal) {
            printf("%d ", sayi);
        }
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
Üst sınır: 50
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

`break` iç döngüden çıkar: bölen bulununca o sayı için aramanın anlamı yok. `bool asal = true;` satırı **dış döngünün içinde** olmalıdır; dışarıya alırsan bir kez `false` olan bayrak bir daha `true` olmaz. (`i * i`, `int` sınırına yakın sayılarda taşabilir; kaygın varsa koşulu `i <= sayi / i` yaz.)

EBOB ve EKOK

Öklid algoritması: `a` ile `b`'nin EBOB'u (en büyük ortak bölen), `b` ile `a % b`'nin EBOB'u ile aynıdır; `b` sıfır olunca cevap `a`'dır. **EKOK**, EBOB'dan çıkar: `EKOK = a / EBOB * b` (bölmeyi önce yapmak `a * b` çarpımının taşma riskini azaltır).

ebobEkok.c

```c
#include <stdio.h>

int main(void) {
    int a = 0, b = 0;
    printf("İki sayı: ");
    scanf("%d %d", &a, &b);

    int x = a;
    int y = b;
    while (y != 0) {
        int kalan = x % y;
        x = y;
        y = kalan;
    }
    printf("EBOB = %d\n", x);
    printf("EKOK = %d\n", a / x * b);
    return 0;
}
```

Çıktı

```
İki sayı: 48 18
EBOB = 6
EKOK = 144
```

`(x, y)` çifti şöyle değişir: (48, 18) → (18, 12) → (12, 6) → (6, 0). `y` sıfır olunca döngü biter, EBOB `x`'te kalır.

Fibonacci

Fibonacci dizisi `0, 1, 1, 2, 3, 5, ...` her terimi önceki iki terimin toplamı olan dizidir. `onceki`, `simdiki` ve geçici `sonraki` değişkenleri "kayar"; atama sırası bozulursa dizi bozulur:

fibonacci.c

```c
#include <stdio.h>

int main(void) {
    long long onceki = 0;
    long long simdiki = 1;
    for (int i = 1; i <= 10; i++) {
        printf("%lld ", onceki);
        long long sonraki = onceki + simdiki;
        onceki = simdiki;
        simdiki = sonraki;
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
0 1 1 2 3 5 8 13 21 34
```

Mükemmel sayı

Kendisi hariç bölenlerinin toplamı kendisine eşit olan sayıya **mükemmel sayı** denir: `6 = 1 + 2 + 3`. Kendisi dışındaki en büyük bölen en fazla `n / 2` olduğundan tarama orada biter. Bölen toplamı her sayı için **sıfırdan** başlar:

mukemmel.c

```c
#include <stdio.h>

int main(void) {
    for (int n = 2; n <= 10000; n++) {
        int bolenToplami = 0;
        for (int i = 1; i <= n / 2; i++) {
            if (n % i == 0) {
                bolenToplami += i;
            }
        }
        if (bolenToplami == n) {
            printf("%d ", n);
        }
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
6 28 496 8128
```

Toplamı dış döngünün **dışında** sıfırlarsan bölenler sayıdan sayıya birikir ve hiçbir sayı mükemmel çıkmaz. Kural: her turda yeniden başlaması gereken sayaç, toplam ve bayraklar o turun içinde ilk değerlerini alır.

5.10 Kullanıcı girdisiyle döngüler

Sentinel: -1 girilene kadar

Kaç değer gireceği baştan belli değilse, girdinin bittiğini bildiren özel bir değer kararlaştırılır: **sentinel** (nöbetçi değer). Notlar 0-100 arasındaysa `-1` hiçbir not olamaz, bitiş işareti olabilir. Kalıp: değeri döngüden **önce** oku, döngü sonunda **yeniden** oku:

ortalama.c

```c
#include <stdio.h>

int main(void) {
    int notu = 0;
    int toplam = 0;
    int adet = 0;
    printf("Notları gir (bitirmek için -1): ");
    scanf("%d", &notu);
    while (notu != -1) {
        toplam += notu;
        adet++;
        scanf("%d", &notu);
    }
    if (adet == 0) {
        printf("Hiç not girilmedi.\n");
    } else {
        printf("%d not, ortalama: %.2f\n", adet, toplam / (double) adet);
    }
    return 0;
}
```

```
Çıktı
Notları gir (bitirmek için -1): 70 85 90 -1
3 not, ortalama: 81.67
```

`(double) adet`, `adet`'i geçici olarak `double`'a çevirir; böylece bölme tamsayı bölmesi olmaz (245 / 3 = 81 değil, 81.67). `-1` ortalamaya katılmaz, çünkü veri değil işarettir. Hiç not girilmezse `adet` sıfır kalır; bölmeyi `if` ile engelliyoruz.

Girdi doğrulama: do-while ve tampon temizleme

Kullanıcı yanlış girerse programın tekrar sorması gerekir; "en az bir kez sor" olduğu için `do-while` uygundur. Burada Bölüm 3'te "Bölüm 5'te anlayacaksın" diye bıraktığımız kalıp devreye girer. Klavye girdisi önce bir **tampona** (buffer) gider, `scanf` oradan okur. `scanf("%d", ...)` harf görürse okuyamaz (`0` döndürür) ve harfleri tamponda **bırakır**. Aynı `scanf`'i tekrar çağırırsan yine aynı harfleri görürsün: sonsuz döngü. Çözüm, tamponu satır sonuna kadar boşaltmaktır:

```c
int c;
while ((c = getchar()) != '\n' && c != EOF) { }
```

`getchar()` tamponun başındaki **bir** karakteri okuyup tüketir; döngü `'\n'` görene kadar bunu tekrarlar, yani bozuk satır silinir. Gövde **bilerek** boştur: noktalı virgül tuzağının meşru hali (okunsun diye `{ }` yazdık; Bölüm 3'teki `while (getchar() != '\n');` ile aynı iştir). `EOF` kontrolü neden var? Girdi tamamen bittiyse `getchar()` `'\n'` yerine `EOF` döndürür ve kısa kalıp sonsuza kadar döner. `c`'nin `char` değil `int` olması da bundandır: `EOF` negatif bir sayıdır.

dogrulama.c

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    int notu = 0;
    bool gecerli = false;
    do {
        printf("Notunu gir (0-100): ");
        if (scanf("%d", &notu) != 1) {
            printf("Sayı girmedin!\n");
            int c = 0;
            while ((c = getchar()) != '\n' && c != EOF) { }
        } else if (notu < 0 || notu > 100) {
            printf("Not 0-100 arasında olmalı!\n");
        } else {
            gecerli = true;
        }
    } while (!gecerli);
    printf("Kabul edildi: %d\n", notu);
    return 0;
}
```

```
Çıktı
Notunu gir (0–100): abc
Sayı girmedin!
Notunu gir (0–100): -5
Not 0–100 arasında olmalı!
Notunu gir (0–100): 150
Not 0–100 arasında olmalı!
Notunu gir (0–100): 85
Kabul edildi: 85
```

`abc` için `scanf` `0` döndürdü ve tampon temizlendi; `-5` ve `150` sayıdır ama aralık dışıdır; `85` kabul edilince `gecerli` doğru oldu ve `do–while` bitti. (Girdi dosyası tümüyle biterse `scanf` `EOF` döndürür ve bu döngü sonsuza kadar sorar; gerçek programlarda önce `EOF` denetlenir.)

Girdi bitene kadar oku: while (scanf(...) == 1)

`scanf` başarıyla okuduğu değer sayısını döndürür (Bölüm 3). Tek `%d` okuyorsan döngü koşulunu doğrudan `scanf`'e bağlayabilirsin:

eofOku.c

```c
#include <stdio.h>

int main(void) {
    int x = 0;
    int adet = 0;
    int toplam = 0;
    while (scanf("%d", &x) == 1) {
        toplam += x;
        adet++;
    }
    printf("%d sayı okundu, toplam %d\n", adet, toplam);
    return 0;
}
```

```
Çıktı
4 8 15
16 23 42
6 sayı okundu, toplam 108
```

Döngü, `scanf` bir sayı okuyamayınca biter: girdi tükenince (**EOF**, *end of file*) ya da sayı yerine harf gelince. Klavyeden çalıştırırken girdiyi bitirmek için Windows'ta **Ctrl+Z** ve Enter, Linux ve macOS'ta satır başında **Ctrl+D** basılır.

> **Sık Yapılan Hata:** `while (scanf(...) != EOF)`
>
> `!= EOF` yazarsan harf girildiğinde sonsuz döngüye girersin: `scanf` harfi okuyamayınca `0` döndürür, `0` ise `EOF`'tan (-1) farklıdır; koşul doğru kalır ve harf tamponda durduğu için `scanf` her turda yine `0` döndürür. Doğru kalıp, **istediğin değer sayısıyla** karşılaştırmaktır: `== 1` (iki sayı okuyorsan `== 2`).

5.11 double sayaçla döngü tuzağı

Ondalıklı sayılar bellekte ikilik tabanda **yaklaşık** tutulur; `0.1` tam yazılamaz. Bu yüzden `0.1` adımla ilerleyen döngülerde küçük hatalar birikir:

ondalikDongu.c

```c
#include <stdio.h>

int main(void) {
    double toplam = 0.0;
    for (int i = 0; i < 10; i++) {
        toplam += 0.1;
    }
    printf("10 kez 0.1 eklendi: %.17f\n", toplam);
    printf("toplam == 1.0 mı? %d\n", toplam == 1.0);

    int adet = 0;
    for (double x = 0.0; x < 1.0; x += 0.1) {
        adet++;
    }
    printf("x < 1.0 döngüsü %d kez döndü\n", adet);
    return 0;
}
```

Çıktı

```
10 kez 0.1 eklendi: 0.99999999999999989
toplam == 1.0 mı? 0
x < 1.0 döngüsü 11 kez döndü
```

On kez `0.1` eklemek tam `1.0` etmedi (`0.99999999999999989`), bu yüzden `x < 1.0` döngüsü beklenen 10 yerine **11** kez döndü. Daha kötüsü, `for (double x = 0.0; x != 1.0; x += 0.1)` yazarsan `x` hiçbir zaman tam `1.0` olmaz ve döngü **sonsuza** kadar sürer (çalıştırmıyoruz). `float` da çare değildir; hatası `double`'dan daha büyüktür.

Kural: Döngü sayacı **hep tamsayı** olsun. Ondalık değer lazımsa tamsayı sayaçtan hesapla: `for (int i = 0; i <= 10; i++)` içinde `double x = i / 10.0;` gibi. Ondalık sayıları `==` ile karşılaştırma da güvenilmezdir.

Bölüm Özeti – Bilmen Gerekenler

- `while` koşulu önce denetler (**0 kez** de dönebilir); `do { } while (koşul);` sonra denetler (**en az 1 kez**) ve sonunda `;` ister; `for (başlangıç; koşul; adım)` sayaçlı işlerde kullanılır.
- `for`'da sıra başlangıç (bir kez) → koşul → gövde → adım şeklindedir. `for (int i = ...)` ile tanımlanan `i` yalnız döngüde yaşar. `for`, başlangıcı önüne, adımı gövde sonuna alınarak `while`'a çevrilir.
- Sayaç ve toplam `0` ile, çarpım `1` ile başlar; ilk değersiz değişkenin içinde **çöp** vardır (tanımsız davranış).
- `break` en içteki döngüyü bitirir, `continue` tur sonuna atlar (`while`'da adımı kendin ilerlet). `while (1)` ve `for (;;)` sonsuz döngüdür, `break` ile çıkılır.
- `for (...);` ve `while (...);` boş gövde demektir: `for` tek yazdırır gibi görünür, `while` sonsuz döngü olur. İç içe döngüde dış her turda iç döngü baştan çalışır; iç sınır dış sayaca bağlıysa tur sayıları **toplanır**, sabitse çarpılır.
- İzleme tablosu çıktı sorusunun anahtarıdır. `i < b` → `b - a` tur, `i <= b` → `b - a + 1` tur. Döngü bitince sayaç, koşulu bozan ilk değerdedir.
- Algoritmalar: basamak için `% 10` ve `/ 10`; asal için `i * i <= n` + bayrak; EBOB için Öklid; `EKOK = a / ebob * b`. Büyük faktöriyel ve Fibonacci için `long long` ve `%lld` (`13!` `int`'e sığmaz, `20!` `long long`'a sığar).
- Sentinel döngüsünde değeri döngüden önce ve tur sonunda oku. Tamponu `while ((c = getchar()) != '\n' && c != EOF) { }` ile boşalt. Girdi bitene kadar `while (scanf("%d", &x) == 1)` (EOF: Windows'ta Ctrl+Z, Linux/macOS'ta Ctrl+D); `!= EOF` kullanma.

- `double` sayaçla döngü kurma: `0.1` adım birikerek şaşar. Sayacı tamsayı yap.

Sorular

A) Çoktan Seçmeli

1 `for (int i = 1; i <= 10; i += 3)` döngüsünün gövdesi kaç kez çalışır?

(A) 3 (B) 10 (C) 4 (D) 5 (E) 9

2 Döngülerle ilgili aşağıdaki ifadelerden hangisi **doğrudur**?

(A) `while` döngüsünde koşul, gövde bir kez çalıştıktan sonra denetlenir.

(B) `break`, yalnızca döngünün o turunun kalanını atlar ve bir sonraki tura geçer.

(C) `continue`, içinde bulunduğu döngüyü tamamen bitirir.

(D) `for (int i = 0; i < 3; i++)` ile tanımlanan `i`, döngü bittikten sonra da kullanılabilir.

(E) `do-while` döngüsünün gövdesi, koşul baştan yanlış olsa bile en az bir kez çalışır.

3 Aşağıdaki iç içe döngüde `say++` satırı toplam kaç kez çalışır?

```c
for (int i = 0; i < 3; i++) {
    for (int j = 0; j <= i; j++) {
        say++;
    }
}
```

(A) 3 (B) 6 (C) 9 (D) 4 (E) 12

4 `while (scanf("%d", &x) == 1) { ... }` döngüsü ne zaman biter?

(A) Yalnızca kullanıcı 0 girdiğinde.

(B) Yalnızca kullanıcı Enter'a bastığında.

(C) Hiçbir zaman; sonsuz döngüdür.

(D) Girdi bittiğinde (Ctrl+D ya da Ctrl+Z) veya sayı yerine sayı olmayan bir şey girildiğinde.

(E) Yalnızca negatif sayı girildiğinde.

5 `long long` yaklaşık 9.2 · 10^18'e kadar tamsayı tutar. Faktöriyeli `long long` ile hesaplayan bir döngü, `n!` değerini **en fazla** hangi `n` için doğru verir?

(A) 20 (B) 12 (C) 13 (D) 25 (E) 30

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```c
int toplam = 0;
for (int i = 1; i <= 6; i++) {
    if (i % 2 == 0) {
        continue;
    }
    if (i == 5) {
        break;
    }
    toplam += i;
    printf("[%d]", toplam);
}
printf("\nSon: %d\n", toplam);
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
int sayac = 0;
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (i == j) {
            break;
        }
        sayac++;
        printf("(%d,%d)", i, j);
    }
}
printf("\nsayac = %d\n", sayac);
```

8 Aşağıdaki program ekrana ne yazdırır? Satır sayısına dikkat et.

```c
int i = 10;
int n = 0;
do {
    n++;
    i += 5;
} while (i < 10);
printf("%d %d\n", i, n);

int k = 0;
for (k = 0; k < 5; k++);
printf("%d\n", k);
```

C) Kod Yazma

9 Kullanıcıdan bir `n` değeri okuyup **ters sayı üçgeni** yazdıran `tersUcgen` programını yaz: her satır `n`'den başlayıp azalır ve her satırda bir sayı eksik yazılır. Örnek (`n = 5`):

```
54321
5432
543
54
5
```

10 Girdi bitene kadar pozitif tamsayılar okuyan ve her sayının **palindrom** (tersiyle aynı) olup olmadığını yazdıran `palindrom` programını yaz. Örnek girdi (her biri ayrı satırda): `12321`, `1234`, `7`, `1001`, `120`. Beklenen çıktı:

```
12321 palindromdur
1234 palindrom değildir
7 palindromdur
1001 palindromdur
120 palindrom değildir
```

Çözümler

Cevap anahtarı

1 C	2 E	3 B	4 D	5 A

1 Cevap: C

`i` değerleri 1, 4, 7, 10 olur (her tur 3 artar); sonraki değer 13, koşulu bozar. Dört değer olduğundan gövde **4** kez çalışır. A, `10` değerini unutmaktan (`<=` yerine `<`), B ve E adımı 1 sanmaktan çıkar; D rastgele bir tahmindir.

2 Cevap: E

`do-while` koşulu gövdeden **sonra** denetler; gövde en az bir kez çalışır. A, bu özelliği `while`'a yakıştırıyor; oysa `while` koşulu önce denetler. B, `continue`'nun tanımıdır; C ise `break`'in tanımıdır. D yanlış: `for` içinde tanımlanan değişken döngünün kapsamındadır, dışarıda kullanılırsa derleme hatası olur.

3 Cevap: B

`i = 0, 1, 2` için iç döngü sırasıyla 1, 2, 3 kez döner (`j <= i`). İç sınır dış sayaca bağlı olduğundan **toplanır**: 1 + 2 + 3 = **6**. C (3 × 3) iç sınırı sabit sanmaktır; A, `j <= i` yerine `j < i` okumaktan çıkar (0 + 1 + 2); E ise 3 × 4'tür; D rastgele bir tahmindir.

4 Cevap: D

`scanf`, başarıyla okuduğu değer sayısını döndürür. Girdi tükenirse (`EOF`) ya da `%d` için sayı yerine harf gelirse `1` döndüremez ve döngü biter. A, B ve E koşulla ilgisiz birer sentinel varsayar; `0`, Enter ya da negatif sayı bu koşulu bozmaz. C yanlış: koşul yanlış olabilir.

5 Cevap: A

`20! = 2432902008176640000`, `long long` sınırının (yaklaşık 9.22 · 10^18) altındadır; `21! ≈ 5.1 · 10^19` ise üstündedir. En büyük doğru `n` **20**'dir (bkz. `faktoriyel` programı). B (12) `int` için geçerlidir, C (13) `int`'in ilk taştığı değerdir; D ve E sınırı aşar.

6

İzleme: `i = 1` tek, 5 değil: `toplam = 1`, `[1]`. `i = 2` çift: `continue`. `i = 3`: `toplam = 4`, `[4]`. `i = 4`: `continue`. `i = 5` tek ve `i == 5`: `break`; `i = 6` hiç denenmez. Döngü sonrası `toplam` 4'tür.

Çıktı

```
[1][4]
Son: 4
```

7

`break` yalnızca **iç** döngüyü bitirir. `i = 1`: `j = 1` iken `i == j`, hemen `break`; hiçbir şey yazılmaz. `i = 2`: `(2,1)` yazılır, `j = 2` iken `break`. `i = 3`: `(3,1)` ve `(3,2)` yazılır, `j = 3` iken `break`. Üç yazdırma var, `sayac = 3`.

Çıktı
```
(2,1)(3,1)(3,2)
sayac = 3
```

8

1. parça: `do-while` gövdesi bir kez çalışır (`n = 1`, `i = 15`), `15 < 10` yanlış olduğundan biter: `15 1`.

2. parça: `for (k = 0; k < 5; k++);` boş gövdelidir, `k` 5 olana kadar döner. `printf` döngünün parçası değildir, bir kez çalışır ve döngü sonrası `k` değerini yazar: `5`.

Çıktı
```
15 1
5
```

9

Dış döngü satır numarasını (`i = 1..n`), iç döngü her satırda `n`'den `i`'ye kadar azalan sayıları üretir. Satır `i`'de `n - i + 1` sayı vardır.

tersUcgen.c
```c
#include <stdio.h>

int main(void) {
    int n = 0;
    printf("n = ");
    scanf("%d", &n);
    for (int i = 1; i <= n; i++) {
        for (int j = n; j >= i; j--) {
            printf("%d", j);
        }
        printf("\n");
    }
    return 0;
}
```

Çıktı
```
n = 5
54321
5432
543
54
5
```

10

Dış döngü girdi bitene kadar sayı okur (`scanf(...) == 1`). Her sayı için ters çevirme algoritması çalışır; `n` bozulmasın diye işlem kopyası `kalan` üzerinde yapılır. `ters` her sayı için sıfırlansın diye dış döngünün içinde tanımlanır.

palindrom.c

```c
#include <stdio.h>

int main(void) {
    int n = 0;
    while (scanf("%d", &n) == 1) {
        int kalan = n;
        int ters = 0;
        while (kalan > 0) {
            ters = ters * 10 + kalan % 10;
            kalan /= 10;
        }
        if (ters == n) {
            printf("%d palindromdur\n", n);
        } else {
            printf("%d palindrom değildir\n", n);
        }
    }
    return 0;
}
```

Çıktı

```
12321
12321 palindromdur
1234
1234 palindrom değildir
7
7 palindromdur
1001
1001 palindromdur
120
120 palindrom değildir
```

BÖLÜM 6

Fonksiyonlar

Bu Bölümde Öğreneceklerin

- Fonksiyonun anatomisini (dönüş tipi, ad, parametre listesi, gövde, `return`), `void` fonksiyonları ve erken `return`'ü
- Prototipin neden gerektiğini ve prototipsiz çağrıda gcc'nin hatasını
- Parametre ile argüman farkını, örtük tip dönüşümünü ve değer ile geçirmeyi (`swap` neden çalışmaz)
- Çağrı yığınında her çağrının nasıl bir çerçeve açtığını
- Kapsamı: yerel, blok ve global değişkenleri, gölgelemeyi; `static` yerel değişkeni
- `return` unutmanın sonucunu; `stdlib.h` ile `rand` ve `srand`'ı
- Klasik algoritmaları fonksiyona dönüştürmeyi ve fonksiyonlarla menü programı yazmayı
- Önişlemcinin ayrıntılarını (parametreli makro, koşullu derleme, `assert`) ve programı `.h` / `.c` dosyalarına bölmeyi

6.1 Neden fonksiyon?

Şimdiye kadar bütün kodu `main` içine yazdın. Program büyüdükçe bu işe yaramaz: aynı hesabı üç yerde yapman gerekir, `main` yüz satıra uzar, hata ararken nereye bakacağını bilemezsin. **Fonksiyon** (function), bir işi yapan ve adı olan kod parçasıdır. Bir kez yazarsın, istediğin kadar çağırırsın. Üç nedeni vardır:

- **Tekrarı önler.** Kopyala-yapıştır yerine fonksiyonu çağırırsın; hata varsa tek yerde düzeltirsin.
- **Okunabilirlik sağlar.** `if (asalMi(n))` satırı, yirmi satırlık bölen arama döngüsünden çok daha okunaklıdır.
- **Böl ve yönet.** Büyük problemi küçük parçalara ayırır, her parçayı ayrı yazar, ayrı denersin.

Aslında fonksiyon kullanıyorsun: `printf`, `scanf` ve `main` birer fonksiyondur. **Kütüphane fonksiyonunu** (`printf`, `sqrt`, `rand`) başkaları yazmıştır; prototipi `#include` ettiğin başlık dosyasındadır, kodunu bağlayıcı programa ekler. **Kendi fonksiyonunu** (`asalMi`, `ebob`) sen yazarsın; prototipi ve kodu programındadır.

6.2 Fonksiyonun anatomisi

Bir fonksiyon tanımı beş parçadan oluşur: `dönüşTipi ad(parametreListesi) { gövde }`, gövdenin içinde de `return`.

Parça	Anlamı	`int kareAl(int x)` için
Dönüş tipi (return type)	Çağırana geri verilen değerin tipi	`int`
Ad	Çağırırken kullanılan isim	`kareAl`
Parametre listesi	Fonksiyonun dışarıdan aldığı değerler (tip + ad)	`int x`
Gövde (body)	`{ ... }` içindeki deyimler	`return x * x;`
`return`	Değeri çağırana verir ve fonksiyonu bitirir	`return x * x;`

kareAl.c

```c
#include <stdio.h>

int kareAl(int x) {
    return x * x;
}

int main(void) {
    int a = kareAl(5);
    int b = kareAl(a);
    printf("%d %d\n", a, b);
    printf("%d\n", kareAl(2) + kareAl(3));
    return 0;
}
```

Çıktı

```
25 625
13
```

Çağrıda şunlar olur: `main` durur, kontrol `kareAl`'a geçer, `x` parametresi `5` değerini alır, `return` değeri geri verir ve `main` kaldığı yerden devam eder. **Çağrı ifadesi (`kareAl(5)`), döndürdüğü değerin yerine geçer**; onu bir değişkene atayabilir, başka bir fonksiyona argüman yapabilir ya da `kareAl(2) + kareAl(3)` gibi işlemlerde kullanabilirsin.

- Fonksiyon adı değişken adı kurallarına uyar. Kitapta Türkçe ASCII camelCase kullanıyoruz ve adı **ne yaptığını söyleyen fiil ya da soru** seçiyoruz: `ortalamaHesapla`, `asalMi`, `menuyuGoster`.
- Aynı adlı iki fonksiyon tanımlayamazsın. Standart C'de fonksiyonun içinde fonksiyon da tanımlanmaz; tanımlar yan yana yazılır.
- Dönüş tipi `int`, `double`, `char`, `long long`, `bool` gibi herhangi bir tip ya da `void` olabilir.

İyi fonksiyon **tek iş** yapar ve adı o işi anlatır: `asalMi` sayının asal olup olmadığına karar verir; ekrana yazmak ya da klavyeden okumak `main`'in işidir. "Hesaplayan fonksiyon yazdırmaz" kuralı fonksiyonu başka programlarda da kullanılır kılar.

6.3 void fonksiyonlar, dönüş değeri ve erken çıkış

Bazı fonksiyonlar değer üretmez, yalnızca bir iş yapar (ekrana yazar, menü gösterir). Bunların dönüş tipi `void`'dir (boş). `void` fonksiyonda `return deger;` yazılmaz; istersen yalnızca `return;` yazarak fonksiyonu erken bitirebilirsin. Sona gelince fonksiyon zaten kendiliğinden biter. Değer üretmediği için `void` fonksiyonun çağrısını bir değişkene **atayamazsın**: `int s = selamVer();` yazarsan gcc `void value not ignored as it ought to be` hatası verir.

`return` çalıştığı anda fonksiyon biter; altındaki satırlar çalışmaz. Bu yüzden özel durumları fonksiyonun başında ele alıp hemen çıkabilirsin:

erkenCikis.c

```c
#include <stdio.h>

void bol(int pay, int payda) {
    if (payda == 0) {
        printf("Sıfıra bölünemez!\n");
        return;
    }
    printf("%d / %d = %d\n", pay, payda, pay / payda);
}
```

```c
int mutlak(int n) {
    if (n < 0) {
        return -n;
    }
    return n;
}

int main(void) {
    bol(17, 5);
    bol(8, 0);
    printf("%d %d\n", mutlak(-7), mutlak(4));
    return 0;
}
```

Çıktı

```
17 / 5 = 3
Sıfıra bölünemez!
7 4
```

`bol(8, 0)` çağrısında `return;` bölmeyi atlattı; Bölüm 2'de tamsayıyı sıfıra bölmenin tehlikeli olduğunu görmüştün. `mutlak` içinde `else` yok: `n < 0` ise `return` fonksiyonu bitirdiği için ikinci `return`'e hiç gelinmez.

Fonksiyonun döndürdüğü değeri kullanmak zorunda değilsin: `topla(4, 5);` tek başına yazılırsa fonksiyon çalışır, değer sessizce atılır. `scanf` ve `printf` de değer döndürür (okunan öğe sayısı, yazılan karakter sayısı); Bölüm 5'te `scanf(...) == 1` diye kullanmıştın, çoğu zaman ise bu değeri atarız. Bir fonksiyon **en fazla bir değer** döndürür; birden fazla sonuç için başka yollar vardır (adres Bölüm 9'da, `struct` Bölüm 11'de).

> **Sık Yapılan Hata: Sonucu kullanmayı unutmak**
>
> `kareAl(x);` tek başına yazarsan `x` kare olmaz; hesaplanan değer atılır. Sonucu istiyorsan bir yere koy: `x = kareAl(x);`. Bu hata, fonksiyonun çağıranın değişkenini değiştirdiğini sanmaktan gelir; nedenini "Değer ile geçirme" başlığında göreceksin.

6.4 Prototip (bildirim)

gcc dosyayı yukarıdan aşağıya okur. Bir fonksiyon çağrısına geldiğinde o fonksiyonu **daha önce görmüş** olmalıdır. Fonksiyonu `main`'den sonra yazarsan sorun çıkar:

prototipsiz.c

```c
#include <stdio.h>

int main(void) {
    int sonuc = kareAl(5);
    printf("%d\n", sonuc);
    return 0;
}

int kareAl(int x) {
    return x * x;
}
```

Derleyici çıktısı – program derlenmez

```
prototipsiz.c: In function 'main':
prototipsiz.c:4:17: error: implicit declaration of function 'kareAl' [-Wimplicit-function-
declaration]
    4 |     int sonuc = kareAl(5);
      |                 ^~~~~~
```

Bu, Bölüm 1'de `prinft` yazınca gördüğün `implicit declaration of function` hatasının aynısıdır: fonksiyon kodda aşağıda **var**, ama gcc çağrı satırında henüz okumadı. (gcc 14'ten önceki sürümler bunu yalnızca uyarı sayardı; yeni sürümler hata verir. Sınavda hata kabul et.) İki çözüm vardır:

1. Fonksiyonu `main`'den **önce** tanımla. Küçük programlarda yeterlidir.
2. Fonksiyonu `main`'den önce **tanıt**: **prototip** (function prototype) yaz. Prototip, tanımın gövdesiz hâlidir ve `;` ile biter: "ad şu, dönüş tipi şu, parametreler şunlar; tanımı aşağıda".

prototipli.c

```c
#include <stdio.h>

int kareAl(int x);
double ortalama(int a, int b);

int main(void) {
    printf("%d\n", kareAl(6));
    printf("%.1f\n", ortalama(3, 4));
    return 0;
}

int kareAl(int x) {
    return x * x;
}

double ortalama(int a, int b) {
    return (a + b) / 2.0;
}
```

Çıktı

```
36
3.5
```

Önce prototipler, sonra `main`, en altta tanımlar: sınavlarda en çok görülen düzen budur, çünkü dosyayı açan kişi ilk olarak programın akışını okur. Prototip kuralları:

- Dönüş tipi, ad ve parametre **tipleri** tanımla birebir aynı olmalıdır. Parametre **adları** serbesttir ve yazılmayabilir: `int kareAl(int);` da geçerlidir.
- `#include <stdio.h>` satırı, `printf`, `scanf` ve onlarca fonksiyonun prototipini programına ekler. Başlık dosyası, prototipler topluluğudur.

Prototip ile tanım uyuşmazsa gcc şikâyet eder:

prototipCakisma.c

```c
#include <stdio.h>

int alan(int en, int boy);

int main(void) {
    printf("%d\n", alan(3, 4));
    return 0;
}

double alan(int en, int boy) {
    return en * boy;
}
```

Derleyici çıktısı – program derlenmez

```
prototipCakisma.c:10:8: error: conflicting types for 'alan'; have 'double(int,  int)'
   10 | double alan(int en, int boy) {
      |        ^~~~
prototipCakisma.c:3:5: note: previous declaration of 'alan' with type 'int(int,  int)'
    3 | int alan(int en, int boy);
      |     ^~~~
```

`conflicting types for 'alan'`: aynı fonksiyon için iki farklı tip görüldü (prototipte `int`, tanımda `double`). Parametre sayısı ya da tipi farklı olsa da aynı hatayı alırsın.

> **● Sık Yapılan Hata: Tanımın başlığına ; koymak**
>
> Prototip `;` ile biter, tanım bitmez. `int topla(int a, int b); {` yazarsan gcc süslü parantezi fonksiyon gövdesi olarak değil, başıboş bir blok olarak görür ve `{` işaretini gösterip `expected identifier or '(' before '{' token` der. Bölüm 5'teki `for (...);` tuzağının akrabasıdır. Bir de `int menu();` ile `int menu(void);` aynı değildir: boş parantez "parametreler belirtilmedi" demektir ve hatalı çağrıları yakalatmaz. Parametresiz fonksiyonda `(void)` yaz.

> **● Sınavda Çıkar**
>
> Fonksiyon `main`'den sonra yazılmış ve prototip yoksa: **derleme hatası** (`implicit declaration of function`). Prototip tanımla uyuşmazsa: `conflicting types`. Bu iki mesajı ayırt etmeni isterler. Prototipteki **parametre adlarının** tanımdakilerle aynı olması gerekmez; tipler ve sayılar aynı olmalıdır.

6.5 Parametre ve argüman

Terim	Nerede?	Örnek
Parametre (parameter)	Fonksiyonun tanımında, parantez içinde	`int topla(int a, int b)`: `a` ve `b`
Argüman (argument)	Fonksiyonun çağrısında	`topla(5, y + 1)`: `5` ve `y + 1`

Argümanlar parametrelere **sırayla** eşlenir. Parametreler fonksiyonun içinde birer **yerel değişken** gibi davranır ve çağrıda argümanların değeriyle başlatılır. Her parametrenin tipi **ayrı ayrı** yazılır: `int topla(int a, int b)` doğrudur; `int topla(int a, b)` yazarsan gcc `unknown type name 'b'` der. Argüman sayısı da parametre sayısına eşit olmalıdır:

eksikArguman.c

```c
#include <stdio.h>

int topla(int a, int b) {
    return a + b;
}

int main(void) {
    printf("%d\n", topla(5));
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
eksikArguman.c: In function 'main':
eksikArguman.c:8:20: error: too few arguments to function 'topla'; expected 2, have 1
    8 |     printf("%d\n", topla(5));
      |                    ^~~~~
eksikArguman.c:3:5: note: declared here
    3 | int topla(int a, int b) {
      |     ^~~~~
```

gcc fonksiyonun tanımlandığı yeri de gösterdi (`note: declared here`). Fazla argümanda mesaj `too many arguments to function` olur. Tip yalnızca tanımda ve prototipte yazılır, çağrıda yazılmaz: `topla(int 5, int 3)` yanlıştır, gcc `expected expression before 'int'` der.

Tip dönüşümü

Prototip ya da tanım görünürken argüman, parametrenin tipine **kendiliğinden** çevrilir; `return` ile döndürülen değer de dönüş tipine çevrilir. Bölüm 2'deki kurallar geçerlidir: `int` → `double` kayıpsızdır (`double` parametreli fonksiyona `5` verirsen `5.0` olur), `double` → `int` ondalık kısmı **atar** (sıfıra doğru).

tipDonusumu.c

```c
#include <stdio.h>

int tamKisim(double x) {
    return x;
}

double ortalamaYanlis(int a, int b) {
    return (a + b) / 2;
}

int main(void) {
    printf("%d %d\n", tamKisim(7.9), tamKisim(-7.9));
    printf("%.2f\n", ortalamaYanlis(3, 4));
    return 0;
}
```

Çıktı

```
7 -7
3.00
```

- `tamKisim(7.9)`: `return x;` `double`'ı `int`'e çevirirken ondalık kısmı attı: `7`. Negatifte `-7` çıkar (yuvarlama değil, kesme).

- `ortalamaYanlis(3, 4)`: **tuzak**. `(a + b) / 2` iki `int`'in bölmesidir, sonuç `7 / 2 = 3` olur ve ancak sonra `double`'a çevrilip `3.00` yazılır. Dönüş tipinin `double` olması bölmeyi `double` yapmaz; `/ 2.0` yazmalısın (`prototipli` programında `ortalama` doğru yazılmıştı).

6.6 Değer ile geçirme

Bu bölümün en önemli kuralı: C'de argümanlar fonksiyona **değer ile** (pass by value) geçirilir. Fonksiyon argümanın **kopyasını** alır; parametreyi fonksiyon içinde ne kadar değiştirirsen değiştir, çağıran fonksiyondaki değişken aynı kalır. Günlük benzetme: bir belgenin fotokopisine not almak asıl belgeyi değiştirmez. Bu kuralın en meşhur sonucu, iki değişkenin değerini değiştirmeyi (takas, swap) deneyen fonksiyondur:

takasCalismaz.c

```c
#include <stdio.h>

void takas(int a, int b) {
    int gecici = a;
    a = b;
    b = gecici;
    printf("takas içinde: a = %d, b = %d\n", a, b);
}

int main(void) {
    int x = 3;
    int y = 8;
    takas(x, y);
    printf("main içinde : x = %d, y = %d\n", x, y);
    return 0;
}
```

Çıktı

```
takas içinde: a = 8, b = 3
main içinde : x = 3, y = 8
```

Takas `a` ve `b` kopyaları üzerinde başarıyla yapıldı, ama `x` ve `y` yerinde durdu: fonksiyon bitince kopyalar yok oldu ve işin hiçbir izi kalmadı. Parametre adlarını `x` ve `y` yapsan da değişmez; aynı adlı iki değişken yine **iki ayrı değişkendir**. Çalışan `takas`, değişkenlerin **adresini** alan bir fonksiyondur; bunu Bölüm 9'da işaretçilerle yazacaksın. Şimdilik kural: **fonksiyon sonucu çağırana yalnızca `return` ile gönderir** (`x = ikiKati(x);` gibi).

> **Sınavda Çıkar**
>
> "`takas(x, y)` çağrıldıktan sonra `x` ve `y` ne olur?" sorusunun cevabı hemen her zaman "**değişmez**"dir. Gerekçe tek cümle: argümanlar değer ile geçirildiğinden fonksiyon kopyalar üzerinde çalışır. (Dizilerde durum görünürde farklıdır; nedenini Bölüm 7'de göreceksin.)

6.7 Çağrı yığını

Değer ile geçirmenin arkasında bir mekanizma vardır. Program çalışırken her fonksiyon çağrısı için bellekte bir **yığın çerçevesi** (stack frame) açılır. Çerçevede fonksiyonun parametreleri, yerel değişkenleri ve fonksiyon bitince nereye dönüleceği bilgisi bulunur. Çerçeveler **çağrı yığını** (call stack) denen bölgede üst üste dizilir; fonksiyon `return` edince çerçevesi **silinir**. Yerel değişkenler ve parametreler bu yüzden fonksiyon bitince yok olur ve aynı adlı değişkenler birbirini etkilemez: her çağrının kendi çerçevesi, yani kendi kopyaları vardır.

Aşağıdaki programda `main`, `karelerToplami`'nı, o da `kare`'yi çağırır:

cagriYigini.c

```c
#include <stdio.h>

int kare(int x) {
    return x * x;
}

int karelerToplami(int a, int b) {
    int ka = kare(a);
    int kb = kare(b);
    return ka + kb;
}

int main(void) {
    int sonuc = karelerToplami(3, 4);
    printf("Sonuç: %d\n", sonuc);
    return 0;
}
```

Çıktı

```
Sonuç: 25
```

Programın baştan sona yığın hareketi (`?`: değişken tanımlanmış ama henüz değer atanmamış; tablodaki ilk çerçeve en üsttedir, yani o an çalışandır):

Adım	Olay	Yığın (üstten alta)
1	`main` başlar	`main` (sonuc=?)
2	`karelerToplami(3, 4)` çağrılır	`karelerToplami` (a=3, b=4, ka=?, kb=?) / `main`
3	`kare(3)` çağrılır	`kare` (x=3) / `karelerToplami` / `main`
4	`kare` 9 döndürür, çerçevesi silinir; `ka = 9`	`karelerToplami` (a=3, b=4, ka=9, kb=?) / `main`
5	`kare(4)` çağrılır, 16 döner; `kb = 16`	`kare` (x=4) / `karelerToplami` / `main`; sonra yine `karelerToplami` / `main`
6	`karelerToplami` 25 döndürür, çerçevesi silinir; `sonuc = 25`	`main` (sonuc=25)

Üç şeye dikkat et. (1) Aynı `kare` fonksiyonu 3. ve 5. adımlarda iki ayrı çerçeve açtı; `x` her seferinde sıfırdan oluştu. (2) Çerçevedeki `a` ve `b`, `main`'deki değerlerin kopyasıdır. (3) Çağıran fonksiyon, çağırdığı bitene kadar **bekler**. `birEkle(ikiKati(5))` gibi iç içe çağrılarda dıştaki fonksiyonun argümanı içteki çağrının sonucudur, bu yüzden **içteki önce çalışır**: `ikiKati(5) = 10`, sonra `birEkle(10) = 11`. Sıra sonucu değiştirir: `ikiKati(birEkle(5))` `12` verir. Sınavda `f(g(x))` gördüğünde içten dışa doğru tek tek yaz. Ancak `f(g(x), h(y))` çağrısında `g` ile `h`'den hangisinin önce çalışacağını C belirlemez; aynı çağrıda yan etkisi olan (ekrana yazan, global değişkeni değiştiren) iki çağrıyı argüman yapma.

Fonksiyon kendi kendini de çağırabilir; buna **özyineleme** (recursion) denir ve her çağrı kendi çerçevesini açtığı için çalışır. Durma koşulu olmazsa çerçeveler sonsuza kadar açılır ve yığın (sınırlı bir bölgedir) dolar. Özyinelemeyi Bölüm 13'te ayrıntılı işleyeceğiz.

6.8 Kapsam

Bir değişkenin **kapsamı** (scope), programın hangi bölgelerinden o değişkene erişilebildiğini belirler. Fonksiyonun içinde tanımlanan değişken **yereldir**: yalnızca o fonksiyonda görünür ve fonksiyon bitince yok olur; parametreler de yereldir. `hesapla` içinde tanımlı `sonucDegeri`'ni `main`'de kullanırsan gcc

`'sonucDegeri' undeclared (first use in this function)` der. Değeri çağırana `return` ile göndermelisin. Yerel değişkene ilk değer vermezsen içinde önceden kalmış **çöp** (garbage) değer bulunur ve okumak tanımsız davranıştır (Bölüm 2); her yerel değişkene değer ver.

Blok kapsamı ve gölgeleme

`{ }` ile oluşturulan her **blok** kendi kapsamını açar. İç blokta dış bloktaki bir değişkenle **aynı adı** kullanırsan iç olan dış olanı **gölgeler** (shadowing): blok boyunca o adın altında iç değişken görünür, blok bitince dış olan yeniden görünür.

kapsamBlok.c

```c
#include <stdio.h>

int main(void) {
    int x = 10;
    printf("1) x = %d\n", x);
    {
        int x = 20;
        int y = 5;
        printf("2) x = %d, y = %d\n", x, y);
    }
    printf("3) x = %d\n", x);
    return 0;
}
```

Çıktı

```
1) x = 10
2) x = 20, y = 5
3) x = 10
```

İç bloktaki `x = 20` dıştaki `x`'i **değiştirmedi**; ayrı, yeni bir değişkendi. Blok bitince `y` ile içteki `x` yok oldu, dıştaki `x` hâlâ 10. Gölgeleme geçerlidir ama okunması zordur; kendi kodunda kaçın, sınavda ise bu soruyu hazır bekle. Tek sınır: fonksiyonun **en dış bloğunda** parametreyle aynı adı tanımlayamazsın (`int ikiKati(int n) { int n = 2; ... }`); gcc `'n' redeclared as different kind of symbol` der.

Global değişkenler

Hiçbir fonksiyonun içinde olmayan, dosyanın üstünde tanımlanan değişkene **global değişken** denir. Tanımından sonraki bütün fonksiyonlar onu görür ve değiştirebilir; program boyunca yaşar. İlk değer verilmezse otomatik olarak **0** olur (yerel değişkenlerden farkı budur).

globalYerel.c

```c
#include <stdio.h>

int sayi = 100;

void yerelGolgeler(void) {
    int sayi = 7;
    printf("yerelGolgeler: sayi = %d\n", sayi);
}

int main(void) {
    printf("main         : sayi = %d\n", sayi);
    yerelGolgeler();
    sayi = sayi + 1;
    printf("main         : sayi = %d\n", sayi);
    return 0;
}
```

Çıktı

```
main         : sayi = 100
yerelGolgeler: sayi = 7
main         : sayi = 101
```

`yerelGolgeler` içinde yeni bir yerel `sayi` tanımlandığı için o fonksiyonda global `sayi` görünmez; `main` ise yerel tanımı olmadığından global olanı değiştirdi (100 → 101).

Global değişken kolaydır: parametre geçirmeden her yerden erişirsin. Ama bu yüzden **az kullanılır**: Değeri hangi fonksiyonun değiştirdiğini bulmak zorlaşır; fonksiyon global değişkene bağımlı olunca başka programa taşınamaz ve tek başına denenemez; girdi ve çıktısı artık parametre ve `return` değil, görünmeyen bir ortak yerdir; yerel değişkenle aynı ad verilirse yanlışlıkla gölgeleme olur. Kural: Veriyi **parametre** ile ver, sonucu `return` ile al. Global değişkeni yalnızca programın her yerinde gereken ve değişmeyen değerler (sabitler) için kullan.

> **Sınavda Çıkar**
>
> Yerel/global soruları "bu program ne yazdırır?" biçiminde gelir. Tek kural: **bir ad, kullanıldığı yerden başlayıp içten dışa doğru aranır**; önce kendi bloğu, sonra dıştaki bloklar, en son global. Bulduğun ilk tanım geçerlidir. Gölgelenmiş global değişken, blok bitince eski hâliyle görünür.

6.9 static yerel değişken

Yerel değişkenler her çağrıda sıfırdan oluşur ve fonksiyon bitince silinir (çağrı yığını!). Bazen fonksiyonun çağrılar arasında bir değeri **hatırlamasını** istersin: kaç kez çağrıldığını saymak gibi. Global değişken çözümdür ama onu herkes görür. Daha iyisi **`static` yerel değişken**dir: değişkenin başına `static` yazarsın.

staticSayac.c

```c
#include <stdio.h>

int normalSayac(void) {
    int sayac = 0;
    sayac++;
    return sayac;
}

int kaliciSayac(void) {
    static int sayac = 0;
    sayac++;
    return sayac;
```

```c
}

int main(void) {
    for (int i = 1; i <= 4; i++) {
        int n = normalSayac();
        int k = kaliciSayac();
        printf("Çağrı %d: normal = %d, static = %d\n", i, n, k);
    }
    return 0;
}
```

Çıktı

```
Çağrı 1: normal = 1, static = 1
Çağrı 2: normal = 1, static = 2
Çağrı 3: normal = 1, static = 3
Çağrı 4: normal = 1, static = 4
```

`normalSayac` her çağrıda `sayac`'ı 0'dan başlatır, 1 yapar ve döndürür; hep `1` verir. `kaliciSayac`'taki `static int sayac = 0;` satırı ise **yalnızca bir kez**, program başlarken işlenir. Değişken fonksiyon bitince silinmez, sonraki çağrıda kaldığı değerle devam eder. `static` değişken çağrı yığınında değil, program boyunca yaşayan ayrı bir bölgede durur; ilk değeri **sabit** olmak zorundadır (`static int n = 0;` olur, `static int n = x;` olmaz) ve başlatılmazsa 0 olur.

	Yerel (normal)	`static` yerel	Global
Ömrü	Fonksiyon çalışırken	Programın tamamı	Programın tamamı
Kim görür?	Tanımlandığı blok	Tanımlandığı blok	Tanımından sonraki bütün fonksiyonlar
İlk değer verilmezse	Çöp	0	0
İlk değer ne zaman verilir?	Her çağrıda	Bir kez	Bir kez

> **Sınavda Çıkar**
>
> `static` yerel değişkenli bir fonksiyonu art arda çağırıp çıktıyı sormak çok sevilen sorudur. Anahtar cümle: `static` değişkenin ilk değeri **yalnızca bir kez** verilir; sonraki çağrılar önceki değerle devam eder. Normal yerel değişken her çağrıda ilk değerinden başlar. Her çağrıdan sonraki değeri bir tabloya yaz.

`static`, fonksiyon dışındaki bir değişkenin ya da fonksiyonun başına yazılırsa farklı anlam taşır: ad yalnızca **o `.c` dosyasında** görünür (birden çok dosyalı programlarda yardımcıları gizlemek için). Tek dosyayla çalıştığımız sürece pratik farkı yoktur; ayrıntısı bölümün sonundaki "Programı dosyalara bölmek" başlığında.

6.10 Fonksiyonu return olmadan bitirmek

Dönüş tipi `void` olmayan bir fonksiyon her yolda bir değer **döndürmelidir**. Aşağıdaki `isaret` fonksiyonu `n` sıfırsa hiçbir `return`'e uğramadan sona erer:

returnYok.c

```c
#include <stdio.h>

int isaret(int n) {
    if (n > 0) {
        return 1;
    } else if (n < 0) {
        return -1;
    }
}

int main(void) {
    printf("%d\n", isaret(7));
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
returnYok.c: In function 'isaret':
returnYok.c:9:1: warning: control reaches end of non-void function [-Wreturn-type]
    9 | }
      | ^
```

gcc `-Wall` ile `control reaches end of non-void function` uyarısı verir. Program derlenir, ama `isaret(0)` çağrılırsa fonksiyon değer döndürmeden biter ve çağıranın aldığı değer **tanımsızdır** (çıktısını göstermiyoruz, çünkü sonuca güvenilemez). Uyarıyı hata gibi ciddiye al: en sık hata, `return`'leri yalnızca `if` / `else if` kollarına koyup "başka ihtimal olamaz" sanmaktır; derleyici ihtimalleri senin kadar bilmez. Çözüm, fonksiyonun son satırını bir `return` yapmaktır: `if (n < 0) { return -1; } return 0;` gibi. `void` fonksiyonlarda sorun yoktur; `main`'in sonundaki `return 0;`'ın zorunlu olmaması da özel bir kuraldır (Bölüm 1), başka hiçbir fonksiyon için geçerli değildir.

6.11 Hazır fonksiyonlar

Her hazır fonksiyon bir başlık dosyasında tanıtılmıştır ve kullanmak için o dosyayı `#include` etmen gerekir (eksik başlık, `implicit declaration of function` hatasıdır).

Başlık	Fonksiyonlar	Not
`math.h`	`sqrt`, `pow`, `fabs`, `ceil`, `floor`, ...	`double` alır, `double` verir; Linux'ta derlerken `-lm` gerekir (Bölüm 2)
`stdlib.h`	`abs`, `rand`, `srand`, `RAND_MAX`	Aşağıda

Hazır fonksiyonları kendi fonksiyonlarının içinde de kullanabilirsin (örneğin iki nokta arasındaki uzaklığı hesaplayan fonksiyonda `sqrt`). `abs` tamsayının mutlak değerini alır ve `stdlib.h`'dedir; `double` için `math.h`'deki `fabs` kullanılır. `abs(-3.7)` yazarsan `-3.7` önce `int`'e çevrilir (`-3`) ve sonuç `3` olur; gcc bunu `-Wabsolute-value` uyarısıyla yakalar.

rand ve srand

`rand()` her çağrıda `0` ile `RAND_MAX` arasında bir **sözde rastgele** (pseudo-random) tamsayı döndürür. `RAND_MAX` en az 32767'dir; Windows'ta (MinGW) tam 32767, Linux ve macOS'ta 2147483647'dir. Sayılar formülle üretildiği için aynı başlangıç değeri (**tohum**, seed) hep aynı diziyi verir. Tohumu `srand(tohum)` ile belirlersin. İstediğin aralığa sığdırmak için kalan (`%`) işlemi kullanılır:

İstenen	İfade	Neden?
0 ile 99 arası	`rand() % 100`	Kalan 0..99
Zar (1 ile 6)	`rand() % 6 + 1`	Kalan 0..5, `+ 1` ile 1..6
`a` ile `b` arası	`a + rand() % (b - a + 1)`	10..20 için `10 + rand() % 11`
0 ile 1 arası `double`	`(double) rand() / RAND_MAX`	Bölme `double` yapılır

`srand` hiç çağrılmazsa her çalıştırmada **aynı** dizi gelir. Sayıların her seferinde değişmesi için tohumu değişen bir değerden alırsın; en yaygın kalıp **şimdiki zaman**dır: `srand(time(NULL));`. `time` fonksiyonu `time.h`'dedir ve 1970'ten beri geçen saniyeyi döndürür.

zarAtma.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    srand(time(NULL));
    for (int i = 1; i <= 5; i++) {
        printf("Zar %d: %d\n", i, rand() % 6 + 1);
    }
    return 0;
}
```

Çıktı

```
Zar 1: 5
Zar 2: 6
Zar 3: 1
Zar 4: 2
Zar 5: 5
```

Bu çıktı sana özgü bir örnektir. Programı kendi bilgisayarında çalıştırınca farklı sayılar görürsün, her çalıştırışta da farklı çıkar. `rand()` Windows, Linux ve macOS'ta farklı formüller kullandığı için aynı tohumla bile farklı sayılar üretir. Sınavlarda `rand` çıktısı sorulmaz; "sonuç hangi aralıkta olabilir?" sorulur. Her sayı yine de 1 ile 6 arasındadır.

> **Sık Yapılan Hata: srand'ı döngü içinde çağırmak**
>
> `srand(time(NULL));` satırı programın başında **bir kez** çalışmalıdır. Döngünün içine koyarsan, `time` saniye çözünürlüklü olduğundan döngü bir saniyeden kısa sürdüğünde her tur aynı tohumla başlar ve `rand()` hep aynı sayıyı verir. Bir de `rand() % 6` yazıp `+ 1`'i unutursan zar 0 ile 5 arasında çıkar.

6.12 Algoritmaları fonksiyona dönüştürme

Bölüm 5'te klasik algoritmaları `main` içinde yazdın. Şimdi fonksiyona çevirelim. Bu kez tanımları `main`'den önce yazdık, bu yüzden prototip gerekmedi. Tarif basittir: algoritmanın **girdileri** parametre olur, **sonucu** `return` ile döner, `scanf` ve `printf` fonksiyonun dışında, `main`'de kalır.

algoritmalar.c

```c
#include <stdio.h>
#include <stdbool.h>

bool asalMi(int n) {
    if (n < 2) {
        return false;
```

```c
    }
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}

int ebob(int a, int b) {
    while (b != 0) {
        int kalan = a % b;
        a = b;
        b = kalan;
    }
    return a;
}

long long faktoriyel(int n) {
    long long sonuc = 1;
    for (int i = 2; i <= n; i++) {
        sonuc *= i;
    }
    return sonuc;
}

int basamakToplami(int n) {
    int toplam = 0;
    while (n > 0) {
        toplam += n % 10;
        n /= 10;
    }
    return toplam;
}

long long ustAl(int taban, int us) {
    long long sonuc = 1;
    for (int i = 0; i < us; i++) {
        sonuc *= taban;
    }
    return sonuc;
}

int main(void) {
    printf("2'den 30'a asallar:");
    for (int n = 2; n <= 30; n++) {
        if (asalMi(n)) {
            printf(" %d", n);
        }
    }
    printf("\n");
    printf("ebob(48, 18) = %d\n", ebob(48, 18));
    printf("20! = %lld\n", faktoriyel(20));
    printf("basamakToplami(4712) = %d\n", basamakToplami(4712));
    printf("2^10 = %lld\n", ustAl(2, 10));
    return 0;
}
```

Çıktı

```
2'den 30'a asallar: 2 3 5 7 11 13 17 19 23 29
ebob(48, 18) = 6
20! = 2432902008176640000
basamakToplami(4712) = 14
2^10 = 1024
```

- **`asalMi`** `bool` döndürür. Bölüm 5'te bayrağı `false` yapıp `break` ile çıkıyorduk; fonksiyonda bölen bulunca doğrudan `return false;` yazarsın, bayrağa ve `break`'e gerek kalmaz. Döngü bölensiz biterse `return true;`. Kullanımı `if (asalMi(n));` `== true` yazmana gerek yok. `bool` için `stdbool.h` gerekir (Bölüm 4).
- **`ebob`** parametreleri doğrudan değiştirir (`a = b; b = kalan;`). Bölüm 5'te `x` ve `y` kopyaları çıkarıyorduk; burada gerek yok, çünkü `a` ve `b` zaten **kopyadır** ve çağıranın değişkenleri korunur. Değer ile geçirmenin işe yaradığı yer burasıdır. `basamakToplami` de aynı şekilde `n`'i eritir; `main`'deki sayı bozulmaz.
- **`faktoriyel`** `long long` döndürür; `20!` sığar, `21!` sığmaz (Bölüm 5). `faktoriyel(0)` döngüye girmez ve `1` verir.
- **`ustAl`** tamsayı kuvvetini döngüyle hesaplar. `pow` `double` döndürür ve büyük tamsayılarda tam sonuç vermeyebilir; tamsayı kuvveti için kendi fonksiyonun daha güvenlidir. Üs negatif verilmez varsayılır.
- Fonksiyonlar birbirini de çağırabilir: `EKOK = a / ebob(a, b) * b` olduğundan `ekok` fonksiyonu tek satırdır.

> **İpucu: Soru çeşidine göre dönüş tipi**
>
> "... mı?" diye sorulan fonksiyonlar `bool` döndürür (`asalMi`, `ciftMi`, `mukemmelMi`). "Kaç tane / ne kadar?" soruları `int` ya da `long long`, ortalama gibi ondalıklı sonuçlar `double` döndürür. Yazdırma ve menü gibi iş yapanlar `void` olur. Dönüş tipini seçerken fonksiyonun adını soru gibi oku.

6.13 Menü programı

Fonksiyonların "böl ve yönet" gücü en iyi menü programında görünür. `main` yalnızca ana döngüyü yönetir; her seçenek kendi fonksiyonuna devredilir. Bölüm 5'teki `do-while` menüsünün fonksiyonlu hâlidir:

menu.c

```c
#include <stdio.h>
#include <stdbool.h>

void menuyuGoster(void);
int sayiOku(void);
void kareIstegi(void);
void ciftIstegi(void);

int main(void) {
    int secim = 0;
    do {
        menuyuGoster();
        secim = sayiOku();
        switch (secim) {
            case 1:
                kareIstegi();
                break;
            case 2:
                ciftIstegi();
                break;
            case 0:
                printf("Güle güle!\n");
                break;
            default:
                printf("Geçersiz seçim!\n");
```

```
        }
    } while (secim != 0);
    return 0;
}

void menuyuGoster(void) {
    printf("\n=== MENÜ ===\n");
    printf("1) Karesini al\n");
    printf("2) Çift mi?\n");
    printf("0) Çıkış\n");
    printf("Seçim: ");
}

int sayiOku(void) {
    int sayi = 0;
    while (scanf("%d", &sayi) != 1) {    // sayı okunamadıysa
        int c = 0;
        while ((c = getchar()) != '\n' && c != EOF) { }   // tamponu boşalt
        if (c == EOF) {
            return 0;                    // girdi bitti: menüde "Çıkış"
        }
        printf("Sayı gir: ");
    }
    return sayi;
}

int kareAl(int x) {
    return x * x;
}

bool ciftMi(int n) {
    return n % 2 == 0;
}

void kareIstegi(void) {
    printf("Sayı: ");
    int n = sayiOku();
    printf("%d * %d = %d\n", n, n, kareAl(n));
}

void ciftIstegi(void) {
    printf("Sayı: ");
    if (ciftMi(sayiOku())) {
        printf("Çift.\n");
    } else {
        printf("Tek.\n");
    }
}
```

Çıktı

```
=== MENÜ ===
1) Karesini al
2) Çift mi?
0) Çıkış
Seçim: 1
Sayı: a
Sayı gir: 12
12 * 12 = 144

=== MENÜ ===
1) Karesini al
2) Çift mi?
0) Çıkış
Seçim: 2
```

```
Sayı: 7
Tek.

=== MENÜ ===
1) Karesini al
2) Çift mi?
0) Çıkış
Seçim: 5
Geçersiz seçim!

=== MENÜ ===
1) Karesini al
2) Çift mi?
0) Çıkış
Seçim: 0
Güle güle!
```

Programda üç fikir var. (1) `main` kısa ve okunaklıdır; ne olduğunu bir bakışta anlarsın. (2) `sayiOku`, `scanf` işini ve hata kontrolünü tek yerde toplar; menü seçimi de sayılar da onunla okunur. `scanf` sayı okuyamazsa (`a` yazıldı) Bölüm 5'teki kalıpla tamponu boşaltıp yeniden sorar. Bu kontrol olmasaydı harf tamponda kalır, `sayi` ilk değeri olan `0`'da kalırdı ve menüde `0` "Çıkış" olduğu için program habersizce kapanırdı. Girdi tamamen biterse (`EOF`) de `0` döndürür ve program düzgünce çıkar. (3) Her menü seçeneği ayrı fonksiyondadır; yeni seçenek eklemek için bir `case` ve bir fonksiyon yazarsın, mevcut kod bozulmaz. `kareAl` ve `ciftMi` prototipsiz kaldı, çünkü tanımları kullanıldıkları `kareIstegi` ve `ciftIstegi`'nden **önce** yazıldı.

6.14 main de bir fonksiyondur

`main` ayrıcalıklı ama sıradan bir fonksiyondur: dönüş tipi `int`, parametre listesi `(void)`. Onu **işletim sistemi** çağırır ve `return` ile verdiği değer **çıkış kodu** olur (Bölüm 1). `main` içinde `return`, programı bitirir; bu yüzden hata durumunda `return 1;` ile erken ve hatalı çıkış yapabilirsin (normal akışta `return 0;`).

> **Not: Büyük programlar birden çok dosyadan oluşur**
>
> Gerçek projelerde fonksiyonlar farklı `.c` dosyalarına bölünür. Bu kitabın örnekleri tek dosyadır; ama bölümün son iki başlığında önişlemcinin ayrıntılarını ve programı dosyalara bölmeyi öğreneceksin.

6.15 Önişlemci: makro ve koşullu derleme

Önişlemci (Bölüm 1), derleyiciden **önce** çalışan ve yalnızca **metin** değiştiren adımdır; `#` ile başlayan satırları işler. Bölüm 2'de onunla sabit tanımlamıştın. Burada iki şey daha öğreneceksin: parametreli makro ve koşullu derleme. Önişlemcinin ürettiği metni görmek için `gcc -E program.c` komutunu kullan.

Parametreli makro

`#define` adının hemen ardına parantezle parametre yazarsan **parametreli makro** (function-like macro) olur. Fonksiyona benzer ama çağrılmaz: önişlemci çağrının yerine gövdeyi, parametreleri yerleştirerek **yazar**. Makro hesap yapmaz, metin yazar; bu yüzden parantezi unutmak ölümcüldür:

makroKare.c

```c
#include <stdio.h>
#define KARE(x) ((x) * (x))
#define KARE_YANLIS(x) x * x

int main(void) {
    printf("%d %.2f\n", KARE(5), KARE(1.5));
    printf("%d %d\n", KARE_YANLIS(2 + 3), KARE(2 + 3));
    printf("%d %d\n", 100 / KARE_YANLIS(5), 100 / KARE(5));
    return 0;
}
```

Çıktı

```
25 2.25
11 25
100 4
```

`KARE(5)`, derleyiciye `((5) * (5))` olarak ulaşır; derleyici `KARE` adını hiç görmez. Makronun tipi yoktur: aynı `KARE` hem `int` hem `double` ile çalıştı. `KARE_YANLIS(2 + 3)` ise `2 + 3 * 2 + 3` olur; çarpma önce işlendiği için 2 + 6 + 3 = 11 çıkar. Üçüncü satırda `100 / 5 * 5` soldan sağa işlenir: 20 * 5 = 100. Kural: **parametrenin her geçişini ve gövdenin tamamını parantez içine al.** Makro adıyla `(` arasına da boşluk koyma: `#define KARE (x) ...` parametresiz bir makro tanımlar.

> **Sınavda Çıkar**
>
> `#define KARE(x) x * x` makrosuyla `KARE(2 + 3)` sorusunun cevabı 25 değil **11**'dir. Makroyu hesap makinesi gibi değil bul-değiştir gibi oku: önce açılmış hâlini yaz, sonra normal öncelikle hesapla.

Yan etki tuzağı

Makro argümanı **metin olarak** kopyalar; parametre gövdede iki kez geçiyorsa argüman iki kez işlenir. `KARE(i++)` şuna dönüşür: `((i++) * (i++))`. Bir değişkeni aynı ifadede iki kez değiştirmek **tanımsız davranıştır** (Bölüm 2), bu yüzden sonucu göstermiyoruz; gcc kodu uyarır:

makroYanEtki.c

```c
#include <stdio.h>
#define KARE(x) ((x) * (x))

int main(void) {
    int i = 3;
    int s = KARE(i++);              // ((i++) * (i++)): tanımsız davranış
    printf("%d %d\n", s, i);
    return 0;
}
```

Derleyici uyarısı — gcc -Wall -Wextra

```
makroYanEtki.c: In function 'main':
makroYanEtki.c:6:19: warning: operation on 'i' may be undefined [-Wsequence-point]
    6 |     int s = KARE(i++);              // ((i++) * (i++)): tanımsız davranış
      |                   ^
makroYanEtki.c:2:25: note: in definition of macro 'KARE'
    2 | #define KARE(x) ((x) * (x))
      |                         ^
```

Fonksiyonda argüman, çağrıdan önce **bir kez** hesaplanır; aynı kod güvenlidir:

fonksiyonYanEtki.c

```c
#include <stdio.h>

int kare(int x) {
    return x * x;
}

int main(void) {
    int i = 3;
    int s = kare(i++);              // kare(3) çağrılır, sonra i artar
    printf("%d %d\n", s, i);
    return 0;
}
```

Çıktı

```
9 4
```

	Makro	Fonksiyon
Kim işler?	Önişlemci (metin değiştirme)	Derleyici
Tip denetimi	Yok	Var
Argüman	Gövdede geçtiği her yerde yeniden yazılır	Çağrıdan önce bir kez hesaplanır
Adresi alınır mı?	Hayır (kod değil metindir)	Evet (Bölüm 9)

Şüphedeysen fonksiyon yaz. Makro, sabitler ve çağrı yerini bilmesi gereken işler (aşağıdaki `__LINE__`) için kalır.

Koşullu derleme

Önişlemci, bir koşula göre kod satırlarını derleyiciye **hiç göndermeyebilir**. Hata ayıklama çıktıları için en çok kullanılan yol budur:

kosulluDerleme.c

```c
#include <stdio.h>
#define HATA_AYIKLA

int main(void) {
    int toplam = 0;

    for (int i = 1; i <= 3; i++) {
        toplam += i;
#ifdef HATA_AYIKLA
        printf("[ayıklama] i = %d, toplam = %d\n", i, toplam);
#endif
    }
    printf("Toplam: %d\n", toplam);
    return 0;
}
```

Çıktı

```
[ayıklama] i = 1, toplam = 1
[ayıklama] i = 2, toplam = 3
[ayıklama] i = 3, toplam = 6
Toplam: 6
```

`#ifdef` ile `#endif` arasındaki satırlar yalnızca `HATA_AYIKLA` adı tanımlıysa derlenir. Programı teslim ederken `#define HATA_AYIKLA` satırını silersen ara çıktıların hepsi birden kaybolur. Adı komut satırından da tanımlayabilirsin; `-D` kaynak dosyaya dokunmadan aynı işi yapar:

```
gcc -std=c17 -Wall -Wextra -DHATA_AYIKLA program.c -o program
```

Komut	Satırlar ne zaman derlenir?
`#ifdef AD` / `#ifndef AD`	`AD` tanımlıysa / tanımlı değilse
`#if ifade`	Sabit ifade sıfırdan farklıysa (tanımsız ad `0` sayılır); değişken kullanılamaz
`#elif ifade`, `#else`	Önceki koşullar sağlanmadıysa
`#endif`	Koşullu bloğu bitirir (her koşul için zorunlu)

`#if 0` … `#endif` bir kod bloğunu geçici olarak devre dışı bırakmanın kolay yoludur (`/* */` yorumları iç içe kullanılamaz).

Başlık koruyucusu ve #pragma once

Bir başlık dosyası aynı kaynağa birden fazla yoldan gelebilir. İçindeki tanımlar iki kez görülürse gcc hata verir; bunu önlemek için her başlığın içeriği **başlık koruyucusu** (include guard) kalıbıyla sarılır:

```
#ifndef NOKTA_H
#define NOKTA_H

/* başlığın içeriği: prototipler, struct tanımları, makrolar */

#endif
```

İlk eklemede `NOKTA_H` tanımlı değildir; içerik alınır ve ad tanımlanır. Sonraki eklemelerde ad tanımlıdır, içerik atlanır. Kısa yolu dosyanın başına yazılan `#pragma once` satırıdır; standart C'nin parçası değildir ama gcc, clang ve MSVC destekler. Çalışan hâlini bir sonraki başlıkta göreceksin.

Hazır makrolar ve assert

Önişlemci birkaç makroyu kendisi tanımlar: `__FILE__` o anki dosyanın adını (metin), `__LINE__` o anki satır numarasını (`int`) verir. Bunlar kodun **yazıldığı yerin** bilgisidir; bu yüzden izleme mesajı veren bir makroda işe yarar:

hazirMakrolar.c

```c
#include <stdio.h>
#define IZLE(mesaj) printf("%s:%d: %s\n", __FILE__, __LINE__, mesaj)

int main(void) {
    IZLE("program başladı");
    IZLE("ortada");
    return 0;
}
```

Çıktı

```
hazirMakrolar.c:5: program başladı
hazirMakrolar.c:6: ortada
```

`__FILE__`, gcc'ye hangi adı verdiysen onu yazar; senin dosyanın adı farklı olacaktır. `assert` (`<assert.h>`) bu makrolardan yararlanır: `assert(koşul)` koşul doğruysa hiçbir şey yapmaz, **yanlışsa** koşulu, dosya adını ve satır numarasını yazıp programı durdurur. "Burada bu doğru olmak zorunda" dediğin varsayımları denetlemenin kısa yoludur:

assertBol.c

```c
#include <assert.h>
#include <stdio.h>

int bol(int a, int b) {
    assert(b != 0);                // varsayım: bölen sıfır olamaz
    return a / b;
}

int main(void) {
    printf("%d\n", bol(10, 2));
    printf("%d\n", bol(5, 0));
    return 0;
}
```

Çıktı – program çöker (Aborted)

```
5
Assertion failed: (b != 0), function bol, file assertBol.c, line 5.
```

Mesajın biçimi sisteme göre değişir (yukarıdaki macOS çıktısıdır) ama koşul, dosya ve satır her zaman yazılır; etiketteki `Aborted`, programın `abort` ile durdurulduğunu gösterir. `assert` **programcı hatasını** yakalamak içindir; kullanıcının yazdığı değeri kontrol etmek için `if` kullan (Bölüm 3). `NDEBUG` adı `<assert.h>`'den **önce** tanımlanırsa (ya da `-DNDEBUG` ile derlersen) bütün `assert`'ler derlemeden silinir. Bu yüzden `assert` içine yan etkili ifade yazma: `assert(i++ > 0)` içindeki `i++` yalnızca `assert` açıkken çalışır.

6.16 Programı dosyalara bölmek

Program büyüdükçe tek dosyada gezinmek zorlaşır. C'de çözüm, programı dosyalara bölmektir: her `.c` dosyası ayrı derlenir, bağlayıcı (Bölüm 1) hepsini birleştirir. Görev paylaşımı sabittir: `.h` **(başlık)** dosyası bildirimleri (prototip, `struct` tanımı, makro, `extern`), `.c` dosyası tanımları (fonksiyon gövdeleri, global değişkenler) taşır. Çok dosyalı örnekler tek bir programa sığmadığı için yalnızca kod kutusu olarak gösterilir, çıktıları metinde söylenir; dosya adları gri başlıklardadır.

topla.h

```c
#ifndef TOPLA_H
#define TOPLA_H

int topla(int a, int b);
int carp(int a, int b);

#endif
```

topla.c

```c
#include "topla.h"

int topla(int a, int b) {
    return a + b;
}

int carp(int a, int b) {
    return a * b;
}
```

main.c

```c
#include <stdio.h>
#include "topla.h"

int main(void) {
    printf("%d\n", topla(3, 4));
    printf("%d\n", carp(3, 4));
    return 0;
}
```

Dosyaları birlikte derleyip çalıştır; çıktı alt alta `7` ve `12` olur:

```
gcc -std=c17 -Wall -Wextra main.c topla.c -o program
./program
```

`main.c` derlenirken derleyici `topla`'nın gövdesini görmez, yalnızca `topla.h`'daki prototipi görür; gövdeyi bağlayıcı `topla.c`'den bulur. `topla.c` de kendi başlığını ekler: prototiple gövde uyuşmazsa derleyici hemen `conflicting types` der. Kendi başlıkların için `#include "topla.h"` (önce o dosyanın klasöründe aranır), sistem başlıkları için `#include <stdio.h>` (sistem klasörlerinde aranır) yaz.

> **Sık Yapılan Hata: `.c` dosyasını `#include` etmek**
>
> Başlıklar eklenir, `.c` dosyaları derlemeye verilir. `main.c` içine `#include "topla.c"` yazıp `topla.c`'yi de derlemeye verirsen `topla` iki dosyada tanımlanmış olur ve bağlayıcı `multiple definition of 'topla'` der. Aynı sebeple başlığa **tanım** (`int sayac = 0;`, fonksiyon gövdesi) yazma: başlık birden çok `.c`'ye girer.

Başlık koruyucusunun çalışması

`#include`, başlığın metnini o yere yapıştırmaktır; aynı başlık iki yoldan gelirse metin iki kez yapışır. Bunu tek dosyada elle yapalım. Önce koruyucusuz (`struct` Bölüm 11'de gelecek; şimdilik "`x` ve `y` taşıyan bir tür" diye oku):

koruyucusuz.c

```c
struct Nokta { int x; int y; };
struct Nokta { int x; int y; };     // nokta.h ikinci kez eklenmiş gibi

int main(void) {
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
koruyucusuz.c:2:8: error: redefinition of 'struct Nokta'
    2 | struct Nokta { int x; int y; };     // nokta.h ikinci kez eklenmiş gibi
      |        ^~~~~
koruyucusuz.c:1:8: note: originally defined here
    1 | struct Nokta { int x; int y; };
      |        ^~~~~
```

Aynı iki yapıştırma koruyucuyla yapılınca ikinci kopya atlanır:

koruyuculu.c

```c
#include <stdio.h>

#ifndef NOKTA_H                     // nokta.h, birinci ekleme
#define NOKTA_H
struct Nokta { int x; int y; };
#endif

#ifndef NOKTA_H                     // nokta.h, ikinci ekleme: atlanır
#define NOKTA_H
struct Nokta { int x; int y; };
#endif

int main(void) {
    struct Nokta n = {3, 4};

    printf("%d %d\n", n.x, n.y);
    return 0;
}
```

Çıktı

```
3 4
```

`stdio.h` gibi sistem başlıkları da böyle korunur; `#include <stdio.h>` satırını iki kez yazsan bile sorun çıkmaz.

extern ve dosya kapsamlı static

Fonksiyon dosyalar arasında prototiple paylaşılır; **global değişkenin** karşılığı `extern`'dir. **Bildirim** ile **tanımı** ayır: `extern int sayac;` bildirimdir (yer ayırmaz, istediğin kadar yazılır); `int sayac = 0;` tanımdır (bellekte yer ayırır, bütün programda **bir** `.c` dosyasında bulunur).

sayac.c

```c
int sayac = 0;                  // tanım: bellekte yer burada ayrılır

void artir(void) {
    sayac++;
}
```

main.c

```c
#include <stdio.h>

extern int sayac;              // bildirim: sayac başka bir dosyada tanımlı
void artir(void);

int main(void) {
    artir();
    artir();
    sayac += 10;
    printf("%d\n", sayac);
    return 0;
}
```

`gcc main.c sayac.c -o program` ile derleyince çıktı `12` olur. Gerçek projede bu `extern` satırı `sayac.h` içine yazılır ve `main.c` onu `#include` eder. Yine de global değişken paylaşmak, her dosyanın onu değiştirebilmesi demektir; mümkünse değişkeni gizleyip fonksiyonla eriş.

Gizlemenin aracı, fonksiyonların **dışında** yazılan `static`'tir. Yerel `static`'in değeri koruduğunu bu bölümde görmüştün; burada anlam başkadır: ad yalnızca tanımlandığı `.c` dosyasına özel olur. Buna **iç bağlantı** (internal linkage) denir; bağlayıcı o adı başka dosyalara göstermez.

sayac.c

```c
static int sayac = 0;          // yalnız bu dosyada görünür

void artir(void) {
    sayac++;
}

int sayacDegeri(void) {        // dışarıya okuma izni fonksiyonla verilir
    return sayac;
}
```

Bu `sayac.c` ile `main.c` içinde `artir()` ve `sayacDegeri()` sorunsuz çalışır. Ama `main.c` içinde `extern int sayac;` yazıp değişkene doğrudan dokunursan program derlenir, **bağlama** aşamasında hata verir. Mesajı elle yazdık (gcc/MinGW biçimi; macOS farklı yazar, uzun yol kısaltıldı):

```
/usr/bin/ld: /tmp/ccXXXXXX.o: in function `main':
main.c:(.text+0x...): undefined reference to 'sayac'
collect2: error: ld returned 1 exit status
```

Aynı kural fonksiyonlara da uygulanır: `static int yardimci(int x)` yalnız o dosyada çağrılabilir; iki dosyada aynı adlı yardımcılar çakışmaz.

Bölüm Özeti – Bilmen Gerekenler

- Fonksiyon: `dönüşTipi ad(parametreler) { gövde }`. Tekrarı önler, okunabilirliği artırır, problemi parçalara böler.
- `void` fonksiyon değer döndürmez (`return;` ile erken çıkabilir); çağrısı bir değişkene atanamaz. Diğerleri `return` ile **en fazla bir** değer verir; `return` çalıştığı anda fonksiyon biter.
- Fonksiyon çağrılmadan önce **tanıtılmış** olmalıdır: tanımı `main`'den önce yaz ya da **prototip** (`int f(int);`) ekle. Prototipsiz çağrı `implicit declaration of function` hatasıdır; prototiple tanım uyuşmazsa `conflicting types`.

- **Parametre** tanımdaki, **argüman** çağrıdaki değerdir. Sayı ve sıra uyuşmalıdır; tip kendiliğinden çevrilir (`double` → `int` ondalığı atar). `(a + b) / 2` tamsayı bölmesidir.
- C'de argümanlar **değer ile** geçirilir: fonksiyon kopya alır, `takas(x, y)` çağıranın değişkenlerini değiştirmez (çözüm Bölüm 9'da).
- Her çağrı yığında kendi **çerçevesini** açar, fonksiyon bitince çerçeve silinir. İç içe çağrıda içteki önce çalışır.
- **Kapsam:** yerel değişken yalnızca kendi bloğunda yaşar; iç blokta aynı ad dıştakini **gölgeler**. Global değişken her yerden görünür, ilk değeri 0'dır ve az kullanılmalıdır.
- `static` yerel değişken çağrılar arasında değerini korur, ilk değeri bir kez verilir, başlatılmazsa 0'dır.
- Değer döndüren fonksiyonun her yolu `return` ile bitmelidir; yoksa `-Wreturn-type` uyarısı ve tanımsız dönüş değeri.
- `abs` `int`, `fabs` `double` içindir. `rand() % n` 0..n-1 verir (zar: `rand() % 6 + 1`); `srand(time(NULL))` programın başında **bir kez** çağrılır; `rand` çıktısı sistemlere göre değişir.
- Algoritma → fonksiyon: girdiler parametre, sonuç `return`, `printf`/`scanf` `main`'de. `main` de bir fonksiyondur: `return 0;` başarı, sıfırdan farklı değer hata.- **Önişlemci:** parametreli makro metin değiştirir; parametreyi ve gövdeyi parantezle (`x * x` ile `KARE(2 + 3)` 11 çıkar), `KARE(i++)` tanımsız davranıştır, şüphede fonksiyon yaz. `#ifdef` / `#ifndef` / `#if` / `#else` / `#endif` koşullu derler (`-DAD` komut satırından tanımlar). `assert(koşul)` yanlışsa dosya ve satırı yazıp programı durdurur, `NDEBUG` onu siler; `__FILE__` ve `__LINE__` hazır makrolardır.
- **Dosyalara bölme:** `.h` bildirim (prototip, `extern`) ve başlık koruyucusu (`#ifndef` ya da `#pragma once`), `.c` tanım taşır; kendi başlığın `"a.h"`, sistem başlığı `<stdio.h>`. Derleme: `gcc main.c topla.c -o program`. `extern int x;` bildirir, `int x = 0;` tanımlar (tek `.c`'de). Dosya düzeyinde `static` adı o dosyaya gizler (iç bağlantı).

Sorular

A) Çoktan Seçmeli

1 `double ortalama(int a, int b)` fonksiyonu `main`'den **sonra** tanımlanmıştır. `main` içinde sorunsuz çağrılabilmesi için `main`'den önce hangi satır yazılmalıdır?

(A) `double ortalama;`
(B) `ortalama(int a, int b);`
(C) `double ortalama(int, int);`
(D) `void ortalama(int a, int b);`
(E) `double ortalama(int a, int b) { }`

2 C'de bir fonksiyona argüman geçirilmesiyle ilgili aşağıdakilerden hangisi **doğrudur**?

(A) Fonksiyon argümanın bir kopyasını alır; parametreyi değiştirmek çağıran fonksiyondaki değişkeni değiştirmez.
(B) Parametre fonksiyon içinde değiştirilirse çağıran fonksiyondaki değişken de aynı değeri alır.
(C) Yalnızca `void` fonksiyonlarda yapılan değişiklikler çağıranı etkiler.
(D) Fonksiyonlar yalnızca sabit değerleri argüman olarak alabilir; değişken geçirilemez.
(E) Parametrenin adı çağırandaki değişkenle aynıysa ikisi aynı bellek hücresini paylaşır.

3 Aşağıdaki `say` fonksiyonu `main` içinde art arda üç kez çağrılıp döndürdüğü değerler sırayla yazdırılırsa ekranda ne görülür?

```c
int say(void) {
    static int n = 10;
    n += 5;
    return n;
}
```

(A) `15 15 15` (B) `10 15 20` (C) `15 25 35` (D) `10 10 10` (E) `15 20 25`

4 `int topla(int a, int b) { return a + b; }` tanımıyla birlikte, tanımdan önce yazılan aşağıdaki prototiplerden hangisi **derleme hatasına** yol açar?

(A) `int topla(int a, int b);`
(B) `int topla(int x, int y);`
(C) `int topla(int, int);`
(D) `int topla(int a, int b, int c);`
(E) Hiçbiri; hepsi geçerlidir.

5 Dönüş tipi `void` olan bir fonksiyonla ilgili aşağıdakilerden hangisi **doğrudur**?

(A) `return 5;` yazarak değer döndürebilir.
(B) `return;` yazılarak fonksiyon erken bitirilebilir.
(C) Çağrısı bir `int` değişkene atanabilir.
(D) `main` dışında `void` fonksiyon tanımlanamaz.
(E) Parametre alamaz.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```c
int f(void) {
    static int a = 1;
    int b = 1;
    a = a * 2;
    b = b * 2;
    return a + b;
}

int main(void) {
    for (int i = 0; i < 3; i++) {
        printf("%d ", f());
    }
    printf("\n");
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
int f(int x) {
    x = x + 1;
    return x * 2;
}

void g(int x) {
    x = x * 10;
}

int main(void) {
    int a = 3;
    int b = f(a);
    g(a);
    printf("%d %d\n", a, b);
    printf("%d\n", f(f(a)));
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır? Her sayıdan sonra bir boşluk olduğuna, son sayıdan sonra ise satır sonu olduğuna dikkat et.

```c
int x = 5;

void yaz(void) {
    printf("%d ", x);
}

void degistir(void) {
    int x = 50;
    printf("%d ", x);
    yaz();
}

int main(void) {
    int x = 1;
    yaz();
    degistir();
    x = x + 10;
    yaz();
    {
        int x = 100;
        printf("%d ", x);
        yaz();
    }
    printf("%d\n", x);
    return 0;
}
```

C) Kod Yazma

9 Bir sayının, **kendisi hariç** bölenlerinin toplamı kendisine eşitse sayıya **mükemmel sayı** denir (`6 = 1 + 2 + 3`). `bool mukemmelMi(int n)` fonksiyonunu yaz. `main`, bir üst sınır okuyup 1'den bu sınıra kadar olan mükemmel sayıları yazdırsın. Fonksiyon yazdırma ya da okuma yapmasın. Örnek (`500` girildiğinde):

```
Üst sınır: 500
Mükemmel sayılar: 6 28 496
```

10 Üç sınav notunu (`int`) okuyan bir program yaz. `double ortalama(int a, int b, int c)` üç notun ortalamasını (ondalıklı) hesaplasın. `char harfNotu(double puan)` ortalamaya göre harf döndürsün: 90 ve üstü `A`, 80 ve üstü `B`, 70 ve üstü `C`, 60 ve üstü `D`, altı `F`. Ortalamayı iki ondalıkla ve harf notunu yazdır. Örnek (`70 85 92` girildiğinde):

```
Notlar: 70 85 92
Ortalama: 82.33
Harf notu: B
```

Çözümler

Cevap anahtarı

1 C	2 A	3 E	4 D	5 B

1 Cevap: C

Prototip, tanımın gövdesiz hâlidir: dönüş tipi, ad, parametre **tipleri**, sonunda `;`; parametre adları yazılmayabilir. A bir değişken bildirimidir. B'de dönüş tipi yok (C99'dan beri geçersiz). D'de dönüş tipi `void` olduğundan `conflicting types` hatası verir. E bir tanımdır, prototip değildir; gerçek tanımla birlikte fonksiyon iki kez tanımlanmış olur.

2 Cevap: A

C'de argümanlar **değer ile** geçirilir; fonksiyon bir kopya üzerinde çalışır. B tam tersini söylüyor (ancak adres geçirilirse olur; Bölüm 9). C yanlış: dönüş tipi bunu değiştirmez. D yanlış: değişken de, ifade de argüman olabilir. E yanlış: aynı adlı parametre ile çağıran değişken farklı çerçevelerde yaşayan iki ayrı değişkendir.

3 Cevap: E

`static int n = 10;` satırı yalnızca ilk çağrıda işlenir. `n` sırayla 10 → 15 (döner 15), 15 → 20 (döner 20), 20 → 25 (döner 25) olur. A, `static` olmasaydı çıkacak sonuçtur (her çağrıda `n = 10`'dan başlar: `15 15 15`). B, değeri artırmadan önce döndürüldüğünü sanmaktır. C her çağrıda 10 eklendiğini sanmaktır. D hiç değişmediğini varsayar.

Çıktı

```
15 20 25
```

4 Cevap: D

Prototip ile tanım aynı tip ve sayıda parametre taşımalıdır. D'de üç parametre vardır, tanımda iki; gcc `conflicting types for 'topla'` hatası verir. A özdeştir. B'de parametre adları farklıdır, adların önemi yoktur. C'de ad hiç yazılmamıştır, o da geçerlidir. E yanlıştır, çünkü D hata verir.

5 Cevap: B

`void` fonksiyonda `return;` (değersiz) yazılabilir ve fonksiyonu hemen bitirir. A yanlış: `void` fonksiyon değer döndüremez. C yanlış: `void value not ignored as it ought to be`. D yanlış: istediğin kadar `void` fonksiyon tanımlayabilirsin (`menuyuGoster`, `bol`). E yanlış: `void` dönüş tipidir, parametreyle ilgisi yoktur; `void bol(int pay, int payda)` geçerlidir.

6

`a` `static`, `b` normal yereldir. `a` çağrılar arasında kalır, `b` her çağrıda 1'den başlar.

Çağrı	a önce	b önce	a sonra	b sonra	Döner
1	1	1	2	2	4
2	2	1	4	2	6
3	4	1	8	2	10

Her sayıdan sonra bir boşluk yazılır; en sonda `\n` alt satıra geçirir.

Çıktı
```
4 6 10
```

7

`f` ve `g` argümanın kopyasıyla çalışır; `main`'deki `a` hep 3 kalır. `f(a)`: `x = 4`, döner `8`, yani `b = 8`. `g(a)` kendi kopyasını 30 yapar ve hiçbir şey döndürmez. İlk `printf` `3 8` yazar. `f(f(a))`: içteki önce, `f(3) = 8`; sonra `f(8)`: `x = 9`, döner `18`.

Çıktı
```
3 8
18
```

8

`yaz` fonksiyonu global `x`'i yazdırır; global `x` hiç değişmez ve hep 5'tir. Yerel `x`'ler yalnızca kendi bloklarında görünür. Sırayla: `yaz()` → `5`. `degistir()`: yerel `x` 50 olur ve `50` yazılır, sonra `yaz()` yine `5`. `main`'de `x = x + 10` yerel `x`'i 11 yapar (global değişmez); `yaz()` yine `5`. İç blokta `int x = 100;` dıştaki `x`'i gölgeler: `100` yazılır, `yaz()` yine `5`. Blok bitince `main`'in `x`'i yazılır: `11`.

Çıktı
```
5 50 5 5 100 5 11
```

9

`mukemmelMi` bölenleri 1'den `n / 2`'ye kadar tarar (kendisi dışındaki en büyük bölen en fazla `n / 2`'dir), toplamı sıfırdan başlatır ve `toplam == n` sonucunu doğrudan `bool` olarak döndürür. `main` yalnızca okur, çağırır ve yazdırır.

mukemmel.c
```c
#include <stdio.h>
#include <stdbool.h>

bool mukemmelMi(int n) {
    if (n < 2) {
        return false;
    }
    int toplam = 0;
    for (int i = 1; i <= n / 2; i++) {
        if (n % i == 0) {
            toplam += i;
        }
    }
    return toplam == n;
}

int main(void) {
    int sinir = 0;
    printf("Üst sınır: ");
    scanf("%d", &sinir);
    printf("Mükemmel sayılar:");
    for (int n = 1; n <= sinir; n++) {
        if (mukemmelMi(n)) {
            printf(" %d", n);
        }
```

```
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
Üst sınır: 500
Mükemmel sayılar: 6 28 496
```

10

`ortalama`, bölmeyi `double` yapmak için `3.0`'a böler (`/ 3` yazarsan tamsayı bölmesi olur ve `82.00` çıkardı). `harfNotu` en büyük eşikten başlayarak erken `return` yapar; böylece `else` gerekmez ve son satırdaki `return 'F';` bütün yolların bir değerle bittiğini garanti eder. Harf `%c` ile yazdırılır.

notHesabi.c

```c
#include <stdio.h>

double ortalama(int a, int b, int c) {
    return (a + b + c) / 3.0;
}

char harfNotu(double puan) {
    if (puan >= 90) {
        return 'A';
    }
    if (puan >= 80) {
        return 'B';
    }
    if (puan >= 70) {
        return 'C';
    }
    if (puan >= 60) {
        return 'D';
    }
    return 'F';
}

int main(void) {
    int n1 = 0, n2 = 0, n3 = 0;
    printf("Notlar: ");
    scanf("%d %d %d", &n1, &n2, &n3);
    double ort = ortalama(n1, n2, n3);
    printf("Ortalama: %.2f\n", ort);
    printf("Harf notu: %c\n", harfNotu(ort));
    return 0;
}
```

Çıktı

```
Notlar: 70 85 92
Ortalama: 82.33
Harf notu: B
```

BÖLÜM 7

Diziler

- **Bu Bölümde Öğreneceklerin**
  - Dizinin ne olduğunu, tanımlamayı, `#define` ile boyutu ve elemanların bellekte ardışık durmasını
  - İlk değer listesini: eksiklerin `0` olmasını, `{0}` ile sıfırlamayı, `int a[] = {...}` kalıbını
  - İndeksin `0`'dan başladığını, son indeksin `N-1` olduğunu ve C'nin sınır taşmasını denetlemediğini
  - `sizeof(a) / sizeof(a[0])` ile eleman sayısını ve bunun fonksiyon içinde neden çalışmadığını
  - Diziyi `for` ile dolaşmayı, `scanf` ile doldurmayı; toplam, ortalama, en büyük/en küçük, sayma
  - Diziyi fonksiyona geçirmeyi, değişikliğin çağırana yansımasını ve `const` parametreyi
  - Kopyalama (`memcpy`), `b = a` ile `a == b` tuzakları, ters çevirme, kaydırma, doğrusal ve ikili arama
  - Kabarcık ve seçmeli sıralama; iki boyutlu diziler (matris, transpoz, köşegen, fonksiyona geçirme)

7.1 Neden dizi? Tanımlama ve bellek

50 öğrencinin notunu tutan bir program yazacaksın. Şimdiye kadarki bilgilerinle `int not1, not2, ..., not50;` diye elli değişken tanımlarsın, ortalama için elli terimli bir toplama yazarsın. Döngü (Bölüm 5) yardım edemez, çünkü `not1`, `not2` adlarını döngüye sokamazsın: "`i`'inci değişken" demenin yolu yok.

Çözüm **dizi**dir (array): aynı türden birçok değeri **tek bir adla** tutar, her değere **sıra numarasıyla** (indeks, *index*) ulaşmanı sağlar. Her değere dizinin **elemanı** denir.

```
tip dizi_adi[boyut];
```

- **tip:** Bütün elemanların ortak türü: `int notlar[50];`, `double fiyat[10];`, `char harf[26];` (karakter dizilerini Bölüm 8'de göreceksin).
- **boyut:** Eleman sayısı. Derleme sırasında bilinen, sıfırdan büyük bir **tamsayı sabiti** olmalıdır; Bölüm 2'deki gibi `#define N 5` ile adlandırılır. `int a[2.5]` geçersizdir (gcc hata verir); `int a[0]` da standartta geçersizdir, gcc onu bir eklenti olarak sessizce derlese de kullanma. Döngü sınırlarını da sayıyla değil `N` ile yaz: boyutu tek yerden değiştirirsin. Dizi tanımlandıktan sonra **büyüyemez ve küçülemez**.

ilkDizi.c

```c
#include <stdio.h>
#define N 4

int main(void) {
    int notlar[N];
    notlar[0] = 70;
    notlar[1] = 85;
    notlar[2] = 90;
    notlar[3] = 55;
    int i = 2;
    printf("İlk not: %d, son not: %d\n", notlar[0], notlar[N - 1]);
    printf("notlar[i - 1] + notlar[i + 1] = %d\n", notlar[i - 1] + notlar[i + 1]);
    printf("Dizi: %zu bayt\n", sizeof(notlar));
    return 0;
}
```

Çıktı

```
İlk not: 70, son not: 55
notlar[i - 1] + notlar[i + 1] = 140
Dizi: 16 bayt
```

`notlar[2]` dizinin 2 numaralı elemanıdır ve **sıradan bir `int` değişkeni gibi** kullanılır: `notlar[2] = 90;`, `notlar[2]++`, `scanf("%d", &notlar[2])` geçerlidir. `sizeof(notlar)` bütün dizinin bayt sayısıdır: 4 eleman × 4 bayt = 16. Elemanlar bellekte **boşluksuz, art arda** durur; bu yüzden `notlar[i]`'ye erişmek bir toplama ve bir çarpma ile olur (dizi hızlıdır) ve aynı düzen Bölüm 9'da işaretçileri anlamanın anahtarı olacak:

İndeks	`notlar[0]`	`notlar[1]`	`notlar[2]`	`notlar[3]`
Değer	70	85	90	55
Başlangıçtan uzaklık	0 bayt	4 bayt	8 bayt	12 bayt

Not: Değişken uzunluklu diziler (VLA)

C99'dan beri boyut, çalışma anında bilinen bir değişkenle de verilebilir: `int n; scanf("%d", &n); int a[n];`. Buna **değişken uzunluklu dizi** (variable length array, VLA) denir. C11'de VLA desteği **isteğe bağlı** oldu: her derleyici sunmak zorunda değildir, VLA'ya ilk değer listesi verilemez ve çok büyük boyut programı çökertir. Sınavda ve bu kitapta **sabit boyut** kullan; boyutu çalışma anında belirlenen diziler için doğru çözüm Bölüm 10'daki dinamik bellektir.

Sık Yapılan Hata: `const int` dizi boyutu olamaz

`const` bir değişkeni **salt okunur** yapar ama **sabit ifade** yapmaz (Bölüm 4'te `case` etiketinde de böyleydi). Standart C'de `const int N = 5;` yazıp `int a[N];` demek, boyutu bir değişkenle vermektir. Dosya düzeyinde (global) böyle bir dizi geçersizdir. Aşağıdaki `#pragma` satırı gcc'ye "standarda uymayan koda hata ver" der:

constBoyut.c

```c
#pragma GCC diagnostic error "-Wpedantic"

const int N = 5;
int a[N];                          // N sabit ifade değil

int main(void) {
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
constBoyut.c:4:1: error: variably modified 'a' at file scope [-Werror=pedantic]
    4 | int a[N];                          // N sabit ifade değil
      | ^~~
cc1: some warnings being treated as errors
```

`#pragma` olmadan gcc bu kodu bir eklenti olarak **sessizce derler**; bunu kanıt sayma, çünkü standart C'de geçersizdir ve sınavda "derlenmez" diye bilinir. Fonksiyonun içinde aynı yazım VLA olur ve VLA'ya `= {0}` ile ilk değer verilemez (gcc bunu da hoşgörebilir). Çözüm: `#define N 5` ya da adsız bir `enum { N = 5 };` (Bölüm 11'de göreceksin).

7.2 İlk değer verme

Tanım satırında küme parantezi içinde **ilk değer listesi** verebilirsin. Biçimlerin hepsi tek programda:

ilkDegerler.c

```c
#include <stdio.h>

int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int b[5] = {1, 2, 3};
    int c[5] = {0};
    int d[5] = {7};
    int e[] = {4, 8, 15, 16, 23, 42};

    printf("i   a   b   c   d\n");
    for (int i = 0; i < 5; i++) {
        printf("%d %3d %3d %3d %3d\n", i, a[i], b[i], c[i], d[i]);
    }
    printf("e[5] = %d\n", e[5]);
    return 0;
}
```

Çıktı

```
i   a   b   c   d
0  10   1   0   7
1  20   2   0   0
2  30   3   0   0
3  40   0   0   0
4  50   0   0   0
e[5] = 42
```

- **Liste boyuttan kısaysa** kalan elemanlar `0` olur (`b`). Çöp değil, `0`!
- **`{0}` bütün diziyi sıfırlar** (`c`): ilk eleman `0`, kalanlar eksik olduğu için `0`.
- **`{7}` hepsini 7 yapmaz** (`d`): yalnız ilk eleman 7'dir, kalanlar `0`.

- **Boyut yazılmazsa** derleyici liste uzunluğunu boyut yapar (`e`: 6 eleman, indeksler 0–5).
- Liste **yalnızca tanım satırında** yazılır; `a = {1, 2, 3};` sonradan yazılamaz (7.8). Fazla elemanlı liste (`int b[3] = {1, 2, 3, 4};`) gcc'den `excess elements in array initializer` uyarısı alır.

> **Sınavda Çıkar**
>
> `int a[5] = {1};` ve `int a[5] = {0};` sorusunun cevabı: ilki `1 0 0 0 0`, ikincisi `0 0 0 0 0`. "Liste kısa olunca kalan elemanlar çöp değer alır" cümlesi **yanlıştır**.

İlk değer listesi olmayan yerel dizi, Bölüm 2'deki ilk değersiz değişken gibi **çöp değer** taşır: `int x[5]; printf("%d", x[0]);` tanımsızdır, çıktısı bazen `0` bazen başka bir sayıdır (bu yüzden çıktı göstermiyoruz). Dizi, **okunmadan önce** değer almalıdır (liste ya da döngüyle). Global ve `static` diziler istisnadır: onlar `0` ile başlar.

7.3 İndeks ve sınır taşması

İndeks **0'dan başlar**: `N` elemanlı dizide ilk eleman `a[0]`, son eleman `a[N-1]`'dir. İndeks sabit olmak zorunda değildir; `a[i]`, `a[i + 1]` (`ilkDizi`'de gördün) ya da `a[b[j]]` gibi her tamsayı ifadesi olabilir. Önce indeks ifadesi hesaplanır, sonra o eleman kullanılır; iç içe indekslerde (`a[a[0]]`) **içten dışa** çöz.

Sınır taşması

`int a[5]` için `a[5]`, `a[-1]`, `a[100]` **geçerli değildir**; dizinin dışındaki belleğe uzanırlar. Kritik nokta: **C bunu denetlemez.** Şu satırlar derlenir:

```c
int a[5] = {10, 20, 30, 40, 50};
a[5] = 99;                  // geçersiz: son indeks 4
a[-1] = 7;                  // geçersiz: indeks negatif
printf("%d\n", a[100]);     // geçersiz: okumak bile
```

Bu bir **tanımsız davranıştır** (undefined behavior). Okumada çöp ya da komşu değişkenin değeri gelir; yazmada **komşu bir değişken sessizce bozulur** ve hata programın ilgisiz bir yerinde ortaya çıkar; program `Segmentation fault` ile çökebilir (Linux ve macOS terminalinin yazdığı mesaj; Windows'ta program çoğu zaman sessizce kapanır); ya da hiçbir şey olmamış gibi **doğru çalışıyormuş** görünür. Java ve Python gibi dillerde sınır dışı erişim programı hata ile durdurur; C'de denetim programcıya aittir. gcc yüksek eniyileme düzeylerinde bazı durumları `-Warray-bounds` uyarısıyla yakalar, ama buna güvenme. Birçok gerçek güvenlik açığı (*buffer overflow*) dizi sınırını aşan yazmalardan doğmuştur.

> **Sık Yapılan Hata: Döngüde `<=` yazmak**
>
> Sınır taşmasının en yaygın nedeni, Bölüm 5'teki off-by-one hatasıdır: `for (int i = 0; i <= N; i++)` yazarsan son turda `i == N` olur ve `a[N]` dizinin dışına çıkar. Kalıbı ezberle: `for (int i = 0; i < N; i++)`. `int a[10]` dizisinin son elemanı `a[9]`'dur; `a[10]` yoktur. "C sınır dışı erişimde hata verir" cümlesi sınavda hep yanlış şıktır.

7.4 Eleman sayısı: sizeof

Eleman sayısını elle yazmak yerine, dizinin bayt sayısını bir elemanın bayt sayısına bölerek bulursun: `sizeof(a) / sizeof(a[0])`. Bölen `sizeof(int)` yerine `sizeof(a[0])` yazılır, çünkü dizinin türü sonradan değişse formül doğru kalır. `sizeof` bir `size_t` üretir, `%zu` ile yazdırılır (Bölüm 2). Formül **yalnız dizinin tanımlandığı fonksiyonda** çalışır:

sizeofTuzak.c

```c
#include <stdio.h>

void yanlis(int a[]) {
    int n = sizeof(a) / sizeof(a[0]);
    printf("Fonksiyon içinde n = %d\n", n);
}

int main(void) {
    int dizi[6] = {1, 2, 3, 4, 5, 6};
    int n = sizeof(dizi) / sizeof(dizi[0]);
    printf("Dizi %zu bayt, eleman %zu bayt\n", sizeof(dizi), sizeof(dizi[0]));
    printf("main içinde n = %d\n", n);
    yanlis(dizi);
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
sizeofTuzak.c: In function 'yanlis':
sizeofTuzak.c:4:19: warning: 'sizeof' on array function parameter 'a' will return size of 'int *' [-
Wsizeof-array-argument]
    4 |     int n = sizeof(a) / sizeof(a[0]);
      |                   ^
sizeofTuzak.c:3:17: note: declared here
    3 | void yanlis(int a[]) {
      |             ~~~~^~~
```

Çıktı

```
Dizi 24 bayt, eleman 4 bayt
main içinde n = 6
Fonksiyon içinde n = 2
```

Fonksiyon dizinin kendisini değil **adresini** alır (nedenini Bölüm 9'da göreceksin); `int a[]` yazılsa da `a` bir adres tutar ve `sizeof(a)` adresin boyutunu verir. 64 bit bilgisayarlarda adres 8 bayt olduğu için 8 / 4 = 2 çıkar (32 bit sistemde 1 çıkardı; senin bilgisayarında farklı olabilir). Sonuç ne olursa olsun yanlıştır. **Çözüm:** eleman sayısını dizinin tanımlandığı yerde hesapla, fonksiyona ayrı bir `int n` parametresiyle geçir.

7.5 Dolaşma, doldurma ve yazdırma

Dizinin omurgası `for` döngüsüdür; `i` indeks olur: baştan sona `for (int i = 0; i < N; i++)`, sondan başa `for (int i = N - 1; i >= 0; i--)`, bir atlayarak `i += 2`. Klavyeden doldururken `scanf`'e elemanın **adresi** (`&notlar[i]`) verilir; `&` unutulursa `scanf` değeri adres sanır (gcc uyarır, program çoğu zaman çöker). Sütunları hizalamak için `%2d`, `%3d` gibi genişlikler kullanılır (Bölüm 3). Kullanıcıya `i + 1` gösteriyoruz (insan 1'den sayar), dizide 0'dan başlayan `i` kullanıyoruz:

notGirisi.c

```c
#include <stdio.h>
#define N 4

int main(void) {
    int notlar[N];
    for (int i = 0; i < N; i++) {
        printf("%d. notu gir: ", i + 1);
        scanf("%d", &notlar[i]);
    }

    printf("\nNo  Not\n");
    for (int i = 0; i < N; i++) {
        printf("%2d  %3d\n", i + 1, notlar[i]);
    }
    return 0;
}
```

Çıktı

```
1. notu gir: 70
2. notu gir: 85
3. notu gir: 90
4. notu gir: 100

No  Not
 1   70
 2   85
 3   90
 4  100
```

7.6 Temel algoritmalar

Sınavdaki dizi sorularının çoğu beş kalıba iner: toplam, ortalama, en büyük/en küçük (ve indeksi), sayma, koşula uyanları bulma. Hepsi tek programda:

istatistik.c

```c
#include <stdio.h>
#define N 8

int main(void) {
    int notlar[N] = {72, 45, 88, 60, 95, 38, 81, 67};
    int toplam = 0;
    int enBuyuk = notlar[0];
    int enBuyukIndeks = 0;
    int enKucuk = notlar[0];
    int enKucukIndeks = 0;
    int gecen = 0;
    for (int i = 0; i < N; i++) {
        toplam += notlar[i];
        if (notlar[i] > enBuyuk) {
            enBuyuk = notlar[i];
            enBuyukIndeks = i;
        }
        if (notlar[i] < enKucuk) {
            enKucuk = notlar[i];
            enKucukIndeks = i;
        }
        if (notlar[i] >= 60) {
            gecen++;
        }
```

```
    }
    double ortalama = (double) toplam / N;
    printf("Toplam: %d, ortalama: %.2f\n", toplam, ortalama);
    printf("En büyük: %d (indeks %d)\n", enBuyuk, enBuyukIndeks);
    printf("En küçük: %d (indeks %d)\n", enKucuk, enKucukIndeks);
    printf("Geçen: %d kişi\n", gecen);
    printf("Ortalamanın üstündekiler:");
    for (int i = 0; i < N; i++) {
        if (notlar[i] > ortalama) {
            printf(" %d", notlar[i]);
        }
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
Toplam: 546, ortalama: 68.25
En büyük: 95 (indeks 4)
En küçük: 38 (indeks 5)
Geçen: 6 kişi
Ortalamanın üstündekiler: 72 88 95 81
```

- **Toplam** `0` ile başlar (Bölüm 5). **Ortalama** için bölenlerden biri `double`'a çevrilmelidir: `toplam / N` yazarsan 546 / 8 tamsayı bölmesi olur ve sonuç 68.25 değil `68` çıkar.
- **En büyük/en küçük** için aday **dizinin ilk elemanıyla** başlatılır (`0` ile değil: bütün sayılar negatifse `0` hep kazanır) ve döngü `1`'den başlayabilir (bu programda `toplam` da aynı döngüde biriktiği için `0`'dan başlıyor; ilk eleman kendisiyle karşılaştırılır, zararsız). İndeksi de saklarsan "kim en yüksek?" sorusuna cevap verebilirsin. Koşulda `>` kullandığın için tekrar eden en büyük değerlerde **ilk** bulunanın indeksi kalır (`>=` olsaydı sonuncusu).
- **Sayma**, Bölüm 5'teki sayaç kalıbıdır: koşul doğruysa `gecen++`.
- **Ortalamanın üstündekiler iki geçiş** ister: ortalama bilinmeden kimin üstte olduğu bilinemez. Önce ortalama hesaplanır, sonra dizi yeniden gezilir. `notlar[i] > ortalama` karşılaştırmasında `int`, `double`'a çevrilir; 68.25'in üstünde 72, 88, 95 ve 81 kalır.

7.7 Diziler ve fonksiyonlar

Dizi parametresi köşeli parantezli ama **boyutsuz** yazılır; eleman sayısı 7.4'teki nedenle ayrı bir `int n` parametresidir. Çağrıda ise dizi **yalnız adıyla** verilir: `yazdir(notlar, N)`.

diziFonksiyon.c

```
#include <stdio.h>
#define N 5

void yazdir(const int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

void ikiyeKatla(int a[], int n) {
    for (int i = 0; i < n; i++) {
        a[i] *= 2;
    }
}

void artir(int x) {
    x++;
```

```
    printf("artir içinde x = %d\n", x);
}

int main(void) {
    int a[N] = {1, 2, 3, 4, 5};
    int sayi = 10;
    ikiyeKatla(a, N);
    yazdir(a, N);
    artir(sayi);
    printf("main içinde sayi = %d\n", sayi);
    return 0;
}
```

Çıktı

```
2 4 6 8 10
artir içinde x = 11
main içinde sayi = 10
```

`ikiyeKatla` diziyi değiştirdi ve **değişiklik `main`'deki dizide göründü**; `artir` ise kendi `x`'ini artırdı, `main`'deki `sayi` 10 kaldı. Bölüm 6'daki değer ile geçirme kuralıyla çelişmiyor:

	`int x` parametresi	`int a[]` parametresi
Fonksiyona giden	Değerin **kopyası**	Dizinin **adresi**
Fonksiyonda yapılan değişiklik	Çağırana **yansımaz**	Çağırana **yansır**

Büyük bir diziyi her çağrıda kopyalamak yavaş olurdu; C bu yüzden diziyi adres olarak geçirir ve fonksiyon **gerçek dizi üzerinde** çalışır (ayrıntısı Bölüm 9'da). Bölüm 6'da çalışmayan `swap(int a, int b)`'nin dizi karşılığı olan `degistir(int a[], int i, int j)` bu yüzden çalışır (7.9). Dizi **döndürülemez**; sonuç, çağıranın verdiği diziye yazılır.

> **Sınavda Çıkar**
>
> İki klasik soru: (1) "Fonksiyon dizinin elemanını değiştirirse `main`'de görünür mü?" **Evet**. "`int` parametreyi değiştirirse?" **Hayır**; `f(d[1])` gibi dizinin bir elemanını geçirmek de `int` geçirmektir (kopya). (2) Fonksiyon içinde `sizeof(a) / sizeof(a[0])` eleman sayısını **vermez**, adresin boyutuna bölünmüş bir sayı verir.

> **Sık Yapılan Hata: Çağrıda köşeli parantez**
>
> Dizi parametresini çağırırken adın dışında bir şey yazma: `yazdir(a[], N)` ve `yazdir(int a[], N)` derleme hatasıdır; `yazdir(a[N], N)` ise `a[N]` tek bir eleman (üstelik dizinin dışında) olduğundan yanlıştır. Parametre listesine `int a[5]` yazsan bile derleyici `5`'i yok sayar: `a` yine bir adrestir ve eleman sayısını `n` ile geçirmek zorundasın.

const: salt okunur dizi parametresi

`yazdir` diziyi yalnızca **okur**. Parametreyi `const int a[]` yazarak bunu derleyiciye de söylersin; fonksiyon bir elemanı değiştirmeye kalkarsa (`a[0] = 0;`) gcc `assignment of read-only location '*a'` hatası verir. Kural: diziyi **okuyan** fonksiyonlar (yazdırma, toplam, arama) `const` alır; **değiştirenler** (sıralama, ters çevirme, doldurma) almaz. Hocalar bu ayrımı sever: başlığa bakan biri diziye ne olacağını hemen anlar.

7.8 Diziyi kopyalama ve karşılaştırma

Dizi **bütün olarak atanamaz**: `b = a;` ve (tanımdan sonra) `a = {1, 2, 3};` derleme hatasıdır.

kopyaHata.c

```c
#include <stdio.h>

int main(void) {
    int a[3] = {1, 2, 3};
    int b[3];
    b = a;
    printf("%d\n", b[0]);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
kopyaHata.c: In function 'main':
kopyaHata.c:6:7: error: assignment to expression with array type
    6 |     b = a;
      |       ^
```

Kopyalamanın iki yolu: **eleman eleman döngüyle** ya da `string.h`'deki `memcpy(hedef, kaynak, bayt)` ile. Sıra, atamadaki gibi: önce hedef. Eşitlik de elle denetlenir; `ayniMi` ilk farkta durur:

kopyala.c

```c
#include <stdio.h>
#include <string.h>
#define N 5

int ayniMi(const int a[], const int b[], int n) {
    for (int i = 0; i < n; i++) {
        if (a[i] != b[i]) {
            return 0;
        }
    }
    return 1;
}

int main(void) {
    int a[N] = {1, 2, 3, 4, 5};
    int b[N];
    int c[N];
    for (int i = 0; i < N; i++) {
        b[i] = a[i];
    }
    memcpy(c, a, sizeof(a));
    printf("Kopyalar a ile aynı mı? %d %d\n", ayniMi(a, b, N), ayniMi(a, c, N));
    b[0] = 99;
    printf("b[0] = %d, a[0] = %d, aynı mı? %d\n", b[0], a[0], ayniMi(a, b, N));
    return 0;
}
```

Çıktı

```
Kopyalar a ile aynı mı? 1 1
b[0] = 99, a[0] = 1, aynı mı? 0
```

Kopyalar birbirinden **bağımsızdır**: `b[0]` değişince `a[0]` etkilenmedi.

> **Sık Yapılan Hata: memcpy'de eleman sayısı vermek**
>
> `memcpy`'nin üçüncü parametresi **bayttır**. `memcpy(c, a, N)` yalnızca `N` = 5 bayt kopyalar (bir `int` ve bir baytlık parça); dizinin geri kalanı çöp kalır. Doğrusu `memcpy(c, a, sizeof(a))` ya da `memcpy(c, a, N * sizeof(a[0]))`'dır.

Karşılaştırmada da tuzak vardır: `a == b` iki dizinin **içeriğini değil adreslerini** karşılaştırır. Farklı iki dizi bellekte farklı yerde durduğundan, içerikleri aynı olsa bile sonuç hep yanlıştır (gcc `comparison between two arrays [-Warray-compare]` uyarısı verir). İçerik, `ayniMi` gibi bir döngüyle ya da `memcmp(a, b, sizeof(a)) == 0` ile karşılaştırılır.

7.9 Ters çevirme ve kaydırma

Bu bölümdeki algoritmaların çoğu iki elemanın **yer değiştirmesine** dayanır. Geçici değişken şarttır: `a[i] = a[j]; a[j] = a[i];` yazarsan ilk satır `a[i]`'nin eski değerini yok eder ve ikisi de `a[j]`'yi alır. Bölüm 6'da `swap(int a, int b)` parametreler kopya olduğu için çalışmıyordu; dizi fonksiyona adres olarak geçtiği için `degistir(a, i, j)` çalışır.

Ters çevirme: biri baştan, biri sondan ilerleyen iki indeks (`bas`, `son`) elemanları değiştirir ve birbirine yaklaşır; birbirini geçince biter. **Sola kaydırma:** ilk eleman kenara alınır, her eleman bir soldaki yere çekilir, kenardaki eleman sona konur. **Sağa kaydırma** bunun aynadaki hâlidir ve **sondan başa** doğru işler:

donusum.c

```c
#include <stdio.h>

void yazdir(const int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

void degistir(int a[], int i, int j) {
    int gecici = a[i];
    a[i] = a[j];
    a[j] = gecici;
}

void tersCevir(int a[], int n) {
    int bas = 0;
    int son = n - 1;
    while (bas < son) {
        degistir(a, bas, son);
        bas++;
        son--;
    }
}

void solaKaydir(int a[], int n) {
    int ilk = a[0];
    for (int i = 0; i < n - 1; i++) {
        a[i] = a[i + 1];
    }
    a[n - 1] = ilk;
}

void sagaKaydir(int a[], int n) {
    int son = a[n - 1];
    for (int i = n - 1; i > 0; i--) {
```

```
        a[i] = a[i - 1];
    }
    a[0] = son;
}

int main(void) {
    int a[6] = {1, 2, 3, 4, 5, 6};
    int b[5] = {1, 2, 3, 4, 5};
    int c[5] = {1, 2, 3, 4, 5};
    tersCevir(a, 6);
    solaKaydir(b, 5);
    sagaKaydir(c, 5);
    yazdir(a, 6);
    yazdir(b, 5);
    yazdir(c, 5);
    return 0;
}
```

Çıktı

```
6 5 4 3 2 1
2 3 4 5 1
5 1 2 3 4
```

`tersCevir(a, 6)`'da (`bas`, `son`) = (0, 5), (1, 4), (2, 3) çiftleri sırayla yer değiştirir: `1 2 3 4 5 6` → `6 2 3 4 5 1` → `6 5 3 4 2 1` → `6 5 4 3 2 1`. Sonra `bas = 3`, `son = 2` olur ve `bas < son` yanlış çıkar. Eleman sayısı **tek** olunca ortadaki eleman yerinde kalır; toplam `n / 2` değiş-tokuş yapılır.

Kaydırma yönü hayati önem taşır: **sola kaydırırken baştan sona, sağa kaydırırken sondan başa** git. Sağa kaydırmayı baştan sona yazarsan (`for (i = 1; i < n; i++) a[i] = a[i - 1];`) her adım az önce yazdığın değeri okur; ilk eleman bütün diziye yayılır (`{1, 2, 3, 4, 5}` için `1 1 1 1 1`). Kural: bir eleman, üzerine yazılmadan **önce** okunmalıdır.

7.10 Arama: doğrusal ve ikili

Doğrusal arama diziyi baştan sona gezer. Bulursa **indeksi** döndürür; bulamazsa geçerli bir indeks olamayacak bir değer, gelenek olarak `-1` döndürür. **İkili arama** (binary search) ise dizi **sıralıysa** çok daha hızlıdır: ortadaki elemana bakılır; aranan ortadakinden küçükse yalnız sol yarıda, büyükse yalnız sağ yarıda aranır. Aday aralığı her adımda yarıya iner (sözlükte sözcük ararken sayfayı ortadan açmak gibi). Üç indeks gerekir: aralığın başı `alt`, sonu `ust`, ortası `orta`.

arama.c

```
#include <stdio.h>

int ara(const int a[], int n, int aranan) {
    for (int i = 0; i < n; i++) {
        if (a[i] == aranan) {
            return i;
        }
    }
    return -1;
}

int ikiliAra(const int a[], int n, int aranan) {
    int alt = 0;
    int ust = n - 1;
    int adim = 0;
    while (alt <= ust) {
        int orta = (alt + ust) / 2;
        adim++;
```

```
        printf("  adım %d: alt=%d ust=%d orta=%d a[orta]=%d\n",
               adim, alt, ust, orta, a[orta]);
        if (a[orta] == aranan) {
            return orta;
        } else if (a[orta] < aranan) {
            alt = orta + 1;
        } else {
            ust = orta - 1;
        }
    }
    return -1;
}

int main(void) {
    int a[] = {3, 7, 12, 19, 25, 31, 42, 56, 68, 77};
    int n = sizeof(a) / sizeof(a[0]);
    printf("ara: 31 -> %d, 10 -> %d\n", ara(a, n, 31), ara(a, n, 10));
    printf("31 ikili aranıyor:\n");
    printf("  sonuç: %d\n", ikiliAra(a, n, 31));
    printf("10 ikili aranıyor:\n");
    printf("  sonuç: %d\n", ikiliAra(a, n, 10));
    return 0;
}
```

Çıktı

```
ara: 31 -> 5, 10 -> -1
31 ikili aranıyor:
  adım 1: alt=0 ust=9 orta=4 a[orta]=25
  adım 2: alt=5 ust=9 orta=7 a[orta]=56
  adım 3: alt=5 ust=6 orta=5 a[orta]=31
  sonuç: 5
10 ikili aranıyor:
  adım 1: alt=0 ust=9 orta=4 a[orta]=25
  adım 2: alt=0 ust=3 orta=1 a[orta]=7
  adım 3: alt=2 ust=3 orta=2 a[orta]=12
  sonuç: -1
```

`ara`'da `return` ilk bulunanda fonksiyonu bitirir (aranan birden çok yerdeyse ilk indeks döner); `-1` döngüden **sonra** gelir. En kötü durumda `n` karşılaştırma yapılır; dizinin sıralı olması gerekmez.

> **Sık Yapılan Hata: Döngünün içinde -1 döndürmek**
>
> `if (a[i] == aranan) { return i; } else { return -1; }` yazarsan fonksiyon yalnız **ilk elemana** bakar: ilk eleman aranan değilse `else` hemen `return`'e girer ve diğer elemanlara sıra gelmez. "Bulunamadı" kararı bütün elemanlar denendikten sonra, **döngünün dışında** verilir.

`31`'in adım tablosu (indeksler 0–9):

Adım	`alt`	`ust`	`orta`	`a[orta]`	Karar
1	0	9	4	25	25 < 31, `alt = orta + 1 = 5`
2	5	9	7	56	56 > 31, `ust = orta - 1 = 6`
3	5	6	5	31	bulundu: indeks 5

`10` dizide yok: `alt` ve `ust` birbirini geçince (`alt = 2`, `ust = 1`) döngü biter, `-1` döner. Dikkat edilecekler: koşul `alt <= ust`'tir (`<` yazarsan tek eleman kalan aralığı atlarsın); `alt = orta + 1` ve `ust = orta - 1` yazılır, çünkü `orta` zaten denendi; dizi **sıralı değilse** sonuç yanlıştır. `n` elemanlı sıralı dizide **en çok** $\lfloor \log_2 n \rfloor + 1$ adım gerekir. Örneğin 10 elemanda en çok 4, 100 elemanda 7, bir milyon elemanda yalnız 20 adım yeter (doğrusal aramada 10, 100 ve 1 000 000). Aynı fikri kendini çağıran bir fonksiyonla Bölüm 13'te yazacaksın.

7.11 Sıralama: kabarcık ve seçmeli

Diziyi küçükten büyüğe dizmek **sıralama** (sorting) problemidir. İki klasik yöntem vardır; ikisi de değiş-tokuş için `degistir` kullanır.

Kabarcık sıralaması (bubble sort): komşu elemanlar karşılaştırılır, soldaki büyükse yer değiştirilir. Bir **geçiş** (pass) sonunda en büyük eleman kabarcık gibi yüzüp sona ulaşır. **Seçmeli sıralama (selection sort):** her turda henüz sıralanmamış kısmın **en küçüğü** bulunur ve o kısmın başındaki elemanla değiştirilir.

siralama.c

```c
#include <stdio.h>

void yazdir(const int a[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

void degistir(int a[], int i, int j) {
    int gecici = a[i];
    a[i] = a[j];
    a[j] = gecici;
}

void kabarcikSirala(int a[], int n) {
    for (int gecis = 1; gecis < n; gecis++) {
        for (int i = 0; i < n - gecis; i++) {
            if (a[i] > a[i + 1]) {
                degistir(a, i, i + 1);
            }
        }
        printf("%d. geçiş: ", gecis);
        yazdir(a, n);
    }
}

void secmeliSirala(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int enKucuk = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[enKucuk]) {
                enKucuk = j;
            }
        }
        degistir(a, i, enKucuk);
        printf("i=%d (en küçük indeks %d): ", i, enKucuk);
        yazdir(a, n);
    }
}

int main(void) {
    int a[5] = {5, 1, 4, 2, 8};
    int b[5] = {64, 25, 12, 22, 11};
    printf("Kabarcık, başlangıç: ");
```

```
    yazdir(a, 5);
    kabarcikSirala(a, 5);
    printf("Seçmeli, başlangıç: ");
    yazdir(b, 5);
    secmeliSirala(b, 5);
    return 0;
}
```

Çıktı

```
Kabarcık, başlangıç: 5 1 4 2 8
1. geçiş: 1 4 2 5 8
2. geçiş: 1 2 4 5 8
3. geçiş: 1 2 4 5 8
4. geçiş: 1 2 4 5 8
Seçmeli, başlangıç: 64 25 12 22 11
i=0 (en küçük indeks 4): 11 25 12 22 64
i=1 (en küçük indeks 2): 11 12 25 22 64
i=2 (en küçük indeks 3): 11 12 22 25 64
i=3 (en küçük indeks 3): 11 12 22 25 64
```

Kabarcıkta ilk geçişin karşılaştırmaları (dizi `5 1 4 2 8`):

Karşılaştırma	Karar	Dizi
`a[0]` ile `a[1]`: 5 > 1	değiştir	1 5 4 2 8
`a[1]` ile `a[2]`: 5 > 4	değiştir	1 4 5 2 8
`a[2]` ile `a[3]`: 5 > 2	değiştir	1 4 2 5 8
`a[3]` ile `a[4]`: 5 < 8	değiştirme	1 4 2 5 8

`gecis`. geçişte iç döngü `n - gecis` karşılaştırma yapar, çünkü sondaki `gecis - 1` eleman zaten yerindedir. Bu örnekte dizi 2. geçişte sıralanmış hâle geldi, 3. ve 4. geçişte hiç değiş-tokuş olmadı. **Hiç değiş-tokuş olmayan geçiş**, dizinin sıralı olduğunu kanıtlar; bir `bool` bayrağı (`degisti`) tutup bu durumda `break` ile çıkarsan gereksiz geçişler atlanır. Seçmelide `i` yalnız `n - 2`'ye kadar gider, çünkü son eleman kendiliğinden yerine oturur; `i = 3` satırında 25 kendisiyle yer değiştirdi (zararsız).

Karşılaştırma sayısı ikisinde de `n(n-1)/2`'dir (bayraklı kabarcık daha azına inebilir); değiş-tokuş kabarcıkta çok olabilir, seçmelide en çok `n - 1`'dir. İkisi de `n` iki katına çıkınca işi yaklaşık **dört katına** çıkarır ("O(n^2)"); büyük veride daha hızlı yöntemler kullanılır. Dersin istediği, ikisini **elle izleyebilmen** ve kodunu yazabilmendir. **Büyükten küçüğe** sıralamak için karşılaştırma tersine çevrilir (`a[i] < a[i + 1]`).

Sınavda Çıkar

"Kabarcık sıralamasının `k` geçişi sonundaki dizi nedir?" sorusunda sondaki `k` eleman **kesinlikle yerindedir** (en büyük `k` eleman); baştakiler hâlâ karışık olabilir. Tabloyu geçiş geçiş çıkar, kestirmeye kalkma. Seçmelide `k` tur sonra ise baştaki `k` eleman, yani en küçükler yerindedir.

İpucu: Sıralamayı kâğıtta izle

Her geçiş ya da tur sonrası dizinin tamamını alt alta yaz; hangi iki elemanı karşılaştırdığını ve yer değiştirip değiştirmediğini yanına not et. İç döngü sınırını karıştırma: kabarcıkta `i < n - gecis`, seçmelide `j = i + 1`'den `n`'e.

7.12 İki boyutlu diziler

Tablo ya da matris gibi **satır ve sütunlu** veriyi iki boyutlu dizi tutar: `tip ad[SATIR][SUTUN];`. Eleman `m[i][j]`'dir; `i` **satır** (0 ile `SATIR-1`), `j` **sütun** (0 ile `SUTUN-1`) numarasıdır. `int m[3][4];` 3 satır, 4 sütun, 12 eleman demektir. Bellekte elemanlar **satır satır** art arda durur; her satır kendi başına bir tek boyutlu dizidir. `int m[2][3]` için `m[2][0]` **yoktur** (satır indeksleri 0 ve 1); dış döngü `i < SATIR`, iç döngü `j < SUTUN` olmalıdır. Elemana `m[i, j]` ile de erişilmez: C'de `i, j` ifadesi (virgül operatörü) önce `i`'yi hesaplayıp atar, sonucu `j` olur; yani bu `m[j]` demektir. Doğrusu `m[i][j]`.

İlk değer, satır başına bir küme parantezi ile verilir; eksikler yine `0` olur: `int z[2][3] = { {1}, {4, 5} };` (`1 0 0 / 4 5 0`). Yalnız **ilk** boyut atlanabilir: `int y[][3] = { {1, 2, 3}, {4, 5, 6} };` geçerlidir, `int y[][] = ...` değildir. Tek parantezli düz liste de derlenir ama gcc `-Wmissing-braces` uyarır; iç içe yaz.

Dizi iç içe iki `for` ile gezilir: dış döngü satırları, iç döngü sütunları dolaşır. Aşağıdaki kare matriste (3 × 3) satır toplamı, köşegenler ve transpoz hesaplanıyor. **Transpoz**, satırları sütun yapmaktır: `t[j][i] = m[i][j]` (kare olmayan 2 × 3 matriste `t` 3 × 2 olur, satır ve sütun sayıları yer değiştirir).

matris.c

```c
#include <stdio.h>
#define N 3

int main(void) {
    int m[N][N] = {
        {5, 1, 2},
        {0, 3, 4},
        {9, 8, 7}
    };
    int t[N][N];
    int ana = 0;
    int ikincil = 0;
    for (int i = 0; i < N; i++) {
        int satirToplam = 0;
        for (int j = 0; j < N; j++) {
            printf("%3d", m[i][j]);
            satirToplam += m[i][j];
            t[j][i] = m[i][j];
            if (i == j) {
                ana += m[i][j];
            }
            if (i + j == N - 1) {
                ikincil += m[i][j];
            }
        }
        printf("  | satır toplamı: %d\n", satirToplam);
    }
    printf("Ana köşegen: %d, ikincil köşegen: %d\n", ana, ikincil);
    printf("Transpoz:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            printf("%3d", t[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

```
Çıktı
  5  1  2  | satır toplamı: 8
  0  3  4  | satır toplamı: 7
  9  8  7  | satır toplamı: 24
Ana köşegen: 15, ikincil köşegen: 14
Transpoz:
  5  0  9
  1  3  8
  2  4  7
```

Satır toplamında iç döngü `j`'dir ve toplam her satırın başında `0`'dan başlar; **sütun toplamında** döngüler yer değiştirir (dış döngü `j`, iç döngü `i`). Köşegenler kare matriste şu koşullarla seçilir:

Bölge	Koşul	Bu matriste
Ana köşegen	`i == j`	5 + 3 + 7 = 15
İkincil köşegen	`i + j == N - 1`	2 + 3 + 9 = 14
Köşegenin üstü / altı	`j > i` / `j < i`	1 + 2 + 4 = 7 / 0 + 9 + 8 = 17

`sizeof(m)` bütün dizinin, `sizeof(m[0])` **bir satırın** bayt sayısıdır; bölümleri satır sayısını verir (3 × 3 × 4 = 36 bayt, bir satır 12 bayt, 36 / 12 = 3). `sizeof(m[0]) / sizeof(m[0][0])` ise sütun sayısıdır.

Fonksiyona iki boyutlu dizi geçirmek

İki boyutlu dizi parametresinde **sütun sayısı yazılmak zorundadır**, satır sayısı yazılmaz (ayrı parametre olur): `int m[][SUTUN]`. Çünkü elemanlar art arda durur ve derleyici `m[i][j]`'nin yerini `başlangıç + (i × SUTUN + j) × 4` bayt diye hesaplar: formülde `SATIR` yoktur ama `SUTUN` vardır. `void yazdir(int m[][], int satir)` yazarsan gcc `array type has incomplete element type 'int[]'` hatası verir.

Aşağıda iki matrisi toplayan bir fonksiyon var; `a` ve `b` yalnız okunduğu için `const` aldı:

matrisIslem.c

```c
#include <stdio.h>
#define SATIR 2
#define SUTUN 3

void yazdirMatris(const int m[][SUTUN], int satir) {
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < SUTUN; j++) {
            printf("%4d", m[i][j]);
        }
        printf("\n");
    }
}

void topla(const int a[][SUTUN], const int b[][SUTUN],
           int c[][SUTUN], int satir) {
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < SUTUN; j++) {
            c[i][j] = a[i][j] + b[i][j];
        }
    }
}

int main(void) {
    int a[SATIR][SUTUN] = {{1, 2, 3}, {4, 5, 6}};
    int b[SATIR][SUTUN] = {{10, 20, 30}, {40, 50, 60}};
    int c[SATIR][SUTUN];
```

```
    topla(a, b, c, SATIR);
    yazdirMatris(c, SATIR);
    return 0;
}
```

Çıktı

```
 11  22  33
 44  55  66
```

Tek boyutlu dizilerde olduğu gibi `c`, `topla`'ya adres olarak geçer; fonksiyonun yazdıkları `main`'deki `c`'de görünür. Dizi döndürülemediği için sonucu çağıran verir.

Sınavda Çıkar

İki boyutlu dizi soruları üç kalıba iner: (1) "`m[i][j]` hangi eleman?": tabloyu çiz, satır `i`, sütun `j`. (2) Köşegen koşulları: `i == j` ve `i + j == N - 1`. (3) Fonksiyon başlığı: `int m[][SUTUN]` doğru, `int m[][]` yanlış. `sizeof(m) / sizeof(m[0])` satır sayısını verir.

Bölüm Özeti – Bilmen Gerekenler

- **Dizi** aynı türden elemanları tek adla tutar: `tip ad[N];`. Boyut derleme sırasında bilinen sabittir (`#define N 5`); elemanlar bellekte art arda durur. VLA (C99) sınavda kullanılmaz.
- `const int N = 5;` **sabit ifade değildir**: global `int a[N];` standartta geçersizdir (gcc sessizce kabul edebilir), yerelde VLA olur ve `= {0}` alamaz. Boyut için `#define N 5` ya da `enum` kullan.
- **İndeks 0'dan başlar**, son eleman `a[N-1]`'dir; `a[N]` yoktur. C sınırı **denetlemez**: sınır dışı erişim tanımsız davranıştır (sessizce komşu belleği bozabilir ya da çökertebilir). Döngüde `i < N` yaz.
- İlk değer listesi: eksikler `0` olur (`{1, 2, 3}`), `{0}` hepsini sıfırlar, `{7}` hepsini 7 yapmaz, `int a[] = {...}` boyutu listeden çıkarır. Listesiz yerel dizi çöp taşır. Liste yalnız tanım satırında yazılır.
- Eleman sayısı `sizeof(a) / sizeof(a[0])` ile bulunur ama yalnız dizinin tanımlandığı yerde; fonksiyonda **yanlış** sonuç verir, bu yüzden `(int a[], int n)` kullanılır. Kalıplar: toplam (`0`'dan başla), ortalama (`(double) toplam / N`), en büyük/en küçük (ilk elemandan başla, indeksi sakla), sayma, ortalamanın üstü (iki geçiş).
- Dizi fonksiyona **adres olarak** geçer: fonksiyon diziyi değiştirirse çağırana yansır; `int` parametre değişirse yansımaz. Okuyan fonksiyon `const int a[]` alır. Çağrıda yalnız ad yazılır: `yazdir(a, N)`.
- `b = a` derlenmez; kopyalamak için döngü ya da `memcpy(hedef, kaynak, bayt)`. `a == b` içeriği değil adresleri karşılaştırır; karşılaştırma döngüyle yapılır. Ters çevirme: iki indeks ve geçici değişken. Kaydırma: sola baştan sona, sağa sondan başa. Doğrusal arama: bulunursa indeks, bulunamazsa `-1` (döngüden **sonra**). İkili arama: dizi **sıralı**, `while (alt <= ust)`, en çok $\lfloor \log_2 n \rfloor + 1$ adım.
- Kabarcık: komşuları karşılaştır, `k` geçiş sonunda en büyük `k` eleman yerindedir (iç döngü `i < n - gecis`). Seçmeli: her turda en küçüğü bul, `i` konumuna koy. İkisi de `n(n-1)/2` karşılaştırma yapar.
- İki boyutlu dizi `m[i][j]` (i satır, j sütun); iç içe iki `for`; transpoz `t[j][i] = m[i][j]`; köşegenler `i == j`, `i + j == N - 1`. Fonksiyona geçirirken **sütun sayısı zorunludur**: `int m[][SUTUN]`.

Sorular

A) Çoktan Seçmeli

1 `int puan[8];` tanımı için aşağıdakilerden hangisi doğrudur?

(A) Geçerli indeksler 1'den 8'e kadardır.
(B) `puan[8]` dizinin son elemanıdır.
(C) Geçerli indeksler 0'dan 7'ye kadardır; `puan[8]` yazılırsa derleyici çoğunlukla hata vermez, ama program tanımsız davranır.
(D) `puan[8]` yazılırsa C çalışma anında hata fırlatıp programı güvenle durdurur.
(E) Dizi 8 bayt yer kaplar.

2 `int a[5] = {3};` tanımından hemen sonra `a[0] + a[1] + a[2] + a[3] + a[4]` ifadesinin değeri kaçtır?

(A) 3
(B) 15
(C) 0
(D) 8
(E) Tanımsızdır, çünkü kalan elemanlar çöp değer taşır.

3 Sıralı ve 100 elemanlı bir dizide ikili arama ile bir değer aranıyor. Döngü, en kötü durumda **en çok** kaç adımda sona erer?

(A) 1 (B) 5 (C) 6 (D) 7 (E) 100

4 Aşağıdaki cümlelerden hangisi doğrudur?

(A) `int a[3], b[3];` tanımından sonra `b = a;` yazmak `a`'nın elemanlarını `b`'ye kopyalar.
(B) Bir fonksiyona dizi geçirilirse fonksiyon içinde yapılan eleman değişiklikleri çağıran fonksiyondaki diziye de yansır.
(C) Dizi parametresi alan bir fonksiyonda `sizeof(a) / sizeof(a[0])` her zaman eleman sayısını verir.
(D) `a == b` ifadesi iki dizinin elemanlarının tek tek eşit olup olmadığını denetler.
(E) Parametresi `const int a[]` olan bir fonksiyon `a[0] = 1;` satırını sorunsuz derler.

5 `int m[3][4];` tanımına göre `sizeof(m) / sizeof(m[0])` ifadesi neyi verir? (`int` 4 bayttır.)

(A) 4 (B) 12 (C) 16 (D) 48 (E) 3

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int a[5] = {3, 0, 4, 1, 2};
    printf("%d\n", a[a[0]]);
    printf("%d\n", a[a[a[0]]]);
    a[a[1]] = 9;
    int toplam = 0;
    for (int i = 0; i < 5; i++) {
        toplam += a[i];
    }
    printf("%d\n", toplam);
    printf("%d %d\n", a[0], a[a[4]]);
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

void f(int x) {
    x = x * 2;
}

void g(int a[]) {
    a[0] = a[0] * 2;
    a[1]++;
}

int main(void) {
    int s = 5;
    int d[3] = {5, 6, 7};
    f(s);
    g(d);
    f(d[2]);
    printf("%d %d %d %d\n", s, d[0], d[1], d[2]);
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır? (Dış döngü iki kez döner.)

```c
#include <stdio.h>

int main(void) {
    int a[5] = {4, 7, 2, 9, 1};
    for (int gecis = 1; gecis <= 2; gecis++) {
        for (int i = 0; i < 4; i++) {
            if (a[i] > a[i + 1]) {
                int t = a[i];
                a[i] = a[i + 1];
                a[i + 1] = t;
            }
        }
    }
    for (int i = 0; i < 5; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    return 0;
}
```

C) Kod Yazma

9 Klavyeden **6 tamsayı** (not) okuyup dizide saklayan `notAnalizi` programını yaz. Sırayla en büyük değeri ve **indeksini**, ortalamayı (iki ondalık basamak) ve ortalamanın **üstündeki** eleman sayısını yazdırsın. Boyut için `#define` kullan.

```
Girdi : 55 80 92 47 80 66
Çıktı : En büyük: 92 (indeks 2)
        Ortalama: 70.00
        Ortalamanın üstünde: 3 eleman
```

10 Klavyeden **3 × 3 bir matris** okuyan `kosegenToplam` programını yaz. Önce **her satırın toplamını**, sonra ana köşegen ve ikincil köşegen toplamlarını yazdırsın.

```
Girdi : 2 7 1
        4 5 9
        3 8 6
Çıktı : 1. satır toplamı: 10
        2. satır toplamı: 18
        3. satır toplamı: 17
        Ana köşegen toplamı: 13
        İkincil köşegen toplamı: 9
```

Çözümler

Cevap anahtarı

1 C	2 A	3 D	4 B	5 E

1 Cevap: C

`N` elemanlı dizide geçerli indeksler `0`'dan `N-1`'e kadardır; `puan[8]` dizinin dışındadır ve C bunu denetlemez: derleyici çoğunlukla ses çıkarmaz, davranış tanımsızdır. A yanlış, indeks 0'dan başlar. B yanlış, son eleman `puan[7]`'dir. D yanlış, C'de böyle bir koruma yoktur. E yanlış, dizi $8 \times 4 = 32$ bayt kaplar.

2 Cevap: A

İlk değer listesi yalnız `a[0]`'a `3` verir; listenin vermediği dört eleman `0` olur: $3 + 0 + 0 + 0 + 0 = 3$. B, "`{3}` hepsini 3 yapar" yanılgısıdır. C, `a[0]`'ı da sıfır sayar. D anlamsız. E yanlış: çöp, ilk değer listesi **hiç olmayan** yerel dizilerdedir; listenin vermediği elemanlar `0` olur.

3 Cevap: D

Her adımda aday sayısı yarılanır: $100 \to 50 \to 25 \to 12 \to 6 \to 3 \to 1$, yedi adım. Formül: $\lfloor \log_2 100 \rfloor + 1 = 6 + 1 = 7$. A (1) yalnız en şanslı durumdur (aranan ilk bakılan ortadaki elemansa). B ve C yetmez; C, `+ 1`'i unutmanın sonucudur. E doğrusal aramanın en kötü durumudur.

4 Cevap: B

Dizi fonksiyona adres olarak geçtiği için fonksiyon gerçek diziyi değiştirir ve değişiklik çağırana yansır. A yanlış, `b = a` derlenmez. C yanlış, fonksiyon içinde `a` bir adrestir ve `sizeof(a)` adresin boyutunu verir. D yanlış, `a == b` **adresleri** karşılaştırır. E yanlış, `const` parametreye yazmak derleme hatasıdır.

5 Cevap: E

`sizeof(m)` bütün dizi: $3 \times 4 \times 4 = 48$ bayt. `sizeof(m[0])` **bir satırın** boyutu: $4 \times 4 = 16$ bayt. $48 / 16 = \mathbf{3}$, yani satır sayısı. A (4) sütun sayısıdır (`sizeof(m[0]) / sizeof(m[0][0])`), B (12) toplam eleman sayısı, C (16) bir satırın bayt sayısı, D (48) bütün dizinin bayt sayısıdır.

6

`a = {3, 0, 4, 1, 2}`. İçten dışa çöz: `a[a[0]]` = `a[3]` = **1**; `a[a[a[0]]]` = `a[1]` = **0**. `a[a[1]] = 9;` satırında `a[1]` = 0 olduğundan `a[0] = 9` olur ve dizi `{9, 0, 4, 1, 2}` olur; toplam $9 + 0 + 4 + 1 + 2 = \mathbf{16}$. Son satırda `a[0]` = 9 ve `a[a[4]]` = `a[2]` = 4: `9 4`.

Çıktı

```
1
0
16
9 4
```

7

`f(s)` ve `f(d[2])` çağrılarında **değer kopyalanır**: `f` kendi `x`'ini değiştirir, `s` ve `d[2]` etkilenmez. `g(d)` ise gerçek diziyi değiştirir: `d[0]` = 5 × 2 = 10 olur, `d[1]` 6'dan 7 olur. `d[2]` 7 kalır, `s` 5 kalır.

Çıktı
```
5 10 7 7
```

8

Birinci geçiş: (4,7) değişmez; (7,2) değişir → `4 2 7 9 1`; (7,9) değişmez; (9,1) değişir → `4 2 7 1 9`. İkinci geçiş: (4,2) değişir → `2 4 7 1 9`; (4,7) değişmez; (7,1) değişir → `2 4 1 7 9`; (7,9) değişmez. Sondaki iki eleman yerine oturdu, ama baştaki `1` hâlâ geride: dizi henüz sıralı değil.

Çıktı
```
2 4 1 7 9
```

9

Yaklaşım: Altı sayıyı `&notlar[i]` adresine oku. En büyüğü ilk elemandan başlayarak ara, indeksini sakla, toplamı aynı döngüde biriktir. Ortalamayı `(double) toplam / N` ile hesapla (tamsayı bölmesi tuzağı!). Ortalamanın üstündekileri saymak için diziyi **ikinci kez** gez.

notAnalizi.c
```c
#include <stdio.h>
#define N 6

int main(void) {
    int notlar[N];
    printf("%d not gir: ", N);
    for (int i = 0; i < N; i++) {
        scanf("%d", &notlar[i]);
    }
    int enBuyuk = notlar[0];
    int enBuyukIndeks = 0;
    int toplam = 0;
    for (int i = 0; i < N; i++) {
        toplam += notlar[i];
        if (notlar[i] > enBuyuk) {
            enBuyuk = notlar[i];
            enBuyukIndeks = i;
        }
    }
    double ortalama = (double) toplam / N;
    int ustunde = 0;
    for (int i = 0; i < N; i++) {
        if (notlar[i] > ortalama) {
            ustunde++;
        }
    }
    printf("En büyük: %d (indeks %d)\n", enBuyuk, enBuyukIndeks);
    printf("Ortalama: %.2f\n", ortalama);
    printf("Ortalamanın üstünde: %d eleman\n", ustunde);
    return 0;
}
```

```
Çıktı
6 not gir: 55 80 92 47 80 66
En büyük: 92 (indeks 2)
Ortalama: 70.00
Ortalamanın üstünde: 3 eleman
```

Toplam 420, ortalama 70.00; 70'in üstünde 80, 92 ve 80 olduğundan 3 eleman.

10

Yaklaşım: Dokuz sayıyı iç içe iki `for` ile `m[i][j]` elemanlarına oku. İkinci iç içe döngüde her `i` turunda `satirToplam` **sıfırdan** başlar; `i == j` ana köşegene, `i + j == N - 1` ikincil köşegene eklenir. Satır toplamı iç döngü bitince yazdırılır, köşegenler ise bütün döngüden sonra.

kosegenToplam.c

```c
#include <stdio.h>
#define N 3

int main(void) {
    int m[N][N];
    printf("Matris (%dx%d): ", N, N);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &m[i][j]);
        }
    }
    int ana = 0;
    int ikincil = 0;
    for (int i = 0; i < N; i++) {
        int satirToplam = 0;
        for (int j = 0; j < N; j++) {
            satirToplam += m[i][j];
            if (i == j) {
                ana += m[i][j];
            }
            if (i + j == N - 1) {
                ikincil += m[i][j];
            }
        }
        printf("%d. satır toplamı: %d\n", i + 1, satirToplam);
    }
    printf("Ana köşegen toplamı: %d\n", ana);
    printf("İkincil köşegen toplamı: %d\n", ikincil);
    return 0;
}
```

```
Çıktı
Matris (3x3): 2 7 1
4 5 9
3 8 6
1. satır toplamı: 10
2. satır toplamı: 18
3. satır toplamı: 17
Ana köşegen toplamı: 13
İkincil köşegen toplamı: 9
```

Ana köşegen $2 + 5 + 6 = 13$, ikincil köşegen $1 + 5 + 3 = 9$; satır toplamları 10, 18 ve 17.

BÖLÜM 8

Karakter Dizileri (String)

Bu Bölümde Öğreneceklerin

- String'in bellekte `'\0'` ile biten bir `char` dizisi olduğunu; tanımlamayı ve `'A'` ile `"A"` farkını
- `%s`, `puts` ile yazdırmayı ve `s[i] != '\0'` ile dolaşmayı
- `scanf("%19s")` ve `fgets` ile okumayı, `'\n'`'i temizlemeyi ve `scanf` sonrası `fgets` tuzağını
- `string.h` fonksiyonlarını (`strlen`, `strcpy`, `strcat`, `strcmp`, `strchr`, `strstr`) ve `==` ile karşılaştırma tuzağını
- `ctype.h` ile karakter denetimini ve dönüşümünü; rakam karakterini sayıya, sayıyı string'e çevirmeyi
- Türkçe karakterlerin neden `strlen` sonucunu şaşırttığını
- String dizisini ve klasik algoritmaları (ters çevirme, palindrom, harf frekansı, sezar şifresi)

8.1 String nedir?

C'de `string` diye ayrı bir tip yoktur. Metin bir `char` dizisinde tutulur ve metnin bittiği yeri dizinin içine konan özel bir karakter gösterir: **null karakter** (null character) `'\0'`. Kodu 0'dır; rakam karakteri `'0'` ile karıştırma (onun kodu 48). `"Ali"` belleğe şöyle yerleşir:

İndeks	0	1	2	3
Karakter	`'A'`	`'l'`	`'i'`	`'\0'`
ASCII kodu	65	108	105	0

`"Ali"` 3 karakterdir ama **4 bayt** yer kaplar. `printf("%s")` gibi fonksiyonlar string'in nerede bittiğini dizinin boyutundan değil, `'\0'`**'a rastlayınca** anlar. Bölüm 1'de gördüğün string sabitlerinin (string literal) sonunda da gizli bir `'\0'` vardır.

Kural: **uzunluk `'\0'` hariç karakter sayısıdır, bellekte kapladığı yer bir fazladır.** Bunu bir sonraki başlıktaki programda kendin göreceksin.

Sınavda Çıkar

"`char s[] = "Merhaba";` için `sizeof s` kaçtır?" Cevap 8'dir, 7 değil: 7 karakter + `'\0'`. Bu tuzak her yıl sorulur. `"A"` 2 bayt, `""` 1 bayttır.

8.2 Tanımlama ve ilk değer

tanimlama.c

```c
#include <stdio.h>

int main(void) {
    char a[20] = "Ali";
    char b[] = "Ali";
    char c[] = {'A', 'l', 'i', '\0'};
    printf("%s %zu\n", a, sizeof a);
    printf("%s %zu\n", b, sizeof b);
    printf("%s %zu\n", c, sizeof c);
    printf("a[3] = %d, a[19] = %d, b[3] = %d\n", a[3], a[19], b[3]);
    printf("%zu %zu\n", sizeof("Merhaba"), sizeof(""));
    return 0;
}
```

Çıktı

```
Ali 20
Ali 4
Ali 4
a[3] = 0, a[19] = 0, b[3] = 0
8 1
```

- `char a[20] = "Ali";`: 20 bayt ayrılır. Bölüm 7'deki ilk değer kuralı geçerlidir: verilmeyen elemanlar 0 olur, yani `a[3]`'ten `a[19]`'a hepsi `'\0'`'dır. Sonradan daha uzun metin koyabileceğin yer bırakmak için kullanılır.
- `char b[] = "Ali";`: Boyutu derleyici bulur: karakter sayısı + 1 = 4.
- `char c[] = {'A', 'l', 'i', '\0'};`: Harf harf yazılan biçim; `'\0'`'ı **sen** yazarsın.
- `sizeof("Merhaba")` 8, boş string `sizeof("")` 1 bayttır: son satırdaki iki sayı. `b[3]` de 0 çıktı; bu `'\0'`'ın kodudur.

Kural: `n` karakterlik metin için dizi boyutu **en az** `n + 1` olmalıdır. İlk değer verilmemiş `char ad[20];` çöp (garbage) taşır; içine `scanf`, `fgets` ya da `strcpy` ile bir şey koymadan yazdırma.

> **Sık Yapılan Hata: `'\0'` için yer bırakmamak**
>
> `char s[3] = {'A', 'l', 'i'};` yazarsan `'\0'` yoktur ve `s` bir string **değildir**; `printf("%s\n", s);` tanımsız davranıştır. `printf` `'\0'` arar, bulamayınca dizinin ötesindeki belleği okumaya devam eder: ekrana çöp karakterler yazılabilir ya da program çökebilir. `"Ali"` için en az 4 bayt ayır; `{...}` biçiminde yazıyorsan sonuna `'\0'` ekle.

'A' ile "A" aynı şey değildir

	`'A'`	`"A"`
Ne?	Tek bir karakter (sayı 65)	İki baytlık dizi: `'A'` ve `'\0'`
`printf` ile	`%c`	`%s`
`char` değişkene atanır mı?	Evet	Hayır

Karıştırırsan gcc durdurur: `char harf = "A";` satırı `initialization of 'char' from 'char *' makes integer from pointer without a cast` hatası verir. Çift tırnaklı değer bir adrestir (`char *`); işaretçileri Bölüm 9'da göreceksin. gcc 14'ten önce bu yalnızca uyarıydı ve program yanlış çalışırdı. Tersi de yanlıştır: `printf("%s\n", 'A');` 65'i adres sanıp okumaya çalışır ve program çökebilir.

8.3 Yazdırma ve karakter karakter dolaşma

`printf` ile string yazdırmak için `%s` kullanılır (`%10s`, `%-10s`, `%.3s` Bölüm 3'te). `puts(s)` string'i yazar ve **kendisi** alt satıra geçer; `printf("%s\n", s)` ile aynı işi yapar. Tek `char` için `putchar(c)` vardır.

String bir dizi olduğu için `s[i]` ile her karakterine ulaşırsın. Uzunluğu önceden bilmen gerekmez, `'\0'`'a gelene kadar ilerlersin. Bir string'i kısaltmak için istediğin yere `'\0'` yazman yeter:

yazdirma.c

```c
#include <stdio.h>

int main(void) {
    char s[] = "Merhaba";
    printf("%s\n", s);
    puts(s);
    for (int i = 0; s[i] != '\0'; i++) {
        printf("%c-", s[i]);
    }
    printf("\n");
    s[3] = '\0';
    printf("%s %c %zu\n", s, s[4], sizeof s);
    return 0;
}
```

Çıktı

```
Merhaba
Merhaba
M-e-r-h-a-b-a-
Mer a 8
```

`s[i] != '\0'` kalıbı string işlemlerinin temelidir; bu bölümdeki hemen her döngü böyle yazılır. `s[3] = '\0'` yapınca `%s` orada durdu ve yalnız `Mer` yazdı. Ama dizi hâlâ 8 bayttır ve `s[4]` bellekte durur (`a`); sadece string'in parçası sayılmaz. Sınavda `'\0'` sonrasındaki karakterleri yazdıran şıklara dikkat et.

Bu kitapta string'i hep `char s[] = "...";` ile tutuyoruz; böyle bir dizinin elemanlarını değiştirmek serbesttir. `char *s = "...";` biçimi başka bir şeydir (string sabiti salt okunur bellekte durur, değiştirmeye çalışmak programı çökertebilir); farkı Bölüm 9'da göreceksin.

8.4 Klavyeden string okumak

scanf ile: tek sözcük

`scanf` ile string okurken `%s` kullanılır ve değişkenin başına **`&` konmaz**: dizinin adı zaten dizinin başlangıç adresi gibi davranır (ayrıntı Bölüm 9). Kural: `int`, `double`, `char` için `&` var, dizi için yok.

scanfString.c

```c
#include <stdio.h>

int main(void) {
    char ad[20], soyad[20];
    printf("Ad ve soyad: ");
    scanf("%19s", ad);
    scanf("%19s", soyad);
    printf("Ad: %s, Soyad: %s\n", ad, soyad);
    return 0;
}
```

Çıktı
```
Ad ve soyad: Ali Veli Yilmaz
Ad: Ali, Soyad: Veli
```

- `%s` **boşlukta durur.** Baştaki boşlukları atlar, ilk boşluk, sekme ya da `'\n'`'de bırakır. `Ali Veli Yilmaz` yazıldığında `Ali` ve `Veli` okundu, `Yilmaz` tamponda kaldı. `scanf("%s")` ile **boşluklu metin okunamaz**.
- **Genişlik, dizi boyutundan 1 eksik** olur: `ad[20]` için `%19s`. En çok 19 karakter okunur; 20. bayta `'\0'`'ı `scanf` kendisi koyar.

> **Sık Yapılan Hata: Genişliksiz %s ve gets**
>
> `char ad[10]; scanf("%s", ad);` genişliksiz `%s` kullanıcının yazdığı kadar karakteri diziye doldurur; 20 karakter yazılırsa dizinin dışına yazılır (Bölüm 7'deki sınır taşması, tanımsız davranış). `gets(ad);` ise boyut almadığı için taşmayı **önlemenin hiçbir yolu yoktur**; güvenlik açıklarının klasik kaynağı olduğundan C11'de standarttan çıkarıldı. Eski kaynaklarda görürsün; sınavda yazarsan yanlış sayılır. Yerine `fgets` kullan.

fgets ile: boşluklu satır

`fgets(dizi, boyut, stdin)` en çok `boyut - 1` karakter okur, `'\n'` görürse orada durur ve sona `'\0'` ekler. `boyut` yerine `sizeof dizi` yazarsan taşma olmaz; satır diziden uzunsa yalnız sığan kısmı alır, kalanı tamponda bırakır. `scanf`'ten iki farkı vardır: **boşlukları okur** ve satır sonundaki `'\n'`**'i diziye koyar**.

fgetsOku.c
```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char satir[30];
    printf("Ad soyad: ");
    fgets(satir, sizeof satir, stdin);
    printf("Ham   : [%s]\n", satir);
    satir[strcspn(satir, "\n")] = '\0';
    printf("Temiz : [%s] (%zu)\n", satir, strlen(satir));
    return 0;
}
```

Çıktı
```
Ad soyad: Ali Veli Yilmaz
Ham   : [Ali Veli Yilmaz
]
Temiz : [Ali Veli Yilmaz] (15)
```

Ham halinde `]` alt satıra düştü, çünkü `'\n'` diziye girmişti. `strcspn(satir, "\n")` string'de ilk `'\n'` karakterinin **indeksini** verir (yoksa uzunluğu); oraya `'\0'` yazınca `'\n'` silinir. `'\n'` yoksa zaten var olan `'\0'`'ın üstüne yazılır, zarar vermez. Bunu kalıp olarak ezberle; her `fgets`'ten sonra kullanacaksın. (`scanf("%19[^\n]", s)` de boşluklu satır okur ama `'\n'`'i tamponda bırakır; biz `fgets` kullanıyoruz.)

scanf'ten sonra fgets tuzağı

Bölüm 3'te `scanf("%d")` sonrasında Enter'ın tamponda kalıp `%c`'yi bozduğunu görmüştün. `fgets` de aynı yüzden bozulur: tamponda hazır bekleyen `'\n'`'i okur ve "satır bitti" sanır.

scanfFgets.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    int yas = 0;
    char ad[30];
    printf("Yas: ");
    scanf("%d", &yas);
    printf("Ad: ");
    fgets(ad, sizeof ad, stdin);
    printf("yas=%d ad=[%s] uzunluk=%zu\n", yas, ad, strlen(ad));
    return 0;
}
```

Çıktı

```
Yas: 20
Ad: yas=20 ad=[
] uzunluk=1
```

`fgets`, kullanıcıya hiçbir şey sormadan tamponda bekleyen `'\n'` ile geri döndü: `ad` yalnızca `"\n"`, uzunluğu 1. Kullanıcı `Ali Veli` yazma fırsatını bile bulamaz. Çare, Bölüm 3'teki kalıpla `scanf`'ten hemen sonra satırın kalanını atmaktır: `scanf("%d", &yas);` satırının altına `while (getchar() != '\n');` yaz. `fgets` bundan sonra gerçekten kullanıcıyı bekler.

Sınavda Çıkar

"`scanf("%s", s)` ile `Ali Veli` okunursa `s`'de ne kalır?" Cevap `Ali`. "`scanf("%d")`'dan sonra `fgets` neden hemen bitiyor?" Cevap: tamponda kalan `'\n'`'i okuduğu için; okunan string'in uzunluğu 1 çıkar. Ayrıca `fgets` sonucu `'\n'` içerir, `scanf("%s")` içermez.

İpucu: Fonksiyona string geçirmek

String'lerde Bölüm 7'deki gibi boyut parametresi gerekmez, çünkü fonksiyon `'\0'`'a kadar gider. Fonksiyonun string'i değiştirmeyeceğini `const` ile belirt: `int sesliSay(const char s[])`. Değiştirecekse (ters çevirme gibi) `const` yazma: `void tersCevir(char s[])`. Fonksiyonun içinde `sizeof s` dizinin boyutunu **vermez** (nedeni Bölüm 9'da); `fgets` için boyutu parametre olarak ilet.

8.5 string.h fonksiyonları

C'de string'e `+`, `=`, `==` gibi operatörler uygulanmaz; her iş için hazır fonksiyon çağırırsın. Hepsi `#include <string.h>` ister:

Fonksiyon	Ne yapar?
`strlen(s)`	`'\0'` hariç uzunluk (tipi `size_t`, `%zu` ile yazdır)
`strcpy(hedef, kaynak)`	`kaynak`'ı `'\0'` dahil `hedef`'e kopyalar
`strncpy(hedef, kaynak, n)`	En çok `n` karakter kopyalar (`'\0'` koymayabilir)
`strcat(hedef, kaynak)`	`kaynak`'ı `hedef`'in sonuna ekler
`strncat(hedef, kaynak, n)`	En çok `n` karakter ekler (`'\0'`'ı her zaman koyar)
`strcmp(a, b)`	`a < b` ise negatif, eşitse 0, `a > b` ise pozitif
`strchr(s, c)`	`s` içinde `c` karakterini arar
`strstr(s, alt)`	`s` içinde `alt` string'ini arar

Ortak kural: **Hedef dizi sonucu alacak kadar büyük olmalıdır.** Fonksiyonlar boyutu denetlemez; yer yetmezse dizinin ötesine yazarlar (tanımsız davranış).

strlen ve sizeof

`char s[20] = "Merhaba";` için `strlen(s)` 7, `sizeof s` 20'dir:

	`strlen(s)`	`sizeof s`
Ne ölçer?	`'\0'`'a kadar **karakter sayısı**	Dizinin **toplam bayt** boyutu
Ne zaman hesaplanır?	Çalışırken, her çağrıda baştan sayar	Derleme sırasında
`s[3] = '\0'` yapınca	Değişir	Değişmez

`strlen` her çağrıda string'i baştan sayar; döngü koşuluna `i < strlen(s)` yazarsan her turda yeniden sayılır. Üstelik `strlen`'in tipi işaretsiz `size_t` olduğundan `int i` ile karşılaştırmak `-Wextra` ile "comparison of integer expressions of different signedness" uyarısı verir. Koşulu `s[i] != '\0'` yap, ya da uzunluğu döngüden önce `int n = (int)strlen(s);` ile al. `strlen(s) - 1` da yazma: boş string'de işaretsiz `0 - 1` eksi bir sayı değil, devasa bir pozitif sayı olur; çıkarmadan önce `(int)strlen(s)` yaz.

strcpy, strncpy, strcat, strncat

`strcpy(hedef, kaynak)` kopyalamanın, `strcat(hedef, kaynak)` birleştirmenin doğru yoludur. `strncpy` ve `strncat` "en çok `n` karakter" der ve güvenli görünür, ama `strncpy` kaynak `n` karakter ya da daha uzunsa hedefe `'\0'` **koymaz**:

kopyaBirlestir.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char kaynak[] = "Merhaba";
    char hedef[20];
    strcpy(hedef, kaynak);
    strcat(hedef, " Dunya");
    printf("%s (%zu karakter)\n", hedef, strlen(hedef));
    char a[8] = "xxxxxxx";
    strncpy(a, kaynak, 4);
    char b[5];
    strncpy(b, kaynak, sizeof b - 1);
    b[sizeof b - 1] = '\0';
    printf("%s | %s\n", a, b);
    char dar[8] = "Ali";
    strncat(dar, " Veli Yilmaz", sizeof dar - strlen(dar) - 1);
    printf("%s\n", dar);
    return 0;
}
```

Çıktı

```
Merhaba Dunya (13 karakter)
Merhxxx | Merh
Ali Vel
```

`a`'ya yalnız `Merh` kopyalandı ve `'\0'` konmadı; `printf` eski `xxx`'lere kadar yürüdü. Doğru kalıp `b`'dedir: en çok `sizeof b - 1` karakter kopyala, ardından son baytı kendin `'\0'` yap; bu iki satırı hep birlikte yaz. `strcat` hedefin `'\0'`'ını bulup ekleyeceği metni oradan yazar, bu yüzden hedef **önce geçerli bir string olmalıdır**: çöp değerli `char tam[30];` yerine `char tam[30] = "";` yaz. `strncat`'ın üçüncü parametresi eklenecek **karakter** sayısıdır ve `'\0'`'ı her zaman koyar; güvenli değer hedefte kalan boş yerdir: `sizeof dar - strlen(dar) - 1` = 8 - 3 - 1 = 4, yani `Vel` eklendi.

> **Sık Yapılan Hata: strcpy ve strncpy tuzakları**
>
> `strcpy` hedefin boyutunu bilmez, `'\0'` görene kadar kopyalar: `char hedef[5]; strcpy(hedef, "Merhaba");` 8 baytı 5 baytlık yere yazar (taşma, tanımsız davranış). Önce `strlen(kaynak) + 1 <= sizeof hedef` olduğundan emin ol. `strncpy(hedef, kaynak, sizeof hedef);` de hatalıdır: `'\0'` için yer bırakmaz. `strcat` için de aynısı geçerlidir: hedefte mevcut metin, eklenecek metin ve `'\0'` için yer olmalı.

strcmp

`strcmp(a, b)` iki string'i **karakter karakter, ASCII kodlarına göre** karşılaştırır. İlk farklı karakter sonucu belirler; biri diğerinden önce biterse (`'\0'` kodu 0) kısa olan küçüktür. Dönüş: `a < b` ise **negatif**, eşitse **0**, `a > b` ise **pozitif**.

karsilastirma.c

```c
#include <stdio.h>
#include <string.h>

void karsilastir(const char a[], const char b[]) {
    int s = strcmp(a, b);
    printf("\"%s\" ile \"%s\": %s\n", a, b,
           s < 0 ? "kucuk" : s > 0 ? "buyuk" : "esit");
}

int main(void) {
    karsilastir("Ali", "Ali");
    karsilastir("Ali", "Veli");
    karsilastir("Ali", "Alice");
    karsilastir("ali", "Ali");
    karsilastir("kedi", "Zebra");
    return 0;
}
```

Çıktı

```
"Ali" ile "Ali": esit
"Ali" ile "Veli": kucuk
"Ali" ile "Alice": kucuk
"ali" ile "Ali": buyuk
"kedi" ile "Zebra": buyuk
```

`Ali` ile `Veli` ilk harfte ayrılır (`'A'` = 65, `'V'` = 86). `Ali` ile `Alice`: üç harf aynı, `Ali` orada biter (`'\0'` = 0, `'c'` = 99), kısa olan küçüktür. `"ali"` ile `"Ali"`: küçük `'a'` (97), büyük `'A'`'dan (65) büyüktür. **Tüm büyük harfler tüm küçük harflerden küçüktür**, bu yüzden `"kedi"`, `"Zebra"`'dan büyük çıkar.

> **Sık Yapılan Hata: strcmp'nin döndürdüğü sayıya güvenme**
>
> `strcmp("Ali", "Veli")` bazı sistemlerde `-1`, bazılarında iki karakterin farkını (`-21` gibi) döndürür; standart yalnız **işareti** vaat eder. `if (strcmp(a, b) == -1)` yazma; sonuca yalnız `< 0`, `== 0`, `> 0` ile bak. Eşitlik denetimi `strcmp(a, b) == 0`'dır; `if (strcmp(a, b))` yazarsan koşul string'ler **farklıyken** doğru olur.

İlk `n` karakteri karşılaştırmak için `strncmp(a, b, n)` vardır. Büyük/küçük harfi yok sayan karşılaştırma standart C'de yoktur; iki kopyayı `tolower` ile küçültüp `strcmp` yaparsın.

strchr ve strstr

`strchr` karakter, `strstr` alt string arar. İkisi de **bulduğu yerin adresini** döndürür, bulamazsa `NULL` döndürür. Adresleri Bölüm 9'da göreceksin; şimdilik iki iş yapabilirsin: **var mı?** (`!= NULL`) ve **nerede?** (bulunan adresten `s`'i çıkarırsan indeks çıkar).

arama.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[] = "Merhaba Dunya";
    if (strchr(s, 'a') != NULL) {
        printf("ilk 'a' indeksi: %d\n", (int)(strchr(s, 'a') - s));
    }
    if (strstr(s, "Dun") != NULL) {
        printf("\"Dun\" indeksi: %d\n", (int)(strstr(s, "Dun") - s));
    }
    return 0;
}
```

Çıktı

```
ilk 'a' indeksi: 4
"Dun" indeksi: 8
```

`M-e-r-h-a` indeksleri 0-4: ilk `'a'` 4'te. `"Dun"` boşluktan sonra, 8. indekste başlar. Aranan yoksa iki fonksiyon da `NULL` döndürür; indeksi hesaplamadan önce hep `!= NULL` denetle. Küçük tuzak: `strchr(s, '\0')` string'in sonundaki `'\0'`'ı "bulur" ve `NULL` döndürmez.

8.6 == ile karşılaştırma ve = ile atama tuzağı

Bölüm 7'de dizilerin `=` ile kopyalanmadığını, `==` ile karşılaştırılmadığını görmüştün; string'ler de dizidir. Başka dillerden gelen herkes buna düşer:

esitlikTuzagi.c

```c
#include <stdio.h>

int main(void) {
    char a[] = "Ali";
    char b[] = "Ali";
    if (a == b) {
        printf("ayni\n");
    }
    if (a == "Ali") {
        printf("ayni\n");
    }
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
esitlikTuzagi.c: In function 'main':
esitlikTuzagi.c:6:11: warning: comparison between two arrays [-Warray-compare]
    6 |     if (a == b) {
      |           ^~
esitlikTuzagi.c:6:11: note: use '&a[0] == &b[0]' to compare the addresses
esitlikTuzagi.c:9:11: warning: comparison with string literal results in unspecified behavior [-
Waddress]
    9 |     if (a == "Ali") {
      |           ^~
```

Program derlenir ama gcc iki uyarı verir. `a == b` **içerikleri değil, dizilerin bellekteki adreslerini** karşılaştırır. `a` ile `b` bellekte farklı yerlerdedir; aynı harfleri taşımaları önemsizdir, adresleri eşit olmaz ve ilk `if` hiçbir zaman doğru çıkmaz. `a == "Ali"` ise string sabitinin adresiyle karşılaştırır ve standart sonucu belirsiz bırakır. Bu yüzden çıktıyı göstermiyoruz: bu koda güvenilemez. Atama da yanlıştır:

atamaHatasi.c

```c
#include <stdio.h>

int main(void) {
    char ad[20];
    ad = "Ali";
    printf("%s\n", ad);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
atamaHatasi.c: In function 'main':
atamaHatasi.c:5:8: error: assignment to expression with array type
    5 |     ad = "Ali";
      |        ^
```

Dizinin adı atama hedefi olamaz; gcc `assignment to expression with array type` der. Doğrusu `strcpy(ad, "Ali");`.

İstediğin	Yanlış	Doğru
String'e metin atamak	`s = "Ali";`	`strcpy(s, "Ali");`
Kopyalamak	`a = b;`	`strcpy(a, b);`
Eşit mi?	`a == b`	`strcmp(a, b) == 0`
`a`, `b`'den küçük mü?	`a < b`	`strcmp(a, b) < 0`
Birleştirmek	`a + b`	`strcat(a, b)`

> ● **Sık Yapılan Hata: String'i == ile karşılaştırmak**
>
> `if (ad == "Ali")` ve `if (a == b)` **derlenir ama yanlıştır**; `-Wall` uyarır ve hiçbir zaman beklediğin gibi çalışmaz. Java ya da Python'dan gelenlerin eli buna gider; sınavda en çok puan kaybettiren hatalardan biridir. String eşitliği yalnız `strcmp(a, b) == 0` ile denetlenir.

8.7 ctype.h: karakter denetimi ve dönüşümü

`#include <ctype.h>` ile gelen fonksiyonlar **tek bir karakter** üzerinde çalışır. Denetleyenler: `isalpha(c)` (harf mi?), `isdigit(c)` (rakam mı?), `isupper(c)` ve `islower(c)` (büyük / küçük harf mi?), `isspace(c)` (boşluk, sekme, `'\n'` gibi boşluk karakteri mi?); ayrıca `isalnum` (harf ya da rakam) ve `ispunct` (noktalama işareti) vardır. Dönüştürenler: `toupper(c)` ve `tolower(c)` büyük / küçük harfe çevrilmiş karakteri **döndürür**; harf değilse aynısını döndürür.

ctypeDeneme.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char s[] = "Merhaba Dunya 2025";
    int harf = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (isalpha(s[i])) {
            harf++;
        }
        s[i] = toupper(s[i]);
    }
    printf("%s (harf: %d)\n", s, harf);
    return 0;
}
```

Çıktı

```
MERHABA DUNYA 2025 (harf: 12)
```

`toupper` ve `tolower` karakteri **değiştirmez**, dönüştürülmüş halini döndürür; `s[i] = toupper(s[i]);` ile geri yazmazsan işe yaramaz (`toupper(s[i]);` tek başına hiçbir şey yapmaz; gcc bazı sistemlerde "statement with no effect" uyarısı verir, bazılarında sessiz kalır). Rakamlar ve boşluklar değişmeden kalır. Bu döngü "string'i büyük harfe çevir" sorusunun cevabıdır; `tolower` aynı kalıpla küçültür. `s[i] - 32` hilesini kullanma, harf olmayanları bozar.

> **Sık Yapılan Hata: isdigit(c) == 1 yazma**
>
> `isalpha`, `isdigit` gibi fonksiyonlar doğru için 1 döndürmek zorunda değildir, sıfırdan farklı herhangi bir sayı olabilir (bazı sistemlerde 2048 gibi). `if (isdigit(c) == 1)` bazı bilgisayarlarda hiç çalışmaz. Doğrusu: `if (isdigit(c))` ya da `if (isdigit(c) != 0)`.

Bir ayrıntı: Bu fonksiyonların parametresi `int`'tir ve yalnız `unsigned char` aralığındaki (0-255) değerler ile `EOF` için tanımlıdır. `char` işaretli olduğundan ASCII dışı baytlar (Türkçe harflerin UTF-8 baytları) negatif görünür ve doğrudan vermek tanımsız davranıştır. ASCII metinde sorun çıkmaz ve sınavda cast'siz yazılır; Türkçe harf içerebilecek metinde güvenli yazım `isalpha((unsigned char)s[i])`'dir.

8.8 String ile sayı dönüşümleri

Rakam karakterinden sayıya: c - '0' ve atoi

`'7'` karakterinin değeri 7 değil, ASCII kodu olan 55'tir. Rakam karakterlerinin kodları art arda gider (`'0'` = 48, `'1'` = 49, ... `'9'` = 57); bu yüzden `'7' - '0'` = 55 - 48 = 7. Bölüm 2'de `char`'ın küçük bir tamsayı olduğunu görmüştün, bu onun doğrudan uygulamasıdır. Tersi de çalışır: `7 + '0'` rakamın karakteridir. Hazır karşılıkları `<stdlib.h>` içindeki `atoi` (`int`) ve `atof`'tur (`double`).

rakamDonusum.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    char s[] = "4096";
    int sayi = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        sayi = sayi * 10 + (s[i] - '0');
    }
    printf("%d %d %c\n", sayi + 1, '7' - '0', 7 + '0');
    printf("%d %d\n", atoi("12abc"), atoi("abc"));
    printf("%d %.2f\n", atoi("  42"), atof("3.14") * 2);
    return 0;
}
```

Çıktı

```
4097 7 7
12 0
42 6.28
```

İlk döngü string'i soldan sağa okuyup her adımda sayıyı 10 ile çarpar: 4, 40 + 0, 400 + 9, 4090 + 6; `atoi`'nin kendisidir. `atoi` baştaki boşlukları atlar, sayı olmayan ilk karakterde durur, hiç sayı bulamazsa 0 döndürür. Zayıflığı da buradan çıkar: `atoi("abc")` ile `atoi("0")` aynı sonucu verir, yani **hatayı anlayamazsın**. Sayı `int`'e sığmayacak kadar büyükse sonuç tanımsızdır.

strtol: hatayı yakalamak

`strtol(s, &son, 10)` string'i `long`'a çevirir (10, sayı tabanı). `son`, dönüşümün **nerede durduğunu** öğrenmek içindir ve bir **işaretçidir** (`char *son = NULL;`, Bölüm 9'da göreceksin; şimdilik kalıp olarak al). `son == s` ise hiç rakam bulunamadı, `*son != '\0'` ise sayıdan sonra artan karakterler var.

strtolDene.c

```c
#include <stdio.h>
#include <stdlib.h>

void dene(const char s[]) {
    char *son = NULL;
    long sayi = strtol(s, &son, 10);
    if (son == s) {
        printf("\"%s\": hic rakam yok\n", s);
    } else if (*son != '\0') {
        printf("\"%s\": sayi %ld, artan \"%s\"\n", s, sayi, son);
    } else {
        printf("\"%s\": gecerli, %ld\n", s, sayi);
    }
}

int main(void) {
    dene("456");
    dene("12ab");
    dene("abc");
    return 0;
}
```

```
Çıktı
"456": gecerli, 456
"12ab": sayi 12, artan "ab"
"abc": hic rakam yok
```

`"12ab"` için `atoi` sessizce 12 verirdi; `strtol` `ab`'nin artık kaldığını gösterdi. Ondalık sayılar için `strtod` vardır. `fgets` ile okuduğun satırı çevirirken önce `strcspn` ile `'\n'`'i sil, yoksa `'\n'` "artan karakter" olarak görünür.

Sayıdan string'e: snprintf

`snprintf(dizi, boyut, biçim, ...)`, `printf`'in ekran yerine **diziye** yazan halidir. En çok `boyut - 1` karakter yazar ve `'\0'`'ı kendisi koyar, bu yüzden taşmaz. Eski kodlarda `sprintf(s, ...)` görürsün; aynı işi yapar ama boyut almaz, taşmaya karşı koruması yoktur. Yeni kodda `snprintf` yaz.

sayidanString.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[40];
    snprintf(s, sizeof s, "OGR-%03d-%.1f", 7, 78.456);
    printf("%s (%zu karakter)\n", s, strlen(s));
    return 0;
}
```

```
Çıktı
OGR-007-78.5 (12 karakter)
```

8.9 Türkçe karakterler ve UTF-8

Bölüm 3'te Türkçe harflerin UTF-8'de 2 bayt tuttuğunu görmüştün. `strlen` ve `sizeof` karakterleri değil **baytları** sayar:

turkceBayt.c

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void) {
    char s[] = "ağaç";
    int karakter = 0, harf = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        unsigned char b = (unsigned char)s[i];
        karakter += (b < 0x80 || b >= 0xC0);
        harf += (isalpha(b) != 0);
    }
    printf("strlen=%zu sizeof=%zu\n", strlen(s), sizeof s);
    printf("karakter=%d isalpha=%d\n", karakter, harf);
    return 0;
}
```

```
Çıktı
strlen=6 sizeof=7
karakter=4 isalpha=2
```

`"ağaç"` 4 karakterdir ama `ğ` ve `ç` ikişer bayt olduğu için `strlen` 6 verir (`sizeof` de `'\0'` ile 7). Çok baytlı bir karakterin ilk baytı 0xC0 ve üzeridir, devam baytları 0x80-0xBF arasındadır; döngü yalnız ilk baytları (ve ASCII'yi) sayarak gerçek karakter sayısını buldu. Son satıra bak: `isalpha` yalnız iki `a`'yı harf saydı, `ğ` ile `ç`'nin baytlarını tanımadı.

Sonuç olarak bu bölümdeki araçlar **ASCII metin içindir**: `strlen` Türkçe harf içeren string'de karakter değil **bayt** sayısı verir; `isalpha`, `toupper` gibi fonksiyonlar `ç ğ ı ö ş ü` harflerini tanımaz; string'i **yerinde ters çevirirsen** çok baytlı karakterlerin baytları ters sıraya girer ve çıktı bozulur; `strcmp` baytlara baktığı için `"çilek"`, `"zeytin"`'den **büyük** çıkar (`ç`'nin ilk baytı 0xC3, `'z'`'den büyüktür). Sınavda ve ödevde string işlemlerinde Türkçe harf kullanmamaya çalış. Yalnız ekrana yazdırmak (`printf("Ağaç\n")`) sorun değildir; Windows konsolunda `chcp 65001` gerektiğini hatırla.

8.10 String dizisi

Birden çok string tutmak için (isim listesi, şehirler) **iki boyutlu `char` dizisi** kullanılır:

```
char isimler[3][20] = {"Zeynep", "Ali", "Can"};   // 3 satır, satır başına 20 bayt
```

`isimler[i]` i. satırı, yani i. string'i verir; `isimler[i][j]` o string'in j. karakteridir. Bellekte 3 satırın her biri 20 baytlık ayrı bir kutudur; kısa isimler de 20 bayt tutar, kalanı `'\0'` ile dolar. Okurken `scanf("%19s", isimler[i])` yazarsın (`&` yok, `isimler[i]` zaten bir string'dir).

Alfabetik sıralama, Bölüm 7'deki sıralamayla aynıdır; yalnız karşılaştırma `strcmp`, yer değiştirme `strcpy` ile yapılır. Dizi elemanları `=` ile atanamadığı için üçüncü bir `gecici` string gerekir:

isimSirala.c

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char isimler[5][20];
    char gecici[20];
    printf("5 isim: ");
    for (int i = 0; i < 5; i++) {
        scanf("%19s", isimler[i]);
    }
    for (int tur = 0; tur < 4; tur++) {
        for (int j = 0; j < 4 - tur; j++) {
            if (strcmp(isimler[j], isimler[j + 1]) > 0) {
                strcpy(gecici, isimler[j]);
                strcpy(isimler[j], isimler[j + 1]);
                strcpy(isimler[j + 1], gecici);
            }
        }
    }
    for (int i = 0; i < 5; i++) {
        printf("%d. %s\n", i + 1, isimler[i]);
    }
    printf("Dizi boyutu: %zu bayt\n", sizeof isimler);
    return 0;
}
```

```
Çıktı
5 isim: Zeynep Ali Mehmet Ayse Can
1. Ali
2. Ayse
3. Can
4. Mehmet
5. Zeynep
Dizi boyutu: 100 bayt
```

Yan yana iki isim yanlış sıradaysa (`strcmp(...) > 0`) üç `strcpy` ile yerlerini değiştirirsin. `isimler[j] > isimler[j + 1]` yazma (adresleri karşılaştırır) ve `isimler[j] = isimler[j + 1]` yazma (derlenmez). Dizi 5 x 20 = 100 bayttır; kısa isimler için israf, Bölüm 9-10'da bunu da düzelteceksin.

8.11 Klasik algoritmalar

Aşağıdakiler string sorularının temelidir. Hepsinde aynı iskelet var: `for (int i = 0; s[i] != '\0'; i++)`. Büyük harfe çevirme algoritmasını `toupper` döngüsüyle zaten gördün.

Ters çevirme ve palindrom

İki yerden yaklaşırsın: `s[i]` ile onun karşısındaki `s[n - 1 - i]`, ortaya (`n / 2`) kadar. **Ters çevirmede** bu iki karakterin yerini değiştirirsin (yeni dizi gerekmez). **Palindromda** (tersiyle aynı okunan sözcük, `kayak`) karşılaştırırsın; bir uyumsuzluk bile varsa palindrom değildir.

tersPalindrom.c

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void tersCevir(char s[]) {
    int n = (int)strlen(s);
    for (int i = 0; i < n / 2; i++) {
        char gecici = s[i];
        s[i] = s[n - 1 - i];
        s[n - 1 - i] = gecici;
    }
}

int palindromMu(const char s[]) {
    int n = (int)strlen(s);
    for (int i = 0; i < n / 2; i++) {
        unsigned char a = (unsigned char)s[i];
        unsigned char b = (unsigned char)s[n - 1 - i];
        if (tolower(a) != tolower(b)) {
            return 0;
        }
    }
    return 1;
}

int main(void) {
    char s[] = "Merhaba";
    tersCevir(s);
    printf("%s\n", s);
    printf("Radar: %d\n", palindromMu("Radar"));
    printf("Ali: %d\n", palindromMu("Ali"));
    return 0;
}
```

Çıktı
```
abahreM
Radar: 1
Ali: 0
```

Döngü `n / 2` kez döner: tek uzunlukta ortadaki karakter kendi karşısındadır ve yerinde kalır; boş string'de döngü hiç dönmez. Dizi fonksiyona adresiyle geçtiği için (Bölüm 7) `tersCevir`'in değişikliği `main`'deki `s`'e yansır. `tolower` sayesinde `Radar` da palindrom sayıldı. Başka bir yol: kopyayı ters çevirip `strcmp(a, b) == 0` ile karşılaştırmak.

Sesli harf ve kelime sayma

Sesli harf için her karakteri küçültüp `"aeiou"` içinde ararız (`strchr`; Türkçe `ı ö ü` ASCII olmadığı için sayılmaz). `strchr("aeiou", '\0')` string'in sonunu "bulurdu" ama döngü `'\0'`'a gelmeden bittiği için sorun çıkmaz. Kelime saymak için "şu an bir kelimenin içinde miyim?" bilgisini bir bayrakta (`icinde`) tutarız: boşluktan harfe geçtiğimiz her yerde yeni kelime başlar.

metinSayim.c
```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void) {
    char s[] = "Programlama  dersi cok guzel";
    int sesli = 0, kelime = 0, icinde = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (strchr("aeiou", tolower(s[i])) != NULL) {
            sesli++;
        }
        if (isspace(s[i])) {
            icinde = 0;
        } else if (!icinde) {
            icinde = 1;
            kelime++;
        }
    }
    printf("Sesli: %d, Kelime: %d\n", sesli, kelime);
    return 0;
}
```

Çıktı
```
Sesli: 9, Kelime: 4
```

Kelime sayarken boşlukları değil, boşluktan **sonra gelen ilk harfi** sayıyoruz; bu yüzden metindeki çift boşluk sonucu bozmaz. "Boşluk sayısı + 1" yöntemi bu metinde yanlış verirdi.

Harf frekansı

Her harfin kaç kez geçtiğini 26 elemanlı bir sayaç dizisinde tutarız: `sayac[0]` `a` için, `sayac[1]` `b` için ... İndeks `harf - 'a'` ile bulunur.

harfFrekansi.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char s[] = "Programlama dersi cok guzel";
    int sayac[26] = {0};
    for (int i = 0; s[i] != '\0'; i++) {
        if (isalpha(s[i])) {
            sayac[tolower(s[i]) - 'a']++;
        }
    }
    for (int k = 0; k < 26; k++) {
        if (sayac[k] > 0) {
            printf("%c:%d ", 'a' + k, sayac[k]);
        }
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
a:3 c:1 d:1 e:2 g:2 i:1 k:1 l:2 m:2 o:2 p:1 r:3 s:1 u:1 z:1
```

`isalpha` denetimi şarttır: boşluğu `sayac[' ' - 'a']` ile saymaya kalkarsan indeks eksi çıkar ve dizinin dışına yazarsın (tanımsız davranış). `'a' + k` bir `int`'tir; `%c` onu karaktere çevirip yazdırır.

Sezar şifresi

Sezar şifresi her harfi alfabede `kaydir` kadar ileri kaydırır, sondan sonra başa döner. Büyük ve küçük harf ayrı işlenir; harf olmayanlar olduğu gibi kalır. `% 26` sarmayı sağlar:

sezar.c

```c
#include <stdio.h>
#include <ctype.h>

void sezar(char s[], int kaydir) {
    for (int i = 0; s[i] != '\0'; i++) {
        if (isupper((unsigned char)s[i])) {
            s[i] = 'A' + (s[i] - 'A' + kaydir) % 26;
        } else if (islower((unsigned char)s[i])) {
            s[i] = 'a' + (s[i] - 'a' + kaydir) % 26;
        }
    }
}

int main(void) {
    char s[] = "Merhaba Dunya";
    sezar(s, 3);
    printf("sifreli : %s\n", s);
    sezar(s, 26 - 3);
    printf("cozulmus: %s\n", s);
    return 0;
}
```

Çıktı

```
sifreli : Phukded Gxqbd
cozulmus: Merhaba Dunya
```

Çözmek için ters yönde kaydırmak yerine `26 - kaydir` kadar ileri gideriz; 26 harf tam bir tur olduğu için sonuç aynıdır ve `%` işlecine negatif sayı sokmamış oluruz. Sarma örneği: `'y'` + 3 = `('y' - 'a' + 3) % 26` = `(24 + 3) % 26` = 1, yani `'b'`.

Bölüm Özeti – Bilmen Gerekenler

- **String**, sonunda `'\0'` (kodu 0) bulunan `char` dizisidir. `"Ali"` 3 karakter, **4 bayttır**; dizi boyutu en az `karakter + 1` olmalı. `'A'` tek bir karakter sabitidir (C'de tipi `int`, `sizeof('A')` 4; bir `char` değişkene koyunca 1 bayt), `"A"` bir string (2 bayt).
- Tanım: `char s[20] = "Ali";`, `char s[] = "Ali";` (boyut = karakter + 1). Dolaşma kalıbı `for (int i = 0; s[i] != '\0'; i++)`; `s[3] = '\0'` string'i keser, `sizeof` değişmez.
- `scanf("%19s", s)` boşlukta durur, `&` yok, genişlik = boyut - 1. `fgets(s, sizeof s, stdin)` satırı okur ve `'\n'`'i diziye koyar; silmek için `s[strcspn(s, "\n")] = '\0';`. `scanf`'ten sonra `fgets` için önce `while (getchar() != '\n');`. `gets` yok.
- `strlen` `'\0'` hariç uzunluktur, `sizeof` dizi boyutudur. `strncpy` `'\0'` koymayabilir: son baytı kendin yap. `strcmp` işaretiyle çalışır (`<0`, `0`, `>0`), büyük harfler küçüklerden önce gelir. Hedef yeterince büyük olmalı.
- String'e `=` ve `==` uygulanmaz: `s = "Ali"` derlenmez, `a == b` adresleri karşılaştırır. Doğrusu `strcpy` ve `strcmp(a, b) == 0`.
- `ctype.h`: `isalpha`, `isdigit`, `isspace`, `isupper`, `toupper`, `tolower`. `isxxx` sıfırdan farklı bir sayı döndürür (`== 1` yazma); `toupper(c)` değeri döndürür, atamazsan işe yaramaz.
- Dönüşüm: rakam karakteri `c - '0'`; `atoi` hatayı anlatmaz, `strtol` anlatır; sayıdan string `snprintf(s, sizeof s, "%d", n)`.
- UTF-8'de `strlen("ağaç") == 6`: `strlen` bayt sayar, karakter değil; `ctype` fonksiyonları Türkçe harfleri tanımaz.
- String dizisi `char isimler[5][20];`: `isimler[i]` bir string'dir; sıralamada `strcmp` ve `strcpy` (+ `gecici`) kullanılır. Klasik algoritmalar: ters çevirme, palindrom (iki indeks), sesli/kelime sayma, harf frekansı (`sayac[harf - 'a']++`), sezar (`'a' + (c - 'a' + k) % 26`).

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki kod parçasının çıktısı nedir?

```c
char s[20] = "Merhaba";
printf("%zu %zu\n", strlen(s), sizeof s);
```

(A) `7 7` (B) `7 8` (C) `7 20` (D) `8 20` (E) `20 7`

2 `strcmp("kedi", "Zebra")` çağrısı için aşağıdakilerden hangisi doğrudur?

(A) Negatif bir sayı döndürür, çünkü “kedi” alfabede “Zebra”dan önce gelir.

(B) 0 döndürür, çünkü büyük/küçük harf farkı yok sayılır.

(C) Derleme hatası verir, çünkü string’ler `strcmp` ile karşılaştırılamaz.

(D) Negatif bir sayı döndürür, çünkü “kedi” “Zebra”dan kısadır.

(E) Pozitif bir sayı döndürür, çünkü `'k'` karakterinin kodu `'Z'` karakterinin kodundan büyüktür.

3 Aşağıdaki programın çıktısı nedir?

```c
char sayi[] = "729";
int t = 0;
for (int i = 0; sayi[i] != '\0'; i++) {
    t += sayi[i] - '0';
}
printf("%d\n", t);
```

(A) `18` (B) `729` (C) `162` (D) `9` (E) `55`

4 `a` ve `b` iki `char` dizisi (string) olsun. Bu iki string’in **içeriğinin aynı olup olmadığını** doğru denetleyen koşul hangisidir?

(A) `a == b`

(B) `a = b`

(C) `strcmp(a, b)`

(D) `strcmp(a, b) == 0`

(E) `strlen(a) == strlen(b)`

5 Aşağıdaki program çalışırken kullanıcı klavyeden `Ali Veli` yazıp Enter’a basıyor. Ekrana ne yazdırılır?

```c
char ad[20];
scanf("%s", ad);
printf("%s %zu\n", ad, strlen(ad));
```

(A) `Ali Veli 8`

(B) `Ali 3`

(C) `Ali 8`

(D) `Veli 4`

(E) Program derlenmez.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[] = "bilgisayar";
    s[4] = '\0';
    printf("%s\n", s);
    printf("%zu %zu\n", strlen(s), sizeof s);
    printf("%c\n", s[5]);
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void) {
    char a[20] = "ab";
    strcat(a, "cd1");
    for (int i = 0; a[i] != '\0'; i++) {
        if (isalpha(a[i]) && i % 2 == 0) {
            a[i] = toupper(a[i]);
        }
    }
    printf("%s %zu\n", a, strlen(a));
    return 0;
}
```

8 Aşağıdaki program çalışırken kullanıcı önce `5` yazıp Enter'a basıyor, `Ali` yazmaya hazırlanıyor. Program ekrana ne yazdırır? Köşeli parantezlerin içine dikkat et.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    int n = 0;
    char ad[20];
    scanf("%d", &n);
    fgets(ad, sizeof ad, stdin);
    printf("%d [%s]\n", n, ad);
    printf("%zu\n", strlen(ad));
    return 0;
}
```

C) Kod Yazma

9 Kullanıcıdan `fgets` ile bir satır oku (en çok 99 karakter). Satırdaki harfleri, rakamları ve boşlukları sayıp aşağıdaki biçimde yazdıran `karakterSay` programını yaz. Satır sonundaki `'\n'` sayılmamalı.

```
Bir satir yaz: Merhaba Dunya 2025!
Harf: 12
Rakam: 4
Bosluk: 2
```

10 Kullanıcıdan 5 isim oku (`scanf`, her isim en çok 19 karakter, boşluksuz). Bu isimlerden **en uzun olanı** ve **alfabetik sıraya göre ilk gelenini** bulup yazdıran `isimAnaliz` programını yaz. En uzun isim birden fazlaysa ilk gelen sayılsın. İsimleri `char isimler[5][20]` dizisinde tut.

```
5 isim: Ali Fatma Mustafa Ayse Can
En uzun isim: Mustafa (7 harf)
Alfabetik ilk: Ali
```

Çözümler

Cevap anahtarı

1 C	2 E	3 A	4 D	5 B

1 Cevap: C

`strlen(s)` `'\0'`'a kadar olan karakterleri sayar: `"Merhaba"` 7 karakter. `sizeof s` dizinin tanımlanan boyutudur: 20. A ve B `sizeof`'u string'in boyutu sanar (B: 7 + `'\0'`), oysa `s` 20 bayt olarak tanımlanmıştır. D iki değeri de bir artırmıştır, `strlen` `'\0'`'ı saymaz. E değerleri ters yazmıştır.

2 Cevap: E

`strcmp` ASCII kodlarını karşılaştırır: `'k'` = 107, `'Z'` = 90. İlk karakterler farklı olduğu için sonuç hemen belli olur ve 107 > 90 olduğundan **pozitiftir**. A alfabe sırası düşüncesidir; `strcmp` alfabeye değil koda bakar ve büyük harfler küçüklerden önce gelir. B yanlış, `strcmp` büyük/küçük harfi ayırt eder. C yanlış, `strcmp`'nin işi tam olarak budur. D yanlış, uzunluk ancak önekler aynıysa devreye girer.

3 Cevap: A

`sayi[i] - '0'` her rakam karakterini kendi değerine çevirir: 7, 2, 9. Toplam 7 + 2 + 9 = 18. B sayıyı kurmak için gereken `* 10` işleminin yapılmadığını görmez. C, `'0'` çıkarılmasaydı çıkacak sonuçtur: 55 + 50 + 57 = 162. D son rakamdır, E ise `'7'` karakterinin kodudur.

4 Cevap: D

`strcmp` iki string aynıysa 0 döndürür; eşitlik denetimi `strcmp(a, b) == 0`'dır. A dizilerin adreslerini karşılaştırır, içeriği değil. B bir atamadır ve diziler için derlenmez. C'de koşul string'ler **farklıyken** doğru olur, çünkü 0 "yanlış"tır. E yalnızca uzunlukları karşılaştırır: `"Ali"` ile `"Can"` aynı uzunluktadır ama aynı string değildir.

5 Cevap: B

`scanf("%s")` boşlukta durur: yalnız `Ali` okunur, `Veli` tamponda kalır; çıktı `Ali 3` olur. A ve C boşluklu metni okuduğunu varsayar; boşluklu satır için `fgets` gerekir. D ikinci sözcüğü alır, oysa ilk sözcük okunur. E yanlış, program derlenir ve çalışır.

6

`s[4] = '\0';` ile "bilgisayar" string'i 4. indekste kesilir (`s[4]` o ana kadar `'i'` idi); dizi artık `b i l g \0 s a y a r \0` biçimindedir.

1. `%s` ilk `'\0'`'da durur: `bilg`.
2. `strlen(s)` = 4. `sizeof s` değişmez: 10 karakter + `'\0'` = 11. Çıktı `4 11`.
3. `s[5]` hâlâ bellektedir: `'s'` karakteri.

Çıktı

```
bilg
4 11
s
```

7

`strcat(a, "cd1")` ile `a` = `"abcd1"` olur (5 karakter). Döngü yalnızca **çift indeksteki harfleri** büyütür: `a[0]` ve `a[2]` çift indekstedir ve harftir, `A` ve `C` olur. `a[1]` ve `a[3]` tek indekstedir, değişmez. `a[4]` çift indekstedir ama `'1'` harf değildir. Sonuç `AbCd1`; `strlen(a)` = 5.

Çıktı
```
AbCd1 5
```

8

`scanf("%d", &n)` yalnız `5`'i okur; Enter'ın `'\n'`'i tamponda kalır. `fgets` tamponda hazır bekleyen `'\n'`'i bulur ve kullanıcıya hiç sorulmadan o tek karakterle döner: `ad` = `"\n"`. Bu yüzden `%s` yalnız bir satır sonu yazdırır ve `]` alt satıra düşer. `strlen(ad)` = 1. `Ali` hiç okunmaz. Düzeltmek için `scanf`'ten sonra `while (getchar() != '\n');` gerekirdi.

Çıktı
```
5
5 [
]
1
```

9

Satırı `fgets` ile okuyup `strcspn` ile `'\n'`'i siliyoruz; böylece `isspace('\n')` yüzünden fazladan boşluk sayılmaz. Her karakter ya harf, ya rakam, ya boşluktur ya da hiçbiridir (`!` gibi); `else if` zinciri tam bir sınıf seçer.

karakterSay.c
```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void) {
    char satir[100];
    int harf = 0, rakam = 0, bosluk = 0;
    printf("Bir satir yaz: ");
    fgets(satir, sizeof satir, stdin);
    satir[strcspn(satir, "\n")] = '\0';
    for (int i = 0; satir[i] != '\0'; i++) {
        unsigned char c = (unsigned char)satir[i];   // Türkçe harf gelebilir
        if (isalpha(c)) {
            harf++;
        } else if (isdigit(c)) {
            rakam++;
        } else if (isspace(c)) {
            bosluk++;
        }
    }
    printf("Harf: %d\nRakam: %d\nBosluk: %d\n", harf, rakam, bosluk);
    return 0;
}
```

Çıktı
```
Bir satir yaz: Merhaba Dunya 2025!
Harf: 12
Rakam: 4
Bosluk: 2
```

10

İsimleri `scanf("%19s", ...)` ile `isimler[i]` satırlarına okuyoruz. En uzun ve alfabetik ilk isimlerin **indekslerini** tutarız (`enUzun`, `ilk`), isimleri kopyalamayız. İkisi de ilk isimle başlar, döngü 1'den devam eder. En uzun için `>` kullandığımızdan eşit uzunlukta ilk gelen korunur.

isimAnaliz.c
```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char isimler[5][20];
    printf("5 isim: ");
    for (int i = 0; i < 5; i++) {
        scanf("%19s", isimler[i]);
    }
    int enUzun = 0, ilk = 0;
    for (int i = 1; i < 5; i++) {
        if (strlen(isimler[i]) > strlen(isimler[enUzun])) {
            enUzun = i;
        }
        if (strcmp(isimler[i], isimler[ilk]) < 0) {
            ilk = i;
        }
    }
    printf("En uzun isim: %s (%zu harf)\n", isimler[enUzun],
           strlen(isimler[enUzun]));
    printf("Alfabetik ilk: %s\n", isimler[ilk]);
    return 0;
}
```

Çıktı
```
5 isim: Ali Fatma Mustafa Ayse Can
En uzun isim: Mustafa (7 harf)
Alfabetik ilk: Ali
```

PROJE 1

Öğrenci Not Sistemi

İlk sekiz bölümde öğrendiklerini ilk kez **tek bir gerçek programda** birleştiriyoruz: `fgets` ile güvenli girdi okuma (Bölüm 3 ve 8), `strtol` ile sayı dönüşümü (Bölüm 8), koşullar ve `switch` (Bölüm 4), döngüler (Bölüm 5), fonksiyonlar ve prototipler (Bölüm 6), `#define` sabitleri ve diziler (Bölüm 2 ve 7), `string.h` ile `ctype.h` (Bölüm 8). Kodu okumadan önce kendin yazmayı dene; takıldığın yerde "Tasarım" ve "Kod" başlıklarındaki açıklamalara bak.

Bu projede `struct`, `malloc` ve işaretçi aritmetiği **bilerek kullanılmadı** (tek istisna, Bölüm 8'de kalıp olarak gördüğün `strtol` ile birlikte gelen `char *son`); onları Bölüm 9–11'de öğreneceksin ve bu projeyi o zaman yeniden yazacaksın.

Amaç

Bir dersin hocasının işini kolaylaştıran bir konsol programı yazacağız. Hoca öğrencilerin adlarını ve notlarını girecek; program her öğrencinin ortalamasını ve harf notunu hesaplayacak, sınıfın durumunu özetleyecek ve bir menü üzerinden istenen bilgiyi gösterecek. Kullanıcı ne yazarsa yazsın program **çökmemeli**.

Gereksinimler

- Program önce öğrenci sayısını sorar (1–50 arası).
- Her öğrenci için **ad soyad**, **vize** ve **final** notu girilir. Notlar 0–100 arası tam sayıdır. Geçersiz girişte (harf, boş satır, aralık dışı sayı) program uyarır ve **aynı soruyu tekrar sorar** (boş bırakılan ad soyad için uyarı yazmadan yalnız tekrar sorar).
- Ortalama = vize × 0.4 + final × 0.6.
- Harf notu aşağıdaki tabloya göre verilir (ortalama, harfin alt sınırına eşit veya ondan büyükse o harf alınır):

Harf	AA	BA	BB	CB	CC	DC	DD	FD	FF
En düşük ortalama	90	85	80	75	70	65	60	50	0

- Veriler girildikten sonra bir **menü** gösterilir ve kullanıcı `0` seçene kadar tekrar edilir:
  - `1` → Bütün öğrencileri tablo halinde listele (no, ad, vize, final, ortalama, harf).
  - `2` → İstatistik: sınıf ortalaması, en yüksek ve en düşük ortalamalı öğrenci.
  - `3` → Sınıf ortalamasının üstündeki öğrenciler.
  - `4` → Harf notu dağılımı: her harf için o harfi alan öğrenci sayısı kadar `*`.
  - `5` → İsimle arama: aranan metin ismin herhangi bir yerinde geçebilir; büyük/küçük harf önemsizdir (`ibra` yazınca `Ibrahim` bulunmalı).
  - `0` → Çıkış.

Tasarım

Veriler: paralel diziler

Her öğrencinin dört bilgisi var: ad, vize, final, ortalama. Bunları dört ayrı dizide tutacağız. Dizilerin **aynı indeksi aynı öğrenciye** aittir; bu yapıya **paralel diziler** (parallel arrays) denir:

```
indeks         0              1              2              3
adlar       Ayse Yilmaz    Mehmet Can     Zeynep Kaya    Ibrahim Demir
vizeler         70             50             95             60
finaller        85             58             88             40
ortalamalar     79.0           54.8           90.8           48.0
```

Adlar, `char adlar[MAKS][AD_BOY]` biçiminde **iki boyutlu bir `char` dizisidir** (Bölüm 8): her satır bir ad tutan, sonu `'\0'` ile biten bir string'dir. `adlar[2]` üçüncü öğrencinin adıdır ve `printf("%s", adlar[2])` ile yazdırılır. Dizi boyutu derleme sırasında sabit olmak zorunda olduğu için `MAKS = 50` yer ayırırız; öğrenci sayısı `n` ise program çalışırken öğrenilir. Yani dizilerin yalnız **ilk `n` hücresi doludur** ve her fonksiyon `n`'i parametre olarak alıp döngüsünü `n`'e kadar çevirir. Dolmamış hücreler çöp taşır; onlara hiç dokunmuyoruz.

Bölüm 11'de bu dört diziyi tek bir `struct`'ta toplayacaksın; o zaman bütün fonksiyonlar dört dizi yerine tek bir dizi alacak.

Sabitler: harf tablosu

Harf tablosunu `if–else` zinciri yerine **iki paralel sabit dizi** ile tutuyoruz: `HARFLER` ve `ALT_SINIRLAR`. Bir ortalamanın harfini bulmak için `ALT_SINIRLAR` dizisinde baştan yürürüz; ortalamanın geçtiği ilk sınırın indeksi, harfin indeksidir. Son sınır 0 olduğu için geçerli bir ortalamada döngü her zaman durur; yine de `harfIndeksi`'ye `i < HARF_SAYISI - 1` koşulunu ekledik: hatalı bir negatif değer gelirse dizinin dışına çıkmak (tanımsız davranış) yerine son harfte (`FF`) durulur. Bu diziler fonksiyonların dışında, dosyanın üstünde `const` olarak tanımlanır; Bölüm 6'da "global değişkeni yalnızca değişmeyen değerler için kullan" demiştik, bu tam öyle bir durumdur. Tablo değişirse yalnız bu iki satırı düzeltmek yeter. Boyutlar da `#define` ile adlandırıldı: `MAKS` (50), `AD_BOY` (40), `SATIR_BOY` (100) ve `HARF_SAYISI` (9).

Fonksiyonlar

Her işi tek bir fonksiyona veriyoruz. `main` yalnız verileri okur ve menü döngüsünü çalıştırır; asıl işi fonksiyonlar yapar:

Fonksiyon	Görevi	Dönüş tipi
`satirOku(mesaj, satir, boy)`	Mesajı yazar, bir satırı güvenle okur, `'\n'`'i siler	`void`
`sagKirp(s)`	Sondaki boşlukları siler (`"70 "` → `"70"`)	`void`
`solKirp(s)`	Baştaki boşlukları siler (`" Ali"` → `"Ali"`)	`void`
`sayiOku(mesaj, min, max)`	`min`–`max` arası geçerli bir tam sayı girilene kadar sorar	`int`
`metinOku(mesaj, hedef, boy)`	Boş olmayan bir satır okur	`void`
`ortalamaHesapla(vize, fin)`	%40 vize + %60 final	`double`
`harfIndeksi(ort)`	Ortalamanın `HARFLER` dizisindeki indeksi	`int`
`sinifOrtalamasi(ort, n)`	Ortalamalar dizisinin ortalaması	`double`
`kucult(s)`	String'i yerinde küçük harfe çevirir	`void`
`menuYazdir()`	Menüyü ekrana yazar	`void`
`listele(ad, vize, fin, ort, n)`	Öğrenci tablosunu yazar	`void`
`istatistik(ad, ort, n)`	Sınıf ortalaması, en yüksek, en düşük	`void`
`ortalamaUstu(ad, ort, n)`	Ortalamanın üstündekileri yazar	`void`
`histogram(ort, n)`	Harf notu dağılımını yıldızlarla çizer	`void`
`ara(ad, ort, n)`	Aranacak metni sorar, isimde geçenleri yazar	`void`

Bu tabloda `harfNotu` gibi harfi **string olarak döndüren** bir fonksiyon yoktur: C'de fonksiyonun string döndürmesi işaretçi bilgisi ister (Bölüm 9). Biz bunun yerine indeksi döndürüp `HARFLER[indeks]` yazdırıyoruz. Diziyi yalnız okuyan fonksiyonlar parametreyi `const` alır (Bölüm 7); `kucult` ve `sagKirp` string'i değiştirdiği için `const` almaz.

Kod

Programı kaydet (`notSistemi.c`), derle ve çalıştır. Windows'ta Türkçe harfler bozuk görünürse Bölüm 3'teki gibi önce `chcp 65001` yaz.

```
gcc -std=c17 -Wall -Wextra notSistemi.c -o notSistemi
./notSistemi
```

Önce programın tamamına göz at; sonraki başlıklar önemli kısımların neden öyle yazıldığını anlatıyor.

Tam program

notSistemi.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAKS 50          // en fazla öğrenci sayısı
#define AD_BOY 40        // ad soyad dizisinin boyutu ('\0' dahil)
#define SATIR_BOY 100    // bir girdi satırı için tampon boyutu
#define HARF_SAYISI 9    // harf notu sayısı

// Harf notları ve alt sınırları: aynı indeksler birbirine karşılık gelir
const char HARFLER[HARF_SAYISI][3] = {
    "AA", "BA", "BB", "CB", "CC", "DC", "DD", "FD", "FF"
```

```
};
const int ALT_SINIRLAR[HARF_SAYISI] = {90, 85, 80, 75, 70, 65, 60, 50, 0};

// Prototipler
void satirOku(const char mesaj[], char satir[], int boy);
void sagKirp(char s[]);
void solKirp(char s[]);
int sayiOku(const char mesaj[], int min, int max);
void metinOku(const char mesaj[], char hedef[], int boy);
double ortalamaHesapla(int vize, int fin);
int harfIndeksi(double ort);
double sinifOrtalamasi(const double ort[], int n);
void kucult(char s[]);
void menuYazdir(void);
void listele(const char ad[][AD_BOY], const int vize[], const int fin[],
             const double ort[], int n);
void istatistik(const char ad[][AD_BOY], const double ort[], int n);
void ortalamaUstu(const char ad[][AD_BOY], const double ort[], int n);
void histogram(const double ort[], int n);
void ara(const char ad[][AD_BOY], const double ort[], int n);

int main(void) {
    char adlar[MAKS][AD_BOY];
    int vizeler[MAKS];
    int finaller[MAKS];
    double ortalamalar[MAKS];

    printf("===== ÖĞRENCİ NOT SİSTEMİ =====\n");
    int n = sayiOku("Öğrenci sayısı: ", 1, MAKS);

    for (int i = 0; i < n; i++) {
        printf("--- %d. öğrenci ---\n", i + 1);
        metinOku("Ad soyad: ", adlar[i], AD_BOY);
        vizeler[i] = sayiOku("Vize: ", 0, 100);
        finaller[i] = sayiOku("Final: ", 0, 100);
        ortalamalar[i] = ortalamaHesapla(vizeler[i], finaller[i]);
    }

    int secim = 0;
    do {
        menuYazdir();
        secim = sayiOku("Seçimin: ", 0, 5);
        switch (secim) {
            case 1:
                listele(adlar, vizeler, finaller, ortalamalar, n);
                break;
            case 2:
                istatistik(adlar, ortalamalar, n);
                break;
            case 3:
                ortalamaUstu(adlar, ortalamalar, n);
                break;
            case 4:
                histogram(ortalamalar, n);
                break;
            case 5:
                ara(adlar, ortalamalar, n);
                break;
            case 0:
                printf("Program kapatıldı. İyi çalışmalar!\n");
                break;
        }
    } while (secim != 0);

    return 0;
```

```c
}

// ---------- Girdi okuma ----------

// Mesajı yazar, bir satır okur; '\n' silinir, baştaki/sondaki boşluklar atılır
void satirOku(const char mesaj[], char satir[], int boy) {
    printf("%s", mesaj);
    if (fgets(satir, boy, stdin) == NULL) {
        printf("\nGirdi bitti, program kapatılıyor.\n");
        exit(1);
    }
    int uzunluk = (int)strcspn(satir, "\n");
    if (satir[uzunluk] == '\0') {        // '\n' yok: satır tampona sığmadı
        int c = 0;
        int atilan = 0;
        while ((c = getchar()) != '\n' && c != EOF) {
            atilan++;
        }
        if (atilan > 0) {                // sessizce kesme, kullanıcıya söyle
            printf("  Uyarı: girdi uzundu, yalnız ilk %d bayt alındı.\n",
                   boy - 1);
        }
    }
    satir[uzunluk] = '\0';
    sagKirp(satir);
    solKirp(satir);
}

void sagKirp(char s[]) {
    int n = (int)strlen(s);
    while (n > 0 && isspace((unsigned char)s[n - 1])) {
        n--;
        s[n] = '\0';
    }
}

// Baştaki boşlukları atlar, kalan karakterleri sola kaydırır
void solKirp(char s[]) {
    int bas = 0;
    while (isspace((unsigned char)s[bas])) {
        bas++;
    }
    int i = 0;
    while (s[bas + i] != '\0') {
        s[i] = s[bas + i];
        i++;
    }
    s[i] = '\0';
}

// min–max aralığında geçerli bir tam sayı girilene kadar tekrar sorar
int sayiOku(const char mesaj[], int min, int max) {
    char satir[SATIR_BOY];
    while (1) {
        satirOku(mesaj, satir, SATIR_BOY);
        char *son = NULL;
        long deger = strtol(satir, &son, 10);
        if (son != satir && *son == '\0' && deger >= min && deger <= max) {
            return (int)deger;
        }
        printf("  Geçersiz! %d–%d arası bir tam sayı gir.\n", min, max);
    }
}

// Boş bırakılamayan bir satır okur (ad soyad için)
```

```c
void metinOku(const char mesaj[], char hedef[], int boy) {
    do {
        satirOku(mesaj, hedef, boy);
    } while (hedef[0] == '\0');
}

// ---------- Hesaplama ----------

// vize * 0.4 + final * 0.6; ondalık hatası olmasın diye tam sayıyla hesaplanır
double ortalamaHesapla(int vize, int fin) {
    return (vize * 4 + fin * 6) / 10.0;
}

// Ortalamanın HARFLER dizisindeki indeksi
int harfIndeksi(double ort) {
    int i = 0;
    // i sınırı: ort negatif gelse bile dizinin dışına çıkma
    while (i < HARF_SAYISI - 1 && ort < ALT_SINIRLAR[i]) {
        i++;
    }
    return i;
}

double sinifOrtalamasi(const double ort[], int n) {
    double toplam = 0;
    for (int i = 0; i < n; i++) {
        toplam += ort[i];
    }
    return toplam / n;
}

// String'i yerinde küçük harfe çevirir (yalnız ASCII harfler değişir)
void kucult(char s[]) {
    for (int i = 0; s[i] != '\0'; i++) {
        s[i] = (char)tolower((unsigned char)s[i]);
    }
}

// ---------- Menü işlemleri ----------

void menuYazdir(void) {
    printf("\n");
    printf("1) Liste  2) İstatistik  3) Ortalama üstü\n");
    printf("4) Harf dağılımı  5) İsimle ara  0) Çıkış\n");
}

void listele(const char ad[][AD_BOY], const int vize[], const int fin[],
             const double ort[], int n) {
    printf("%-3s %-20s %5s %6s %6s %5s\n",
           "No", "Ad Soyad", "Vize", "Final", "Ort.", "Harf");
    printf("-----------------------------------------------------\n");
    for (int i = 0; i < n; i++) {
        printf("%-3d %-20s %5d %6d %6.1f %5s\n", i + 1, ad[i], vize[i], fin[i],
               ort[i], HARFLER[harfIndeksi(ort[i])]);
    }
}

void istatistik(const char ad[][AD_BOY], const double ort[], int n) {
    int enYuksek = 0;
    int enDusuk = 0;
    for (int i = 1; i < n; i++) {
        if (ort[i] > ort[enYuksek]) {
            enYuksek = i;
        }
        if (ort[i] < ort[enDusuk]) {
```

```
            enDusuk = i;
        }
    }
    printf("Sınıf ortalaması: %.2f\n", sinifOrtalamasi(ort, n));
    printf("En yüksek       : %s (%.1f)\n", ad[enYuksek], ort[enYuksek]);
    printf("En düşük        : %s (%.1f)\n", ad[enDusuk], ort[enDusuk]);
}

void ortalamaUstu(const char ad[][AD_BOY], const double ort[], int n) {
    double sinifOrt = sinifOrtalamasi(ort, n);
    int sayac = 0;
    printf("Sınıf ortalamasının (%.2f) üstündekiler:\n", sinifOrt);
    for (int i = 0; i < n; i++) {
        if (ort[i] > sinifOrt) {
            printf("  %-20s %5.1f\n", ad[i], ort[i]);
            sayac++;
        }
    }
    printf("Toplam %d öğrenci.\n", sayac);
}

void histogram(const double ort[], int n) {
    int sayac[HARF_SAYISI] = {0};
    for (int i = 0; i < n; i++) {
        sayac[harfIndeksi(ort[i])]++;
    }
    printf("Harf notu dağılımı:\n");
    for (int i = 0; i < HARF_SAYISI; i++) {
        printf("  %s | ", HARFLER[i]);
        for (int j = 0; j < sayac[i]; j++) {
            putchar('*');
        }
        putchar('\n');
    }
}

// Büyük/küçük harf duyarsız, isim parçasıyla arama
void ara(const char ad[][AD_BOY], const double ort[], int n) {
    char aranan[AD_BOY];
    char kAranan[AD_BOY];
    char kAd[AD_BOY];
    int bulundu = 0;

    metinOku("Aranacak isim: ", aranan, AD_BOY);
    strcpy(kAranan, aranan);            // ikisi de AD_BOY boyutlu: sığar
    kucult(kAranan);
    for (int i = 0; i < n; i++) {
        strcpy(kAd, ad[i]);
        kucult(kAd);
        if (strstr(kAd, kAranan) != NULL) {
            printf("  %s -> ortalama %.1f (%s)\n",
                   ad[i], ort[i], HARFLER[harfIndeksi(ort[i])]);
            bulundu = 1;
        }
    }
    if (!bulundu) {
        printf("  \"%s\" bulunamadı.\n", aranan);
    }
}
```

1. Çökmeyen girdi okuma

Programın en çok emek isteyen kısmı budur. **Bütün** girdileri, sayı olsun ad olsun, `fgets` ile **satır olarak** okuyoruz; `scanf` hiç kullanılmıyor. Böylece Bölüm 3 ve 8'de gördüğün `scanf` ile `fgets`'i karıştırma tuzağı (tamponda kalan `'\n'`'in sonraki `fgets`'e boş satır olarak gitmesi) hiç oluşmaz. `satirOku` her çağrıda sırayla şunları yapar:

Adım	Kod	Ne işe yarar?
1	`printf("%s", mesaj)`	Soruyu yazar. `printf(mesaj)` yazma: mesajda `%` olursa biçim belirteci sanılır
2	`fgets(...) == NULL`	Girdi tümüyle bittiyse (Ctrl+D, Windows'ta Ctrl+Z) `NULL` döner; tekrar soran döngüler sonsuza kadar dönmesin diye programı `exit(1)` ile bitiririz (`stdlib.h`)
3	`strcspn(satir, "\n")`	`'\n'`'in indeksini verir (yoksa uzunluğu)
4	`satir[uzunluk] == '\0'`	O indekste `'\n'` yoksa satır tampona **sığmamıştır**. Artan harfler sonraki soruya karışmasın diye `getchar` döngüsüyle (Bölüm 5) satırın sonuna kadar silinir; atılan bir şey varsa kullanıcı uyarılır (girdi sessizce kesilmez)
5	`satir[uzunluk] = '\0'`	`'\n'`'i siler
6	`sagKirp`, `solKirp`	`"70 "`, `" Ali"` gibi sonda ve başta kalan boşlukları atar (baştaki boşluk tablo hizasını bozardı); boşluk dolu satır boş sayılır

`sayiOku` bu satırı sayıya çevirir. `strtol(satir, &son, 10)`, `son` değişkenine **dönüşümün durduğu yeri** yazar (`char *son = NULL;` bir işaretçidir; Bölüm 9'a kadar kalıp olarak al). Sayının geçerli olması için dört koşul birlikte doğru olmalıdır:

Koşul	Yakaladığı geçersiz girdi
`son != satir`	`"abc"` ve `""` (boş satır): hiç rakam okunamadı
`*son == '\0'`	`"7.5"` ve `"12ab"`: sayıdan sonra artan karakter var
`deger >= min`	`"-3"` ve `"0"` (öğrenci sayısı sorulurken)
`deger <= max`	`"105"` ve `"99999999999999999999"` (taşan sayıda `strtol` `LONG_MAX` döndürür, yine aralığa takılır)

Bir koşul bile yanlışsa uyarı yazılır ve `while (1)` döngüsü soruyu tekrarlar. Geçerli sayı bulununca `return` hem fonksiyondan hem döngüden çıkar. `metinOku` da benzer biçimde, kullanıcı boş satır girdiği sürece soruyu tekrarlayan bir `do-while`'dır. Ad soyad `AD_BOY` (40) baytlık diziye okunur; daha uzun bir isim kesilir, fazlası `satirOku` tarafından silinir.

2. Ortalama ve harf notu

`ortalamaHesapla`, `vize * 0.4 + fin * 0.6` yerine `(vize * 4 + fin * 6) / 10.0` hesaplar. İki formül matematikte aynıdır; ama Bölüm 2'de gördüğün gibi `0.4` ve `0.6` bilgisayarda **tam olarak** saklanamaz ve harf sınırında yanlış not verilebilir:

Sık Yapılan Hata: Sınırda yanlış harf notu

```c
#include <stdio.h>

int main(void) {
    int vize = 6, fin = 96;
    double a = vize * 0.4 + fin * 0.6;
    double b = (vize * 4 + fin * 6) / 10.0;
    printf("0.4 ve 0.6 ile: %.1f, 60 ve üstü mü? %d\n", a, a >= 60);
    printf("tam sayıyla   : %.1f, 60 ve üstü mü? %d\n", b, b >= 60);
    return 0;
}
```

Çıktı

```
0.4 ve 0.6 ile: 60.0, 60 ve üstü mü? 0
tam sayıyla   : 60.0, 60 ve üstü mü? 1
```

Gerçek ortalama tam 60, yani **DD**. İlk formülün sonucu ise 60'ın çok az altında kalıyor; `%.1f` yuvarlayarak yazdığı için ekranda yine `60.0` görünür, ama `>= 60` karşılaştırması yanlış çıkar ve öğrenci tabloda "60.0" yazmasına rağmen **FD** alır. İkinci formülde çarpma ve toplama tam sayılarla yapılır, en sonda tek bir bölme kalır; sonuç doğru çıkar.

`harfIndeksi`, ortalama `ALT_SINIRLAR[i]`'den küçük olduğu sürece `i`'yi artırır. Örneğin 79.0 için 90, 85 ve 80'den küçüktür ama 75'ten küçük değildir; döngü `i = 3`'te durur ve `HARFLER[3]` = `"CB"` olur. Sınırdaki `60.0` için döngü `i = 6`'da durur (`60.0 < 60` yanlıştır), yani `"DD"`. Eşitlik durumunda yukarıdaki harfe geçmek için karşılaştırmanın `<` olması gerekir; `<=` yazsaydın 60.0 bir alt harfe, `"FD"`'ye düşerdi.

3. Menü

Menü bir `do-while` döngüsüdür: Menü **en az bir kere** gösterilmeli, kullanıcı `0` seçene kadar tekrarlanmalıdır. Seçim yine `sayiOku` ile 0–5 aralığında okunduğu için `switch`'e geçersiz bir değer hiç gelmez; bu yüzden `default` dalına gerek kalmadı. Her `case`'in sonundaki `break`'i unutma (Bölüm 4); unutursan seçim 1 bütün sonraki seçenekleri de çalıştırır.

`main` dört diziyi tanımlar, `n` kadar öğrenciyi `for` ile doldurur, sonra menüyü çalıştırır. Dizilere dokunan her fonksiyona `n` geçirilir. Dizileri fonksiyona geçirmek **kopya** oluşturmaz: fonksiyon `main`'deki dizinin kendisiyle çalışır (Bölüm 7), bu yüzden `const` onları yanlışlıkla bozmaktan korur.

4. Tablo, istatistik ve histogram

- `listele`, Bölüm 3'teki `printf` genişlikleriyle hizalı bir tablo çizer: `%-20s` adı 20 karakterlik alana **sola**, `%6.1f` ortalamayı 6 karakterlik alana **sağa** yaslar. Her sütun genişliği başlık satırıyla veri satırında aynıdır; ayraç çizgisi bu genişliklerin toplamı kadardır (50).
- `istatistik`, en yüksek ve en düşük ortalamanın **değerini değil indeksini** tutar (Bölüm 7). İndeksi bilirsek o öğrencinin adına da `ad[enYuksek]` ile ulaşırız. Arama `i = 1`'den başlar, çünkü ilk eleman başlangıçta hem en yüksek hem en düşük sayılır; `n` en az 1 olduğu için `ort[0]` her zaman vardır.
- `histogram`, Bölüm 8'deki harf frekansı (`int sayac[26]`) fikrinin aynısıdır: `HARFLER` ile aynı boyutta bir sayaç dizisi tanımlayıp her öğrenci için `sayac[harfIndeksi(ort[i])]++` yapar. Yıldızları iç `for` döngüsü tek tek `putchar('*')` ile çizer.

İpucu: Tek komutla deneme

Menüyü denemek için öğrencileri her seferinde elle yazmak sıkıcıdır. Girdileri satır satır bir metin dosyasına (`girdi.txt`) yaz ve programa dosyadan ver. `fgets` zaten satır okuduğu için hiçbir şeyi değiştirmen gerekmez:

```
./notSistemi < girdi.txt                      (Linux, macOS, MSYS2 terminali)
.\notSistemi.exe < girdi.txt                  (Windows cmd)
Get-Content girdi.txt | .\notSistemi.exe      (Windows PowerShell)
```

Hata ayıklarken önce `n = 1` ile başla; bir fonksiyon bozuksa tek öğrenciyle de bozuktur.

5. Büyük/küçük harf duyarsız arama

`ara`, hem aranan metnin hem her ismin **küçültülmüş kopyasını** yapar ve `strstr` ile karşılaştırır. Kopya şarttır: `ad` dizisi `const`'tur ve ayrıca asıl isim tabloda büyük harfle yazıldığı gibi görünmeye devam etmelidir. `strcpy` güvenlidir, çünkü kopyalanan stringler (`aranan`, `ad[i]`) ile hedefler (`kAranan`, `kAd`) aynı `AD_BOY` boyutundaki dizilerdir (Bölüm 8'de `strcpy` için koyduğumuz "sığıyor mu?" kuralı burada yapıdan gelir). `strstr`, `NULL` döndürmediyse aranan metin isimde geçiyor demektir.

Sık Yapılan Hata: `tolower` Türkçe harfleri çeviremez

`tolower` **tek bir bayt** üzerinde çalışır ve yalnız ASCII harfleri (`A–Z`) küçültür. UTF-8'de `İ`, `Ş`, `Ü` gibi Türkçe harfler **iki bayttır** ve ikisi de ASCII aralığının dışındadır; `tolower` onlara dokunmaz. Sonuç olarak `İ` küçülmez, `ibra` araması `İbrahim`'i bulamaz:

turkceTolower.c

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

void kucult(char s[]) {
    for (int i = 0; s[i] != '\0'; i++) {
        s[i] = (char)tolower((unsigned char)s[i]);
    }
}

int main(void) {
    char ad[] = "İbrahim";
    char a1[] = "ibra";
    char a2[] = "İBRA";
    kucult(ad);
    kucult(a1);
    kucult(a2);
    printf("ad = %s\n", ad);
    printf("ibra -> %s\n", strstr(ad, a1) != NULL ? "bulundu" : "bulunamadı");
    printf("İBRA -> %s\n", strstr(ad, a2) != NULL ? "bulundu" : "bulunamadı");
    printf("strlen(\"İ\") = %zu\n", strlen("İ"));
    return 0;
}
```

Çıktı

```
ad = İbrahim
ibra -> bulunamadı
İBRA -> bulundu
strlen("İ") = 2
```

`İ` iki bayt olduğu için `strlen("İ")` 2 verir ve `kucult` o iki baytı olduğu gibi bırakır. Bu yüzden `İBRA` bulundu (iki taraftaki `İ` de aynı kaldı) ama `ibra` bulunamadı. Aynı sorun `Ş` ile `ş`, `Ü` ile `ü` için de geçerlidir; üstelik Türkçede `I`'nın küçüğü `ı`'dır, oysa `tolower('I')` `i` verir. Doğru çözüm UTF-8'i bilen bir kütüphane ister; bu kitabın kapsamı dışındadır. **Bu yüzden örnek çalışmada arama ASCII harflerle yapıldı** ve test verisindeki adlarda Türkçe harf yok. Sınavda "tolower ile Türkçe harf küçültülür mü?" sorusunun cevabı hayırdır.

Not: Hizalama ve Türkçe harfler

`printf("%-20s")` genişliği **bayt** olarak sayar, karakter olarak değil. `Ayşe` 4 karakterdir ama 5 bayttır; tabloda bir boşluk eksik kalır ve sonraki sütunlar bir karakter sola kayar. Kitaptaki örnek veride Türkçe harf olmaması bu yüzdendir. Çözüm yolunu "Kendin Geliştir" bölümünde bulacaksın.

Sık Yapılan Hata: `scanf` ile `fgets`'i karıştırma

Projeyi yazarken öğrenci sayısını `scanf("%d", &n)` ile, adı `fgets` ile okumak çok cazip gelir. Yapma: `scanf` sayıyı alır ama sonundaki `'\n'`'i tamponda bırakır; ardından gelen `fgets` hemen o `'\n'`'i görür ve ad olarak **boş satır** okur. Program ad sormadan vize sormaya geçmiş gibi görünür. Bu projede tek kural şudur: **her şeyi `fgets` ile satır olarak oku, sayıya sonradan `strtol` ile çevir.** Okuduğun satırın sonundaki `'\n'`'i silmeyi de unutma: silmezsen `strtol` işleminde `*son` `'\n'` olur ve her sayı "geçersiz" sayılır, adlar da tabloda ikiye bölünür.

Örnek çalışma

Aşağıdaki çalışmada 6 öğrenci giriliyor. Öğrenci sayısına önce `0`, sonra `dört` yazılıyor; ikisi de reddediliyor. Mehmet'ten sonra ad yerine boş satır bırakılıyor ve Zeynep'in vizesine önce `105`, sonra `abc` yazılıyor; Ibrahim'in finaline `40` yerine yanlışlıkla `40` (harf O) yazılıyor. Burak'ın notları `6` ve `96` olarak seçildi: ortalama tam 60.0 olmalı ve **DD** çıkmalı. Ardından menünün bütün seçenekleri sırayla deneniyor: iki farklı arama (`AY` ve `IBRA`), bulunamayan bir arama (`xyz`), geçersiz bir menü seçimi (`7`) ve çıkış.

Çıktı

```
===== ÖĞRENCİ NOT SİSTEMİ =====
Öğrenci sayısı: 0
  Geçersiz! 1-50 arası bir tam sayı gir.
Öğrenci sayısı: dört
  Geçersiz! 1-50 arası bir tam sayı gir.
Öğrenci sayısı: 6
--- 1. öğrenci ---
Ad soyad: Ayse Yilmaz
Vize: 70
Final: 85
--- 2. öğrenci ---
Ad soyad: Mehmet Can
Vize: 50
Final: 58
--- 3. öğrenci ---
Ad soyad:
Ad soyad: Zeynep Kaya
Vize: 105
  Geçersiz! 0-100 arası bir tam sayı gir.
Vize: abc
  Geçersiz! 0-100 arası bir tam sayı gir.
Vize: 95
Final: 88
--- 4. öğrenci ---
Ad soyad: Ibrahim Demir
```

```
Vize: 60
Final: 40
  Geçersiz! 0-100 arası bir tam sayı gir.
Final: 40
--- 5. öğrenci ---
Ad soyad: Burak Sen
Vize: 6
Final: 96
--- 6. öğrenci ---
Ad soyad: Elif Aydin
Vize: 82
Final: 76

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 1
No  Ad Soyad              Vize  Final   Ort.  Harf
-------------------------------------------------
1   Ayse Yilmaz             70     85   79.0    CB
2   Mehmet Can              50     58   54.8    FD
3   Zeynep Kaya             95     88   90.8    AA
4   Ibrahim Demir           60     40   48.0    FF
5   Burak Sen                6     96   60.0    DD
6   Elif Aydin              82     76   78.4    CB

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 2
Sınıf ortalaması: 68.50
En yüksek       : Zeynep Kaya (90.8)
En düşük        : Ibrahim Demir (48.0)

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 3
Sınıf ortalamasının (68.50) üstündekiler:
  Ayse Yilmaz           79.0
  Zeynep Kaya           90.8
  Elif Aydin            78.4
Toplam 3 öğrenci.

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 4
Harf notu dağılımı:
  AA | *
  BA |
  BB |
  CB | **
  CC |
  DC |
  DD | *
  FD | *
  FF | *

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 5
Aranacak isim: AY
  Ayse Yilmaz -> ortalama 79.0 (CB)
  Zeynep Kaya -> ortalama 90.8 (AA)
  Elif Aydin -> ortalama 78.4 (CB)

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
```

```
Seçimin: 5
Aranacak isim: IBRA
  Ibrahim Demir -> ortalama 48.0 (FF)

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 5
Aranacak isim: xyz
  "xyz" bulunamadı.

1) Liste  2) İstatistik  3) Ortalama üstü
4) Harf dağılımı  5) İsimle ara  0) Çıkış
Seçimin: 7
  Geçersiz! 0-5 arası bir tam sayı gir.
Seçimin: 0
Program kapatıldı. İyi çalışmalar!
```

Sonucu kontrol et: Burak'ın ortalaması 60.0 ve harfi **DD**; formülü `0.4/0.6` ile yazsaydık tabloda "60.0 FD" görecektik. `AY` araması büyük harfle yazılmasına rağmen **Ay**se Yilmaz'ı, Zeynep K**ay**a'yı ve Elif **Ay**din'i buldu; `IBRA` da `Ibrahim Demir`'i buldu. `xyz` için "bulunamadı" mesajı çıktı ve menü kapanmadı; `7` ise menüye ulaşmadan `sayiOku` tarafından reddedildi.

Kendin Geliştir

- **Final barajı:** Finalden 50'nin altında alan öğrenci, ortalaması ne olursa olsun `FF` alsın. Hangi fonksiyonu değiştirmen gerektiğini düşün: `ortalamaHesapla` yalnız vize ve final alıyor, harfi ise `harfIndeksi` buluyor.
- **Geçti/kaldı:** İstatistik ekranına geçen (DD ve üstü) ve kalan öğrenci sayısını ekle. `ALT_SINIRLAR[6]` değerini sabit olarak adlandırmak işini kolaylaştırır.
- **Sıralı liste:** Öğrencileri ortalamaya göre büyükten küçüğe listeleyen yeni bir menü seçeneği ekle. Sıralarken dört dizinin elemanlarını **birlikte** yer değiştirmelisin; adı yer değiştirmek için `strcpy` ve geçici bir `char` dizisi gerekir. Bölüm 11'de bu dört diziyi tek bir `struct` dizisinde toplayınca yer değiştirme tek bir atamaya iner.
- **Not güncelleme:** İsmi verilen öğrencinin vize/final notunu değiştirip ortalamasını yeniden hesaplayan bir seçenek ekle. Tam isim karşılaştırması için `strcmp` kullan.
- **Hizalı Türkçe isimler:** UTF-8'de bir karakter, ilk iki biti `10` olmayan bayttan başlar (`(bayt & 0xC0) != 0x80`). Bir ismin karakter sayısını bu kuralla say, `%-20s` yerine `%s` yazıp kalan boşluğu `20 - karakterSayisi` kadar elle yazdır.
- **Standart sapma:** Sınıf ortalamasının yanında notların standart sapmasını da göster: $\sqrt{\Sigma(x_i - ort)^2 / n}$. `math.h` kullanırsan Linux'ta `-lm` ile derlemeyi unutma.
- **Bütünleme:** `FD` veya `FF` alan öğrenciler için bütünleme notu iste; bütünleme notu final yerine geçsin.
- **Dosyaya kaydetme:** Çıkarken bütün öğrencileri bir metin dosyasına yaz, açılışta oradan oku (Bölüm 12). Böylece hoca her seferinde notları baştan girmek zorunda kalmaz.
- **Sınırsız öğrenci:** `MAKS` sınırını kaldırmak için diziler `malloc` ile `n` boyutunda oluşturulabilir (Bölüm 10).

DENEME SINAVI

Deneme Vize Sınavı

Bu deneme sınavı kitabın ilk sekiz bölümünü kapsar: derleme süreci, değişkenler ve operatörler, `printf` / `scanf`, koşullar, döngüler, fonksiyonlar, diziler ve string'ler. Soru türleri ve zorluk, bilgisayar mühendisliği 1. sınıf vize sınavlarına göre ayarlandı; tuzaklar **bilerek** kondu. Kolay görünen sorulara özellikle dikkat et: Hocalar tamsayı bölmesini, `++` / `--` sırasını, kısa devreyi, `switch` düşmesini ve `'\0'`'ı sever.

Bölüm	Soru sayısı	Soru başına puan	Toplam
A) Çoktan seçmeli	10	4	40
B) Çıktı tahmini	4	6	24
C) Kod yazma	3	12	36
Toplam	**17**		**100**

Süre: 75 dakika. Bilgisayar, kitap ve not kullanmak yok; sadece kâğıt ve kalem. Bütün sorularda standart C17 ve `gcc` geçerlidir; `int` 4 bayttır.

> **İpucu: Önce kâğıtta çöz**
>
> Sınavı gerçek sınav gibi çöz: 75 dakikalık bir sayaç kur ve süre bitene kadar VS Code'u açma, çözümlere bakma. Kabaca **20 dakikayı** testlere, **20 dakikayı** çıktı sorularına, **35 dakikayı** kod yazmaya ayır. Bir soruda takılırsan işaretle ve geç; sona dönersin. Süre bitince kodlarını bilgisayarda derleyip çalıştır, çözümlerle karşılaştır ve kendini puanla.

> **Not: Kendini nasıl puanlarsın?**
>
> - **Çıktı sorularında** çıktının her satırı eşit puan değerindedir. Eksik boşluk, fazladan satır veya `5` yerine `5.0` yazmak o satırı yanlış yapar; gerçek sınavda da böyledir.
> - **Kod sorularında** çözümdeki program tek doğru cevap değildir. Senin kodun aynı işi yapıyorsa tam puan al. Çözümlerin sonunda "hangi adım kaç puan" notu var. Derleme hatası olan her satır için 1 puan, yanlış sonuç veren her mantık hatası için 2–3 puan kır.
> - Her cevabın sonunda konunun anlatıldığı bölüm yazıyor. Yanlış yaptığın soruların bölümlerindeki **Sınavda Çıkar** ve **Sık Yapılan Hata** kutularını tekrar oku.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki programda `main` fonksiyonunun adı yanlışlıkla `mian` yazılmıştır. Program `gcc program.c -o program` komutuyla derlenirse ne olur?

```c
#include <stdio.h>

int mian(void) {
    printf("Merhaba\n");
    return 0;
}
```

(A) Önişlemci `mian` adını tanımadığı için daha ilk aşamada hata verir.

(B) Derleyici `mian` için sözdizimi hatası verir; hiçbir aşama tamamlanmaz.

(C) Derleme başarıyla biter; bağlama (link) aşamasında `main` bulunamadığı için hata verilir ve çalıştırılabilir dosya oluşmaz.

(D) Program derlenir ve `Merhaba` yazdırır; fonksiyonun adı serbesttir.

(E) Program derlenir, çalıştırılınca `Merhaba` yazar ve sonra çöker.

2 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = -2;
    printf("%d %d %d %.1f\n", a / b, a % b, b % a, (double) a / b);
    return 0;
}
```

(A) `-4 1 -2 -3.5` (B) `-3 1 -2 -3.5` (C) `-3 -1 -2 -3.5` (D) `-3 1 5 -3.5` (E) `-3 1 -2 -3.0`

3 Aşağıdaki program çalışırken kullanıcı klavyeden `12 abc` yazıp Enter'a basıyor. Program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int a = 0, b = 0;
    int k = scanf("%d %d", &a, &b);
    printf("%d %d %d\n", k, a, b);
    return 0;
}
```

(A) `2 12 0` (B) `0 0 0` (C) `-1 12 0` (D) `1 0 0` (E) `1 12 0`

4 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int a = 2, b = 0, c = 5;
    int r1 = (a-- > 1) || (b++ > 0);
    int r2 = (b++ > 0) && (c++ > 0);
    printf("%d %d %d %d %d\n", r1, r2, a, b, c);
    return 0;
}
```

(A) `1 0 1 1 5` (B) `1 1 1 2 6` (C) `1 0 1 2 5` (D) `1 0 1 1 6` (E) `1 0 2 1 5`

5 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int x = 3, y = 5;
    if (x > 5)
        if (y > 10)
            y = 1;
    else
        y = 2;
    printf("%d\n", y);
    return 0;
}
```

(A) `1`
(B) `2`
(C) `3`
(D) `5`
(E) Derleme hatası verir.

6 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int i = 3;
    while (i--) {
        printf("%d ", i);
    }
    printf("| %d\n", i);
    return 0;
}
```

(A) `3 2 1 | 0` (B) `2 1 0 | -1` (C) `2 1 0 | 0` (D) `3 2 1 | -1` (E) `2 1 | 0`

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int kare(int n) {
    return n * n;
}

int topla(int a, int b) {
    a += kare(b);
    return a;
}

int main(void) {
    int x = 2, y = 3;
    int z = topla(topla(x, 2), y);
    printf("%d %d %d\n", x, y, z);
    return 0;
}
```

(A) `2 3 15` (B) `6 3 15` (C) `2 3 11` (D) `2 3 13` (E) `15 3 15`

8 Aşağıdaki kod parçalarından hangisi gcc ile **derlenmez**? (Hepsi bir `main` fonksiyonunun içindedir.)

(A) `int a[] = {4, 5, 6};`

(B) `int a[5] = {0};`

(C) `int a[3] = {1, 2, 3}; int b[3]; b = a;`

(D) `int m[2][3] = { {1, 2, 3}, {4, 5, 6} };`

(E) `int k[4] = {7, 8};`

9 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[] = "Ali Veli";
    s[3] = '\0';
    printf("%zu %zu [%s]\n", strlen(s), sizeof(s), s);
    return 0;
}
```

(A) `3 3 [Ali]` (B) `8 9 [Ali Veli]` (C) `3 8 [Ali]` (D) `3 9 [Ali Veli]` (E) `3 9 [Ali]`

10 `char s[] = "kalem";` tanımından sonra hangi döngü ekrana tam olarak `melak` yazdırır? (`<string.h>` eklenmiştir.)

A) `for (int i = 0; s[i] != '\0'; i++) printf("%c", s[i]);`

B) `for (int i = strlen(s); i > 0; i--) printf("%c", s[i]);`

C) `for (int i = (int)strlen(s) - 1; i > 0; i--) printf("%c", s[i]);`

D) `for (int i = (int)strlen(s) - 1; i >= 0; i--) printf("%c", s[i]);`

E) `for (size_t i = strlen(s) - 1; i >= 0; i--) printf("%c", s[i]);`

B) Çıktı Tahmini

11 Aşağıdaki program ekrana ne yazdırır? Boşlukları dikkatle say.

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = 2;
    a += b++;
    b *= a - 4;
    printf("%4d|%-4d|\n", a, b);
    double d = a / 2;
    printf("%.1f %.1f\n", d, a / 2.0);
    char ch = 'A' + a % 5;
    printf("%c %d\n", ch, ch - 'A');
    printf("%d %d\n", a / b * 2, a % b);
    return 0;
}
```

12 Aşağıdaki program ekrana ne yazdırır? Satır sayısına dikkat et.

```c
#include <stdio.h>

int main(void) {
    int t = 0;
    for (int k = 1; k <= 6; k++) {
        if (k == 5) {
            break;
        }
        switch (k % 3) {
            case 0:
                continue;
            case 1:
                t += 1;
            default:
                t += 10;
        }
        printf("k=%d t=%d\n", k, t);
    }
    printf("son: %d\n", t);
    return 0;
}
```

13 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int sayac(void) {
    static int n = 0;
    return n += 2;
}

void degistir(int x, int d[]) {
    x = 100;
    d[0] = x + 1;
}

int main(void) {
    int a = 5, dizi[3] = {1, 2, 3};
    sayac();
    sayac();
    printf("%d\n", sayac());
    degistir(a, dizi);
    printf("%d %d\n", a, dizi[0] + sayac());
    return 0;
}
```

14 Aşağıdaki program ekrana ne yazdırır? (`char` işaretli, `size_t` işaretsizdir; `'a'` 97, `'A'` 65'tir.)

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[] = "bilgisayar";
    int say = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] == 'a' || s[i] == 'i') {
            s[i] -= 32;
            say++;
        }
    }
    printf("%s %d\n", s, say);
    s[4] = '\0';
    printf("%s %zu %zu\n", s, strlen(s), sizeof(s));
    printf("%d\n", strcmp("elma", "Elma") > 0);
    printf("%d\n", strlen("ab") - strlen("abc") > 0);
    return 0;
}
```

C) Kod Yazma

15 Kullanıcıdan 2 veya daha büyük bir tamsayı okuyan `asalCarpan` programını yaz. Program sayıyı **asal çarpanlarına** ayırıp küçükten büyüğe bir çarpım olarak yazdırsın ve kaç **farklı** asal çarpanı olduğunu söylesin. Sayı 2'den küçükse sadece `Geçersiz sayı!` yazıp bitsin. Örnek çalışmalar:

```
Sayı: 360
360 = 2 x 2 x 2 x 3 x 3 x 5
Farklı asal çarpan: 3
```

```
Sayı: 97
97 = 97
Farklı asal çarpan: 1
```

16 Bir `int` dizisi üzerinde çalışan üç fonksiyon yaz ve `main` içinde `{7, 3, 9, 1, 9, 4}` dizisiyle dene:

- `void diziYaz(const int d[], int n)`: elemanları aralarında birer boşlukla, sonunda satır sonuyla yazdırır.
- `void tersCevir(int d[], int n)`: diziyi **yeni bir dizi kullanmadan** (yerinde) ters çevirir.
- `int ikinciBuyuk(const int d[], int n)`: dizideki ikinci en büyük **farklı** değeri döndürür (`{9, 9, 7}` için `7`). Diziyi sıralama; tek `for` kullan. Dizide en az iki farklı değer olduğunu varsayabilirsin.

Beklenen çıktı (ters çevirmeden sonra):

```
Önce : 7 3 9 1 9 4
Sonra: 4 9 1 9 3 7
İkinci en büyük: 7
```

17 Kullanıcıdan `fgets` ile en çok 99 karakterlik bir cümle okuyan `cumleAnaliz` programını yaz. Program şunları yazdırsın:

- **kelime sayısı** (kelimeler arasında, başta ya da sonda birden fazla boşluk olabilir),
- **sesli harf sayısı** (`a e i o u`, büyük ve küçük; cümlede yalnız İngilizce harfler olduğunu varsay),
- boşluklar ve büyük/küçük harf farkı yok sayıldığında cümlenin **palindrom** (tersten okunuşu aynı) olup olmadığı.

Örnek çalışma:

```
Cümle: Ey Edip Adanada pide ye
Kelime sayısı: 5
Sesli harf sayısı: 10
Palindrom: evet
```

Çözümler

Cevap anahtarı

1 C	2 B	3 E	4 A	5 D	6 B	7 A	8 C
9 E	10 D						

1 Cevap: C

`mian` geçerli bir fonksiyon adıdır, bu yüzden **derleme** aşaması (`.c` → `.o`) sorunsuz biter; `gcc -c program.c` komutu hata vermez. Ama programın giriş noktası olan `main`'i **bağlayıcı** (linker) arar, bulamaz ve çalıştırılabilir dosya üretemez. Linux'ta mesaj şuna benzer (MinGW'de genellikle `WinMain` adı geçer, macOS'ta `Undefined symbols ... "_main"` yazar; aşama her yerde aynıdır):

```
/usr/bin/ld: in function `_start':
(.text+0x1b): undefined reference to `main'
collect2: error: ld returned 1 exit status
```

- A) Önişlemci yalnızca `#include` gibi `#` satırlarıyla ilgilenir; fonksiyon adlarına bakmaz.
- B) Sözdizimi doğrudur; hata derleme değil, bağlama aşamasındadır.
- D) `main` adı zorunludur; program başlangıcı onunla bulunur.
- E) Çalıştırılabilir dosya hiç oluşmadığı için çalıştırılacak bir şey yoktur.

Konu: Bölüm 1.

2 Cevap: B

C99'dan beri tamsayı bölmesi sıfıra doğru **kırpar** ve `%` işleminin işareti **bölünenin** işaretidir (`a == (a / b) * b + a % b` her zaman geçerlidir):

- `7 / -2` → `-3.5` kırpılır → **-3** (A şıkkı aşağı yuvarlıyor, bu C'de olmaz).
- `7 % -2` → `7 - (-3 * -2)` = **1** (C şıkkı işareti bölenden alıyor).
- `-2 % 7` → `-2 - (0 * 7)` = **-2**. D şıkkı Python'daki gibi "hep pozitif" kalan varsayıyor; C'de öyle değil.
- `(double) a / b` önce `a`'yı `double` yapar, bölme ondalıklı olur: `-3.5`. `.1f` tek basamak yazdırır; E'deki `-3.0` tamsayı bölmesinin sonucunu `double`'a çevirmek olurdu.

Çıktı

```
-3 1 -2 -3.5
```

Konu: Bölüm 2.

3 Cevap: E

`scanf` geri dönüş değeri **başarıyla okunan değer sayısıdır**. `%d` önce `12`'yi okur (`a = 12`), sonra boşluğu atlar ve ikinci `%d` için `abc`'ye bakar; `a` harfi bir rakam olmadığı için okuma burada durur. Dönüş değeri 1, `b` dokunulmadığı için 0 kalır; `abc` tamponda bekler.

- A) İkinci değerin de okunduğunu varsayıyor.
- B) Başarılı okumayı hiç saymıyor; `a` da 12 olmuş.
- C) `-1` (`EOF`), girdi hiçbir değer okunamadan bittiğinde döner; burada girdi bitmedi.

- D) `scanf` "ya hep ya hiç" çalışmaz; başarıyla okunan değer korunur.

```
Çıktı
12 abc
1 12 0
```

Konu: Bölüm 3.

4 Cevap: A

`||` ve `&&` soldan sağa değerlendirilir ve sonuç belli olunca **kısa devre** yapılır.

1. `r1`: `a-- > 1` → `2 > 1` doğru, sonra `a` = 1. `||`'nin sol tarafı doğru olduğu için sağ taraf **hiç çalışmaz**; `b++` yapılmaz, `b` 0 kalır. `r1 = 1`.
2. `r2`: `b++ > 0` → `0 > 0` yanlış, sonra `b` = 1. `&&`'nin sol tarafı yanlış olduğu için `c++` **çalışmaz**, `c` 5 kalır. `r2 = 0`.

Adım	a	b	c
Başlangıç	2	0	5
`r1` sonrası	1	0	5
`r2` sonrası	1	1	5

```
Çıktı
1 0 1 1 5
```

B hiç kısa devre olmadığını, C ilk satırda `b++`'ın çalıştığını, D `c++`'ın çalıştığını, E ise `a--`'nın çalışmadığını varsayıyor.

Konu: Bölüm 4.

5 Cevap: D

Sarkan else: `else`, girintiye değil, kendisinden önceki ve henüz `else` almamış **en yakın** `if`'e bağlanır, yani `if (y > 10)`'a. Kod gerçekte şöyle okunur:

```
if (x > 5) {
    if (y > 10) y = 1;
    else        y = 2;
}
```

`x > 5` yanlış olduğu için iç `if` hiç değerlendirilmez; `y` 5 kalır. B şıkkı `else`'in dış `if`'e bağlandığını sanır (girintiye aldanır). A, iç `if`'in çalıştığını, C `x`'in değerini yazdığını varsayar. E yanlıştır: gcc bunu hata değil uyarı sayar.

```
Derleyici uyarısı – gcc -Wall -Wextra
test5.c: In function 'main':
test5.c:5:8: warning: suggest explicit braces to avoid ambiguous 'else' [-Wdangling-else]
    5 |     if (x > 5)
      |        ^
```

```
Çıktı
5
```

Konu: Bölüm 4.

6 Cevap: B

`i--` ifadesi koşulda `i`'nin **eski** değerini verir, azaltma her kontrolde yapılır:

Kontrol	Eski `i`	Doğru mu?	`i` (sonra)	Yazılan
1.	3	evet	2	`2`
2.	2	evet	1	`1`
3.	1	evet	0	`0`
4.	0	**hayır**	**-1**	–

Koşul yanlış çıksa bile `i--` çalışmıştır; bu yüzden döngüden sonra `i` 0 değil **-1**'dir. A ve D gövdede `i`'nin eski değerini sanar, C son azaltmayı unutur, E ise son turu atlar.

```
Çıktı
2 1 0 | -1
```

Konu: Bölüm 5.

7 Cevap: A

Fonksiyonlar parametreleri **değer ile** alır; `topla` içindeki `a`, `x`'in kopyasıdır. Dıştaki çağrıdan başlayıp içten dışa çöz:

1. `topla(x, 2)`: `a` = 2, `kare(2)` = 4, `a` = 6 olur ve **6 döndürülür**. `x` hâlâ 2.
2. `topla(6, 3)`: `a` = 6, `kare(3)` = 9, `a` = 15, **15 döndürülür**.

`z = 15`; `x` ve `y` değişmez. B ve E, `topla`'nın `x`'i değiştirdiğini sanıyor. C iç çağrıyı hesaba katmadan `x + kare(y)` = 2 + 9 = 11 buluyor, D iç çağrıda `kare`'yi unutup 2 + 2 = 4 buluyor (4 + 9 = 13).

```
Çıktı
2 3 15
```

Konu: Bölüm 6.

8 Cevap: C

C'de bir dizinin adı **atanabilir bir değer değildir**; `b = a;` ile dizi kopyalanamaz. gcc şu hatayı verir (diziyi kopyalamak için döngü ya da Bölüm 7'deki `memcpy` kullanılır):

```
Derleyici çıktısı – program derlenmez
test8C.c: In function 'main':
test8C.c:4:7: error: assignment to expression with array type
    4 |     b = a;
      |       ^
```

Diğer şıklar derlenir: A'da boyut ilk değer listesinden çıkar (3 eleman). B'de ilk eleman 0, kalanlar da 0'dır. D, 2 satırlı 3 sütunlu geçerli bir matristir. E'de `k[2]` ve `k[3]` otomatik 0 olur. A–E şıklarındaki geçerli tanımların bir programda çalışması:

Çıktı
```
3 0 6 0
```

Konu: Bölüm 7.

9 Cevap: E

`char s[] = "Ali Veli";` dizisi 8 karakter + `'\0'` = **9 bayt** tutar; `sizeof(s)` bunu verir ve `s[3] = '\0'` ile değişmez. `strlen` ise ilk `'\0'`'a kadar sayar: `A`, `l`, `i` → **3**. `%s` de aynı yerde durur ve yalnız `Ali` yazar. Boşluk, 3. indeksteydi ve artık bitiş karakteri oldu.

- A) `sizeof`'u uzunluk sanıyor.
- B) `s[3] = '\0'`'ın hiçbir etkisi olmadığını varsayıyor.
- C) `sizeof`'ta `'\0'`'ı saymayı unutuyor.
- D) `strlen` doğru, ama `%s` bütün diziyi yazdırır sanılmış; `%s` ilk `'\0'`'da durur.

Çıktı
```
3 9 [Ali]
```

Konu: Bölüm 8.

10 Cevap: D

`"kalem"` için `strlen(s)` = 5, indeksler 0–4'tür; son harf `s[4]`'tür. Tersten yazmak için `i`, `strlen(s) - 1`'den başlamalı ve **0 dahil** inmelidir. Çıkarmadan önce `(int)` dönüşümü Bölüm 8'deki kuraldır: boş string'de işaretsiz `0 - 1` devasa bir sayı olurdu.

- A) Düz sırada yazar: `kalem`.
- B) `s[5]` bitiş karakteri `'\0'`'dır; önce o yazılır, sonra `mela`. Hem bozuk karakter çıkar hem `k` eksik kalır.
- C) `i > 0` olduğu için `s[0]` hiç yazılmaz: `mela`. Klasik off-by-one hatası.
- E) `size_t` **işaretsizdir**; `i >= 0` her zaman doğrudur. `i` 0'dan sonra bir daha azalınca çok büyük bir sayıya sarar ve dizi dışından okur. gcc bunu yakalar:

Derleyici uyarısı – gcc -Wall -Wextra
```
test10E.c: In function 'main':
test10E.c:6:38: warning: comparison of unsigned expression in '>= 0' is always true [-Wtype-limits]
    6 |     for (size_t i = strlen(s) - 1; i >= 0; i--) printf("%c", s[i]);
      |                                      ^~
```

D şıkkının çıktısı:

Çıktı
```
melak
```

Konu: Bölüm 5 ve 8.

11

Önce `a` ve `b`'yi izle:

Satır	a	b
`a = 7, b = 2`	7	2
`a += b++` (önce `a = 7 + 2`, sonra `b++`)	9	3
`b *= a - 4` (`b = 3 * 5`)	9	15

- `%4d|%-4d|`: `9` dört genişlikte sağa yaslanır (üç boşluk + `9`), `15` dört genişlikte sola yaslanır (`15` + iki boşluk).
- `double d = a / 2;` işleminde bölme **tamsayıdır**: `9 / 2 = 4`, sonra `double`'a çevrilir. `d` = `4.0`. `a / 2.0` ise `4.5` verir.
- `'A' + a % 5` = `65 + 4` = 69 → `'E'`. `ch - 'A'` = 4.
- `a / b * 2`: `9 / 15` tamsayı bölmesi → 0; `0 * 2` = 0. `a % b` = 9.

Çıktı

```
   9|15  |
4.0 4.5
E 4
0 9
```

12

İki tuzak var. `case 0: continue;` — `switch` içindeki `continue` döngünün **sonraki turuna** geçer (`break` olsaydı yalnız `switch`'ten çıkardı). `case 1`'de `break` yoktur: `t += 1` yapıldıktan sonra `default`'a **düşülür** ve `t += 10` de çalışır. gcc de bu yüzden uyarır:

Derleyici uyarısı – gcc -Wall -Wextra

```
soru12.c: In function 'main':
soru12.c:13:19: warning: this statement may fall through [-Wimplicit-fallthrough=]
   13 |                 t += 1;
      |                 ~~^~~~
soru12.c:14:13: note: here
   14 |             default:
      |             ^~~~~~~
```

k	k % 3	İşlem	t	Yazılır mı?
1	1	`+1`, düşüp `+10`	11	`k=1 t=11`
2	2	`default`: `+10`	21	`k=2 t=21`
3	0	`continue`	21	yazılmaz
4	1	`+1`, düşüp `+10`	32	`k=4 t=32`
5	–	`break`: döngü biter	32	–

Son satır `son: 32`.

Çıktı

```
k=1 t=11
k=2 t=21
k=4 t=32
son: 32
```

13

`static int n` yalnız **bir kez** 0'lanır ve çağrılar arasında değerini korur; her çağrıda 2 artar. İlk iki çağrının sonucu atılır ama `n` 4 olur. Üçüncü çağrı 6 döndürür ve yazdırılır.

`degistir(a, dizi)`: `x`, `a`'nın **kopyasıdır**; `x = 100` olur ama `a` 5 kalır. `d` ise gerçek diziyi gösterir: `d[0] = 100 + 1 = 101` ana programdaki `dizi[0]`'ı değiştirir.

Son satırda `dizi[0] + sayac()`: `sayac()` dördüncü çağrıdır ve 8 döndürür; `101 + 8` = 109. `a` hâlâ 5.

Çıktı

```
6
5 109
```

14

1. satır: Döngü `a` ve `i` harflerini bulur, `s[i] -= 32` ile büyük harfe çevirir (`'a'` 97 - 32 = 65 = `'A'`). `bilgisayar` içinde `i` (1. ve 4. indeks) ve `a` (6. ve 8. indeks) olmak üzere 4 harf değişir: `bIlgIsAyAr 4`.

2. satır: `s[4] = '\0'` dizinin 4. indeksine bitiş karakteri koyar; `%s` ve `strlen` için dizi burada biter ve `bIlg` olur (uzunluk 4). `sizeof(s)` ise dizinin tanımlanan boyutudur: `"bilgisayar"` 10 karakter + `'\0'` = 11.

3. satır: `strcmp` küçük ve büyük harfi ayırır. `'e'` (101) `'E'`'den (69) büyüktür, sonuç pozitiftir; `> 0` doğru → `1`.

4. satır: `strlen` `size_t` (işaretsiz) döndürür. `2 - 3` işaretsiz sayıda **-1 olmaz**, çok büyük bir pozitif sayıya sarar; `> 0` doğru → `1`. Çıkarma sonucu negatif beklemek klasik tuzaktır.

Çıktı

```
bIlgIsAyAr 4
bIlg 4 11
1
1
```

15

Yaklaşım: `c = 2`'den başla. `c`, `kalan`'ı **tam bölüyorsa** asal bir çarpandır (küçük çarpanlar önce temizlendiği için `c` hiç bileşik olamaz). `while` ile `c` kaç kez bölüyorsa o kadar yazdır. Döngü `c <= kalan / c` iken sürer (`c * c <= kalan` ile aynıdır ama taşma riski yoktur). Döngü bitince `kalan > 1` ise `kalan`'ın kendisi son (ve en büyük) asal çarpandır; `97` gibi asal sayılarda ve `360`'taki `5`'te bu dal çalışır.

asalCarpan.c

```c
#include <stdio.h>

int main(void) {
    int n = 0;
    printf("Sayı: ");
    scanf("%d", &n);
    if (n < 2) {
        printf("Geçersiz sayı!\n");
        return 0;
    }
    printf("%d =", n);
    int kalan = n;
    int ilk = 1;
    int farkli = 0;
    for (int c = 2; c <= kalan / c; c++) {
        if (kalan % c == 0) {
            farkli++;
```

```
            while (kalan % c == 0) {
                printf("%s%d", ilk ? " " : " x ", c);
                ilk = 0;
                kalan /= c;
            }
        }
    }
    if (kalan > 1) {
        farkli++;
        printf("%s%d", ilk ? " " : " x ", kalan);
    }
    printf("\nFarklı asal çarpan: %d\n", farkli);
    return 0;
}
```

Çıktı

```
Sayı: 360
360 = 2 x 2 x 2 x 3 x 3 x 5
Farklı asal çarpan: 3
```

`360` için izleme: `c = 2` → 360, 180, 90 bölünür (üç kez `2`), `kalan` = 45. `c = 3` → iki kez `3`, `kalan` = 5. `c = 4` için `4 <= 5 / 4` yanlış, döngü biter. `kalan` = 5 > 1 olduğu için `x 5` yazılır. Üç farklı çarpan: 2, 3, 5.

Puanlama (12): okuma ve `n < 2` denetimi 2; dış döngü ve sınırı 3; `while` ile aynı çarpanı tekrar tekrar bölme 3; döngü sonrası `kalan > 1` durumu 2; `x` ayracının biçimi 1; farklı çarpan sayacı 1. `c * c <= kalan` yazan da tam puan alır.

16

Yaklaşım: **`tersCevir`**: baştaki ve sondaki elemanı takas et, ortaya doğru ilerle. Döngü `n / 2` kez döner; `n`'e kadar giderse dizi iki kez ters çevrilip eski haline döner. **`ikinciBuyuk`**: iki değişken tut. Yeni bir en büyük gelirse eski en büyük ikinciye kayar. Eşit değerlerin (`9`, `9`) ikinci sayılmaması için `d[i] < birinci` şartı gerekir. Başlangıçta `ikinci`'ye `INT_MIN` ver (`<limits.h>`, Bölüm 2). Fonksiyonlar diziyi **adres ile** aldığı için `tersCevir` çağıranın dizisini değiştirir; `ikinciBuyuk` ve `diziYaz` değiştirmediği için `const` yazdık.

dizilslemleri.c

```c
#include <stdio.h>
#include <limits.h>

void diziYaz(const int d[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d%s", d[i], i < n - 1 ? " " : "\n");
    }
}

void tersCevir(int d[], int n) {
    for (int i = 0; i < n / 2; i++) {
        int gecici = d[i];
        d[i] = d[n - 1 - i];
        d[n - 1 - i] = gecici;
    }
}

int ikinciBuyuk(const int d[], int n) {
    int birinci = d[0];
    int ikinci = INT_MIN;
    for (int i = 1; i < n; i++) {
        if (d[i] > birinci) {
            ikinci = birinci;
            birinci = d[i];
```

```
        } else if (d[i] < birinci && d[i] > ikinci) {
            ikinci = d[i];
        }
    }
    return ikinci;
}

int main(void) {
    int dizi[] = {7, 3, 9, 1, 9, 4};
    int n = sizeof(dizi) / sizeof(dizi[0]);
    printf("Önce : ");
    diziYaz(dizi, n);
    tersCevir(dizi, n);
    printf("Sonra: ");
    diziYaz(dizi, n);
    printf("İkinci en büyük: %d\n", ikinciBuyuk(dizi, n));
    return 0;
}
```

Çıktı

```
Önce : 7 3 9 1 9 4
Sonra: 4 9 1 9 3 7
İkinci en büyük: 7
```

`n`, dizinin tanımlandığı yer olan `main`'de hesaplanır: fonksiyonun içinde `sizeof(d) / sizeof(d[0])` yazsaydın yanlış sonuç alırdın, çünkü `d` orada bir diziyi değil adresi temsil eder (Bölüm 7).

Puanlama (12): `diziYaz` 2; `tersCevir` 4 (döngü sınırı `n / 2`: 1, takas: 2, indeks `n - 1 - i`: 1); `ikinciBuyuk` 5 (başlangıç değerleri 1, yeni birincide kaydırma 2, eşit değerleri eleme 2); `main` ve çağrılar 1. Sıralayarak çözen (tek `for` şartı çiğnendi) en çok 3 puan kaybeder.

17

Yaklaşım: Üç iş tek `for` döngüsünde yapılabilir. **Kelime sayısı:** `icinde` bayrağı tut: boşluktan boşluk olmayana geçerken (`icinde` 0 iken harf görünce) kelime sayacını artır; boşlukta bayrağı 0 yap. Birden fazla boşlukta sayaç şaşmaz. **Sesli harf:** `tolower` ile küçültüp `switch` içinde beş etiket alt alta dizilir (Bölüm 4). **Palindrom:** Cümleyi kopyalamadan, `bas` baştan ve `son` sondan ilerler. Boşluğa gelen taraf bir adım atlar, iki harf farklıysa palindrom değildir.

cumleAnaliz.c

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main(void) {
    char cumle[100];
    printf("Cümle: ");
    fgets(cumle, sizeof(cumle), stdin);
    cumle[strcspn(cumle, "\n")] = '\0';

    int kelime = 0, sesli = 0, icinde = 0;
    for (int i = 0; cumle[i] != '\0'; i++) {
        unsigned char c = (unsigned char)cumle[i];   // ctype için (Bölüm 8)
        if (isspace(c)) {
            icinde = 0;
        } else if (!icinde) {
            icinde = 1;
            kelime++;
        }
        switch (tolower(c)) {
```

```
            case 'a':
            case 'e':
            case 'i':
            case 'o':
            case 'u':
                sesli++;
                break;
        }
    }

    int bas = 0, son = (int) strlen(cumle) - 1;
    int palindrom = 1;
    while (bas < son) {
        unsigned char a = (unsigned char)cumle[bas];
        unsigned char b = (unsigned char)cumle[son];
        if (isspace(a)) {
            bas++;
        } else if (isspace(b)) {
            son--;
        } else if (tolower(a) != tolower(b)) {
            palindrom = 0;
            break;
        } else {
            bas++;
            son--;
        }
    }

    printf("Kelime sayısı: %d\n", kelime);
    printf("Sesli harf sayısı: %d\n", sesli);
    printf("Palindrom: %s\n", palindrom ? "evet" : "hayır");
    return 0;
}
```

Çıktı

```
Cümle: Ey Edip Adanada pide ye
Kelime sayısı: 5
Sesli harf sayısı: 10
Palindrom: evet
```

`fgets` satır sonunu (`'\n'`) diziye de koyar; `strcspn` ile silmezsen palindrom denetiminde `son` `'\n'`'e denk gelir ve sonuç hep "hayır" çıkar. `icinde` bayrağı yerine "boşluk sayısı + 1" yazanlar, birden fazla boşlukta ve başta/sonda boşlukta yanlış sonuç alır. Türkçe karakterler UTF-8'de iki bayt tuttuğu için bu program yalnız İngilizce harflerle doğru çalışır (Bölüm 8).

Puanlama (12): `fgets` ve `'\n'` temizliği 2; kelime sayımı 4 (bayrak mantığı 3, çoklu boşluk 1); sesli harf sayımı 2 (büyük harf dahil); palindrom 4 (iki indeks 1, boşluk atlama 1, küçük harfe çevirerek karşılaştırma 1, sonuç 1).

BÖLÜM 9

İşaretçiler (Pointers)

Bu Bölümde Öğreneceklerin

- Her değişkenin bellekte bir adresi olduğunu ve `&` ile nasıl alındığını
- İşaretçi (pointer) tanımlamayı, `*` ile gösterdiği yere erişmeyi (dolaylama) ve `int *p, q;` tuzağını
- İşaretçinin tipinin neden önemli olduğunu ve `sizeof(int *)` değerini
- `NULL` işaretçiyi, başlatılmamış işaretçinin tehlikesini ve kullanmadan önce kontrol etmeyi
- Adres ile geçirmeyi: çalışan `swap`, birden çok sonuç döndürme ve `scanf`'teki `&`'ın gerçek anlamını
- İşaretçi aritmetiğini (`p + 1`, `p++`, `p - q`) ve `*p++` ile `(*p)++` farkını
- Dizi adı ile işaretçi ilişkisini (`a[i]` ≡ `*(a + i)`) ve fonksiyona dizi geçirmenin gerçek anlamını
- `char s[]` ile `char *p` farkını, string sabitinin salt okunur olduğunu ve `const` ile işaretçi birleşimlerini
- Yerel değişkenin adresini döndürme hatasını, `int **pp`'yi, `argc`/`argv`'yi, `void *` ve fonksiyon işaretçisini (`qsort`)

Bu, kitabın en kritik bölümüdür. Üç şeyi "sonra anlatacağız" diye erteledik: `scanf`'te değişkenin önüne neden `&` konduğunu (Bölüm 3), Bölüm 6'da çalışmayan `swap`'ın nasıl düzeltileceğini ve dizilerin fonksiyona neden kopyalanmadan geçtiğini (Bölüm 7). Üçünün cevabı da aynıdır: **işaretçi**. Sınavlarda en çok puan kaybettiren sorular da bunlardır; sebep çoğu zaman konunun zorluğu değil, belleğin kafada canlandırılmamasıdır. Bu yüzden her örnekte bir bellek tablosu çizeceğiz. Sen de sınavda aynısını yap: **kâğıda kutuları çiz, okları çiz, satır satır izle.**

9.1 Bellek ve adres

Bilgisayarın belleği (RAM), sıra numaralı bayt hücrelerinden oluşan çok uzun bir sıradır. Bu numaraya **adres** (address) denir. Bölüm 2'de değişkeni "adı olan kutu" diye anlatmıştık; kutunun bir de **numarası** vardır. `int yas = 25;` yazınca derleyici 4 baytlık yer ayırır, adına `yas` der ve 25'i oraya yazar. Değişkenin adresi, kapladığı baytların ilkinin numarasıdır.

```
Bellek (adresler örnektir; gerçek adresler çok daha büyük sayılardır)

  adres       değer      ad
+--------+----------+--------+
|  1000  |    25    |  yas   |   int: 1000–1003 arası 4 bayt
+--------+----------+--------+
|  1004  |   'A'    |  harf  |   char: 1 bayt
+--------+----------+--------+
```

Adresi **adres operatörü** (address-of operator) `&` verir: `&yas`. Bölüm 3'te `scanf("%d", &yas)` yazarken kullandığın işaret budur. Adresler `printf`'te `%p` ile yazdırılır:

adresGoster.c

```c
#include <stdio.h>

int main(void) {
    int yas = 25;
    char harf = 'A';

    printf("yas  : deger = %d, adres = %p\n", yas, (void *)&yas);
    printf("harf : deger = %c, adres = %p\n", harf, (void *)&harf);
    return 0;
}
```

Çıktı

```
yas  : deger = 25, adres = 0x16baf66dc
harf : deger = A, adres = 0x16baf66db
```

`%p` için argümanı `(void *)` ile çevirmek gerekir (`void *`'ı bölümün sonunda göreceksin). Adresler onaltılık (hexadecimal) yazılır. **Senin bilgisayarında adresler farklı çıkar**, aynı programı iki kez çalıştırsan bile değişebilir: yerleştirmeyi işletim sistemi yapar. Windows'ta adres `0x` öneki olmadan, başı sıfırlarla dolu görünebilir. Sınavda adresin sayısal değeri sorulmaz; önemli olan her değişkenin bir adresi olmasıdır.

9.2 İşaretçi tanımlama ve dolaylama

İşaretçi (pointer), değeri başka bir değişkenin **adresi** olan değişkendir. Tanımlarken tipin yanına `*` konur. Üç adımda kullanılır:

Adım	Kod	Anlamı
1. Tanımla	`int *p;`	`p`, bir `int`'in adresini tutabilir
2. Adres ver	`p = &x;`	`p`'nin içine `x`'in adresi yazılır
3. Kullan	`*p`	`p`'nin gösterdiği yerdeki değer

`*p` işlemine **dolaylama** (indirection, dereference) denir: "p'nin içindeki adrese git, orada duran değişkenin kendisini kullan". `*p` hem okunabilir hem yazılabilir; `x` ile aynı kutudur.

ilkIsaretci.c

```c
#include <stdio.h>

int main(void) {
    int x = 10;
    int *p = &x;                    // p, x'in adresini tutar

    printf("x  = %d\n", x);
    printf("*p = %d\n", *p);        // p'nin gösterdiği yerdeki değer
    printf("p == &x : %d\n", p == &x);
    *p = 20;                        // x'in kendisi değişir
    printf("x  = %d\n", x);
    *p = *p + 5;
    printf("x  = %d\n", x);
    return 0;
}
```

Çıktı

```
x  = 10
*p = 10
p == &x : 1
x  = 20
x  = 25
```

Bellek tablosu (adresler örnek): `p` de bir değişkendir, kendi kutusu vardır ve içinde `x`'in adresi yazılıdır.

```
int x = 10;        int *p = &x;

   x  (adres 1000)              p  (adres 1008)
+------------------+         +------------------+
|        10        |<--------|       1000       |
+------------------+         +------------------+
                             p'nin içindeki sayı = x'in adresi
```

Satır satır izle: `int *p = &x;` sonrası `p` = `&x`; `*p = 20;` ile `x` 20 olur, `p` değişmez; `*p = *p + 5;` ile sağdaki `*p` okur (20), soldaki `*p` yazar, `x` 25 olur.

`*` işareti C'de üç iş yapar; hangisi olduğunu **bulunduğu yer** belirler:

Nerede?	Anlamı	Örnek
Tanımda, tipin yanında	"bu bir işaretçi"	`int *p;`
İfadede, tek başına ve önde	dolaylama: "gösterilen yer"	`*p = 3;` , `y = *p;`
İki değerin arasında	çarpma	`a * b`

> **Sık Yapılan Hata: `p` ile `*p`'yi karıştırmak**
>
> `p` işaretçinin kendisidir (adres), `*p` gösterdiği değerdir. `x`'i değiştirmek için `*p = 20;` yazılır. `p = 20;` ise işaretçiye adres yerine **sayı** atar: gcc 14'ten beri `assignment to 'int *' from 'int' makes pointer from integer without a cast` derleme hatasıdır (eski sürümlerde uyarıydı). Aynı şekilde `double d; int *p = &d;` da tip uyuşmadığı için (`incompatible pointer type`) derlenmez.

int *p, q; tuzağı

`*` tipe değil, **o değişkene** aittir. Aynı satırda iki değişken tanımlarsan:

ikiTanim.c

```c
#include <stdio.h>

int main(void) {
    int x = 7;
    int *p, q;          // p işaretçi, q sıradan int!

    p = &x;
    q = x;
    printf("sizeof(p) = %zu, sizeof(q) = %zu\n", sizeof(p), sizeof(q));
    printf("*p = %d, q = %d\n", *p, q);
    return 0;
}
```

Çıktı
```
sizeof(p) = 8, sizeof(q) = 4
*p = 7, q = 7
```

`q` işaretçi değil, sıradan bir `int`'tir (`sizeof` farkına bak). `int* p, q;` diye yazsan da değişmez: boşluğun yeri anlamı etkilemez. İki işaretçi için `int *p, *q;` yaz; en temizi her işaretçiyi ayrı satırda tanımlamaktır.

> **Sınavda Çıkar**
>
> "`int *p, q;` tanımında `q` neyin tipindedir?" klasik bir sorudur. Cevap: `int`. `*`, yalnız hemen sağındaki ada yapışır. `int *p, *q, r;` yazılırsa `p` ve `q` işaretçi, `r` sıradan `int` olur.

9.3 İşaretçinin tipi

İşaretçinin içinde yalnızca bir adres vardır. Peki neden `int *`, `double *`, `char *` diye ayrı tipler var? Çünkü adres bir yerin başlangıcını söyler ama **orada ne olduğunu** söylemez. İşaretçinin tipi iki şeyi belirler: (1) `*p` yazdığında **kaç bayt** okunacak ve bunun nasıl yorumlanacağı; (2) `p + 1` yazdığında adresin **kaç bayt** ilerleyeceği.

Bugünkü bilgisayarlarda bütün işaretçiler **aynı boyuttadır**, çünkü hepsi bir adres tutar (C standardı bunu her sistem için garanti etmez; bazı mikrodenetleyicilerde farklı işaretçi tipleri farklı boyutta olabilir): `sizeof(char *)`, `sizeof(int *)` ve `sizeof(double *)` 64 bitlik sistemlerde (bugün Windows, macOS ve Linux'ta kullanılan bilgisayarların hepsi) **8** bayttır; az önceki `ikiTanim` programında `sizeof(p)` 8 çıkmıştı. Eski 32 bitlik sistemlerde 4'tü. Gösterdiği değerin boyutu ise tipe göre değişir: `sizeof(*p)` bir `int` olduğu için 4'tür.

9.4 NULL ve başlatılmamış işaretçi

Gösterecek bir şeyin henüz yoksa işaretçiye **hiçbir yeri göstermeyen** özel değeri verirsin: `NULL` (boş işaretçi, null pointer). `stdio.h`, `stdlib.h` ve `string.h` içinde tanımlıdır. Kullanmadan önce `if (p == NULL)` ile kontrol etmek standart güvenlik adımıdır. Kontrol etmeden `NULL` işaretçinin gösterdiği yere **erişmek** tanımsız davranıştır; bugünkü masaüstü sistemlerde program neredeyse her zaman çöker. Aşağıda bunu kasıtlı olarak gösteriyoruz:

nullCokme.c
```c
#include <stdio.h>

int main(void) {
    int *p = NULL;                  // henüz hiçbir yeri göstermiyor

    if (p == NULL) {
        printf("p henuz bir yeri gostermiyor.\n");
    }
    printf("Yazmadan once\n");
    *p = 5;                         // kontrol yok: NULL'in gosterdigi yere yazma
    printf("Yazdiktan sonra\n");
    return 0;
}
```

Çıktı – program çöker (Segmentation fault)
```
p henuz bir yeri gostermiyor.
Yazmadan once
```

`Segmentation fault` yazısını Linux ve macOS terminali yazar (Bölüm 1'de gördüğün çökme budur); Windows'ta program çoğu zaman mesajsız kapanır. İkinci `printf` hiç çalışmaz. Bu çökme aslında iyi haberdir, çünkü hata hemen ortaya çıkar. Daha kötüsü **başlatılmamış işaretçidir**:

```
int *p;           // ilk değer verilmedi: içinde rastgele bir adres var
*p = 5;           // TANIMSIZ DAVRANIŞ: bilinmeyen bir yere yazar
```

Bölüm 2'deki çöp değerin işaretçideki karşılığıdır: `p`'nin içinde o bellekte kalmış rastgele bir sayı vardır ve bu sayı adres sayılır. Program çökebilir, başka bir değişkeni sessizce bozabilir ya da hiçbir şey olmamış gibi çalışabilir; "benim bilgisayarımda çalıştı" bir kanıt değildir. Üstelik içinde `NULL` değil çöp olduğu için `p != NULL` ile de yakalayamazsın.

İşaretçinin durumu	İçinde ne var?	`*p` kullanılırsa
Geçerli bir değişkeni gösteriyor	O değişkenin adresi	Sorun yok
`NULL`	Hiçbir yeri göstermeyen özel değer	Çökme; `if (p != NULL)` ile önlenir
Başlatılmamış	Rastgele bir sayı	Tanımsız davranış; kontrol edilemez

Altın kural: **İşaretçiyi tanımlarken ya geçerli bir adres ya da `NULL` ver** (`int *p = NULL;`).

> **Not: 0 ve NULL**
>
> `p = 0;` da geçerlidir ve `p`'yi `NULL` yapar (sabit `0`, işaretçiye atanınca "boş işaretçi" olur). Ama okuyan sayı mı işaretçi mi anlayamaz; her zaman `NULL` yaz. C23'te bunun için `nullptr` geldi, ama kitapta ve sınavda `NULL` kullanılır.

9.5 Adres ile geçirme

Bölüm 6'dan hatırla: argümanlar **değer ile** geçer. Fonksiyondaki parametre çağıran taraftaki değişkenin **kopyasıdır**; o yüzden `swap(x, y)` main'deki değişkenlere dokunamıyordu. Çözüm, değişkenin kendisini değil **adresini** geçirmektir. Fonksiyon adresin kopyasını alır, ama kopya da aynı yeri gösterdiği için `*a = ...` yazınca main'deki değişken değişir.

swapDuzgun.c

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int gecici = *a;
    *a = *b;
    *b = gecici;
}

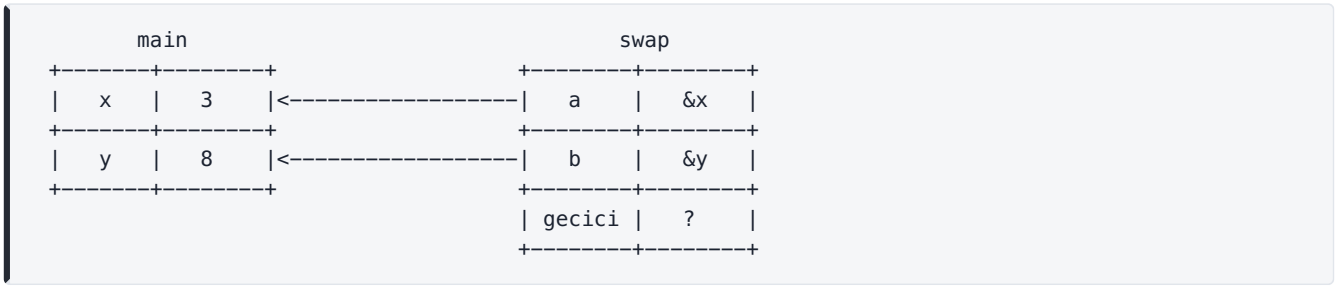
int main(void) {
    int x = 3;
    int y = 8;

    printf("Once  : x = %d, y = %d\n", x, y);
    swap(&x, &y);                   // değişkenlerin adreslerini ver
    printf("Sonra : x = %d, y = %d\n", x, y);
    return 0;
}
```

Çıktı

```
Once  : x = 3, y = 8
Sonra : x = 8, y = 3
```

`swap` çalışırken bellek aşağıdaki gibidir. `a` ve `b`, `swap`'ın yerel değişkenleridir (içlerinde adres var); `gecici` sıradan bir `int`'tir:



Adım adım: çağrıda `gecici` çöptür; `gecici = *a;` ile 3 olur; `*a = *b;` ile `x` 8 olur; `*b = gecici;` ile `y` 3 olur.

Dönüşte `a`, `b` ve `gecici` yok olur (Bölüm 6'daki çağrı yığını), ama `x` ile `y` main'e aittir ve değişmiş hâlde kalır. Kritik cümle: **C'de "adres ile geçirme" diye ayrı bir mekanizma yoktur.** Her zaman değer ile geçirilir; yalnızca geçirilen değer **bir adrestir**.

> **Sık Yapılan Hata: Çağırırken `&` unutmak**
>
> `void swap(int *a, int *b)` fonksiyonunu `swap(x, y);` diye çağırırsan `int` değerini işaretçi parametreye vermiş olursun. gcc 14'ten beri bu bir derleme hatasıdır (`passing argument 1 of 'swap' makes pointer from integer without a cast`); eski sürümlerde uyarıydı ve program `3` sayısını adres sanıp çökerdi. Parametre `T *` ise çağırırken `&değişken` ver.

Birden çok sonuç döndürme

`return` tek değer döndürür. Birden çok sonuç gerekiyorsa sonuçların yazılacağı yerlerin **adreslerini** parametre olarak verirsin:

bolVeKalan.c

```c
#include <stdio.h>

void bolVeKalan(int a, int b, int *bolum, int *kalan) {
    *bolum = a / b;
    *kalan = a % b;
}

int main(void) {
    int bolum = 0;
    int kalan = 0;

    bolVeKalan(17, 5, &bolum, &kalan);
    printf("17 / 5 = %d, kalan %d\n", bolum, kalan);
    return 0;
}
```

Çıktı

```
17 / 5 = 3, kalan 2
```

scanf'teki & işareti

Artık `scanf("%d", &yas)` yazarken `&`'ın nedeni açık: `scanf` de bir fonksiyondur ve `yas` değişkenine bir değer **yazması** gerekir. Bunun için `yas`'ın değerini değil **adresini** almalıdır; tıpkı `swap`'taki `*a = ...` gibi. `scanf`'in ikinci parametresi aslında bir `int *`'tır. Bu yüzden elinde zaten bir işaretçi varsa `&` **koyma**: `int *p = &sayi;` `scanf("%d", p);` ile `scanf("%d", &sayi);` aynı işi yapar.

> **Sık Yapılan Hata: scanf'e işaretçinin adresini vermek**
>
> `p` bir işaretçiyken `scanf("%d", &p);` yazarsan `p`'nin **kendi adresini** (`int **`) vermiş olursun; sayı `p`'nin kutusuna yazılır ve içindeki adres bozulur. gcc `-Wall` ile `format '%d' expects argument of type 'int *', but argument 2 has type 'int **'` uyarısı verir; ciddiye al. Bölüm 8'de `scanf("%s", s)` yazarken `&` koymamanın nedeni de aynı: dizi adı zaten adrestir (birazdan göreceksin).

9.6 İşaretçi aritmetiği

İşaretçilerle toplama ve çıkarma yapabilirsin, ama kuralı bilmelisin: `p + 1` adresi **1 bayt değil, 1 eleman** ilerletir. Eleman boyutu işaretçinin tipinden gelir (`char *` için 1, `int *` için 4, `double *` için 8 bayt; yani `sizeof(*p)`). İşaretçinin tipinin önemi tam burada ortaya çıkar.

```
  a[0]   a[1]   a[2]   a[3]   a[4]
+------+------+------+------+------+
|  10  |  20  |  30  |  40  |  50  |
+------+------+------+------+------+
 1000   1004   1008   1012   1016        (örnek adresler)
  ^
  p          p + 1 = 1004 (1001 değil!),   p + 3 = 1012
```

İşlem	Anlamı	Sonuç
`p + n`, `p - n`	`n` eleman ileri/geri	işaretçi
`p++`, `p--`	bir eleman ileri/geri (`p`'yi değiştirir)	işaretçi
`p - q`	aralarındaki **eleman sayısı** (aynı dizide)	tamsayı (`ptrdiff_t`)
`p < q`, `p == q`	adres karşılaştırması	0 ya da 1
`p + q`, `p * 2`, `p / 2`	anlamsız, **derlenmez**	

aritmetik.c

```c
#include <stdio.h>

int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = &a[0];

    printf("%d\n", *p);             // 10
    p++;                            // a[1]'e geç
    printf("%d\n", *p);             // 20
    p = p + 2;                      // a[3]'e geç
    printf("%d\n", *p);             // 40
    p--;                            // a[2]'ye dön
    printf("%d\n", *p);             // 30
    printf("%d\n", *(p + 1));       // 40, p değişmez

    int *q = &a[4];
    printf("q - p = %d\n", (int)(q - p));   // 4 - 2
    return 0;
}
```

Çıktı
```
10
20
40
30
40
q - p = 2
```

İki işaretçiyi **çıkarmak** eleman sayısını verir (`q - p` = 2), bayt farkını değil. Çıkarma ve karşılaştırma yalnız **aynı dizinin** elemanlarını gösteren işaretçiler için anlamlıdır. Dizinin **bir ötesini** gösteren işaretçi (`a + 5`) oluşturmak ve karşılaştırmak serbesttir (döngülerde kullanacaksın), ama onu `*` ile okumak ya da dizinin daha ötesine gitmek tanımsızdır.

*p++ ile (p)++ farkı

`++` sonek operatörü `*`'dan **daha yüksek önceliklidir**. Bu yüzden `*p++` "`p`'nin gösterdiğini kullan, sonra `p`'yi ilerlet" demektir; gösterilen değeri artırmaz:

İfade	Nasıl okunur	Ne yapar?
`*p++`	`*(p++)`	Önce `*p`'yi kullan, sonra `p`'yi bir eleman ilerlet
`(*p)++`		`p` sabit kalır, gösterilen **değer** 1 artar (kullanılan, artmadan önceki değerdir)
`*++p`	`*(++p)`	Önce `p`'yi ilerlet, sonra yeni yerdeki değeri kullan

artirmaOncelik.c
```c
#include <stdio.h>

int main(void) {
    int a[4] = {10, 20, 30, 40};
    int *p = a;

    int ilk = *p++;                 // ilk = 10, sonra p -> a[1]
    printf("ilk = %d, *p = %d\n", ilk, *p);
    (*p)++;                         // a[1]: 20 -> 21, p yerinde
    printf("*p = %d, a[1] = %d\n", *p, a[1]);
    int ucuncu = *++p;              // önce p -> a[2], sonra oku
    printf("ucuncu = %d\n", ucuncu);
    printf("%d %d %d %d\n", a[0], a[1], a[2], a[3]);
    return 0;
}
```

Çıktı
```
ilk = 10, *p = 20
*p = 21, a[1] = 21
ucuncu = 30
10 21 30 40
```

> **Sınavda Çıkar**
>
> `*p++` ile `(*p)++` ayrımı neredeyse her işaretçi sınavındadır. Takip yöntemi: iki şeyi ayrı ayrı yaz, "`p` nereyi gösteriyor?" ve "dizide ne var?". `*p++`'da **p** değişir, dizi değişmez. `(*p)++`'da **dizi** değişir, `p` değişmez. Parantez yoksa `++` önce `p`'ye yapışır.

9.7 Diziler ve işaretçiler

İşte Bölüm 7'deki sözün cevabı: Dizi adı, ifadelerde çoğu zaman **ilk elemanın adresine** dönüşür.

```
int a[5] = {10, 20, 30, 40, 50};

a           ≡  &a[0]       (ilk elemanın adresi)
a + i       ≡  &a[i]
*(a + i)    ≡  a[i]        (indeks yazımı, bu işlemin kısa yoludur)
```

`a[i]` yazımı derleyici için tam olarak `*(a + i)` demektir. İşaretçilere de indeks uygulanabilir: `p[i]` ile `*(p + i)` aynıdır.

dizilsaretci.c

```c
#include <stdio.h>

int main(void) {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;                     // &a[0] ile aynı

    printf("a[2]     = %d\n", a[2]);
    printf("*(a + 2) = %d\n", *(a + 2));
    printf("p[2]     = %d\n", p[2]);
    printf("*(p + 2) = %d\n", *(p + 2));
    return 0;
}
```

Çıktı

```
a[2]     = 30
*(a + 2) = 30
p[2]     = 30
*(p + 2) = 30
```

Diziyi dolaşmanın yollarından biri **gezen işaretçidir** (kalıp):

```c
for (int *p = a; p < a + n; p++) {      // a: dizi, n: eleman sayısı
    toplam += *p;                       // a[i] yerine *p
}
```

Burada `a + n` dizinin **bir ötesini** gösterir; `p < a + n` bitiş koşuludur ve `*p` o adrese hiç erişmez. Bu kalıbı C kodlarında sık göreceksin.

Dizi adı ile işaretçi aynı şey değildir

Aralarındaki ilişki "dönüşüm"dür, eşitlik değil. Dizi adı bir **işaretçi değişkeni** olmadığı için: `sizeof(a)` bütün dizinin boyutunu verir (`sizeof(p)` işaretçinin boyutunu, 8) ve `a`'ya **atama yapılamaz**, `p`'ye yapılır: dizinin yeri sabittir.

```c
int main(void) {
    int a[3] = {1, 2, 3};
    int b[3] = {4, 5, 6};
    int *p = b;                     // olur: p değişken

    a = p;                          // hata: dizi adına atama
    a++;                            // hata: dizi adı ilerletilemez
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
diziAdiAtama.c: In function 'main':
diziAdiAtama.c:6:7: error: assignment to expression with array type
    6 |     a = p;                          // hata: dizi adına atama
      |       ^
diziAdiAtama.c:7:6: error: lvalue required as increment operand
    7 |     a++;                            // hata: dizi adı ilerletilemez
      |      ^~
diziAdiAtama.c:2:9: warning: variable 'a' set but not used [-Wunused-but-set-variable=]
    2 |     int a[3] = {1, 2, 3};
      |         ^
```

Fonksiyona dizi geçirmek

Dizi adı adrese dönüştüğü için fonksiyon **dizinin kopyasını değil, ilk elemanın adresini** alır. Parametrede `int a[]` yazmak `int *a` yazmakla aynıdır. Bölüm 7'de "dizi neden adres olarak geçiyor?" diye sormuştun: sebep budur. Fonksiyon ana dizinin kendisini kullanır:

diziGecir.c

```c
#include <stdio.h>

void ikiyeKatla(int *a, int n) {            // int a[] ile aynı
    for (int i = 0; i < n; i++) {
        a[i] = a[i] * 2;
    }
}

int main(void) {
    int sayilar[4] = {1, 2, 3, 4};

    ikiyeKatla(sayilar, 4);
    printf("%d %d %d %d\n", sayilar[0], sayilar[1], sayilar[2], sayilar[3]);
    return 0;
}
```

Çıktı

```
2 4 6 8
```

Fonksiyonda yapılan değişiklik main'deki diziyi değiştirdi, çünkü aynı belleği gösteriyordu. Fonksiyon yalnız **adresi** bildiği için dizinin kaç elemanlı olduğunu bilemez; uzunluğu (`n`) ayrıca geçirmek zorundasın. Bölüm 7'de uyardığımız `sizeof` tuzağı da buradan doğar:

boyutFonk.c

```c
#include <stdio.h>

void boyutYaz(int a[]) {
    printf("fonksiyonda: sizeof(a)    = %zu\n", sizeof(a));
}

int main(void) {
    int dizi[5] = {1, 2, 3, 4, 5};

    printf("main'de    : sizeof(dizi) = %zu\n", sizeof(dizi));
    boyutYaz(dizi);
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
boyutFonk.c: In function 'boyutYaz':
boyutFonk.c:4:55: warning: 'sizeof' on array function parameter 'a' will return size of 'int *' [-
Wsizeof-array-argument]
    4 |     printf("fonksiyonda: sizeof(a)    = %zu\n", sizeof(a));
      |                                                       ^
boyutFonk.c:3:19: note: declared here
    3 | void boyutYaz(int a[]) {
      |               ~~~~^~
```

Çıktı

```
main'de    : sizeof(dizi) = 20
fonksiyonda: sizeof(a)    = 8
```

gcc uyarıyor: fonksiyondaki `a` bir dizi değil `int *`'tır; `sizeof(a)` işaretçinin boyutunu verir (8 değeri 64 bitlik sistem içindir). Yani fonksiyon içinde `sizeof(a) / sizeof(a[0])` eleman sayısını **vermez**: 8 / 4 = 2 çıkar. Eleman sayısını her zaman dizinin tanımlandığı yerde hesapla, fonksiyona ayrı parametre olarak ver.

> **Sınavda Çıkar**
>
> "`void f(int a[])` içinde `sizeof(a)` kaçtır?" Cevap: dizinin boyutu **değil**, işaretçinin boyutu (64 bitte 8). Bir de: "fonksiyonda `a[0] = 99;` yapılırsa main'deki dizi değişir mi?" Evet; dizi kopyalanmaz, adresi geçer. Bu, `int` parametresinden farklıdır: `int` kopyalanır, dizi kopyalanmaz.

9.8 Stringler ve işaretçiler

Bölüm 8'de string'in `'\0'` ile biten bir `char` dizisi olduğunu öğrendin. Dizi adı adrese dönüştüğü için string'ler de çoğunlukla `char *` ile taşınır. Ama string tanımlamanın iki yolu aynı değildir:

diziVsIsaretci.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char dizi[] = "Ali";            // 4 baytlık dizi: 'A' 'l' 'i' '\0'
    const char *p = "Ali";          // işaretçi + salt okunur "Ali"

    dizi[0] = 'E';                  // olur: dizi senin kopyan
    printf("dizi = %s, p = %s\n", dizi, p);
    printf("sizeof(dizi) = %zu, sizeof(p) = %zu\n", sizeof(dizi), sizeof(p));
    printf("strlen(dizi) = %zu, strlen(p) = %zu\n", strlen(dizi), strlen(p));
    p = "Veli";                     // olur: p başka yeri gösterebilir
    printf("p = %s\n", p);
    return 0;
}
```

Çıktı

```
dizi = Eli, p = Ali
sizeof(dizi) = 4, sizeof(p) = 8
strlen(dizi) = 3, strlen(p) = 3
p = Veli
```

```
char dizi[] = "Ali";                const char *p = "Ali";

    dizi (4 bayt)                     p (8 bayt)      salt okunur bölge
+-----+-----+-----+------+         +--------+      +-----+-----+-----+------+
| 'A' | 'l' | 'i' | '\0' |         |  adres |----->| 'A' | 'l' | 'i' | '\0' |
+-----+-----+-----+------+         +--------+      +-----+-----+-----+------+
 senin kopyan, değiştirilebilir                     programa gömülü, değişmez
```

	`char dizi[] = "Ali";`	`const char *p = "Ali";`
Bellekte	4 baytlık dizi (metnin kopyası)	8 baytlık işaretçi + salt okunur metin
`sizeof`	4	8
Harfi değiştirme	Olur	Yapılamaz (`const` ile derleme hatası)
Başka metne yöneltme	`dizi = "Veli";` **derlenmez**	`p = "Veli";` olur

Çift tırnak içindeki `"Ali"` **string sabitidir** (string literal) ve programın salt okunur bir bölgesinde durur. `char *p = "Ali"; p[0] = 'E';` ile değiştirmeye çalışmak **tanımsız davranıştır** (Linux ve macOS'ta çoğu zaman `Segmentation fault`, ama garanti yok):

```c
char *p = "Ali";
p[0] = 'E';        // TANIMSIZ DAVRANIŞ: salt okunur belleğe yazma
```

Bu yüzden string sabitini gösteren işaretçiyi `const char *` olarak tanımla: yanlışlıkla değiştirmeye kalkarsan hata derleyicide yakalanır. Değiştireceğin metin için `char dizi[] = "..."` kullan. Fonksiyona değiştirilmeyecek bir string veriyorsan parametreyi `const char *` yap (`strlen`, `puts`, `printf` böyle tanımlanmıştır).

İşaretçiyle `strlen`'in kendi sürümünü yazalım:

ozelStrlen.c

```c
#include <stdio.h>

size_t uzunluk(const char *s) {
    const char *basla = s;
    while (*s != '\0') {
        s++;
    }
    return (size_t)(s - basla);     // iki işaretçinin farkı = harf sayısı
}

int main(void) {
    printf("%zu\n", uzunluk("Merhaba"));
    printf("%zu\n", uzunluk(""));
    return 0;
}
```

Çıktı

```
7
0
```

9.9 const ve işaretçiler

`const` işaretçide iki farklı şeyi sabitleyebilir: **gösterilen değeri** ya da **işaretçinin kendisini**. Ayrımı `const`'ın `*`'a göre yeri belirler:

Tanım	Gösterilen değer (`*p = ...`)	İşaretçi (`p = ...`)	Okunuşu
`int *p`	değişir	değişir	`int`'e işaretçi
`const int *p`	**değişmez**	değişir	sabit `int`'e işaretçi
`int *const p`	değişir	**değişmez**	`int`'e sabit işaretçi
`const int *const p`	**değişmez**	**değişmez**	sabit `int`'e sabit işaretçi

Ezber kuralı: `const` `*`'ın **solundaysa** gösterilen veri sabittir; `*`'ın **sağındaysa** işaretçi sabittir. Karışırsan tanımı **sağdan sola** oku: `int *const p` → "p, sabit bir işaretçidir, `int`'e işaret eder".

```c
int main(void) {
    int a = 1;
    int b = 2;
    const int *p1 = &a;            // gösterilen değer sabit
    int *const p2 = &a;            // işaretçi sabit
    const char *s = "Ali";

    p1 = &b;                       // olur: işaretçi değişebilir
    *p2 = 5;                       // olur: gösterilen değişebilir
    *p1 = 9;                       // HATA
    p2 = &b;                       // HATA
    s[0] = 'E';                    // HATA
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
constHata.c: In function 'main':
constHata.c:10:9: error: assignment of read-only location '*p1'
   10 |     *p1 = 9;                      // HATA
      |         ^
constHata.c:11:8: error: assignment of read-only variable 'p2'
   11 |     p2 = &b;                      // HATA
      |        ^
constHata.c:12:10: error: assignment of read-only location '*s'
   12 |     s[0] = 'E';                   // HATA
      |          ^
```

Üç hata mesajı da "salt okunur" (read-only) diyor. En çok kullanacağın biçim `const T *`'tır: fonksiyona "bu veriyi sadece okuyacağım" sözü vermek için (`uzunluk(const char *s)`). Derleyici o sözü senin yerine denetler.

> **Sınavda Çıkar**
>
> "Aşağıdaki satırlardan hangisi derlenmez?" biçiminde bir `const` sorusu mutlaka çıkar. Yöntem: Tanımı yaz (`const int *p`), sonra her satıra bak. `*p = ...` yazıyor ve `const` `*`'ın solundaysa **hata**. `p = ...` yazıyor ve `const` `*`'ın sağındaysa **hata**. İki tarafta da `const` varsa ikisi de hata.

Not: `restrict` ve `volatile`

`const` gibi iki **tip niteleyici** (type qualifier) daha vardır. Kendi programlarında nadiren yazarsın, ama kütüphane prototiplerinde ve gömülü sistem kodunda görürsün:

- **`restrict`** (C99) yalnız işaretçilere yazılır. "Bu işaretçinin gösterdiği belleğe, fonksiyon boyunca başka işaretçiyle erişilmeyecek" sözüdür; derleyici bu sözle daha hızlı kod üretebilir. Sözü tutmak sana düşer: iki `restrict` işaretçi aynı belleği gösterirse davranış tanımsızdır. `memcpy` prototipi (Bölüm 7) tipik örnektir; kaynak ile hedefin örtüşmeyeceği sözü verilmiştir (örtüşüyorsa `memmove` kullanılır):

```
void *memcpy(void *restrict hedef, const void *restrict kaynak, size_t n);
```

- **`volatile`**, "bu değişkenin değeri program dışından değişebilir" der (donanım yazmacı, sinyal işleyicisi). Derleyici onun okumalarını ve yazmalarını "gereksiz" sayıp silemez: `volatile int dugmeDurumu;`. Masaüstü programlarında nadirdir, mikrodenetleyici kodunda çok görülür.

9.10 Yerel değişkenin adresini döndürmek

Yerel değişken fonksiyon bitince yok olur (Bölüm 6'daki çağrı yığını). Adresini döndürürsen çağıranın elinde **artık var olmayan bir yeri gösteren** işaretçi kalır; buna **sarkan işaretçi** (dangling pointer) denir:

yerelAdres.c

```
#include <stdio.h>

int *yanlis(void) {
    int sayi = 42;
    return &sayi;                   // HATA: sayi, fonksiyon bitince yok olur
}

int main(void) {
    int *p = yanlis();
    (void)p;                        // p'yi kullanmak tanımsız davranıştır
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
yerelAdres.c: In function 'yanlis':
yerelAdres.c:5:12: warning: function returns address of local variable [-Wreturn-local-addr]
    5 |     return &sayi;                   // HATA: sayi, fonksiyon bitince yok olur
      |            ^~~~
```

Bu programı **çalıştırmadık**, çünkü `*p` ile okumak tanımsız davranıştır: O bellek sonraki çağrılarda başka işler için kullanılır; `*p` eski değeri de verebilir, çöp de verebilir, program da çökebilir. gcc'nin `-Wreturn-local-addr` uyarısı bu hatayı yakalar; **uyarı olsa da asla yok sayma**. Aynı hata yerel dizide de olur: `char sonuc[50]; ...; return sonuc;` yazan bir fonksiyon çağırana sarkan işaretçi verir.

Çözümler: (1) Sonucu çağıranın verdiği yere yaz (`bolVeKalan` gibi, ya da `char *sonuc` parametresi). (2) `static` yerel değişkenin adresini döndür; `static` değişken program boyunca yaşar. (3) Kalıcı yer ayır: `malloc`, Bölüm 10'da.

9.11 İşaretçinin işaretçisi

İşaretçi de bir değişken olduğuna göre onun da adresi vardır; o adresi tutan değişkene **işaretçinin işaretçisi** denir: `int **pp`. Her `*` bir basamak gösterir:

ciftIsaretci.c

```c
#include <stdio.h>

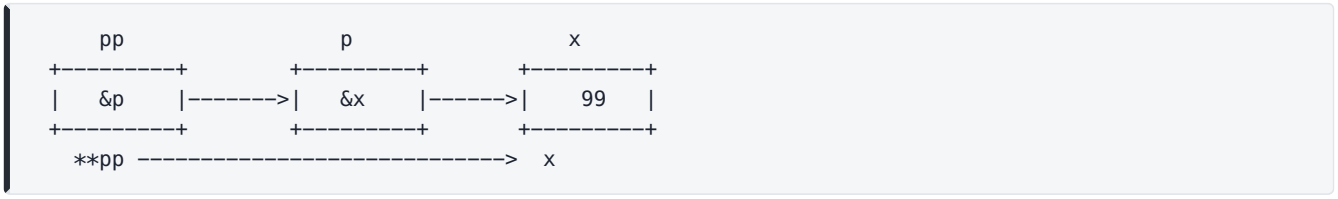
void yonelt(int **hedef, int *yeni) {
    *hedef = yeni;                  // main'deki işaretçiyi değiştir
}

int main(void) {
    int x = 5;
    int y = 8;
    int *p = &x;                    // p, x'i gösterir
    int **pp = &p;                  // pp, p'yi gösterir

    printf("x = %d, *p = %d, **pp = %d\n", x, *p, **pp);
    **pp = 99;                      // pp -> p -> x
    printf("x = %d\n", x);
    yonelt(&p, &y);                 // p artık y'yi gösterir
    printf("*p = %d\n", *p);
    return 0;
}
```

Çıktı

```
x = 5, *p = 5, **pp = 5
x = 99
*p = 8
```



Asıl kullanım yeri, fonksiyonun **işaretçiyi değiştirmesi** gereken durumdur. `swap`'ta bir `int`'i değiştirmek için `int *` geçirmiştik; bir `int *`'ı değiştirmek için de `int **` geçirmek gerekir (`yonelt`). Aynı kural, bir seviye yukarıda. Bölüm 14'te bağlı listenin başını değiştiren fonksiyonlarda (`Dugum **bas`) bunu kullanacaksın.

9.12 İşaretçi dizisi ve diziye işaretçi

İki tanım yalnızca bir çift parantezle ayrılır, ama anlamları çok farklıdır:

Tanım	Okunuşu	`sizeof` (64 bit)
`int *p[3]`	`p` bir **dizidir**: 3 elemanlı, elemanları `int *` (işaretçi dizisi)	24
`int (*q)[3]`	`q` bir **işaretçidir**: 3 elemanlı bir `int` dizisini gösterir (diziye işaretçi)	8

`[ ]` operatörü `*`'dan önceliklidir (Bölüm 2'deki öncelik sırası gibi). `int *p[3]` içinde `p` önce `[3]` ile birleşir: dizidir, elemanları `int *`'tır. `int (*q)[3]` içinde parantez `*`'ı öne alır: `q` önce işaretçidir, gösterdiği tip `int[3]`'tür.

isaretciDizisi.c

```c
#include <stdio.h>

int main(void) {
    int a = 10, b = 20, c = 30;
    int *p[3] = {&a, &b, &c};       // 3 tane int işaretçisi
    int m[3] = {1, 2, 3};
    int (*q)[3] = &m;               // 3 elemanlı diziyi gösteren tek işaretçi

    printf("sizeof(p) = %zu, sizeof(q) = %zu\n", sizeof(p), sizeof(q));
    printf("sizeof(*p) = %zu, sizeof(*q) = %zu\n", sizeof(*p), sizeof(*q));
    printf("*p[1] = %d, (*q)[1] = %d\n", *p[1], (*q)[1]);
    return 0;
}
```

Çıktı

```
sizeof(p) = 24, sizeof(q) = 8
sizeof(*p) = 8, sizeof(*q) = 12
*p[1] = 20, (*q)[1] = 2
```

`p`'nin her elemanı ayrı bir `int`'i gösterir (`*p[1]` → `b`). `q` ise **tek bir adres** tutar, ama gösterdiği şey bütün dizidir: `sizeof(*q)` 12 çıkar. İşaretçi dizisi, uzunlukları farklı string listeleri için doğal yoldur; bir sonraki başlıktaki `argv` böyledir. Diziye işaretçi ise iki boyutlu diziyi satır satır gezmeye yarar:

diziyeIsaretci.c

```c
#include <stdio.h>

int satirToplami(int (*m)[3], int satir) {      // int m[][3] ile aynıdır
    return m[satir][0] + m[satir][1] + m[satir][2];
}

int main(void) {
    int m[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int (*satir)[3] = m;            // m[0] satırının tamamını gösterir

    printf("%d %d\n", satir[0][1], satir[1][2]);
    satir++;                        // 3 int (12 bayt) ileri: sonraki satır
    printf("%d\n", (*satir)[0]);
    printf("%d\n", satirToplami(m, 1));
    return 0;
}
```

Çıktı

```
2 6
4
15
```

`satir++` işaretçiyi 3 `int` (12 bayt) ilerletti, çünkü işaretçi aritmetiği gösterilen tipin boyutuyla çalışır. Bölüm 7'de iki boyutlu diziyi fonksiyona `int m[][SUTUN]` diye geçirirken sütun sayısının zorunlu olduğunu görmüştün: derleyici o parametreyi `int (*m)[SUTUN]` diye okur ve satır başına kaç bayt atlayacağını sütun sayısından bulur.

> **Sınavda Çıkar**
>
> `int *p[3];` ile `int (*p)[3];` ayrımı sık sorulur. Parantez yoksa `[ ]` önce gelir ve `p` **dizidir**; parantez `*`'ı öne alırsa `p` **işaretçidir**. Emin olamazsan tanımı adından başlayarak oku.

9.13 Komut satırı argümanları

Programı çalıştırırken adının yanına yazdığın sözcükler programa **komut satırı argümanı** (command-line argument) olarak ulaşır. Bunun için `main`'in ikinci biçimi kullanılır: `int main(int argc, char *argv[])`.

Parametre	Anlamı
`argc`	Argüman sayısı (argument count). Program adı da sayıldığı için en az 1'dir
`argv`	Argüman dizisi (argument vector): her eleman bir string'i gösteren `char *`
`argv[0]`	Programın adı
`argv[1]`, `argv[2]`, ...	Sen yazdıkların; hepsi **string**'tir (`"5"` bir sayı değil, metindir)
`argv[argc]`	Her zaman `NULL`

`char *argv[]` parametrede `char **argv` ile aynıdır (dizi parametresi işaretçiye dönüşür). Metni sayıya çevirmek için `stdlib.h`'deki `atoi` kullanılır.

argumanlar.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int toplam = 0;

    printf("Arguman sayisi: %d\n", argc);
    for (int i = 1; i < argc; i++) {
        printf("argv[%d] = %s\n", i, argv[i]);
        toplam += atoi(argv[i]);
    }
    printf("Toplam: %d\n", toplam);
    return 0;
}
```

Çıktı

```
Arguman sayisi: 1
Toplam: 0
```

Kitabın derleme sistemi programları **argümansız** çalıştırdığı için `argc` 1 çıktı ve döngü hiç dönmedi. Argümanlı çalıştırmak için terminalde program adının yanına boşlukla ayırarak yaz (Windows `cmd`'de `./argumanlar` yerine `.\argumanlar` yaz; PowerShell ikisini de kabul eder):

```
gcc -std=c17 -Wall -Wextra argumanlar.c -o argumanlar
./argumanlar 12 30 elma
Arguman sayisi: 4
argv[1] = 12
argv[2] = 30
argv[3] = elma
Toplam: 42
```

`atoi("elma")` sayıya çevrilemediği için 0 döndürür: 12 + 30 + 0 = 42. Bellekte `argv`, elemanları `char *` olan (her biri bir string'in ilk harfini gösteren) bir dizidir:

```
argv (argc = 4)
+--------+
| adres -|--> "./argumanlar"     argv[0]
| adres -|--> "12"               argv[1]
| adres -|--> "30"               argv[2]
| adres -|--> "elma"             argv[3]
|  NULL  |                       argv[4] = argv[argc]
+--------+
```

> **İpucu: Argüman kontrolü**
>
> Programın belli sayıda argüman beklediğini varsay: `if (argc != 3) { printf("Kullanim: ..."); return 1; }` ile başla. VS Code'daki ▷ düğmesi argüman vermez; argümanlı denemeyi terminalden yap.

9.14 void işaretçi ve fonksiyon işaretçisi

İşaretçilerin son iki türüne kısa bir bakış; ikisini de `qsort` içinde göreceksin.

void *

`void *` **tipi olmayan** genel bir adrestir. Herhangi bir işaretçi `void *`'a, `void *` da herhangi bir işaretçiye cast olmadan atanabilir. Ama tipi olmadığı için ne `*` ile okunabilir (kaç bayt okunacağını bilmez), ne de standart C'de `v + 1` ile ilerletilebilir. Önce gerçek tipe çevirmek gerekir: `*(int *)v`. `malloc` (Bölüm 10) `void *` döndürür; `qsort` da elemanların tipini bilmediği için `void *` ile çalışır.

Fonksiyon işaretçisi

Fonksiyonların da bellekte adresi vardır; fonksiyon adı, dizi adı gibi o adrese dönüşür. **Fonksiyon işaretçisi** (function pointer) tanımı fonksiyonun dönüş tipini ve parametre tiplerini içerir:

fonksiyonIsaretcisi.c

```c
#include <stdio.h>

int topla(int a, int b) {
    return a + b;
}

int main(void) {
    int (*islem)(int, int);         // (int, int) alıp int döndüren fonksiyonun
                                    // işaretçisi
    islem = topla;                  // fonksiyon adı = fonksiyonun adresi
    printf("%d\n", islem(3, 4));    // topla(3, 4)
    return 0;
}
```

Çıktı

```
7
```

Parantezler çok önemlidir: `int (*islem)(int, int)` fonksiyon işaretçisidir; `int *islem(int, int)` ise "`int *` döndüren bir fonksiyon"dur.

qsort

`stdlib.h`'deki `qsort` her tipten diziyi sıralar. Tipi bilmediği için dört şeye ihtiyaç duyar:

```
void qsort(void *dizi, size_t adet, size_t boyut,
           int (*karsilastir)(const void *, const void *));
```

Sırayla: dizinin adresi, eleman sayısı, bir elemanın bayt boyutu (`sizeof(dizi[0])`) ve iki elemanı kıyaslayan **senin yazdığın** fonksiyonun adresi. Karşılaştırma fonksiyonu iki elemanın adresini alır; `x` önce gelecekse negatif, eşitse 0, `y` önce gelecekse pozitif döndürür:

qsortOrnek.c

```c
#include <stdio.h>
#include <stdlib.h>

int kucuktenBuyuge(const void *x, const void *y) {
    int a = *(const int *)x;        // void * -> int *, sonra oku
    int b = *(const int *)y;
    if (a < b) {
        return -1;
    }
    if (a > b) {
        return 1;
    }
    return 0;
}

int main(void) {
    int sayilar[6] = {42, 7, 19, 3, 88, 7};
    size_t n = sizeof(sayilar) / sizeof(sayilar[0]);

    qsort(sayilar, n, sizeof(sayilar[0]), kucuktenBuyuge);
    for (size_t i = 0; i < n; i++) {
        printf("%d ", sayilar[i]);
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
3 7 7 19 42 88
```

`qsort(..., kucuktenBuyuge)` yazarken fonksiyon adının yanına **parantez koymadın**: `kucuktenBuyuge(...)` fonksiyonu çağırır, `kucuktenBuyuge` ise adresini verir. Büyükten küçüğe sıralamak için karşılaştırma fonksiyonunun sonuçlarını tersine çevirmen yeterlidir. Karşılaştırmada `return a - b;` yazma: `int` taşabilir (işaretli taşma tanımsız davranıştır); `<` ve `>` kullan.

Bölüm Özeti – Bilmen Gerekenler

- Her değişkenin bir **adresi** vardır; `&x` adresi verir (`%p` ile yazdırılır, bilgisayardan bilgisayara değişir).
- İşaretçi adres tutan değişkendir: `int *p = &x;`. `*p` gösterilen yere erişir (dolaylama), `*p = 20;` `x`'i değiştirir. `p` adrestir, `*p` değerdir.
- `int *p, q;` yalnız `p`'yi işaretçi yapar. Bugünkü bilgisayarlarda tüm işaretçiler aynı boyuttadır (64 bitte 8 bayt); tip, `*p`'nin kaç bayt okuyacağını ve `p + 1`'in ne kadar ilerleyeceğini belirler.
- `NULL` hiçbir yeri göstermez; `*p` ile erişmek tanımsız davranıştır, masaüstünde genellikle çökertir (`Segmentation fault`). Başlatılmamış işaretçi tanımsızdır. Tanımlarken adres ya da `NULL` ver, kullanmadan önce `p != NULL` kontrol et.
- C'de her zaman **değer ile** geçirilir; "adres ile geçirme" adresin değer olarak geçirilmesidir. `swap(&x, &y)`, `scanf("%d", &x)` ve birden çok sonuç döndürme aynı kalıptır.

- `p + 1` bir **eleman** ilerler (bayt değil); `p - q` eleman sayısını verir. `*p++` işaretçiyi ilerletir, `(*p)++` gösterilen değeri artırır.
- `a[i]` ≡ `*(a + i)`. Fonksiyona dizi geçince adres geçer (`int a[]` ≡ `int *a`), bu yüzden fonksiyon içinde `sizeof(a)` işaretçi boyutunu verir. Dizi adına atama yapılamaz.
- `char s[] = "Ali"` değiştirilebilir kopyadır; `const char *p = "Ali"` salt okunur metni gösterir. `const int *p`: değer sabit; `int *const p`: işaretçi sabit.
- Yerel değişkenin adresini döndürmek hatadır. `int **pp` işaretçiyi değiştiren fonksiyonlarda ve `argv`'de çıkar; `main(int argc, char *argv[])` komut satırı argümanlarını alır. `qsort`, `void *` ve fonksiyon işaretçisini birlikte kullanır.
- `int *p[3]` işaretçi **dizisidir** (24 bayt), `int (*p)[3]` 3 elemanlı diziyi gösteren tek **işaretçidir** (8 bayt); `int m[][3]` parametresi ikincisidir. `restrict` "bu belleğe başka işaretçiyle erişilmez" sözüdür (`memcpy`), `volatile` değerin dışarıdan değişebileceğini söyler.

Sorular

A) Çoktan Seçmeli

1 `int *p, q;` tanımından sonra aşağıdakilerden hangisi doğrudur?

(A) `p` ve `q` ikisi de `int` işaretçisidir.
(B) `p` bir `int` işaretçisidir, `q` sıradan bir `int`'tir.
(C) `p` sıradan bir `int`'tir, `q` bir `int` işaretçisidir.
(D) İkisi de sıradan `int`'tir; `*` işareti burada anlam taşımaz.
(E) Tanım derlenmez.

2 Aşağıdaki kod parçasından sonra `p[2] + *(p + 1)` ifadesinin değeri kaçtır?

```c
int a[] = {10, 20, 30, 40, 50};
int *p = a + 1;
```

(A) 50 (B) 60 (C) 70 (D) 80 (E) 90

3 `int a = 1, b = 2; const int *p = &a; int *const q = &a;` tanımlarından sonra aşağıdaki satırlardan hangisi **derlenmez**?

(A) `p = &b;`
(B) `*q = 5;`
(C) `a = 7;`
(D) `*p = 5;`
(E) `printf("%d", *p);`

4 `void f(int a[]) { ... }` fonksiyonu ana programdan `f(dizi);` diye çağrılıyor. Aşağıdakilerden hangisi doğrudur?

(A) `a`, fonksiyon içinde bir işaretçidir; `sizeof(a)` dizinin boyutunu değil işaretçinin boyutunu verir.

(B) `a`, `dizi`'nin kopyasıdır; fonksiyonda yapılan değişiklikler `dizi`'yi etkilemez.

(C) `int a[]` ile `int *a` farklı anlama gelir; ikisi aynı parametre olarak kullanılamaz.

(D) `sizeof(a) / sizeof(a[0])` her zaman dizinin eleman sayısını verir.

(E) Fonksiyon, dizinin kaç elemanlı olduğunu `a`'dan otomatik öğrenir.

5 `int *p = NULL;` tanımı için aşağıdakilerden hangisi doğrudur?

(A) `*p` her zaman 0 değerini verir.

(B) `NULL`, `p`'nin bellekte 0 numaralı değişkeni gösterdiği anlamına gelir.

(C) `NULL` işaretçiye `*p = 5;` yazmak bir derleme hatasıdır.

(D) `if (p)` koşulu bu durumda doğru (true) sayılır.

(E) `*p`'yi kullanmadan önce `if (p != NULL)` yazmak, NULL işaretçiye erişimi önler.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int a[4] = {5, 10, 15, 20};
    int *p = a;
    int x = *p++;
    int y = (*p)++;
    int z = *++p;

    printf("%d %d %d\n", x, y, z);
    printf("%d %d %d %d\n", a[0], a[1], a[2], a[3]);
    printf("%d\n", *p);
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

void degistir(int *a, int *b) {
    int gecici = *a;
    *a = *b;
    *b = gecici;
}

int main(void) {
    int s = 1;
    int t = 2;
    int *p = &s;
    int *q = &s;

    *p = *p + 5;
    *q = *q * 2;
    degistir(&s, &t);
    degistir(p, q);
    printf("%d %d\n", s, t);
    printf("%d %d\n", *p, *q);
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int a[5] = {2, 4, 6, 8, 10};
    int *p = a + 4;
    int *q = &a[1];

    printf("%d\n", (int)(p - q));
    printf("%d\n", *(p - 2));
    printf("%d\n", *q + *(q + 1));
    p -= 3;
    printf("%d %d\n", *p, p[2]);
    printf("%d\n", p == q);
    return 0;
}
```

C) Kod Yazma

9 Klavyeden 5 tamsayı okuyup **en küçük** ile **en büyük** sayıyı yazdıran `minMaks` programını yaz. Bulma işi `void minMaks(const int *a, int n, int *enKucuk, int *enBuyuk)` fonksiyonunda yapılsın; iki sonuç işaretçi parametrelerle döndürülsün.

```
Sayilari gir: 8 -3 15 0 7
En kucuk: -3
En buyuk: 15
```

10 Bir kelime okuyup **tersini** yazdıran `ters` programını yaz. Ters çevirme `void tersCevir(char *s)` fonksiyonunda, köşeli parantez (`[]`) kullanmadan, yalnız iki işaretçiyle (biri başta, biri sonda; ortada buluşana kadar harf değiştirerek) yapılsın. Boş string'de fonksiyon hiçbir şey yapmamalı. Kelime en fazla 49 harfli olacak.

```
Kelime: Merhaba
Ters  : abahreM
```

Çözümler

Cevap anahtarı

1 B	2 C	3 D	4 A	5 E

1 Cevap: B

`*` tipe değil, hemen sağındaki ada aittir: `p` işaretçi, `q` sıradan `int` olur. A, `int *p, *q;` yazılsaydı doğru olurdu. C tersini söylüyor. D yanlış, `*` burada işaretçi anlamı taşır. E yanlış, tanım geçerlidir.

2 Cevap: C

`p = a + 1` olduğundan `p`, `a[1]`'i gösterir. `p[2]` = `*(p + 2)` = `a[3]` = 40; `*(p + 1)` = `a[2]` = 30. Toplam 70. A, `p`'yi `a` sanınca çıkar (`a[2] + a[1]` = 50). E, `p`'yi `a + 2` sanınca çıkar (`a[4] + a[3]` = 90). B (40 + 20) ve D (40 + 40) ikinci terimi yanlış elemandan alır.

3 Cevap: D

`const int *p`: gösterilen değer sabit, yani `*p = 5;` yazılamaz (D derlenmez). A olur, çünkü `p`'nin kendisi sabit değil. B olur: `q` sabit işaretçidir ama gösterdiği `int` sabit değildir. C olur, `a` sıradan değişken. E yalnız okuyor. (`q = &b;` yazılsaydı o da derlenmezdi, ama seçeneklerde yok.)

4 Cevap: A

Dizi fonksiyona geçerken ilk elemanın adresine dönüşür; parametredeki `int a[]` aslında `int *a`'dır. Bu yüzden `sizeof(a)` işaretçi boyutudur (64 bitte 8). B yanlış, kopya yapılmaz, değişiklik `dizi`'yi etkiler. C yanlış, ikisi tamamen aynıdır. D yanlış, fonksiyon içinde 8 / 4 = 2 gibi yanlış bir sayı verir. E yanlış, uzunluk ayrıca parametre olarak geçirilmelidir.

5 Cevap: E

Kullanmadan önce `p != NULL` kontrolü, NULL'a erişimi önleyen standart yöntemdir. A yanlış: `*p` ile erişmek tanımsız davranıştır (masaüstünde genellikle çökme), 0 vermez. B yanlış: `NULL` hiçbir değişkeni göstermez. C yanlış: `*p = 5;` derlenir, çalışırken çöker. D yanlış: `NULL` sıfırdır, `if (p)` **yanlış** (false) sayılır.

6

Önce `p` dizinin başını gösterir. Her satırdan sonra `p`'nin yerini ve dizinin içeriğini yaz:

Satır	p	Dizi	Açıklama
`int x = *p++;`	`a[1]`	5 10 15 20	`x` = 5 (eski `*p`), sonra `p` ilerler
`int y = (*p)++;`	`a[1]`	5 **11** 15 20	`y` = 10 (artmadan önceki değer), `a[1]` artar, `p` yerinde
`int z = *++p;`	`a[2]`	5 11 15 20	Önce `p` ilerler, sonra `z` = `a[2]` = 15

Son `printf` `p`'nin gösterdiği yeri, yani `a[2]` = 15'i yazar.

Çıktı
```
5 10 15
5 11 15 20
15
```

7

`p` ve `q` **aynı değişkeni** (`s`) gösterir; biri üzerinden yapılan değişiklik ikisinde de görünür.

1. `*p = *p + 5;` → `s` = 6.
2. `*q = *q * 2;` → `s` = 12.
3. `degistir(&s, &t);` → `s` = 2, `t` = 12.
4. `degistir(p, q);` → iki parametre de `s`'yi gösterir: `gecici = 2;` `*a = *b;` (`s` = 2) `*b = gecici;` (`s` = 2). Hiçbir şey değişmez.

İlk satır `s t`, ikinci satır `*p *q` yani yine `s` ile `s`:

Çıktı
```
2 12
2 2
```

8

`a` = {2, 4, 6, 8, 10}; `p` = `&a[4]`, `q` = `&a[1]`.

1. `p - q` = 4 - 1 = **3** (eleman sayısı).
2. `*(p - 2)` = `a[2]` = **6**.
3. `*q + *(q + 1)` = `a[1] + a[2]` = 4 + 6 = **10**.
4. `p -= 3;` → `p` = `&a[1]`. `*p` = 4, `p[2]` = `a[3]` = 8.
5. Artık `p` ve `q` aynı yeri gösterir, `p == q` = **1**.

Çıktı
```
3
6
10
4 8
1
```

9

Sonuçları `*enKucuk` ve `*enBuyuk` ile main'deki değişkenlere yazıyoruz; ilk eleman iki sonucun da başlangıç değeridir. `scanf("%d", &sayilar[i])` ile `scanf("%d", sayilar + i)` aynı şeydir.

minMaks.c
```c
#include <stdio.h>

void minMaks(const int *a, int n, int *enKucuk, int *enBuyuk) {
    *enKucuk = a[0];
    *enBuyuk = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] < *enKucuk) {
            *enKucuk = a[i];
        }
        if (a[i] > *enBuyuk) {
            *enBuyuk = a[i];
```

```
        }
    }
}

int main(void) {
    int sayilar[5];
    int kucuk = 0;
    int buyuk = 0;

    printf("Sayilari gir: ");
    for (int i = 0; i < 5; i++) {
        scanf("%d", &sayilar[i]);
    }
    minMaks(sayilar, 5, &kucuk, &buyuk);
    printf("En kucuk: %d\n", kucuk);
    printf("En buyuk: %d\n", buyuk);
    return 0;
}
```

Çıktı

```
Sayilari gir: 8 -3 15 0 7
En kucuk: -3
En buyuk: 15
```

10

`bas` ilk harfi, `son` son harfi gösterir. Harfleri değiştirip iki işaretçiyi birbirine doğru yürütürüz; `bas < son` olduğu sürece devam ederiz. Boş string'de `s + strlen(s) - 1` dizinin **önüne** düşerdi (tanımsız), bu yüzden en başta `*s == '\0'` kontrolü var. `scanf("%49s", kelime)` içinde dizi adı zaten adres olduğu için `&` yoktur.

ters.c

```
#include <stdio.h>
#include <string.h>

void tersCevir(char *s) {
    if (*s == '\0') {
        return;                     // boş string: yapacak bir şey yok
    }
    char *bas = s;
    char *son = s + strlen(s) - 1;

    while (bas < son) {
        char gecici = *bas;
        *bas = *son;
        *son = gecici;
        bas++;
        son--;
    }
}

int main(void) {
    char kelime[50];

    printf("Kelime: ");
    scanf("%49s", kelime);
    tersCevir(kelime);
    printf("Ters  : %s\n", kelime);
    return 0;
}
```

Çıktı

```
Kelime: Merhaba
Ters  : abahreM
```

BÖLÜM 10

Dinamik Bellek Yönetimi

Bu Bölümde Öğreneceklerin

- Sabit boyutlu dizinin neden yetmediğini ve değişken uzunluklu dizinin (VLA) risklerini
- Bellek bölgelerini: statik alan, yığın (stack) ve öbek (heap)
- `malloc`, `calloc`, `realloc` ve `free` kullanımını; `NULL` kontrolünü
- Çalışma zamanında boyutu belirlenen ve büyüyen dizileri
- Fonksiyondan dinamik dizi döndürmeyi ve belleği kimin bırakacağını (sahiplik)
- Dinamik string ve 2 boyutlu dinamik dizi oluşturmayı
- Bellek sızıntısı, sarkan işaretçi ve çift `free` hatalarından korunmayı
- Valgrind, Dr. Memory ve AddressSanitizer'ın ne işe yaradığını

10.1 Neden dinamik bellek?

Bölüm 7'de dizinin boyutunun programı yazarken belli olması gerektiğini gördün. Bir derste kaç öğrenci olacağını bilmiyorsan `int notlar[100];` yazmak bir kumardır: 101. öğrenci sığmaz (taşma, tanımsız davranış), 10 öğrencili sınıfta ise 90 yerlik bellek boşa gider. Çözüm, boyutu **program çalışırken** öğrenip o kadar yeri o anda istemektir. Buna **dinamik bellek ayırma** (dynamic memory allocation) denir.

C99 bunun bir kestirmesini getirdi: **değişken uzunluklu dizi** (VLA, variable length array).

```
int n;
scanf("%d", &n);
int a[n];              // VLA: boyut çalışma anında belli oluyor
```

Kısa ve cazip görünür, ama kitapta ve sınavda `malloc` kullanacağız, çünkü:

- VLA **yığında** (stack) yer alır. `n` büyükse yığın taşar ve program çöker; hatayı yakalamanın yolu yoktur. `malloc` ise yer bulamayınca `NULL` döndürür, sen kontrol edersin.
- Fonksiyon bitince yok olur; Bölüm 9'daki yerel dizi gibi güvenle döndürülemez.
- Boyutunu sonradan büyütemezsin.
- C11'den beri derleyicilerin VLA desteklemesi zorunlu değildir; bazıları (ör. MSVC) hiç desteklemez.

10.2 Bellek bölgeleri

Çalışan bir program belleği üç ana bölgede kullanır:

Bölge	İçinde ne var?	Ömrü	Kim yönetir?	Boyut
Statik alan	global ve `static` değişkenler, string sabitleri	program boyunca	derleyici	derleme anında sabit
Yığın (stack)	yerel değişkenler, parametreler	fonksiyon bitene kadar	otomatik	küçük (genelde birkaç MB)
Öbek (heap)	`malloc`, `calloc`, `realloc` ile alınan bloklar	sen `free` edene kadar	**sen**	büyük (RAM'in elverdiği kadar)

Yerel değişkenlerin çağrı yığınında yaşadığını Bölüm 6'dan, `static` değişkenin program boyunca durduğunu da oradan biliyorsun. Öbek üçüncü ve tek **elle yönetilen** bölgedir: yeri sen istersin, işin bitince sen geri verirsin.

> **Not: Yığın ve öbek karışmasın**
>
> Türkçe kaynakların çoğu hem *stack*'e hem *heap*'e "yığın" der. Bu kitapta *stack* = **yığın**, *heap* = **öbek**. Sınavda karışıklık olmasın diye parantez içinde İngilizcesini de yaz.

Bir işaretçi ile öbekteki blok ilişkisi şöyledir:

```
  YIĞIN (stack)                       ÖBEK (heap)
+----------------+                  +----+----+----+----+----+
| n = 5          |                  | 10 | 20 | 30 | 40 | 50 |
| p = (adres) ---+----------------> +----+----+----+----+----+
+----------------+                   malloc(5 * sizeof *p) ile alındı
main'in yerel                        free(p) yazılana kadar yaşar
değişkenleri
```

`p` yığındadır; fonksiyon bitince yok olur. Gösterdiği blok ise **öbektedir** ve `p` yok olsa bile durmaya devam eder. `free` yazmadan `p`'yi kaybedersen bloğa bir daha ulaşamazsın; buna **bellek sızıntısı** denir (aşağıda).

10.3 malloc ve free

```
void *malloc(size_t boyut);       // <stdlib.h>
void  free(void *p);
```

`malloc` (memory allocation) öbekten `boyut` **bayt** ister ve bloğun ilk baytının adresini döndürür. Yer bulamazsa `NULL` döndürür. Ayrılan bloğun içi **ilk değersizdir** (çöp). `void *` döndürdüğü için (Bölüm 9) sonucu cast yapmadan herhangi bir işaretçiye atayabilirsin.

mallocTemel.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;
    int *dizi = malloc(n * sizeof *dizi);   // 5 int'lik yer iste

    if (dizi == NULL) {                     // her malloc'tan hemen sonra kontrol
        printf("Bellek ayrilamadi.\n");
        return 1;
    }
    int toplam = 0;
    for (int i = 0; i < n; i++) {
        dizi[i] = (i + 1) * 10;             // sıradan dizi gibi kullan
```

```
        toplam += dizi[i];
    }
    printf("Ayrilan bayt: %zu\n", n * sizeof *dizi);
    printf("Ilk: %d, son: %d, toplam: %d\n", dizi[0], dizi[n - 1], toplam);
    free(dizi);                             // işin bitti: yeri geri ver
    return 0;
}
```

Çıktı

```
Ayrilan bayt: 20
Ilk: 10, son: 50, toplam: 150
```

Satır satır:

- `malloc(n * sizeof *dizi)`: bayt sayısı = eleman sayısı × bir elemanın boyutu. `sizeof *dizi`, "`dizi`'nin gösterdiği şeyin boyutu" demektir (`int`, yani 4 bayt); 5 × 4 = **20** bayt çıkar. `sizeof(int)` de yazabilirsin. `sizeof *dizi` tercih edilir, çünkü ileride `dizi`'yi `double *` yaparsan bu satır kendiliğinden doğru kalır.
- Dönen adres bir diziyi gösterir. Bölüm 9'daki `a[i] == *(a + i)` eşitliği burada da geçerli: `dizi[i]` yazabilirsin.
- `free(dizi)`: bloğu öbeğe geri verir. **Her `malloc` için tam bir `free`** yazılır.

> **Not: C'de cast yazılmaz**
>
> Eski kitaplarda `int *p = (int *)malloc(...)` görürsün. C'de cast **gerekmez** (`void *` her işaretçiye kendiliğinden dönüşür); cast'i C++ ister, ama C++ ayrı bir dildir. Cast zararsızdır, yalnızca gereksizdir; bu kitapta yazmıyoruz. `malloc`, `calloc`, `realloc` ve `free` için `#include <stdlib.h>` şarttır.

> **Sık Yapılan Hata: NULL kontrolünü atlamak**
>
> `malloc` yer bulamayınca `NULL` döndürür; kontrol etmeden kullanırsan Bölüm 9'daki `NULL` işaretçi çökmesini yaşarsın (`Segmentation fault`; Windows'ta program sessizce kapanır). Küçük bir programda `malloc`'ın başarısız olması nadirdir, ama çok büyük bir blok isteyince ya da gömülü sistemlerde olur. Alışkanlık: `malloc` satırının hemen altında `if (p == NULL)`. Kontrolü `main`'de yaparsan `return 1;` ile çık; bellek ayıran bir fonksiyondaysan `NULL` döndürüp kararı çağırana bırak.

`free` kuralları:

- Yalnızca `malloc`, `calloc` ya da `realloc`'tan gelen adres `free` edilir, ve **bloğun başlangıç adresiyle** (ortasıyla değil).
- `free`'den sonra `p`'nin içindeki adres değişmez, ama o bloğa artık dokunamazsın.
- `free(NULL)` güvenlidir; hiçbir şey yapmaz.

10.4 calloc

```
void *calloc(size_t adet, size_t boyut);
```

`calloc` (clear allocation) `adet × boyut` bayt ayırır ve **hepsini 0'a ayarlar**. Çarpmayı kendisi yaptığı için parametreleri ayrıdır:

callocFark.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 5;
    int *a = calloc(n, sizeof *a);          // 5 int, hepsi 0

    if (a == NULL) {
        return 1;
    }
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    free(a);
    return 0;
}
```

Çıktı

```
0 0 0 0 0
```

`malloc` ise içeriği **olduğu gibi bırakır**. Onu doğrudan okumak Bölüm 2'deki çöp değer sorunudur:

```c
int *p = malloc(5 * sizeof *p);
printf("%d\n", p[0]);           // TANIMSIZ: henüz değer verilmedi, çöp okur
```

	`malloc(n * sizeof *p)`	`calloc(n, sizeof *p)`
Parametre	toplam bayt sayısı	eleman sayısı ve eleman boyutu
Başlangıç içeriği	çöp (belirsiz)	tüm baytlar 0
Ne zaman?	hemen üstüne yazacaksan	sayaç, toplam gibi 0'dan başlayacak diziler

> **Sınavda Çıkar**
>
> "`malloc` ile `calloc` farkı nedir?" klasiktir: **`calloc` sıfırlar, `malloc` sıfırlamaz** ve parametre biçimleri farklıdır. Bayt hesabı da sorulur: `malloc(25 * sizeof(int))` → 25 × 4 = **100 bayt**; `calloc(25, sizeof(int))` de aynı 100 baytı verir, ama hepsi 0'dır.

10.5 Boyutu kullanıcıdan alıp dinamik dizi

Dinamik belleğin asıl işi budur: eleman sayısı `n`, ancak çalışma anında belli olur. Aşağıdaki program önce kaç not gireceğini sorar, sonra tam o kadar yer ayırır:

dinamikOrtalama.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int n = 0;

    printf("Kac not girilecek? ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Gecersiz sayi.\n");
        return 1;
```

```
    }
    int *notlar = malloc(n * sizeof *notlar);
    if (notlar == NULL) {
        printf("Bellek ayrilamadi.\n");
        return 1;
    }
    printf("Notlari gir: ");
    int toplam = 0;
    for (int i = 0; i < n; i++) {
        scanf("%d", &notlar[i]);
        toplam += notlar[i];
    }
    printf("Ortalama: %.2f\n", (double)toplam / n);
    free(notlar);
    return 0;
}
```

Çıktı

```
Kac not girilecek? 5
Notlari gir: 70 85 90 60 95
Ortalama: 80.00
```

`n`'yi okuduktan **sonra** ayırdığımıza dikkat et; önce kontrol (`n <= 0`), sonra `malloc`, sonra `NULL` kontrolü. `scanf("%d", &notlar[i])` yerine `notlar + i` de yazabilirdin (Bölüm 9).

10.6 realloc ile büyüyen dizi

Eleman sayısını baştan hiç bilmiyorsan? "0 girilene kadar sayı oku" gibi durumlarda diziyi doldukça büyütürsün:

```
void *realloc(void *p, size_t yeniBoyut);
```

`realloc` (re-allocation) `p`'nin gösterdiği bloğu `yeniBoyut` bayta büyütür ya da küçültür:

- **Eski içerik korunur** (küçültüyorsan sığan kısmı).
- Blok yerinde genişleyemiyorsa `realloc` yeni bir yer bulur, içeriği oraya kopyalar ve eski bloğu **kendisi `free` eder**. Dönen adres bu yüzden eskisinden farklı olabilir; eski adresi kullanmaya devam etme.
- Eklenen yeni kısım **çöptür** (sıfırlanmaz).
- Başarısız olursa `NULL` döndürür ve **eski blok olduğu gibi durur**.
- `realloc(NULL, n)` tam olarak `malloc(n)` gibi çalışır. `realloc(p, 0)` yazma: davranışı taşınabilir değildir; bırakmak için `free` kullan.

```
int *yeni = realloc(p, 5 * sizeof *yeni);   // 3 elemandan 5 elemana büyüt
if (yeni == NULL) {
    free(p);                                 // eski blok hâlâ bizim
    return 1;
}
p = yeni;                                    // artık yeni adres geçerli
```

Büyütme başarılıysa ilk üç eleman olduğu gibi korunur; yalnız yeni eklenen 3. ve 4. indeksler çöptür ve değer vermen gerekir. `yeni` geçici işaretçisi tesadüf değildir, sıradaki kutuya bak.

● Sık Yapılan Hata: p = realloc(p, ...) yazmak

En çok görülen `realloc` hatası:

```
p = realloc(p, yeniBoyut);      // YANLIŞ: başarısız olursa p = NULL olur
```

`realloc` başarısız olursa `NULL` döner, bu da `p`'ye atanır. Eski bloğun adresini tutan **tek işaretçi** gitmiş olur: blok hâlâ öbekte durur ama ona bir daha ulaşamazsın, yani **sızıntı** (üstelik veri de kaybolur). Doğru kalıp: sonucu önce **geçici bir işaretçiye** al, `NULL` değilse `p`'ye ver (yukarıdaki gibi): `int *yeni = realloc(p, ...); if (yeni == NULL) {...} p = yeni;`

Kapasiteyi ikiye katlamak

Her yeni sayıda diziyi 1 büyütürsen `n` eleman için `n` kez `realloc` çağrılır ve bloğun her seferinde kopyalanma ihtimali vardır. Bunun yerine **kapasiteyi** (capacity) ikiye katlarsın: `kapasite` kadar elemanlık yer ayırırsın, `adet` kadarını kullanırsın; `adet` kapasiteye ulaşınca kapasiteyi iki katına çıkarırsın. Böylece `realloc` yaklaşık log₂(n) kez çağrılır.

buyuyenDizi.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int kapasite = 2;                       // başlangıçta 2 elemanlık yer
    int adet = 0;                           // şu an kaç eleman var
    int *sayilar = malloc(kapasite * sizeof *sayilar);
    int x = 0;

    if (sayilar == NULL) {
        return 1;
    }
    printf("Sayilari gir (0 ile biter): ");
    while (scanf("%d", &x) == 1 && x != 0) {
        if (adet == kapasite) {             // dizi dolu: büyüt
            int yeniKapasite = kapasite * 2;
            int *yeni = realloc(sayilar, yeniKapasite * sizeof *yeni);
            if (yeni == NULL) {
                free(sayilar);
                return 1;
            }
            sayilar = yeni;
            printf("[kapasite %d -> %d]\n", kapasite, yeniKapasite);
            kapasite = yeniKapasite;
        }
        sayilar[adet] = x;
        adet++;
    }
    int toplam = 0;
    for (int i = 0; i < adet; i++) {
        toplam += sayilar[i];
    }
    printf("Adet: %d, kapasite: %d, toplam: %d\n", adet, kapasite, toplam);
    free(sayilar);
    return 0;
}
```

Çıktı
```
Sayilari gir (0 ile biter): 4 8 15 16 23 42 0
[kapasite 2 -> 4]
[kapasite 4 -> 8]
Adet: 6, kapasite: 8, toplam: 108
```

İzleyelim: Kapasite 2 iken 4 ve 8 sığar. 15 gelince `adet == kapasite` olur, kapasite 4'e çıkar; 15 ve 16 yerleşir. 23 gelince kapasite 8'e çıkar; 23 ve 42 de sığar. `0` gelince okuma biter. Sonuçta 6 eleman, 8 kapasite vardır (`4+8+15+16+23+42 = 108`). Bir eleman eklemeden önce **dolu mu** diye bakmak dinamik dizinin temel kalıbıdır.

10.7 Fonksiyondan dinamik dizi döndürmek

Bölüm 9'da yerel diziyi döndürmenin sarkan işaretçi verdiğini görmüştün: dizi yığında olduğu için fonksiyon bitince yok olur. **Öbekteki blok ise `free` edilene kadar yaşar**, bu yüzden fonksiyon onu rahatça döndürebilir:

kareDizisi.c
```c
#include <stdio.h>
#include <stdlib.h>

// 1'den n'e kadar sayıların karelerini tutan yeni bir dizi döndürür.
// Hata olursa NULL. Dönen belleği ÇAĞIRAN free etmelidir.
int *kareleriOlustur(int n) {
    int *a = malloc(n * sizeof *a);

    if (a == NULL) {
        return NULL;
    }
    for (int i = 0; i < n; i++) {
        a[i] = (i + 1) * (i + 1);
    }
    return a;
}

int main(void) {
    int n = 5;
    int *kareler = kareleriOlustur(n);

    if (kareler == NULL) {
        return 1;
    }
    for (int i = 0; i < n; i++) {
        printf("%d ", kareler[i]);
    }
    printf("\n");
    free(kareler);
    return 0;
}
```

Çıktı
```
1 4 9 16 25
```

Burada yeni bir soru çıkar: **belleği kim serbest bırakacak?** Bloğu `kareleriOlustur` ayırdı ama `main` kullandı; `free`'yi `main` yazar. Bu sorumluluğa **sahiplik** (ownership) denir: Fonksiyon bloğu döndürdüğü anda sahipliği çağırana devreder. Yorum satırında yazmak iyi alışkanlıktır.

Kalıp	Kim ayırır?	Kim `free` eder?
`p = olustur(...)` (yukarıdaki)	fonksiyon	**çağıran**
`doldur(tampon, n)` (`fgets` gibi)	çağıran	çağıran
`char s[50]; return s;`	yığın	**geçersiz!** (Bölüm 9)

Kural: Fonksiyon "yeni blok döndürür" diyorsa onu çağıran `free` eder. Fonksiyon içinde `free(a)` yapıp `return a` yazarsan sarkan işaretçi döndürürsün.

10.8 Dinamik string

String, sonu `'\0'` ile biten `char` dizisidir (Bölüm 8). Öbekte string tutmak için `strlen(s) + 1` bayt ayırırsın: **`+1` sondaki `'\0'` içindir**. `sizeof(char)` her zaman 1 olduğundan çarpmaya gerek yoktur.

kopyaOlustur.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// kaynak'ın öbekte yeni bir kopyasını döndürür (strdup benzeri).
// Hata olursa NULL. Dönen belleği ÇAĞIRAN free etmelidir.
char *kopyaOlustur(const char *kaynak) {
    char *kopya = malloc(strlen(kaynak) + 1);   // +1: '\0' için

    if (kopya == NULL) {
        return NULL;
    }
    strcpy(kopya, kaynak);
    return kopya;
}

int main(void) {
    const char *sabit = "Merhaba";              // string sabiti: değiştirilemez
    char *kopya = kopyaOlustur(sabit);

    if (kopya == NULL) {
        return 1;
    }
    kopya[0] = 'm';                             // kopya öbekte: değiştirilebilir
    printf("sabit: %s\n", sabit);
    printf("kopya: %s\n", kopya);
    printf("uzunluk: %zu, ayrilan: %zu bayt\n", strlen(kopya), strlen(kopya) + 1);
    free(kopya);
    return 0;
}
```

Çıktı

```
sabit: Merhaba
kopya: merhaba
uzunluk: 7, ayrilan: 8 bayt
```

İki kazanç: kopya, orijinal değişse bile **bağımsızdır**; ve sabitin aksine **değiştirilebilir** (Bölüm 9'daki salt okunur string sabitini hatırla). Fonksiyonun `kaynak` parametresini `const char *` yaptık, çünkü okuyoruz, değiştirmiyoruz.

Sık Yapılan Hata: Dinamik stringde +1'i unutmak

```
char *kopya = malloc(strlen(kaynak));   // YANLIŞ: '\0' için yer yok
strcpy(kopya, kaynak);                  // '\0' ayrılan bloğun 1 bayt DIŞINA yazılır
```

`"Merhaba"` için 7 bayt ayrılır ama `strcpy` 8 bayt yazar. Bu **tanımsız davranıştır**: bazen hiçbir şey olmaz, bazen sonraki `free` ya da `malloc` çöker. Test ederken fark edilmez, bu yüzden tehlikelidir. Kural: `malloc(strlen(s) + 1)`. Türkçe karakter varsa `strlen` baytı sayar (Bölüm 8); ayrılan miktar da bayttır, yani tutarlıdır.

C23 Notu: strdup

Bu `kopyaOlustur` fonksiyonunun işini standart kütüphanedeki `strdup` yapar. C17'de `strdup` standart değildir (POSIX sistemlerinde vardır, ama taşınabilir değildir); C23'te `string.h`'ye girdi. Sınavda genelde bu fonksiyonu kendin yazarsın, o yüzden kalıbı iyi öğren.

10.9 2 boyutlu dinamik dizi

Matrisin boyutu da çalışma anında belli olabilir. İki yol var. Birincisi **işaretçi dizisi**: önce satır sayısı kadar `int *` tutan bir dizi (`int **m`, Bölüm 9'daki işaretçinin işaretçisi), sonra her satır için ayrı bir `int` dizisi ayırırsın.

Bütün bu blokların her biri ayrı `malloc`'tır; bu yüzden her birine ayrı `free` gerekir ve **ters sırada** bırakılır: önce satırlar, en son satır işaretçileri. Önce `m`'yi bırakırsan `m[i]` adreslerini okuyacak yer kalmaz.

matris.c

```
#include <stdio.h>
#include <stdlib.h>

int **matrisOlustur(int satir, int sutun) {
    int **m = malloc(satir * sizeof *m);        // satır işaretçileri

    if (m == NULL) {
        return NULL;
    }
    for (int i = 0; i < satir; i++) {
        m[i] = malloc(sutun * sizeof **m);      // i. satırın elemanları
        if (m[i] == NULL) {
            for (int k = 0; k < i; k++) {       // şimdiye kadarkileri geri ver
                free(m[k]);
            }
            free(m);
            return NULL;
        }
    }
    return m;
}

void matrisBirak(int **m, int satir) {
    for (int i = 0; i < satir; i++) {
        free(m[i]);                             // önce satırlar
    }
    free(m);                                    // en son satır işaretçileri
}

int main(void) {
    int satir = 3;
    int sutun = 4;
```

```
    int **m = matrisOlustur(satir, sutun);
    int toplam = 0;

    if (m == NULL) {
        return 1;
    }
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
            m[i][j] = i * sutun + j + 1;
            toplam += m[i][j];
            printf("%3d", m[i][j]);
        }
        printf("\n");
    }
    printf("Toplam: %d\n", toplam);
    matrisBirak(m, satir);
    return 0;
}
```

Çıktı

```
  1  2  3  4
  5  6  7  8
  9 10 11 12
Toplam: 78
```

`m[i][j]` yazımı sabit boyutlu `int m[3][4]` ile aynıdır. Fark, satırların bellekte **peş peşe durmak zorunda olmaması**; bunun bir yararı da satır uzunluklarının farklı olabilmesidir (üçgen matris gibi).

İkinci yol **tek blok**: `satir × sutun` elemanı tek seferde ayırıp `[i][j]` yerine indeksi kendin hesaplarsın: `m[i * sutun + j]`.

```
int *m = malloc(satir * sutun * sizeof *m);   // tek malloc
m[i * sutun + j] = 5;                          // m[i][j] yerine
free(m);                                       // tek free
```

	İşaretçi dizisi (`int **m`)	Tek blok (`int *m`)
`malloc` / `free` sayısı	`satir + 1`	1
Eleman erişimi	`m[i][j]`	`m[i * sutun + j]`
Elemanlar bellekte bitişik mi?	hayır (satırlar ayrı)	evet
Satır uzunlukları farklı olabilir mi?	evet	hayır

10.10 Bellek hataları

Dinamik bellek hataları genellikle **derleyiciyi geçer**. Program çalışır, çoğu zaman doğru sonuç da verir; hata günler sonra, başka bir yerde çöküş olarak ortaya çıkar. Aşağıdaki kodlar bilerek hatalıdır; bunları çalıştırmıyoruz, çünkü sonuçları tanımsızdır (ya da hiçbir belirti vermez).

1. Bellek sızıntısı (memory leak)

```c
void sizinti(void) {
    int *p = malloc(100 * sizeof *p);
    if (p == NULL) {
        return;
    }
    p[0] = 1;
}                       // free(p) yok: p yok olur, 400 baytlık blok öbekte kalır
```

Ayrılan bloğun adresini veren son işaretçi kaybolunca blok sonsuza kadar (program bitene kadar) kullanılamaz kalır. Çalışmayı hemen bozmaz; ama sunucu ya da oyun gibi uzun çalışan programlarda bellek yavaş yavaş tükenir. Program bitince işletim sistemi belleği geri alır, ama "sızıntı var" yine de hatadır. Klasik sızıntı yolları:

- `free` yazmayı unutmak (yukarıdaki).
- Adresi kaybetmek: `p = malloc(...); p = malloc(...);`. İlk bloğun adresi `p`'den silindi.
- Erken çıkış: hata durumunda `return` ederken önceden ayrılan blokları `free` etmeyi unutmak.
- `p = realloc(p, ...)` başarısız olunca (yukarıda gördün).

2. free sonrası kullanım (sarkan işaretçi)

```c
int *p = malloc(sizeof *p);
*p = 7;
free(p);
printf("%d\n", *p);     // TANIMSIZ: p artık bizim olmayan bir yeri gösteriyor
```

`free` sonrasında `p`'nin içindeki adres değişmez ama blok artık sana ait değildir; başka bir `malloc` o yeri başkasına verebilir. `*p` eski değeri de verebilir, çöp de, program da çökebilir. Böyle işaretçiye **sarkan işaretçi** (dangling pointer) denir; Bölüm 9'da yerel değişkenin adresini döndürünce de aynısını görmüştün.

3. Çift free ve yanlış adresi free etmek

```c
free(p);
free(p);                // TANIMSIZ: aynı bloğu ikinci kez bırakmak
int x = 5;
free(&x);               // TANIMSIZ: x yığında, öbekte değil
free(p + 1);            // TANIMSIZ: bloğun başı değil, ortası
```

Üçü de öbeğin iç düzenini bozar; çoğu sistemde program çöker, ama garantisi yoktur.

4. Blok sınırını aşmak

```c
int *p = malloc(5 * sizeof *p);
p[5] = 1;               // TANIMSIZ: geçerli indeksler 0..4
```

Bölüm 7'deki dizi taşmasının öbekteki hâlidir. Komşu bloğun bilgilerini bozduğu için hata bazen çok sonra, bir `free`'de patlar.

> **Sınavda Çıkar**
>
> Bellek hatası soruları genelde **kod okuma** olarak gelir ve çalıştırılmaz: "Hangi satır sızıntıya yol açar?", "`free`'den sonra `*p` yazmak ne doğurur?" Çözüm yöntemi: Her `malloc` için bloğa "bu bloğun adresini şu an hangi işaretçiler tutuyor?" diye bak. Blok bırakılmadan önce son işaretçi başka adrese atanırsa ya da yok olursa sızıntı vardır. Bırakıldıktan sonra o bloğa erişim **sarkan işaretçidir**.

Korunma yolu: free(p); p = NULL;

Çift `free` ve sarkan işaretçiden korunmanın en ucuz yolu, `free`'den hemen sonra işaretçiyi `NULL` yapmaktır. `free(NULL)` güvenli olduğundan ikinci `free` zarar vermez; `if (p != NULL)` ile de bloğun hâlâ geçerli olup olmadığına bakabilirsin:

freeNull.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = malloc(sizeof *p);

    if (p == NULL) {
        return 1;
    }
    *p = 7;
    printf("Once : %d\n", *p);
    free(p);
    p = NULL;                               // sarkan işaretçiyi temizle
    if (p == NULL) {
        printf("Sonra: p bos, kullanilmaz\n");
    }
    free(p);                                // free(NULL): güvenli, bir şey yapmaz
    printf("free(NULL) sorun cikarmadi\n");
    return 0;
}
```

Çıktı

```
Once : 7
Sonra: p bos, kullanilmaz
free(NULL) sorun cikarmadi
```

> **İpucu: Her malloc için bir free**
>
> Kod yazarken `malloc` satırını yazdığın anda `free` satırını da (ya da yorum olarak "burada free edilecek") yaz. Fonksiyondan erken çıkan her `return` yolunun da `free`'yi atlamadığına bak. `p = NULL` alışkanlığı yalnızca **o işaretçiyi** korur: `int *q = p;` ile oluşan ikinci işaretçi `free(p)` sonrasında hâlâ sarkan kalır.

10.11 Bellek hatalarını bulan araçlar

Bellek hataları gözle bulmak zordur; bunun için araçlar var. Hepsi için programı **hata ayıklama bilgisiyle** (`-g`) derlersen rapor satır numarasını da gösterir:

Araç	Platform	Kullanım
Valgrind	Linux	`valgrind --leak-check=full ./program`
Dr. Memory	Windows	`drmemory -- program.exe`
AddressSanitizer	gcc ve clang	derlerken `-fsanitize=address`

```
gcc -std=c17 -g -fsanitize=address program.c -o program
./program
```

AddressSanitizer, derleyicinin programa eklediği denetimlerle taşma, `free` sonrası kullanım ve çift `free` hatalarını çalışma anında yakalar ve hatanın bulunduğu satırı yazar. Sızıntı raporu Linux'ta gelir. Windows'ta MinGW gcc çoğu kurulumda AddressSanitizer içermez; orada Dr. Memory kullan. macOS'ta (özellikle Apple işlemcilerde) Valgrind çalışmaz; `-fsanitize=address` çalışır, sızıntı için `leaks --atExit -- ./program` kullanılabilir. Valgrind sızıntıyı `definitely lost` ("kesin kaybolmuş", hiçbir işaretçinin göstermediği blok) ifadesiyle bildirir. Bu araçlar sınavda sorulmaz ama laboratuvarda ve ödevlerde çok işine yarar; teslimden önce bir kez çalıştır.

10.12 sizeof ve ayrılan boyut

Bir dizinin `sizeof` değeri tüm dizinin bayt sayısıdır (Bölüm 7). `malloc`'la alınan bloğu **gösteren işaretçi** için ise `sizeof` yalnızca işaretçinin kendi boyutunu verir:

sizeofIsaretci.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int sabit[10];
    int *dinamik = malloc(10 * sizeof *dinamik);

    if (dinamik == NULL) {
        return 1;
    }
    printf("sizeof(sabit)   = %zu\n", sizeof(sabit));
    printf("sizeof(dinamik) = %zu\n", sizeof(dinamik));
    free(dinamik);
    return 0;
}
```

Çıktı

```
sizeof(sabit)   = 40
sizeof(dinamik) = 8
```

64 bitlik sistemlerde bir işaretçi 8 bayttır (Bölüm 9), 32 bitlikte 4'tür; yani `sizeof(dinamik)` bloğun 40 baytını **vermez**. Bloğun büyüklüğünü sonradan işaretçiden öğrenemezsin; **eleman sayısını ayrı bir değişkende tut** (`n`, `adet`, `kapasite`) ve diziyi fonksiyona geçirirken boyutu da geçir (Bölüm 7). Bölüm 11'de diziyi ve eleman sayısını tek bir `struct` içinde birlikte taşımayı göreceksin.

Bölüm Özeti – Bilmen Gerekenler

- Sabit dizi boyutu derleme anında belli olmalı; boyut çalışma anında belliyse **dinamik bellek** kullan. VLA yığında yer aldığı için riskli, kitapta kullanılmaz.
- Bellek bölgeleri: **statik alan** (global, `static`), **yığın/stack** (yerel değişkenler, otomatik), **öbek/heap** (`malloc` ile, elle yönetilir).
- `malloc(n * sizeof *p)`: `n` eleman için yer; içi **çöp**. Dönen `void *` C'de cast istemez. **Hemen `NULL` kontrol et.** Her `malloc` için bir `free`.
- `calloc(adet, boyut)`: yeri **sıfırlayarak** ayırır. `malloc(25 * sizeof(int))` = 100 bayt.
- `realloc`: bloğu büyütür/küçültür, eski içeriği korur, adres değişebilir, yeni kısım çöptür. `p = realloc(p, ...)` yazma; **geçici işaretçi** kullan. Büyüyen dizide kapasiteyi ikiye kat.
- Fonksiyon öbekte blok döndürebilir (yerel diziden farklı); bloğu **çağıran** `free` eder (sahiplik). Fonksiyon içinde `free` edilen adres döndürülmez.
- Dinamik string: `malloc(strlen(s) + 1)` (`+1` unutulursa taşar), sonra `strcpy`.
- 2B dinamik dizi: `int **m`, satır satır `malloc`; `free` **ters sırada** (önce satırlar, sonra `m`). Alternatif: tek blok, `m[i * sutun + j]`.
- Hatalar: **sızıntı** (adres kaybı, `free` unutmak), **sarkan işaretçi** (`free` sonrası kullanım), **çift `free`**, öbekte olmayan/başı olmayan adresi `free`, blok sınırını aşmak. Hepsi tanımsız ya da sinsi hatadır.
- `free(NULL)` güvenlidir. `free(p); p = NULL;` alışkanlığı kazan.
- `sizeof(p)` bloğun boyutunu vermez, işaretçinin boyutunu verir (8 bayt); boyutu ayrıca tut.
- Araçlar: Valgrind (Linux), Dr. Memory (Windows), `-fsanitize=address` (gcc/clang).

Sorular

A) Çoktan Seçmeli

1 `int` 4 bayt olduğuna göre `int *p = malloc(25 * sizeof(int));` satırı öbekten kaç bayt ister?

(A) 25 (B) 29 (C) 100 (D) 104 (E) 8

2 Aşağıdakilerden hangisi `malloc` ile `calloc` arasındaki farkı doğru anlatır?

(A) `malloc` ayrılan baytları sıfırlar, `calloc` sıfırlamaz.

(B) `calloc(adet, boyut)` ayırdığı baytların hepsini 0 yapar; `malloc` içeriği olduğu gibi (çöp) bırakır.

(C) `malloc` öbekten, `calloc` yığından yer ayırır.

(D) `calloc` ile ayrılan bloğa `free` uygulanmaz.

(E) `malloc` yer bulamazsa `NULL` döndürür, `calloc` yer bulamazsa programı sonlandırır.

3 Aşağıdaki kod parçası çalışıp bittiğinde öbekte ne kalır?

```
int *a = malloc(10 * sizeof *a);
int *b = malloc(10 * sizeof *b);
a = b;
free(a);
```

(A) Hiçbir şey; iki blok da serbest bırakıldı.

(B) Bir blok sızar: ikinci `malloc`'ın bloğu, çünkü `b` doğrudan `free` edilmedi.

(C) İki blok sızar: `free(a)` yalnız işaretçiyi siler, blokları değil.

(D) Bir blok sızar, ama hangisi olduğu belirsizdir.

(E) Bir blok sızar: ilk `malloc`'ın bloğu, çünkü `a = b` ile adresi kayboldu.

4 `p = realloc(p, yeniBoyut);` yazımının asıl sorunu nedir?

(A) `realloc` başarısız olup `NULL` döndürürse `p`'nin eski adresi kaybolur; eski blok sızar.

(B) `realloc` her zaman yeni bir yere taşır, bu yüzden eski veriler silinir.

(C) `realloc` yalnızca bloğu küçültmek için kullanılabilir.

(D) `realloc`'un döndürdüğü adres `p`'ye atanmamalıdır; `p` kendiliğinden güncellenir.

(E) `realloc` yeni eklenen elemanları sıfırladığı için programı gereksiz yavaşlatır.

5 Dinamik bellekle ilgili aşağıdakilerden hangisi **doğrudur**?

(A) `free(p)` çağrısından sonra `p` kendiliğinden `NULL` olur.

(B) `malloc`'ın döndürdüğü adres C'de mutlaka `(int *)` ile cast edilmek zorundadır.

(C) `malloc` ile ayrılan bloğun bayt sayısı `sizeof(p)` ile öğrenilir.

(D) `free(NULL)` çağrısı güvenlidir; hiçbir şey yapmaz.

(E) Aynı adrese ikinci kez `free` yazmak güvenlidir, çünkü blok zaten serbesttir.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = calloc(4, sizeof *p);
    int toplam = 0;

    if (p == NULL) {
        return 1;
    }
    p[1] = 5;
    p[3] = p[1] * 2;
    for (int i = 0; i < 4; i++) {
        toplam += p[i];
    }
    int *yeni = realloc(p, 6 * sizeof *yeni);
    if (yeni == NULL) {
        free(p);
        return 1;
    }
    p = yeni;
    p[4] = toplam;
    p[5] = p[4] - p[3];
    printf("%d\n", toplam);
    for (int i = 0; i < 6; i++) {
        printf("%d ", p[i]);
    }
    printf("\n");
    free(p);
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>
#include <stdlib.h>

int *kopyala(const int *a, int n) {
    int *k = malloc(n * sizeof *k);

    if (k == NULL) {
        return NULL;
    }
    for (int i = 0; i < n; i++) {
        k[i] = a[n - 1 - i] * 2;
    }
    return k;
}

int main(void) {
    int a[4] = {1, 2, 3, 4};
    int *k = kopyala(a, 4);

    if (k == NULL) {
        return 1;
    }
    a[0] = 100;
    for (int i = 0; i < 4; i++) {
        printf("%d ", k[i]);
    }
    printf("\n%d %d\n", a[0], k[3]);
    free(k);
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır? (Kısalık için satırların `calloc` kontrolleri yazılmamıştır.)

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int satir = 4;
    int adet = 0;
    int toplam = 0;
    int **m = malloc(satir * sizeof *m);

    if (m == NULL) {
        return 1;
    }
    for (int i = 0; i < satir; i++) {
        m[i] = calloc(i + 1, sizeof **m);
        for (int j = 0; j <= i; j++) {
            m[i][j] = i + j;
            adet++;
            toplam += m[i][j];
        }
    }
    printf("%d %d\n", adet, toplam);
    printf("%d %d\n", m[3][2], m[2][1]);
    for (int i = 0; i < satir; i++) {
        free(m[i]);
    }
    free(m);
    return 0;
}
```

C) Kod Yazma

9 Klavyeden **0 girilene kadar** tamsayı okuyan `ortalamaUstu` programını yaz (kaç sayı gireceği belli değil; 0 diziye eklenmez). Sayıları `malloc` ile 2 elemanlık başlayan, dolunca `realloc` ile kapasitesi ikiye katlanan dinamik dizide sakla. Okuma bitince sayıların **ortalamasını** ve ortalamadan **büyük** olanların adedini yazdır. Hiç sayı girilmediyse `Sayi girilmedi.` yazıp çık. Bellek ayırma hatalarını kontrol et ve diziyi serbest bırak.

```
Sayilari gir (0 ile biter): 10 20 30 40 50 60 0
Ortalama: 35.00
Ortalamadan buyuk: 3
```

10 Ad ve soyadı okuyup **yeni bir string** olarak birleştiren `birlestir` programını yaz. `char *birlestir(const char *ad, const char *soyad)` fonksiyonu sonucu (`ad`, bir boşluk, `soyad`) `malloc` ile **tam gereken kadar** yer ayırarak oluştursun; yer yoksa `NULL` döndürsün. `main` sonucu ve uzunluğunu yazdırsın, sonra belleği serbest bıraksın. Ad ve soyad en fazla 49 karakterdir ve boşluk içermez.

```
Ad Soyad: Elif Demir
Tam ad: Elif Demir (10 karakter)
```

Çözümler

Cevap anahtarı

1 C	2 B	3 E	4 A	5 D

1 Cevap: C

Bayt sayısı = eleman sayısı × eleman boyutu = 25 × 4 = **100**. A yalnızca eleman sayısıdır. B ve D anlamsız toplamlardır (25 + 4, 100 + 4). E, bir işaretçinin (`p`) boyutudur; `malloc` ise `p` için değil, gösterdiği blok için yer ayırır.

2 Cevap: B

`calloc` ayırdığı bloğun tüm baytlarını 0 yapar, `malloc` içeriğe dokunmaz (çöp). A tersini söylüyor. C yanlış: ikisi de **öbekten** ayırır. D yanlış: `calloc` ile alınan blok da `free` ile bırakılır. E yanlış: ikisi de yer bulamazsa `NULL` döndürür.

3 Cevap: E

İlk `malloc` bloğunun adresi `a`'da, ikincisininki `b`'dedir. `a = b;` ile `a`'nın eski değeri (birinci bloğun adresi) **silinir**; başka hiçbir işaretçi o bloğu göstermediği için ona bir daha ulaşılamaz: **sızıntı**. Sonra `free(a)` ikinci bloğu bırakır (`a` ve `b` aynı bloğu gösterir), yani ikinci blok sızmaz (B yanlış). Bu arada `b` sarkan işaretçi olur ama kullanılmadığı için sorun yoktur. A yanlış, ilk blok bırakılmadı. C yanlış, `free(a)` bloğu gerçekten bırakır. D yanlış, hangisinin sızdığı kesin belli.

4 Cevap: A

`realloc` başarısız olunca `NULL` döndürür ve eski bloğa dokunmaz. Sonucu doğrudan `p`'ye atarsan `p` `NULL` olur ve eski bloğun adresi kaybolur: sızıntı. Doğrusu geçici işaretçidir. B yanlış, eski veri korunur. C yanlış, büyütmek `realloc`'ın asıl işidir. D yanlış, `p` kendiliğinden güncellenmez; adresin değişmiş olabileceği için sonucu atamak gerekir. E yanlış, `realloc` yeni kısmı sıfırlamaz, çöp bırakır.

5 Cevap: D

`free(NULL)` bir şey yapmaz ve güvenlidir; bu yüzden `free(p); p = NULL;` alışkanlığı çift `free`'yi zararsız yapar. A yanlış: `free` `p`'nin değerini değiştirmez, `NULL` yapmayı sen yazarsın. B yanlış: C'de cast gerekmez (C++'ta gerekir). C yanlış: `sizeof(p)` işaretçinin boyutudur (8 bayt), bloğun değil. E yanlış: çift `free` tanımsız davranıştır ve çoğu zaman çökmeyle sonuçlanır.

6

Adım adım izle:

1. `calloc(4, ...)` ile `p = {0, 0, 0, 0}`.
2. `p[1] = 5` → `{0, 5, 0, 0}`; `p[3] = p[1] * 2 = 10` → `{0, 5, 0, 10}`.
3. `toplam = 0 + 5 + 0 + 10 = 15`.
4. `realloc` 6 elemana büyütür; ilk 4 eleman korunur. `p[4] = toplam = 15`, `p[5] = p[4] - p[3] = 15 - 10 = 5`.
5. İlk satır `toplam`'ı (15), ikinci satır 6 elemanı yazar. Satırın sonunda her sayıdan sonra bir boşluk vardır.

Çıktı
```
15
0 5 0 10 15 5
```

7

Tuzak: Kopya `kopyala` çağrılırken alınır, sonradan `a[0] = 100` yapmak `k`'yi etkilemez (`k` öbekte ayrı bir bloktur).

1. `k[i] = a[3 − i] ∗ 2`: `k[0] = a[3] ∗ 2 = 8`, `k[1] = a[2] ∗ 2 = 6`, `k[2] = a[1] ∗ 2 = 4`, `k[3] = a[0] ∗ 2 = 2` (`a[0]` o anda hâlâ 1).
2. `a[0] = 100` yalnız `a`'yı değiştirir.
3. Döngü `8 6 4 2` yazar. Son satır `a[0]` (100) ve `k[3]` (2) yazar.

Çıktı
```
8 6 4 2
100 2
```

8

`i`. satırda `i + 1` eleman vardır ve `m[i][j] = i + j` yazılır:

i	satırın elemanları	satır toplamı
0	0	0
1	1, 2	3
2	2, 3, 4	9
3	3, 4, 5, 6	18

Eleman sayısı `1+2+3+4 = 10`, toplam `0+3+9+18 = 30`. `m[3][2] = 3 + 2 = 5`, `m[2][1] = 2 + 1 = 3`.

Çıktı
```
10 30
5 3
```

9

Diziyi 2 elemanla başlatıp `adet == kapasite` olunca ikiye katlıyoruz; `realloc` sonucunu geçici işaretçiye alıyoruz. Ortalama `double` hesaplanır. Ortalamadan büyük olanları saymak için sayıları **saklamak** zorundayız, bu yüzden dizi gerekir.

ortalamaUstu.c
```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int kapasite = 2;
    int adet = 0;
    int *sayilar = malloc(kapasite * sizeof *sayilar);
    int x = 0;

    if (sayilar == NULL) {
        printf("Bellek ayrilamadi.\n");
        return 1;
    }
```

```
    printf("Sayilari gir (0 ile biter): ");
    while (scanf("%d", &x) == 1 && x != 0) {
        if (adet == kapasite) {
            int *yeni = realloc(sayilar, 2 * kapasite * sizeof *yeni);
            if (yeni == NULL) {
                printf("Bellek ayrilamadi.\n");
                free(sayilar);
                return 1;
            }
            sayilar = yeni;
            kapasite *= 2;
        }
        sayilar[adet++] = x;
    }
    if (adet == 0) {
        printf("Sayi girilmedi.\n");
        free(sayilar);
        return 0;
    }
    int toplam = 0;
    for (int i = 0; i < adet; i++) {
        toplam += sayilar[i];
    }
    double ortalama = (double)toplam / adet;
    int ustunde = 0;
    for (int i = 0; i < adet; i++) {
        if (sayilar[i] > ortalama) {
            ustunde++;
        }
    }
    printf("Ortalama: %.2f\n", ortalama);
    printf("Ortalamadan buyuk: %d\n", ustunde);
    free(sayilar);
    return 0;
}
```

Çıktı

```
Sayilari gir (0 ile biter): 10 20 30 40 50 60 0
Ortalama: 35.00
Ortalamadan buyuk: 3
```

Toplam 210, 6 sayı, ortalama 35. 40, 50 ve 60 ortalamadan büyüktür: 3 sayı.

10

Gereken bayt sayısı: `strlen(ad) + 1 (boşluk) + strlen(soyad) + 1 ('\0')`. Önce `strcpy` ile adı yazıyoruz, sonra boşluğu ve `strcat` ile soyadı ekliyoruz. Bloğu `birlestir` ayırıyor, sahiplik çağırana geçiyor: `free`'yi `main` yazar.

birlestir.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// ad + " " + soyad için yeni blok döndürür; hata olursa NULL.
// Dönen belleği ÇAĞIRAN free etmelidir.
char *birlestir(const char *ad, const char *soyad) {
    char *sonuc = malloc(strlen(ad) + 1 + strlen(soyad) + 1);

    if (sonuc == NULL) {
        return NULL;
```

```
    }
    strcpy(sonuc, ad);
    strcat(sonuc, " ");
    strcat(sonuc, soyad);
    return sonuc;
}

int main(void) {
    char ad[50];
    char soyad[50];

    printf("Ad Soyad: ");
    if (scanf("%49s %49s", ad, soyad) != 2) {
        return 1;
    }
    char *tamAd = birlestir(ad, soyad);
    if (tamAd == NULL) {
        printf("Bellek ayrilamadi.\n");
        return 1;
    }
    printf("Tam ad: %s (%zu karakter)\n", tamAd, strlen(tamAd));
    free(tamAd);
    return 0;
}
```

Çıktı

```
Ad Soyad: Elif Demir
Tam ad: Elif Demir (10 karakter)
```

BÖLÜM 11

Yapılar: struct, union ve enum

Bu Bölümde Öğreneceklerin

- Paralel diziler yerine neden tek bir kayıt (`struct`) kullanıldığını
- `struct` tanımlamayı, `.` ile üyeye erişmeyi, ilk değer vermeyi ve `typedef` ile kısa ad vermeyi
- Struct atamasının **tüm üyeleri kopyaladığını** (dizi üyesi dahil) ve `==` ile karşılaştırılamadığını
- Struct dizisiyle kayıt listesi okumayı, aramayı ve sıralamayı (kabarcık ve `qsort`)
- Fonksiyona struct geçirmeyi (değer ve işaretçi), `->` operatörünü, `const Ogrenci *` ve struct döndürmeyi
- İç içe struct'ı, `malloc` ile dinamik struct dizisini, `sizeof` ve hizalamayı (padding)
- `union` ile etiketli union'ı; `enum` ile adlandırılmış sabitleri

11.1 Neden struct?

Proje: Öğrenci Not Sistemi'nde bir öğrencinin bilgisini üç ayrı dizide tutmuştun: `ad`, `no`, `ortalama`. Öğrenci `i`'nin bilgisi üç dizinin `i`. gözündedir; buna **paralel diziler** (parallel arrays) denir. Sorun, öğrenciyle ilgili her işlemin üç diziye birden dokunmasıdır. İki öğrencinin yerini değiştirmek bile üç ayrı takas ister: adlar için `strcpy` ve geçici bir string, numaralar için geçici bir `int`, ortalamalar için geçici bir `double`. Birini unutursan Ayse'nin adı başka öğrencinin numarasıyla kalır ve derleyici bunu fark etmez. Bir fonksiyona öğrenci geçirmek de üç parametre demektir. **Yapı** (`struct`), farklı tipteki verileri tek bir tipte toplar: öğrenci tek bir değişken olur, takas tek satırdır. Bu bölümde yapıyı, iki kardeşini (`union`, `enum`) ve yapıların diziler, fonksiyonlar, işaretçiler ve dinamik bellekle kullanımını öğreneceksin.

11.2 struct tanımı ve üye erişimi

Yapı iki adımda kullanılır: önce yeni **tipi tanıtırsın**, sonra o tipten **değişken** tanımlarsın.

ilkYapi.c

```c
#include <stdio.h>
#include <string.h>

struct Ogrenci {
    char ad[30];
    int no;
    double ortalama;
};

int main(void) {
    struct Ogrenci a;
    strcpy(a.ad, "Ali Yilmaz");
    a.no = 101;
    a.ortalama = 78.5;

    struct Ogrenci b = {"Ayse Demir", 102, 91.25};
    struct Ogrenci c = {.no = 103, .ad = "Can Kaya"};

    printf("%s %d %.2f\n", a.ad, a.no, a.ortalama);
```

```
    printf("%s %d %.2f\n", b.ad, b.no, b.ortalama);
    printf("%s %d %.2f\n", c.ad, c.no, c.ortalama);
    return 0;
}
```

Çıktı

```
Ali Yilmaz 101 78.50
Ayse Demir 102 91.25
Can Kaya 103 0.00
```

Kod	Anlamı
`struct Ogrenci { ... };`	Yeni bir tip tanıtır; adı **`struct Ogrenci`** (iki sözcük). **Bellek ayırmaz**; içindekiler yapının **üyeleridir** (members)
`struct Ogrenci a;`	Değişken tanımlar; **bellek şimdi ayrılır**, üç üyenin hepsi içindedir
`a.no = 101;`	**Nokta operatörü** `.` ile üyeye erişir: "`a`'nın `no`'su"
`{"Ayse Demir", 102, 91.25}`	İlk değer listesi: değerler üyelerin **tanım sırasıyla** eşleşir
`{.no = 103, .ad = "Can Kaya"}`	**Atanmış ilk değer** (designated initializer, C99): üyeyi adıyla seçersin, sıra önemsizdir; verilmeyen üye **sıfırlanır**

Üye, kendi tipinin sıradan bir değişkenidir: `scanf("%d", &a.no)`, `a.no++` geçerlidir; dizi üyesi için `&` gerekmez (`scanf("%29s", a.ad)`). Konumla verilen listeyi eksik bırakırsan (`{"Ali"}`) `-Wextra` "missing initializer" uyarısı verir; eksik bırakacaksan `.üye = değer` yaz. Hepsini sıfırlamak için `struct Ogrenci bos = {0};`.

Tanımın sonundaki `;` **zorunludur**. Unutursan gcc hatayı `}`'nin yanında değil **sonraki** satırda bildirir (`struct Nokta {...} int main(void)` "bu yapı tipinden `main` adlı bir şey tanımlıyorsun" gibi okunur):

structNokta.c

```c
#include <stdio.h>

struct Nokta {
    int x;
    int y;
}

int main(void) {
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
structNokta.c:8:1: error: expected ';', identifier or '(' before 'int'
    8 | int main(void) {
      | ^~~
```

Hata mesajı `int main` satırını gösteriyorsa bir üst satırdaki yapının `}` işaretinden sonra `;` olup olmadığına bak.

11.3 typedef ile kısa ad

Her yerde `struct Ogrenci` yazmak yorucudur. **`typedef`** mevcut bir tipe ikinci bir ad verir. Yapıyla üç kalıp görürsün:

Kalıp	Değişken tanımı
`struct Ogrenci { ... };`	`struct Ogrenci a;`
`typedef struct { ... } Ogrenci;`	`Ogrenci a;` (yapının etiketi yok, tek adı `Ogrenci`)
`typedef struct Ogrenci { ... } Ogrenci;`	`Ogrenci a;` **ve** `struct Ogrenci a;`

Kitapta çoğunlukla ikinci kalıbı kullanacağız. Üçüncüsü, yapının **kendi türünden bir işaretçi** taşıması gerektiğinde şarttır (bağlı liste, Bölüm 14). Tip adları büyük harfle başlar (`Ogrenci`); `ogrenci` değişkeniyle karışmaz.

● Sık Yapılan Hata: Etiketsiz typedef'ten sonra struct yazmak

`typedef struct { ... } Nokta;` yapının **etiketi yoktur**, yalnızca `Nokta` adı vardır. Sonradan `struct Nokta c;` yazarsan C, `struct Nokta` adlı **başka, tanımsız** bir tip arar:

yanlisEtiket.c

```c
typedef struct {
    int x;
    int y;
} Nokta;

int main(void) {
    struct Nokta c;
    c.x = 1;
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
yanlisEtiket.c: In function 'main':
yanlisEtiket.c:7:18: error: storage size of 'c' isn't known
    7 |     struct Nokta c;
      |                  ^
yanlisEtiket.c:7:18: warning: unused variable 'c' [-Wunused-variable]
```

`unused variable` uyarısı asıl hatanın yan etkisidir. Çözüm: `Nokta c;` yaz ya da yapıya etiket ver (`typedef struct Nokta {...} Nokta;`).

11.4 Atama ve karşılaştırma

Aynı tipte iki yapı `=` ile atanabilir ve atama **bütün üyeleri** kopyalar. Bölüm 7'de dizilerin `=` ile kopyalanamadığını görmüştün; yapının **içindeki** dizi üyesi ise kopyalanır:

yapiAtama.c

```c
#include <stdio.h>
#include <string.h>

typedef struct {
    char ad[30];
    int no;
} Ogrenci;

int main(void) {
    Ogrenci a = {"Ali", 101};
    Ogrenci b = a;                // b, a'nın üye üye kopyası

    strcpy(b.ad, "Veli");
    b.no = 102;

    printf("a: %s %d\n", a.ad, a.no);
    printf("b: %s %d\n", b.ad, b.no);
    return 0;
}
```

Çıktı

```
a: Ali 101
b: Veli 102
```

`b`, `a`'nın **bağımsız bir kopyasıdır**; `b.ad`'ı değiştirmek `a.ad`'ı etkilemez.

İki yapıyı `==` ile karşılaştıramazsın: C bu işlemi tanımlamamıştır, üyeleri (string üyeleri `strcmp` ile) tek tek kıyaslarsın. String üyeye `=` ile metin de atanamaz; Bölüm 8'deki kural aynen geçerli:

yapiHatalari.c

```c
typedef struct { char ad[30]; int no; } Ogrenci;

int main(void) {
    Ogrenci a = {"Ali", 101};
    Ogrenci b = {"Ali", 101};

    if (a == b) {
        a.no = 0;
    }
    a.ad = "Veli";
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
yapiHatalari.c: In function 'main':
yapiHatalari.c:7:11: error: invalid operands to binary == (have 'Ogrenci' and 'Ogrenci')
    7 |     if (a == b) {
      |           ^~
yapiHatalari.c:10:10: error: assignment to expression with array type
   10 |     a.ad = "Veli";
      |          ^
```

Doğrusu: `strcmp(a.ad, b.ad) == 0 && a.no == b.no` ile karşılaştırmak ve `strcpy(a.ad, "Veli")` ile kopyalamaktır.

> **Sınavda Çıkar**
>
> "Aşağıdaki kodun çıktısı nedir?" sorularında `Ogrenci b = a;` gördüğünde hatırla: **yapı ataması üye üye kopyadır**, içindeki diziler de kopyalanır; sonradan `b` üzerindeki değişiklik `a`'ya yansımaz. Dizilerin `=` ile kopyalanamaması kuralı yapı **üyesi** olan diziyi bağlamaz. İşaretçi üyeler ise yalnız **adresi** kopyalar (aynı belleği paylaşırlar); bunu Bölüm 14'te göreceksin.

11.5 Struct dizisi

Asıl güç, yapıları diziye dizdiğinde ortaya çıkar. `Ogrenci liste[50];` elli kayıt tutar; kayıt `liste[i]`, üyesi `liste[i].no` ile erişilir. Program öğrencileri okur, tablo yazar, en yüksek ortalamayı bulur ve ada göre arar (diziyi fonksiyona geçirmeyi Bölüm 7'de gördün):

ogrenciListe.c

```c
#include <stdio.h>
#include <string.h>

#define MAX 50

typedef struct { char ad[30]; int no; double ortalama; } Ogrenci;

int enYuksek(const Ogrenci liste[], int n) {
    int enIyi = 0;
    for (int i = 1; i < n; i++) {
        if (liste[i].ortalama > liste[enIyi].ortalama) {
            enIyi = i;
        }
    }
    return enIyi;
}

int adaGoreBul(const Ogrenci liste[], int n, const char *aranan) {
    for (int i = 0; i < n; i++) {
        if (strcmp(liste[i].ad, aranan) == 0) {
            return i;
        }
    }
    return -1;
}

int main(void) {
    Ogrenci liste[MAX];
    int n = 0;
    char aranan[30];

    if (scanf("%d", &n) != 1 || n < 1 || n > MAX) {
        return 1;
    }
    for (int i = 0; i < n; i++) {
        scanf("%29s %d %lf", liste[i].ad, &liste[i].no, &liste[i].ortalama);
    }
    scanf("%29s", aranan);

    for (int i = 0; i < n; i++) {
        printf("%d %-6s %6.2f\n", liste[i].no, liste[i].ad, liste[i].ortalama);
    }
    int en = enYuksek(liste, n);
    printf("En yüksek: %s (%.2f)\n", liste[en].ad, liste[en].ortalama);

    int yer = adaGoreBul(liste, n, aranan);
    if (yer >= 0) {
```

```
        printf("%s bulundu: no %d\n", aranan, liste[yer].no);
    } else {
        printf("%s listede yok\n", aranan);
    }
    return 0;
}
```

Çıktı

```
4
Ali 101 72.5
Ayse 102 91.25
Can 103 64
Deniz 104 85.75
Can
101 Ali     72.50
102 Ayse    91.25
103 Can     64.00
104 Deniz   85.75
En yüksek: Ayse (91.25)
Can bulundu: no 103
```

`liste[i].ad` için `&` yok (dizi adı zaten adrestir), `liste[i].no` için `&` var. `liste[i]` bir **yapıdır**, işaretçi değildir; bu yüzden `.` kullanırsın (`liste[i]->ad` derlenmez). Arama ve en büyüğü bulma algoritmaları Bölüm 7'dekiyle aynıdır; yalnızca karşılaştırdığın şey `liste[i].ortalama` ya da `strcmp(liste[i].ad, ...)` olur.

Kabarcık sıralaması

Kabarcık algoritması Bölüm 7'dekiyle aynıdır; fark, karşılaştırdığın üyenin `.ortalama` olması ve takasın **bütün yapıyı** tek seferde taşımasıdır. Paralel dizilerdeki üç ayrı takas, tek bir yapı takasına indi:

kabarcikYapi.c

```c
#include <stdio.h>

typedef struct { char ad[20]; int no; double ortalama; } Ogrenci;

int main(void) {
    Ogrenci liste[5] = {
        {"Ali", 101, 72.5}, {"Ayse", 102, 91.25}, {"Can", 103, 64.0},
        {"Deniz", 104, 85.75}, {"Efe", 105, 91.25}
    };
    int n = 5;

    for (int tur = 0; tur < n - 1; tur++) {         // büyükten küçüğe
        for (int j = 0; j < n - 1 - tur; j++) {
            if (liste[j].ortalama < liste[j + 1].ortalama) {
                Ogrenci gecici = liste[j];
                liste[j] = liste[j + 1];
                liste[j + 1] = gecici;
            }
        }
    }
    for (int i = 0; i < n; i++) {
        printf("%d. %-6s %.2f\n", i + 1, liste[i].ad, liste[i].ortalama);
    }
    return 0;
}
```

```
Çıktı
1. Ayse   91.25
2. Efe    91.25
3. Deniz  85.75
4. Ali    72.50
5. Can    64.00
```

Ayse ile Efe'nin ortalaması eşit; eşitlikte takas yapmadığımız (`<`) için aralarındaki sıra korunur. Buna kararlılık (stability) denir.

> **İpucu: Yapıyı tek parça takas et**
>
> Üyeleri tek tek takas etmeye kalkma: biri eksik kalırsa kayıt bozulur. `Ogrenci gecici = liste[j];` bütün kaydı taşır.

11.6 Fonksiyona struct: değer ve işaretçi

C'de her şey **değer ile** geçer (Bölüm 6). Yapıyı doğrudan verirsen **bütün yapı** kopyalanır ve fonksiyon kopya üzerinde çalışır; orijinali değiştirmek için Bölüm 9'daki gibi **adresini** geçirirsin. Fonksiyon yapıyı `return` ile de döndürebilir; yapı kopya olarak döner, bu yüzden yerel değişkenin adresini döndürmenin sorunu (Bölüm 9) burada yoktur:

degerAdres.c

```c
#include <stdio.h>

typedef struct { char ad[20]; int no; double ortalama; } Ogrenci;

Ogrenci yeniOgrenci(const char *ad, int no, double ortalama) {
    Ogrenci o;
    snprintf(o.ad, sizeof o.ad, "%s", ad);
    o.no = no;
    o.ortalama = ortalama;
    return o;                       // kopya olarak döner
}

void zamDeger(Ogrenci o) {
    o.ortalama += 10;
    printf("zamDeger içinde: %.1f\n", o.ortalama);
}

void zamAdres(Ogrenci *p) {
    p->ortalama += 10;
    printf("zamAdres içinde: %.1f\n", p->ortalama);
}

int main(void) {
    Ogrenci a = yeniOgrenci("Ali", 101, 60.0);

    zamDeger(a);
    printf("dışarıda: %.1f\n", a.ortalama);
    zamAdres(&a);
    printf("dışarıda: %.1f\n", a.ortalama);
    return 0;
}
```

```
Çıktı
zamDeger içinde: 70.0
dışarıda: 60.0
zamAdres içinde: 70.0
dışarıda: 70.0
```

`zamDeger` kopyayı değiştirdi, dışarıdaki `a` aynı kaldı; `zamAdres` gerçek `a`'yı değiştirdi. `zamAdres`'teki `p->ortalama`, "`p`'nin gösterdiği yapının `ortalama` üyesi" demektir; yazımı hemen aşağıda açıklıyoruz. `snprintf` Bölüm 8'deki güvenli kopyalamadır: uzun adı keser, taşırmaz.

Ok operatörü

İşaretçi üzerinden üyeye `(*p).no` ile erişilir; parantez şarttır, çünkü `.` operatörü `*`'tan **önce** işlenir. Kalıp çok sık yazıldığı için C kısa yol verir: **ok operatörü** (arrow operator) `->`.

Yazım	Anlamı
`p->no`	`p`'nin gösterdiği yapının `no` üyesi
`(*p).no`	Aynısı, parantezli uzun yazım
`*p.no`	`*(p.no)` demektir: `p` işaretçi olduğu için **derlenmez**

Hangisini kullanacağını soldaki ifade belirler: **yapıysa `.`, yapıya işaretçiyse `->`.**

Örnekler: `a.no` (yapı değişkeni), `liste[i].no` (dizi elemanı yapıdır), `p->no` (işaretçi), `ptrler[i]->no` (işaretçi dizisinin elemanı işaretçidir).

const Ogrenci * ile büyük yapıyı geçirmek

Yapı büyükse her çağrıda kopyalamak israftır (içinde büyük diziler olan yapılar yüzlerce bayt olabilir). Adres geçirmek hızlıdır; fonksiyonun yapıyı **değiştirmeyeceğini** garanti etmek için `const` yaz (Bölüm 9). İki hatayı birlikte görelim:

okVeConst.c

```c
#include <stdio.h>

typedef struct { char ad[20]; int no; } Ogrenci;

void yazdir(const Ogrenci *o) {
    printf("%d %s\n", o->no, o->ad);
    o->no = 0;
}

int main(void) {
    Ogrenci a = {"Ali", 101};
    Ogrenci *p = &a;
    printf("%c\n", *p.ad);
    yazdir(p);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
okVeConst.c: In function 'yazdir':
okVeConst.c:7:11: error: assignment of member 'no' in read-only object
    7 |     o->no = 0;
      |           ^
okVeConst.c: In function 'main':
okVeConst.c:13:22: error: 'p' is a pointer; did you mean to use '->'?
   13 |     printf("%c\n", *p.ad);
      |                     ^
      |                     ->
```

`*p.ad` hatasında `.` önce işlendiği için `p.ad` aranır; doğrusu `p->ad` ya da `(*p).ad`'dır (parantez şart). İkinci hata salt-okunur yapıya yazmaktır. Kural: **yalnız okuyan** fonksiyon `const Ogrenci *`, **değiştiren** fonksiyon `Ogrenci *` alır.

11.7 qsort ile yapı dizisi sıralamak

Bölüm 9'daki `qsort` yapı dizisini de sıralar. Karşılaştırma fonksiyonu iki elemanın **adresini** alır; `void *`'ı `const Ogrenci *`'a atayıp `->` ile üyeye ulaşırsın. Ortalamaya göre büyükten küçüğe, eşitlikte ada göre:

```
int ortalamayaGore(const void *x, const void *y) {
    const Ogrenci *a = x;
    const Ogrenci *b = y;
    if (a->ortalama > b->ortalama) return -1;      // a önce gelsin
    if (a->ortalama < b->ortalama) return 1;
    return strcmp(a->ad, b->ad);                   // eşitlikte ada göre
}
...
qsort(liste, n, sizeof liste[0], ortalamayaGore);
```

`qsort` **kararlı değildir**: eşit elemanların sırası kütüphaneye göre değişebilir (Windows'ta ve Linux'ta farklı çıkabilir). Çıktının her yerde aynı olması için eşitliği ikinci bir anahtarla (burada `ad`) bozarsın.

11.8 İç içe struct

Bir yapının üyesi başka bir yapı olabilir. İç yapıya erişirken noktalar art arda gelir: `o.dogum.yil`. İç ilk değer listesi de iç içe süslü parantez alır.

iclceYapi.c

```
#include <stdio.h>

typedef struct { int gun; int ay; int yil; } Tarih;

typedef struct { char ad[20]; int no; Tarih dogum; } Ogrenci;

int main(void) {
    Ogrenci o = {"Ali", 101, {7, 3, 2005}};
    Ogrenci *p = &o;

    o.dogum.yil = 2004;
    p->dogum.ay = 11;

    printf("%s: %02d.%02d.%d\n", o.ad, o.dogum.gun, o.dogum.ay, o.dogum.yil);
    return 0;
}
```

```
Çıktı
Ali: 07.11.2004
```

Soldan sağa oku: `p->dogum.ay` = "`p`'nin gösterdiği yapının `dogum` üyesinin `ay`'ı". `->` yalnızca ilk adımdır; `dogum` bir yapı olduğu için ikinci adım yine `.` ile atılır. Bir yapı **kendi türünden bir üye** taşıyamaz (boyutu sonsuz olurdu), ama kendi türünden bir **işaretçi** taşıyabilir; bağlı liste böyle kurulur ve Bölüm 14'te göreceksin.

11.9 Struct ve dinamik bellek

`malloc` ile yapı için bellek ayırmak diğer tiplerden farksızdır (Bölüm 10): boyutu `sizeof(Ogrenci)` ya da `sizeof *p` ile verirsin; sonuç `Ogrenci *`'dır. Öğrenci sayısını çalışırken öğrenen program `n * sizeof *liste` ile yapı dizisi ayırır:

dinamikOgrenci.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct { char ad[20]; int no; double ortalama; } Ogrenci;

int main(void) {
    int n = 0;
    if (scanf("%d", &n) != 1 || n < 1) {
        return 1;
    }
    Ogrenci *liste = malloc(n * sizeof *liste);        // n kayıtlık dizi
    if (liste == NULL) {
        return 1;
    }
    double toplam = 0;
    for (int i = 0; i < n; i++) {
        scanf("%19s %d %lf", liste[i].ad, &liste[i].no, &liste[i].ortalama);
        toplam += liste[i].ortalama;
    }
    printf("Ortalama: %.2f\n", toplam / n);
    free(liste);
    return 0;
}
```

```
Çıktı
3
Ali 101 72.5
Ayse 102 91.25
Can 103 64
Ortalama: 75.92
```

Tek bir kayıt için `Ogrenci *tek = malloc(sizeof(Ogrenci));` yazılır ve üyeye `tek->no` ile erişilir. `liste` de bir işaretçidir ama `liste[i]` **yapının kendisidir** (`liste[i]` ≡ `*(liste + i)`): `liste[i].ad`. `sizeof *liste`, tip değişirse bu satıra dokunmanı gerektirmez; `*liste`'yi değerlendirmez, yalnızca tipinin boyutunu alır. Her `malloc` için bir `free` yaz.

11.10 sizeof ve hizalama

Üyelerin boyutlarını toplayıp yapının boyutunu bulmaya çalışırsan yanılırsın: derleyici üyeler arasına **boşluk baytları** (padding) koyar. İşlemci, adresi 4'ün katı olan `int`'i ve 8'in katı olan `double`'ı tek hamlede okur. Bu yüzden her üye kendi **hizalama** (alignment) gereksinimine uyan adreste başlar ve yapının toplam boyutu en büyük hizalamanın katına yuvarlanır.

hizalama.c

```c
#include <stdio.h>
#include <stddef.h>

struct A { char c; int i; };
struct B { char c1; int i; char c2; };
struct C { int i; char c1; char c2; };

typedef struct { char ad[30]; int no; double ortalama; } Ogrenci;

int main(void) {
    printf("struct A: %zu bayt, i %zu. bayttan başlar\n",
           sizeof(struct A), offsetof(struct A, i));
    printf("struct B: %zu bayt\n", sizeof(struct B));
    printf("struct C: %zu bayt\n", sizeof(struct C));
    printf("Ogrenci : %zu bayt\n", sizeof(Ogrenci));
    printf("no %zu, ortalama %zu\n", offsetof(Ogrenci, no),
           offsetof(Ogrenci, ortalama));
    return 0;
}
```

Çıktı

```
struct A: 8 bayt, i 4. bayttan başlar
struct B: 12 bayt
struct C: 8 bayt
Ogrenci : 48 bayt
no 32, ortalama 40
```

`offsetof(tip, üye)` (`stddef.h`), üyenin yapının başından kaç bayt sonra başladığını verir. Sayıları elle şöyle bulursun:

- **A**: `c` 0. baytta; `i` 4'ün katında başlamalı, 1-3. baytlar boş kalır: 1 + 3 + 4 = **8**.
- **B**: `c1` (1) + 3 boşluk + `i` (4) + `c2` (1) = 9, en büyük hizalama olan 4'ün katına yuvarlanır: **12**.
- **C**: `i` (4) + `c1` (1) + `c2` (1) = 6, 4'ün katına yuvarlanır: **8**. B ile aynı üyeler, ama büyükten küçüğe dizildi.
- **Ogrenci**: `ad` 0-29, 30-31 boşluk, `no` 32'de, 36-39 boşluk, `ortalama` 40'ta: **48**. Üyelerin toplamı 30 + 4 + 8 = 42'dir.

Bu sayılar bu kitapta kullanılan gcc'de (64 bit: arm64 ve x86-64; Windows'ta MinGW-w64 dahil) böyledir. C standardı yapı boyutunu garanti etmez; 32 bit bir derleyicide `double` 4 bayta hizalanırsa `Ogrenci` 44 bayt olur. Sınav soruları bu yüzden "bu derleyicide" der. Boşluk baytların içeriği belirsizdir; yapıları `memcmp` ile de karşılaştırma, üye üye karşılaştır.

> **Sınavda Çıkar**
>
> Padding hesabı: her üyeyi sırayla koy, kendi hizalamasına uyan adrese getir (gerekirse boşluk ekle), en sonda toplamı **en büyük hizalamanın** katına yuvarla. `struct { char a; double b; char c; }` → 1 + 7 (boşluk) + 8 + 1 + 7 (kuyruk) = **24**; `struct { double b; char a; char c; }` → 8 + 1 + 1 + 6 = **16**. Üyeleri büyükten küçüğe dizmek boşluğu azaltır.

Bit alanları

Bazen bir üyenin tam bir `int` kaplamasına gerek yoktur: gün 1-31 arasıdır, 5 bit yeter. Üye adından sonra `: bitSayısı` yazarsan o üye yalnızca o kadar bit kaplar; buna **bit alanı** (bit-field) denir. Tipi hep `unsigned int` yaz:

bitAlani.c

```c
#include <stdio.h>

struct NormalTarih {
    int gun;
    int ay;
    int yil;
};

struct SikTarih {
    unsigned int gun : 5;          // 0 ... 31
    unsigned int ay : 4;           // 0 ... 15
    unsigned int yil : 12;         // 0 ... 4095
};

int main(void) {
    struct SikTarih t = {9, 10, 2026};
    int fazla = 33;

    printf("%u/%u/%u\n", t.gun, t.ay, t.yil);
    printf("normal: %zu bayt, sık: %zu bayt\n",
           sizeof(struct NormalTarih), sizeof(struct SikTarih));
    t.gun = fazla;                 // 5 bit'e sığmaz
    printf("gun = %u\n", t.gun);
    return 0;
}
```

Çıktı

```
9/10/2026
normal: 12 bayt, sık: 4 bayt
gun = 1
```

- 5 + 4 + 12 = 21 bit tek bir 4 baytlık birime sığar; `sizeof` 12'den 4'e iner (bu kitabın gcc'sinde). Bitlerin hangi sırayla dizildiği derleyiciye bağlıdır, bu yüzden bit alanlı yapıyı olduğu gibi dosyaya ya da ağa yazma.
- Sığmayan değer, genişliğe göre mod alınarak kesilir: 5 bit en çok 31 tutar, `33` yazınca `33 % 32 = 1` kalır.
- `unsigned` yaz. Düz `int` bit alanının işaretli mi işaretsiz mi olduğunu standart derleyiciye bırakır.
- Bit alanının **adresi alınamaz**: bellekte baytlar adreslenir, bitler değil. `&t.gun` da, `scanf("%u", &t.gun)` de derlenmez; önce sıradan bir değişkene oku, sonra ata.

bitAlaniAdres.c

```c
struct Bayrak {
    unsigned int acik : 1;
};

int main(void) {
    struct Bayrak b = {1};
    unsigned int *p = &b.acik;     // hata: bit alanının adresi alınamaz
    return (int)*p;
}
```

Derleyici çıktısı – program derlenmez

```
bitAlaniAdres.c: In function 'main':
bitAlaniAdres.c:7:23: error: cannot take address of bit-field 'acik'
    7 |     unsigned int *p = &b.acik;      // hata: bit alanının adresi alınamaz
      |                       ^
```

11.11 union

`union` yapıya benzer ama üyeleri art arda değil, **aynı bellek alanında üst üste** saklar. Boyutu en büyük üyenin boyutu kadardır (hizalama gerekirse yuvarlanır) ve **aynı anda yalnızca bir üye geçerlidir**: en son yazdığın.

birlesim.c

```c
#include <stdio.h>

union Deger {
    int tam;
    double ondalik;
    char harf;
};

int main(void) {
    union Deger d;

    printf("boyut: %zu bayt\n", sizeof d);

    d.tam = 65;
    printf("tam = %d\n", d.tam);
    d.ondalik = 2.5;                // tam'ın yerini ezer
    printf("ondalik = %.1f\n", d.ondalik);
    d.harf = 'Z';                   // ondalik'in ilk baytını ezer
    printf("harf = %c\n", d.harf);
    return 0;
}
```

Çıktı

```
boyut: 8 bayt
tam = 65
ondalik = 2.5
harf = Z
```

Boyut 8'dir: `int` (4), `double` (8) ve `char` (1) üyelerinin **en büyüğü**, toplamları (13) değil. Her yazma öncekini ezer.

> **Sık Yapılan Hata: Yazılmayan union üyesini okumak**
>
> `d.ondalik = 2.5;` yazıp `d.tam` okursan aynı baytları `int` olarak yorumlarsın: anlamlı bir sayı çıkmaz, sonuç derleyiciye ve işlemciye bağlıdır. "Hangi üye geçerli?" bilgisini kendin saklaman gerekir; standart yol aşağıdaki **etiketli union**'dur.

Etiketli union

Bir değişken bazen tam sayı, bazen ondalıklı sayı taşıyacaksa union'ı bir **etiketle** (tag) birlikte yapıya koyarsın; etiket hangi üyenin geçerli olduğunu söyler. Etiket için `enum` uygundur (hemen sonraki başlık; şimdilik "adlandırılmış tam sayı sabitleri" say):

etiketliBirlesim.c

```c
#include <stdio.h>

typedef enum { TUR_TAM, TUR_ONDALIK } Tur;

typedef struct {
    Tur tur;                        // hangi üye geçerli?
    union {
        int tam;
        double ondalik;
    } deger;
} Sayi;

int main(void) {
    Sayi liste[2] = {
        {TUR_TAM, {.tam = 42}},
        {TUR_ONDALIK, {.ondalik = 3.14159}}
    };

    for (int i = 0; i < 2; i++) {
        switch (liste[i].tur) {
            case TUR_TAM:
                printf("tam: %d\n", liste[i].deger.tam);
                break;
            case TUR_ONDALIK:
                printf("ondalık: %.2f\n", liste[i].deger.ondalik);
                break;
        }
    }
    return 0;
}
```

Çıktı

```
tam: 42
ondalık: 3.14
```

İki farklı tipte değer aynı dizide durdu; `switch` etiketi okuyup yalnızca geçerli üyeye baktı. Etiketli union, diğer dillerdeki “toplam tip” (sum type) fikrinin C’deki karşılığıdır.

11.12 enum

Programlarda “gün”, “renk”, “durum” gibi **sınırlı sayıda seçenek** olur. Bunları `1`, `2`, `3` diye sihirli sayılarla taşımak okunamaz; hepsine `#define` yazmak (Bölüm 2) sıkıcıdır. `enum` (enumeration) bir sabit kümesine ad verir:

gunler.c

```c
#include <stdio.h>

enum Gun { PAZARTESI, SALI, CARSAMBA, PERSEMBE, CUMA, CUMARTESI, PAZAR };

typedef enum {
    DUSUK = 1,
    ORTA,
    YUKSEK = 10,
    COK_YUKSEK
} Seviye;

int main(void) {
    const char *gunAdi[] = {"Pazartesi", "Sali", "Carsamba", "Persembe",
                            "Cuma", "Cumartesi", "Pazar"};
```

```
    enum Gun bugun = CUMARTESI;

    printf("SALI = %d, PAZAR = %d\n", SALI, PAZAR);
    printf("bugun = %d (%s)\n", bugun, gunAdi[bugun]);
    switch (bugun) {
        case CUMARTESI:
        case PAZAR:
            printf("Hafta sonu\n");
            break;
        default:
            printf("Hafta içi\n");
    }

    printf("%d %d %d %d\n", DUSUK, ORTA, YUKSEK, COK_YUKSEK);
    Seviye s = ORTA;
    s = s + 1;                      // enum bir int gibi toplanır
    printf("s = %d, s == YUKSEK? %d\n", s, s == YUKSEK);
    return 0;
}
```

Çıktı

```
SALI = 1, PAZAR = 6
bugun = 5 (Cumartesi)
Hafta sonu
1 2 10 11
s = 3, s == YUKSEK? 0
```

- Sabitler **0'dan başlayarak** birer birer artar: `PAZARTESI` 0, `SALI` 1, … `PAZAR` 6.
- Bir sabite **açık değer** verebilirsin; sonraki sabit, değer verilmediği sürece bir öncekinin **bir fazlası** olur: `ORTA` = 2, `COK_YUKSEK` = 11. İki sabite aynı değer verilebilir.
- Sabitler **`int` sabitidir**: `printf("%d", SALI)` çalışır, dizi indeksi (`gunAdi[bugun]`) ve `case` etiketi olur, toplanır. `s + 1`'den sonra `s` 3'tür; hiçbir sabitle eşleşmez ama C itiraz etmez.
- Sabit adları büyük harfle yazılır ve değişken adlarıyla **aynı alanı** paylaşır; iki farklı `enum` içinde aynı ad (`KIRMIZI`) kullanılamaz.
- Enum değişkeni aslında bir `int` gibidir: C, `enum Gun g = 42;` yazmana da izin verir, aralığı denetlemez. `default` olmayan bir `switch`'te bir sabiti eksik bırakırsan gcc `-Wall` ile `enumeration value 'PAZAR' not handled in switch` uyarısı verir.
- Standart enum'un tipini derleyiciye bırakır (sabitleri `int`'tir); bu kitabın gcc'sinde `int` boyutundadır. Birbirine bağlı sabitler için `enum`, tek tük sabit (`MAX`) için `#define` kullanılır; `enum` sabitlerini derleyici tanır ve `switch` uyarıları verir.

11.13 Uygulama: sınıf kayıt sistemi

Öğrendiklerini küçük bir kayıt sisteminde birleştirelim. Sınıf, en çok `MAX` öğrenci tutan bir dizi ile bir sayaçtan oluşur; ikisi tek yapıda toplanır, böylece fonksiyonlara tek parametre (`Sinif *`) yeter. `ekle` yer varsa 1, doluysa 0 döndürür:

sinifKayit.c

```
#include <stdio.h>

#define MAX 3

typedef struct { char ad[20]; int no; double ortalama; } Ogrenci;

typedef struct { Ogrenci liste[MAX]; int adet; } Sinif;
```

```c
int ekle(Sinif *s, const char *ad, int no, double ortalama) {
    if (s->adet == MAX) {
        return 0;
    }
    Ogrenci *o = &s->liste[s->adet];
    snprintf(o->ad, sizeof o->ad, "%s", ad);
    o->no = no;
    o->ortalama = ortalama;
    s->adet++;
    return 1;
}

void listele(const Sinif *s) {
    for (int i = 0; i < s->adet; i++) {
        const Ogrenci *o = &s->liste[i];
        printf("%d. %-6s %d %6.2f\n", i + 1, o->ad, o->no, o->ortalama);
    }
}

double sinifOrtalamasi(const Sinif *s) {
    double toplam = 0;
    for (int i = 0; i < s->adet; i++) {
        toplam += s->liste[i].ortalama;
    }
    return s->adet > 0 ? toplam / s->adet : 0.0;
}

int main(void) {
    Sinif sinif = {.adet = 0};

    ekle(&sinif, "Ali", 101, 72.5);
    ekle(&sinif, "Ayse", 102, 91.25);
    ekle(&sinif, "Can", 103, 64.0);
    if (!ekle(&sinif, "Deniz", 104, 85.75)) {
        printf("Liste dolu: Deniz eklenemedi\n");
    }
    listele(&sinif);
    printf("Sınıf ortalaması: %.2f\n", sinifOrtalamasi(&sinif));
    return 0;
}
```

Çıktı

```
Liste dolu: Deniz eklenemedi
1. Ali    101  72.50
2. Ayse   102  91.25
3. Can    103  64.00
Sınıf ortalaması: 75.92
```

Dördüncü öğrenci sığmadığı için `ekle` 0 döndürdü; liste bozulmadı. `&s->liste[s->adet]` şöyle okunur: `s->liste` dizidir, `[s->adet]` ilk boş gözdür, `&` o gözün adresini verir. Gerçek sistemde bu dizinin yerine Bölüm 10'daki dinamik dizi ya da Bölüm 14'teki bağlı liste kullanılır; kayıtları dosyaya yazmayı Bölüm 12'de göreceksin.

Bölüm Özeti – Bilmen Gerekenler

- **struct** farklı tipte üyeleri tek tipte toplar. Tanım bellek ayırmaz, değişken ayırır; `;` ile biter. Üye: `.`. İlk değer: `{"Ali", 101, 72.5}` ya da `{.no = 101}` (verilmeyenler 0).
- `typedef struct { ... } Ogrenci;` ile `Ogrenci a;` yazılır. Etiketsiz typedef'ten sonra `struct Ogrenci` yazılamaz.
- Struct ataması (`b = a`) **bütün üyeleri, dizi üyeler dahil, kopyalar**. `a == b` ve `o.ad = "Ali"` derlenmez (`strcmp`, `strcpy`).
- Struct dizisi: `liste[i].no`; takas için `Ogrenci gecici = liste[j];` yeter. `qsort` karşılaştırması `const Ogrenci *` alır, eşitlikte ikinci anahtar ekle.
- Fonksiyona **değer ile** (kopya) ya da **işaretçi ile** geçer. `p->no` ≡ `(*p).no`; `*p.no` derlenmez. Yalnız okuyan fonksiyon `const Ogrenci *` alır. Struct `return` ile döndürülebilir.
- Soldaki yapıysa `.`, yapıya işaretçiyse `->`: `liste[i].ad`, `p->dogum.yil`. `malloc(n * sizeof *p)` yapı dizisi ayırır.
- **Padding**: üyeler kendi hizalamasına göre yerleşir, toplam en büyük hizalamanın katı olur. `struct {char c; int i;}` bu kitabın gcc'sinde 8 bayttır.
- **union**: üyeler aynı belleği paylaşır, boyut en büyük üyeye bağlıdır, aynı anda tek üye geçerlidir; hangisinin geçerli olduğunu bir etiketle (`enum`) tut.
- **enum**: sabitler 0'dan başlar, açık değerden sonra birer artar, `int`'tir, aralık denetlenmez. `{A, B = 5, C}` → C = 6.
- **Bit alanı**: `unsigned int gun : 5;` üyeyi 5 bit'e sığdırır; `sizeof` küçülür (bu kitabın gcc'sinde 12 yerine 4 bayt). Sığmayan değer mod alır, adresi alınamaz, yerleşimi derleyiciye bağlıdır.

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki kod çalıştırıldıktan sonra hangisi doğrudur?

```c
typedef struct { char ad[10]; int puan; } Kayit;
Kayit a = {"Ali", 50};
Kayit b = a;
strcpy(b.ad, "Veli");
b.puan = 70;
```

A) `a.ad` de `"Veli"` olur, çünkü `ad` bir dizidir ve `b = a` yalnız adresini kopyalar.

B) `a.ad` `"Ali"`, `a.puan` 50 kalır; `b` bağımsız bir kopyadır.

C) `a.ad` `"Ali"` kalır ama `a.puan` 70 olur.

D) `Kayit b = a;` satırı derlenmez; yapılar `=` ile kopyalanamaz.

E) `a.puan` 50 kalır ama `a.ad` `"Veli"` olur.

2 `Ogrenci o = {"Ali", 101}; Ogrenci *p = &o;` tanımlarından sonra aşağıdaki ifadelerden hangisi **derlenmez**?

A) `o.no` B) `p->no` C) `(*p).no` D) `p.no` E) `(&o)->no`

3 Aşağıdaki tanımda `M + 0` ifadesinin değeri kaçtır?

```
enum E { K, L = 5, M, N = 2, O };
```

(A) 6 (B) 8 (C) 9 (D) 11 (E) 13

4 `union V { int i; double d; char c; };` tanımı ve `union V v;` için hangisi doğrudur? (`int` 4, `double` 8, `char` 1 bayttır.)

(A) `sizeof(v)` 8'dir; üyeler aynı belleği paylaşır ve `v.d = 2.5;` yazıldıktan sonra geçerli üye `d`'dir.

(B) `sizeof(v)` 13'tür; üyeler art arda saklanır.

(C) `sizeof(v)` 4'tür; boyutu en küçük üye belirler.

(D) `v.i = 5; v.d = 2.5;` sonrasında `v.i` hâlâ 5'tir.

(E) Farklı tipte üye taşıdığı için bu union tanımı derlenmez.

5 `typedef struct { int x; int y; } Nokta;` tanımından sonra hangi satır **derlenmez**?

(A) `Nokta a = {1, 2};` (B) `Nokta b[3];`

(C) `Nokta *p = NULL;` (D) `Nokta d = {.y = 5};`

(E) `struct Nokta c;`

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```
#include <stdio.h>

typedef struct { int no; int puan; } Kayit;

void artirDeger(Kayit k) {
    k.puan += 10;
    printf("%d ", k.puan);
}

void artirAdres(Kayit *k) {
    k->puan += 5;
    printf("%d ", k->puan);
}

int main(void) {
    Kayit x = {1, 50};
    Kayit y = x;
    artirDeger(x);
    artirAdres(&x);
    y.puan = 0;
    printf("%d %d\n", x.puan, y.puan);
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

enum Durum { BEKLE, CALIS = 3, BITTI, HATA = -1, YENI };

int main(void) {
    printf("%d %d %d %d %d\n", BEKLE, CALIS, BITTI, HATA, YENI);

    enum Durum d = BITTI;
    d++;
    printf("%d %d\n", d, YENI == BEKLE);
    printf("%d\n", BITTI + HATA * 2);
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

typedef struct { int x; int y; } Nokta;

typedef struct { Nokta bas; Nokta son; } Dogru;

int main(void) {
    Dogru d = {{1, 2}, {4, 6}};
    Dogru *p = &d;
    Nokta *q = &d.son;

    q->x -= p->bas.x;
    p->bas = d.son;
    d.son.y = d.bas.x + q->y;
    printf("(%d,%d) (%d,%d)\n", d.bas.x, d.bas.y, d.son.x, d.son.y);
    return 0;
}
```

C) Kod Yazma

9 Önce kitap sayısını, sonra her satırda bir kitabın adını (boşluksuz) ve fiyatını okuyan `kitapListesi` programını yaz. `Kitap` yapısı ve yapı dizisi kullan (en çok 20 kitap). Kitapları fiyata göre **küçükten büyüğe** sırala (kabarcık sıralaması, yapı takası ile) ve yazdır; ardından en pahalı kitabı ve fiyat ortalamasını yazdır.

```
Girdi:            Çıktı:
4                 Cprog 85.00
Algoritma 120.5   Ag 99.90
Cprog 85          Algoritma 120.50
Veri 150          Veri 150.00
Ag 99.9           En pahali: Veri
                  Ortalama: 113.85
```

10 Pay ve paydadan oluşan `Kesir` yapısını tanımla. İki kesri `a/b c/d` biçiminde okuyan, toplamlarını ve çarpımlarını **sadeleştirilmiş** yazdıran `kesirler` programını yaz. `Kesir topla(Kesir x, Kesir y)`, `Kesir carp(Kesir x, Kesir y)` ve `Kesir sadelestir(Kesir k)` fonksiyonlarını yaz; fonksiyonlar `Kesir` döndürsün. Pay ve paydaların pozitif olduğunu varsay. Girdi `3/4 5/6` için çıktı:

```
Toplam: 19/12
Carpim: 5/8
```

Çözümler

Cevap anahtarı

1	2	3	4	5
B	D	C	A	E

1 Cevap: B

Yapı ataması **bütün üyeleri kopyalar**, `ad` dizisi dahil; `b` bağımsız bir kopyadır. `strcpy(b.ad, ...)` ve `b.puan = 70` yalnız `b`'yi değiştirir, `a` aynı kalır. A yanlış: dizi üyesi adres olarak değil, içeriğiyle kopyalanır. C ve E, `b`'deki değişikliğin `a`'ya kısmen yansıdığını söylüyor; hiçbir üye paylaşılmaz. D yanlış: aynı tipte yapılar `=` ile atanabilir.

2 Cevap: D

`p` bir **işaretçidir**, `.` ise yapı değişkenine uygulanır; işaretçi için `->` gerekir. `p.no` derlenmez. A geçerli (`o` yapı). B ve C eşdeğer, geçerli yazımlardır (`p->no` ≡ `(*p).no`). E de geçerli: `(&o)` yapıya işaretçi olduğu için `->` kullanılabilir.

3 Cevap: C

Değerler: `K` 0, `L` 5 (açık), `M` 6 (5'ten sonra), `N` 2 (açık), `O` 3 (2'den sonra). `M + O = 6 + 3 = 9`. A (6), açık değerleri yok sayıp 0, 1, 2, 3, 4 diye sayanın cevabıdır (`M` = 2, `O` = 4). B (8), `O`'yu 2 sanmaktan (6 + 2); D (11), `O` yerine `L`'yi toplamaktan (6 + 5); E (13), `O`'yu `M`'den sonra gelen 7 sanmaktan (6 + 7) çıkar.

4 Cevap: A

Union'ın üyeleri **aynı belleği paylaşır**; boyutu en büyük üyeye (`double`, 8 bayt) eşittir ve her an yalnızca en son yazılan üye geçerlidir. B yanlış: 13, üyelerin art arda saklandığı yapı için (hizalamasız) bir toplamdır. C yanlış: boyutu en **büyük** üye belirler. D yanlış: `v.d` yazmak `v.i`'nin baytlarını ezer. E yanlış: union'ın amacı farklı tipte üyeleri aynı yerde tutmaktır.

5 Cevap: E

`typedef struct { ... } Nokta;` yapının **etiketi yoktur**, tek adı `Nokta`'dır. `struct Nokta c;` ise "etiketi `Nokta` olan yapı"yı arar; böyle bir tip tanımlanmadığı için gcc `storage size of 'c' isn't known` hatası verir. A (liste ile ilk değer), B (dizi), C (işaretçi) ve D (atanmış ilk değer) geçerlidir.

6

Adım adım:

1. `Kayit y = x;` → `y` bağımsız kopya, `y.puan = 50`.
2. `artirDeger(x)` yapıyı **kopyalayarak** alır; kopyada `puan` 60 olur ve `60` yazılır. Dışarıdaki `x.puan` 50'dir.
3. `artirAdres(&x)` gerçek `x`'i değiştirir: `x.puan` 55 olur, `55` yazılır.
4. `y.puan = 0;` yalnız `y`'yi etkiler. Son `printf` `55 0` yazar.

Çıktı

```
60 55 55 0
```

7

Değerler: `BEKLE` 0, `CALIS` 3, `BITTI` 4, `HATA` -1, `YENI` 0 (-1'den sonra gelir). `YENI` ile `BEKLE` **aynı değerdedir**; enum bunu yasaklamaz.

1. İlk `printf`: `0 3 4 -1 0`.
2. `d = BITTI` (4); `d++` ile 5 olur. Hiçbir sabit 5 değildir.
3. İkinci `printf`: `d` = 5 ve `YENI == BEKLE` → `0 == 0` doğru, yani `1`.
4. Üçüncü `printf`: `BITTI + HATA * 2` = `4 + (-1) * 2` = `2`.

Çıktı

```
0 3 4 -1 0
5 1
2
```

8

Başlangıç: `d.bas = (1, 2)`, `d.son = (4, 6)`; `q`, `d.son`'u gösterir.

1. `q->x -= p->bas.x;` → `d.son.x = 4 - 1 = 3`; `son = (3, 6)`.
2. `p->bas = d.son;` → yapı ataması: `bas = (3, 6)`.
3. `d.son.y = d.bas.x + q->y;` → `3 + 6 = 9`; `son = (3, 9)`.

Çıktı

```
(3,6) (3,9)
```

9

Yaklaşım: `Kitap` yapısı (ad + fiyat) ve `Kitap liste[20]`. Okurken toplamı biriktir. Kabarcık sıralamasında `liste[j].fiyat > liste[j + 1].fiyat` ise üç satırla **bütün yapıyı** takas et. Sıralı olduğundan en pahalı kitap son elemandır: `liste[n - 1]`. `n`'yi okur okumaz 1–20 aralığında olduğunu kontrol et: `n > 20` diziyi taşırır, `n = 0` ise `liste[-1]` dizi dışına erişir ve `toplam / n` sıfıra bölme olur.

kitapListesi.c

```c
#include <stdio.h>

typedef struct { char ad[30]; double fiyat; } Kitap;

int main(void) {
    Kitap liste[20];
    int n = 0;
    double toplam = 0;

    if (scanf("%d", &n) != 1 || n < 1 || n > 20) {   // dizi 20 kitaplık
        printf("Kitap sayisi 1 ile 20 arasinda olmali\n");
        return 1;
    }
    for (int i = 0; i < n; i++) {
        scanf("%29s %lf", liste[i].ad, &liste[i].fiyat);
        toplam += liste[i].fiyat;
    }
    for (int tur = 0; tur < n - 1; tur++) {
        for (int j = 0; j < n - 1 - tur; j++) {
            if (liste[j].fiyat > liste[j + 1].fiyat) {
                Kitap gecici = liste[j];
                liste[j] = liste[j + 1];
```

```
                liste[j + 1] = gecici;
            }
        }
    }
    for (int i = 0; i < n; i++) {
        printf("%s %.2f\n", liste[i].ad, liste[i].fiyat);
    }
    printf("En pahali: %s\n", liste[n - 1].ad);
    printf("Ortalama: %.2f\n", toplam / n);
    return 0;
}
```

Çıktı

```
4
Algoritma 120.5
Cprog 85
Veri 150
Ag 99.9
Cprog 85.00
Ag 99.90
Algoritma 120.50
Veri 150.00
En pahali: Veri
Ortalama: 113.85
```

10

Yaklaşım: `a/b + c/d = (a*d + c*b) / (b*d)`, çarpım `(a*c) / (b*d)`'dir. İkisinin sonucunu da `sadelestir` ile en büyük ortak bölenine (EBOB) bölersin. Fonksiyonlar `Kesir`'i **değer olarak** alır ve döndürür; `sadelestir` parametre kopyasını değiştirip döndürdüğü için dışarıdaki yapıya dokunmaz. `scanf("%d/%d %d/%d", ...)` eğik çizgiyi formatta karşılayarak okur.

kesirler.c

```
#include <stdio.h>

typedef struct { int pay; int payda; } Kesir;

Kesir sadelestir(Kesir k) {
    int a = k.pay;
    int b = k.payda;
    while (b != 0) {                // Öklid: a, EBOB'a dönüşür
        int kalan = a % b;
        a = b;
        b = kalan;
    }
    k.pay /= a;
    k.payda /= a;
    return k;
}

Kesir topla(Kesir x, Kesir y) {
    Kesir s = {x.pay * y.payda + y.pay * x.payda, x.payda * y.payda};
    return sadelestir(s);
}

Kesir carp(Kesir x, Kesir y) {
    Kesir s = {x.pay * y.pay, x.payda * y.payda};
    return sadelestir(s);
}

int main(void) {
```

```
    Kesir a, b;

    scanf("%d/%d %d/%d", &a.pay, &a.payda, &b.pay, &b.payda);
    Kesir t = topla(a, b);
    Kesir c = carp(a, b);
    printf("Toplam: %d/%d\n", t.pay, t.payda);
    printf("Carpim: %d/%d\n", c.pay, c.payda);
    return 0;
}
```

Çıktı

```
3/4 5/6
Toplam: 19/12
Carpim: 5/8
```

BÖLÜM 12

Dosya İşlemleri

Bu Bölümde Öğreneceklerin

- Verinin neden dosyada saklandığını; metin ve ikili (binary) dosya farkını
- `FILE *`, `fopen` modlarını, `fclose`'u, `NULL` kontrolünü ve `perror`'u
- `fprintf`, `fputs`, `fputc` ile yazmayı; `fscanf`, `fgets`, `fgetc` ile okumayı
- `while (!feof(f))` tuzağını; sayma, kopyalama, ekleme gibi tipik programları
- CSV satırlarını `sscanf` / `strtok` ile ayrıştırıp struct dizisine okumayı
- `stdin`, `stdout`, `stderr` akışlarını ve terminalde yönlendirmeyi (`>`, `<`)
- `fwrite` / `fread` ile ikili kayıtları; `fseek`, `ftell`, `rewind` ile rastgele erişimi
- Windows'ta `\r\n` farkını, dosya yollarını ve UTF-8 dosyaları

12.1 Dosya neden gerekir?

Şimdiye kadar bütün veriler değişkenlerdeydi, yani RAM'de; program bitince hepsi kaybolur. **Dosya** (file) diskte durur ve program bittikten sonra da yaşar: program veriyi yazar, bir sonraki çalışmada geri okur. İki tür dosya vardır:

	Metin dosyası (text file)	İkili dosya (binary file)
İçerik	Okunabilir karakterler (`.txt`, `.csv`, `.c`)	Bellekteki baytların kopyası (`.bin`, `.dat`)
Editörle açınca	Okunur	Anlamsız semboller
Fonksiyonlar	`fprintf`, `fscanf`, `fgets`, `fputs`	`fwrite`, `fread`
`int` değer 12345	5 karakter	Bellekteki 4 bayt

C dosyaya bir **akım** (stream) üzerinden erişir. Her dosya işlemi üç adımdır: **aç** (`fopen`) → **kullan** (oku/yaz) → **kapat** (`fclose`).

12.2 FILE, fopen ve fclose

`FILE`, `<stdio.h>` içinde tanımlı bir türdür; içini bilmen gerekmez, hep işaretçi olarak tutarsın:

```
FILE *f = fopen("dosyaAdi.txt", "mod");   // açar; başarısızsa NULL döner
```

`fopen`'ın ikinci parametresi **mod**dur ve dosyayla ne yapacağını belirler:

Mod	Anlamı	Dosya yoksa	Dosya varsa
`"r"`	Yalnız okuma	`NULL` döner	Baştan okur
`"w"`	Yalnız yazma	Oluşturur	**İçeriği siler** (0 bayt olur)
`"a"`	Sona ekleme (append)	Oluşturur	İçeriği korur, hep **sona** yazar
`"r+"`	Okuma + yazma	`NULL` döner	İçeriği korur
`"w+"`	Okuma + yazma	Oluşturur	**İçeriği siler**
`"a+"`	Okuma + sona ekleme	Oluşturur	İçeriği korur, yazma hep sonda

Moda `b` eklersen dosya **ikili** açılır: `"rb"`, `"wb"`, `"r+b"`. `b` yoksa metin modudur; farkını bölümün sonunda göreceksin.

> **Sınavda Çıkar**
>
> "`w` ile açılan dosyaya ne olur?" (içeriği silinir), "`a` ile?" (sona eklenir), "`r` ile olmayan dosya?" (`NULL`) üçlüsü her sınavda gelir. `r+` ile `w+` ayrımı da sevilir: ikisi de okur-yazar, ama yalnız `w+` içeriği siler.

İlk program bir dosya oluşturur, iki satır yazar, kapatır, sonra aynı dosyayı okuyup ekrana basar (okuma kısmını ve `perror`'u birazdan ayrıntılı göreceksin):

ilkDosya.c

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("selam.txt", "w");
    if (f == NULL) {
        perror("selam.txt");
        return 1;
    }
    fprintf(f, "Merhaba dosya!\n");
    fprintf(f, "Sayı: %d\n", 42);
    fclose(f);

    f = fopen("selam.txt", "r");
    if (f == NULL) { perror("selam.txt"); return 1; }
    char satir[100];
    while (fgets(satir, sizeof satir, f) != NULL) {
        printf("Okunan: %s", satir);
    }
    fclose(f);
    return 0;
}
```

Çıktı

```
Okunan: Merhaba dosya!
Okunan: Sayı: 42
```

w modunun tuzağı

`"w"` dosyayı **açar açmaz** siler; tek bir şey yazmasan bile. Önceki içerik geri gelmez:

```c
FILE *f = fopen("liste.txt", "w");   // liste.txt'nin eski içeriği BURADA silinir
fclose(f);                           // hiçbir şey yazmadık: dosya artık 0 bayt
```

Olduğu gibi okumak istiyorsan `"r"`, eklemek istiyorsan `"a"` aç; `"w"` yalnız sıfırdan yazarken doğrudur.

fopen başarısız olursa

Dosya açılamayabilir: ad yanlış, dosya yok, klasör yanlış, izin yok. Bu durumda `fopen` `NULL` döndürür; her `fopen`'dan hemen sonra denetle. `perror("başlık")` (print error) sistemin hata nedenini stderr'e `başlık: neden` biçiminde yazar:

olmayanDosya.c

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("yok.txt", "r");
    if (f == NULL) {
        perror("yok.txt");
        return 1;
    }
    printf("Dosya açıldı\n");
    fclose(f);
    return 0;
}
```

Çıktı – program 1 çıkış koduyla biter

```
yok.txt: No such file or directory
```

Mesaj Windows, Linux ve macOS'ta aynıdır. `return 1;` programın sorunla bittiğini söyler (çıkış kodu, Bölüm 1); bu yüzden çıktı etiketinde "1 çıkış koduyla biter" yazar. Bundan sonra yer kazanmak için kontrolü tek satıra yazacağız: `if (f == NULL) { return 1; }`; sen sınavda `perror` ile yaz.

> **Sık Yapılan Hata: fopen sonucunu denetlememek**
>
> `NULL` olan `f`'yi `fgets`, `fprintf` ya da `fscanf`'e verirsen program çöker (Linux/macOS terminali `Segmentation fault` yazar, Windows'ta program çoğu zaman sessizce kapanır). Dosya adındaki tek bir yazım hatası ödevi çöken programa çevirir.

fclose ve tampon

`fclose(f)` dosyayı kapatır. Yazdığın veri hemen diske gitmez: C veriyi bellekte bir **tamponda** (buffer) biriktirir ve toplu yazar; `fclose` tamponu boşaltır. Kapatmazsan program çöktüğünde bekleyen veri **yazılmaz** (dosya eksik ya da boş kalır), aynı dosyayı başka bir `fopen` ile okuyunca içerik henüz diske inmemiş olabilir ve Windows'ta açık dosya silinemez. `main` normal bitince kütüphane dosyaları kendisi kapatır, ama buna güvenme: işin biter bitmez `fclose`. Kapatmadan tamponu boşaltmak için `fflush(f)` vardır.

12.3 Dosyaya yazma: fprintf, fputs, fputc

Yazma fonksiyonları, tanıdığın ekran fonksiyonlarının dosyaya yazan halleridir; tek fark **ilk parametrenin dosya olması**:

Fonksiyon	Ne yazar?	Not
`fprintf(f, "biçim", ...)`	Biçimli çıktı	`printf` ile aynı belirteçler (Bölüm 3)
`fputs(s, f)`	Bir string	`'\n'` eklemez (`puts` ekler)
`fputc(c, f)`	Tek karakter	`putchar(c)` ile aynı iş

Program üçünü kullanır (`fputs` satır sonu eklemediği için `\n`'i `fputc` ile koyuyoruz), sonra dosyayı `fgetc` ile karakter karakter okuyup ekrana basar (`fgetc`'yi birazdan göreceksin):

yazmaCesitleri.c

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("yazma.txt", "w");
    if (f == NULL) { return 1; }
    fprintf(f, "Pi = %.2f, sayı = %d\n", 3.14159, 7);
    fputs("fputs satır sonu eklemez", f);
    fputc('\n', f);
    fputc('A', f);
    fputc('\n', f);
    fclose(f);

    f = fopen("yazma.txt", "r");
    if (f == NULL) { return 1; }
    int c = 0;
    while ((c = fgetc(f)) != EOF) {
        putchar(c);
    }
    fclose(f);
    return 0;
}
```

Çıktı

```
Pi = 3.14, sayı = 7
fputs satır sonu eklemez
A
```

12.4 Dosyadan okuma: fscanf, fgets, fgetc

Fonksiyon	Ne okur?	Dönüş değeri
`fscanf(f, "biçim", ...)`	`scanf` gibi; boşlukta durur	Doldurulan alan sayısı; hiçbir şey okunamadan dosya biterse `EOF`
`fgets(s, n, f)`	**Satır**; en çok `n - 1` karakter, `'\n'` dahil	`s`; dosya bitmişse `NULL`
`fgetc(f)`	**Tek karakter**	Karakterin kodu (`int`); dosya bitmişse `EOF`

Üçünde de kural aynı: **döngü koşulunu okuma fonksiyonunun dönüş değeriyle kur.**

fscanf

`fscanf`, Bölüm 3 ve 5'teki `scanf`'in kardeşidir; yalnızca ilk parametresi dosyadır. Bölüm 5'teki `while (scanf(...) == 1)` kalıbı burada da geçerlidir. `notlar.txt` dosyasında dört satır var; program notları yazdırıp ortalamayı bulur (`adet > 0` denetimi boş dosyada sıfıra bölmeyi önler):

notlar.txt

```
Ali 85
Ayse 92
Mehmet 70
Zeynep 100
```

notOrtalamasi.c

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("notlar.txt", "r");
    if (f == NULL) { perror("notlar.txt"); return 1; }
    char ad[30];
    int puan, toplam = 0, adet = 0;
    while (fscanf(f, "%29s %d", ad, &puan) == 2) {
        printf("%-8s -> %d\n", ad, puan);
        toplam += puan;
        adet++;
    }
    fclose(f);
    if (adet > 0) {
        printf("%d öğrenci, ortalama %.2f\n", adet, (double)toplam / adet);
    }
    return 0;
}
```

Çıktı

```
Ali      -> 85
Ayse     -> 92
Mehmet   -> 70
Zeynep   -> 100
4 öğrenci, ortalama 86.75
```

`%29s` genişliği `ad[30]` dizisini taşırmamak içindir (Bölüm 8); `== 2` "iki alanı da doldurdum" demektir. Dönüş değeri üç durumu ayırt eder: istenen alan sayısı kadarsa başarılıdır; `0` ise veri biçime uymuyordur (sayı beklerken `abc` gelmiş gibi) ve okunamayan yazı **dosyada kalır**; `EOF` (derleyicilerde -1) ise okunacak bir şey kalmamıştır. Dosya kullanıcıdan geliyorsa satırı `fgets` ile alıp `sscanf` ile ayrıştırmak daha sağlamdır (CSV bölümünde).

fgets

`fgets(satir, n, f)` bir satırı `'\n'` dahil, en çok `n - 1` karakter olarak diziye koyar ve sonuna `'\0'` ekler. Klavyeden okuyan `fgets(s, n, stdin)` ile aynı fonksiyondur; `stdin` yerine `f` yazarsın. Bölüm 8'deki gibi `'\n'`'i `strcspn` ile silebilirsin. `metin.txt` dört satırdır, üçüncüsü boştur:

metin.txt

```
Dosyalar verileri kalıcı tutar.
Program kapanır, dosya kalır.

Son satır.
```

satirOku.c

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    FILE *f = fopen("metin.txt", "r");
    if (f == NULL) { perror("metin.txt"); return 1; }
    char satir[80];
    int n = 0;
    while (fgets(satir, sizeof satir, f) != NULL) {
        satir[strcspn(satir, "\n")] = '\0';
        n++;
        printf("%d: [%s]\n", n, satir);
    }
    fclose(f);
    return 0;
}
```

Çıktı

```
1: [Dosyalar verileri kalıcı tutar.]
2: [Program kapanır, dosya kalır.]
3: []
4: [Son satır.]
```

Boş satır tek bir `"\n"` olarak okunur ve silinince `[]` kalır; satır sayısında atlanmaz. Satır dizinden uzunsa `fgets` yalnız `n - 1` karakter alır, kalanını **bir sonraki çağrıda** verir; diziyi en uzun satıra göre seç.

Son fonksiyon `fgetc(f)` tek karakter okur. Dönüş türü `int`'tir, çünkü sonuç iki türlüdür: bir karakter kodu (0-255) ya da dosya sonu işareti **`EOF`** (negatif sayı); `char` değişkenine atarsan ikisi karışabilir. Kalıbı `yazmaCesitleri` programında gördün: `int c;` ve `while ((c = fgetc(f)) != EOF)`. (`putchar(c)` = `fputc(c, stdout)`, `getchar()` = `fgetc(stdin)`.)

12.5 feof tuzağı

`feof(f)` ("dosya sonu mu?") adı yüzünden herkesi yanıltır. Gerçek kural: `feof`, dosya sonunu aşan bir **okuma girişimi başarısız olduktan sonra** doğru olur. Son satırı başarıyla okumak onu doğru yapmaz. Üç satırlık `meyve.txt` ile deneyelim; son satır da dahil her satır `'\n'` ile biter:

meyve.txt

```
elma
kiraz
üzüm
```

feofTuzak.c

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("meyve.txt", "r");
    if (f == NULL) { return 1; }
    char satir[40] = "";
    int n = 0;
    while (!feof(f)) {                    // YANLIŞ KALIP
        fgets(satir, sizeof satir, f);
        n++;
        printf("%d: %s", n, satir);
    }
    fclose(f);
    return 0;
}
```

Çıktı

```
1: elma
2: kiraz
3: üzüm
4: üzüm
```

Üç satırlık dosya **dört** satır yazdırdı. Üçüncü turda `üzüm\n` okundu, ama dosya sonu henüz "görülmedi": `feof` hâlâ yanlış. Döngü bir tur daha döndü; `fgets` okuyacak şey bulamayıp `NULL` döndürdü ve C standardı gereği `satir` dizisine **dokunmadı**, `printf` eski içeriği yazdı. Bu davranış tanımlıdır, her derleyicide aynı çıkar. `fscanf` ile de aynısı olur: değişken eski değerini korur ve son sayı iki kez toplanır.

Tuzak bazen saklanır: dosyanın son satırı `'\n'` ile **bitmiyorsa** `fgets` o satırı okurken dosya sonuna çarpar ve `feof` hemen doğru olur. Hata, biri dosyaya Enter ekleyince ortaya çıkar. Doğru kalıp, okuma fonksiyonunun **dönüş değerine** bakmaktır: `while (fgets(satir, sizeof satir, f) != NULL)`, `while (fscanf(f, "%d", &x) == 1)` ya da `while ((c = fgetc(f)) != EOF)`. Döngü bitince nedenini `feof(f)` (dosya bitti) ve `ferror(f)` (okuma hatası) söyler; `feof`'un meşru kullanımı budur.

> **● Sık Yapılan Hata: while (!feof(f)) ile döngü kurmak**
>
> Ders kitaplarının en yaygın dosya hatasıdır. Dosya `'\n'` ile bitiyorsa son kayıt **iki kez** işlenir; sınavda "bu kod kaç satır yazdırır" diye tam bu hata sorulur. Her zaman `fgets(...) != NULL`, `fscanf(...) == n` ya da `(c = fgetc(f)) != EOF` yaz.

12.6 Uygulamalar

Satır, kelime ve karakter sayma

Terminaldeki `wc` programının işi: `metin.txt` içindeki satırı, kelimeyi ve karakteri say. Kelime, boşluk olmayan ardışık karakterlerdir; "şu an bir kelimenin içinde miyim?" bilgisi `kelimeIcinde` bayrağında tutulur.

sayac.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    FILE *f = fopen("metin.txt", "r");
    if (f == NULL) { perror("metin.txt"); return 1; }
    int karakter = 0, satir = 0, kelime = 0, kelimeIcinde = 0;
    int c = 0;
```

```
    while ((c = fgetc(f)) != EOF) {
        karakter++;
        if (c == '\n') {
            satir++;
        }
        if (isspace(c)) {
            kelimeIcinde = 0;
        } else if (!kelimeIcinde) {
            kelimeIcinde = 1;
            kelime++;
        }
    }
    fclose(f);
    printf("Satır: %d, kelime: %d, bayt: %d\n", satir, kelime, karakter);
    return 0;
}
```

Çıktı

```
Satır: 4, kelime: 10, bayt: 79
```

Boş satır kelimeyi bozmaz, çünkü `isspace('\n')` doğrudur. `fgetc` **bayt** okur: dosyada 74 karakter vardır ama `ı` UTF-8'de iki bayt olduğu için (5 tane) program 79 saydı.

Dosya kopyalama

Kopyalama, birinden `fgetc` ile okuyup diğerine `fputc` ile yazmaktır. `FILE *` bir değişken olduğu için dosyaları **fonksiyona parametre** olarak verebilirsin; aşağıdaki `kopyala` hedef olarak `stdout`'u da kabul eder (sonraki başlıkta anlatılan ekran akımı). `buyut` doluysa harfleri büyütür: `toupper` yalnız ASCII harfleri çevirir, Türkçe `ı` gibi harflere dokunmaz (Bölüm 8). İki dosya açık olduğundan ikincisi açılamazsa birincisini **kapatıp** çıkmak gerekir:

kopyalama.c

```c
#include <stdio.h>
#include <ctype.h>

int kopyala(FILE *giris, FILE *cikis, int buyut) {
    int c, adet = 0;
    while ((c = fgetc(giris)) != EOF) {
        fputc(buyut ? toupper(c) : c, cikis);
        adet++;
    }
    return adet;
}

int main(void) {
    FILE *giris = fopen("metin.txt", "r");
    if (giris == NULL) { perror("metin.txt"); return 1; }
    FILE *cikis = fopen("kopya.txt", "w");
    if (cikis == NULL) {
        perror("kopya.txt");
        fclose(giris);                  // birincisini unutma
        return 1;
    }
    printf("%d bayt kopyalandı\n", kopyala(giris, cikis, 0));
    fclose(cikis);

    rewind(giris);                      // başa dön; bu kez ekrana, büyük harfle
    kopyala(giris, stdout, 1);
    fclose(giris);
    return 0;
}
```

```
Çıktı
79 bayt kopyalandı
DOSYALAR VERILERI KALıCı TUTAR.
PROGRAM KAPANıR, DOSYA KALıR.

SON SATıR.
```

`stdout` yerine `fopen("BUYUK.txt", "w")` ile açılmış bir dosya verseydin büyük harfli kopya dosyaya yazılırdı. Resim, PDF, `.exe` gibi ikili dosyaları kopyalarken `"rb"` / `"wb"` kullanmak zorundasın; metin modu Windows'ta baytları değiştirir (bölümün sonunda).

Dosyaya ekleme

`"a"` modu dosyanın sonuna ekler, içeriği silmez, dosya yoksa oluşturur; günlük (log) dosyaları böyle tutulur. Program her çalıştığında aynı satırı ekler, dosya büyür; `"w"` olsaydı her seferinde sıfırdan başlardı:

```c
FILE *f = fopen("gunluk.txt", "a");
if (f == NULL) { perror("gunluk.txt"); return 1; }
fprintf(f, "14:00 Laboratuvar\n");     // her zaman dosyanın SONUNA yazılır
fclose(f);
```

`"a"` modunda araya `fseek` koysan bile yazma sona yapılır (Sorular'da göreceksin).

12.7 CSV dosyaları

CSV (comma-separated values), her satırı bir kayıt, alanları virgülle ayrılmış metin dosyasıdır; Excel'in "CSV olarak kaydet" çıktısı budur. Yöntem: satırı `fgets` ile al, alanlara ayır. Gerçek veride bozuk satır çıkar; bu dosyada ilk satır başlık, beşinci satır da bilerek bozuktur:

ogrenciler.csv

```
ad,no,not
Ali Yilmaz,101,85.5
Ayse Demir,102,92
Mehmet Kaya,103,70.25
Can Er,abc,60
Zeynep Can,104,100
```

sscanf ve strtok ile ayrıştırma

`sscanf(satir, biçim, ...)`, `scanf`'in klavye yerine bir **string**'ten okuyan kardeşidir; dönüş değeri doldurulan alan sayısıdır. Boşluklu bir ad için `%s` yetmez (boşlukta durur); `%[^,]` kullanılır: "virgül **olmayan** her karakteri oku". Biçimdeki `,` gerçek virgülle eşleşir. `"Ali Yilmaz,101,85.5"` satırı için `sscanf(satir, "%29[^,],%d,%lf", ad, &no, &puan)` üç alanı da doldurur ve `3` döner; bozuk `"Can Er,abc,60"` satırında ad okunur, `%d` başarısız olunca `sscanf` durur ve `1` döner. **Dönüş değeri satırın sağlam olup olmadığını anlama yoludur:** `== 3` ise üç alan da geldi. `%29`, `ad[30]` dizisini taşırmamak içindir.

`strtok` (`<string.h>`) alternatiftir: string'i ayırıcı karakterlerden **parçalar**. İlk çağrı `strtok(satir, ",")` ilk parçayı, sonraki çağrılar `strtok(NULL, ",")` diğerlerini verir; parça kalmayınca `NULL` döner. Ama `strtok` ayırıcıların yerine `'\0'` yazar, yani **orijinal string'i bozar**; değiştirilebilir bir dizi gerekir (`char *satir = "..."` string sabitiyle kullanmak tanımsız davranıştır). Ardışık ayırıcıları tek sayar: `"a,,b"` iki parça verir, boş alanı **atlar**. Parçalar yazı olarak gelir; sayılar için ayrıca `atoi`/`atof` gerekir (Bölüm 8). Bu yüzden CSV'de `sscanf` hem ayırır hem çevirir ve tercih edilir.

Struct dizisine okuma ve rapor yazma

Hepsini birleştirelim: `ogrenciler.csv`'yi okuyup `Ogrenci` struct dizisine (Bölüm 11) doldur, bozuk satırları bildir, `rapor.txt` dosyasını yaz ve göster. Başlık satırı ilk `fgets` ile atlanır:

csvOku.c

```c
#include <stdio.h>

#define MAX 100

typedef struct {
    char ad[30];
    int no;
    double puan;
} Ogrenci;

int main(void) {
    FILE *f = fopen("ogrenciler.csv", "r");
    if (f == NULL) { perror("ogrenciler.csv"); return 1; }
    Ogrenci liste[MAX];
    int adet = 0;
    char satir[100];
    if (fgets(satir, sizeof satir, f) == NULL) {      // başlık satırı
        fclose(f);
        return 1;
    }
    while (adet < MAX && fgets(satir, sizeof satir, f) != NULL) {
        Ogrenci o;
        if (sscanf(satir, "%29[^,],%d,%lf", o.ad, &o.no, &o.puan) == 3) {
            liste[adet++] = o;
        } else {
            printf("Hatalı satır atlandı: %s", satir);
        }
    }
    fclose(f);

    f = fopen("rapor.txt", "w");
    if (f == NULL) { perror("rapor.txt"); return 1; }
    double toplam = 0;
    fprintf(f, "%-4s %-12s %6s\n", "No", "Ad", "Not");
    for (int i = 0; i < adet; i++) {
        fprintf(f, "%-4d %-12s %6.2f\n", liste[i].no, liste[i].ad, liste[i].puan);
        toplam += liste[i].puan;
    }
    if (adet > 0) {
        fprintf(f, "Ortalama: %.2f\n", toplam / adet);
    }
    fclose(f);

    f = fopen("rapor.txt", "r");                      // raporu göster
    if (f == NULL) { return 1; }
    while (fgets(satir, sizeof satir, f) != NULL) {
        printf("%s", satir);
    }
    fclose(f);
    return 0;
}
```

```
Çıktı
Hatalı satır atlandı: Can Er,abc,60
No   Ad              Not
101  Ali Yilmaz    85.50
102  Ayse Demir    92.00
103  Mehmet Kaya   70.25
104  Zeynep Can   100.00
Ortalama: 86.94
```

Bozuk satır atlanıp bildirildi. Dosya Windows'ta `\r\n` ile kaydedilip Linux/macOS'ta okunursa satır sonunda `\r` kalır; `%lf` sayıyı orada bitirdiği için sorun çıkmaz, ama son alan string olsaydı `\r` ona karışırdı.

> **İpucu: Türkçe Excel CSV'yi ; ile kaydeder**
>
> Türkçe ayarlı Excel ondalık ayırıcı virgül olduğu için alanları **noktalı virgülle** ayırır (`Ali;101;85,5`). Böyle bir dosyayı okurken biçimi `"%29[^;];%d;%lf"` yap ve ondalık virgülü noktaya çevir.

12.8 stdin, stdout, stderr

Her C programı başlarken üç akım açık gelir ve her biri bir `FILE *`'dır:

Akım	Varsayılan	Eşdeğeri
`stdin`	Klavye	`scanf(...)` = `fscanf(stdin, ...)`
`stdout`	Ekran	`printf(...)` = `fprintf(stdout, ...)`
`stderr`	Ekran (hata kanalı)	`fprintf(stderr, ...)`

Bölüm 8'deki `fgets(s, n, stdin)` çağrısındaki üçüncü parametre bu yüzdendi. `stderr` hata ve uyarı mesajları içindir; normal çıktıdan **ayrı bir kanaldır** ve tamponsuzdur (yazılan hemen görünür):

akimlar.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 0;
    fprintf(stdout, "Bir sayı gir: ");
    if (fscanf(stdin, "%d", &sayi) != 1) {
        fprintf(stderr, "Hata: sayı okunamadı\n");
        return 1;
    }
    fprintf(stdout, "Karesi: %d\n", sayi * sayi);
    fprintf(stderr, "Bilgi: hesaplama bitti\n");
    return 0;
}
```

```
Çıktı
Bir sayı gir: 7
Karesi: 49
Bilgi: hesaplama bitti
```

Ekranda ikisi iç içe görünür; fark terminalde **yönlendirme** (redirection) yapınca ortaya çıkar: `>` stdout'u dosyaya gönderir, `<` stdin'i dosyadan okutur, `2>` stderr'i yönlendirir.

```
./akimlar > cikti.txt             ← "Karesi" dosyaya gider; stderr ekranda kalır
./akimlar < girdi.txt             ← sayıyı klavye yerine dosyadan okur
./akimlar 2> hatalar.txt          ← yalnız stderr dosyaya gider
```

Windows Komut İstemi'nde (cmd) komutlar `akimlar > cikti.txt` biçiminde aynen çalışır. PowerShell'de `<` desteklenmez; yerine `Get-Content girdi.txt | .\akimlar` yaz. Windows PowerShell 5.1'in `>` işlemi dosyayı UTF-16 yazar; Türkçe karakterli çıktıyı cmd ile yönlendirmek daha güvenlidir.

12.9 İkili dosyalar: fwrite ve fread

Metin dosyasında sayıyı yazıya çevirir, okurken geri çevirirsin. **İkili** dosyada bellekteki baytlar olduğu gibi diske geçer: çevirme yok, kayıp yok.

```
size_t fwrite(const void *veri, size_t birimBoyut, size_t adet, FILE *f);
size_t fread(void *hedef, size_t birimBoyut, size_t adet, FILE *f);
```

İkisi de `adet` tane `birimBoyut` baytlık öğeyi taşır ve **başarıyla taşınan öğe sayısını** döndürür; eksik dönerse dosya bitmiş (ya da hata olmuş) demektir. Struct dizisini tek çağrıda yazarsın; `fread(&k, sizeof k, 1, f) == 1`, tek kaydı okuyup başarıyı denetleme kalıbıdır. Dosyayı `"wb"` / `"rb"` ile aç.

Program üç kaydı `kayit.bin`'e yazıp geri okur, sonra aynı veriyi metin dosyasına da yazıp boyutları karşılaştırır. Boyutu, yazma bittiği anda `ftell(f)` ile alıyoruz: konum göstergesi dosyanın sonundadır, yani dosya boyutunu verir (`ftell`'i bir sonraki başlıkta ayrıntılı göreceksin). Bu ikili (`"wb"`) dosya için standartça garantidir; metin modunda açılmış dosyada `ftell`'in değeri yalnızca `fseek` ile geri dönmek için güvenilir bir konumdur, bayt sayısı olması standartça garanti edilmez (Linux ve macOS'ta bayt sayısıdır).

ikiliKayit.c

```c
#include <stdio.h>

typedef struct {
    char ad[15];
    int no;
    double puan;
} Kayit;

int main(void) {
    Kayit yaz[3] = {{"Ali", 101, 85.5}, {"Ayse", 102, 92.0},
                    {"Mehmet", 103, 70.25}};
    FILE *f = fopen("kayit.bin", "wb");
    if (f == NULL) { perror("kayit.bin"); return 1; }
    size_t yazilan = fwrite(yaz, sizeof(Kayit), 3, f);
    printf("%zu kayıt yazıldı; kayıt %zu bayt, dosya %ld bayt\n",
           yazilan, sizeof(Kayit), ftell(f));
    fclose(f);

    Kayit oku[3];
    f = fopen("kayit.bin", "rb");
    if (f == NULL) { perror("kayit.bin"); return 1; }
    size_t okunan = fread(oku, sizeof(Kayit), 3, f);
    fclose(f);
    for (size_t i = 0; i < okunan; i++) {
        printf("%-7s %d %.2f\n", oku[i].ad, oku[i].no, oku[i].puan);
    }

    f = fopen("kayit.txt", "w");
    if (f == NULL) { perror("kayit.txt"); return 1; }
    for (int i = 0; i < 3; i++) {
        fprintf(f, "%s,%d,%.2f\n", yaz[i].ad, yaz[i].no, yaz[i].puan);
```

```
    }
    printf("Aynı veri metin dosyasında: %ld bayt\n", ftell(f));
    fclose(f);
    return 0;
}
```

Çıktı

```
3 kayıt yazıldı; kayıt 32 bayt, dosya 96 bayt
Ali     101 85.50
Ayse    102 92.00
Mehmet  103 70.25
Aynı veri metin dosyasında: 46 bayt
```

- Alan toplamı 15 + 4 + 8 = 27 bayt olsa da 64 bitlik derleyicilerde struct 32 bayt çıkar: derleyici alanları hizalamak için araya boş bayt (padding) koyar. Senin derleyicinde sayı farklı olabilir; kural şu: dosya boyutu = kayıt sayısı × `sizeof`. Dolgu baytları da dosyaya yazılır ve içerikleri belirsizdir (bellekte o an ne varsa); dosyada anlamsız baytlar görmek istemiyorsan yapıyı doldurmadan önce `string.h`'deki `memset(&k, 0, sizeof k);` ile bütün baytlarını sıfırlayabilirsin.
- İkili dosya **her zaman küçük değildir**: burada ad dizisinin boş baytları da yazıldığı için ikili dosya (96 bayt) metin dosyasından büyük çıktı (46 bayt; Windows'ta metin modu her `\n`'e bir `\r` eklediği için 49). Asıl kazancı boyut değil, **sabit boyutlu kayıt** (n'inci kayda doğrudan atlama), dönüşümsüz hız ve `double`'ın kayıpsız saklanmasıdır.

> **Not: İkili dosya taşınabilir değildir**
>
> İkili dosya baytları aynen yazdığı için bir bilgisayarda yazılıp başkasında okunabilmesi garanti değildir: `long` Windows'ta 4, Linux/macOS'ta 8 bayttır; struct hizalaması derleyiciye göre değişir; bazı işlemciler çok baytlı sayıları ters sırada (endianness) saklar. Yalnız aynı programla okuyacaksan ikili kullan; paylaşacaksan metin (CSV) güvenlidir. Ayrıca struct içinde `char *ad` gibi bir **işaretçi** varsa `fwrite` metni değil **adresi** yazar ve adres sonra geçersizdir; kaydedilecek metinler `char ad[15]` gibi **dizi** olmalıdır.

12.10 Rastgele erişim: fseek, ftell, rewind

Dosya açılınca bir **konum göstergesi** vardır: sonraki okuma/yazma dosyanın hangi baytından başlayacak. Her okuma/yazma göstergeyi o kadar ilerletir. Göstergeyi elle oynatmak için:

Fonksiyon	Ne yapar?
`fseek(f, kaydirma, nereden)`	Göstergeyi `nereden`'den `kaydirma` bayt öteye taşır; başarılıysa `0` döner
`ftell(f)`	Göstergenin dosya başından itibaren bayt konumunu verir (`long`)
`rewind(f)`	Göstergeyi başa alır (`fseek(f, 0, SEEK_SET)` gibi)

`nereden`: `SEEK_SET` dosyanın **başı**, `SEEK_CUR` **şimdiki konum** (kaydırma negatif olabilir), `SEEK_END` dosyanın **sonu** (geri gitmek için negatif kaydırma). Konumlar dizi indeksi gibi 0'dan sayılır; son baytın ötesi dosya boyutudur, yani `fseek(f, 0, SEEK_END)` ardından `ftell(f)` **dosya boyutunu** verir (`ikiliKayit` programında `ftell` ile aynı bilgiyi yazma sırasında aldık). `"w+b"` ile açıp yazdığımız 26 harflik dosyada deneyelim:

harfler.c

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("harfler.bin", "w+b");
    if (f == NULL) { return 1; }
    fputs("abcdefghijklmnopqrstuvwxyz", f);
    fseek(f, 0, SEEK_END);
    printf("Boyut: %ld\n", ftell(f));

    fseek(f, 5, SEEK_SET);              // baştan 5 ileri
    printf("%c ", fgetc(f));            // okuyunca konum 1 ilerler: 6
    fseek(f, 2, SEEK_CUR);              // 6 + 2 = 8
    printf("%c ", fgetc(f));
    fseek(f, -3, SEEK_END);             // 26 - 3 = 23
    printf("%c ", fgetc(f));
    printf("konum: %ld\n", ftell(f));
    fclose(f);
    return 0;
}
```

Çıktı

```
Boyut: 26
f i x konum: 24
```

`fgetc` her okuyuşta konumu 1 ilerletir; konum izleme sorularının püf noktası budur. Yeni bir `fseek`'i her zaman **o anki** konuma göre hesapla.

> **Sınavda Çıkar**
>
> "`fseek(f, 3, SEEK_CUR)` sonrası hangi karakter okunur?" gibi sorularda önce konumu yaz: önceki okuma konumu 1 artırmıştır. `SEEK_END` ile hep **negatif** kaydırma verilir (`-1` son baytın başıdır). Dosya `'\n'` ile bitiyorsa son bayt `'\n'`'dir ve boyuta o da dahildir.

Kayıt numarasıyla okuma ve güncelleme

Kayıtlar sabit boyutlu olduğundan n'inci kaydın konumu **n × sizeof(Kayit)**'tir (kayıt sayısı da `ftell` ile bulunan dosya boyutu / `sizeof(Kayit)`'tir). Aşağıda dört kayıtlı dosyanın 2 numaralı (0'dan sayılır) kaydını okuyup puanını 10 artırır ve yerine geri yazarız. Dosyayı `"r+b"` ile açıyoruz: hem okur hem yazar ama içeriği silmez (`"w+b"` silerdi). `Kayit` yapısı önceki programdakiyle aynıdır.

```c
Kayit liste[4] = {{"Ali", 101, 85.5}, {"Ayse", 102, 92.0},
                  {"Mehmet", 103, 70.25}, {"Zeynep", 104, 100.0}};
FILE *f = fopen("kayit.bin", "wb");
if (f == NULL) { return 1; }
fwrite(liste, sizeof(Kayit), 4, f);
fclose(f);

f = fopen("kayit.bin", "r+b");
if (f == NULL) { return 1; }
Kayit k;
fseek(f, 2L * (long)sizeof(Kayit), SEEK_SET);     // 2 numaralı kayıt
if (fread(&k, sizeof k, 1, f) == 1) {
    printf("Okunan: %s %.2f\n", k.ad, k.puan);
    k.puan += 10;
    fseek(f, -(long)sizeof k, SEEK_CUR);          // bir kayıt geri
    fwrite(&k, sizeof k, 1, f);
}
```

```
rewind(f);
while (fread(&k, sizeof k, 1, f) == 1) {
    printf("%d %-7s %.2f\n", k.no, k.ad, k.puan);
}
fclose(f);
```

Çıktı

```
Okunan: Mehmet 70.25
101 Ali     85.50
102 Ayse    92.00
103 Mehmet  80.25
104 Zeynep  100.00
```

`fread` konumu bir kayıt ilerlettiği için yazmadan önce `-sizeof k` kadar **geri** gitmek şarttır; yoksa güncel kayıt bir sonrakinin üstüne yazılırdı. Bir kural daha: `"+"` ile açılmış dosyada okumadan yazmaya geçerken araya `fseek` ya da `rewind`, yazmadan okumaya geçerken `fseek`, `rewind` ya da `fflush` konmalıdır; yoksa davranış tanımsızdır. Yukarıdaki `fseek` ve `rewind` bunu karşılar.

12.11 Metin ve ikili mod, dosya yolu

Windows'ta \r\n

Windows satır sonunu iki baytla (`\r\n`), Linux ve macOS tek baytla (`\n`) yazar. **Metin modunda** (`"r"`, `"w"`) C bu farkı saklar: `"w"` ile `'\n'` yazarsan Windows'ta diske `\r\n` gider, `"r"` ile okurken `\r\n` yine `'\n'` olarak gelir. **İkili modda** (`"rb"`, `"wb"`) çeviri yapılmaz.

Durum	Linux / macOS	Windows
`"w"` ile `"a\nb\n"` yazınca dosya boyutu	4 bayt	6 bayt
`"rb"` ile `fgetc` döngüsü	`\n` görür	`\r` ve `\n` ayrı ayrı görür

Kural: **metin dosyasına metin modu; ikili veri, bayt sayma ve `fseek` ile gezinme için ikili mod.** (Windows'ta metin modunda `fseek` yalnız `ftell`'den aldığın konumlarla ya da 0 ile güvenlidir.)

Dosya yolu ve çalışma klasörü

`fopen("notlar.txt", ...)` gibi yalnız ad yazarsan bu **göreli yoldur** (relative path): dosya, programın **çalıştığı klasörde** aranır. Bu klasör `.c` ya da `.exe` dosyanın yeri değil, **terminalin o anda bulunduğu** klasördür. VS Code terminali açtığın klasörde başlar:

```
CDersleri> gcc -std=c17 -Wall -Wextra ders/dosya.c -o dosya
CDersleri> ./dosya        ← notlar.txt'yi CDersleri klasöründe arar, ders/ içinde değil!
```

"Dosya açılamadı" hatasının en sık nedeni dosyanın yanlış klasörde olmasıdır; önce `perror` mesajına bak, sonra `ls` (Windows'ta `dir`) ile dosyanı gör. Tam yol (absolute path) da yazabilirsin; Windows'ta ters bölü kaçış dizisi olduğu için **çift** yazılır: `fopen("C:\\ders\\notlar.txt", "r")`. Eğik çizgi de kabul edilir: `"C:/ders/notlar.txt"`.

> **Sık Yapılan Hata: Yolda tek ters bölü**
>
> `fopen("C:\notlar.txt", "r")` yazarsan `\n` yeni satır karakteri olur ve aranan ad bozulur; `"C:\test"` içindeki `\t` de sekmedir. Dosya bulunamaz, `perror` garip bir ad gösterir. Kural Bölüm 1'deki gibidir: her ters bölü `\\` yazılır. Ayrıca Windows uzantıları gizliyorsa `notlar.txt` aslında `notlar.txt.txt` olabilir.

12.12 Türkçe karakterli dosyalar

Bölüm 3'te gördüğün gibi `ş`, `ç`, `ğ`, `ü`, `ö`, `ı` UTF-8'de **iki bayt**tır ve dosyada da iki bayt tutarlar. `fputs`, `fgets`, `fprintf` baytları anlamadan taşıdığı için Türkçe metni sorunsuz kopyalar; sorun sayarken (`sayac` programındaki 74 karakter, 79 bayt) ve `toupper` gibi ASCII'ye özgü fonksiyonlarda çıkar. Windows konsolunda bozuk görünürse çözüm `chcp 65001`'dir (Bölüm 3). Bir uyarı daha: Not Defteri bazen dosyayı "UTF-8 with BOM" kaydeder ve başa 3 görünmez bayt (`EF BB BF`) eklenir; ilk alan bu baytlarla başlar. Dosyayı VS Code'da **UTF-8** (BOM'suz) kaydet.

Bölüm Özeti – Bilmen Gerekenler

- Dosya kalıcıdır; **metin** dosyası okunabilir karakterlerden, **ikili** dosya bellek baytlarından oluşur. Dosya işlemi: `fopen` → kullan → `fclose`.
- Modlar: `r` oku (yoksa `NULL`), `w` yaz (**var olanı siler**), `a` sona ekle (`fseek` etkisiz), `+` hem oku hem yaz, `b` ikili. Her `fopen`'dan sonra `if (f == NULL)` + `perror`. `fclose` tamponu boşaltır.
- Yazma: `fprintf(f, ...)`, `fputs(s, f)` (satır sonu eklemez), `fputc(c, f)`. Okuma: `fscanf` (dönüş = doldurulan alan sayısı), `fgets(s, n, f)` (`'\n'` dahil), `fgetc` (`int c`, dosya sonu `EOF`).
- **`while (!feof(f))` yanlıştır:** son satır iki kez işlenir. Doğru kalıp: `fgets(...) != NULL`, `fscanf(...) == n`, `(c = fgetc(f)) != EOF`.
- CSV: `fgets` ile satırı al, `sscanf(satir, "%29[^,],%d,%lf", ...)` ile ayrıştır, `== 3` ile doğrula. `strtok` string'i bozar, boş alanı atlar.
- `stdin`, `stdout`, `stderr` birer `FILE *`'dır. Terminalde `>` stdout'u, `<` stdin'i, `2>` stderr'i yönlendirir.
- İkili: `fwrite(veri, boyut, adet, f)` / `fread(...)`, dönüş = taşınan öğe sayısı; `"wb"` / `"rb"`. Struct'ta işaretçi olmasın; ikili dosya taşınabilir değildir.
- `fseek(f, kaydirma, SEEK_SET | SEEK_CUR | SEEK_END)`, `ftell(f)` (konum; sonda dosya boyutu), `rewind(f)`. n'inci kayıt = `n * sizeof(Kayit)`. `"+"` modunda okuma ile yazma arasında `fseek`/`rewind`.
- Windows metin modu `\n` → `\r\n` çevirir; ikili modda çeviri yok. Göreli yol terminalin bulunduğu klasörden aranır; Windows yolunda `\\` yaz. Türkçe harfler dosyada 2 bayttır.

Sorular

A) Çoktan Seçmeli

1 Var olan `defter.txt` dosyası 5 satır içeriyor. Programda yalnızca `FILE *f = fopen("defter.txt", "w");` ve hemen ardından `fclose(f);` çalışıyor. Dosyanın durumu nedir?

(A) 5 satırı da korunur.
(B) `fopen` dosya var olduğu için `NULL` döndürür, dosya değişmez.
(C) Dosya boşalır (0 bayt olur).
(D) Dosya diskten silinir.
(E) Yalnız ilk satır korunur.

2 Dosyadan karakter okurken `int c = fgetc(f);` yazılır. `c` neden `char` değil `int` olarak tanımlanır?

(A) `char` dosyadan okunan harfi tutamaz.

(B) `int` kullanınca okuma döngüsü daha hızlı çalışır.

(C) `fgetc` yalnız rakam karakterlerini okuyabilir ve rakamlar `int`'tir.

(D) `putchar` yalnız `int` değişken kabul eder.

(E) Dosya sonu değeri `EOF` (negatif bir sayı), geçerli hiçbir karakter koduyla karışmamalıdır; bunu tutmak için `char`'dan geniş bir tür gerekir.

3 Aşağıdakilerden hangisi `feof(f)` fonksiyonu için **doğrudur**?

(A) Dosya sonunu aşan bir okuma girişimi başarısız olduktan sonra doğru olur; son veriyi okumak tek başına onu doğru yapmayabilir.

(B) Dosyadaki son satır okunur okunmaz doğru olur.

(C) Dosya boşsa bile ilk okumadan önce doğrudur.

(D) Yalnız ikili dosyalarda çalışır.

(E) Dosyanın kaç bayt olduğunu döndürür.

4 Her kaydı 20 bayt olan ikili bir dosyada, 0'dan başlayarak numaralanan **3 numaralı** kaydın başına gitmek için hangi çağrı yazılır?

(A) `fseek(f, 3, SEEK_SET);`

(B) `fseek(f, 3, SEEK_CUR);`

(C) `fseek(f, 20, SEEK_SET);`

(D) `fseek(f, 3L * 20, SEEK_SET);`

(E) `fseek(f, -3L * 20, SEEK_END);`

5 İçinde `A` ve satır sonu bulunan `gunluk.txt`, `"a"` modunda açılıyor. Sırayla `fseek(f, 0, SEEK_SET);` ve `fputs("B\n", f);` çalışıp dosya kapanıyor. Dosyanın içeriği ne olur?

(A) `B` satırı başa yazılır; dosya `B`, `A` olur.

(B) `"a"` modunda yazma her zaman sona yapılır; dosya `A`, `B` olur.

(C) `B` eski içeriğin üstüne yazılır; dosya yalnız `B` olur.

(D) `fseek` `"a"` modunda hata verip programı durdurur.

(E) Kod derlenmez.

B) Çıktı Tahmini

6 `soru6.txt` dosyası aşağıdaki gibidir (üç satır; sonuncusu dahil her satır `'\n'` ile biter):

soru6.txt

```
kalem
silgi
defter
```

Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("soru6.txt", "r");
    char satir[20] = "";
    int n = 0;

    while (!feof(f)) {
        fgets(satir, sizeof satir, f);
        n++;
        printf("%d-%s", n, satir);
    }
    fclose(f);
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("harf.bin", "w+b");
    fputs("ABCDEFGHIJ", f);
    printf("%ld\n", ftell(f));
    rewind(f);
    printf("%c\n", fgetc(f));
    fseek(f, 3, SEEK_CUR);
    printf("%c\n", fgetc(f));
    fseek(f, -2, SEEK_END);
    int c = fgetc(f);
    printf("%c %ld\n", c, ftell(f));
    fclose(f);
    return 0;
}
```

8 `soru8.txt` dosyasının içeriği aşağıdadır:

soru8.txt

```
12 7 abc 5
```

Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("soru8.txt", "r");
    int x, d, n = 0, toplam = 0;
    char s[10];

    while ((d = fscanf(f, "%d", &x)) == 1) {
        toplam += x;
        n++;
    }
    printf("%d %d %d\n", n, toplam, d);
    fscanf(f, "%9s", s);
    printf("%s\n", s);
    fclose(f);
    return 0;
}
```

C) Kod Yazma

9 `soru9.txt` dosyasının her satırında bir öğrencinin adı ve notu vardır. Dosyayı okuyup **ortalamanın üstünde** not alan öğrencileri `ustu.txt` dosyasına (`ad not` biçiminde, aynı sırayla) yazan ve aynı satırları ekrana da yazdıran `ortalamaUstu` programını yaz. Önce ortalama ekrana yazılsın. Dosyayı iki kez okumak için `rewind` kullan.

soru9.txt

```
Ali 60
Ayse 90
Can 75
Demet 45
Efe 80
```

Beklenen ekran çıktısı:

```
Ortalama: 70.00
Ayse 90
Can 75
Efe 80
```

10 Bir kırtasiye için `Urun` struct'ı tanımla: `ad` (en çok 14 karakter), `adet` (`int`), `fiyat` (`double`). Programda üç ürünü (`Kalem 20 12.50`, `Defter 10 40.00`, `Silgi 50 5.00`) `urunler.bin` dosyasına **ikili** yaz. Sonra dosyayı `"r+b"` ile aç, 1 numaralı (0'dan sayılan) ürünün fiyatını **%10 artırıp** dosyadaki yerine geri yaz. Son olarak dosyadaki bütün ürünleri baştan okuyup yazdır. Programın adı `urunGuncelle` olsun.

Beklenen ekran çıktısı:

```
Kalem     20  12.50
Defter    10  44.00
Silgi     50   5.00
```

Çözümler

Cevap anahtarı

1 C	2 E	3 A	4 D	5 B

1 Cevap: C

`"w"` modu dosyayı açar açmaz **siler** (0 bayta indirir); hiçbir şey yazılmasa da önceki içerik gider. A ve E içeriğin korunduğunu söylüyor, yanlış. B yanlış: `"w"` dosya varsa da açılır, `NULL` yalnız açılamazsa döner. D yanlış, dosya diskte durur ama boştur.

2 Cevap: E

`fgetc` geçerli karakter kodlarını (0-255) ya da dosya sonunu bildiren `EOF`'u (negatif) döndürür. `EOF`'u karakterlerden güvenle ayırmak için hem karakter kodlarını hem negatif değeri tutabilen `int` gerekir. A yanlış, `char` karakter tutabilir; sorun `EOF`'u ayırt etmektir. B'nin hızla ilgisi yok. C yanlış, `fgetc` her karakteri okur. D'de `putchar` parametresi `int`'tir ama `int c` yazmamızın sebebi bu değil.

3 Cevap: A

`feof`, dosya sonunu aşan bir okuma **denemesi** başarısız olduktan sonra doğru olur; bu yüzden `while (!feof(f))` son kaydı iki kez işler. B tersini söylüyor, tuzak tam budur. C yanlış, boş dosyada da ilk okuma girişimine kadar `feof` yanlıştır. D yanlış, her akımda çalışır. E yanlış, boyutu `fseek` + `ftell` verir.

4 Cevap: D

3 numaralı kaydın konumu `3 * 20 = 60` bayttır ve baştan ölçüldüğü için `SEEK_SET` kullanılır. A kayıt boyutunu unutmuş, 3. bayta gider. B şimdiki konumdan 3 bayt ilerler. C 1 numaralı kaydın başına gider. E dosyanın **sonundan** 60 bayt geri gider; dosyada 6 kayıt yoksa bu 3 numaralı kayıt olmaz.

5 Cevap: B

`"a"` modunda yazılan her veri, araya `fseek` girse bile **dosyanın sonuna** eklenir (C standardı böyle der). A ve C `fseek`'in işe yaradığını varsayıyor. D yanlış, `fseek` hata vermez, sadece yazmayı etkilemez. E yanlış, kod derlenir.

Çıktı

```
A
B
```

6

Dosya `kalem\nsilgi\ndefter\n` içerir. İlk üç turda `fgets` sırayla `kalem\n`, `silgi\n`, `defter\n` okur ve `1-kalem`, `2-silgi`, `3-defter` yazılır. Üçüncü turda son satır okundu ama dosya sonu henüz "görülmediği" için `feof` yanlış kaldı. Dördüncü turda `fgets` başarısız oldu (`NULL` döndürdü), `satir` **değişmedi** ve `printf` `defter\n`'i bir kez daha yazdı: `4-defter`. Bu turdan sonra `feof` doğru olur ve döngü biter.

Çıktı
```
1-kalem
2-silgi
3-defter
4-defter
```

7

Dosya `ABCDEFGHIJ`, 10 bayt (konumlar 0-9). `fputs` sonrası konum 10 olduğundan ilk satır `10`. `rewind` konumu 0 yapar; `fgetc` `A` okur (konum 1). `fseek(f, 3, SEEK_CUR)` konumu 1 + 3 = 4 yapar; `fgetc` `E`'yi okur (konum 5). `fseek(f, -2, SEEK_END)` konumu 10 - 2 = 8 yapar; `fgetc` `I`'yı okur ve konum 9 olur: `I 9`. Dosya `'\n'` ile bitmediği için sondan 2 geri `I`'dır; sonda `'\n'` olsaydı `J` okunurdu.

Çıktı
```
10
A
E
I 9
```

8

Döngü `12` ve `7`'yi okur (`n = 2`, `toplam = 19`). Üçüncü okumada `abc` sayı değildir; `fscanf` `0` döndürür (`EOF` değil, çünkü dosya bitmedi) ve döngü biter, yani `d = 0`. Okunamayan `abc` dosyada kaldığı için `%9s` onu okur.

Çıktı
```
2 19 0
abc
```

9

İlk geçişte `fscanf` ile toplam ve adet bulunur. `rewind` göstergeyi başa alır; ikinci geçişte ortalamadan büyük notlar hem `ustu.txt`'ye hem ekrana yazılır. Boş dosya ihtimaline karşı `adet == 0` denetlenir.

ortalamaUstu.c
```c
#include <stdio.h>

int main(void) {
    FILE *giris = fopen("soru9.txt", "r");
    if (giris == NULL) { perror("soru9.txt"); return 1; }
    char ad[30];
    int puan, toplam = 0, adet = 0;
    while (fscanf(giris, "%29s %d", ad, &puan) == 2) {
        toplam += puan;
        adet++;
    }
    if (adet == 0) { fclose(giris); return 0; }
    double ortalama = (double)toplam / adet;
    printf("Ortalama: %.2f\n", ortalama);

    rewind(giris);
    FILE *cikis = fopen("ustu.txt", "w");
    if (cikis == NULL) { perror("ustu.txt"); fclose(giris); return 1; }
    while (fscanf(giris, "%29s %d", ad, &puan) == 2) {
        if (puan > ortalama) {
            fprintf(cikis, "%s %d\n", ad, puan);
            printf("%s %d\n", ad, puan);
        }
```

```
    }
    fclose(giris);
    fclose(cikis);
    return 0;
}
```

Çıktı

```
Ortalama: 70.00
Ayse 90
Can 75
Efe 80
```

10

Önce üç ürün `"wb"` ile yazılır. Dosya `"r+b"` ile yeniden açılır (içeriği silmez). 1 numaralı ürüne `fseek` ile gidilip `fread` ile okunur, fiyat güncellenir; `fread` konumu bir kayıt ilerlettiği için `-sizeof` kadar geri gidilip `fwrite` yapılır. Son döngüden önce `rewind` gerekir.

urunGuncelle.c

```c
#include <stdio.h>

typedef struct { char ad[15]; int adet; double fiyat; } Urun;

int main(void) {
    Urun liste[3] = {{"Kalem", 20, 12.50}, {"Defter", 10, 40.00},
                     {"Silgi", 50, 5.00}};
    FILE *f = fopen("urunler.bin", "wb");
    if (f == NULL) { perror("urunler.bin"); return 1; }
    fwrite(liste, sizeof(Urun), 3, f);
    fclose(f);

    f = fopen("urunler.bin", "r+b");
    if (f == NULL) { perror("urunler.bin"); return 1; }
    Urun u;
    fseek(f, 1L * (long)sizeof(Urun), SEEK_SET);
    if (fread(&u, sizeof u, 1, f) == 1) {
        u.fiyat *= 1.10;
        fseek(f, -(long)sizeof u, SEEK_CUR);
        fwrite(&u, sizeof u, 1, f);
    }
    rewind(f);
    while (fread(&u, sizeof u, 1, f) == 1) {
        printf("%-8s %3d %6.2f\n", u.ad, u.adet, u.fiyat);
    }
    fclose(f);
    return 0;
}
```

Çıktı

```
Kalem     20  12.50
Defter    10  44.00
Silgi     50   5.00
```

BÖLÜM 13

Özyineleme (Recursion)

Bu Bölümde Öğreneceklerin

- Özyinelemenin (kendini çağıran fonksiyon) ne olduğunu ve hangi problemlerde işe yaradığını
- Temel durum (base case) ile özyinelemeli durumu; her çağrının temel duruma yaklaşması gerektiğini
- `faktoriyel(4)` çağrısında çağrı yığınının adım adım nasıl dolup boşaldığını
- Faktöriyel, üs alma (hızlı üs dahil), basamak toplamı, EBOB ve ikilik tabana çevirmeyi özyinelemeyle yazmayı
- Fibonacci'de çağrı ağacını, tekrar eden hesapları ve bellekleme (memoization) ile hızlandırmayı
- Dizi ve string üzerinde özyinelemeyi (toplam, en büyük, ters yazdırma, uzunluk, palindrom)
- Hanoi kulelerini ve özyinelemeli ikili aramayı
- Yazdırmayı çağrıdan önce ya da sonra yapmanın çıktıyı nasıl tersine çevirdiğini
- Temel durum unutulunca olan yığın taşmasını; özyineleme ile döngü arasında seçim yapmayı

13.1 Özyineleme nedir

Rus **matruşka** bebeklerini bilirsin: bebeği açarsın, içinden daha küçük bir bebek çıkar; onu açarsın, içinden bir küçüğü daha çıkar. Her bebeğin içinde "kendisinin daha küçük hâli" vardır. En küçük bebek açılmaz; işlem orada biter. **Özyineleme** (recursion) tam olarak bu fikirdir: bir problemi, **aynı türden ama daha küçük** bir probleme indirgersin; en küçük problemi ise doğrudan çözersin.

Programlamada bunun anlamı basittir: **bir fonksiyon kendi kendini çağırır.** Bölüm 6'da bunun mümkün olduğunu söylemiştik; şimdi nasıl yazıldığına bakalım. 1'den `n`'e kadar sayıların toplamı şöyle düşünülebilir: `n` için toplam, `n` sayısına `n - 1` için toplamı eklemektir; 0 için toplam 0'dır.

ilkOzyineleme.c

```c
#include <stdio.h>

int topla(int n) {
    if (n == 0) {
        return 0;
    }
    return n + topla(n - 1);
}

int main(void) {
    printf("topla(1) = %d\n", topla(1));
    printf("topla(5) = %d\n", topla(5));
    printf("topla(100) = %d\n", topla(100));
    return 0;
}
```

Çıktı
```
topla(1) = 1
topla(5) = 15
topla(100) = 5050
```

`topla(3)` hesaplanırken fonksiyon `3 + topla(2)` ifadesine gelir ve `topla(2)`'nin cevabını **bekler**. O da `2 + topla(1)` der, o da `1 + topla(0)` der. `topla(0)` kendini çağırmadan `0` döndürür ve zincir geriye doğru çözülür: `1 + 0 = 1`, `2 + 1 = 3`, `3 + 3 = 6`.

Bu işi `for` ile de yazabilirsin (Bölüm 5). Özyinelemeyi öğrenmemizin nedeni, bazı problemlerin (Hanoi kuleleri, ikili arama, bağlı liste ve ağaçlar) özyinelemeyle döngüden çok daha kısa ve doğal yazılmasıdır.

13.2 Temel durum ve özyinelemeli durum

Her özyinelemeli fonksiyonun iki parçası vardır:

```
int fonksiyon(int n) {
    if (en küçük problem) {          ← TEMEL DURUM: kendini çağırmadan cevap ver
        return cevap;
    }
    ... fonksiyon(daha küçük problem) ...   ← ÖZYİNELEMELİ DURUM
}
```

Parça	Görevi	`topla` örneğinde
Temel durum (base case)	Kendini çağırmadan doğrudan cevap verir; zinciri durdurur	`n == 0` ise `0`
Özyinelemeli durum (recursive case)	Problemi küçültüp kendini çağırır	`n + topla(n - 1)`
Yaklaşma	Her çağrı, temel duruma biraz daha yakın olmalıdır	`n` her çağrıda 1 azalır, sonunda 0 olur

Üçüncü satır en çok unutulandır: Temel durumun yazılması yetmez, çağrıların ona **ulaşması** gerekir. `topla(n + 1)` yazarsan `n` temel durumdan uzaklaşır ve zincir hiç bitmez.

Özyinelemeli fonksiyon yazarken en rahat yöntem **güven ilkesidir**: "`topla(n - 1)` doğru çalışıyor" diye kabul et ve yalnızca şunu sor: "`topla(n - 1)`'in cevabından `topla(n)`'in cevabını nasıl çıkarırım?" Çağrının içini kafanda açmaya çalışmak seni yorar; zinciri açıp izlemek ise sınavda iz sürerken gerekir.

> **Sınavda Çıkar**
>
> "Aşağıdaki özyinelemeli fonksiyonda hata nerede?" sorusunun cevabı hemen her zaman şunlardan biridir: **temel durum yok**, **temel duruma hiç ulaşılmıyor** (argüman yanlış yöne gidiyor) ya da **temel durum bazı girdileri kaçırıyor** (`n == 0` yazıp negatif sayı göndermek gibi). Önce temel durumu, sonra argümanın yönünü kontrol et.

13.3 Çağrı yığını ve faktoriyel

Bölüm 6'da her çağrının **yığın çerçevesi** (stack frame) açtığını, çerçevede parametrelerin ve yerel değişkenlerin durduğunu ve fonksiyon bitince çerçevenin silindiğini gördün. Özyinelemede aynı fonksiyonun **birçok çerçevesi aynı anda** yığındadır ve her birinin kendi `n`'i, kendi yerel değişkenleri vardır. Faktöriyeli (`n! = n · (n−1)!`, `0! = 1! = 1`) izleyelim. Program, girinti ile derinliği göstererek her aşamayı yazdırıyor:

faktoriyellz.c

```c
#include <stdio.h>

int derinlik = 0;                   // çıktıdaki girintiyi ayarlamak için

void girinti(void) {
    for (int i = 0; i < derinlik; i++) {
        printf("  ");
    }
}

int faktoriyel(int n) {
    int sonuc = 0;

    girinti();
    printf("faktoriyel(%d) başladı\n", n);
    if (n <= 1) {
        girinti();
        printf("faktoriyel(%d): temel durum, 1 döndürüyor\n", n);
        return 1;
    }
    derinlik++;
    sonuc = n * faktoriyel(n - 1);
    derinlik--;
    girinti();
    printf("faktoriyel(%d): sonuç = %d\n", n, sonuc);
    return sonuc;
}

int main(void) {
    printf("Sonuç: %d\n", faktoriyel(4));
    return 0;
}
```

Çıktı

```
faktoriyel(4) başladı
  faktoriyel(3) başladı
    faktoriyel(2) başladı
      faktoriyel(1) başladı
      faktoriyel(1): temel durum, 1 döndürüyor
    faktoriyel(2): sonuç = 2
  faktoriyel(3): sonuç = 6
faktoriyel(4): sonuç = 24
Sonuç: 24
```

Girinti her çağrının bir kat içeride başladığını gösterir. Aynı olayın yığındaki karşılığı (`f(4)`: `faktoriyel`'in `n = 4` olan çerçevesi; tablodaki ilk çerçeve en üsttedir, yani o an çalışandır):

Adım	Olay	Yığın (üstten alta)
1	`main`, `faktoriyel(4)`'ü çağırır	`f(4)` (n=4, sonuc=?) / `main`
2	`f(4)`, `faktoriyel(3)`'ü çağırır ve bekler	`f(3)` / `f(4)` / `main`
3	`f(3)`, `faktoriyel(2)`'yi çağırır	`f(2)` / `f(3)` / `f(4)` / `main`
4	`f(2)`, `faktoriyel(1)`'i çağırır	`f(1)` / `f(2)` / `f(3)` / `f(4)` / `main`
5	`f(1)` temel durum: `1` döndürür, çerçevesi silinir	`f(2)` / `f(3)` / `f(4)` / `main`
6	`f(2)`: `sonuc = 2 * 1 = 2` döndürür, silinir	`f(3)` / `f(4)` / `main`
7	`f(3)`: `sonuc = 3 * 2 = 6` döndürür, silinir	`f(4)` / `main`
8	`f(4)`: `sonuc = 4 * 6 = 24` döndürür, silinir	`main`

Üç şeye dikkat et:

- **Gidişte** (adım 1–4) hiçbir çarpma yapılmaz; çağrılar açılır ve cevap beklenir. Çarpmalar **dönüşte** (5–8) yapılır, çerçeveler kapanırken.
- Dört çerçevede dört ayrı `n` ve dört ayrı `sonuc` vardır; hiçbiri ötekini bozmaz. `f(2)`'nin `n`'ini `2` yapması `f(4)`'ün `n`'ini değiştirmez.
- Yığının en derin hâlinde `faktoriyel`'in 4 çerçevesi vardır (`main` ile 5). Yığın belleği çağrı derinliği kadar büyür.

`int` ile `faktoriyel(13)` taşar (Bölüm 5'te gördüğün gibi); büyük `n` için `long long` kullan.

> **Sınavda Çıkar**
>
> "Yığında aynı anda kaç çerçeve bulunur?" sorusunun cevabı **özyineleme derinliğidir**. Temel durumu `n <= 1` olan `faktoriyel(5)` için derinlik 5'tir (`n = 5, 4, 3, 2, 1`); soru `main` dahil mi diyorsa 6. Temel durumu `n == 0` olsaydı derinlik 6 olurdu (`n = 0` da bir çerçeve açar). Sayarken temel durumun çerçevesini **unutma**.

13.4 Sayılarla özyineleme

Üs alma: us(a, n) ve hızlı üs

$a^n = a \cdot a^{n-1}$ ve $a^0 = 1$. Bu doğrudan `us(a, n)` fonksiyonudur. **Hızlı üs** ise daha akıllıdır: `n` çiftse $a^n = (a^{n/2})^2$, tekse $a^n = a \cdot (a^{n/2})^2$. Yani `n` her çağrıda **yarıya** iner. İkisinde de kaç çağrı yapıldığını global `cagri` sayacı ile sayalım:

usAlma.c

```c
#include <stdio.h>

int cagri = 0;

long long us(long long a, int n) {
    cagri++;
    if (n == 0) {
        return 1;
    }
    return a * us(a, n - 1);
}

long long hizliUs(long long a, int n) {
    long long yarim = 0;

    cagri++;
```

```
    if (n == 0) {
        return 1;
    }
    yarim = hizliUs(a, n / 2);          // a^(n/2) bir kez hesaplanır
    if (n % 2 == 0) {
        return yarim * yarim;
    }
    return yarim * yarim * a;
}

int main(void) {
    long long sonuc = 0;

    cagri = 0;
    sonuc = us(3, 20);
    printf("us(3, 20) = %lld, çağrı sayısı: %d\n", sonuc, cagri);
    cagri = 0;
    sonuc = hizliUs(3, 20);
    printf("hizliUs(3, 20) = %lld, çağrı sayısı: %d\n", sonuc, cagri);
    return 0;
}
```

Çıktı

```
us(3, 20) = 3486784401, çağrı sayısı: 21
hizliUs(3, 20) = 3486784401, çağrı sayısı: 6
```

`hizliUs`'ta `n` şöyle iner: 20 → 10 → 5 → 2 → 1 → 0. `n = 5` ve `n = 1` tek olduğundan bu iki çağrı fazladan bir `a` ile çarpar. `us`'ta `n + 1` çağrı gerekirken `hizliUs`'ta yaklaşık $\log_2 n$ çağrı yeter (burada 6); `n = 1 000 000` için fark 1 000 001 çağrıya karşı yaklaşık 21 çağrıdır.

> **Sık Yapılan Hata: Aynı özyinelemeli çağrıyı iki kez yazmak**
>
> `return hizliUs(a, n / 2) * hizliUs(a, n / 2);` yazarsan sonuç doğru çıkar ama hız kazancı **yok olur**: Her çağrı artık iki çağrı açar ve toplam çağrı sayısı yine `n` ile orantılı olur. Yarım sonucu bir değişkene (`yarim`) koy ve **bir kez** hesapla.

Basamak toplamı, EBOB ve ikilik tabana çevirme

Bölüm 5'te döngüyle yazdığın üç algoritma, özyinelemeyle çok kısalır:

sayilarlaOzyineleme.c

```
#include <stdio.h>

int basamakToplami(int n) {
    if (n < 0) {
        return basamakToplami(-n);   // -45 → 45'in basamakları
    }
    if (n < 10) {
        return n;
    }
    return n % 10 + basamakToplami(n / 10);
}

int ebob(int a, int b) {
    if (b == 0) {
        return a;
    }
    return ebob(b, a % b);
}
```

```c
void ikiliYaz(int n) {
    if (n < 0) {                    // önce işaret, sonra mutlak değer
        printf("-");
        ikiliYaz(-n);
        return;
    }
    if (n > 1) {
        ikiliYaz(n / 2);
    }
    printf("%d", n % 2);
}

int main(void) {
    printf("basamakToplami(2749) = %d\n", basamakToplami(2749));
    printf("ebob(48, 18) = %d\n", ebob(48, 18));
    printf("13 ikilik tabanda: ");
    ikiliYaz(13);
    printf("\n0 ikilik tabanda: ");
    ikiliYaz(0);
    printf("\n-5 ikilik tabanda: ");
    ikiliYaz(-5);
    printf("\n");
    return 0;
}
```

Çıktı

```
basamakToplami(2749) = 22
ebob(48, 18) = 6
13 ikilik tabanda: 1101
0 ikilik tabanda: 0
-5 ikilik tabanda: -101
```

- **`basamakToplami`**: Son basamak `n % 10`, geri kalanı `n / 10`'dur. `basamakToplami(2749)` = `9 + basamakToplami(274)` = `9 + 4 + basamakToplami(27)` = `9 + 4 + 7 + basamakToplami(2)`. `2` tek basamaklı olduğundan temel durumdur: `9 + 4 + 7 + 2 = 22`.
- **`ebob`**: Öklid algoritması, Bölüm 5'teki döngüyle aynıdır: `(48, 18)` → `(18, 12)` → `(12, 6)` → `(6, 0)`. Döngüde `x` ve `y` değişkenlerine atama yapıyordun; burada yeni çağrının **argümanları** olarak geçer. `b` sıfır olunca cevap `a`'dır.
- **`ikiliYaz`**: Bir sayıyı 2'ye bölüp kalanları sırayla not alırsın; bunlar sondan başa doğru okunur (13 → 6 → 3 → 1 için kalanlar 1, 0, 1, 1, ters okuyunca `1101`). Özyineleme bu "ters okumayı" **bedavaya** yapar: `printf` özyinelemeli çağrıdan **sonra** olduğundan önce en yüksek bit yazılır. `printf`'i çağrıdan önceye alsan `1011` (ters) çıkardı. Bu ayrıntıyı "Yazdırmayı çağrıdan önce mi sonra mı" bölümünde ayrıca işleyeceğiz. `0` için `n > 1` yanlıştır, doğrudan `0` yazılır. Negatif sayıda önce `-` yazılır ve aynı iş sayının mutlak değeriyle yapılır; bu kontrol olmasaydı `ikiliYaz(-5)` için C'de `-5 % 2` = `-1` olduğundan ekrana `-1` çıkardı. `basamakToplami` da negatif sayıyı aynı yolla çevirir; yoksa `-45 < 10` doğru olduğundan temel durum hemen `-45` döndürürdü.

13.5 Fibonacci

Fibonacci dizisi `0, 1, 1, 2, 3, 5, 8, ...`: `fib(0) = 0`, `fib(1) = 1`, diğerleri iki öncekinin toplamı. Tanım zaten özyinelemelidir ve çevirmesi çok kolaydır; ama bu kez fonksiyon **iki kez** kendini çağırır.

fibonacci.c

```c
#include <stdio.h>

int cagri = 0;
```

```c
int fib(int n) {
    cagri++;
    if (n < 2) {
        return n;
    }
    return fib(n - 1) + fib(n - 2);
}

int main(void) {
    int degerler[] = {5, 10, 20};

    for (int i = 0; i < 3; i++) {
        int sonuc = 0;

        cagri = 0;
        sonuc = fib(degerler[i]);
        printf("fib(%2d) = %4d, çağrı sayısı: %5d\n", degerler[i], sonuc, cagri);
    }
    return 0;
}
```

Çıktı

```
fib( 5) =    5, çağrı sayısı:    15
fib(10) =   55, çağrı sayısı:   177
fib(20) = 6765, çağrı sayısı: 21891
```

`fib(20)` için 21 891 çağrı yapıldı; oysa cevap yalnızca 20 adımla bulunabilirdi. Neden, `fib(5)`'in **çağrı ağacında** (call tree) görünür:

```
                        fib(5)
                  /                \
              fib(4)                fib(3)
             /      \               /     \
         fib(3)    fib(2)       fib(2)   fib(1)
         /    \     /   \         /   \
    fib(2) fib(1) fib(1) fib(0) fib(1) fib(0)
     /   \
 fib(1) fib(0)
```

`fib(3)` iki kez, `fib(2)` üç kez, `fib(1)` beş kez hesaplanıyor. Aynı işler defalarca tekrarlanıyor. Toplam 15 çağrı vardır (programın `fib(5)` satırında da 15 çıktı). Çağrı sayısı `C(n)` için `C(0) = C(1) = 1` ve `C(n) = 1 + C(n−1) + C(n−2)` kuralı geçerlidir; yani `C(2) = 3`, `C(3) = 5`, `C(4) = 9`, `C(5) = 15`. Formül olarak `C(n) = 2 · fib(n+1) − 1`.

`n` bir artınca çağrı sayısı yaklaşık 1,6 katına çıkar (**üstel** büyüme). `fib(30)` 2 692 537 çağrı, `fib(40)` 331 milyondan fazla çağrı ister; `fib(40)` bilgisayarına göre saniyenin altında ile birkaç saniye arası sürer, ama her 10 adımda süre yaklaşık 120 katına çıkar; `fib(50)` onlarca saniye ile dakikalar alır. Bu yüzden `fib(40)` ve üstünü çalıştırmıyoruz.

> **Sınavda Çıkar**
>
> "`fib(n)` toplam kaç kez çağrılır?" sorusu sık gelir. Formülü ezberleme, `C(n) = 1 + C(n−1) + C(n−2)` ile küçük `n` için tablo çıkar: **1, 1, 3, 5, 9, 15, 25** (`n = 0...6`). İlk çağrı da sayılır. Soru "kaç toplama yapılır?" diye sorarsa: toplama yalnız temel durum olmayan çağrılarda yapılır, `fib(4)` için `(9 − 1) / 2 = 4`.

Bellekleme (memoization)

Çözüm basit: bir kez hesapladığını bir **dizide sakla**, tekrar gerekirse hesaplamak yerine diziden oku. Buna **bellekleme** (memoization) denir. `hafiza[n] == 0` demek "henüz hesaplanmadı" demektir (`n >= 1` için `fib(n)` en az 1'dir). Global diziler `0` ile başladığı için ayrıca doldurmaya gerek yok:

fibHafiza.c

```c
#include <stdio.h>

long long hafiza[91];              // global: başlangıçta hepsi 0
int cagri = 0;

long long fibHafiza(int n) {
    cagri++;
    if (n < 2) {
        return n;
    }
    if (hafiza[n] != 0) {           // daha önce hesaplandı
        return hafiza[n];
    }
    hafiza[n] = fibHafiza(n - 1) + fibHafiza(n - 2);
    return hafiza[n];
}

int main(void) {
    long long sonuc = fibHafiza(50);

    printf("fib(50) = %lld, çağrı sayısı: %d\n", sonuc, cagri);
    return 0;
}
```

Çıktı

```
fib(50) = 12586269025, çağrı sayısı: 99
```

`fib(50)` 40 milyar çağrı isterken burada 99 çağrı yeterli: `fibHafiza(n-1)` zinciri sonuna kadar iner, dönüşte diziyi doldurur; `fibHafiza(n-2)` ise artık hazır cevabı okur. Genel olarak `fibHafiza(n)` için `2n - 1` çağrı yapılır.

Karşılaştır: Bölüm 5'teki **döngülü** Fibonacci de `n` adımda biter ve dizi gerektirmez. Bellekleme özyinelemenin okunaklı yapısını korurken üstel maliyeti ortadan kaldırır, ama dizi ve derinliği `n` olan bir yığın kullanır. "Aynı alt problem tekrar tekrar çözülüyor" gördüğün her yerde bellekleme akla gelmelidir.

> **İpucu: Çağrı ağacını kâğıda çiz**
>
> Sınavda iki çağrılı bir fonksiyonun çıktısını ya da çağrı sayısını sorarlarsa en güvenli yol ağacı çizmektir. Her düğüme argümanı yaz; yapraklar temel durumlardır. Çıktı yazdırılıyorsa ağacı **soldan sağa, derinlemesine** gez.

13.6 Dizi ve string ile özyineleme

Bir dizi üzerinde özyinelemede "daha küçük problem", dizinin **daha kısa hâlidir**. Diziyi kopyalamazsın; yalnızca "ilk `n` eleman" demek için `n`'i azaltırsın (ya da başlangıcı ilerletirsin):

diziOzyineleme.c

```c
#include <stdio.h>

int toplam(const int a[], int n) {          // ilk n elemanın toplamı
    if (n == 0) {
```

```
        return 0;
    }
    return a[n - 1] + toplam(a, n - 1);
}

int enBuyuk(const int a[], int n) {        // n >= 1 olmalı
    int diger = 0;

    if (n == 1) {
        return a[0];
    }
    diger = enBuyuk(a, n - 1);
    return (a[n - 1] > diger) ? a[n - 1] : diger;
}

void tersYazdir(const int a[], int n) {
    if (n == 0) {
        return;
    }
    printf("%d ", a[n - 1]);
    tersYazdir(a, n - 1);
}

int main(void) {
    int a[] = {4, 9, 2, 7, 5};

    printf("toplam = %d\n", toplam(a, 5));
    printf("enBuyuk = %d\n", enBuyuk(a, 5));
    printf("ters: ");
    tersYazdir(a, 5);
    printf("\n");
    return 0;
}
```

Çıktı

```
toplam = 27
enBuyuk = 9
ters: 5 7 2 9 4
```

- `toplam`: Son elemanı ayır, geri kalanın toplamını özyinelemeyle bul. Temel durum **boş dizi** (`n == 0`) ve toplamı 0'dır.
- Aynı işi **başı ilerleterek** de yazabilirsin: `return a[0] + toplam(a + 1, n - 1);`. Bölüm 9'daki işaretçi aritmetiğiyle `a + 1`, ikinci elemandan başlayan "kısalmış dizi"dir.
- `enBuyuk`: Boş dizinin en büyüğü olmadığı için temel durum **tek elemandır** (`n == 1`). `n >= 1` olmayan çağrı temel duruma ulaşamaz.
- `tersYazdir`: Önce son elemanı yazar, sonra kalan kısmı çağırır. Elemanları baştan yazmak için `printf` çağrıdan **sonra** yazılırdı.

String'lerde "küçük problem", ilk karakteri atmış string'dir; `s + 1` bunu verir. Bitiş `'\0'` karakteridir (Bölüm 8):

stringOzyineleme.c

```
#include <stdio.h>
#include <string.h>

int uzunluk(const char *s) {
    if (*s == '\0') {
        return 0;
    }
    return 1 + uzunluk(s + 1);
}
```

```c
int palindromMu(const char *s, int bas, int son) {
    if (bas >= son) {
        return 1;
    }
    if (s[bas] != s[son]) {
        return 0;
    }
    return palindromMu(s, bas + 1, son - 1);
}

void tersYaz(const char *s) {
    if (*s == '\0') {
        return;
    }
    tersYaz(s + 1);
    putchar(*s);
}

int main(void) {
    const char *kelimeler[] = {"kayak", "araba", "radar"};

    printf("uzunluk(\"merhaba\") = %d\n", uzunluk("merhaba"));
    for (int i = 0; i < 3; i++) {
        int n = (int)strlen(kelimeler[i]);

        printf("%s: %s\n", kelimeler[i],
               palindromMu(kelimeler[i], 0, n - 1) ? "palindrom" : "değil");
    }
    printf("merhaba tersi: ");
    tersYaz("merhaba");
    printf("\n");
    return 0;
}
```

Çıktı

```
uzunluk("merhaba") = 7
kayak: palindrom
araba: değil
radar: palindrom
merhaba tersi: abahrem
```

- `uzunluk("abc")` = `1 + uzunluk("bc")` = `1 + 1 + uzunluk("c")` = `1 + 1 + 1 + uzunluk("")` = `3`. `strlen`'in yaptığı iş budur.
- `palindromMu`: **İki ucu** karşılaştırır, içeri doğru daralır. İki temel durum var: uçlar karşılaşırsa (`bas >= son`) hepsi eşleşmiştir, yani palindromdur; uçlardaki karakterler farklıysa palindrom değildir. Uzunluk tek de olsa çift de olsa `bas >= son` yakalar.
- `tersYaz`: Yazdırma özyinelemeli çağrıdan **sonradır**: önce string'in sonuna inilir, harfler dönüşte sondan başa yazılır. `putchar(*s)` çağrıdan önce olsaydı string düz yazılırdı.

13.7 Hanoi kuleleri

Üç çubuk (A, B, C) ve A'da üst üste dizili, küçükten büyüğe `n` disk var. Amaç, tüm diskleri **C'ye** taşımak. Kurallar: bir seferde yalnızca bir disk taşınır ve **büyük disk küçüğün üstüne konamaz**. Döngüyle yazması çok zordur; özyinelemeyle ise üç satırdır. Fikir: `n` diski A'dan C'ye taşımak için (B yedek)

1. üstteki `n - 1` diski A'dan B'ye taşı (C yedek),
2. en büyük diski A'dan C'ye taşı,

3. `n - 1` diski B'den C'ye taşı (A yedek).

Temel durum diskin kalmamasıdır (`n == 0`: yapılacak iş yok).

hanoi.c

```c
#include <stdio.h>

void hanoi(int n, char kaynak, char hedef, char yedek) {
    if (n == 0) {
        return;
    }
    hanoi(n - 1, kaynak, yedek, hedef);
    printf("Disk %d: %c -> %c\n", n, kaynak, hedef);
    hanoi(n - 1, yedek, hedef, kaynak);
}

long long hamleSayisi(int n) {
    if (n == 0) {
        return 0;
    }
    return 2 * hamleSayisi(n - 1) + 1;
}

int main(void) {
    int olculer[] = {1, 2, 3, 4, 10, 20, 30};

    hanoi(3, 'A', 'C', 'B');
    for (int i = 0; i < 7; i++) {
        printf("n = %2d: %lld hamle\n", olculer[i], hamleSayisi(olculer[i]));
    }
    return 0;
}
```

Çıktı

```
Disk 1: A -> C
Disk 2: A -> B
Disk 1: C -> B
Disk 3: A -> C
Disk 1: B -> A
Disk 2: B -> C
Disk 1: A -> C
n =  1: 1 hamle
n =  2: 3 hamle
n =  3: 7 hamle
n =  4: 15 hamle
n = 10: 1023 hamle
n = 20: 1048575 hamle
n = 30: 1073741823 hamle
```

`hanoi(3, 'A', 'C', 'B')` çağrısının yapısı (7 hamle):

```
hanoi(3: A -> C)
+-- hanoi(2: A -> B) :  1. Disk 1: A -> C   2. Disk 2: A -> B   3. Disk 1: C -> B
+-- Disk 3: A -> C   :  4. hamle
+-- hanoi(2: B -> C) :  5. Disk 1: B -> A   6. Disk 2: B -> C   7. Disk 1: A -> C
```

Hamle sayısı `H(n)` için `H(0) = 0` ve `H(n) = 2 · H(n−1) + 1` yazılır: iki kez `n − 1` diski taşıyorsun, arada bir hamle var. Çözüm **`H(n) = 2ⁿ − 1`**: 1, 3, 7, 15, 31, … Programdaki tablo da bunu doğruluyor (`2¹⁰ − 1 = 1023`, `2²⁰ − 1 = 1 048 575`). Her disk hamle sayısını yaklaşık **iki katına** çıkarır; 64 diskin hamle sayısı `2⁶⁴ − 1`, yaklaşık 1,8 · 10¹⁹'dur. Saniyede bir hamle yapsan yaklaşık 585 milyar yıl sürer. Özyinelemenin kısa yazılması, işin hızlı bitmesi demek değildir.

13.8 Özyinelemeli ikili arama

Bölüm 7'de ikili aramayı `while (alt <= ust)` döngüsüyle yazmıştın. Fikir özyinelemeye çok uygundur: ortaya bak; bulduysan bitti, bulamadıysan **aynı aramayı yarım dizide** yinele. Aynı dizi ve aynı iz çıktısı:

ikiliAraOzyineleme.c

```c
#include <stdio.h>

int ikiliAra(const int a[], int alt, int ust, int aranan) {
    int orta = 0;

    if (alt > ust) {
        return -1;
    }
    orta = (alt + ust) / 2;
    printf("  alt=%d ust=%d orta=%d a[orta]=%d\n", alt, ust, orta, a[orta]);
    if (a[orta] == aranan) {
        return orta;
    }
    if (a[orta] < aranan) {
        return ikiliAra(a, orta + 1, ust, aranan);
    }
    return ikiliAra(a, alt, orta - 1, aranan);
}

int main(void) {
    int a[] = {3, 7, 12, 19, 25, 31, 42, 56, 68, 77};

    printf("31 aranıyor:\n");
    printf("  sonuç: %d\n", ikiliAra(a, 0, 9, 31));
    printf("10 aranıyor:\n");
    printf("  sonuç: %d\n", ikiliAra(a, 0, 9, 10));
    return 0;
}
```

Çıktı

```
31 aranıyor:
  alt=0 ust=9 orta=4 a[orta]=25
  alt=5 ust=9 orta=7 a[orta]=56
  alt=5 ust=6 orta=5 a[orta]=31
  sonuç: 5
10 aranıyor:
  alt=0 ust=9 orta=4 a[orta]=25
  alt=0 ust=3 orta=1 a[orta]=7
  alt=2 ust=3 orta=2 a[orta]=12
  sonuç: -1
```

`31` için üç satır Bölüm 7'deki adım tablosuyla aynıdır. Döngü ile özyinelemenin eşleşmesi:

	Döngü (Bölüm 7)	Özyineleme
Aranacak aralık	`alt` ve `ust` **değişkenleri** güncellenir	Yeni çağrının **argümanları** olur: `orta + 1, ust` ya da `alt, orta - 1`
Bitiş (bulunamadı)	`while (alt <= ust)` koşulu bozulunca döngüden çıkılır, `-1` döner	`alt > ust` temel durumdur: `-1`
Bitiş (bulundu)	`return orta;`	Temel durum: `return orta;`
Bellek	Sabit	Her adımda bir çerçeve: en çok $\lfloor \log_2 n \rfloor + 2$ çerçeve

Özyinelemeli çağrı `return` ile **hemen sonuç olarak** döner, çağrıdan sonra yapılacak başka iş yoktur. Bu yapıya **kuyruk özyinelemesi** diyeceğiz (bölümün sonunda).

> **Sık Yapılan Hata: Özyinelemeli çağrının sonucunu return etmeyi unutmak**
>
> `ikiliAra(a, orta + 1, ust, aranan);` satırını `return` **olmadan** yazarsan çağrı yapılır ama sonuç atılır; akış fonksiyonun altındaki satıra düşer ve yanlış sonuç (burada `-1`) çıkar, üstelik gcc uyarı vermez. Aynı `return` fonksiyonun **son** çağrısından silinirse fonksiyon değer döndürmeden biter: sonuç tanımsızdır (çöp değer) ve gcc `-Wall` ile `control reaches end of non-void function` uyarısını verir. Kural: Sonuç döndüren fonksiyonda özyinelemeli çağrı `return ikiliAra(...)` biçiminde yazılır ya da değeri başka bir ifadede kullanılır.

13.9 Yazdırmayı çağrıdan önce mi sonra mı yapmalı

Özyinelemeli fonksiyonda `printf`'i çağrıdan önce ya da sonra yazmak **çıktıyı tamamen değiştirir**. Çağrıdan önceki satırlar **gidişte** (yığın dolarken), sonraki satırlar **dönüşte** (yığın boşalırken, ters sırayla) çalışır:

onceSonra.c

```c
#include <stdio.h>

void once(int n) {                  // önce yaz, sonra çağır
    if (n == 0) {
        return;
    }
    printf("%d ", n);
    once(n - 1);
}

void sonra(int n) {                 // önce çağır, sonra yaz
    if (n == 0) {
        return;
    }
    sonra(n - 1);
    printf("%d ", n);
}

void cetvel(int n) {                // iki özyinelemeli çağrı
    if (n == 0) {
        return;
    }
    cetvel(n - 1);
    printf("%d ", n);
    cetvel(n - 1);
}

int main(void) {
```

```
    printf("once(3):   ");
    once(3);
    printf("\nsonra(3):  ");
    sonra(3);
    printf("\ncetvel(3): ");
    cetvel(3);
    printf("\n");
    return 0;
}
```

Çıktı

```
once(3):   3 2 1
sonra(3):  1 2 3
cetvel(3): 1 2 1 3 1 2 1
```

Yazdırma yeri	Ne zaman çalışır?	`n = 3` için çıktı
Çağrıdan **önce**	Giderken: `n` büyükten küçüğe	`3 2 1`
Çağrıdan **sonra**	Dönerken: en son açılan çerçeve ilk kapanır	`1 2 3`

`sonra(3)`'ü yığında izle: `sonra(3)` → `sonra(2)` → `sonra(1)` → `sonra(0)` temel durumdur ve döner. Şimdi çerçeveler kapanır: `sonra(1)` `1`'i yazar, `sonra(2)` `2`'yi, `sonra(3)` `3`'ü. Yığın "son giren ilk çıkar" çalıştığı için özyineleme bir şeyi **tersine çevirmenin** doğal yoludur; `ikiliYaz` ve `tersYaz` bu yüzden bedavaya ters sırada çalışıyordu.

İki özyinelemeli çağrılı `cetvel` için tanımı açık yaz: `cetvel(3)` = `cetvel(2)`, `3`, `cetvel(2)`. `cetvel(2)` = `cetvel(1)`, `2`, `cetvel(1)`; `cetvel(1)` = `1` (çünkü `cetvel(0)` hiçbir şey yazmaz). Yani `cetvel(2)` = `1 2 1` ve `cetvel(3)` = `1 2 1 3 1 2 1`.

> **Sınavda Çıkar**
>
> En klasik özyineleme sorusu budur: `printf` çağrıdan önce mi sonra mı? Yöntem: her çağrıyı kendi satırına aç. Çağrıdan **önceki** `printf`'leri aşağı inerken, **sonraki** `printf`'leri yukarı çıkarken yaz. `f(n-1)` iki kez yazılmışsa ikisini ayrı ayrı aç ve soldan sağa oku. Aşağı inerken yazdırdıklarını, çıkarken yazdıklarına karıştırma.

13.10 Temel durumu unutmak: yığın taşması

Temel durum yoksa zincir bitmez, ama bellek sonsuz değildir. Her çağrı bir çerçeve açar ve **çağrı yığını sınırlıdır** (genellikle birkaç MB: Windows'ta 1–2 MB, Linux ve macOS'ta 8 MB). Çerçeveler yığını doldurunca program **yığın taşması** (stack overflow) ile çöker. Aşağıdaki program kasıtlı olarak hatalıdır; çıktı karışmasın diye yalnızca ilk üç çağrıda yazdırıyor:

sonsuz.c

```c
#include <stdio.h>

int sayac = 0;

int sonsuzToplam(int n) {
    sayac++;
    if (sayac <= 3) {
        printf("çağrı %d: n = %d\n", sayac, n);
    }
    return n + sonsuzToplam(n - 1);        // temel durum yok!
}

int main(void) {
    printf("Başlıyor\n");
    printf("%d\n", sonsuzToplam(5));
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
sonsuz.c: In function 'sonsuzToplam':
sonsuz.c:5:5: warning: infinite recursion detected [-Winfinite-recursion]
    5 | int sonsuzToplam(int n) {
      |     ^~~~~~~~~~~~
sonsuz.c:10:16: note: recursive call
   10 |     return n + sonsuzToplam(n - 1);        // temel durum yok!
      |                ^~~~~~~~~~~~~~~~~~
```

Çıktı – program çöker (Segmentation fault)

```
Başlıyor
çağrı 1: n = 5
çağrı 2: n = 4
çağrı 3: n = 3
```

gcc bu hatayı `-Wall` ile (gcc 12'den beri) **derlerken** yakalayıp `infinite recursion detected` uyarısı verir; her zaman yakalayamaz, uyarıya güvenme. Program ilk üç çağrıdan sonra sessizce çerçeve açmaya devam eder, yığın dolar ve çöker. `Segmentation fault` yazısını Linux ve macOS terminali yazar; Windows'ta program çoğu zaman hiçbir şey yazmadan kapanır (arka planda "stack overflow" hatası). Bu, bilerek gösterebildiğimiz, her çalıştırmada aynı biçimde ortaya çıkan bir çökmedir.

İkinci tür hata temel durum **var ama ulaşılamıyor**:

```c
int topla(int n) {
    if (n == 0) {                  // negatif n bu koşula hiç ulaşamaz
        return 0;
    }
    return n + topla(n - 1);       // topla(-1): -1, -2, -3, ... sonsuza gider
}
```

● Sık Yapılan Hata: Temel durumu == ile yazmak

`topla(-1)` çağrısı `n == 0` koşuluna hiç ulaşamaz: `-1`, `-2`, `-3` diye gider ve yığın taşar. Girdinin negatif olabileceği her yerde temel durumu `n <= 0` yaz (ya da girdiyi önceden denetle). Aynı hata `faktoriyel(-1)` ve `n == 1` ile biten fonksiyonlarda `n = 0` gönderince de olur.

Doğru yazılmış fonksiyon bile yeterince derin çağrılırsa yığını doldurabilir: `topla(1000000)` gibi bir çağrı, kitaptaki gibi `-O0` ile derlenen programda çoğu sistemde çöker (sınır sisteme ve çerçeve boyutuna bağlıdır). Bu yüzden milyonlarca adım süren işler döngüyle yazılır.

13.11 Özyineleme mi, döngü mü?

Her özyinelemeli çözüm döngüye (gerekirse kendi yığınınla birlikte) çevrilebilir. Seçim, hız, bellek ve okunabilirlik içindir:

Ölçüt	Döngü	Özyineleme
Bellek	Sabit (birkaç değişken)	Derinlik kadar çerçeve; çok derinse yığın taşar
Hız	Genellikle biraz daha hızlı	Her çağrının çerçeve açma/kapama maliyeti vardır; `fib` gibi yerlerde tekrar eden hesap üstel yavaşlık yaratır
Okunabilirlik	Doğrusal işlerde (toplam, faktöriyel, dizi gezme) daha doğal	Problem kendine benzeyen parçalara bölünüyorsa (Hanoi, ikili arama, liste ve ağaç gezme) çok daha kısa ve net
Tipik hata	Sonsuz döngü	Temel durum eksikse yığın taşması

Pratik kural: İş **doğrusal** ise (1'den `n`'e topla, dizi gez) döngü. Problem aynı türden **alt problemlere** bölünüyorsa (Hanoi, ağaç) ya da çözüm doğal olarak özyinelemeli tanımlanmışsa (Fibonacci, EBOB) özyineleme. Sınavda "özyinelemeyle yaz" dendiyse döngü kullanma.

Kuyruk özyinelemesi (tail recursion)

Özyinelemeli çağrı fonksiyonun **son işi** ise (çağrıdan sonra çarpma, toplama gibi bir şey yapılmıyorsa) buna **kuyruk özyinelemesi** denir. `ebob` ve `ikiliAra` böyledir: `return ebob(b, a % b);`. `faktoriyel` böyle değildir: `n * faktoriyel(n - 1)` ifadesinde çağrıdan **sonra** çarpma yapılır. Sonucu fazladan bir parametrede biriktirirsen (`faktoriyelKuyruk(int n, int biriken)`; temel durumda `biriken` döner, diğer durumda `return faktoriyelKuyruk(n - 1, n * biriken);` yazılır, ilk çağrı `faktoriyelKuyruk(5, 1)`) faktöriyel de kuyruk özyinelemeli olur.

Kuyruk çağrısında çağıran çerçevenin işi bittiği için derleyici `-O2` gibi optimizasyonlarla çerçeveyi yeniden kullanıp bunu döngüye çevirebilir; ama C standardı bunu **garanti etmez** ve kitaptaki `-O0` derlemede çerçeve yine açılır. Sınav için yeter bilgi: tanımı ve `ebob`'un kuyruk özyinelemeli olduğunu bil.

Özyinelemeyi Bölüm 14'te yeniden göreceksin: bağlı listeyi özyinelemeyle gezmek ve yazdırmak.

Bölüm Özeti – Bilmen Gerekenler

- **Özyineleme**: fonksiyonun kendini çağırması. Her özyinelemeli fonksiyonda **temel durum** (kendini çağırmadan cevap) ve **özyinelemeli durum** (daha küçük problemle kendini çağırma) vardır; her çağrı temel duruma **yaklaşmalıdır**.
- Her çağrı kendi **yığın çerçevesini** açar; parametreler ve yerel değişkenler çağrıya özeldir. Yığındaki çerçeve sayısı = özyineleme derinliği. Hesaplamalar genellikle **dönüşte** yapılır (`n * faktoriyel(n - 1)`).
- Kalıplar: `topla(n) = n + topla(n-1)`; `basamakToplami(n) = n % 10 + basamakToplami(n / 10)`; `ebob(a, b) = ebob(b, a % b)`, `b == 0` iken `a`; `us(a, n) = a * us(a, n-1)`; **hızlı üs** `n/2` ile `log n` çağrı (yarım sonucu **bir kez** hesapla).
- **Fibonacci** özyinelemeli: iki çağrı, çağrı ağacı, aynı değerler tekrar hesaplanır; çağrı sayısı üstel (`C(n) = 2·fib(n+1) - 1`: 1, 1, 3, 5, 9, 15, 25). **Bellekleme** (dizide sakla) bunu `2n - 1` çağrıya indirir.

- Dizi/string: ilk `n` elemana ya da `s + 1`'e indirge. Boş dizi/boş string (`n == 0`, `*s == '\0'`) temel durumdur; en büyük için `n == 1`. Palindromda iki uç karşılaştırılır.
- **Hanoi**: `n − 1` diski yedeğe, en büyüğü hedefe, `n − 1` diski hedefe. Hamle sayısı $2^n - 1$.
- **Özyinelemeli ikili arama**: temel durum `alt > ust` (`−1`) ve `a[orta] == aranan`; aralık argümanla daralır; çağrıyı `return` et.
- **Çağrıdan önce yazdırma** gidişte (sıra: büyükten küçüğe), **sonra yazdırma** dönüşte (ters sıra) çalışır. İki çağrılı fonksiyonlarda çağrıları sırayla aç.
- Temel durum yok ya da ulaşılamıyor → **yığın taşması** (Linux/macOS'ta `Segmentation fault`, Windows'ta sessiz kapanma). `n == 0` yerine `n <= 0` yaz. Döngü: sabit bellek, genelde hızlı; özyineleme: yığın kullanır ama kendine benzeyen problemlerde kısa. **Kuyruk özyinelemesi**: çağrı son iştir (`ebob`).

Sorular

A) Çoktan Seçmeli

1 `n >= 0` için 1'den `n`'e kadar sayıların toplamını (`topla(0) = 0`) doğru hesaplayan ve sonsuz özyinelemeye girmeyen fonksiyon aşağıdakilerden hangisidir?

(A) `int topla(int n) { if (n <= 0) return 0; return n + topla(n - 1); }`
(B) `int topla(int n) { return n + topla(n - 1); }`
(C) `int topla(int n) { if (n == 1) return 1; return n + topla(n - 1); }`
(D) `int topla(int n) { if (n == 0) return 0; return n + topla(n + 1); }`
(E) `int topla(int n) { if (n > 0) return 0; return n + topla(n - 1); }`

2 `int fib(int n) { if (n < 2) return n; return fib(n - 1) + fib(n - 2); }` fonksiyonu `fib(4)` ile çağrılırsa `fib` ilk çağrı dahil toplam kaç kez çağrılır?

(A) 3 (B) 9 (C) 11 (D) 15 (E) 25

3 Temel durumu `n <= 1` olan, `faktoriyel(n)` fonksiyonuna `faktoriyel(5)` ile başlanıyor. Çağrı yığınında aynı anda **en çok** kaç `faktoriyel` çerçevesi bulunur? (`main`'in çerçevesi sayılmaz.)

(A) 1 (B) 4 (C) 5 (D) 6 (E) 120

4 Üç çubuklu Hanoi kulelerinde 4 diski bir çubuktan diğerine, kurallara uyarak taşımak için **en az** kaç hamle gerekir?

(A) 8 (B) 12 (C) 14 (D) 15 (E) 16

5 Özyineleme ve döngü ile ilgili aşağıdakilerden hangisi **doğrudur**?

(A) Bazı problemler yalnızca özyinelemeyle çözülebilir; döngüyle yazılamaz.

(B) Özyinelemeli çözüm, aynı işi yapan döngüden her zaman daha hızlıdır.

(C) Bir özyinelemeli çağrı bitince yerel değişkenlerinin değerleri sonraki çağrıya aktarılır.

(D) Temel durumu olmayan özyinelemeli fonksiyon derleme hatası verir.

(E) Her çağrı yığında bir çerçeve açtığından çok derin özyineleme yığın taşmasıyla programı çökertebilir; aynı işi yapan döngü ek çerçeve açmaz.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

void yaz(int n) {
    if (n <= 0) {
        return;
    }
    printf("%d ", n);
    yaz(n - 2);
    printf("%d ", n + 1);
}

int main(void) {
    yaz(5);
    printf("\n");
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır? (`static` değişkeni unutma.)

```c
#include <stdio.h>

int f(int n) {
    static int cagri = 0;

    cagri++;
    if (n <= 1) {
        return cagri;
    }
    return f(n - 1) + n;
}

int main(void) {
    printf("%d\n", f(4));
    printf("%d\n", f(2));
    return 0;
}
```

8 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

void f(int n) {
    if (n <= 0) {
        return;
    }
    f(n - 1);
    printf("%d ", n);
    f(n - 2);
}

int main(void) {
    f(4);
    printf("\n");
    return 0;
}
```

C) Kod Yazma

9 Klavyeden pozitif bir tamsayı okuyan `rakamlar` programını yaz. İki **özyinelemeli** fonksiyon kullan (döngü yok): `void rakamYaz(int n)` rakamları soldan sağa, aralarında birer boşlukla yazdırsın; `int basamakSayisi(int n)` basamak sayısını döndürsün. Sayı en fazla 9 basamaklıdır.

```
Sayi: 2749
Rakamlar: 2 7 4 9
Basamak sayisi: 4
```

10 Klavyeden boşluksuz bir sözcük (en fazla 49 karakter) ve ardından bir harf okuyan `harfSay` programını yaz. Özyinelemeli `int kacKez(const char *s, char harf)` fonksiyonu harfin sözcükte kaç kez geçtiğini döndürsün (döngü yok). Sonucu aşağıdaki gibi yazdır.

```
Sozcuk ve harf: ananas a
a harfi 3 kez geciyor.
```

Çözümler

Cevap anahtarı

1 A	2 B	3 C	4 D	5 E

1 Cevap: A

Doğru fonksiyonun temel durumu `n <= 0` olmalı ve çağrı temel duruma yaklaşmalı. A bunu yapar: `n = 0` için `0` döner, pozitif `n` için `n - 1`'e iner. B'de **temel durum yok**: yığın taşar. C'de temel durum `n == 1`'dir; `n = 0` ile çağırınca `0, -1, -2, ...` diye temel duruma hiç ulaşılmaz. D'de temel durum var ama argüman `n + 1` ile **temel durumdan uzaklaşır**. E'de koşul ters yazılmış: `n > 0` için hemen `0` döner (yanlış toplam), `n = 0` için ise `topla(-1)`'e girer ve sonsuz iner.

2 Cevap: B

Çağrı sayısı `C(n) = 1 + C(n-1) + C(n-2)` ve `C(0) = C(1) = 1`: `C(2) = 3`, `C(3) = 5`, `C(4) = 5 + 3 + 1 = 9`. Ağaçta `fib(4)` → `fib(3)`, `fib(2)`; `fib(3)` → `fib(2)`, `fib(1)`; her `fib(2)` iki çağrı daha açar. Toplam 9 düğüm. A (3) `fib(4)`'ün **değeridir**, çağrı sayısı değil. C (11) bir tahmindir. D (15) `fib(5)`'in çağrı sayısıdır (ya da `2⁴ - 1`'dir, tam ağaç değil). E (25) `fib(6)`'nın çağrı sayısıdır.

3 Cevap: C

Yığın `n = 5, 4, 3, 2, 1` çerçeveleriyle en derin hâle ulaşır; `n = 1` temel durumdur ve kendini çağırmadan döner. Yani 5 çerçeve. A (1) her çağrının bitmeden yenisini açtığını unutur. B (4), temel durum çerçevesini saymayı unutmaktır. D (6), `main`'i ya da `n = 0` çerçevesini fazladan sayar; soru `main`'i dışarıda bırakıyor ve temel durum `n <= 1` olduğu için `n = 0` çağrısı yapılmaz. E (120) sonuçtur, çerçeve sayısı değil.

4 Cevap: D

`H(n) = 2ⁿ - 1` olduğundan `H(4) = 2⁴ - 1 = 15`. Adım adım: `H(1) = 1`, `H(2) = 2·1 + 1 = 3`, `H(3) = 2·3 + 1 = 7`, `H(4) = 2·7 + 1 = 15`. A (8) `2³`, B (12) rastgele bir tahmin, C (14) `2⁴ - 2`, E (16) ise `2⁴`'tür; hepsinde "−1" hatalı ya da unutulmuştur.

5 Cevap: E

Doğrusu E: Yığın sınırlıdır; derinliği milyonlara varan özyineleme taşmaya yol açar, döngü ise aynı yerde değişkenleri güncelleyerek sabit bellekle çalışır. A yanlış: **her** özyinelemeli çözüm döngüyle de yazılabilir (gerekirse kendi yığınınla). B yanlış: Çağrı maliyeti yüzünden genelde döngü biraz daha hızlıdır; Fibonacci'de özyineleme üstel yavaşlar. C yanlış: Her çağrının yerel değişkenleri kendisinindir, çağrı bitince silinir (bilgi yalnızca `return` değeri ve argümanlarla taşınır). D yanlış: Temel durumsuz fonksiyon **derlenir** (gcc en fazla uyarı verir); hata çalışma zamanında yığın taşması olarak çıkar.

6

Her çağrının `n`'ini yaz; `printf`'ler çağrının önünde ve arkasında:

Çağrı	Önce yazar	Çağırır	Sonra yazar (`n + 1`)
`yaz(5)`	`5`	`yaz(3)`	`6`
`yaz(3)`	`3`	`yaz(1)`	`4`
`yaz(1)`	`1`	`yaz(-1)`: `n <= 0`, hemen döner	`2`

Gidişte `5 3 1` yazılır. `yaz(-1)` temel durumdur ve hiçbir şey yazmaz. Dönüşte sırayla `yaz(1)` `2`, `yaz(3)` `4`, `yaz(5)` `6` yazar. Tuzaklar: Dönüşte yazılan sayı `n` değil `n + 1`'dir; çağrı `n - 2` ile indiği için gidişte yalnız tek sayılar (`5 3 1`) görünür.

Çıktı
```
5 3 1 2 4 6
```

7

`cagri` `static` olduğu için **bütün çağrılarda ortaktır** ve `main`'deki ikinci çağrıda da kalan değerle devam eder.

1. `f(4)`: `cagri = 1`, `f(3)`: `cagri = 2`, `f(2)`: `cagri = 3`, `f(1)`: `cagri = 4`; temel durum, `4` döner.
2. Dönüşte: `f(2) = 4 + 2 = 6`, `f(3) = 6 + 3 = 9`, `f(4) = 9 + 4 = 13`. İlk satır `13`.
3. `f(2)` yeniden çağrılır: `cagri` `5` olur; `f(1)`: `cagri = 6`, `6` döner. `f(2) = 6 + 2 = 8`. İkinci satır `8`.

Tuzak: `cagri` her çağrıda sıfırlanmaz ve temel durum `n` yerine `cagri`'yi döndürür.

Çıktı
```
13
8
```

8

`f(n)`: önce `f(n-1)`, sonra `n` yaz, sonra `f(n-2)`. Alt çağrıları sırayla aç:

- `f(1)`: `f(0)` hiçbir şey yazmaz, `1` yazar, `f(-1)` hiçbir şey yazmaz → `1`.
- `f(2)`: `f(1)` = `1`, sonra `2`, sonra `f(0)` (boş) → `1 2`.
- `f(3)`: `f(2)` = `1 2`, sonra `3`, sonra `f(1)` = `1` → `1 2 3 1`.
- `f(4)`: `f(3)` = `1 2 3 1`, sonra `4`, sonra `f(2)` = `1 2` → `1 2 3 1 4 1 2`.

Tuzak: `f(n - 2)` çağrısı `printf`'ten sonradır; son parça (`1 2`) `f(2)`'nin kendi çıktısıdır.

Çıktı
```
1 2 3 1 4 1 2
```

9

`rakamYaz`'da yazdırma çağrıdan **sonra** olmalı: önce `n / 10` (soldaki rakamlar) yazılır, sonra `n % 10`. İki rakam arasındaki boşluk için, `n >= 10` ise soldaki rakamlar yazıldıktan sonra bir boşluk basılır. `basamakSayisi` temel durumu tek basamaklı sayıdır (`n < 10`).

rakamlar.c
```c
#include <stdio.h>

void rakamYaz(int n) {
    if (n >= 10) {
        rakamYaz(n / 10);
```

```
        printf(" ");
    }
    printf("%d", n % 10);
}

int basamakSayisi(int n) {
    if (n < 10) {
        return 1;
    }
    return 1 + basamakSayisi(n / 10);
}

int main(void) {
    int sayi = 0;

    printf("Sayi: ");
    if (scanf("%d", &sayi) != 1 || sayi <= 0) {
        printf("Gecersiz sayi.\n");
        return 1;
    }
    printf("Rakamlar: ");
    rakamYaz(sayi);
    printf("\nBasamak sayisi: %d\n", basamakSayisi(sayi));
    return 0;
}
```

Çıktı

```
Sayi: 2749
Rakamlar: 2 7 4 9
Basamak sayisi: 4
```

10

Temel durum string'in sonudur (`*s == '\0'`). Aksi hâlde ilk karakter harfe eşitse `1` ekle ve kalan string'de say (`s + 1`); eşit değilse yalnızca kalanı say. `scanf`'te `%49s` taşmayı önler, `%c`'den önceki boşluk araya giren boşluğu atlar (Bölüm 3).

harfSay.c

```
#include <stdio.h>

int kacKez(const char *s, char harf) {
    if (*s == '\0') {
        return 0;
    }
    if (*s == harf) {
        return 1 + kacKez(s + 1, harf);
    }
    return kacKez(s + 1, harf);
}

int main(void) {
    char sozcuk[50];
    char harf = ' ';

    printf("Sozcuk ve harf: ");
    if (scanf("%49s %c", sozcuk, &harf) != 2) {
        printf("Girdi okunamadi.\n");
        return 1;
```

```
    }
    printf("%c harfi %d kez geciyor.\n", harf, kacKez(sozcuk, harf));
    return 0;
}
```

Çıktı

```
Sozcuk ve harf: ananas a
a harfi 3 kez geciyor.
```

BÖLÜM 14

Veri Yapılarına Giriş: Bağlı Liste, Yığın ve Kuyruk

Bu Bölümde Öğreneceklerin

- Veri yapısının ne olduğunu ve dizinin hangi sınırları yüzünden başka yapılara ihtiyaç duyulduğunu
- Öz-referanslı yapıyı (`struct Dugum *sonraki`) ve düğüm (node) kavramını
- Tek yönlü bağlı listede düğüm oluşturmayı, bağlamayı, dolaşmayı ve aramayı
- Başa, sona ve sıralı eklemeyi; baştan ve değere göre silmeyi; listeyi sızıntısız serbest bırakmayı
- Liste başını fonksiyonda değiştirmek için neden `Dugum **bas` gerektiğini
- Yığını (stack, LIFO) dizi ve bağlı listeyle; parantez dengesi denetimini
- Kuyruğu (queue, FIFO) dairesel dizi ve bağlı listeyle; bilet simülasyonunu
- Dizi ile bağlı listenin işlem maliyetlerini (O(1), O(n)) karşılaştırmayı

14.1 Neden veri yapısı?

Bölüm 7'de diziyi öğrendin: elemanlar bellekte yan yana durur, `dizi[i]` ile istediğin elemana tek adımda ulaşırsın. Ama dizinin üç sınırı var:

Sınır	Sebep
Boyut sabittir	Kaç eleman geleceğini baştan bilmen gerekir. Fazla ayırırsan bellek boşa gider, az ayırırsan taşar
Başa ve araya ekleme/silme pahalıdır	Boşluk açmak (ya da kapatmak) için sonraki elemanların **hepsi** kaydırılır. 1000 elemanlı dizinin başına eleman eklemek 1000 kaydırma demektir
Büyütmek kopyalama demektir	`realloc` (Bölüm 10) çoğu zaman daha büyük yeni bir blok ayırıp eski elemanların hepsini oraya kopyalar

Veri yapısı (data structure), veriyi bellekte düzenleme biçimi ile o düzen üzerindeki işlemlerin (ekle, sil, ara) toplamıdır. Dizi de bir veri yapısıdır; bu bölümdekiler zayıf yanlarını giderir, karşılığında başka bedeller ister. **Bağlı liste** (linked list) elemanları dağınık tutup her elemana bir sonrakinin adresini yükler. **Yığın** (stack) son giren ilk çıkar (LIFO), **kuyruk** (queue) ilk giren ilk çıkar (FIFO) kuralıdır; ikisi de dizi ya da bağlı liste üzerine kurulabilir. Bu yüzden önce bağlı listeyi öğreneceğiz.

14.2 Öz-referanslı yapı ve düğüm

Bağlı listenin her elemanı bir **düğümdür** (node): **veri** ve bir sonraki düğümün **adresi**. Bölüm 11'de gördüğün gibi bir yapı kendi türünden bir üye taşıyamaz (boyutu sonsuz olurdu), ama kendi türünden bir **işaretçi** taşıyabilir. Böyle yapıya **öz-referanslı yapı** denir. Alışkanlıkla şöyle yazmak istersin:

yanlisDugum.c

```c
typedef struct {
    int veri;
    Dugum *sonraki;     // HATA: Dugum adı henüz yok
} Dugum;

int main(void) {
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
yanlisDugum.c:3:5: error: unknown type name 'Dugum'
    3 |     Dugum *sonraki;     // HATA: Dugum adı henüz yok
      |     ^~~~~
```

`typedef`'in verdiği kısa ad `}` ile `;` arasında doğar; yapının gövdesi yazılırken `Dugum` adı henüz yoktur. Çözüm: yapıya bir **etiket** (tag) ver ve içeride `struct Dugum *` yaz:

```c
typedef struct Dugum {          // Dugum: yapının etiketi
    int veri;
    struct Dugum *sonraki;      // içeride etiket kullanılır
} Dugum;                        // dışarıda kısa ad: Dugum
```

Kitabın geri kalanında düğüm hep budur. Son düğümün `sonraki`'si `NULL`'dır ve "liste burada biter" demektir. Listeye girişi tek bir işaretçi sağlar: **baş işaretçisi** `bas`. Boş listede `bas == NULL` olur.

14.3 Düğüm oluşturma, bağlama ve dolaşma

Düğümler `malloc` ile öbekte (heap) yaratılır. Bunu bir fonksiyona koymak işi kısaltır: `dugumOlustur` `malloc`'u kontrol eder (Bölüm 10), `sonraki`'ni `NULL` yapar ve düğümün adresini döndürür. Öbekteki blok fonksiyon bitse de yaşar. Üç düğümü elle şöyle bağlarsın: `bas->sonraki = ikinci;` ve `ikinci->sonraki = ucuncu;`. Bellekteki durum:

```
  bas
   |
   v
+------+-------+      +------+-------+      +------+--------+
|  10  |   o---+----->|  20  |   o---+----->|  30  |  NULL  |
+------+-------+      +------+-------+      +------+--------+
  veri  sonraki         ^ ikinci              ^ ucuncu
```

Her kutunun sağ yarısı bir **adres** taşır; ok "o adresteki düğüme git" demektir. `bas->sonraki` ikinci düğümü, `bas->sonraki->sonraki->veri` üçüncü düğümün verisini verir. Zincirin ortasında `NULL` varsa ondan ötesini kullanmak programı çökertir (Bölüm 9).

Dolaşmanın kalıbı tektir: bir gezgin işaretçi `bas`'tan başlar, her turda bir düğüm ilerler ve `NULL`'a gelince durur:

```c
for (Dugum *p = bas; p != NULL; p = p->sonraki) { ... p->veri ... }
```

Gezgin olarak **ayrı bir** `p` kullan; `bas`'ı kaydırırsan listenin başını kaybedersin. Aşağıdaki program üç düğümü kurar, dolaşarak yazdırır, sayar, toplar, arar ve ters sırada yazdırır:

liste.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

Dugum *dugumOlustur(int veri) {
    Dugum *d = malloc(sizeof(Dugum));
    if (d != NULL) {
        d->veri = veri;
        d->sonraki = NULL;
    }
    return d;
}

void yazdir(const Dugum *bas) {
    for (const Dugum *p = bas; p != NULL; p = p->sonraki) {
        printf("%d -> ", p->veri);
    }
    printf("NULL\n");
}

Dugum *ara(Dugum *bas, int aranan) {
    for (Dugum *p = bas; p != NULL; p = p->sonraki) {
        if (p->veri == aranan) {
            return p;
        }
    }
    return NULL;
}

void tersYazdir(const Dugum *p) {
    if (p == NULL) {
        return;
    }
    tersYazdir(p->sonraki);
    printf("%d ", p->veri);
}

int main(void) {
    Dugum *bas = dugumOlustur(10);
    Dugum *ikinci = dugumOlustur(20);
    Dugum *ucuncu = dugumOlustur(30);
    if (bas == NULL || ikinci == NULL || ucuncu == NULL) {
        free(bas);
        free(ikinci);
        free(ucuncu);
        return 1;
    }
    bas->sonraki = ikinci;
    ikinci->sonraki = ucuncu;

    printf("%d %d\n", bas->sonraki->veri, bas->sonraki->sonraki->veri);
    yazdir(bas);
    int adet = 0;
    int toplam = 0;
    for (Dugum *p = bas; p != NULL; p = p->sonraki) {
        adet++;
        toplam += p->veri;
    }
    printf("Eleman sayısı: %d, toplam: %d\n", adet, toplam);
```

```
    Dugum *bulunan = ara(bas, 20);
    if (bulunan != NULL) {
        printf("20 bulundu, sonraki: %d\n", bulunan->sonraki->veri);
    }
    if (ara(bas, 99) == NULL) {
        printf("99 listede yok\n");
    }
    tersYazdir(bas);
    printf("\n");

    free(ucuncu);
    free(ikinci);
    free(bas);
    return 0;
}
```

Çıktı

```
20 30
10 -> 20 -> 30 -> NULL
Eleman sayısı: 3, toplam: 60
20 bulundu, sonraki: 30
99 listede yok
30 20 10
```

`ara`, bulduğu düğümün **adresini** döndürür, bulamazsa `NULL` döndürür; çağıran kullanmadan önce `NULL` kontrolü yapmak zorundadır. `free(NULL)` güvenli olduğu için hata dalındaki üç `free` rahatça yazılır.

`tersYazdir` özyinelemelidir (Bölüm 13): liste ya boştur (`NULL`, temel durum) ya da bir düğüm ile geri kalan bir listedir. Önce kalan listeyi yazdırır, dönüşte kendi verisini yazar; böylece listeyi ek bir yapı olmadan sondan başa yazdırırsın. `yazdir`'in özyinelemeli eşi sıralamayı değiştirmez: önce `printf`, sonra `p->sonraki` ile çağrı. Her düğüm için bir çağrı yığında birikir, bu yüzden çok uzun listelerde döngü tercih edilir.

> **Sık Yapılan Hata: `p->sonraki != NULL` ile dolaşmak**
>
> `while (p->sonraki != NULL)` döngüsü **son düğümde durur**, son düğümü işlemeden çıkar; boş listede (`p == NULL`) ise `p->sonraki` ifadesi `NULL`'ı kullanır ve program çöker. Her düğümü işlemek için koşul `p != NULL` olmalıdır. Son düğüme **varmak** istediğin yerde (sona ekleme) `p->sonraki != NULL` doğrudur; hangisini neden kullandığını bil.

14.4 Tüm listeyi serbest bırakma

Her `malloc` için bir `free` gerekir (Bölüm 10); liste için bu, **her düğümü** serbest bırakmak demektir. İlk akla gelen döngü yanlıştır:

```
for (Dugum *p = bas; p != NULL; p = p->sonraki) {
    free(p);                    // TANIMSIZ: sonraki tur p->sonraki'yi okuyacak
}
```

`free(p)` düğümü geri verir; döngü bir sonraki turda `p = p->sonraki` işlemini yapınca **serbest bırakılmış belleği** okur (dangling pointer, Bölüm 10): program çökebilir de, "çalışıyor gibi" görünebilir de. Doğru kalıp: `sonraki`'ni **free'den önce** sakla.

```c
void listeyiSil(Dugum *bas) {
    while (bas != NULL) {
        Dugum *sonraki = bas->sonraki;   // önce sakla
        free(bas);                       // sonra serbest bırak
        bas = sonraki;                   // saklananla devam et
    }
}
```

`bas` parametresi kopya olduğu için çağıranın `bas`'ı artık serbest bırakılmış düğümü gösterir; çağıran gerekirse `bas = NULL;` yazar. Bundan sonraki programlar `listeyiSil` ile biter.

> **Sık Yapılan Hata:** `free(p); p = p->sonraki;`
>
> Listeyi silen döngünün en sık yapılan hatasıdır. `free(p)`'den sonra `p`'nin gösterdiği blok senin değildir. **Önce sonrakini sakla, sonra free et, sonra saklananı al.** Hocalar "listeyi sil" sorusunda bu kalıbı mutlaka arar.

14.5 Ekleme: başa, sona, sıralı

Başa ekleme listenin en ucuz eklemesidir; liste ne kadar uzun olursa olsun üç adımda biter:

```
Önce:     bas -> [20] -> [10] -> NULL
Adım 1:   yeni = dugumOlustur(30)
Adım 2:   yeni->sonraki = bas;     yeni -> [20] -> [10] -> NULL
Adım 3:   bas = yeni;              bas -> [30] -> [20] -> [10] -> NULL
```

Sıra önemlidir. 3. adımı 2.'den önce yaparsan eski listeye giden tek yol kaybolur ve 20 ile 10 sızar.

Asıl soru `bas`'ı kimin değiştireceğidir. Bölüm 6'dan biliyorsun: parametre **kopyadır**. Fonksiyonun içinde `bas = yeni;` yazarsan yalnız kopya değişir. İki çözüm var: (1) fonksiyon yeni başı **döndürür**, çağıran `bas = basaEkle(bas, 10);` yazar; (2) fonksiyon başın **adresini** alır: `Dugum **bas`. Bu, Bölüm 9'daki `swap`'ın bir seviye yukarısıdır; `*bas` çağıranın `bas`'ıdır, çağıran `&bas` geçirir.

Sona ekleme için önce **son düğümü bulmak** gerekir (`sonraki`'si `NULL` olan düğüm); koşul `p->sonraki != NULL`'dır. Liste boşsa yeni düğüm baş olur, bu yüzden `Dugum **` gerekir. Baş işaretçisinin yanında bir de **son işaretçisi** tutarsan aramak gerekmez; bunu kuyrukta kullanacağız.

Sıralı ekleme listeyi küçükten büyüğe sıralı tutar. Liste boşsa ya da yeni değer baştakinden küçükse başa eklenir; değilse `p->sonraki->veri` yeni değerden küçük olduğu sürece ilerlenir ve `p`, yeni düğümden hemen önceki düğümde durur. Bağlama sırası: önce `yeni->sonraki = p->sonraki;`, sonra `p->sonraki = yeni;`.

Programda beş fonksiyon var. `basaEkleYanlis` bilerek **yanlıştır**:

ekleme.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

Dugum *dugumOlustur(int veri) {
    Dugum *d = malloc(sizeof(Dugum));
    if (d != NULL) {
        d->veri = veri;
        d->sonraki = NULL;
    }
```

```
    return d;
}

void yazdir(const Dugum *bas) {
    for (const Dugum *p = bas; p != NULL; p = p->sonraki) {
        printf("%d -> ", p->veri);
    }
    printf("NULL\n");
}

void listeyiSil(Dugum *bas) {
    while (bas != NULL) {
        Dugum *sonraki = bas->sonraki;
        free(bas);
        bas = sonraki;
    }
}

Dugum *basaEkle(Dugum *bas, int veri) {          // yeni başı döndürür
    Dugum *yeni = dugumOlustur(veri);
    if (yeni == NULL) {
        return bas;
    }
    yeni->sonraki = bas;
    return yeni;
}

void basaEkle2(Dugum **bas, int veri) {          // başın adresini alır
    Dugum *yeni = dugumOlustur(veri);
    if (yeni != NULL) {
        yeni->sonraki = *bas;
        *bas = yeni;
    }
}

void basaEkleYanlis(Dugum *bas, int veri) {      // YANLIŞ: kopya değişir
    Dugum *yeni = dugumOlustur(veri);
    if (yeni != NULL) {
        yeni->sonraki = bas;
        bas = yeni;
    }
}

int sonaEkle(Dugum **bas, int veri) {
    Dugum *yeni = dugumOlustur(veri);
    if (yeni == NULL) {
        return 0;
    }
    if (*bas == NULL) {                  // boş liste: yeni düğüm baş olur
        *bas = yeni;
        return 1;
    }
    Dugum *p = *bas;
    while (p->sonraki != NULL) {         // son düğümü bul
        p = p->sonraki;
    }
    p->sonraki = yeni;
    return 1;
}

int siraliEkle(Dugum **bas, int veri) {
    Dugum *yeni = dugumOlustur(veri);
    if (yeni == NULL) {
        return 0;
    }
```

```c
    if (*bas == NULL || veri < (*bas)->veri) {   // başa eklenecek
        yeni->sonraki = *bas;
        *bas = yeni;
        return 1;
    }
    Dugum *p = *bas;
    while (p->sonraki != NULL && p->sonraki->veri < veri) {
        p = p->sonraki;
    }
    yeni->sonraki = p->sonraki;          // önce yeni düğümü sonrakine bağla
    p->sonraki = yeni;                   // sonra öncekini yeni düğüme bağla
    return 1;
}

int main(void) {
    Dugum *a = NULL, *b = NULL, *c = NULL, *d = NULL, *s = NULL;
    int degerler[] = {40, 10, 30, 20, 30};
    for (int i = 1; i <= 3; i++) {
        a = basaEkle(a, i * 10);
        basaEkle2(&b, i * 10);
        basaEkleYanlis(c, i * 10);
        sonaEkle(&d, i * 10);
    }
    for (int i = 0; i < 5; i++) {
        siraliEkle(&s, degerler[i]);
    }
    printf("basaEkle      : ");
    yazdir(a);
    printf("basaEkle2     : ");
    yazdir(b);
    printf("basaEkleYanlis: ");
    yazdir(c);
    printf("sonaEkle      : ");
    yazdir(d);
    printf("siraliEkle    : ");
    yazdir(s);
    listeyiSil(a);
    listeyiSil(b);
    listeyiSil(d);
    listeyiSil(s);
    return 0;
}
```

Çıktı

```
basaEkle      : 30 -> 20 -> 10 -> NULL
basaEkle2     : 30 -> 20 -> 10 -> NULL
basaEkleYanlis: NULL
sonaEkle      : 10 -> 20 -> 30 -> NULL
siraliEkle    : 10 -> 20 -> 30 -> 30 -> 40 -> NULL
```

`10, 20, 30` sırasıyla başa eklenenler **ters sırada** listelenir: son eklenen en öndedir. `sonaEkle` ise eklenme sırasını korur. `basaEkleYanlis`'te `c` hiç değişmedi: içerideki `bas` kopyadır. Üstelik **bellek sızıntısı** da vardır: yeni düğümü gösteren tek işaretçi fonksiyon bitince yok olur ve üç düğüme bir daha ulaşılamaz. Program yanlışı göstermek için bu üç düğümü bilerek serbest bırakmıyor; gerçek kodda böyle bırakma. Sıralı ekleme yinelenen değeri de kabul eder (iki `30`); koşulun `<` ya da `<=` olması yalnızca eşit elemanların kendi arasındaki sırasını etkiler.

> **Sık Yapılan Hata: Liste başını `Dugum *` ile değiştirmeye çalışmak**
>
> Liste başını değiştiren her fonksiyon (başa ekleme, baştan silme, boş listeye ekleme) ya yeni başı döndürmeli ya da `Dugum **` almalıdır. `Dugum *bas` alan fonksiyon başı değiştiremez, yalnızca **düğümlerin içeriğini** değiştirebilir. Derleyici hata vermez, program sessizce yanlış çalışır. Kural: Hangi değişkeni değiştirmek istiyorsan **onun adresini** geçir; değişken `Dugum *` ise adres `Dugum **` olur.

14.6 Silme

Silmenin kuralı: **Düğümü `free` etmeden önce listeden çıkar ve `sonraki`'ni kaybetme.** Düğümü çıkarmak için **ondan önceki** düğümün `sonraki`'sini değiştirmen gerekir; tek yönlü listede geri gidemediğin için silerken hep **önceki** düğümü tutarsın.

- **Baştan silme:** Önceki düğüm yoktur; `bas` ikinci düğüme taşınır (`Dugum **` gerekir). Silinecek düğümü önce bir değişkende sakla, `bas`'ı ilerlet, sonra `free` et.
- **Değere göre silme:** Değer baştaysa baştan silme çağrılır. Değilse `onceki->sonraki->veri` kontrol edilerek ilerlenir; bulununca `onceki->sonraki = silinecek->sonraki;` yazılır ve düğüm `free` edilir.
- **Bulunamazsa:** Hiçbir şey değişmez, fonksiyon `0` döndürür.

silme.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

Dugum *basaEkle(Dugum *bas, int veri) {
    Dugum *yeni = malloc(sizeof(Dugum));
    if (yeni == NULL) {
        return bas;
    }
    yeni->veri = veri;
    yeni->sonraki = bas;
    return yeni;
}

void yazdir(const Dugum *bas) {
    for (const Dugum *p = bas; p != NULL; p = p->sonraki) {
        printf("%d -> ", p->veri);
    }
    printf("NULL\n");
}

void listeyiSil(Dugum *bas) {
    while (bas != NULL) {
        Dugum *sonraki = bas->sonraki;
        free(bas);
        bas = sonraki;
    }
}

int basiSil(Dugum **bas) {
    if (*bas == NULL) {
        return 0;
    }
    Dugum *silinecek = *bas;
    *bas = silinecek->sonraki;
```

```
    free(silinecek);
    return 1;
}

int sil(Dugum **bas, int deger) {
    if (*bas == NULL) {
        return 0;
    }
    if ((*bas)->veri == deger) {
        return basiSil(bas);
    }
    Dugum *onceki = *bas;
    while (onceki->sonraki != NULL && onceki->sonraki->veri != deger) {
        onceki = onceki->sonraki;
    }
    if (onceki->sonraki == NULL) {
        return 0;                           // bulunamadı
    }
    Dugum *silinecek = onceki->sonraki;
    onceki->sonraki = silinecek->sonraki;   // silinecek düğümü atla
    free(silinecek);
    return 1;
}

int main(void) {
    Dugum *bas = NULL;
    for (int i = 4; i >= 1; i--) {
        bas = basaEkle(bas, i * 10);
    }
    yazdir(bas);
    printf("sil(20): %d\n", sil(&bas, 20));
    printf("sil(10): %d\n", sil(&bas, 10));
    printf("sil(40): %d\n", sil(&bas, 40));
    yazdir(bas);
    printf("sil(99): %d\n", sil(&bas, 99));
    printf("basiSil: %d\n", basiSil(&bas));
    printf("basiSil: %d\n", basiSil(&bas));
    yazdir(bas);
    listeyiSil(bas);
    return 0;
}
```

Çıktı

```
10 -> 20 -> 30 -> 40 -> NULL
sil(20): 1
sil(10): 1
sil(40): 1
30 -> NULL
sil(99): 0
basiSil: 1
basiSil: 0
NULL
```

Üç farklı konumdaki silme çalıştı: ortadaki (`20`), baştaki (`10`), sondaki (`40`). `99` bulunamadığı için `0` döndü ve liste değişmedi. İki `basiSil`'den ilki kalan `30`'u sildi, ikincisi **boş listede** `0` döndürdü; boş listeyi kontrol etmek şarttır, yoksa `(*bas)->sonraki` `NULL` üzerinde çöker. Sonda `listeyiSil` boş listede çağrıldı: `bas == NULL` olduğu için döngü hiç dönmez, zararsızdır.

Sınavda Çıkar: Bağlantı sırası ve `->` zincirleri

Liste sorularında puan **işaretçilerin değiştirilme sırasında** gider. Başa eklemede önce `yeni->sonraki = bas;`, sonra `bas = yeni;`. Silmede önce `onceki->sonraki = silinecek->sonraki;`, sonra `free(silinecek);`. Sırayı ters yazarsan listenin kalanını ya da silinecek düğümün adresini kaybedersin. `p->sonraki->sonraki->veri` gibi zincirleri **soldan sağa** oku ve her adımda işaretçinin `NULL` olup olmadığını kontrol et.

İpucu: Önce kağıda çiz

İşaretçi sorularının neredeyse tamamı bir çizimle çözülür. Düğümleri kutu, `sonraki`'leri ok olarak çiz; `bas`, `p`, `onceki` gibi değişkenleri ayrı etiketler olarak yaz ve her satırda yönü değişen oku silip yeniden çiz. Kod yazarken de önce çiz, sonra yaz.

14.7 Yığın (stack)

Yığın, elemanların **tek ucundan** eklenip çıkarıldığı yapıdır; son eklenen ilk çıkar (**LIFO**). Üst üste konmuş tabaklar ya da editördeki "geri al" geçmişi buna örnektir. Bölüm 10'daki yığın yerel değişkenlerin yaşadığı **bellek bölgesiydi**; buradaki yığın bir **veri yapısıdır**. İkisi de LIFO ile çalıştığı için adı aynıdır, kavramlar ayrıdır. Üç işlem vardır:

İşlem	Anlamı
`push`	Tepeye eleman koyar
`pop`	Tepedeki elemanı çıkarır ve değerini verir
`tepe` (peek)	Tepedeki elemana bakar, çıkarmaz

Boş yığından `pop`, dolu (dizi ile yapılmış) yığına `push` yapılamaz. Fonksiyonlar bu iki durumu kontrol eder ve başarıyı `1`/`0` ile bildirir; çıkan değer adres ile geçirilen bir parametreye (Bölüm 9) yazılır.

Dizi ile yığın

Yığın bir dizi ve `tepe` adlı bir indeksten oluşur. Boşken `tepe` `-1`'dir. `push` önce `tepe`'yi artırır, sonra yazar; `pop` önce okur, sonra `tepe`'yi azaltır. Çıkan elemanın dizideki değeri silinmez, yalnız sahipsiz kalır.

yiginDizi.c

```c
#include <stdio.h>

#define KAPASITE 3

typedef struct {
    int dizi[KAPASITE];
    int tepe;                   // en üstteki elemanın indeksi, boşken -1
} Yigin;

int yiginBosMu(const Yigin *y) {
    return y->tepe == -1;
}

int yiginDoluMu(const Yigin *y) {
    return y->tepe == KAPASITE - 1;
}

int push(Yigin *y, int deger) {
    if (yiginDoluMu(y)) {
        return 0;
```

```
    }
    y->tepe++;
    y->dizi[y->tepe] = deger;
    return 1;
}

int pop(Yigin *y, int *deger) {
    if (yiginBosMu(y)) {
        return 0;
    }
    *deger = y->dizi[y->tepe];
    y->tepe--;
    return 1;
}

int tepeOku(const Yigin *y, int *deger) {
    if (yiginBosMu(y)) {
        return 0;
    }
    *deger = y->dizi[y->tepe];
    return 1;
}

int main(void) {
    Yigin y = { {0}, -1 };
    int deger = 0;
    for (int i = 1; i <= 4; i++) {
        if (push(&y, i * 10)) {
            printf("push(%d)\n", i * 10);
        } else {
            printf("push(%d) başarısız: yığın dolu\n", i * 10);
        }
    }
    if (tepeOku(&y, &deger)) {
        printf("tepe: %d\n", deger);
    }
    while (pop(&y, &deger)) {
        printf("pop -> %d\n", deger);
    }
    printf("boş mu? %d\n", yiginBosMu(&y));
    return 0;
}
```

Çıktı

```
push(10)
push(20)
push(30)
push(40) başarısız: yığın dolu
tepe: 30
pop -> 30
pop -> 20
pop -> 10
boş mu? 1
```

Kapasite 3 olduğu için dördüncü `push` reddedildi. Elemanlar `10, 20, 30` sırasıyla girdi, **ters** sırayla çıktı: LIFO. `Yigin y = { {0}, -1 };` diziyi sıfırlar ve `tepe`'yi `-1` yapar.

Bağlı liste ile yığın

Bağlı listede başa ekleme ve baştan silme zaten O(1)'dir (eleman sayısından bağımsız, sabit adım; ayrıntısı bölümün sonunda); yığın tam olarak budur. Listenin **başı yığının tepesidir**, kapasite sınırı yoktur (sınırı `malloc` koyar). Yukarıdaki `basaEkle2` ve `basiSil` fonksiyonlarının adı değişmiş hâlidir:

```
int push(Dugum **tepe, int deger) {          // = başa ekleme
    Dugum *yeni = malloc(sizeof(Dugum));
    if (yeni == NULL) {
        return 0;
    }
    yeni->veri = deger;
    yeni->sonraki = *tepe;
    *tepe = yeni;
    return 1;
}

int pop(Dugum **tepe, int *deger) {          // = baştan silme
    if (*tepe == NULL) {
        return 0;
    }
    Dugum *silinecek = *tepe;
    *deger = silinecek->veri;
    *tepe = silinecek->sonraki;
    free(silinecek);
    return 1;
}
```

Kullanım: `Dugum *tepe = NULL;` ile boş yığın başlar, `push(&tepe, 10);` ekler, `pop(&tepe, &deger)` çıkarır; yığın boşken `tepe == NULL` olur. `pop` her düğümü serbest bıraktığı için tüm elemanlar çıkarılınca bellek sızıntısı kalmaz.

Uygulama: parantez dengesi

Derleyiciler, `({[]})` gibi bir ifadenin parantezlerinin **dengeli** olup olmadığını yığınla denetler. Kural: Açan parantezi (`(`, `[`, `{`) yığına koy. Kapatan parantez gelince tepedeki açan parantezle **aynı türden** olup olmadığına bak; değilse ya da yığın boşsa ifade dengesizdir. Metin bitince yığın **boş** olmalıdır.

parantez.c

```
#include <stdio.h>

int dengeliMi(const char *s) {
    char yigin[64];
    int tepe = -1;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') {
            if (tepe == 63) {
                return 0;                    // yığın dolu
            }
            yigin[++tepe] = c;               // push
        } else if (c == ')' || c == ']' || c == '}') {
            if (tepe == -1) {
                return 0;                    // kapatacak açan yok
            }
            char acan = yigin[tepe--];       // pop
            if ((c == ')' && acan != '(') ||
                (c == ']' && acan != '[') ||
                (c == '}' && acan != '{')) {
                return 0;                    // tür uyuşmuyor
            }
        }
```

```
    }
    return tepe == -1;                      // açıkta kalan var mı?
}

int main(void) {
    const char *ifadeler[] = {
        "({[]})", "(a + b) * [c - d]", "(]", "((", ")(", "{[(])}"
    };
    for (int i = 0; i < 6; i++) {
        printf("%-20s %s\n", ifadeler[i],
               dengeliMi(ifadeler[i]) ? "dengeli" : "dengesiz");
    }
    return 0;
}
```

Çıktı

```
({[]})              dengeli
(a + b) * [c - d]   dengeli
(]                  dengesiz
((                  dengesiz
)(                  dengesiz
{[(])}              dengesiz
```

Dört dengesizlik türü var: tür uyuşmuyor (`(]`), açan açıkta kaldı (`((`), kapatandan önce açan yok (`)(`) ve iç içe geçme sırası bozuk (`{[(])}`). Sayaç tutmak (kaç `(`, kaç `)` var) ilk ikisini yakalar ama sıra bozukluğunu yakalayamaz; bu yüzden yığın gerekir.

> **Sık Yapılan Hata: Boş yığından `pop`, dolu yığına `push`**
>
> Dizi ile yapılmış yığında `tepe == -1` iken `dizi[tepe]` okumak ya da kapasite doluyken yazmak dizi sınırı dışına çıkmaktır (Bölüm 7): tanımsız davranış. Bağlı liste ile yapılmış yığında boşken `tepe->veri` okumak `NULL` kullanmaktır, program çöker. `push` ve `pop` her zaman önce **dolu mu / boş mu** diye bakmalı ve başarıyı döndürmelidir.

14.8 Kuyruk (queue)

Kuyruk, elemanların **bir uçtan eklenip öbür uçtan çıkarıldığı** yapıdır; ilk giren ilk çıkar (**FIFO**). Banka sırası, yazıcı kuyruğu buna örnektir. `enqueue` elemanı kuyruğun **sonuna** ekler, `dequeue` **başındaki** elemanı çıkarıp değerini verir.

Dairesel dizi ile kuyruk

Diziyle yapmanın saf yolu, her çıkarmada herkesi bir adım kaydırmaktır (pahalı). Daha iyisi iki indeks tutmaktır: `bas` (çıkacak ilk elemanın yeri) ve `son` (**bir sonraki elemanın yazılacağı** yer). `dequeue` `bas`'ı, `enqueue` `son`'u ilerletir. İndeks dizinin sonuna gelince başa sarar: `(indeks + 1) % KAPASITE`. Dizi bir halka gibi düşünüldüğü için buna **dairesel kuyruk** (circular queue) denir. `bas == son` hem boş hem dolu kuyrukta oluşabileceğinden eleman sayısını `adet` ile ayrıca tutarız.

kuyrukDizi.c

```
#include <stdio.h>

#define KAPASITE 4

typedef struct {
    int dizi[KAPASITE];
    int bas;                    // çıkacak ilk elemanın indeksi
```

```c
    int son;                    // bir sonraki elemanın yazılacağı indeks
    int adet;                   // kuyruktaki eleman sayısı
} Kuyruk;

int enqueue(Kuyruk *k, int deger) {
    if (k->adet == KAPASITE) {
        return 0;
    }
    k->dizi[k->son] = deger;
    k->son = (k->son + 1) % KAPASITE;
    k->adet++;
    return 1;
}

int dequeue(Kuyruk *k, int *deger) {
    if (k->adet == 0) {
        return 0;
    }
    *deger = k->dizi[k->bas];
    k->bas = (k->bas + 1) % KAPASITE;
    k->adet--;
    return 1;
}

void durum(const Kuyruk *k) {
    printf("bas=%d son=%d adet=%d | sıra:", k->bas, k->son, k->adet);
    for (int i = 0; i < k->adet; i++) {
        printf(" %d", k->dizi[(k->bas + i) % KAPASITE]);
    }
    printf("\n");
}

int main(void) {
    Kuyruk k = { {0}, 0, 0, 0 };
    int deger = 0;
    for (int i = 1; i <= 3; i++) {
        enqueue(&k, i * 10);
    }
    durum(&k);
    dequeue(&k, &deger);
    dequeue(&k, &deger);
    durum(&k);
    for (int i = 4; i <= 6; i++) {
        enqueue(&k, i * 10);
    }
    durum(&k);
    printf("enqueue(70): %d\n", enqueue(&k, 70));
    printf("çıkanlar:");
    while (dequeue(&k, &deger)) {
        printf(" %d", deger);
    }
    printf("\n");
    durum(&k);
    return 0;
}
```

Çıktı

```
bas=0 son=3 adet=3 | sıra: 10 20 30
bas=2 son=3 adet=1 | sıra: 30
bas=2 son=2 adet=4 | sıra: 30 40 50 60
enqueue(70): 0
çıkanlar: 30 40 50 60
bas=2 son=2 adet=0 | sıra:
```

İzle: Üç eleman girince `son = 3`. İki `dequeue` `bas`'ı `2`'ye getirdi. `40` dizinin son gözüne (`dizi[3]`) yazıldı ve `son`, `(3 + 1) % 4 = 0`'a **sardı**; `50` ile `60` dizinin başındaki boş gözlere yazıldı. Dizideki yerleşim `50 60 30 40` olsa da mantıksal sıra `bas`'tan başlayarak `30 40 50 60`'tır. Kuyruk dolu olduğu için `70` reddedildi; çıkış sırası giriş sırasıyla aynıdır: FIFO.

Bağlı liste ile kuyruk

Bağlı listede **iki uç** gerekir: çıkarmak için `bas`, eklemek için `son`. `son` olmasa her `enqueue` listeyi sonuna kadar gezerdi (O(n)); `son` ile ekleme tek adımdır. İki işaretçiyi bir yapıda toplayıp fonksiyonlara tek parametre olarak geçiriyoruz. `enqueue`'da kuyruk boşsa yeni düğüm hem `bas` hem `son` olur, değilse `son->sonraki`'ne bağlanır. `dequeue` kuyruğu boşaltırsa (`bas` `NULL` olursa) **`son` da `NULL`'a çekilmelidir**; yoksa `son`, serbest bırakılmış düğümü göstermeye devam eder.

Uygulama olarak bir **bilet simülasyonu** yazalım: Müşteriler gelince sıra numarası alır, gişe numaraları geliş sırasıyla çağırır (ilk gelen ilk çağrılır).

biletSistemi.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

typedef struct {
    Dugum *bas;                 // sıradaki ilk müşteri: buradan çıkar
    Dugum *son;                 // en son gelen müşteri: buraya eklenir
} Kuyruk;

int enqueue(Kuyruk *k, int deger) {
    Dugum *yeni = malloc(sizeof(Dugum));
    if (yeni == NULL) {
        return 0;
    }
    yeni->veri = deger;
    yeni->sonraki = NULL;
    if (k->son == NULL) {
        k->bas = yeni;
    } else {
        k->son->sonraki = yeni;
    }
    k->son = yeni;
    return 1;
}

int dequeue(Kuyruk *k, int *deger) {
    if (k->bas == NULL) {
        return 0;
    }
    Dugum *silinecek = k->bas;
    *deger = silinecek->veri;
    k->bas = silinecek->sonraki;
    if (k->bas == NULL) {
        k->son = NULL;
    }
    free(silinecek);
    return 1;
}

int main(void) {
```

```
    Kuyruk gise = {NULL, NULL};
    int siradakiBilet = 1;
    int cagrilan = 0;

    for (int i = 0; i < 3; i++) {
        printf("Bilet alındı: %d\n", siradakiBilet);
        enqueue(&gise, siradakiBilet++);
    }
    if (dequeue(&gise, &cagrilan)) {
        printf("Gişeye çağrıldı: %d\n", cagrilan);
    }
    printf("Bilet alındı: %d\n", siradakiBilet);
    enqueue(&gise, siradakiBilet++);
    while (dequeue(&gise, &cagrilan)) {
        printf("Gişeye çağrıldı: %d\n", cagrilan);
    }
    printf("Sırada kimse kalmadı.\n");
    return 0;
}
```

Çıktı

```
Bilet alındı: 1
Bilet alındı: 2
Bilet alındı: 3
Gişeye çağrıldı: 1
Bilet alındı: 4
Gişeye çağrıldı: 2
Gişeye çağrıldı: 3
Gişeye çağrıldı: 4
Sırada kimse kalmadı.
```

Sonradan gelen `4` numaralı müşteri, daha önce gelenlerin arkasına geçti ve en son çağrıldı. `dequeue` kuyruğu tamamen boşalttığı için her düğüm serbest bırakıldı.

> **Sınavda Çıkar: Dairesel kuyrukta `son` ve `% KAPASITE`**
>
> "`KAPASITE` 5 iken `son = 4` ve bir `enqueue` yapılırsa `son` kaç olur?" sorusunun cevabı `(4 + 1) % 5 = 0`'dır. `bas` ve `son` hiçbir zaman `KAPASITE`'ye eşit olmaz; `%` sarmayı yapar. Kuyruğun dolu mu boş mu olduğunu `bas == son` ile anlamaya çalışma, iki durumda da eşittir: `adet` tut ya da bir göz boş bırak.

14.9 Dizi mi, bağlı liste mi?

Hangisinin seçileceği, en çok hangi işlemi yaptığına bağlıdır. Maliyeti **sezgisel** karşılaştıracağız: n eleman varken işlem kabaca kaç adım sürer? **O(1)** eleman sayısından bağımsız, sabit sayıda adımdır (10 eleman da olsa 1 milyon da). **O(n)** eleman sayısı katlanınca adımın da kabaca katlanmasıdır (listeyi baştan sona gezmek gibi).

İşlem	Dizi	Bağlı liste
`i`'inci elemana erişim	O(1), `dizi[i]`	O(n), `i` adım gezmek gerekir
Başa ekleme	O(n), hepsi kaydırılır	O(1)
Sona ekleme	O(1), yer varsa	O(n); `son` işaretçisi varsa O(1)
Arama (sırasız)	O(n)	O(n)
Konumu belli düğümden sonra ekleme	O(n), kaydırma	O(1)
Boyut	Önceden belli, büyütmek kopyalama	İhtiyaç kadar büyür

Sıralı dizide **ikili arama** (Bölüm 7) çok hızlıdır; bağlı listede ortaya atlayamadığın için yapılamaz. **Bellek:** Dizi bitişik durur, ek yük taşımaz. Bağlı listede her düğüm ayrı `malloc` ister ve işaretçi kadar ek yer taşır (64 bit sistemlerde 8 bayt; hizalama boşluğuyla birlikte `int` verili düğüm genellikle 16 bayt tutar). Özet: sık `dizi[i]` erişimi ya da ikili arama gerekiyorsa **dizi**; sık başa ekleme/silme varsa ya da eleman sayısı sürekli değişiyorsa **bağlı liste**.

Sınavda Çıkar: Dizi ve bağlı liste karşılaştırması

"Başa ekleme hangisinde daha hızlıdır?" → bağlı liste (O(1), kaydırma yok). "`i`'inci elemana erişim?" → dizi (O(1)). "Bağlı listenin dezavantajı?" → rastgele erişim yok, her düğümde ekstra işaretçi, her düğüm ayrı `malloc`/`free`. Sınavda genelde iki sütunlu tablo doldurtulur; yukarıdaki tabloyu ezberle.

Not: Çift yönlü ve dairesel bağlı liste

Bu bölümdeki liste **tek yönlüdür**: yalnızca ileri gidebilirsin. **Çift yönlü bağlı listede** (doubly linked list) her düğümde `sonraki`'nin yanında bir de `onceki` işaretçisi bulunur; geri de gidilir ve silerken öncekini aramak gerekmez. Karşılığında düğüm büyür ve her ekleme/silmede dört işaretçi güncellenir. **Dairesel bağlı listede** son düğümün `sonraki`'si `NULL` değil **baş düğümü** gösterir; dolaşırken durma koşulu `p != NULL` değil, "başa dönüldü" (`p != bas`) olur. İkisinin kodunu bu kitapta yazmıyoruz.

Bölüm Özeti – Bilmen Gerekenler

- **Veri yapısı**, veriyi düzenleme biçimidir. Dizi sabit boyutludur; başa/araya ekleme-silme kaydırma, büyütme ise kopyalama ister.
- **Öz-referanslı yapı:** `typedef struct Dugum { int veri; struct Dugum *sonraki; } Dugum;`. İçeride `struct Dugum *` yazılır, çünkü `typedef` adı henüz doğmamıştır. Son düğümün `sonraki`'si `NULL`'dır.
- `dugumOlustur`: `malloc` + `NULL` kontrolü + `sonraki = NULL`. Dolaşma: `for (Dugum *p = bas; p != NULL; p = p->sonraki)`; `bas`'ı kaydırma, `p` kullan.
- **Başa ekleme:** `yeni->sonraki = bas; bas = yeni;` (sıra önemli). Liste başını değiştiren fonksiyon yeni başı döndürmeli ya da `Dugum **bas` almalıdır; `Dugum *bas` yalnız kopyayı değiştirir ve düğümü sızdırır.
- **Sona ekleme:** son düğümü bul (`p->sonraki != NULL`), boş listeyi ayrı ele al. **Silme:** önceki düğümü tut, `onceki->sonraki = silinecek->sonraki;`, sonra `free`.
- **Listeyi silme:** `sonraki`'ni önce sakla, sonra `free`. `free(p); p = p->sonraki;` tanımsız davranıştır.
- **Yığın** LIFO: `push`, `pop`, `tepe`. Dizide `tepe` indeksi (boşken `-1`), listede baş = tepe. Uygulama: parantez dengesi.
- **Kuyruk** FIFO: `enqueue` sona, `dequeue` baştan. Dairesel dizide `bas`, `son`, `adet` ve `% KAPASITE`; listede `bas` + `son` işaretçisi.
- Dizi: erişim O(1), başa ekleme O(n). Bağlı liste: başa ekleme O(1), `i`'inci eleman O(n).

Sorular

A) Çoktan Seçmeli

1 `typedef struct { int veri; Dugum *sonraki; } Dugum;` tanımı neden derlenmez?

(A) Bir yapı işaretçi üye taşıyamaz.
(B) `sonraki` yazılırken `Dugum` adı henüz tanıtılmamıştır; yapıya etiket verilip `struct Dugum *` yazılmalıdır.
(C) `typedef` yapılarla birlikte kullanılamaz.
(D) `int` ve işaretçi aynı yapıda bulunamaz.
(E) Yapı adı küçük harfle başlamalıdır.

2 `void basaEkle(Dugum *bas, int veri)` fonksiyonu yeni düğümü oluşturup `yeni->sonraki = bas; bas = yeni;` yapıyor. `main`'de `basaEkle(liste, 5);` çağrılırsa ne olur?

(A) `liste`'nin başına 5 eklenir.
(B) Derleme hatası oluşur.
(C) Program çöker.
(D) `main`'deki `liste` değişmez ve yeni düğüm erişilemez kalır (sızıntı).
(E) Liste boşaltılır.

3 Bir listeyi silerken `free(p); p = p->sonraki;` yazmak neden yanlıştır?

(A) `free(p)`'den sonra `p->sonraki` serbest bırakılmış belleği okur (tanımsız davranış); `sonraki` önce saklanmalıdır.
(B) `free` yalnızca dizilerde kullanılabilir.
(C) `p` bir işaretçi olduğu için `free`'ye geçirilemez.
(D) `free` bütün listeyi zaten sildiği için ikinci satır gereksizdir.
(E) `free` `NULL` döndürür ve döngü sonsuza kadar sürer.

4 Boş bir yığına ve boş bir kuyruğa ayrı ayrı sırayla 1, 2, 3 ekleniyor; sonra her birinden iki eleman çıkarılıyor. Çıkan elemanlar sırasıyla nedir?

(A) Yığın: 1, 2; kuyruk: 3, 2
(B) Yığın: 1, 2; kuyruk: 1, 2
(C) Yığın: 3, 2; kuyruk: 3, 2
(D) Yığın: 2, 3; kuyruk: 1, 2
(E) Yığın: 3, 2; kuyruk: 1, 2

5 Yalnızca `bas` işaretçisi tutulan tek yönlü bir bağlı listenin **sonuna** eleman eklemek için gereken iş, eleman sayısı n arttıkça nasıl artar?

(A) O(1): Listeyi gezmeye gerek yoktur.
(B) Hiç eklenemez; bağlı listeye yalnız başa eklenir.
(C) O(n): Son düğümü bulmak için listeyi baştan gezmek gerekir.
(D) O(n): Sonraki bütün elemanlar kaydırılır.
(E) O(log n): İkili arama kullanılır.

B) Çıktı Tahmini

6 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

int main(void) {
    Dugum d1 = {1, NULL};
    Dugum d2 = {2, NULL};
    Dugum d3 = {3, NULL};
    Dugum *bas = NULL;

    d1.sonraki = bas;
    bas = &d1;
    d2.sonraki = bas;
    bas = &d2;
    d3.sonraki = bas;
    bas = &d3;

    printf("%d\n", bas->sonraki->veri);
    printf("%d\n", bas->sonraki->sonraki->veri);
    for (Dugum *p = bas; p != NULL; p = p->sonraki) {
        printf("%d ", p->veri);
    }
    printf("\n");
    return 0;
}
```

7 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int yigin[5];
    int tepe = -1;

    yigin[++tepe] = 4;
    yigin[++tepe] = 7;
    yigin[++tepe] = 2;
    printf("%d\n", yigin[tepe--]);
    yigin[++tepe] = 9;
    printf("%d\n", yigin[tepe--]);
    tepe--;
    printf("%d %d\n", tepe, yigin[tepe]);
    yigin[++tepe] = 6;
    printf("%d %d\n", tepe, yigin[tepe]);
    return 0;
}
```

8 Aşağıdaki program dairesel bir kuyruğu benzetir. Ekrana ne yazdırır?

```c
#include <stdio.h>

#define K 4

int main(void) {
    int dizi[K] = {0};
    int bas = 0;
    int son = 0;
    int adet = 0;

    for (int i = 1; i <= 3; i++) {
        dizi[son] = i;
        son = (son + 1) % K;
        adet++;
    }
    for (int i = 0; i < 2; i++) {
        bas = (bas + 1) % K;
        adet--;
    }
    for (int i = 4; i <= 5; i++) {
        dizi[son] = i;
        son = (son + 1) % K;
        adet++;
    }
    printf("%d %d %d\n", bas, son, adet);
    for (int i = 0; i < adet; i++) {
        printf("%d ", dizi[(bas + i) % K]);
    }
    printf("\n");
    return 0;
}
```

C) Kod Yazma

9 Klavyeden okunan negatif olmayan bir tamsayıyı **yığın** kullanarak ikilik tabana çeviren `ikilik` programını yaz. Sayıyı 2'ye bölerek kalanları yığına `push` et, sonra yığını boşaltarak (`pop`) yazdır. Giriş `0` ise `0` yazmalıdır.

```
Girdi : 13
Çıktı : İkilik: 1101
```

10 Tek yönlü bir bağlı listeyi **ters çeviren** `Dugum *tersCevir(Dugum *bas)` fonksiyonunu yaz: Yeni düğüm oluşturmadan, yalnızca `sonraki` işaretçilerini çevir ve yeni başı döndür. `main`'de `1, 2, 3` verili üç düğümü yerel değişken olarak kur, listeyi önce ve sonra yazdır.

```
1 -> 2 -> 3 -> NULL
3 -> 2 -> 1 -> NULL
```

Çözümler

Cevap anahtarı

1 B	2 D	3 A	4 E	5 C

1 Cevap: B

`typedef` ile verilen `Dugum` adı yapının gövdesi bittikten sonra doğar; gövdede `Dugum *sonraki;` yazılınca derleyici adı bilmez. Çözüm: `typedef struct Dugum { ... struct Dugum *sonraki; } Dugum;`. A yanlış, işaretçi üye taşınabilir (öz-referanslı yapı budur). C ve D yanlış, böyle kurallar yoktur. E yanlış, harf büyüklüğü sebep değildir.

2 Cevap: D

`Dugum *bas` parametresi kopyadır; `bas = yeni;` yalnız kopyayı değiştirir, `main`'deki `liste` aynı kalır. Yeni düğümü gösteren tek işaretçi fonksiyon bitince yok olduğundan düğüm bir daha bulunamaz: bellek sızıntısı. A yanlış, ekleme çağırana yansımaz. B ve C yanlış, kod derlenir ve çökmez. E yanlış, liste hiç değişmez. Doğrusu `Dugum **bas` almak ya da yeni başı döndürmektir.

3 Cevap: A

`free(p)` düğümü geri verir; sonraki satırdaki `p->sonraki` serbest bırakılmış belleği okur. Doğru kalıp: `Dugum *sonraki = p->sonraki; free(p); p = sonraki;`. B yanlış, `free` her `malloc` bloğunda kullanılır. C yanlış, `free` zaten işaretçi bekler. D yanlış, `free` yalnızca tek düğümü bırakır. E yanlış, `free` değer döndürmez (`void`).

4 Cevap: E

Yığın LIFO'dur: 1, 2, 3 girer, önce 3 sonra 2 çıkar. Kuyruk FIFO'dur: önce 1 sonra 2 çıkar. A ikisini tersine karıştırmıştır. B yığını FIFO sanmıştır. C kuyruğu LIFO sanmıştır. D'deki yığın sırası (2, 3) imkânsızdır: 3, 2'den sonra girdiği için 2'den önce çıkar.

5 Cevap: C

Yalnızca `bas` bilindiği için son düğüme ulaşmanın tek yolu baştan `sonraki`'leri izlemektir: n düğümde $n-1$ adım, yani O(n). Ayrıca bir `son` işaretçisi tutulsaydı iş O(1) olurdu (A bu durumda doğru olurdu). B yanlış, sona eklenebilir. D diziler için geçerli bir ifadedir (başa eklemede kaydırma); sonda kaydırma yoktur. E yanlış, bağlı listede ikili arama yapılamaz.

6

Her `d.sonraki = bas; bas = &d;` çifti düğümü **başa** ekler; son eklenen `d3` en öndedir. Liste adım adım: `1 -> NULL`, sonra `2 -> 1 -> NULL`, sonra `3 -> 2 -> 1 -> NULL`. `bas->sonraki->veri` ikinci düğümün verisidir (`2`), `bas->sonraki->sonraki->veri` üçüncünün verisidir (`1`). Döngü listeyi baştan sona gezer.

Çıktı

```
2
1
3 2 1
```

7

`yigin[++tepe]` önce `tepe`'yi artırır, sonra yazar (push). `yigin[tepe--]` mevcut `tepe`'deki değeri okur, sonra `tepe`'yi azaltır (pop).

Satır	`tepe`	Yığın (alttan üste)	Yazılan
üç `push` (4, 7, 2)	2	4 7 2	
`yigin[tepe--]`	1	4 7	`2`
`push(9)`, sonra `yigin[tepe--]`	1	4 7	`9`
`tepe--` (değer atılır)	0	4	
`printf("%d %d", tepe, yigin[tepe])`	0	4	`0 4`
`push(6)`	1	4 6	`1 6`

Çıktı

```
2
9
0 4
1 6
```

8

`K = 4` olduğundan indeksler `0, 1, 2, 3, 0, 1, ...` diye döner.

Aşama	`dizi`	`bas`	`son`	`adet`
1, 2, 3 eklenince	1 2 3 0	0	3	3
İki çıkarma	1 2 3 0	2	3	1
4 eklenince	1 2 3 4	2	0	2
5 eklenince (başa sardı)	5 2 3 4	2	1	3

İlk satır `bas son adet` = `2 1 3`. Elemanlar `bas = 2`'den başlayarak `adet` tane okunur: `dizi[2]`, `dizi[3]`, `dizi[0]`. Dizideki sıra `5 2 3 4` olsa da mantıksal sıra `3 4 5`'tir.

Çıktı

```
2 1 3
3 4 5
```

9

Yaklaşım: `sayi % 2` en sağdaki biti verir. Bitler sağdan sola üretilir ama soldan sağa yazılmalıdır; yığın sırayı tersine çevirir. `do-while`, giriş `0` iken de bir kez döner ve `0` basar. 32 bitlik `int` en fazla 31 bit üretir, 32 gözlük yığın yeter.

ikilik.c

```c
#include <stdio.h>

int main(void) {
    int sayi = 0;
    int yigin[32];
    int tepe = -1;
```

```
    if (scanf("%d", &sayi) != 1 || sayi < 0) {
        printf("Geçersiz giriş\n");
        return 1;
    }
    do {
        yigin[++tepe] = sayi % 2;       // push
        sayi /= 2;
    } while (sayi > 0);

    printf("İkilik: ");
    while (tepe >= 0) {
        printf("%d", yigin[tepe--]);    // pop
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
13
İkilik: 1101
```

10

Yaklaşım: Her turda `sonraki`'ni sakla, bu düğümün `sonraki`'sini **önceki** düğüme çevir, sonra iki işaretçiyi bir ilerlet. Döngü bitince `onceki` yeni başı (eski son düğümü) gösterir. Sakla-çevir-ilerlet sırası bozulursa liste kopar.

tersCevir.c

```c
#include <stdio.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

Dugum *tersCevir(Dugum *bas) {
    Dugum *onceki = NULL;
    while (bas != NULL) {
        Dugum *sonraki = bas->sonraki;   // 1. sakla
        bas->sonraki = onceki;           // 2. ok'u çevir
        onceki = bas;                    // 3. ilerle
        bas = sonraki;
    }
    return onceki;
}

void yazdir(const Dugum *bas) {
    for (const Dugum *p = bas; p != NULL; p = p->sonraki) {
        printf("%d -> ", p->veri);
    }
    printf("NULL\n");
}

int main(void) {
    Dugum d3 = {3, NULL};
    Dugum d2 = {2, &d3};
    Dugum d1 = {1, &d2};
    Dugum *bas = &d1;

    yazdir(bas);
```

```
    bas = tersCevir(bas);
    yazdir(bas);
    return 0;
}
```

Çıktı

```
1 -> 2 -> 3 -> NULL
3 -> 2 -> 1 -> NULL
```

DENEME SINAVI

Deneme Final Sınavı

Bu deneme sınavı kitabın tamamını kapsar, ama ağırlık ikinci yarıdadır: işaretçiler, dinamik bellek, `struct` / `union` / `enum`, dosyalar, özyineleme ve veri yapıları (bağlı liste, yığın, kuyruk). İlk sekiz bölümden yalnız dört karışık temel soru var; çünkü gerçek final sınavında da temel bilgi, yeni konuların içine gömülü olarak sorulur. Tuzaklar **bilerek** kondu. Hocaların sevdiği şeylere dikkat et: `*p++` ile `(*p)++` farkı, `sizeof` ile `strlen` farkı, `const` olan satır, `realloc` kalıbı, `fopen` modları, `feof` tuzağı, özyinelemede çağrı sırası ve bağlı listeye ekleme sırası.

Bölüm	Soru sayısı	Soru başına puan	Toplam
A) Çoktan seçmeli	12	3	36
B) Çıktı tahmini	5	4	20
C) Kod yazma	4	11	44
Toplam	**21**		**100**

Süre: 100 dakika. Bilgisayar, kitap ve not kullanmak yok; sadece kâğıt ve kalem. Bütün sorularda standart C17 ve `gcc` geçerlidir; `int` 4 bayt, `double` 8 bayttır. `sizeof` soruları bu kitaptaki `gcc` için sorulmuştur.

> **İpucu: Önce kâğıtta çöz**
>
> Sınavı gerçek sınav gibi çöz: 100 dakikalık bir sayaç kur ve süre bitene kadar VS Code'u açma, çözümlere bakma. Kabaca **25 dakikayı** testlere, **20 dakikayı** çıktı sorularına, **55 dakikayı** kod yazmaya ayır. Bir soruda takılırsan işaretle ve geç; sona dönersin. Süre bitince kodlarını bilgisayarda derleyip çalıştır, çözümlerle karşılaştır ve kendini puanla.

> **Not: Kendini nasıl puanlarsın?**
>
> - **Çıktı sorularında** çıktının her satırı eşit puan değerindedir. Eksik boşluk, fazladan satır veya `80.0` yerine `80` yazmak o satırı yanlış yapar; gerçek sınavda da böyledir.
> - **Kod sorularında** çözümdeki program tek doğru cevap değildir. Senin kodun aynı işi yapıyorsa tam puan al. Çözümlerin sonunda "hangi adım kaç puan" notu var. Derleme hatası olan her satır için 1 puan, yanlış sonuç veren her mantık hatası için 2–3 puan kır. Ayırdığın belleği `free` etmeyen ya da açtığın dosyayı kapatmayan çözümden her seferinde 1 puan kır.
> - Her cevabın sonunda konunun anlatıldığı bölüm yazıyor. Yanlış yaptığın soruların bölümlerindeki **Sınavda Çıkar** ve **Sık Yapılan Hata** kutularını tekrar oku.

Not: Hangi soru hangi bölümden?

Bölüm	Sorular
1–8 (temel)	Test 1–4
9 İşaretçiler	Test 5, 6; Çıktı 13; Kod 18
10 Dinamik bellek	Test 7; Çıktı 14; Kod 19
11 Yapılar	Test 8, 9; Kod 19
12 Dosyalar	Test 10; Çıktı 15; Kod 20
13 Özyineleme	Test 11; Çıktı 16; Kod 21
14 Veri yapıları	Test 12; Çıktı 17; Kod 21

Sorular

A) Çoktan Seçmeli

1 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[20] = "merhaba";
    s[3] = '\0';
    printf("%zu %zu %s\n", strlen(s), sizeof(s), s + 4);
    return 0;
}
```

(A) `3 8 aba` (B) `7 20 aba` (C) `3 20 aba` (D) `3 20 ba` (E) `3 3 aba`

2 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int a = 7, b = 2;
    double d = a / b * 2.0;
    int e = (int) (a / 2.0) + a % b * 3;
    printf("%.1f %d %d\n", d, e, a > b ? a++ : b++);
    return 0;
}
```

(A) `6.0 6 7` (B) `7.0 6 7` (C) `6.0 7 7` (D) `6.0 6 8` (E) `7.0 7 8`

3 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int sayac = 0;
    for (int i = 1; i <= 4; i++) {
        for (int j = i; j <= 4; j++) {
            if ((i + j) % 3 == 0) {
                continue;
            }
            if (j == 4) {
                break;
            }
            sayac++;
        }
    }
    printf("%d\n", sayac);
    return 0;
}
```

(A) 2 (B) 3 (C) 5 (D) 4 (E) 6

4 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

void f(int a[], int n) {
    for (int i = 0; i < n; i++) {
        a[i] *= 2;
    }
}

int g(int x) {
    x += 10;
    return x;
}

int main(void) {
    int d[3] = {1, 2, 3};
    f(d, 3);
    g(d[0]);
    f(d + 1, 2);
    printf("%d %d %d\n", d[0], d[1], d[2]);
    return 0;
}
```

(A) 2 4 6 (B) 2 8 12 (C) 12 8 12 (D) 4 8 12 (E) 2 8 6

5 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    int d[4] = {10, 20, 30, 40};
    int *p = d;
    int a = *p++;
    int b = (*p)++;
    int c = *++p;
    printf("%d %d %d %d %d\n", a, b, c, d[1], (int) (p - d));
    return 0;
}
```

(A) `10 21 30 21 2` (B) `10 20 30 20 2` (C) `20 21 30 21 2` (D) `10 20 30 21 3` (E) `10 20 30 21 2`

6 Aşağıdaki programın `(1)`–`(4)` numaralı satırlarından hangileri **derleme hatası** verir? (Diğer satırlar doğrudur.)

```c
#include <stdio.h>

int main(void) {
    int x = 10, y = 20;
    const int *p = &x;
    int *const q = &x;
    p = &y;      // (1)
    *p = 5;      // (2)
    q = &y;      // (3)
    *q = 7;      // (4)
    printf("%d %d\n", x, y);
    return 0;
}
```

(A) Yalnız `(2)`

(B) `(2)` ve `(3)`

(C) `(3)` ve `(4)`

(D) `(1)`, `(2)` ve `(3)`

(E) Hiçbiri; program derlenir ve çalışır.

7 Aşağıdaki kod parçasında son satırdaki `realloc` kalıbı hakkında hangisi **doğrudur**?

```c
int *a = malloc(4 * sizeof(int));
if (a == NULL) {
    return 1;
}
/* a[0] ... a[3] doldurulur */
a = realloc(a, 8 * sizeof(int));
```

(A) `realloc` başarısız olursa eski bloğu kendiliğinden `free` eder; bu yüzden kalıp her zaman güvenlidir.

(B) `realloc` eski bloğun içeriğini sıfırlar; yeni dizi her zaman 0 ile başlar.

(C) `realloc` büyütülen bloğu her zaman aynı adreste tutar; `a`'ya yeniden atama gereksizdir.

(D) `realloc` başarısız olup `NULL` döndürürse eski blok hâlâ ayrılmıştır ama `a`'ya `NULL` yazıldığı için adresi kaybolur (bellek sızıntısı). Doğrusu sonucu önce geçici bir işaretçiye atamaktır.

(E) `malloc` ile ayrılmış blok `realloc` ile büyütülemez; yalnız `calloc` bloğu büyütülebilir.

8 Aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

typedef struct {
    int x;
    int y;
} Nokta;

int tasi(Nokta n) {
    n.x += 10;
    return n.x;
}

void yukari(Nokta *n) {
    n->y += 10;
}

int main(void) {
    Nokta a = {1, 2};
    Nokta b = a;
    b.x = 5;
    int r = tasi(a);
    yukari(&b);
    printf("%d %d %d %d %d\n", r, a.x, a.y, b.x, b.y);
    return 0;
}
```

(A) `11 1 2 5 12` (B) `11 11 2 5 12` (C) `11 5 2 5 12` (D) `11 1 12 5 12` (E) `11 1 2 5 2`

9 Bu kitaptaki `gcc`'de (`int` 4, `double` 8 bayt) aşağıdaki program ekrana ne yazdırır?

```c
#include <stdio.h>

enum Renk { KIRMIZI, YESIL = 5, MAVI, SARI = 2, MOR };
union Veri { int i; char c; double d; };

int main(void) {
    printf("%d %d %d %zu\n", MAVI, SARI, MOR, sizeof(union Veri));
    return 0;
}
```

(A) `6 2 3 13` (B) `2 2 3 8` (C) `6 2 3 8` (D) `6 2 3 16` (E) `5 2 3 8`

10 Aşağıdaki program çalışmadan önce `t10.txt` dosyası diskte **yoktur**. Program ekrana ne yazdırır?

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("t10.txt", "w");
    fprintf(f, "1\n2\n3\n");
    fclose(f);
    f = fopen("t10.txt", "a");
    fprintf(f, "4\n");
    fclose(f);
    f = fopen("t10.txt", "r");
    int x = 0, toplam = 0, tur = 0;
    while (!feof(f)) {
        fscanf(f, "%d", &x);
        toplam += x;
        tur++;
    }
    fclose(f);
    printf("%d %d\n", toplam, tur);
    return 0;
}
```

(A) `10 4` (B) `10 5` (C) `14 4` (D) `14 6` (E) `14 5`

11 Aşağıdaki program ekrana ne yazdırır? (`sayac`, `f` fonksiyonunun toplam kaç kez çağrıldığını sayar.)

```c
#include <stdio.h>

int sayac = 0;

int f(int n) {
    sayac++;
    if (n <= 1) {
        return 1;
    }
    return f(n - 1) + f(n - 2);
}

int main(void) {
    int sonuc = f(5);
    printf("%d %d\n", sonuc, sayac);
    return 0;
}
```

(A) `5 5` (B) `8 15` (C) `8 8` (D) `8 14` (E) `13 15`

12 Başlangıçta boş olan bir **yığına** ve başlangıçta boş olan bir **kuyruğa** sırasıyla şu işlemler uygulanıyor: `ekle(10)`, `ekle(20)`, `ekle(30)`, `cikar()`, `ekle(40)`, `cikar()`, `ekle(50)`, `cikar()`. (`ekle` yığında `push`, kuyrukta `enqueue`; `cikar` yığında `pop`, kuyrukta `dequeue` demektir.) Çıkarılan değerler, çıkış sırasıyla hangisidir?

(A) Yığın: 30, 40, 50 — Kuyruk: 30, 40, 50
(B) Yığın: 10, 20, 30 — Kuyruk: 30, 40, 50
(C) Yığın: 30, 40, 50 — Kuyruk: 10, 20, 40
(D) Yığın: 30, 40, 50 — Kuyruk: 10, 20, 30
(E) Yığın: 10, 20, 30 — Kuyruk: 10, 20, 30

B) Çıktı Tahmini

13 Aşağıdaki program ekrana ne yazdırır? Dört satırı da yaz.

```c
#include <stdio.h>

void degis(int a, int b) { int t = a; a = b; b = t; }
void takas(int *a, int *b) { int t = *a; *a = *b; *b = t; }

int main(void) {
    int x = 3, y = 8, d[4] = {1, 2, 3, 4};
    int *p = d + 1;
    degis(x, y);
    printf("%d %d\n", x, y);
    takas(&x, &y);
    printf("%d %d\n", x, y);
    takas(p, p + 2);
    *p++ = 9;
    (*p)++;
    printf("%d %d %d %d\n", d[0], d[1], d[2], d[3]);
    printf("%d %d %d\n", *p, p[-1], (int) (&d[3] - p));
    return 0;
}
```

14 Aşağıdaki program ekrana ne yazdırır? (Kısalık için `calloc` denetimi yazılmamıştır; bellek sorunsuz ayrılır.)

```c
#include <stdio.h>
#include <stdlib.h>

void ekle(int **dizi, int *n, int deger) {
    int *yeni = realloc(*dizi, (*n + 1) * sizeof(int));
    if (yeni == NULL) return;
    *dizi = yeni;
    (*dizi)[*n] = deger;
    (*n)++;
}

int main(void) {
    int n = 2, *a = calloc(n, sizeof(int));
    a[1] = 7;
    ekle(&a, &n, a[1] + 1);
    ekle(&a, &n, n * 10);
    printf("%d", a[0]);
    for (int i = 1; i < n; i++) printf(" %d", a[i]);
    printf("\nn = %d, son = %d\n", n, a[n - 1]);
    free(a);
    return 0;
}
```

15 Aşağıdaki program çalışmadan önce `notlar15.txt` dosyası diskte yoktur. Program ekrana ne yazdırır? (Kısalık için `fopen` denetimleri yazılmamıştır; dosyalar sorunsuz açılır.)

```c
#include <stdio.h>

int main(void) {
    FILE *f = fopen("notlar15.txt", "w");
    fprintf(f, "Ali 60\nVeli 80\n");
    fclose(f);
    f = fopen("notlar15.txt", "a");
    fprintf(f, "Ayse 100\n");
    fclose(f);
    char ad[16];
    int puan = 0, toplam = 0, sayi = 0;
    f = fopen("notlar15.txt", "r");
    while (fscanf(f, "%15s %d", ad, &puan) == 2) {
        toplam += puan;
        sayi++;
        if (puan >= 80) printf("%s gecti\n", ad);
    }
    printf("%d kisi, ortalama %.1f\n", sayi, (double) toplam / sayi);
    printf("eof: %d\n", feof(f) != 0);
    fclose(f);
    return 0;
}
```

16 Aşağıdaki program ekrana ne yazdırır? İlk satırdaki `<` ve `-` işaretlerini ve sırayı dikkatle yaz.

```c
#include <stdio.h>

void f(int n) {
    if (n <= 0) {
        return;
    }
    printf("<%d", n);
    f(n - 1);
    printf("-%d", n);
    f(n - 2);
}

int g(int a, int b) {
    if (b == 0) return a;
    return g(b, a % b);
}

int main(void) {
    f(3);
    printf("\n%d %d\n", g(48, 18), g(7, 5));
    return 0;
}
```

17 Aşağıdaki program ekrana ne yazdırır? (Kısalık için `malloc` denetimi ve `free` çağrıları yazılmamıştır; gerçek programda yazılır.)

```c
#include <stdio.h>
#include <stdlib.h>

struct Dugum { int veri; struct Dugum *sonraki; };

void basaEkle(struct Dugum **bas, int v) {
    struct Dugum *d = malloc(sizeof(struct Dugum));
    d->veri = v;
    d->sonraki = *bas;
    *bas = d;
}

int main(void) {
    struct Dugum *bas = NULL, *p;
    for (int i = 1; i <= 4; i++) basaEkle(&bas, i * i);
    p = bas->sonraki;
    bas->sonraki = p->sonraki;
    printf("Silinen: %d\nListe:", p->veri);
    for (p = bas; p != NULL; p = p->sonraki) printf(" %d", p->veri);
    printf("\n");
    return 0;
}
```

C) Kod Yazma

18 `minMaksTers` programını yaz. Program önce `n` değerini (1 ile 20 arasında) sonra `n` tamsayıyı klavyeden okuyup `int d[20]` dizisine koysun. `n` geçersizse sadece `Geçersiz n!` yazıp bitsin. İki fonksiyon yazman gerekiyor; ikisinde de **işaretçi aritmetiği** kullan, `[]` operatörünü kullanma:

- `void minMaks(const int *d, int n, int *kucuk, int *buyuk)`: dizinin en küçük ve en büyük elemanını **adres ile** `main`'e döndürür.
- `void tersCevir(int *bas, int *son)`: `bas` ve `son` dizinin ilk ve son elemanını gösterir; elemanları iki uçtan ortaya doğru takas ederek diziyi **yerinde** ters çevirir.

`main` önce en küçük ve en büyüğü, sonra ters çevrilmiş diziyi yazdırsın. Örnek çalışma:

```
n: 6
4 -2 9 0 9 3
En küçük: -2, en büyük: 9
Ters: 3 9 0 9 -2 4
```

19 `ogrenciSirala` programını yaz. Önce öğrenci sayısı `n` okunur (1'den küçükse `Geçersiz sayı!` yazıp bitsin). Sonra her satırda bir öğrencinin adı (boşluksuz, en çok 19 karakter), vize ve final notu okunur. Öğrencileri **`n` elemanlı dinamik bir diziye** (`malloc`) kaydet; her öğrencinin ortalaması `0.4 * vize + 0.6 * final` olsun. Öğrencileri ortalamaya göre **büyükten küçüğe** sırala (ortalaması eşit olanlar giriş sırasını korusun), sıra numarası, ad ve ortalamayı `%d. %-8s %6.2f` biçiminde yazdır, en sonda sınıf ortalamasını yazdır. Öğrenci için bir `struct` tanımla, belleği `free` et. Örnek çalışma:

```
Öğrenci sayısı: 4
Ali 70 60
Ayse 90 85
Veli 50 40
Can 80 80
1. Ayse      87.00
2. Can       80.00
3. Ali       64.00
4. Veli      44.00
Sınıf ortalaması: 68.75
```

20 Bir kırtasiyenin satış kayıtları `satis.csv` dosyasında tutuluyor. İlk satır başlıktır, her sonraki satırda `ürün,adet,fiyat` bilgisi vardır (ürün adı boşluksuz ve virgülsüz, en çok 19 karakter):

satis.csv

```
urun,adet,fiyat
kalem,10,12.50
defter,4,45.00
silgi,25,3.75
cetvel,6,18.20
```

`satisRapor` programını yaz. Program `satis.csv` dosyasını okusun (başlık satırını atlasın), her ürünün tutarını (`adet * fiyat`) hesaplasın ve `rapor.txt` dosyasına her ürün için `ad;tutar` biçiminde bir satır, en sona da `TOPLAM;genelToplam` satırı yazsın (tutarlar iki ondalıklı). Sonra `rapor.txt` dosyasını **yeniden açıp** içeriğini ekrana yazdırsın. `satis.csv` açılamazsa `satis.csv açılamadı!` yazıp `1` ile bitsin. Açtığın her dosyayı kapat. Beklenen ekran çıktısı:

```
kalem;125.00
defter;180.00
silgi;93.75
cetvel;109.20
TOPLAM;507.95
```

21 `listeTers` programını yaz. Program klavyeden **0 girilene kadar** tamsayı okusun ve her sayıyı tek yönlü bağlı listenin **sonuna** eklesin (`int sonaEkle(Dugum **bas, int v)`: başarıda 1, bellek yetmezse 0 döndürür). Sonra listeyi ileri doğru `->` ile ayırarak, ardından **özyinelemeli** bir fonksiyonla (`void tersYaz(const Dugum *d)`) tersten yazdırsın; eleman sayısını ve toplamı da yazsın. Hiç sayı girilmezse (ilk sayı `0` ise) sadece `Liste boş.` yazıp bitsin. Program bitmeden listedeki bütün düğümleri `free` et. Örnek çalışma:

```
Sayılar (0 ile biter): 5 8 3 8 1 0
Liste: 5 -> 8 -> 3 -> 8 -> 1
Ters: 1 8 3 8 5
Eleman: 5, toplam: 25
```

```
Sayılar (0 ile biter): 0
Liste boş.
```

Çözümler

Cevap anahtarı

1 C	2 A	3 D	4 B	5 E	6 B	7 D	8 A
9 C	10 E	11 B	12 D				

1 Cevap: C

`char s[20]` dizisi 20 bayttır; `sizeof(s)` içeriğe bakmaz, **dizinin boyutunu** verir: `20`. `s[3] = '\0'` yazınca dizi `m e r \0 a b a ...` olur. `strlen` ilk `'\0'`'a kadar sayar: `3`. `s + 4`, `s[4]`'ten başlayan adrestir; orada `aba` vardır (`s[3]` silinen `h` idi).

- A) `8`, `char s[] = "merhaba";` yazılsaydı `sizeof` değeri olurdu. Burada boyut açıkça 20.
- B) `7`, `s[3] = '\0'` satırından **önceki** uzunluktur.
- D) `ba` için `s + 5` gerekirdi.
- E) `sizeof` metnin uzunluğunu değil, dizinin kapladığı baytı verir.

Konu: Bölüm 8 ve 9.

2 Cevap: A

Soldan sağa gidelim:

- `a / b * 2.0`: önce `a / b` **tamsayı bölmesi**dir: `7 / 2 = 3`; sonra `3 * 2.0 = 6.0`. (`7.0` çıkması için `a / (double) b * 2` gerekirdi.)
- `(int) (a / 2.0)`: `3.5` kesilerek `3` olur; `a % b * 3` = `1 * 3` = `3`; toplam `e = 6`.
- `a > b ? a++ : b++`: koşul doğru, `a++` seçilir. Son ek `++` **eski değeri** verir: `7`. (`a` sonra 8 olur ama yazdırılmaz.)

Çıktı `6.0 6 7`.

- B) `7.0` bölmeyi ondalıklı yapmak demektir; bölme `int / int` olduğu için bu olmaz.
- C) `(int)` kesme yapar, yuvarlamaz; `3.5` → `3`.
- D) ve E) `8`, `++` ön ek olsaydı çıkardı.

Konu: Bölüm 2 ve 4.

3 Cevap: D

İzleme (`sayac` yalnız iki `if`'e de takılmayınca artar):

`i`	`j`	`i + j`	ne olur	`sayac`
1	1	2	artar	1
1	2	3	`continue`	1
1	3	4	artar	2
1	4	5	`j == 4` → `break`	2
2	2	4	artar	3
2	3	5	artar	4
2	4	6	`continue`	4
3	3	6	`continue`	4
3	4	7	`break`	4
4	4	8	`break`	4

Sonuç `4`. `break` yalnız **içteki** döngüden çıkar; dıştaki `i` döngüsü devam eder. İki `if`'in sırası da önemlidir: `(i + j) % 3 == 0` olan tur `continue` ile `j == 4` denetimine bile gelmez.

- A) `break`'in dıştaki döngüyü de bitirdiğini sanmaktır (o zaman `i = 1` turundan sonra durulur ve `2` çıkar).
- B), C) ve E) tablodaki bir turu eksik ya da fazla saymaktan çıkar.

Konu: Bölüm 5.

---

4 Cevap: B

`f(d, 3)` dizinin **kendisini** değiştirir (dizi fonksiyona adres olarak gider): `{2, 4, 6}`. `g(d[0])` `int` parametreyi **değer ile** alır; `x += 10` yalnız kopyayı değiştirir, dönüş değeri de kullanılmadığı için `d[0]` yine `2` kalır. `f(d + 1, 2)` ikinci elemandan başlayarak **iki** elemanı ikiye katlar: `d[1] = 8`, `d[2] = 12`. Sonuç `2 8 12`.

- A) `f(d + 1, 2)` çağrısını unutmaktır.
- C) `g(d[0])` çağrısının dizi elemanını değiştirdiğini sanmaktır.
- D) `f(d, 3)` çağrısını iki kez uygulamaktır (ilk elemanı da ikiye katlamak).
- E) `f(d + 1, 2)` çağrısında yalnız bir eleman değiştiğini sanmaktır.

Konu: Bölüm 6, 7 ve 9.

---

5 Cevap: E

Son ek `++` işleci `*`'dan **önce** bağlanır:

Satır	Anlamı	`p` nerede	Sonuç
`int a = *p++;`	`*(p++)`: `d[0]` okunur, sonra `p` ilerler	`d + 1`	`a = 10`
`int b = (*p)++;`	`d[1]` okunur (`20`), sonra **o eleman** artar	`d + 1`	`b = 20`, `d[1] = 21`
`int c = *++p;`	önce `p` ilerler, sonra `d[2]` okunur	`d + 2`	`c = 30`

Çıktı `10 20 30 21 2`; son sayı işaretçi farkıdır: `p - d` = `2` eleman.

- A) `(*p)++` ifadesinin yeni değeri verdiğini sanmaktır; son ek, **eski** değeri verir.
- B) `(*p)++` ile `d[1]`'in değişmediğini sanmaktır.
- C) `*p++` ifadesinde `p`'nin okumadan **önce** ilerlediğini (`a = 20`) ve `(*p)++` ifadesinin yeni değeri verdiğini (`b = 21`) sanmaktır.

- D) İşaretçi farkını `3` bulmaktır; `p` yalnız iki kez ilerledi.

Konu: Bölüm 9.

---

6 Cevap: B

`const int *p`: **gösterilen değer** sabit, `p` değişebilir. `int *const q`: **`q` sabit**, gösterilen değer değişebilir.

- `(1) p = &y;` doğru, `p`'nin kendisi sabit değil.
- `(2) *p = 5;` hata: `p` üzerinden değer değiştirilemez.
- `(3) q = &y;` hata: `q` sabit işaretçidir, başka adres alamaz.
- `(4) *q = 7;` doğru, `q`'nun gösterdiği `int` sabit değil.

gcc'nin verdiği hatalar (satır numaraları dosyadaki yerleşime göre değişir):

Derleyici çıktısı – program derlenmez

```
test6.c: In function 'main':
test6.c:8:8: error: assignment of read-only location '*p'
    8 |     *p = 5;     // (2)
      |       ^
test6.c:9:7: error: assignment of read-only variable 'q'
    9 |     q = &y;     // (3)
      |       ^
```

Okuma kuralı: `const`'u `*`'ın **soluna** yazdıysan değer, **sağına** yazdıysan işaretçi sabittir.

- A) `(3)` de hatadır.
- C) ve D) `(4)` ve `(1)` doğru satırlardır.
- E) Derleyici yazma girişimlerini kabul etmez.

Konu: Bölüm 9.

---

7 Cevap: D

`a = realloc(a, ...)` yazarsan ve bellek yetmeyip `realloc` `NULL` döndürürse eski blok **hâlâ ayrılmıştır** ama `a`'ya `NULL` yazıldığı için ona erişemezsin ve `free` edemezsin: bellek sızar. Güvenli kalıp:

```c
int *yeni = realloc(a, 8 * sizeof(int));
if (yeni == NULL) {
    free(a);
    return 1;
}
a = yeni;
```

- A) `realloc` başarısız olursa eski bloğa **dokunmaz**; `free` etmek senin işindir.
- B) `realloc` eski içeriği **korur**; yeni eklenen baytların değeri ise belirsizdir (sıfırlanmaz).
- C) `realloc` gerekirse bloğu **başka adrese taşır**; bu yüzden dönen adresi kullanmak zorundasın.
- E) `malloc`, `calloc` ve `realloc` ile ayrılan her blok `realloc` ile büyütülebilir.

Konu: Bölüm 10.

---

8 Cevap: A

`Nokta b = a;` bir **kopya** yapar; `b.x = 5` `a`'yı etkilemez. `tasi(a)` yapıyı **değer ile** alır: fonksiyon kendi kopyasını 11 yapar ve `11` döndürür; `a.x` `1` kalır. `yukari(&b)` adres alır, `n->y += 10` doğrudan `b.y`'yi `12` yapar.

`r = 11`, `a = (1, 2)`, `b = (5, 12)` → `11 1 2 5 12`.

- B) `tasi` içindeki değişikliğin `a`'ya yansıdığını sanmaktır.
- C) `b = a` atamasının iki yapıyı birbirine bağladığını sanmaktır (adres kopyalanmaz, **içerik** kopyalanır).
- D) `yukari(&b)` yerine `a`'nın değiştiğini sanmaktır.
- E) `b.y`'nin `yukari` sonrası `2` kaldığını sanmaktır.

Konu: Bölüm 11.

9 Cevap: C

`enum` sabitleri açıkça değer verilmedikçe **bir öncekinin bir fazlasıdır**: `KIRMIZI = 0`, `YESIL = 5`, `MAVI = 6`, `SARI = 2`, `MOR = 3`. `union`'ın bütün üyeleri aynı adresten başlar; boyutu **en büyük üyenin** boyutudur (`double` = 8), toplamı değil. Bu kitaptaki `gcc`'de `sizeof(union Veri)` = `8`. Çıktı `6 2 3 8`.

- A) `13`, üyelerin boyutlarının toplamıdır (`4 + 1 + 8`); `union` üyeleri üst üste biner, toplanmaz.
- B) `MAVI`'nin `2` olduğunu sanmaktır; `MAVI`, `YESIL`'den sonra gelir.
- D) `16`, `struct` içinde hizalama (padding) yüzünden görülebilecek bir boyuttur; burada en büyük üye 8 bayttır ve ek boşluk gerekmez.
- E) `YESIL = 5` satırından sonraki sabitin de `5` olduğunu sanmaktır.

Konu: Bölüm 11.

10 Cevap: E

`"w"` modu dosyayı **oluşturur ya da sıfırlar**; `1 2 3` yazılır. `"a"` modu **sona ekler**: dosya artık `1 2 3 4`. Okuma döngüsündeki asıl tuzak `feof`'tur: `feof`, **okuma girişimi dosya sonunu aştıktan sonra** doğru olur. `4` okunduğunda imleç `'\n'`'in önündedir, `feof` hâlâ yanlıştır; döngü bir tur daha döner. Bu turda `fscanf` hiçbir şey okuyamaz (`EOF` döndürür), `x` **eski değerini** (`4`) korur ve `toplam`'a bir kez daha eklenir.

Tur	okunan	`x`	`toplam`
1	1	1	1
2	2	2	3
3	3	3	6
4	4	4	10
5	okuyamaz	4	14

Çıktı `14 5`. Doğru kalıp `while (fscanf(f, "%d", &x) == 1)`'dir.

- A) `feof` tuzağını görmemektir: döngü 4 tur döner ve toplam 10 sanılır.
- B) Fazladan turu fark edip `fscanf` başarısız olunca `x`'in eski değerini koruduğunu (ve `toplam`'a yine eklendiğini) atlamaktır.
- C) Toplamı doğru bulup tur sayısını eksik saymaktır.
- D) Bir tur fazla saymaktır; döngü yalnız bir kez fazladan döner.

Konu: Bölüm 12.

11 Cevap: B

`f(n)` aslında Fibonacci dizisidir (temel durumlar 1): `f(0) = f(1) = 1`, `f(2) = 2`, `f(3) = 3`, `f(4) = 5`, `f(5) = 8`. Çağrı sayısı `C(n)` için `C(0) = C(1) = 1`, `C(n) = 1 + C(n−1) + C(n−2)`:

n	0	1	2	3	4	5
f(n)	1	1	2	3	5	8
C(n)	1	1	3	5	9	15

Çıktı `8 15`. Özyinelemeli Fibonacci'nin yavaş olmasının nedeni budur: aynı alt problemler defalarca çözülür.

- A) Sonucu ve çağrı sayısını `n`'ye eşit sanmaktır.
- C) Çağrı sayısını sonuca eşit saymaktır; her çağrı sayaç artırır, yalnız yapraklar değil.
- D) Bir çağrıyı atlamaktır (ilk çağrı `f(5)` de sayılır).
- E) Fibonacci dizisinde bir adım ileri kaymaktır (`f(5)` = 8, `f(6)` = 13).

Konu: Bölüm 13.

12 Cevap: D

Yığın **son giren ilk çıkar** (LIFO), kuyruk **ilk giren ilk çıkar** (FIFO) çalışır.

İşlem	Yığın (üst sağda)	Kuyruk (ön solda)	Yığından çıkan	Kuyruktan çıkan
`ekle(10)`, `ekle(20)`, `ekle(30)`	10 20 30	10 20 30		
`cikar()`	10 20	20 30	30	10
`ekle(40)`	10 20 40	20 30 40		
`cikar()`	10 20	30 40	40	20
`ekle(50)`	10 20 50	30 40 50		
`cikar()`	10 20	40 50	50	30

Yığın: 30, 40, 50. Kuyruk: 10, 20, 30.

- A) Kuyruğu da yığın gibi çalıştırmaktır.
- B) İki yapıyı da ters çalıştırmaktır (yığından en eski, kuyruktan en yeni eleman çıkıyor sanmak).
- E) Yığını kuyruk gibi çalıştırmaktır (en eski eleman çıkıyor sanmak).
- C) Kuyruğun üçüncü çıkışında `30` yerine `40` almaktır; `40` hâlâ içeridedir.

Konu: Bölüm 14.

13

`degis` parametreleri **değer ile** alır: yalnız kopyalar yer değiştirir, `x` ve `y` değişmez. `takas` **adres** alır, asıl değişkenler yer değiştirir. Dizi ve işaretçi izlemesi:

Satır	d	p
başlangıç	`{1, 2, 3, 4}`	`d + 1`
`takas(p, p + 2)`: `d[1]` ile `d[3]` yer değiştirir	`{1, 4, 3, 2}`	`d + 1`
`*p++ = 9;`: `d[1] = 9`, sonra `p` ilerler	`{1, 9, 3, 2}`	`d + 2`
`(*p)++;`: `d[2]` artar	`{1, 9, 4, 2}`	`d + 2`

Son satır: `*p` = `d[2]` = `4`; `p[-1]` = `*(p - 1)` = `d[1]` = `9`; `&d[3] - p` = `3 - 2` = `1`.

Çıktı
```
3 8
8 3
1 9 4 2
4 9 1
```

14

`calloc` iki elemanı da sıfırlar: `a = {0, 0}`, `a[1] = 7` yapılınca `{0, 7}`. `ekle` işlevi `int **` alır; çünkü `realloc` bloğu **taşıyabilir** ve `main`'deki `a` yeni adresi görmelidir. `n` de adresle geçtiği için fonksiyon içinde artırılabilir.

Çağrı	Eklenen değer	`a`	`n`
başlangıç		`{0, 7}`	2
`ekle(&a, &n, a[1] + 1)`	`7 + 1 = 8`	`{0, 7, 8}`	3
`ekle(&a, &n, n * 10)`	`3 * 10 = 30`	`{0, 7, 8, 30}`	4

`realloc` eski elemanları **korur**. Çıktının ilk satırı `0 7 8 30`; ikinci satırda `n = 4`, son eleman `a[3] = 30`.

Çıktı
```
0 7 8 30
n = 4, son = 30
```

15

`"w"` dosyayı yaratır ve `Ali` ile `Veli` satırlarını yazar. `"a"` modu dosyayı **silmez**, sonuna `Ayse 100` ekler; dosyada üç satır var. Okuma döngüsü `fscanf` iki değer okuduğu (`== 2`) sürece döner; bu doğru kalıp olduğu için `feof` tuzağı yoktur:

Okunan	`puan`	ekrana	`toplam`	`sayi`
`Ali 60`	60		60	1
`Veli 80`	80	`Veli gecti`	140	2
`Ayse 100`	100	`Ayse gecti`	240	3
(dosya sonu)	`fscanf` `EOF` döndürür			

Ortalama `240 / 3` = `80.0`. Döngü `fscanf` dosya sonunu görüp başarısız olduğunda bittiği için `feof(f)` doğrudur; `!= 0` ifadesi bunu `1`'e çevirir.

Çıktı
```
Veli gecti
Ayse gecti
3 kisi, ortalama 80.0
eof: 1
```

16

`f(3)`'ün çağrı ağacı, sırayla: `<n` yazılır, sonra `f(n-1)`, sonra `-n` yazılır, sonra `f(n-2)`.

- `f(3)`: `<3` → `f(2)`
- `f(2)`: `<2` → `f(1)`
  - `f(1)`: `<1` → `f(0)` (hemen döner) → `-1` → `f(-1)` (hemen döner)

- `-2` → `f(0)` (hemen döner)
- `-3` → `f(1)`: `<1` → `f(0)` → `-1` → `f(-1)`

Sıra: `<3 <2 <1 -1 -2 -3 <1 -1` (bitişik yazılır). Ağacın iki kolu olduğu için `f(1)` iki kez çalışır.

`g` Öklid'in en büyük ortak bölen algoritmasıdır: `g(48, 18)` → `g(18, 12)` → `g(12, 6)` → `g(6, 0)` = `6`. `g(7, 5)` → `g(5, 2)` → `g(2, 1)` → `g(1, 0)` = `1`.

Çıktı
```
<3<2<1-1-2-3<1-1
6 1
```

17

`basaEkle` her düğümü listenin **başına** ekler; bu yüzden sayılar ters sırada dizilir. `i = 1, 2, 3, 4` için `1, 4, 9, 16` eklenir:

Adım	Liste (baştan sona)
`basaEkle(1)`	1
`basaEkle(4)`	4 → 1
`basaEkle(9)`	9 → 4 → 1
`basaEkle(16)`	16 → 9 → 4 → 1

`p = bas->sonraki` ikinci düğümü (`9`) gösterir. `bas->sonraki = p->sonraki` ile `16`, doğrudan `4`'e bağlanır; `9` listeden çıkar. `p` hâlâ `9`'u gösterir ve düğüm `free` edilmediği için `p->veri` geçerlidir (`free` etseydik bu satır tanımsız olurdu). Son liste: `16 → 4 → 1`.

Çıktı
```
Silinen: 9
Liste: 16 4 1
```

Bu programda hiçbir `free` yok: dört düğümün hepsi sızar. Gerçek programda listeyi bitirirken hepsini serbest bırakırız.

18

Yaklaşım: **`minMaks`**'ta `*kucuk` ve `*buyuk` adreslerine yazarak iki sonucu birden döndürürüz; ikisi de önce ilk elemana eşitlenir, sonra `d + 1`'den `d + n`'e kadar işaretçiyle gezilir. **`tersCevir`**'de `bas` ve `son` birbirine yaklaşır: `bas < son` iken takas yapılır. İki işaretçi karşılaşınca (ya da geçince) iş bitmiştir; eleman sayısı tek ise ortadaki eleman yerinde kalır. Diziyi `d[i]` yerine `*p` ile okuduk; `d + n` bir sonraki konumdur ve yalnız karşılaştırmada kullanılabilir, okunamaz.

minMaksTers.c
```c
#include <stdio.h>

void minMaks(const int *d, int n, int *kucuk, int *buyuk) {
    *kucuk = *d;
    *buyuk = *d;
    for (const int *p = d + 1; p < d + n; p++) {
        if (*p < *kucuk) {
            *kucuk = *p;
        }
        if (*p > *buyuk) {
            *buyuk = *p;
```

```
        }
    }
}

void tersCevir(int *bas, int *son) {
    while (bas < son) {
        int gecici = *bas;
        *bas = *son;
        *son = gecici;
        bas++;
        son--;
    }
}

int main(void) {
    int d[20];
    int n = 0;
    printf("n: ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 20) {
        printf("Geçersiz n!\n");
        return 0;
    }
    for (int *p = d; p < d + n; p++) {
        scanf("%d", p);
    }
    int kucuk = 0, buyuk = 0;
    minMaks(d, n, &kucuk, &buyuk);
    printf("En küçük: %d, en büyük: %d\n", kucuk, buyuk);
    tersCevir(d, d + n - 1);
    printf("Ters:");
    for (const int *p = d; p < d + n; p++) {
        printf(" %d", *p);
    }
    printf("\n");
    return 0;
}
```

Çıktı

```
n: 6
4 -2 9 0 9 3
En küçük: -2, en büyük: 9
Ters: 3 9 0 9 -2 4
```

`scanf("%d", p)` içinde `&` yok: `p` zaten adrestir. Dizi `n` elemanlı olduğu için son elemanın adresi `d + n - 1`'dir; `d + n` yazan `tersCevir`'de dizinin dışına yazar. `minMaks` dizinin içeriğini değiştirmediği için `const int *` aldı.

Puanlama (11): okuma ve `n` denetimi 2; `minMaks` 3 (iki sonucu adresle yazmak 1, başlangıç değeri 1, karşılaştırma 1); `tersCevir` 3 (iki uç 1, takas 1, ilerleme ve durma koşulu 1); `[]` kullanmadan işaretçi aritmetiği 1; çağrılar ve çıktı biçimi 2.

19

Yaklaşım: `Ogrenci` yapısı ad, vize, final notu ve hesaplanmış ortalamayı tutar. `malloc(n * sizeof(Ogrenci))` ile dizi açılır, `NULL` denetimi yapılır. Sıralama için **kabarcık sıralaması**: komşu iki öğrencide `o[j].ort < o[j + 1].ort` ise yer değiştirilir; bu karşılaştırma **kesin** küçüktür olduğu için eşit ortalamalarda sıra bozulmaz. `Ogrenci` bir bütün olarak kopyalanabildiği için takas tek `Ogrenci t = o[j];` ile yapılır (Bölüm 11).

ogrenciSirala.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char ad[20];
    int vize;
    int finalNotu;
    double ort;
} Ogrenci;

int main(void) {
    int n = 0;
    printf("Öğrenci sayısı: ");
    if (scanf("%d", &n) != 1 || n < 1) {
        printf("Geçersiz sayı!\n");
        return 0;
    }
    Ogrenci *o = malloc(n * sizeof(Ogrenci));
    if (o == NULL) {
        printf("Bellek ayrılamadı!\n");
        return 1;
    }
    double toplam = 0;
    for (int i = 0; i < n; i++) {
        scanf("%19s %d %d", o[i].ad, &o[i].vize, &o[i].finalNotu);
        o[i].ort = 0.4 * o[i].vize + 0.6 * o[i].finalNotu;
        toplam += o[i].ort;
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (o[j].ort < o[j + 1].ort) {
                Ogrenci t = o[j];
                o[j] = o[j + 1];
                o[j + 1] = t;
            }
        }
    }
    for (int i = 0; i < n; i++) {
        printf("%d. %-8s %6.2f\n", i + 1, o[i].ad, o[i].ort);
    }
    printf("Sınıf ortalaması: %.2f\n", toplam / n);
    free(o);
    return 0;
}
```

Çıktı

```
Öğrenci sayısı: 4
Ali 70 60
Ayse 90 85
Veli 50 40
Can 80 80
1. Ayse     87.00
2. Can      80.00
3. Ali      64.00
4. Veli     44.00
Sınıf ortalaması: 68.75
```

`o[i].ad` bir dizidir, adresi zaten `&` olmadan verilir; `vize` ve `finalNotu` için `&` şart. `%19s`, 20 baytlık `ad` dizisinin taşmasını (`'\0'` dahil) önler. `malloc` sonucunu `free(o)` ile geri vermeyi unutma; sınıf ortalamasını sıralamadan önce de sonra da hesaplayabilirsin (toplam değişmez).

Puanlama (11): `struct` tanımı 1; `n` denetimi, `malloc` ve `NULL` denetimi 2; okuma (`%19s`, `&` kullanımı) 2; ortalama hesabı 1; sıralama 3 (iki döngü 1, doğru karşılaştırma yönü 1, `struct` takası 1); yazdırma ve sınıf ortalaması 1; `free` 1.

20

Yaklaşım: `satis.csv` `fopen(..., "r")` ile, `rapor.txt` `"w"` ile açılır. Başlık satırı bir `fgets` ile okunup atılır. Sonraki satırlar `fgets` döngüsüyle alınır (`NULL` dönünce dosya biter; `feof` tuzağı yok) ve `sscanf` ile ayrıştırılır: `%19[^,]` virgüle kadar olan adı, `%d` adeti, `%lf` fiyatı okur; `== 3` kontrolü bozuk satırı atlar. Rapor yazıldıktan sonra dosya **kapatılıp** `"r"` ile yeniden açılır ve satır satır ekrana basılır.

satisRapor.c

```c
#include <stdio.h>

int main(void) {
    FILE *giris = fopen("satis.csv", "r");
    if (giris == NULL) {
        printf("satis.csv açılamadı!\n");
        return 1;
    }
    FILE *rapor = fopen("rapor.txt", "w");
    if (rapor == NULL) {
        printf("rapor.txt oluşturulamadı!\n");
        fclose(giris);
        return 1;
    }
    char satir[100];
    fgets(satir, sizeof(satir), giris);
    double genel = 0;
    while (fgets(satir, sizeof(satir), giris) != NULL) {
        char ad[20];
        int adet = 0;
        double fiyat = 0.0;
        if (sscanf(satir, "%19[^,],%d,%lf", ad, &adet, &fiyat) == 3) {
            double tutar = adet * fiyat;
            fprintf(rapor, "%s;%.2f\n", ad, tutar);
            genel += tutar;
        }
    }
    fprintf(rapor, "TOPLAM;%.2f\n", genel);
    fclose(giris);
    fclose(rapor);

    rapor = fopen("rapor.txt", "r");
    if (rapor == NULL) {
        return 1;
    }
    while (fgets(satir, sizeof(satir), rapor) != NULL) {
        printf("%s", satir);
    }
    fclose(rapor);
    return 0;
}
```

Çıktı

```
kalem;125.00
defter;180.00
silgi;93.75
cetvel;109.20
TOPLAM;507.95
```

`fgets` satırın sonundaki `'\n'`'i de diziye koyar; `sscanf` buna takılmaz, ama ekrana yazarken `printf("%s", satir)` ekstra satır sonu eklememelidir (`rapor.txt` satırlarında `'\n'` zaten var). Dosyayı yazdıktan sonra `fclose` etmeden okumaya geçersen veriler diske inmemiş olabilir; bu yüzden önce `fclose`, sonra `"r"` ile yeniden açıyoruz.

Puanlama (11): `fopen` ve `NULL` denetimi 2; başlık satırını atlama 1; `fgets` döngüsü ve `sscanf` ile ayrıştırma 2; tutar ve genel toplam 1; `rapor.txt` biçimi ve `TOPLAM` satırı 2; kapatıp yeniden açarak ekrana yazdırma 2; her dosyanın kapatılması 1.

21

Yaklaşım: `sonaEkle`, yeni düğümü oluşturup `sonraki`'ni `NULL` yapar. Liste boşsa `*bas` yeni düğüm olur (`Dugum **` almamızın nedeni budur: `main`'deki `bas` değişebilir). Boş değilse son düğüme kadar gidilip ona bağlanır. `malloc` başarısız olursa `sonaEkle` 0 döndürür; `main` sayacı ve toplamı yalnız ekleme başarılıysa artırır, yoksa eklenmeyen bir sayı da sayılırdı. `tersYaz` özyinelemelidir: **önce** geri kalan listeyi tersten yazdırır, **sonra** kendi değerini yazar. Temel durum `d == NULL`'dır. `temizle` her düğümü silmeden önce `sonraki`'ni saklar; yoksa `free` sonrası `->sonraki` okumak tanımsız davranış olur.

listeTers.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;

int sonaEkle(Dugum **bas, int v) {
    Dugum *yeni = malloc(sizeof(Dugum));
    if (yeni == NULL) {
        return 0;                       // eklenemedi: çağıran bilsin
    }
    yeni->veri = v;
    yeni->sonraki = NULL;
    if (*bas == NULL) {
        *bas = yeni;
        return 1;
    }
    Dugum *p = *bas;
    while (p->sonraki != NULL) {
        p = p->sonraki;
    }
    p->sonraki = yeni;
    return 1;
}

void tersYaz(const Dugum *d) {
    if (d == NULL) {
        return;
    }
    tersYaz(d->sonraki);
    printf(" %d", d->veri);
}

void temizle(Dugum **bas) {
    while (*bas != NULL) {
        Dugum *silinecek = *bas;
        *bas = silinecek->sonraki;
        free(silinecek);
    }
}
```

```c
int main(void) {
    Dugum *bas = NULL;
    int x = 0, sayi = 0, toplam = 0;
    printf("Sayılar (0 ile biter): ");
    while (scanf("%d", &x) == 1 && x != 0) {
        if (!sonaEkle(&bas, x)) {       // eklenmediyse sayma, dur
            printf("Bellek yetersiz, %d eklenemedi.\n", x);
            break;
        }
        sayi++;
        toplam += x;
    }
    if (bas == NULL) {
        printf("Liste boş.\n");
        return 0;
    }
    printf("Liste: ");
    for (const Dugum *p = bas; p != NULL; p = p->sonraki) {
        printf("%s%d", p == bas ? "" : " -> ", p->veri);
    }
    printf("\nTers:");
    tersYaz(bas);
    printf("\nEleman: %d, toplam: %d\n", sayi, toplam);
    temizle(&bas);
    return 0;
}
```

Çıktı

```
Sayılar (0 ile biter): 5 8 3 8 1 0
Liste: 5 -> 8 -> 3 -> 8 -> 1
Ters: 1 8 3 8 5
Eleman: 5, toplam: 25
```

Özyineleme izlemesi: `tersYaz(5)` → `tersYaz(8)` → `tersYaz(3)` → `tersYaz(8)` → `tersYaz(1)` → `tersYaz(NULL)` döner; sonra çağrılar **geri sarılırken** `1`, `8`, `3`, `8`, `5` yazılır. `printf` özyinelemeli çağrıdan **sonra** yazıldığı için sıra ters çıkar; önce yazsaydık düz sırada yazardı. Her sayı için listeyi baştan gezmek yavaştır (`O(n)`); `son` işaretçisi tutarak `O(1)`'e inebilirsin, ama sınavda bu kadarı beklenmez.

Puanlama (11): `struct` ve `sonaEkle` 3 (boş liste 1, son düğümü bulma 1, bağlama 1); okuma döngüsü ve boş liste durumu 2; ileri yazdırma 1; `tersYaz` 3 (temel durum 1, özyineleme sonra yazma 2); `temizle` ile `free` 2 (`sonraki`'ni önceden saklamak 1).

EK A

C Cheat Sheet

Sınavdan önceki son tekrar için kitabın sıkıştırılmış hali; anlamadığın satırda ilgili bölüme dön.

```c
#include <stdio.h>                  // önişlemci: ; YOK

int main(void) {                    // program buradan başlar
    printf("Merhaba\n");            // her deyim ; ile biter
    return 0;                       // 0 = başarı
}
```

Temeller

Derleme ve tipler

```
gcc -std=c17 -Wall -Wextra a.c -o a
önişlemci → derleyici → çevirici → bağlayıcı
```

```
char       1 B   'A'=65 'a'=97 '0'=48
int        4 B   %d
long       4 B Windows, 8 B diğerleri
long long  8 B   %lld
float      4 B   %f
double     8 B   %f printf, %lf scanf
```

- `sizeof` sonucu `size_t`: `%zu` ile yaz. `sqrt` için sonuna `-lm`.
- `int` taşması tanımsız (UB); `unsigned` taşması mod 2^n.
- `#define MAX 100` tipsiz, `;` yok; `const int MAX = 100;` tipli.

Operatörler (yüksekten düşüğe)

```
()  []  .  ->  x++  x--
!  ~  ++x  --x  -x  *p  &x  (tip)  sizeof
*  /  %
+  -
<<  >>
<  <=  >  >=
==  !=
&  ^  |
&&  ||
?:  =  +=  -=  *=  /=  %=
```

- Tekli operatörler ve atama sağdan sola çalışır.
- `7 / 2` → `3`; `7 / 2.0` → `3.5`; `-7 % 2` → `-1`.
- `(double)a / b` ≠ `(double)(a / b)`.
- `x++` eski değeri, `++x` yeni değeri verir.
- `&&` ve `||` kısa devre yapar: sağı çalışmayabilir.
- `x & 1 == 0` → `x & (1 == 0)`: parantez koy.
- Bit: `&` VE, `|` VEYA, `^` XOR, `~` değil, `<<` `>>` kaydırma; öncelik `&` > `^` > `|`. `^` üs değildir; bitlerde `unsigned` kullan.
- Maske: `x | (1u << n)` biti açar, `x & ~(1u << n)` kapatır, `x & 1` ≡ tek mi?

Girdi ve çıktı

printf

```
%d    int          %u    unsigned
%ld   long         %lld  long long
%zu   size_t       %x    onaltılık
%f    double       %e    1.234560e+04
%c    karakter     %s    string
%p    adres        %%    yüzde
%5d   ␣␣␣42       %-5d  42␣␣␣
%05d  00042        %.2f  3.14
%8.3f ␣␣␣3.142
```

- `printf` satır sonu eklemez: `\n` yaz. `%d` ile `double` UB.

scanf

```c
int n; double d; char ad[20], ch;
scanf("%d", &n);        // int: & şart
scanf("%lf", &d);       // double: %lf
scanf("%19s", ad);      // dizi: & YOK
scanf(" %c", &ch);      // boşlukları atla
```

- `if (scanf("%d", &n) != 1)` → okunan öğe sayısı 1 değil.
- `%19s` boşlukta durur, `ad[20]` sınırını aşmaz.
- Kalan `\n`'yi temizle: `int c; while ((c = getchar()) != '\n' && c != EOF) { }`

Satır okuma

```c
char satir[100];
fgets(satir, sizeof satir, stdin);
satir[strcspn(satir, "\n")] = '\0';
```

- `fgets` `\n`'yi tutar; `strcspn` ile at. `gets` YOK (C11'de kalktı).

Kontrol ve döngüler

if, switch, ?:

```c
if (x > 0) {
    puts("pozitif");
} else if (x == 0) {
    puts("sıfır");
} else {
    puts("negatif");
}
switch (gun) {
    case 1:
    case 7:
        puts("Tatil");
        break;  // yoksa alta düşer
    default:
        puts("İş");
}
int m = a > b ? a : b;
```

- Yanlış = `0`: `if (x)` ≡ `if (x != 0)`.
- `case` etiketi sabit tamsayı olmalı; `string` ile `switch` olmaz.

Döngüler

```c
for (int i = 0; i < 5; i++)
    printf("%d ", i);
while (n > 0) n /= 10;      // n = n / 10
do {
    n--;
} while (n > 0);            // ; ZORUNLU
```

- `break` döngüden/`switch`ten çıkar; `continue` sonraki tura geçer.
- Sonsuz döngü: `for (;;)` ya da `while (1)`.

Fonksiyonlar

İskelet ve prototip

```c
int kare(int x);  // prototip
int main(void) {
    printf("%d\n", kare(4));  // 16
    return 0;
}
int kare(int x) { return x * x; }
```

- Kullanılmadan önce prototip ya da tanım gelmeli.
- Parametre yoksa `(void)`; değer dönmeyen `void`.

Değer ile mi, adres ile mi?

```c
// adres ile (işaretçi)
void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}
int x = 1, y = 2;
swap(&x, &y);  // x=2, y=1
```

- Argümanlar **kopyalanır**; değiştirmek için `&` gönder.
- Dizi adres olarak gider; kopyalanmaz.
- `static int sayac = 0;` yerel, ama çağrılar arasında korunur.

Diziler ve string

Diziler

```c
int a[5] = {0};                 // hepsi 0
int b[] = {3, 1, 4};            // boyut 3
int n = sizeof a / sizeof a[0]; // 5
int m[2][3] = { {1, 2, 3}, {4, 5, 6} };
```

- İndeks `0 … n−1`; `a[5]` UB. Fonksiyona: `int a[]` + `int n`.
- 2B dizi fonksiyona: `int m[][3]`; ikinci boyut zorunlu.

String ve '\0'

```c
char s[] = "Ali";  // A l i \0 → 4 B
```

- `'\0'` sonlandırıcı: `strlen("Ali")` → `3`, bellekte +1 bayt.

```c
strlen(s)         uzunluk ('\0' hariç)
strcpy(d, s)      kopyala (taşma riski)
strncpy(d, s, n)  en fazla n kopyala
strcat(d, s)      sona ekle
strcmp(a, b)      0 = eşit (== KULLANMA)
strchr / strstr   karakter / alt dize ara
```

- `strncpy` uzun kaynakta `'\0'` koymaz: kendin ekle. `strcmp` `<0`, `0`, `>0` döner.

ctype.h

```c
#include <ctype.h>
int r = c - '0';   // '7' → 7
char b = 'a' + 1;  // 'b' (98)
```

- `isdigit`, `isalpha`, `isspace` → `0` hayır. `toupper(c)` yeni karakteri döndürür.

İşaretçiler ve bellek

İşaretçi

```c
int x = 7;
int *p = &x;   // p: x'in adresi
*p = 5;        // x artık 5
int *q = NULL; // boş işaretçi
```

```c
const int *p   değer sabit, p değişir
int *const p   p sabit, değer değişir
int **pp       işaretçinin işaretçisi
```

- `p + 1` sonraki elemanı gösterir (int ise +4 B); `a[i] == *(a + i)`.
- `*p++` adresi ilerletir (eski değeri alır); `(*p)++` değeri artırır.
- `int *a, b;` → yalnız `a` işaretçi; `b` bir `int`.

malloc, calloc, free

```c
#include <stdlib.h>
int *p = malloc(n * sizeof *p);
if (p == NULL) return 1;   // kontrol!
/* ... kullan ... */
free(p);
p = NULL;
```

```c
// büyütme: eski p kaybolmasın
int *tmp = realloc(p, 2 * n * sizeof *p);
if (tmp == NULL) { free(p); return 1; }
p = tmp;
```

- `calloc(n, sizeof *p)` sıfırlar; `malloc` çöp verir.
- Her `malloc`'a bir `free`; `free(NULL)` güvenli.

Yapılar, union, enum

```c
typedef struct {
    char ad[20];
    int yas;
} Kisi;
Kisi k = {"Ali", 20};
k.yas = 21;     // nokta
Kisi *pk = &k;
pk->yas = 22;   // ok
```

```c
union Veri { int i; float f; };
// KIRMIZI 0, YESIL 5, MAVI 6
enum Renk { KIRMIZI, YESIL = 5, MAVI };
```

- `struct =` ile kopyalanır; `==` ile karşılaştırılamaz.
- `union` üyeler aynı belleği paylaşır; boyut en büyüğü kadar.

Dosyalar

Açma modları

```
"r"   oku (yoksa NULL)    "w"  yaz (SİLER)
"a"   sona ekle           "r+" oku/yaz
"rb" "wb"  ikili modlar
```

```c
FILE *f = fopen("veri.txt", "r");
if (f == NULL) {
    perror("veri.txt");
    return 1;
}
char s[100];
while (fgets(s, sizeof s, f) != NULL)
    printf("%s", s);
fclose(f);
```

```
fprintf fscanf fgets fputs fgetc fputc
fread fwrite fseek ftell rewind fclose
```

- `while (!feof(f))` tuzağı: son satır iki kez işlenir.
- `fgetc` `int` döndürür; `EOF` için `int c` kullan.

Özyineleme ve veri yapıları

Özyineleme

```c
long fakt(int n) {
    if (n <= 1) return 1;     // taban
    return n * fakt(n - 1);   // küçülür
}
```

- `fakt(5)` → `120`. Taban yoksa yığın taşar.

Bağlı liste

```c
typedef struct Dugum {
    int veri;
    struct Dugum *sonraki;
} Dugum;
Dugum *bas = NULL;  // boş liste
Dugum *yeni = malloc(sizeof *yeni);
if (yeni == NULL) exit(1);  // kontrol
yeni->veri = 7;
yeni->sonraki = bas;  // başa ekle
bas = yeni;
```

```c
Dugum *p;
for (p = bas; p != NULL; p = p->sonraki)
    printf("%d\n", p->veri);
```

```c
Dugum *q;
while (bas != NULL) {
    q = bas->sonraki;  // önce sakla
    free(bas);
    bas = q;
}
```

Yığın (LIFO)

```c
#define KAP 100
typedef struct {
    int dizi[KAP];
    int tepe;               // boşken -1
} Yigin;
int push(Yigin *y, int x) {
    if (y->tepe == KAP - 1) return 0;
    y->dizi[++y->tepe] = x;
    return 1;               // 1 = başarılı
}
int pop(Yigin *y, int *x) {
    if (y->tepe == -1) return 0;
    *x = y->dizi[y->tepe--];
    return 1;
}
// Yigin y = { {0}, -1 };
```

Kuyruk (FIFO, dairesel)

```c
typedef struct {
    int dizi[KAP];
    int bas, son, adet;
} Kuyruk;
int enqueue(Kuyruk *k, int x) {
    if (k->adet == KAP) return 0;  // dolu
    k->dizi[k->son] = x;
    k->son = (k->son + 1) % KAP;
    k->adet++;
    return 1;
}
int dequeue(Kuyruk *k, int *x) {
    if (k->adet == 0) return 0;    // boş
    *x = k->dizi[k->bas];
    k->bas = (k->bas + 1) % KAP;
    k->adet--;
    return 1;
}
```

- Boş: `adet == 0`; dolu: `adet == KAP`. `(son + 1) % KAP` sarar. Boşken `pop`/`dequeue` 0 döndürür; dönüş değerini her zaman kontrol et.

Algoritmalar

```c
int asal(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) return 0;
    return 1;
}
int ebob(int a, int b) {  // Öklid
    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }
    return a;
}
```

```c
// basamak toplamı
int topla = 0;
while (n > 0) {
    topla += n % 10;      // son basamak
    n /= 10;
}
int max = a[0];           // N: dizi boyu
for (int i = 1; i < N; i++)
    if (a[i] > max) max = a[i];
```

```c
// a sıralı olmalı
int ikili(const int a[], int n, int x) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int orta = lo + (hi - lo) / 2;
        if (a[orta] == x) return orta;
        if (a[orta] < x) lo = orta + 1;
        else hi = orta - 1;
    }
    return -1;  // bulunamadı
}
```

```c
// kabarcık sıralama
for (int i = 0; i < n - 1; i++)
    for (int j = 0; j < n - 1 - i; j++)
        if (a[j] > a[j + 1]) {
            int t = a[j];
            a[j] = a[j + 1];
            a[j + 1] = t;
        }
```

En sık 10 sınav tuzağı

1. `if (x = 5)` **atar**; karşılaştırma `==` ile yapılır.
2. `if (x > 0);` → `;` ayrı deyim; sonraki satır her zaman çalışır.
3. `switch`'te `break` yoksa alttaki `case`ler de çalışır.
4. `7 / 2` → `3`: ondalık için en az bir taraf `double` olmalı.
5. `for (i = 0; i <= n; i++)` → `a[n]` dışarıda; son indeks `n - 1`.
6. `scanf("%d", n)` yanlış → `&n`; `double` için `%lf`; kalan `\n` yüzünden `fgets` boş okur.
7. `s1 == s2` adresleri karşılaştırır → `strcmp(s1, s2) == 0`.
8. `char s[3] = "abc";` → `'\0'` sığmaz → `char s[4]` olmalı.
9. `free(p)` sonrası `p` kullanmak ya da iki kez `free` etmek UB (çökebilir).
10. `void main`, `gets`, `fflush(stdin)` YASAK → `int main(void)`, `fgets`.

EK B

Terimler Sözlüğü

Bu sözlükte kitapta geçen C terimleri Türk alfabesi sırasıyla dizilmiştir. Kitapta ya da derleyicinin İngilizce mesajında bir terimle karşılaşırsan, İngilizce sütununa bakarak Türkçe karşılığını bulabilirsin.

A	İngilizce	Açıklama
açma kipi	*open mode*	`fopen` fonksiyonunun ikinci argümanı; okuma (`r`), yazma (`w`) ya da ekleme (`a`).
adres	*address*	Bellekteki bir baytın sıra numarası; `&x` ile alınır.
adres operatörü	*address-of operator*	`&` operatörü; bir değişkenin bellekteki adresini verir: `&x`.
akım	*stream*	Veri akışı; `stdin`, `stdout` ve açılmış dosyalar birer akımdır.
ana fonksiyon	`main`	Programın çalışmaya başladığı fonksiyon; her C programında bulunmalıdır.
anahtar sözcük	*keyword*	C'de ayrılmış, değişken adı olarak kullanılamayan kelime; `int`, `return` gibi.
argüman	*argument*	Fonksiyon çağrısında gönderilen gerçek değer; parametreye karşılık gelir.
artırma operatörü	*increment operator*	`++` değeri 1 artırır; `++i` ve `i++` ifadede farklı sonuç verir.
atama	*assignment*	`=` ile değişkene değer yazma işlemi: `x = 5;`.

B	İngilizce	Açıklama
bağlama hatası	*linker error*	Bir adın tanımı bulunamadığında bağlayıcının verdiği hata; `undefined reference` yazar.
bağlayıcı	*linker*	Nesne dosyalarını birleştirip çalıştırılabilir programı üreten araç.
bağlı liste	*linked list*	Düğümlerin birbirine işaretçiyle bağlandığı, bellekte dağınık tutulabilen liste.
başlık dosyası	*header file*	`.h` uzantılı; prototip ve makroları `#include` ile birden çok kaynak dosyaya taşır.
başlık koruyucusu	*include guard*	Başlık dosyasını `#ifndef` / `#define` / `#endif` (ya da `#pragma once`) ile saran kalıp; aynı başlığın bir kaynakta iki kez işlenmesini önler.
bayrak	*flag*	Bir durumu (ör. bulundu mu) tutan, genellikle 0 ya da 1 değerli değişken.
bayt	*byte*	Adreslenebilen en küçük bellek birimi; `sizeof(char)` her zaman 1'dir.
bellek	*memory (RAM)*	Programın verilerini tuttuğu, adresleriyle erişilen geçici donanım alanı.
bellek sızıntısı	*memory leak*	`malloc` ile alınan belleğin `free` edilmeden erişilemez hâle gelmesi.
bellekleme	*memoization*	Hesaplanan sonucu saklayıp aynı işi tekrar yapmamak; özyinelemeyi hızlandırır.
biçim belirteci	*format specifier*	`printf` ve `scanf` içinde değerin türünü belirten `%d`, `%f`, `%s` gibi kalıp.
biçim metni	*format string*	`printf` ve `scanf` fonksiyonlarına verilen, biçim belirteçleri içeren metin.

B	İngilizce	Açıklama
bildirilmemiş tanımlayıcı	*undeclared identifier*	Kullanılmadan önce bildirilmeyen ad; derleyici `undeclared` hatası verir.
bildirim	*declaration*	Bir adın türünü derleyiciye duyurur; tek başına yer ayırmayabilir.
bileşik atama	*compound assignment*	Bir operatörle atamayı birleştirir: `x += 2;` komutu `x = x + 2;` ile aynıdır.
birlik	`union`	Tüm üyeleri aynı bellek alanını paylaşan tür; boyutu en büyük üyeye göre belirlenir.
bit alanı	*bit-field*	Yapı üyesinin kaç bit kaplayacağını belirten yazım (`unsigned int gun : 5;`); adresi alınamaz.
boş deyim	*empty statement*	Yalnız `;` içeren, hiçbir iş yapmayan deyim; yanlış yere konursa döngüyü bozar.
boş işaretçi	*null pointer*	Hiçbir nesneyi göstermeyen işaretçi; `NULL` ile gösterilir ve dolaylanamaz.
büyük/küçük harf duyarlı	*case-sensitive*	C'de `Ad` ile `ad` farklı tanımlayıcılardır.

Ç-D	İngilizce	Açıklama
çağrı ağacı	*call tree*	Özyinelemeli çağrıların dallanarak oluşturduğu ağaç biçimli çağrı yapısı.
çağrı yığını	*call stack*	Açık fonksiyon çağrılarının sırasını tutan bellek bölgesi; `main` en altta olur.
çalışma zamanı hatası	*runtime error*	Program çalışırken ortaya çıkan hata; derleyici bunu önceden yakalayamaz.
çevirici	*assembler*	Derleyicinin ürettiği assembly kodunu makine koduna (`.o` nesne dosyası) çeviren derleme aşaması.
çıkarma	*pop*	Yığının en üstündeki elemanı çıkarır ve geri döndürür.
çıkış kodu	*exit status*	`main`'in `return` ettiği değer; `0` başarı, sıfırdan farklı değer hata demektir.
çift yönlü bağlı liste	*doubly linked list*	Her düğümün hem öncekini hem sonrakini gösterdiği bağlı liste.
çoklu seçim	`switch`	Bir tamsayı değere göre birden fazla koldan birini seçen yapı.
çöp değer	*garbage value*	Başlatılmamış değişkenin içinde duran, anlamsız rastgele değer.
dairesel kuyruk	*circular queue*	Sonu başa bağlanmış sabit boyutlu kuyruk; dizi ile verimli uygulanır.
değer ile geçirme	*pass by value*	Fonksiyona değerin kopyası gönderilir; çağrılan fonksiyon orijinali değiştiremez.
değişken	*variable*	Değeri çalışma sırasında değişebilen, bellekte bir yere verilmiş ad.
değişken uzunluklu dizi	*variable length array (VLA)*	Boyutu çalışma zamanında belirlenen dizi (C99); C11'de isteğe bağlıdır.
derleme hatası	*compile-time error*	Kod C kurallarına uymadığında derleyicinin programı üretmeden verdiği hata.
derlenen dil	*compiled language*	Kodu önce makine diline çevrilip sonra çalıştırılan dil; C bu gruptadır.
derleyici	*compiler*	Kaynak kodu makine diline çeviren program; C için `gcc` gibi.
deyim	*statement*	`;` ile biten, bir iş yapan kod parçası.
dış bildirim	`extern`	Değişkenin başka bir `.c` dosyasında tanımlı olduğunu duyurur (`extern int sayac;`); bellekte yer ayırmaz.

Ç–D	İngilizce	Açıklama
dinamik bellek ayırma	*dynamic memory allocation*	Çalışma zamanında `malloc` ya da `calloc` ile bellek talep etme.
dizi	*array*	Aynı türden elemanların bellekte ardışık tutulduğu yapı.
diziye işaretçi	*pointer to array*	Belirli uzunluktaki bir diziyi bütün olarak gösteren işaretçi: `int (*q)[3]`.
doğru	*true*	C'de sıfırdan farklı her değer doğru kabul edilir.
doğrulama makrosu	`assert`	`assert.h` içindeki makro; koşul yanlışsa dosya adını ve satırı yazıp programı durdurur, `NDEBUG` tanımlıysa derlemeden silinir.
dolaylama	*dereference*	`*p` ile işaretçinin gösterdiği bellek yerine erişme.
dolgu baytı	*padding*	Yapıda hizalama için eklenen, kullanılmayan boş baytlar; `sizeof` bunları da sayar.
dosya işaretçisi	`FILE *`	Açık bir dosyayı ya da akımı gösteren işaretçi; `fopen` döndürür.
dosya sonu	*end of file (EOF)*	Dosyanın bittiğini belirten, genellikle `-1` olan `EOF` sabiti.
döngü	*loop*	Bir kod bloğunu koşul doğru olduğu sürece tekrarlayan yapı.
dönüş türü	*return type*	Fonksiyonun döndüreceği değerin türü; `void` ise değer dönmez.
düğüm	*node*	Bağlı yapılarda veriyi ve sonraki bağlantıyı tutan parça.
düşme	*fall-through*	`switch`'te `break` yoksa sıradaki `case`'e geçilmesi.

F–H	İngilizce	Açıklama
fonksiyon	*function*	Belirli bir işi yapan, adı ve parametreleriyle çağrılabilen kod bloğu.
fonksiyon işaretçisi	*function pointer*	Bir fonksiyonun adresini tutan işaretçi; `int (*f)(int)` biçimindedir.
genel değişken	*global variable*	Hiçbir fonksiyonun dışında tanımlanır; dosyanın her yerinden erişilir.
gölgeleme	*shadowing*	İç kapsamdaki aynı adlı değişkenin dış değişkeni geçici olarak gizlemesi.
göreli yol	*relative path*	Çalışma klasörüne göre verilen dosya yolu; `veri.txt` gibi.
hizalama	*alignment*	Verinin belleğin belli adreslerinden (ör. 4'ün katı) başlaması kuralı.

İ	İngilizce	Açıklama
iç bağlantı	*internal linkage*	Fonksiyon dışındaki `static` ile oluşur; ad yalnız tanımlandığı `.c` dosyasında görünür, bağlayıcı onu başka dosyalara göstermez.
ifade	*expression*	Bir değer üreten kod parçası; `a + b` bir ifadedir.
ikili arama	*binary search*	Sıralı dizide ortadaki elemana bakarak aramayı yarıya indiren algoritma.
ikili dosya	*binary file*	Baytları olduğu gibi yazılan dosya; `rb` ve `wb` kipleriyle açılır.
ilişkisel operatör	*relational operator*	`<`, `>`, `==` gibi değerleri karşılaştıran operatör; sonucu `0` ya da `1`.
ilk giren ilk çıkar	*FIFO*	Kuyruğun çalışma ilkesi; ilk eklenen eleman ilk çıkarılır.
indis	*index*	Dizideki bir elemanın sıra numarası; ilk eleman `0` indisindedir.
işaretçi	*pointer*	Bir bellek adresini tutan değişken; `int *p;` ile tanımlanır.
işaretçi aritmetiği	*pointer arithmetic*	İşaretçiye tamsayı eklemek; `p + 1` bir sonraki elemanı gösterir.
işaretçi dizisi	*array of pointers*	Elemanları işaretçi olan dizi: `int *p[3]`; `argv` buna örnektir.
işaretli	*signed*	Negatif değer de tutabilen tamsayı türü; `int` varsayılan olarak işaretlidir.
işlenen	*operand*	Operatörün üzerinde işlem yaptığı değer; `3 + 4` ifadesinde `3` ve `4`.
itme	*push*	Yığının en üstüne yeni bir eleman ekler.

K	İngilizce	Açıklama
kaçış dizisi	*escape sequence*	`\n`, `\t` gibi ters eğik çizgiyle başlayan özel karakter gösterimi.
kalan operatörü	*modulus operator*	`%` tamsayılarda bölmenin kalanını verir: `7 % 3` değeri `1`.
kapsam	*scope*	Bir adın geçerli olduğu kod bölgesi; süslü parantez blokları kapsamı sınırlar.
kayan noktalı sayı	*floating-point number*	Ondalıklı sayı; `float` ve `double` ile tutulur, `==` ile karşılaştırma güvenilmez.
kaynak kod	*source code*	İnsanın yazdığı, derleyicinin çevireceği `.c` uzantılı metin dosyası.
kesme	*truncation*	Ondalık kısmın atılması; `(int)2.9` değeri `2` olur.
kısa devre	*short-circuit*	`&&` (ve) ile `||` (veya) işleçlerinde sonuç belli olunca sağ taraf hesaplanmaz.
kısıtlı işaretçi	`restrict`	Gösterdiği belleğe fonksiyon boyunca yalnız bu işaretçiyle erişileceğine dair derleyiciye verilen söz; `memcpy` prototipinde görülür.
komut satırı argümanı	*command-line argument*	Programı çalıştırırken komut satırına yazılan değer; `argv` dizisinde tutulur.
koşul	*condition*	Doğru ya da yanlış sonuç veren ifade; `if` ve döngülerde kullanılır.
koşullu derleme	*conditional compilation*	`#ifdef`, `#ifndef`, `#if` ve `#endif` ile kod satırlarının derlemeye alınıp alınmayacağını önişlemcinin belirlemesi.
kuyruk	*queue*	İlk giren ilk çıkar (FIFO) ilkesiyle çalışan veri yapısı.
kütüphane	*library*	Hazır fonksiyon ve tanımların derlenmiş topluluğu; C standart kütüphanesi bunlardan biridir.

M–N	İngilizce	Açıklama
makro	*macro*	`#define` ile tanımlanan metin değiştirme kuralı; tip denetimi yapmaz.
mantıksal operatör	*logical operator*	`&&` (ve), `||` (veya) ve `!` (değil) ile koşulları birleştirir; sonucu `0` ya da `1`.
metin	*string*	Sonunda `'\0'` bulunan karakter dizisi; C'de `char` dizisi olarak tutulur.
metin dosyası	*text file*	İnsanın okuyabileceği karakterlerden oluşan dosya; ikili dosyanın karşıtıdır.
metin sabiti	*string literal*	Kodda çift tırnak içine doğrudan yazılan metin değeri.
nesne dosyası	*object file*	Derlenmiş ama henüz bağlanmamış makine kodu dosyası; `.o` uzantılıdır.
null karakter	*null character*	`'\0'`, metin sonunu işaretleyen sıfır değerli karakter.

O–Ö	İngilizce	Açıklama
ok operatörü	*arrow operator*	Yapı işaretçisinin üyesine erişir: `p->ad`, `(*p).ad` ile aynıdır.
onaltılık	*hexadecimal*	16 tabanlı sayı sistemi; C'de `0x1F` biçiminde yazılır.
operatör	*operator*	Değerler üzerinde işlem yapan simge; `+`, `==`, `&&` gibi.
öbek	*heap*	Dinamik bellek için ayrılan, `malloc` ile alınıp `free` ile geri verilen bölge.
ömür	*lifetime*	Nesnenin bellekte var olduğu süre; yerel değişkende blok, `static` için program boyunca.
ön ek	*prefix*	İşlenenden önce yazılan operatör; `++i` içindeki `++` ön ektir.
öncelik	*precedence*	İfadede hangi operatörün önce işleneceğini belirleyen sıra; `*`, `+`'tan önce gelir.
önişlemci	*preprocessor*	Derlemeden önce `#include` ve `#define` satırlarını işleyen aşama.
özyineleme	*recursion*	Bir fonksiyonun kendi kendini çağırması; temel durum bulunmalıdır.
özyinelemeli durum	*recursive case*	Fonksiyonun kendini daha küçük bir girdiyle çağırdığı kısım.

P–S	İngilizce	Açıklama
paralel dizi	*parallel arrays*	Aynı indisteki elemanları birbirine karşılık gelen birden çok dizi.
parametre	*parameter*	Fonksiyon tanımında yer alan ve çağrıda gelen değeri alan değişken.
prototip	*function prototype*	Fonksiyonun dönüş türünü ve parametrelerini önceden bildiren satır.
sabit	*constant*	Değeri değiştirilemeyen nesne; `const` anahtar sözcüğüyle tanımlanır.
sabit değer	*literal*	Kodda doğrudan yazılan değer: `42`, `3.14`, `'a'`.
sahiplik	*ownership*	Bir bellek bloğunu kimin serbest bırakacağını belirleyen sorumluluk.
sarkan else	*dangling else*	`else`'in hangi `if`'e ait olduğu belirsizliği; süslü parantezle önlenir.
sarkan işaretçi	*dangling pointer*	Serbest bırakılmış belleği hâlâ gösteren işaretçi; kullanmak tanımsız davranıştır.
sayım tipi	`enum`	Adlandırılmış tamsayı sabitlerinden oluşan tür; `enum` anahtar sözcüğüyle yazılır.
segmentasyon hatası	*segmentation fault*	Programın izinsiz bir bellek bölgesine erişmesi üzerine işletim sisteminin onu sonlandırması.
serbest bırakma	`free`	`free()` ile dinamik belleği geri verme; sonrasında işaretçi sarkar.
sıralama	*sorting*	Elemanları belirli bir ölçüte göre düzenleme işlemi.
sizeof operatörü	`sizeof`	Bir türün ya da nesnenin bayt cinsinden boyutunu verir.
son ek	*postfix*	İşlenenden sonra yazılan operatör; `i++` içindeki `++` son ektir.
son giren ilk çıkar	*LIFO*	Yığının çalışma ilkesi; en son eklenen eleman ilk çıkarılır.
sözde rastgele sayı	*pseudo-random number*	Tohuma bağlı olarak belirli bir dizi üreten, gerçekten rastgele olmayan sayı.
standart çıktı	`stdout`	Varsayılan çıktı akımı; `printf` buraya yazar.
standart girdi	`stdin`	Varsayılan giriş akımı; `scanf` buradan okur.
standart kütüphane	*standard library*	C'nin hazır fonksiyonları; `stdio.h`, `stdlib.h` gibi başlıklarla gelir.

T	İngilizce	Açıklama
tampon	*buffer*	Verinin geçici tutulduğu bellek alanı; klavye girişi önce tampona düşer.
tampon taşması	*buffer overflow*	Tamponun sınırının dışına yazılması; ciddi bir güvenlik açığıdır.
tamponu boşaltma	`fflush`	Çıkış tamponunu zorla yazdırır; `fflush(stdin)` standart C'de tanımsız davranıştır.
tamsayı	*integer*	Ondalıksız sayı; `int`, `long`, `char` gibi türlerle tutulur.
tamsayı yükseltmesi	*integer promotion*	`char` ve `short` değerlerin aritmetikte otomatik olarak `int`'e çevrilmesi; `char c = 100; c + c` bu yüzden `200` olur.
tanım	*definition*	Değişkene yer ayıran ya da fonksiyonun gövdesini veren yazım.
tanımlayıcı	*identifier*	Değişken ya da fonksiyon gibi varlıklara verilen ad; harfle ya da `_` ile başlar.
tanımsız davranış	*undefined behavior*	Standardın sonucunu tanımlamadığı kod; sonuç her derleyicide farklı olabilir.
taşma	*overflow*	Bir türün taşıyabileceğinden büyük değer; `int` taşması tanımsız davranıştır.
tekli operatör	*unary operator*	Tek işlenen alan operatör; `-x`, `!x`, `&x` gibi.
temel durum	*base case*	Özyinelemenin durduğu, kendini çağırmadan sonuç veren durum.
tip	*type*	Değerin boyutunu ve üzerindeki işlemleri belirleyen sınıflandırma; `int`, `char`.
tip dönüştürme	*cast (type casting)*	Değeri açıkça başka türe çevirme; `(float) x` biçiminde yazılır.
tip tanımı	`typedef`	Bir türe yeni ad verir; uzun `struct` adlarını kısaltmak için sık kullanılır.
tohum	*seed*	Sözde rastgele sayı üretecinin başlangıç değeri; `srand` ile verilir.

U-Ü	İngilizce	Açıklama
uçucu değişken	`volatile`	Değeri program dışından değişebilen (donanım, sinyal) değişken; derleyici okuma ve yazmalarını optimizasyonla silemez.
UTF-8	*Unicode Transformation Format-8*	Unicode karakterlerini 1–4 bayta kodlayan standart; Türkçe `ş` harfi 2 bayttır.
uyarı	*warning*	Derlemeyi durdurmayan ama şüpheli bulunan kod için derleyicinin verdiği mesaj.
üye	*member*	Yapının içindeki her bir alan; `ogrenci.ad` ile erişilir.

V–Y	İngilizce	Açıklama
veri yapısı	*data structure*	Verileri bellekte düzenli tutan ve üzerinde işlem yapılan yapı; bağlı liste, yığın, kuyruk gibi.
yan etki	*side effect*	Fonksiyonun değer dönmenin dışında yaptığı değişiklik; `printf` ekrana yazar.
yanlış	*false*	C'de sıfır yanlış kabul edilir.
yapı	`struct`	Farklı türdeki değişkenleri tek bir ad altında toplayan, kullanıcı tanımlı tür.
yapı etiketi	*struct tag*	`struct` sözcüğünden sonra gelen tür adı; `struct Dugum` içinde `Dugum` etikettir.
yerel değişken	*local variable*	Bir blok ya da fonksiyon içinde tanımlanan, yalnız orada geçerli değişken.
yığın	*stack (memory)*	Yerel değişkenlerin ve çağrı bilgilerinin tutulduğu bellek bölgesi; öbeğin karşıtı.
yığın çerçevesi	*stack frame*	Bir fonksiyon çağrısının yerel değişkenlerini ve dönüş bilgisini tuttuğu yığın bölümü.
yığın taşması	*stack overflow*	Yığın belleği dolduğunda oluşan hata; sonsuz özyinelemede sık görülür.
yığın veri yapısı	*stack*	Son giren ilk çıkar (LIFO) ilkesiyle çalışan veri yapısı; `push` ve `pop` ile kullanılır.
yorum	*comment*	`//` ya da `/* */` ile yazılan, derleyicinin yok saydığı açıklama.

EK C

Sık Karşılaşılan Hata Mesajları

Bu ekte, C öğrenirken gcc'den en sık alacağın mesajları topladık. Derleme hatalarıyla uyarılar, onları **gerçekten** üreten küçük programlarla gösteriliyor; altındaki çıktı gcc 16'nın verdiği mesajın kendisidir. Mesajlar İngilizcedir; Türkçe karşılıkları ve çözüm yolları açıklamalarda. Bağlama ve çalışma zamanı hatalarının mesajları işletim sistemine göre değiştiği için bunlar için ayrıca belirtilmiş örnekler verdik.

Bir gcc mesajı şu kalıptadır:

```
dosya.c:SATIR:SÜTUN: error: MESAJ
    5 |     printf("Merhaba\n")
      |                       ^   ← gcc'nin takıldığı yer
```

Başındaki sözcük mesajın ağırlığını söyler:

Sözcük	Anlamı	Sonuç
`error`	Kod C kurallarına uymuyor	Program **derlenmez**, çalıştırılabilir dosya oluşmaz
`warning`	Kod derlenir ama büyük olasılıkla bir yanlışlık var	Program çalışır, ama sonucu güvenilmez olabilir
`note`	Bir önceki mesaja ek bilgi (ör. "tanım burada")	Tek başına bir sorun değildir

- **İpucu: Hata mesajı okuma alışkanlığı**
  - Derleme hatalarında **her zaman ilk hatadan başla**. Tek bir eksik `}` ya da `"`, arkasından sahte hatalar zinciri doğurabilir; ilkini düzeltip yeniden derle.
  - Satır numarası, hatanın **fark edildiği** yeri gösterir; asıl neden bir üst satırda olabilir. Bulamazsan yukarıya doğru bak.
  - Uyarıyı hata gibi ciddiye al: `-Wall -Wextra` ile derle (Bölüm 1) ve uyarısız derlenene kadar bırakma.

- **Not: Senin gcc'n farklı yazabilir**
  - gcc 14'ten itibaren bazı eski **uyarılar hata** oldu: `implicit declaration of function`, `makes pointer from integer without a cast`, `incompatible pointer type`, `'return' with no value`. Eski bir MinGW kullanıyorsan (gcc 13 ve öncesi) aynı kod için yalnızca `warning` görürsün ve program derlenir. Düzeltmesi değişmez.
  - Linux'ta UTF-8 terminalde tırnaklar düz `'x'` yerine kıvrık `‘x’` görünebilir. Anlam aynıdır.
  - Aşağıdaki çıktılar `-std=c17 -Wall -Wextra` ile alınmıştır.

Derleme hataları

`expected ';' before 'return'`

```c
#include <stdio.h>

int main(void) {
    printf("Merhaba\n")
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
noktaliVirgul.c: In function 'main':
noktaliVirgul.c:4:24: error: expected ';' before 'return'
    4 |     printf("Merhaba\n")
      |                        ^
      |                        ;
    5 |     return 0;
      |     ~~~~~~
```

Ne demek? Komutun sonundaki noktalı virgül eksik. gcc bunu, sonraki komuta (`return`) gelince fark eder. Değişken tanımında (`int x = 5`) aynı eksiklik `expected ',' or ';' before 'printf'` olarak görünür.

Çözüm:

- `^` ve `;` işaretinin gösterdiği yere `;` koy. gcc 16 çoğu durumda hatayı eksik `;`'nin olduğu satırda gösterir (yukarıdaki gibi); değişken tanımında (`int x = 5`) ve eski sürümlerde ise bir **sonraki** satırı gösterir, o yüzden bir üst satıra da bak.
- Benzer mesajlar: `expected ')' before ...` (kapanmayan parantez), `expected identifier or '(' before ...` (fonksiyon dışına yazılmış komut ya da fazladan `;`).

`expected declaration or statement at end of input`

```c
#include <stdio.h>

int main(void) {
    int x = 5;
    if (x > 3) {
        printf("buyuk\n");
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
kapanmayanSus.c: In function 'main':
kapanmayanSus.c:8:1: error: expected declaration or statement at end of input
    8 | }
      | ^
```

Ne demek? Dosyanın sonuna gelindi ama açık bir `{` hâlâ kapanmamış. Yukarıda `if`'in `}` karakteri eksik; son `}` ona gitti, `main` açık kaldı.

Çözüm:

- Girintilere bak: her `{`'nin altına, aynı girintide bir `}` düşmeli. Hata satırı hep dosyanın sonunu gösterir; asıl eksiklik daha yukarıdadır.

- VS Code'da kodu otomatik biçimlendir (`Shift+Alt+F`); girinti kayan yer, eksik `}` ya da fazla `{`'nin yeridir. Parantez eşlemesi renkli gösterilir.

`'Toplam' undeclared (first use in this function)`

```c
#include <stdio.h>

int main(void) {
    int toplam = 10;
    printf("%d\n", Toplam);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
tanimsizAd.c: In function 'main':
tanimsizAd.c:5:20: error: 'Toplam' undeclared (first use in this function); did you mean 'toplam'?
    5 |     printf("%d\n", Toplam);
      |                     ^~~~~~
      |                     toplam
tanimsizAd.c:5:20: note: each undeclared identifier is reported only once for each function it
appears in
tanimsizAd.c:4:9: warning: unused variable 'toplam' [-Wunused-variable]
    4 |     int toplam = 10;
      |         ^~~~~~
```

Ne demek? "Bu adı tanımıyorum." gcc çoğu zaman `did you mean 'toplam'?` diye en yakın adı önerir. `note` satırı, aynı yanlış adı bir fonksiyonda kaç kez kullanırsan kullan yalnızca ilk kullanımda bildireceğini söyler.

Çözüm: Üç olası neden var:

- Ad yanlış yazılmış ya da büyük/küçük harf farklı (`Toplam` ≠ `toplam`, `Printf` ≠ `printf`). C büyük/küçük harfe duyarlıdır.
- Değişken **kapsam dışında**: bir `for` ya da `{ }` bloğunun içinde tanımlanıp dışında kullanılmış (Bölüm 5, 6).
- Değişken hiç tanımlanmamış ya da kullanıldığı satırdan **sonra** tanımlanmış.

`implicit declaration of function 'printf'`

```c
int main(void) {
    printf("Merhaba\n");
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
includeYok.c: In function 'main':
includeYok.c:2:5: error: implicit declaration of function 'printf' [-Wimplicit-function-declaration]
    2 |     printf("Merhaba\n");
      |     ^~~~~~
includeYok.c:1:1: note: include '<stdio.h>' or provide a declaration of 'printf'
  +++ |+#include <stdio.h>
    1 | int main(void) {
includeYok.c:2:5: warning: incompatible implicit declaration of built-in function 'printf' [-
Wbuiltin-declaration-mismatch]
    2 |     printf("Merhaba\n");
      |     ^~~~~~
includeYok.c:2:5: note: include '<stdio.h>' or provide a declaration of 'printf'
```

Ne demek? gcc, `printf` adında bir fonksiyon tanımıyor; çünkü onu tanıtan başlık dosyası eklenmemiş. `note` satırı çözümü doğrudan söyler: `#include <stdio.h>`.

Çözüm:

- Eksik `#include`'u ekle. Sık gerekenler: `printf`, `scanf`, `fgets`, `fopen` için `<stdio.h>`; `malloc`, `free`, `atoi`, `rand` için `<stdlib.h>`; `strlen`, `strcpy`, `strcmp` için `<string.h>`; `sqrt`, `pow` için `<math.h>`.
- Kendi fonksiyonunu `main`'den **sonra** tanımladıysan `main`'den önce fonksiyonun **prototipini** yaz: `int kare(int x);` (Bölüm 6). Bu durumda mesaj `implicit declaration of function 'kare'` olur.
- Bu hata gcc 14 ve sonrasında `error`'dur; eski sürümlerde `warning` görünür ve program yine de (tehlikeli biçimde) derlenir.

`too few arguments to function` ve `too many arguments to function`

```c
int topla(int a, int b) {
    return a + b;
}

int main(void) {
    int x = topla(1);
    int y = topla(1, 2, 3);
    return x + y;
}
```

Derleyici çıktısı – program derlenmez

```
argumanSayisi.c: In function 'main':
argumanSayisi.c:6:13: error: too few arguments to function 'topla'; expected 2, have 1
    6 |     int x = topla(1);
      |             ^~~~~
argumanSayisi.c:1:5: note: declared here
    1 | int topla(int a, int b) {
      |     ^~~~~
argumanSayisi.c:7:13: error: too many arguments to function 'topla'; expected 2, have 3
    7 |     int y = topla(1, 2, 3);
      |             ^~~~~       ~
argumanSayisi.c:1:5: note: declared here
    1 | int topla(int a, int b) {
      |     ^~~~~
```

Ne demek? Fonksiyon çağrısında verdiğin argüman sayısı, tanımdaki parametre sayısıyla uyuşmuyor. Mesaj `expected 2, have 1` diyerek ikisini de söyler; `note: declared here` fonksiyonun tanımlandığı satırı gösterir.

Çözüm: Parametre sayısıyla argüman sayısını eşitle. Sıra ve tipler de aynı olmalı (Bölüm 6). Prototipi `int topla();` diye boş parantezle yazarsan C17'de gcc argüman sayısını denetlemez (C23'te `()`, `(void)` demektir); prototipte hep tam parametre listesi yaz.

`assignment to expression with array type`

```c
int main(void) {
    char ad[10];
    ad = "Ali";
    return ad[0];
}
```

Derleyici çıktısı – program derlenmez

```
diziAtama.c: In function 'main':
diziAtama.c:3:8: error: assignment to expression with array type
    3 |     ad = "Ali";
      |        ^
```

Ne demek? Bir dizinin adına `=` ile yeni değer atayamazsın; dizi adı, sabit bir bellek bloğunun adresidir. `ad = "Ali"` ancak tanımla **aynı satırda** (ilk değer olarak) yazılabilir.

Çözüm:

- Tanımda değer ver: `char ad[10] = "Ali";`
- Sonradan değiştireceksen kopyala: `strcpy(ad, "Ali");` (`<string.h>`, Bölüm 8). Sınır aşımına karşı `snprintf(ad, sizeof ad, "%s", "Ali");` daha güvenlidir.
- Dizilerde `==` ile karşılaştırma ve `=` ile atama çalışmaz; `strcmp` ve `strcpy` gerekir.

`invalid operands to binary %`

```c
#include <stdio.h>

int main(void) {
    double d = 7.5;
    printf("%f\n", d % 2);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
ondalikMod.c: In function 'main':
ondalikMod.c:5:22: error: invalid operands to binary % (have 'double' and 'int')
    5 |     printf("%f\n", d % 2);
      |                     ^
```

Ne demek? `%` (mod) işlemi yalnızca **tamsayılarla** yapılır. Parantez içindeki `have 'double' and 'int'` operandların tipini söyler. Aynı mesaj `+`, `&`, `<<` gibi işleçlerde de çıkar: `"abc" + "def"` için `(have 'char *' and 'char *')`.

Çözüm: Ondalık sayılarda `<math.h>` içindeki `fmod(d, 2)` kullan (Linux'ta `-lm`), ya da değerin tamsayı olduğundan eminsen `(int) d % 2` yaz (Bölüm 2).

`'p' is a pointer; did you mean to use '->'?`

```c
#include <stdio.h>

struct Dugum {
    int deger;
    struct Dugum *sonraki;
};

int main(void) {
    struct Dugum d = {1, NULL};
    struct Dugum *p = &d;
    int a = p.deger;
    int b = d->deger;
    int c = p->deger;
    return a + b + c;
}
```

Derleyici çıktısı – program derlenmez

```
uyeErisim.c: In function 'main':
uyeErisim.c:11:14: error: 'p' is a pointer; did you mean to use '->'?
   11 |     int a = p.deger;
      |             ^
      |             ->
uyeErisim.c:12:14: error: invalid type argument of '->' (have 'struct Dugum')
   12 |     int b = d->deger;
      |             ^~
```

Ne demek? Yapının (struct) üyesine erişirken işleç tipe göre seçilir:

- `d.deger`: `d` yapının **kendisi** ise nokta.
- `p->deger`: `p` yapıyı gösteren **işaretçi** ise ok. (`p->deger`, `(*p).deger` ile aynıdır.)

İlk mesaj noktayı işaretçiye uygulamaktan, ikincisi (`invalid type argument of '->' (have 'struct Dugum')`) oku yapının kendisine uygulamaktan doğar. Eski gcc sürümleri ilk durumu `request for member 'deger' in something not a structure or union` diye anlatır.

Çözüm: Değişkenin tanımına bak: `struct Dugum d` ise `.`, `struct Dugum *p` ise `->` (Bölüm 11, bağlı listelerde Bölüm 14).

`unknown type name 'string'`

```c
struct Ogrenci {
    int no;
};

void yaz(string ad, bool dolu);
Ogrenci o;

int main(void) {
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
bilinmeyenTip.c:5:10: error: unknown type name 'string'
    5 | void yaz(string ad, bool dolu);
      |          ^~~~~~
bilinmeyenTip.c:5:21: error: unknown type name 'bool'
    5 | void yaz(string ad, bool dolu);
      |                     ^~~~
bilinmeyenTip.c:1:1: note: 'bool' is defined in header '<stdbool.h>'; this is probably fixable by
adding '#include <stdbool.h>'
  +++ |+#include <stdbool.h>
    1 | struct Ogrenci {
bilinmeyenTip.c:6:1: error: unknown type name 'Ogrenci'; use 'struct' keyword to refer to the type
    6 | Ogrenci o;
      | ^~~~~~~
      | struct
```

Ne demek? gcc bu tip adını tanımıyor. Üç yaygın durum, hepsi yukarıda:

- `string`: C'de `string` tipi yoktur; metin `char` dizisidir (`char ad[20]` ya da `const char *ad`, Bölüm 8). Bu hata Java ve Python'dan gelenlerin ilk sürprizidir.
- `bool`: `#include <stdbool.h>` gerekir (C23'te `bool` anahtar sözcüktür, include gerekmez). `note` satırı bunu söyler.

- `Ogrenci`: `struct Ogrenci { ... };` ile tanımlanmış bir yapıya başvururken `struct` sözcüğünü yazmak zorundasın (`struct Ogrenci o;`). Her seferinde yazmak istemezsen tanımda `typedef` kullan (Bölüm 11).

`stray '\342' in program`

Aşağıdaki programda tırnaklar klavyeden değil, Word ya da bir web sayfasından kopyalanmış **kıvrık tırnaklardır** (`“` ve `”`). Gözle düz tırnağa çok benzerler.

```c
#include <stdio.h>

int main(void) {
    printf(“Merhaba”);
    return 0;
}
```

Derleyici çıktısı – program derlenmez

```
akilliTirnak.c: In function 'main':
akilliTirnak.c:4:12: error: stray '\342' in program
    4 |     printf(<U+201C>Merhaba<U+201D>);
      |            ^~~~~~~
akilliTirnak.c:4:20: error: stray '\342' in program
    4 |     printf(<U+201C>Merhaba<U+201D>);
      |                    ^~~~~~~
akilliTirnak.c:4:13: error: 'Merhaba' undeclared (first use in this function)
    4 |     printf(“Merhaba”);
      |             ^~~~~~
akilliTirnak.c:4:13: note: each undeclared identifier is reported only once for each function it
appears in
```

Ne demek? `\342` kıvrık tırnağın UTF-8 kodunun ilk baytıdır (sekizlik düzende). C'de dize ancak düz `"` ile açılır; gcc bu karakteri ne olduğunu anlamadan “yabancı” (stray) sayar. Çıktıdaki `<U+201C>`, karakterin Unicode numarasıdır; UTF-8 bir terminalde tırnağın kendisini görürsün. Bu yüzden bir de `'Merhaba' undeclared` hatası çıkar: gcc `Merhaba`'yı dize değil, değişken adı sanar.

Çözüm: Kıvrık tırnakları silip klavyeden yeniden `"` ve `'` yaz. Aynı mesaj kopyalanmış `–` (uzun tire), `…` (üç nokta karakteri) ve görünmeyen boşluk karakterlerinde de çıkar. Ders notundan kod kopyalıyorsan düz metin olarak yapıştır (`Ctrl+Shift+V`).

`makes pointer from integer without a cast`

```c
void artir(int *p) {
    (*p)++;
}

int main(void) {
    int sayi = 5;
    int *p = 5;
    artir(sayi);
    return *p;
}
```

Derleyici çıktısı – program derlenmez

```
intIsaretci.c: In function 'main':
intIsaretci.c:7:14: error: initialization of 'int *' from 'int' makes pointer from integer without a
cast [-Wint-conversion]
    7 |     int *p = 5;
      |              ^
intIsaretci.c:8:11: error: passing argument 1 of 'artir' makes pointer from integer without a cast
[-Wint-conversion]
    8 |     artir(sayi);
      |           ^~~~
      |           |
      |           int
intIsaretci.c:8:11: note: possible fix: take the address with '&'
    8 |     artir(sayi);
      |           ^~~~
      |           &
intIsaretci.c:1:17: note: expected 'int *' but argument is of type 'int'
    1 | void artir(int *p) {
      |            ~~~~~^
```

Ne demek? Bir tamsayıyı, işaretçi beklenen yere verdin. gcc iki yerde yakaladı: `int *p = 5;` ve işaretçi parametre alan `artir`'a `sayi`'yı doğrudan vermek. İkincisinde `note` satırı çözümü söylüyor: `take the address with '&'`.

Çözüm:

- Bir değişkenin **adresini** vermek için `&` kullan: `artir(&sayi);` ve `int *p = &sayi;` (Bölüm 9).
- Ters yönde `int x = &y;` için `makes integer from pointer without a cast` çıkar.
- İşaretçi tipleri uymazsa (`int *p = &d;`, `d` bir `double`) `initialization of 'int *' from incompatible pointer type 'double *'` hatası alırsın. İşaretçinin tipi, gösterdiği değişkenin tipiyle aynı olmalıdır.

Önemli uyarılar

Uyarı veren program **derlenir ve çalışır**; ama uyarı çoğu zaman bir mantık hatasını ya da tanımsız davranışı işaret eder. Aşağıdaki programlar çalıştırılmadı: çoğu tanımsız davranış içerdiği için çıktıları belirsizdir, yalnız gcc'nin uyarısı gösteriliyor.

`format '%d' expects argument of type 'int', but argument 2 has type 'double'`

```c
#include <stdio.h>

int main(void) {
    double not1 = 85.5;
    int sayi = 7;
    printf("%d\n", not1);
    printf("%f\n", sayi);
    printf("%d %d\n", sayi);
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
bicimHatasi.c: In function 'main':
bicimHatasi.c:6:14: warning: format '%d' expects argument of type 'int', but argument 2 has type
'double' [-Wformat=]
    6 |     printf("%d\n", not1);
      |             ~^     ~~~~
      |              |     |
      |              int   double
      |             %f
bicimHatasi.c:7:14: warning: format '%f' expects argument of type 'double', but argument 2 has type
'int' [-Wformat=]
    7 |     printf("%f\n", sayi);
      |             ~^     ~~~~
      |              |     |
      |              |     int
      |              double
      |             %d
bicimHatasi.c:8:17: warning: format '%d' expects a matching 'int' argument [-Wformat=]
    8 |     printf("%d %d\n", sayi);
      |                ~^
      |                 |
      |                 int
```

Ne demek? `printf`'in biçim belirteci ile verdiğin değerin tipi uyuşmuyor. `argument 2`, biçim dizesinden sonraki **ilk** değerdir. gcc altına uygun belirteci de önerir (`%f`). Son satırda belirteç sayısı değer sayısından fazla (`expects a matching 'int' argument`).

Çözüm: Tip ile belirteci eşleştir: `int` → `%d`, `double` → `%f`, `char` → `%c`, metin → `%s`, `size_t` → `%zu` (Bölüm 3). Uyarıyı görmezden gelirsen ekrana çöp ya da `0` yazılır.

`format '%d' expects argument of type 'int *'`

```c
#include <stdio.h>

int main(void) {
    int yas = 0;
    double boy = 0;
    scanf("%d", yas);
    scanf("%d", &boy);
    printf("%d %f\n", yas, boy);
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
scanfAdres.c: In function 'main':
scanfAdres.c:6:13: warning: format '%d' expects argument of type 'int *', but argument 2 has type
'int' [-Wformat=]
    6 |     scanf("%d", yas);
      |            ~^   ~~~
      |             |   |
      |             |   int
      |             int *
scanfAdres.c:7:13: warning: format '%d' expects argument of type 'int *', but argument 2 has type
'double *' [-Wformat=]
    7 |     scanf("%d", &boy);
      |            ~^   ~~~~
      |             |   |
      |             |   double *
      |             int *
      |            %le
```

Ne demek? `scanf` değeri değişkene **yazmak** için değişkenin adresini ister. İlk satırda `&` unutulmuş: `yas`'ın değeri (`int`) verilmiş, `int *` beklenirken. İkinci satırda `&` var ama `double` için `%d` yazılmış (`%lf` olmalı).

Çözüm: `scanf("%d", &yas);` ve `scanf("%lf", &boy);`. Tek istisna dizilerdir: `scanf("%s", ad);` (dizi adı zaten adrestir, Bölüm 3 ve 8). `&` unutmak programı çalışma zamanında **çökertebilir**.

`suggest parentheses around assignment used as truth value`

```c
#include <stdio.h>

int main(void) {
    int x = 3;
    if (x = 5) {
        printf("bes\n");
    }
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
atamaIf.c: In function 'main':
atamaIf.c:5:9: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    5 |     if (x = 5) {
      |         ^
```

Ne demek? `if` koşulunda `=` (atama) kullandın; karşılaştırma `==` ile yapılır. `x = 5` ifadesi `x`'e 5 yazar ve değeri 5 (sıfırdan farklı → doğru) olduğu için koşul **her zaman doğrudur**. Üstelik `x` değişir. Sınavın klasik tuzağıdır (Bölüm 4).

Çözüm: `if (x == 5)` yaz. Atamayı bilerek yapıyorsan fazladan parantez ekle: `if ((x = 5))` (ya da atamayı ayrı satıra al).

`'toplam' is used uninitialized`

```c
#include <stdio.h>

int main(void) {
    int toplam;
    int adet = 3;
    printf("%d\n", toplam + 1);
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
kullanilmamis.c: In function 'main':
kullanilmamis.c:5:9: warning: unused variable 'adet' [-Wunused-variable]
    5 |     int adet = 3;
      |         ^~~~
kullanilmamis.c:6:5: warning: 'toplam' is used uninitialized [-Wuninitialized]
    6 |     printf("%d\n", toplam + 1);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~
kullanilmamis.c:4:9: note: 'toplam' was declared here
    4 |     int toplam;
      |         ^~~~~~
```

Ne demek? `toplam`'a hiç değer verilmeden okundu. Yerel değişkenlerin ilk değeri **çöptür**; hangi sayı çıkacağı bilinmez. `note` satırı değişkenin tanımlandığı yeri gösterir. Aynı çıktıdaki ilk uyarı (`unused variable 'adet'`) tanımlanıp hiç kullanılmayan değişkeni anlatır: gereksiz tanımı sil ya da kullanmayı unuttuğun yeri bul.

Çözüm: Değişkene tanımda ilk değer ver: `int toplam = 0;` (Bölüm 2). Bu uyarı bazı derleyici ayarlarında (`-O0`) yalnızca basit durumları yakalar; uyarı yok diye değişkenin başlatıldığından emin olma.

`control reaches end of non-void function`

```c
#include <stdio.h>

int isaret(int n) {
    if (n > 0) {
        return 1;
    } else if (n < 0) {
        return -1;
    }
}

int main(void) {
    printf("%d\n", isaret(5));
    return 0;
}
```

Derleyici uyarısı – gcc -Wall -Wextra

```
donusYok.c: In function 'isaret':
donusYok.c:9:1: warning: control reaches end of non-void function [-Wreturn-type]
    9 | }
      | ^
```

Ne demek? Fonksiyon `int` döndüreceğini söylüyor, ama en az bir yolda (`n == 0`) `return` görmeden bitiyor. gcc koşulların içini değerlendirmez; "ya hiçbiri doğru olmazsa?" diye düşünür. O yolda dönen değer çöptür.

Çözüm: Sona her durumu karşılayan bir `return` ekle (burada `return 0;`) ya da son `else if`'i düz `else` yap (Bölüm 6).

Diğer sık uyarılar

Uyarı (kısaltılmış)	Ne demek?	Çözüm
`comparison of integer expressions of different signedness: 'int' and 'size_t' ...`	İşaretli (`int`) ile işaretsiz (`size_t`, `unsigned`) değer karşılaştırılıyor; `strlen(s)` ve `sizeof` işaretsizdir (Bölüm 2)	Sayaç tipini `size_t` yap ve uzunluğu bir kez hesapla: `size_t n = strlen(s); for (size_t i = 0; i < n; i++)` (koşula `strlen(s)` yazarsan her turda metin baştan sayılır, döngü O(n²) olur)
`this statement may fall through`	`switch`'te `break` unutulmuş; akış sonraki `case`'e düşecek (Bölüm 4)	`break;` ekle; düşme bilerek ise `// fall through` yaz
`function returns address of local variable`	Fonksiyon bittiğinde yok olan yerel değişkenin adresi döndürülüyor	Değeri döndür, ya da `malloc` / `static` kullan (Bölüm 9, 10)
`unused parameter 'b'`	Parametre fonksiyonda hiç kullanılmamış	Kullan ya da parametreyi kaldır

Bağlama (linker) hataları

Bağlama hatası, derleme **bittikten sonra** gelir: söz diziminde sorun yoktur, ama bağlayıcı (linker, `ld`) çağrılan bir fonksiyonun kodunu hiçbir yerde bulamaz (Bölüm 1'deki dördüncü aşama). Satır numarası yerine `.o` dosyası ve `in function 'main'` gibi ifadeler görürsün, son satır hep aynıdır: `collect2: error: ld returned 1 exit status`.

Bu mesajlar macOS'ta farklı yazılır (`Undefined symbols for architecture arm64`), o yüzden aşağıdaki örnekleri **Windows (MinGW) ve Linux'taki gcc mesajları** olarak elle verdik; uzun yolları `...` ile kısalttık. Eski `ld` sürümleri tırnağı `'main'` yerine `` `main' `` biçiminde yazar.

`undefined reference to 'WinMain'`

```
C:/.../ld.exe: C:/.../crt2.o:crtexe.c:(.text+0x...): undefined reference to 'WinMain'
collect2.exe: error: ld returned 1 exit status
```

Linux'ta aynı hata şöyle görünür:

```
/usr/bin/ld: .../Scrt1.o: in function `_start':
(.text+0x1b): undefined reference to 'main'
collect2: error: ld returned 1 exit status
```

Ne demek? Bağlayıcı programın başlangıç noktasını, yani `main`'i bulamadı. Windows'ta `main` bulunmayınca `WinMain` (grafik arayüzlü programların başlangıcı) aranır ve mesaj onu söyler.

Çözüm:

- `main` yazımını kontrol et: `Main`, `mian`, `int main` yerine `int maın` (Türkçe `ı`!) gibi hataları gcc ancak bu aşamada yakalar.
- `main` başka bir `.c` dosyasındaysa o dosyayı da derlemeye ekle (aşağıdaki son girdiye bak).
- Dosya bir `.h` ya da boş değil mi, doğru dosyayı mı derliyorsun? VS Code'da önce doğru dosyayı seç.

`undefined reference to 'sqrt'`

`<math.h>` eklenmiş, program derlenmiş ama bağlama sırasında şu mesajı alırsın:

```
/usr/bin/ld: /tmp/ccXXXXXX.o: in function `main':
kok.c:(.text+0x1e): undefined reference to 'sqrt'
collect2: error: ld returned 1 exit status
```

Ne demek? `#include <math.h>` yalnız fonksiyonun **prototipini** getirir; kodu ayrı bir kütüphanededir (`libm`) ve Linux'ta ona açıkça bağlanmak gerekir. `pow`, `sin`, `floor`, `fmod` için de aynıdır.

Çözüm: Komutun **sonuna** `-lm` ekle:

```
gcc -std=c17 -Wall -Wextra kok.c -o kok -lm
```

`-lm` kaynak dosyadan sonra gelmelidir; öne yazarsan Linux'ta yine aynı hatayı alabilirsin. Windows'ta (MinGW) matematik fonksiyonları ana kütüphanede olduğu için `-lm` gerekmez, yazman da zarar vermez.

`undefined reference to 'topla'`

```c
int topla(int a, int b);        // prototip var

int main(void) {
    return topla(1, 2);         // çağrı var, ama fonksiyonun gövdesi yok
}
```

```
/usr/bin/ld: /tmp/ccXXXXXX.o: in function `main':
program.c:(.text+0x...): undefined reference to 'topla'
collect2: error: ld returned 1 exit status
```

Ne demek? Derleyici prototipe bakıp çağrının doğru olduğuna karar verdi; ama bağlayıcı `topla`'nın gövdesini hiçbir `.o` dosyasında bulamadı.

Çözüm:

- Fonksiyonun gövdesini yaz ya da yazımını kontrol et (`topla` ≠ `Topla`).
- Fonksiyon başka bir dosyadaysa **tüm** `.c` dosyalarını birlikte derle: `gcc main.c topla.c -o program` (Bölüm 6, "Programı dosyalara bölmek"). VS Code'un varsayılan derleme görevi yalnızca açık dosyayı derler.
- Fonksiyonun ya da değişkenin başında `static` varsa ad o `.c` dosyasına gizlidir; başka dosyadan çağırınca da aynı hata çıkar (Bölüm 6).
- Ters durum: aynı fonksiyonu **iki** `.c` dosyasında tanımlarsan `multiple definition of 'topla'` alırsın; gövde yalnız bir yerde olmalı. Başlığa (`.h`) gövde ya da `int sayac = 0;` gibi tanım yazma, yalnız bildirim koy (Bölüm 6).

Çalışma zamanı hataları

Çalışma zamanı hatası, program **derlendikten ve başladıktan sonra** ortaya çıkar. C'de çoğu zaman düzgün bir hata mesajı yoktur: Dizi taşması gibi hatalar çoğunlukla sessizce yanlış sonuç verir; program çöktüğünde ise yazıyı işletim sistemi ya da kabuk (terminal) yazar, gcc değil. Bu yüzden mesajlar sistemden sisteme değişir.

`Segmentation fault`

```c
#include <stdio.h>

int main(void) {
    int *p = NULL;
    printf("Önce\n");
    *p = 5;
    printf("Sonra\n");
    return 0;
}
```

Çıktı – program çöker (Segmentation fault)

```
Önce
```

Ne demek? Program, kendisine ait olmayan bir bellek adresine eriştiği için işletim sistemi tarafından durduruldu (segmentation fault, kısaca *segfault*). Yukarıda `NULL` işaretçiye yazıldı: `Önce` yazıldı, `Sonra` hiç yazılmadı. Mesajı yazan terminaldir, programın kendisi değil.

- **Linux:** `Segmentation fault (core dumped)`. **macOS:** `Segmentation fault: 11` ya da `Bus error: 10`.
- **Windows (PowerShell, cmd):** Hiçbir yazı çıkmaz; program **sessizce kapanır** (MSYS2 terminalinde `Segmentation fault` yazabilir). Çıkış kodu `-1073741819` (`0xC0000005`) olur. PowerShell'de `echo $LASTEXITCODE`, cmd'de `echo %errorlevel%` ile görürsün.

Aşağıdakilerin hepsi segfault'a yol açabilir. Hepsi **tanımsız davranıştır**: program çökebilir de çökmeyebilir de, o yüzden kodu çalıştırmadan gösteriyoruz:

```c
int *p;
*p = 5;                      // ilk değer verilmemiş işaretçi: rastgele adres

int dizi[3];
dizi[100000] = 1;            // dizi sınırının çok dışına yazma

char *s = "Ali";
s[0] = 'E';                  // string sabiti salt okunur bellektedir

int *q = malloc(sizeof(int));
free(q);
*q = 7;                      // free sonrası kullanım

int f(int n) { return f(n + 1); }   // temel durumu olmayan özyineleme
```

Sonuncusu yığın taşmasıdır (stack overflow): her çağrı yığında yer ayırır, yığın dolunca program segfault ile (Windows'ta `0xC00000FD` koduyla) çöker (Bölüm 13).

Çözüm:

- Her işaretçiye tanımda ya bir adres ya da `NULL` ver; `NULL` olabilecek işaretçiyi kullanmadan önce `if (p != NULL)` ile denetle (Bölüm 9).
- `malloc` sonucunu kontrol et, `free`'den sonra işaretçiye `NULL` ata (Bölüm 10).
- Dizi indislerinin `0` ile `boyut - 1` arasında kaldığından emin ol (Bölüm 7). Metni değiştireceksen `char *s = "Ali"` yerine `char s[] = "Ali"` yaz (Bölüm 8).
- Nerede çöktüğünü bulmak için hata ayıklayıcıyı kullan (aşağıdaki bölüm).

`Floating point exception`

```c
int a = 10;
int b = 0;
printf("%d\n", a / b);        // tamsayı sıfıra bölme
```

Ne demek? Bu kod x86 işlemcilerde (çoğu Windows ve Linux bilgisayarı) program **çöker**. Adı yanıltıcıdır: hata ondalık sayıdan değil, **tamsayı** bölmesinden gelir. Aynı hata `a % b` için de çıkar. ARM işlemcilerde (ör. Apple M serisi) aynı kod çökmediği için bu örneği çalıştırıp çıktısını göstermiyoruz.

- **Linux:** `Floating point exception (core dumped)`.
- **Windows:** Mesaj yok, program kapanır; çıkış kodu `-1073741676` (`0xC0000094`).

`double` ile sıfıra bölmede ise program **çökmez**: sonuç `inf` ya da `nan` olur ve ekrana öyle yazılır.

Çözüm: Bölmeden önce bölenin sıfır olmadığını denetle: `if (b != 0) { ... } else { printf("Sıfıra bölünemez\n"); }` (Bölüm 4). Bölen kullanıcıdan geliyorsa bu denetim zorunludur.

`free(): double free detected in tcache 2`

```c
int *p = malloc(sizeof(int));
free(p);
free(p);                      // aynı bloğu ikinci kez serbest bırakma
```

```
free(): double free detected in tcache 2
Aborted (core dumped)
```

Ne demek? Bu mesaj Linux'taki glibc'nin `malloc`/`free` kodundan gelir ve programı `abort()` ile durdurur. Aynı bloğu iki kez `free` etmek tanımsız davranıştır; glibc bunu fark ederse program böyle sonlanır. Windows'ta mesaj çıkmaz, program sessizce kapanır (çıkış kodu `-1073740940`, `0xC0000374`: yığın/heap bozulması) ya da hiçbir şey olmayıp ileride başka yerde çöker.

Linux'ta bellek yönetimiyle ilgili sık görülen diğer glibc mesajları:

Mesaj	Olası neden
`free(): invalid pointer`, `munmap_chunk(): invalid pointer`	`malloc`'tan gelmeyen ya da kaydırılmış (`p + 1`) bir işaretçiyi `free` etmek; yerel dizi ya da string sabiti `free` etmek
`double free or corruption (!prev)`, `free(): invalid next size (fast)`	Ayrılan bloğun **dışına** yazılmış (heap taşması) ve `free` bunu fark etti
`malloc(): corrupted top size`	Yine heap taşması; hata taşmanın olduğu satırda değil, sonraki `malloc`/`free`'de ortaya çıkar

Çözüm:

- Her `malloc` için tam **bir** `free` yaz; `free(p); p = NULL;` alışkanlığı edin, `free(NULL)` zararsızdır (Bölüm 10).
- Ayırdığın bayt sayısını (`n * sizeof(int)`) ve yazdığın en büyük indeksi karşılaştır; bir eksik ya da fazla sorun çıkarır.
- Linux ve macOS'ta programı `-fsanitize=address -g` ile derle: bu hataları **olduğu satırda** adıyla raporlar.

Hata ayıklama ipuçları

- **İlk hatadan başla.** Sonraki mesajlar çoğunlukla zincirleme sonuçtur. Düzelt, yeniden derle, yeni ilk hataya bak.

- **Bir üst satıra da bak.** Eksik `;`, kapanmayan `(`, `{` ve `"` hatayı sonraki satırda ya da dosya sonunda ortaya çıkarır.
- **Her zaman `-Wall -Wextra` ile derle:** `gcc -std=c17 -Wall -Wextra -g program.c -o program`. Uyarısız derlenmeyen programı teslim etme; bu ekteki uyarıların hepsi gerçek hataların habercisidir. `-g`, hata ayıklayıcının satır numarasını göstermesini sağlar.
- **`printf` ile izle.** Şüpheli yerlere değişkenin değerini yazdır: `printf("i=%d toplam=%d\n", i, toplam);`. Satırı `\n` ile bitirmezsen çıktı tamponda bekler ve program çökerse hiç görünmeyebilir. Tamponsuz yazmak için `fprintf(stderr, ...)` kullanabilirsin.
- **Program takıldıysa** (hiçbir şey yazmıyor, geri dönmüyor) büyük olasılıkla sonsuz döngüdedir: `i++` unutulmuştur, koşul hiç yanlış olmaz ya da `while (x < 10);` sonundaki `;` döngüyü boşa çevirmiştir (Bölüm 5). Terminalde `Ctrl+C` ile durdur. Program bir `scanf` içinde bekliyor da olabilir; bu takılma değil, girdi bekleyişidir.
- **VS Code hata ayıklayıcısı.** Satırın soluna tıklayıp kırmızı nokta (*breakpoint*) koy, `F5` ile başlat. `F10` bir sonraki satıra geçer, `F11` çağrılan fonksiyonun içine girer; sol paneldeki *Variables* bölümünde değişkenlerin değerlerini, *Call Stack* bölümünde programın nereden geldiğini görürsün. Segfault'ta ayıklayıcı doğrudan çöken satırda durur. Kurulum için Bölüm 1'e bak.
- **Mesajı ara.** Hata mesajını, kendi değişken adlarını çıkararak arama motoruna yapıştır (ör. `makes pointer from integer without a cast`). Aynı hatayı yüzlerce kişi yaşamıştır.

EK D

ASCII Tablosu

C'de `char` aslında küçük bir tamsayıdır (Bölüm 2): `'A'` yazdığında bellekte 65 sayısı durur. Hangi sayının hangi karaktere karşılık geldiğini **ASCII** (*American Standard Code for Information Interchange*) tablosu belirler. Tablo 0–127 arasındaki 128 kodu kapsar; 0–31 ve 127 ekranda görünmeyen **kontrol karakterleridir**. Türkçe harfler (`ç`, `ğ`, `ı`, `ö`, `ş`, `ü`) ASCII'de yoktur; UTF-8'de 2 baytla yazılırlar (Bölüm 3 ve 8).

Akılda tutulacak kodlar

Karakter	Kod	Neden önemli?
`'\0'`	0	String sonlandırıcı (Bölüm 8)
`'\n'`	10	Yeni satır; `fgets` ve tamponda kalan Enter (Bölüm 3)
`' '`	32	Boşluk; büyük-küçük harf farkı da 32'dir
`'0'` – `'9'`	48 – 57	Rakam karakteri → sayı: `c - '0'`
`'A'` – `'Z'`	65 – 90	Büyük harfler
`'a'` – `'z'`	97 – 122	Küçük harfler: `'a' - 'A' == 32`

`%c` karakteri, `%d` kodunu yazdırır:

asciiOrnek.c

```c
#include <stdio.h>

int main(void) {
    char harfler[] = "Aa0";

    for (int i = 0; harfler[i] != '\0'; i++) {
        printf("'%c' = %3d\n", harfler[i], harfler[i]);
    }
    printf("'A' + 2 = '%c'\n", 'A' + 2);
    printf("'7' - '0' = %d\n", '7' - '0');
    printf("'b' - 32 = '%c'\n", 'b' - 32);
    return 0;
}
```

Çıktı

```
'A' =  65
'a' =  97
'0' =  48
'A' + 2 = 'C'
'7' - '0' = 7
'b' - 32 = 'B'
```

Kodları ezberlemen gerekmez: `'0'`, `'A'` ve `'a'`'nın 48, 65 ve 97 olduğunu bil, gerisini aradaki farktan bulursun.

Tablo (0–127)

Dec	Hex	Kr.	Dec	Hex	Kr.	Dec	Hex	Kr.	Dec	Hex	Kr.
0	00	NUL `'\0'`	32	20	boşluk	64	40	`@`	96	60	`` ` ``
1	01	SOH	33	21	`!`	65	41	`A`	97	61	`a`
2	02	STX	34	22	`"`	66	42	`B`	98	62	`b`
3	03	ETX	35	23	`#`	67	43	`C`	99	63	`c`
4	04	EOT	36	24	`$`	68	44	`D`	100	64	`d`
5	05	ENQ	37	25	`%`	69	45	`E`	101	65	`e`
6	06	ACK	38	26	`&`	70	46	`F`	102	66	`f`
7	07	BEL `'\a'`	39	27	`'`	71	47	`G`	103	67	`g`
8	08	BS `'\b'`	40	28	`(`	72	48	`H`	104	68	`h`
9	09	TAB `'\t'`	41	29	`)`	73	49	`I`	105	69	`i`
10	0A	LF `'\n'`	42	2A	`*`	74	4A	`J`	106	6A	`j`
11	0B	VT	43	2B	`+`	75	4B	`K`	107	6B	`k`
12	0C	FF	44	2C	`,`	76	4C	`L`	108	6C	`l`
13	0D	CR `'\r'`	45	2D	`-`	77	4D	`M`	109	6D	`m`
14	0E	SO	46	2E	`.`	78	4E	`N`	110	6E	`n`
15	0F	SI	47	2F	`/`	79	4F	`O`	111	6F	`o`
16	10	DLE	48	30	`0`	80	50	`P`	112	70	`p`
17	11	DC1	49	31	`1`	81	51	`Q`	113	71	`q`
18	12	DC2	50	32	`2`	82	52	`R`	114	72	`r`
19	13	DC3	51	33	`3`	83	53	`S`	115	73	`s`
20	14	DC4	52	34	`4`	84	54	`T`	116	74	`t`
21	15	NAK	53	35	`5`	85	55	`U`	117	75	`u`
22	16	SYN	54	36	`6`	86	56	`V`	118	76	`v`
23	17	ETB	55	37	`7`	87	57	`W`	119	77	`w`
24	18	CAN	56	38	`8`	88	58	`X`	120	78	`x`
25	19	EM	57	39	`9`	89	59	`Y`	121	79	`y`
26	1A	SUB	58	3A	`:`	90	5A	`Z`	122	7A	`z`
27	1B	ESC	59	3B	`;`	91	5B	`[`	123	7B	`{`
28	1C	FS	60	3C	`<`	92	5C	`\`	124	7C	`|`
29	1D	GS	61	3D	`=`	93	5D	`]`	125	7D	`}`
30	1E	RS	62	3E	`>`	94	5E	`^`	126	7E	`~`
31	1F	US	63	3F	`?`	95	5F	`_`	127	7F	DEL